

第一章 ZIGBEE 协议概述

1.1 协议描述

ZigBee 联盟正在开发一种低成本、极低功耗的双向无线通信标准。采用 ZigBee 标准的解决方案将被嵌入到消费型电子产品、家庭和楼宇自动化、工业控制、PC 外设、医疗传感应用、玩具和游戏中。

1.1.1 范围

本文件包括有关 ZigBee 协议标准的规范、接口描述、对象描述、协议和算法。包括应用支持子层（APS）、ZigBee 设备对象（ZDO）、ZigBee 设备配置（ZDP）、应用程序构架层、网络层（NWK）和 ZigBee 安全服务。

1.1.2 目的

本文件的目的是为今后实施 ZigBee 协议标准提供一个明确的描述，以便于许多将 ZigBee 标准纳入平台和设备的公司，根据本文件为新兴的无线市场产生兼容的、低成本和高可用性的产品。

1.1.3 协议栈结构

ZigBee 协议栈结构由一组被称作层的模块组成。每一层为上面的层执行一组特定的服务：数据实体提供了数据传输服务，管理实体提供了所有其它的服务。

每个服务实体通过一个服务接入点（SAP）为上层提供一个接口，每个 SAP 支持多种服务原语来实现要求的功能。

IEEE 802.15.4-2003 标准定义了两个较低层：物理层（PHY）和媒体访问控制子层（MAC）。ZigBee 联盟在此基础上建立了网络层（NWK）和应用层构架。应用层构架由应用支持子层（APS）、ZigBee 设备对象（ZDO）组成。制造商定义的应用对象使用构架层，和 ZDO 共享 APS 和安全服务。

IEEE 802.15.4-2003 有两个 PHY 层，运行在两个不同的频率范围：868/915 MHz 和 2.4 GHz。较低频率的 PHY 层覆盖了欧洲 868 MHz 频带和 915 MHz，使用的国家如美国和澳大利亚。较高频率的 PHY 层几乎在世界各地使用。关于 IEEE 802.15.4-2003 PHY 层的完整描述可以在[B1]中找到。

IEEE 802.15.4-2003 MAC 子层使用 CSMA-CA 机制来控制无线电信道的访问。其职责也可能包括传输信标帧，同步和提供一个可靠的传输机制。关于 IEEE 802.15.4-2003 MAC 子层的完整描述可以在[B1]中找到。

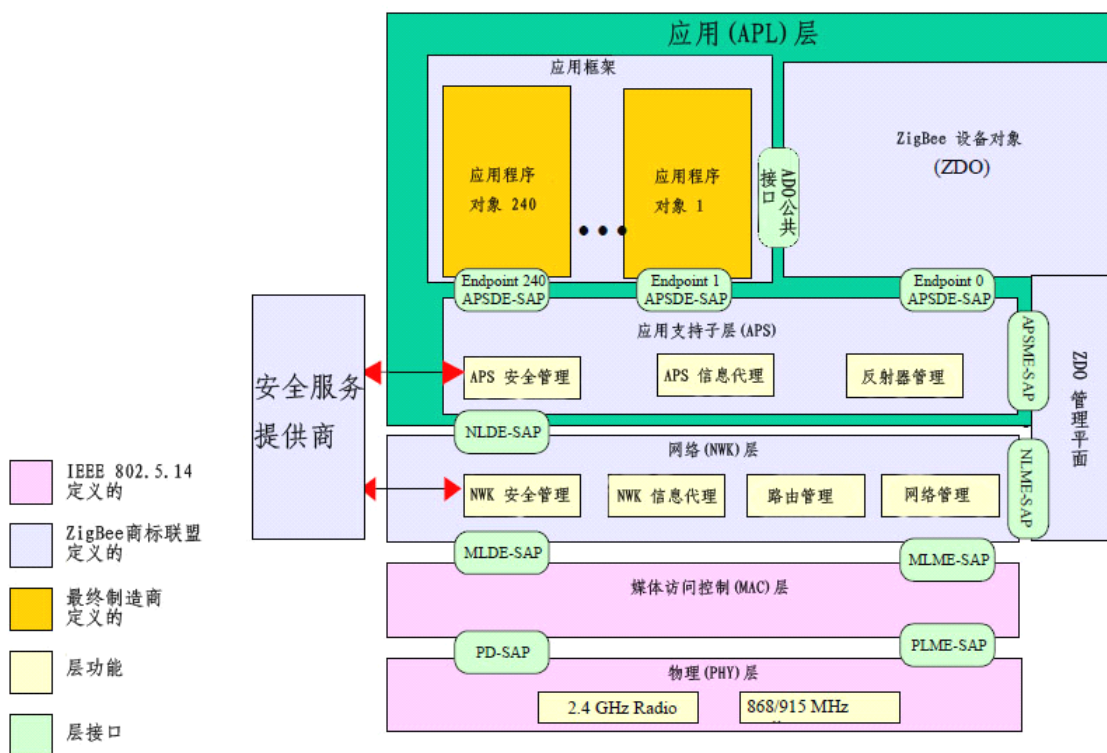


图 1.1 ZigBee 协议栈结构概览

1.1.4 网络拓扑

ZigBee 网络层（NWK）支持星型、树型和网状网络拓扑。在星型拓扑中，网络由一个叫做 ZigBee 协调器的设备控制。ZigBee 协调器负责发起和维护网络中的设备，以及所有其它设备，称为终端设备，直接与 ZigBee 协调器通信。在网状和树型拓扑中，ZigBee 协调器负责启动网络，选择某些关键的网络参数，但是网络可以通过使用 ZigBee 路由器进行扩展。在树型网络中，路由器使用一个分级路由策略在网络中传送数据和控制信息。树型网络可以使用 IEEE 802.15.4-2003 规范描述的以信标为导向的通信。网状网络允许完全的点对点通信。网状网络中的 ZigBee 路由器不会定期发出 IEEE 802.15.4-2003 信标。本规范仅描述了内部 PAN 网络，即通信开始和终止都是在同一个网络。

1.2 约定和缩写

1.2.1 约定

1.2.1.1 符号和表示法

表示法遵守 ANSI X9.63 - 2001, § 2.2 [B7]

1.2.1.2 整数、八位字节及其表示

从附录 A 到 D，整数表示为八位字节的字符串，八位字节表示为二进制字符串，会被固定下来。所有整数应该以最高有效的八位字节首序的形式表示为八位字节的字符串。这个表示遵循 4.3 节中的 ANSI X9.63-2001 [B7] 中的转换。所有八位字节应该以高位八位字节首序形式表示为二进制字符串。

1.2.1.3 传输顺序

除非另有说明，本规范中所有帧的传输顺序按照 IEEE Std. 802.15.4-2003 [B1]) 中使用的约定：

- 帧格式按照它们被 PHY 层传来的顺序描述——从左至右——最左边的位首先传输。
- 每个域中的位从 0（最左边，最低）编号，直到 k-1（最右边，最高），域的长度就是 k 位。

● 长于一个八位字节的域被发送到 PHY 层，按照位编号最低的八位字节到位编号最高的八位字节的顺序。

1.2.1.4 字符串和字符串操作

字符串是一个经过特定设置的符号的序列（例如，二进制字母表{0, 1}或所有八位字节的集合）。字符串的长度是它所包含的符号个数（在同一字母表中）。空字符串的长度为 0。两个长度分别为 m 和 n 的字符串 x 和 y 的右串联（表示法：x||y），是长度为 m+n 的字符串 z，它的最左边 m 位与 x 一致，最右边 n 位与 y 一致。一个八位字节是一个长度为 8 的字符串。在我们的上下文，所有的八位字节都是二进制字母表中的字符串。

1.3 缩略语和简称

为施行本标准，使用以下缩略语和简称：

AIB	应用支持层的信息库
AF	应用框架
APDU	应用支持子层协议数据单元
APL	应用层
APS	应用支持子层
APSDE	应用支持子层数据实体
APSDE-SAP	应用支持子层数据实体-服务接入点
APSME	应用支持子层管理实体
APSME-SAP	应用支持子层管理实体—服务接入点
ASDU	APS 服务数据单元
BRT	广播重试计时器
BTR	广播事务记录
BTT	广播事件务表
CCM*	CBC-MAC 模式增强计数器选项
CSMA-CA	载波侦听多路访问——冲突检测
EPID	扩展 PAN ID
FFD	全功能设备
GTS	同步时隙

HDR	头
IB	信息库
LQI	链路质量指标
LR-WPAN	低速率无线个人区域网
MAC	媒体访问控制
MCPS-SAP	媒体访问控制公用部分子层—服务接入点
MIC	消息完整性代码
MLME-SAP	媒体访问控制子层管理实体—服务接入点
MSC	消息序列图
MSDU	介质访问控制子层服务数据单元
MSG	信息服务类型
NBDT	网络广播传送时间
NHLE	上层实体
NIB	网络层信息库
NLDE	网络层数据实体
NLDE-SAP	网络层的数据实体—服务接入点
NLME	网络层管理实体
NLME-SAP	网络层管理实体—服务接入点
NPDU	网络层协议数据单元
NSDU	网络服务数据单元
NWK	网络
OSI	开放式系统互连
PAN	个人区域网络
PD-SAP	物理层数据—服务接入点
PDU	协议数据单元
PHY	物理层
PIB	个人区域网络信息库
PLME-SAP	物理层管理实体—服务接入点
POS	个人操作空间
QOS	服务质量
RFD	精简功能设备
RREP	路由应答
RREQ	路由请求

RN	路由节点
SAP	服务接入点
SKG	密钥生成
SKKE	对称密钥的密钥建立
SSP	安全服务提供商
SSS	安全服务规范
WPAN	无线个人区域网
XML	可扩展标记语言
ZB	ZigBee
ZDO	ZigBee 设备对象

1.4 词汇

1.4.1 定义

1.4.1.1 一致性级别

一致性级别定义应该遵循 IEEE Style Manual [B13]13 条第 1 节中的这些标准。

Expected: 一个关键词，用于描述本规范假设的设计模型的软件或硬件行为。其它硬件和软件设计模型也可以执行。

May: 一个关键词，指明在标准的限制范围内允许的一个行动方针（*may* 等于允许）。

Shall: 一个关键词，指明为了符合标准要严格遵守的强制性要求；禁止偏离“Shall”（*shall* 等于要求）。

Should: 一个关键词，表示在一些可能性之中，某一个因为特别合适被推荐，没有提到或排除其它的；某个行动方针是首选但不是必要的；或者，（以否定形式）不赞成某个行动方针，但是也不禁止（*should* 等于建议）。

Reserved Codes: 本规范定义的一组代码，但是不以其它方式使用。未来的规范的执行可能使用这些代码。一个执行本规范的产品不能产生这些代码。

Reserved Fields: 本规范定义的一组域，但是不以其它方式使用。执行本规范的产品必须使这些域为 0，且不能作出关于这些域的任何假设，或者执行基于它们的内容的处理。

ZigBee Protocol Version: 本规范管理的 ZigBee 协议版本的名称。所有遵守本规范的 ZigBee 协议栈的帧 NWK 头的帧控制域的协议本子域的值应为 0x02。各种 ZigBee 规范协议修订版需要支持的协议版本如表 1.1。

表 1.1 ZigBee 协议版本

规范	协议版本	注解
现行	0x02	要求和 ZigBee 2006 向后兼容 不要求和 ZigBee 2004 向后兼容
ZigBee 2006	0x02	不要求和 ZigBee 2004 向后兼容
ZigBee 2004	0x01	原始 ZigBee 版本

一个遵守本规范版本的 ZigBee 设备可以选择向后兼容本规范的 2004 修订版。如果选择了向后兼容，除了支持本规范版本描述的帧格式和功能，还要支持旧版本指定的所有帧格式和功能。[一个运行网络中的所有设备，不管其内部支持 ZigBee 规范的哪个修订版本，就其外部看的见的行为，都应该始终遵守一个 ZigBee 协议版本。] 从其外部行为的角度来说，一个 ZigBee 网络不应该包含遵守多个 ZigBee 协议版本的设备。[要加入的网络的协议版本应该由一个向后兼容的设备在决定加入网络之前，通过信标负载审查确定；或者如果设备是 ZigBee 协调器，应该由应用程序确定。] 一个遵守本规范的 ZigBee 设备可以选择只支持 0x02 协议版本，这样它仅能加入支持适当信标负载的网络。一个遵守本规范的 ZigBee 设备应该丢弃所有携带的协议本子域值不是 0x01 或 0x02 的帧，并应只处理协议版本为 0x01 或 0x02，符合设备参与进来的网络协议版本的帧。

1.4.1.2 ZigBee 定义

为了履行本标准，使用以下名词和定义。本条中定义的名词可以在 IEEE P802.15.4 § 3 [B1] 或 ANSI X9.63-2001 § 2.1 [B7] 中找到。

Access control list: 一个设备使用的一个表，来决定哪些设备被授权执行某个特定功能。这个表还可以存储用于与其它设备安全通信的安全信息（比如，密钥，帧计数，密钥计数，安全级别信息）。

Active network key: ZigBee 设备使用的密钥，来加密送输出的 NWK 帧，还可用于处理输入的 NWK 帧。

Alternate network key: 一个可以代替主动网络密钥使用的密钥来处理输入的 NWK 帧。

Application domain: 这描述了应用的广泛领域，比如楼宇自动化。

Application key: 这是一个主密钥或者连接密钥，通过信任中心发送到设备，用于加密端到端通信。

Application object: 应用层顶部的一个组成部分，由实际执行此应用的制造商定义。

Application profile: 这是设备描述的一个集合，加在一起形成一个合作的应用。例如，一个节点的恒温器可以与另外一个节点的炉控制器通信。它们一起合作，形成了一个供暖应用 profile。

Application support sub-layer protocol data unit: 一个数据单元，在支持两个对等实体子层的应用程序之间交换。

APS command frame: 这是一个命令帧，来自一个设备的 APSME，发给另一个设备的对等实体。

Association: IEEE 802.15.4-2003 MAC 子层提供的服务，用于建立一个网络的成员。

Attribute: 一个数据实体，表示一个物理量或状态。这个数据通过命令被传递给其它设备。

Beacon-enabled personal area network: 这是一个个人区域网络，至少包括一个设备，以一个定期间隔传输信标帧。

Binding: 在一个源端点/cluster 标识符对一个目的端点之间创建的单向逻辑连接，这可能存在于一个或多个设备上。

Broadcast: 传输信息到特定 PAN 的每个设备，这个 PAN 属于静态定义的广播群的少数之一，例如所有路由器，在给定的以跳测量的传输半径范围内。

Broadcast jitter: 随机的延迟时间，由一个设备在中继一个广播事务之前产生。

Broadcast transaction record: 设备发起或中继一个广播信息的一个本地接收说明。

Broadcast transaction table: 广播事务记录的收集。

Cluster: 这是一个应用程序信息，是一个或多个属性的容器。例如，ZigBee 设备 Profile 定义了命令和响应。这些包含在为每个命令和响应列举了 cluster 标识符的 cluster 中。然后每个 ZigBee 设备规范(Profile)信息定义为一个 cluster。或者，在已知属性的 cluster 中一个应用 profile 可以创建子类型。在这种情况下，cluster 是一个属性的集合，伴随着具体 cluster 标识符定义（子类型信息）。

Cluster identifier: 这是在一个具体应用 profile 或应用 profile 集合中的 cluster 的枚举参考。cluster 标识符是一个 16 位的数字，在每个应用规范的范围中是唯一的，指明了一个具体的 cluster。约定可以通过 cluster 标识符的普通定义的应用 profile 建立，这时每个应用 profile 定义了同样的一组 cluster 标识符。cluster 标识符在简单描述符中指定为输入或输出，用于创建一个绑定表。

Coordinator: 这是一个负责连接或断开到它的 PAN 设备的 IEEE 802.15.4-2003 设备。协调器必须是一个完全功能设备（FFD）。

Data integrity: 保证数据的原有形式没有被修改。

Data key: 这是一个密钥，来源于连接密钥，用于保护数据信息。

Device: 这是包含 ZigBee 协议栈实现的任一实体。

Device application: 一个具体的应用，负责设备运行。设备应用驻留在端点 0，通过约定和包含的逻辑来管理设备网络和常用的维护功能。

Device description: 一个应用 profile 内的一个特定设备的描述。例如，光传感器设备描述是一个家庭自动化应用 profile 的成员。设备描述还有一个唯一的标识符，作为发现程序的一部分交换。

Direct addressing: 一种寻址模式，帧的目的地是在帧本身中完全指定的。

Direct transmission: 使用直接寻址的帧传输。

Disassociation: IEEE 802.15.4-2003 MAC 子层提供的服务，用于终止网络中设备的成员身份。

End application: 驻留在一个设备的端点 1 到 240 的应用。终端应用执行非网络的以及和 ZigBee 协议相关的功能。

End device binding: 创建或移除一个绑定连接的程序，由每个将要形成连接的终端设备发起。程序可以涉及或不涉及用户干预。

Endpoint: 一个单元内的特定组件。每个 ZigBee 设备可以支持多达 240 个这样的组件。

Endpoint address: 分配给一个端点的地址。除了唯一的 64 位 IEEE 地址和 16 位网络地址，又分配了这个地址。

Extended PAN ID: 全球唯一的 64 位 PAN 网络标识符。这个标识符应该在给定区域众多 PAN 中是唯一的。这个标识符用于避免不同网络之间的 PAN ID 冲突。

Information base: 一个变量的集合，定义了一个层特定的行为。这些变量可以指定或通过其管理服务从层中获得。

Key establishment: 一种机制，涉及两个设备制定出一个共同密钥的协议的执行。

Key-load key: 这是一个密钥，来源于连接密钥，用于保护带有主密钥的密钥传输信息。

Key transport: 一种机制，用于在一个设备到另外一个设备或其它设备进行密钥通信。

Key-transport key: 这是一个密钥，来源于一个连接密钥，用来保护密钥传输信息，带有不是主密钥的密钥。

Key update: 一种机制，通过给同一个组提供另外一个密钥，在两个或多个设备之间实施共享密钥的更换。

Local device: 这是一个 ZDP 命令的发起者。

Link key: 这是一个密钥，PAN 内，在两个（且只能是两个）对等的应用层实体之间专门共享。

Master key: 这是一个共享的密钥，用于执行一个对称加密密钥期间建立协议。主密钥是两个设备之间长期安全的基础，并可以用于产生连接密钥。

Mesh network: 这是一个网络，其中路由信息的执行是一个分散的、合作的过程，在为每个代表寻找路由时涉及许多对等设备。

Multicast: 传输信息到特定 PAN 的每个设备，这个 PAN 属于动态定义的多播组，在给定的以跳测量的传输半径范围内。

Multihop network: 一个网络，尤其是无线网络，其中给定信息的发送端和接收端不能保证是互相连接的。这意味着中间设备必须作为路由器。

Non-beacon-enabled personal area network: 一个个人区域网络，不包括每隔一定间隔传输信标帧的任何设备。

Neighbor table: ZigBee 设备使用一个表，用来在 POS 内保持跟踪其它设备。

Network address: 网络层分配给设备的地址，由网络层使用在设备之间为信息寻找路由。

Network broadcast delivery time: 给定网络中发送给每个设备的一次广播事务的时间要求。

Network manager: 这是一个 ZigBee 设备，执行第 3 章的网络管理功能，包括 PAN ID 冲突解决和面对干扰时的频率捷变措施。

Network protocol data unit: 在两个对等实体网络层之间交换的一个数据单元。

Network service data unit: 通过网络服务接入点作为一个单元传送的信息。

Node: 独立设备描述和应用的一个集合，驻留在一个单元，共享一个共同的 802.14.5 无线电。

Normal operating state: 所有启动和初始化程序发生之后，关闭程序发起之前，发生的程序。

NULL: 一个参数或变量值，意味着未指定，未定义或未知。空的具体值应具体执行，且不能和任何其它参数或值相冲突。

Octet: 八位数据，作为字节的同义词使用。

One-way function: 一个函数，比其相反更易计算和执行。

Orphaned device: 一个设备通过成为 ZigBee 设备的 PAN 成员，失去与它的通信。典型是 ZigBee 终端设备。

PAN coordinator: 基于 IEEE 802.15.4-2003 的网络的主要控制器，负责网络形成。PAN 协调器必须是一个全功能设备（FFD）。

PAN information base: IEEE 802.15.4-2003 标准中的变量的一个集合，为了交换信息在层之间传递。这个数据库可以包括访问控制列表，它存储了安全信息。

Personal operating space: 单个设备接收范围内的区域。

Private method: 只有 ZigBee 设备对象可以使用，终端应用程序不能使用的属性和命令。

Protocol data unit: 在两个对等实体之间交换的数据单元。

Public method: 终端应用程序可以使用的属性和命令。

Radio: IEEE 802.15.4-2003 无线设备，是每个 ZigBee 设备的一部分。

Remote device: 一个 ZDP 命令的目标。

Route discovery: 一个 ZigBee 协调器或 ZigBee 路由器通过发出一个路由请求命令帧，尝试发现到远程设备的一条路由的一个操作。

Route discovery table: 由 ZigBee 协调器或 ZigBee 路由器使用的一个表，来存储路由发现期间使用的临时信息。

Route reply: 一个 ZigBee 网络层命令帧，用于回复路由请求。

Route request: 一个 ZigBee 网络层命令帧，用于发现随后的信息可以通过网络传送的路径。

Routing table: 一个 ZigBee 协调器或 ZigBee 路由器中的一个表，存储参与为帧寻找路由时需要的信息。

Service discovery: 一个设备查找有效的服务的功能。

Stack profile: ZigBee 规范范围之外约定的一组额外限制的协议，与规范本身宣布的可选功能有关。

Symmetric-key key establishment: 一种机制，基于一个预先共享密钥（一个所谓的主密钥），使双方建立一个共享的密钥。

Trust center: 被一个 ZigBee 网络内设备信任的设备，为了网络和端到端应用配置管理来分发密钥。

Unicast: 传输一个信息到网络中的一个设备。

ZigBee coordinator: 一个 IEEE 802.15.4-2003 PAN 协调器。

ZigBee device object: 应用层的组成部分，负责定义网络中设备的角色（比如，ZigBee 协调器或终端设备），初始化和/或响应绑定，发现请求，以及建立网络设备之间的一个安全关系。

ZigBee end device: 一个 IEEE 802.15.4-2003 RFD 或 FFD，参与 ZigBee 网络，既不是 ZigBee 协调器也不是 ZigBee 路由器。

ZigBee router: 一个 IEEE 802.15.4-2003 FFD，参与 ZigBee 网络，不是 ZigBee 协调器，但是可以在其个人操作空间作为一个 IEEE 802.15.4-2003 协调器。能为信息在设备之间寻找路由，支持连接。

1.5 参考书目

以下标准包含一些规定，通过本文件中的参考书目，它们构成了本标准的规定。规范性参考书目给定在“ZigBee/IEEE 参考书目”中，“规范性参考书目”和资料性参考书目，给定在“资料性参考书目”中。在发表时，该版本说明是有效的。所有的标准以修订版为准，基于本标准作出协议的各方最好调查采用参考书目最新版本的可能性，如本小节中所指明的。

1.5.1 ZigBee/IEEE 参考书目

[B1] 电子与电气工程师协会，IEEE802.15.4-2003 标准公司，信息技术的 IEEE 标准——电信和系统之间信息交换——地方和都市区域网络——具体要求——第一部分 15.4: 低数据率的无线个人区域网（WPANs）的无线媒体访问控制（MAC）和物理层（PHY）规范。纽约：IEEE 出版社。2003 年。

[B2] IEEE 754-1985, 二进制浮点算法的 IEEE 标准, IEEE, 1985 年。

[B3] 文件 03285r0: IEEE802.15.4 标准的改进建议, 2003 年 7 月。

[B4] 文件 02055r4: 网络需求定义, 2003 年 8 月。

1.5.2 规范性参考书目

[B5] ISO/IEC 639-1: 2002 语言名称表示代码——第 1 部分: Alpha-2 代码。

[B6] ISO/IEC 646: 199 信息技术——用于信息交换的 ISO7 位编码字符集

[B7] ANSI X9.63-2001, 金融服务业的开放密钥加密——密钥协议和使用椭圆曲线加密传输密钥, 美国银行家协会, 2001 年 11 月 20 日。可从 <http://www.ansi.org> 上找到。

[B8] FIPS Pub 197, 高级加密标准 (AES), 联邦信息处理标准出版物 197, 美国商务部商/N.I.S.T., 春田市, 弗吉尼亚州, 2001 年 11 月 26 日。可从 <http://csrc.nist.gov/>上找到。

[B9] FIPS 出版物 198, 键控哈希消息认证码 (HMAC), 联邦信息处理标准出版物 198, 美国商务部/N.I.S.T., 春田市, 弗吉尼亚州, 2002 年 3 月 6 日。可从 <http://csrc.nist.gov/>上找到。

[B10] ISO/IEC 9798-2, 信息技术——安全技术——实体验证机制——第 2 部分: 使用对称加密算法的机制, 国际标准化组织, 日内瓦, 瑞士, 1994(第一期)。可从 <http://www.iso.org/>上找到。

[B11] NIST Pub 800-38A 2001 ED, 分组加密编码操作建议——方法和技术, NIST 特别出版物 800-38A, 2001 年版, 美国商务部/N.I.S.T., 2001 年 12 月。可从 <http://csrc.nist.gov/>上找到。

[B12] NIST, 随机数产生和测试。可从 <http://csrc.nist.gov/rng/>上找到。

1.5.3 资料性的参考书目

[B13] FIPS Pub 140-2, 加密模块的安全要求, 美国商务部/ N.I.S.T., 春田市, 弗吉尼亚州, 2001 年 6 月 (取代 FIPS 的出版物 140-1)。可从 <http://csrc.nist.gov/>上找到。

[B14] IEEE 标准样式手册, 在 2000 年 5 月出版并发行, 2001 年 9 月 20 日修订。可从 <http://standards.ieee.org/guides/style/>上得到。

[B15] ISO/IEC 7498-1: 1994 信息技术——开放系统互联——基本参考模型: 基本模型。

[B16] ISO/IEC 10731: 1994, 信息技术——开放系统互联——OSI 服务定义的约定。

[B17] ISO/IEC 9646-1: 1991, 信息技术——开放系统互联——一致性测试方法和框架——第 1 部分: 一般的概念。

[B18] ISO/IEC 9646-7: 1995, 信息技术——开放系统互联——一致性测试方法和框架——第 7 部分: 一致性声明的实现。

[B19] A.J. Menezes, P.C. van Oorschot, S.A. Vanstone, 加密使用手册, Boca Raton: CRC 出版社, 1997 年。

[B20] FIPS 出版物 113, 计算机数据认证, 联邦信息处理标准出版物 113, 美国商务部/ N.I.S.T., 1985 年 5 月 30 日。可从 <http://csrc.nist.gov/>上找到。

[B21] R. Housley, D. Whiting, N. Ferguson, 计数器与 CBC-MAC (CCM), 2002 年 6 月 3 日提交给 N.I.S.T.。可从 <http://csrc.nist.gov/encryption/modules/proposedmodes/>上找到。

[B22] J. Jonsson, 关于 CTR + CBC-MAC 的安全, 选定区域加密进行中——SAC, 2002 年, K. Nyberg, H. Heys, Eds., 计算机科学讲义, 2595 卷, 76-93 页, 柏林: 施普林格, 2002 年。

[B23] J. Jonsson, 关于 CTR + CBC-MAC 的安全, NIST 操作模式——附加的 CCM 文件。可从 <http://csrc.nist.gov/encryption/modes/proposedmodes/>上找到。

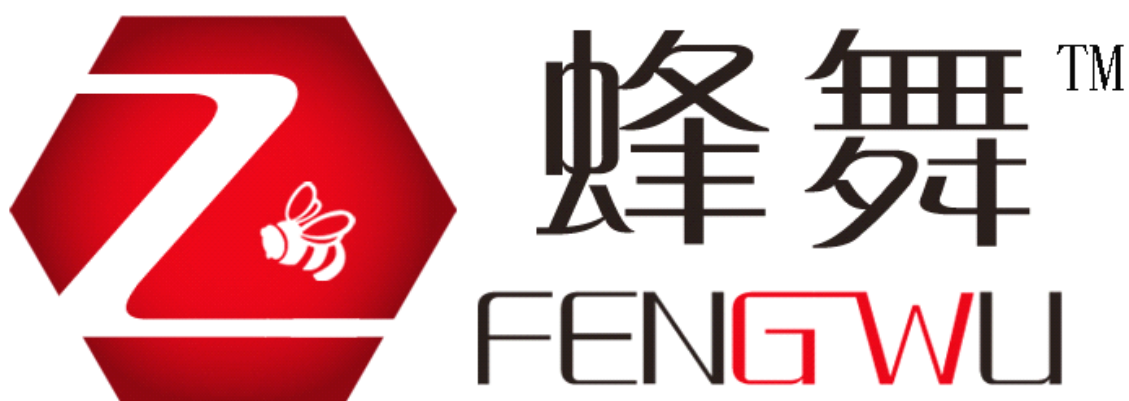
[B24] P. Rogaway, D. Wagner, CCM 评论, IACR ePrint 存档 2003-070, 2003 年 4 月 13 日。

版权声明

郑州新双恒信息技术有限公司拥有本译文的版权。本译文免费供大家一起学习交流。译文出现的错误及误差，您可以通过以下方式联系本公司，我们将非常感谢。欢迎在网络上转载本译文，但必须保证本译文的完整，否则本公司不承担任何后果。如果你在转载的时候，需要对本译文进行修改或重新编辑，请征得本公司的同意。谢谢！

网址：<http://www.zigbee-sh.cn>

QQ 群：83028739



第二章 应用层规范

2.1 概述

ZigBee 协议栈结构包括许多层次的组件，包括 IEEE802.15.4 2003 媒体访问控制（MAC）层，物理（PHY）层和 ZigBee 网络（NWK）层。每个组件提供了含有一组它自己的服务和功能的一个应用。尽管本章可能提到 ZigBee 协议栈结构的其它组件，它的主要目的是描述图 1.1 “ZigBee 协议概览”中显示的组件标记应用（APL）层。

如图 1.1 所示，ZigBee 应用层包括 APS 子层，ZDO（包括 ZDO 管理平面）和制造商定义的应用对象。

2.1.1 应用支持子层

应用支持子层（APS）提供了网络层（NWK）和应用层（APL）之间的一个接口，通过一组 ZDO 和制造商定义的应用对象都使用的常用服务。该服务通过两个实体提供：

- APS 数据实体（APSDE），通过 APSDE 服务接入点（APSDE-SAP）。
- APS 管理实体（APSME），通过 APSME 服务接入点（APSME-SAP）。

APSDE 在位于同一个网络的两个或多个应用实体之间提供了数据传输服务。

APSME 为应用对象提供了多种服务，包括安全服务和绑定设备。它还维护管理对象的数据库，叫做 APS 信息库（AIB）。

2.1.2 应用构架

ZigBee 中的应用构架是 ZigBee 设备承载应用对象的环境。

可以定义多达 240 个不同的应用对象，每个端点上的接口索引从 1 到 240。为使用 APSDE-SAP 定义了两个另外的端点：端点 0 保留给到 ZDO 的数据接口，端点 255 保留给到所有应用对象的广播数据接口功能。端点 241-254 保留，供今后使用。

2.1.2.1 应用 Profile

应用 profile 是信息，信息格式，处理行为的协议，使开发者可以创建一个可以使用定居在不同设备上的应用实体的共同操作、分散的应用。这些应用 profile 可以发送命令，请求数据，处理命令和请求。

2.1.2.2 Cluster

Cluster 用 cluster 标识符识别，和数据流出、流进设备有关。Cluster 标识符在特定应用 profile 中是唯一的。

2.1.3 ZigBee 设备对象

ZigBee 设备对象（ZDO），代表一个基础的函数类，提供应用对象、设备 profile 和 APS 之间的一个接口。ZDO 位于应用程序构架和应用支持子层之间。它满足了 ZigBee 协议栈所有应用程序操作的一般需求。ZDO 的职责如下：

- 初始化应用支持子层（APS）、网络层（NWK）和安全服务提供者。
- 组装来自于终端应用的配置信息，以决定并执行发现，安全管理，网络管理和绑定管理。

ZDO 通过应用对象，为设备控制和网络功能提供了应用构架层的应用对象的公共接口。到 ZigBee 协议栈低层部分的 ZDO 接口，在端点 0，通过 APSDE-SAP 提供数据服务，并通过 APSME-SAP 来控制信息。公共的接口在 ZigBee 协议栈的应用构架层内部，提供了设备的地址管理、发现、绑定以及安全功能。这些设备在以下小节中描述。ZDO 在 2.5 节做了全面的描述。

2.1.3.1 设备发现

设备发现，是一个 ZigBee 设备发现其他 ZigBee 设备的过程。有两种形式的设备发现请求：IEEE 地址请求和 NWK 地址请求。IEEE 地址请求单播到一个特定的设备，并假设 NWK 地址已知。NWK 地址请求是广播的，携带已知的 IEEE 地址到作为数据负载。

2.1.3.2 服务发现

服务发现是一个给定设备的能力被其他设备发现的过程。服务发现可以通过为特定设备的每个端点发布一个查询，或者使用一个匹配服务功能（广播或者单播）。服务发现可以定义和使用各种描述符来描述一个设备的能力。

在发现操作发生时，设备提供的某种服务可能无法访问，在这种情况下设备发现信息也可以在网络中缓存。

2.2 ZigBee 应用支持(APS)子层

2.2.1 范围

本节指明了应用层部分，提供服务规范以及到制造商定义的应用对象和 ZigBee 设备对象的接口。本规范定义了一个数据服务，它允许应用对象传输数据，以及一个管理服务，它提供了绑定机制。另外，还定义了应用支持子层的帧格式和帧类型规范。

2.2.2 目的

本节的目的是定义 ZigBee 应用支持（APS）子层的功能。本功能基于使得 ZigBee 网络层正确运行的驱动器功能和制造商定义的应用对象要求的功能。

2.2.3 应用支持(APS)子层概述

应用支持子层通过一组 ZigBee 设备对象 (ZDO) 和制造商定义的应用对象共用的服务, 提供了网络层和应用层之间的接口。这些服务通过两个实体提供: 数据服务和管理服务。APS 数据实体 (APSDE) 通过其相关的 SAP 提供了数据传输服务, 即 APSDE-SAP。APS 管理实体 (APSME) 通过其相关的 SAP 提供了管理服务, 即 APSME-SAP, 并维护管理对象的数据库, 叫做 APS 信息库 (AIB)。

2.2.3.1 应用支持子层数据实体(APSDE)

APSDE 应该为网络层、ZDO 和应用对象提供一个数据服务, 使得应用程序的 PDU 在两个或多个设备之间传输。设备本身必须位于同一个网络。

APSDE 将提供以下服务:

- **发出应用层 PDU (APDU):** APSDE 会接收应用 PDU, 并通过增加合适的协议开销发出一个 APS PDU。
- **绑定:** 一旦两个设备绑定, APSDE 就可以从一个绑定设备传输一个信息给第二个设备。
- **组地址过滤:** 这提供了根据是否是终端组的成员过滤组地址信息的功能。
- **可靠传输:** 在 NWK 层采用端到端重试, 增加了事务的可靠性。
- **重复拒绝:** 传输所提供的信息将不会被多次接收。
- **分裂:** 可以分裂和重组长于一个 NWK 层帧负载的信息。

2.2.3.2 应用支持子层管理实体 (APSME)

APSME 将提供一个管理服务, 允许一个应用和栈相互作用。

APSME 将提供根据其服务和需求一起匹配两个设备的功能。这个服务叫做绑定服务, APSME 将建立和维护一个表来存储该信息。

另外，APSME 将提供以下服务：

- **绑定管理**：这是根据其服务和需求一起匹配两个设备的功能。
- **AIB 管理**：在设备的 AIB 中获取并设置属性的功能。
- **安全**：通过使用安全密钥，与其它设备建立可信关系的功能。
- **组管理**：这提供了宣布多个设备共享一个地址，增加设备到组，以及从组中移除设备的功能。

2.2.4 服务规范

APS 子层提供了上层实体（NHLE）和 NWK 层之间的一个接口。APS 子层的概念包括一个管理实体，叫做 APS 子层管理实体（APSME）。这个实体通过激发子层管理功能提供了服务接口。APSME 还负责维护一个管理有关 APS 子层对象的数据库。该数据库叫做 APS 子层信息库（AIB）。图 2.1 描绘了 APS 子层的组件和接口。

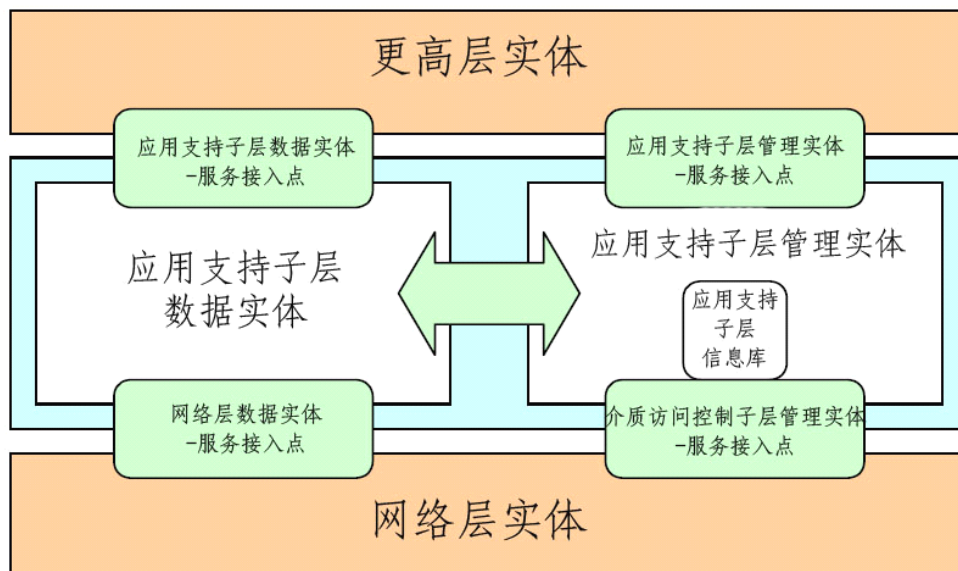


图 2.1 应用支持子层参考模型

APS 子层提供了两种服务，通过两种服务访问点（SAPs）访问。它们是 APS 数据服务，通过 APS 子层数据实体 SAP（APSDE-SAP）访问，以及 APS 管理服务，通过 APS 子层管理实体 SAP（APSME-SAP）访问。这两种服务通过 NLDE-SAP 和（在有限的范围内）NLME-SAP 接口（参见 3.1 小节），提供了 NHLE 和 NWK 层之间的接口。NWK 层和 APS 子层之间的 NLME-SAP 接口只支持 NLME-GET 和 NLME-SET 原语：所有其它 NLME-SAP 原语只能通过 ZDO 访问（见 2.5 小节）。除了这些外部的接口，在 APSME 和 APSDE 之间还有一个隐蔽接口，它允许 APSME 使用 APS 数据服务。

2.2.4.1 APS 数据服务

APS 子层数据实体 SAP（APSDE-SAP）支持应用协议数据单元在对等应用实体之间的传输。表 2.1 列出了 APSDE-SAP 支持的原语。每个原语将在下面分节中讨论。

表 2.1 APSDE – SAP 原语

APSDE-SAP 原语	请求	确认	指示
APSDE – DATA	2.2.4.1.1	2.2.4.1.2	2.2.4.1.3

2.2.4.1.1 APSDE-DATA.request

该原语请求一个从本地 NHLE 到一个或多个对等 NHLE 实体传输一个 NHLE PDU（ASDU）。

2.2.4.1.1.1 服务原语的语义

该原语的语义如下：

APSDE-DATA.request	{
	DstAddrMode,
	ProfileId,
	ClusterId,
	SrcEndpoint,
	ADSULength,
	ADSU,
	TxOptions,
	RadiusCounter
	}

表 2.2 指明了 APSDE-DATA.request 原语的参数。

表 2.2 APSDE-DATA.request 参数

名称	类型	有效范围	描述
DstAddrMode	Integer	0x00-0xff	本原语使用的目标地址以及传输 APDU 的寻址模式。该参数可从下列表中取一个非保留值。 0x00= DstAddress 和 DstEndpoint 不存在 0x01= DstAddress 16 位组地址；DstEndpoint 不存在 0x02 = DstAddress 16 位地址；DstEndpoint 不存在 0x03= DstAddress 和 DstEndpoint 都存在，是 64 位扩展地址 0x04-0xff=保留
DstAddress	Address	通过 DstAddrMode 参数指定	ASDU 要被传输到的实体的单独的设备地址或者组地址
DstEndpoint	Integer	0x00- 0xf0 ,0xff	当且仅当 DstAddrMode 参数的值为 0x02 或者 0x03，该参数才存在，如果存在，该参数或者是 ASDU 要被传输到的实体的单个端点的编号，或者是广播端点(0xff)

表2.2 APSDE-DATA.request参数

名称	类型	有效范围	描述
ProfileId	Integer	0x0000-0xffff	帧要被送到的 profile 的标识符
ClusterId	Integer	0x0000-0xffff	帧要被送到的那个对象的标识符
SrcEndpoint	Integer	0x0000-0xf0	被传输的 ASDU 来自的实体的单个端点
ASDULength	Integer	0x00 - 256*(NsduLength - apscMinHeaderOverhead)	构成被传输的 ASDU 的字节数。单个 APS 帧负载的最大长度由 NsduLength-apscMinHeaderOverhead 给定。如果使用分段，用 256 个这样的块组成单个最大的 ASDU。
AsDu	Set of octets	-	组成被传输的 ASDU 的字节集合
TxOptions	Bitmap	0000 0xxx (x 为 0 或 1)	被传输的 ASDU 的传输选项。是下面一个或多个按位逻辑或运算： 0x01=启用安全传输 0x02=使用网络密钥 0x04=确认传输 0x08 =允许分段
Radius	Unsigned Integer	0x00-0xff	允许一个被传输的帧通过网络进行传输的距离，以跳为单位

2.2.4.1.1.2 原语生成

每当一个数据 PDU (ASDU) 要被传输给一个对等的 NHLE 时，该原语由一个本地的 NHLE 生成。

2.2.4.1.1.3 接收结果

在接收此原语时，APS 子层实体开始传输所提供的 ASDU。

如果 DstAddrMode 参数设置为 0x00 且本原语被支持绑定表的一个设备的 APSDE 接收，就分别根据 SrcEndpoint 和 ClusterId 参数指定的端点和 cluster 标识符，查询绑定表，寻找相关的绑定表条目。如果没有找到绑定表条目，APSDE 发出状态为 NO_BOUND_DEVICE 的 APSDE-DATA.confirm 原语。如果找到一个或多个绑定表条目，那么 APSDE 检查每个绑定表条目的目标地址信息。如果这表示一个设备本身，那么 APSDE 就发出一个 APSDE-DATA.indication 原语给上层，DstEndpoint 参数设置为绑定表条目的目标端点标识符。否则，如果存在的话，APSDE 建立端点信息来自绑定表条目的 APDU，并当通过 NWK 层传输该帧时，使用绑定表条目的目标地址信息。如果存在多个绑定表条目，那么 APSDE 按照上面所述处理每个绑定表条目；直到没有剩余的绑定表条目。如果这个原语由不支持绑定

表的一个设备的 APSDE 接收, APSDE 发出状态为 NOT_SUPPORTED 的 APSDE-DATA.confirm 原语。

如果 DstAddrMode 参数设置为 0x03, DstAddress 参数包含一个扩展 64 位 IEEE 地址, 必须首先通过使用 *nwkAddressMap* NIB 属性 (见表 3.43) 映射为一个相应的 16 位 NWK 地址。如果找不到一个相应的 16 位 NWK 地址, APSDE 就发出状态为 NO_SHORT_ADDRESS 的 APSDE-DATA.confirm 原语。如果找到了一个相应的 16 位 NWK 地址, 它将被用于 NLDE-DATA.request 原语的调用, DstEndpoint 参数的值将被放在由此产生的 APDU 中。在这种情况下, APS 头的帧控制域的传送模式子域值应该是 0x00。

如果 DstAddrMode 参数的值是 0x01, 表示组寻址, DstAddress 参数将被解释为一个 16 位的组地址。该地址将被放在 APS 头的组地址域中, DstEndpoint 参数将被忽略, 目标端点域将从 APS 头中省略。在这种情况下, APS 头的帧控制域的传送模式子域值应该是 0x03。

如果 DstAddrMode 参数设置为 0x02, DstAddress 参数包含一个 16 位的 NWK 地址, 且 DstEndpoint 参数被提供。在目标 NWK 地址用于立即应用响应的情况下, 上层只使用 DstAddrMode 值为 0x02 的模式, 且 NWK 地址不再为以后的数据传输请求保留。

应用程序可以通过设置 NLDEDATA.request 原语的 RadiusCounter 参数为一个非零值, 限制一个帧传输可以通过网络转发的跳数。

如果 DstAddrMode 参数的值是 0x01, 表示组寻址, 或 DstAddrMode 参数的值是 0x00 且相应的绑定表条目包含一个组地址, 那么 APSME 检查 *nwkUseMulticast* NIB 属性 (见表 3.44) 的值。如果这个属性值为 FALSE, 那么由此产生的 APDU 的帧控制域的传送模式子域设置为 0b11, 目标组的 16 位地址将被放在输出帧的 APS 头的组地址域中, 且 NSDU 帧将被作为一个广播传输。传输该帧时, NLDE-DATA.request 的 DstAddr 参数被提供一个值 0xffffd, 即广播给所有 macRxOnWhenIdle = TRUE 的设备。如果 *nwkUseMulticast* 属性值为 TRUE, 那么输出帧将使用 NWK 层单播传输, APDU 的帧控制域的传送模式子域设置为 0b10, 目标端点域设置为 0xff, 且组地址不放在 APS 头中。

如果 TxOptions 参数指明要求安全传输, APS 子层应该使用安全服务供应商 (参见 4.2.3 小节) 来加密 ASDU。如果加密过程失败, APSDE 应该发出状态值为 SECURITY_FAIL 的 APSDE-DATA.confirm 原语。

APSDE 通过给 NWK 层发出 NLDE-DATA.request 原语来传输所建立的帧。当 APSDE 完成所有关于这个传输请求的操作, 包括按照需要传输帧, 任何重传, 以及任何确认的接收或超时, 然后 APSDE 发出 APSDE-DATA.confirm 原语 (见 2.2.4.1.2 小节)。如果一个或多个 NLDE-DATA.confirm 原语失败, 那么状态参数设置为从 NWK 层收到的状态。否则, 如果没有正确收到一个或多个 APS 确认, 那么状态参数设置为 NO_ACK。如果 ASDU 成功传输到所有预定目标, 那么状态参数设置为 SUCCESS。

如果使用了 NWK 层多播, NLDE-DATA.request 原语的 NonmemberRadius 参数设置为 *apsNonmemberRadius*。

APSDE 通过每次发出 NLDE-DATA.request 原语时, 设置它的 DiscoverRoute 参数值为 0x01, 将确保网络层的保证路由发现功能总是开启的。

如果要传输的 ASDU 大于一个合适的帧且不能分段, 那么 ASDU 就不被传输, APSDE 应该发出状态值为 ASDU_TOO_LONG 的 APSDE-DATA.confirm 原语。如果传输没有请求带有一个确认, 或如果 TxOptions 域的分段允许标志设置为 0, 或如果 ASDU 太大而不能被 APSDE 处理, 分段就不可能。

如果要传输的 ASDU 大于一个合适的帧, 传输请求了一个确认, 且 TxOptions 域的分段允许标志设置为 1, 且 ASDU 不是太大而不能被 APSDE 处理, 那么 ASDU 将分成多个 APDU, 如 2.2.8.4.5 小节所述。每个单独的 APDU 独立必须携带所请求的传输和安全处理。注意除非相关的高层文件



和/或条款明确指明被发送的帧允许分段，且无论块的个数和总传输大小，另一端都可以接收分段的传输，分段就不能使用。

2.2.4.1.2 APSDE-DATA.confirm

该原语报告请求从一个本地 NHLE 传输一个数据 PDU (ASDU) 到一个对等 NHLE 的结果。

2.2.4.1.2.1 服务原语的语义

本原语的语义如下：

APSDE-DATA.confirm {

DstAddrMode,

DstAddress,

DstEndpoint,

SrcEndpoint,

Status,

TxTime

}

表 2.3 指明了 APSDE-DATA.confirm 原语的参数。

表 2.3 APSDE-DATA.confirm 参数

名称	类型	有效范围	描述
DstAddrMode	Integer	0x00-0xff	该原语和被传输的 PDU(APDU)使用的目标地址寻址模式。该参数可从下列表中取一个非保留值： 0x00=DstAddress 和 DstEndpoint 不存在 0x01=DstAddress 16 位组地址；DstEndpoint 不存在 0x02=DstAddress 和 DstEndpoint 都存在，是 16 位地址 0x03=DstAddress 和 DstEndpoint 都存在，是 64 位扩展地址 0x04-0xff=保留
DstAddress	Address	通过 DstAddrMode 参数来指定	ASDU 要被传输到的实体的单个设备的地址或者组地址。
DstEndpoint	Integer	0x00-0xf0,0xff	当且仅当 DstAddrMode 参数的值为 0x02 或者 0x03 时，该参数才存在，如果存在，该参数是 APDU 要被传输到的实体的单个端点的编号。

表 2.3 APSDE-DATA.confirm 参数

SrcEndpoint	Integer	0x00-0xf0	ASDU 要被传输到的实体的单个端点
status	Enumeration	SUCCESS, NO_SHORT_ADDRESS, NO_BOUND_DEVICE, SECURITY_FAIL, NO_ACK , ASDU_TOO_LONG 以及从 NLDE-DATA.confirm 原语返回的任何状态返回值。	相应请求的状态
TxTime	Integer	具体执行	对基于本地时钟的传输包的时间指示，如 NMK 层所提供的那样。

2.2.4.1.2.2 原语生成

本原语由本地 APS 子层实体在响应一个 APSDE-DATA.request 原语时生成。本原语返回一个状态值 SUCCESS，表示传输请求成功，或一个 NO_SHORT_ADDRESS，NO_BOUND_DEVICE 或 SECURITY_FAIL 的错误代码，或任何 NLDE-DATA.confirm 原语返回的状态值。这些状态值的原因在 2.2.4.1.1.3 节中做了详细描述。

2.2.4.1.2.3 接收结果

在接收本原语后，发出设备的上层被通知其传输请求的结果。如果尝试传输成功，状态参数将被设置为 SUCCESS。否则，状态参数将指示错误。

2.2.4.1.3 APSDE-DATA.indication

本原语表示从 APS 子层传输一个数据 PDU（ASDU）到本地应用实体。

2.2.4.1.3.1 服务原语的语义

本原语的语义如下：

APSDE-DATA.indication	{
	DstAddrMode,
	DstAddress,
	DstEndpoint,
	SrcAddrMode,
	SrcAddress,
	SrcEndpoint,
	ProfileId,
	ClusterId,
	asduLength,
	asdu,
	Status,
	SecurityStatus,
	LinkQuality,
	RxTime
	}

表 2.4 指明了 APSDE-DATA.indication 原语的参数。

表 2.4 APSDE-DATA.indication 参数

名称	类型	有效范围	描述
DstAddrMode	Integer	0x00 – 0xff	该原语和收到的 APDU 使用的目标地址寻址模式。此参数可以从下面的列表中取一个非保留值： 0x00 = 保留 0x01=DstAddress 是 16 位组地址，DstEndpoint 不存在 0x02=DstAddress 是 16 位地址且 DstEndpoint 存在 0x03-0xff=保留
DstAddress	Address	通过 DstAddrMode 所指定	ASDU 是针对单独地址或组地址
DstEndpoint	Integer	0x00 – 0xf0	ASDU 所针对的是本地实体上的目标端点

表 2.4 APSDE-DATA.indication 参数

名称	类型	有效范围	描述
SrcAddrMode	Integer	0x00 – 0xff	该原语和收到的 APDU 使用的源地址寻址模式。此参数可以从下面的列表中取一个非保留值： 0x00 = 保留 0x01 = 保留 0x02 =SrcAddress 和 SrcEndpoint 都存在，是 16 位短地址 0x03 = SrcAddress 和 SrcEndpoint 都存在，是 64 位扩展地址 0x04 – 0xff=保留
SrcAddress	Address	通过 SrcAddrMode 参数指定	来自接收到的 ASDU 的实体的单个设备的地址或组地址
SrcEndpoint	Integer	0x00 – 0xf0	来自接收到的 ASDU 的实体的单个端点的编号
ProfileId	Integer	0x0000-0xffff	发起这一帧的 profile 的标识符
ClusterId	Integer	0x0000-0xffff	接收对象的标识符
asduLength	Integer		通过 APSDE 表示的 ASDU 包括的字节数
asdu	Set of octets	-	通过 APSDE 表示的 ASDU 包括的字节集合
Status	Enumeration	SUCCESS, DEFRAG_UN SUPPORTED, DEFRAG_DE FRED 或者 帧安全处理返回的任何状态	输入帧处理的状态

表 2.4 APSDE-DATA.indication 参数

名称	类型	有效范围	描述
SecurityStatus	Enumeration	UNSECURED, SECURED_NWK_KEY, SECURED_LINK_KEY	如果在没有任何安全情况下接收 ASDU, 是 UNSECURED 如果接收到的 ASDU 用网络密钥加密, 是 SECURED_NWK_KEY 如果 ASDU 使用一个连接密钥加密, 是 SECURED_LINK_KEY
LinkQuality	Integer	0x00 - 0xff	NLDE 传送的链路质量指示
RxTime	Integer	具体执行	对基于本地时钟的传输包的时间指示, 如 NMK 层所提供的那样。

2.2.4.1.3.2 原语生成

当从本地 NWK 层实体收到一个适当寻址数据帧, 或随后收到一个 DstAddrMode 参数设置为 0x00 且绑定表条目指示帧送到设备本身的 APSDE-DATA.request 时, 本原语由 APS 子层生成并发给上层。如果 ASDU 头的帧控制域指明帧被保护, 应该执行 4.2.3 小节中规定的安全程序。

当从本地网络层实体收到一个适当寻址数据帧时, 本原语由 APS 子层实体产生, 通过 NLDE-DATA.indication 原语发给上层实体。

如果帧 APDU 头的控制域指明该帧被保护, 那么必须执行 4.2.3 小节中规定的安全程序。如果帧处理失败, APSDE 设置状态参数为安全处理返回的安全错误代码。

如果帧未被保护或安全处理成功, APSDE 必须检查帧是否被分段。如果扩展头包含在 APDU 头中且扩展帧控制域的分段子域表示帧被分段, 但是这个设备不支持分段, APSDE 就设置状态参数为 DEFRAG_UNSUPPORTED。如果扩展头包含在 APDU 头中且扩展帧控制域的分段子域表示帧被分段, 且这个设备支持分段, 但是当前不能重组该帧, APSDE 就设置状态参数为 DEFRAG_DEFERRED。

在所有其它情况下, APSDE 设置状态参数为 SUCCESS。

如果状态参数不是设置为 SUCCESS，APSDE 设置 ASDULength 参数为 0，ASDU 参数为空字节集合。

APS 子层实体必须通过使用 NIB 属性 *nwkAddressMap*（见表 3.43）尝试把收到的帧的源地址映射为它相应的扩展 64 位 IEEE 地址。如果相应的 64 位 IEEE 地址可以找到，APSDE 发出本原语，SrcAddrMode 参数设置为 0x03，且 SrcAddress 参数设置为相应的 64 位 IEEE 地址。如果相应的 64 位 IEEE 地址找不到，APSDE 发出本原语，SrcAddrMode 参数设置为 0x02，且 SrcAddress 参数设置为收到的帧所包含的 16 位源地址。

2.2.4.1.3.3 接收结果

在接收到本原语后，上层被通知数据到达设备。

2.2.4.2 APS 管理服务

APS 管理实体 SAP (APSME-SAP) 支持上层和 APSME 之间管理命令的传输。表 2.5 总结了 APSME 通过 APSME-SAP 接口支持的原语。参见以下小节获得更多关于各个原语的信息。

表 2.5 通过 APSME-SAP 原语访问综述

名称	请求	指示	响应	确认
APSME-BIND	2.2.4.3.1			2.2.4.3.2
APSME -UNBIND	2.2.4.3.3			2.2.4.3.4
APSME-GET	2.2.4.4.1			2.2.4.4.2
APSME-SET	2.2.4.4.3			2.2.4.4.4
APSME-ADD-GROUP	2.2.4.5.1			2.2.4.5.2
APSME-REMOVE- GROUP	2.2.4.5.3			2.2.4.5.4
APSME-REMOVE- ALL-GROUPS	2.2.4.5.5			2.2.4.5.6

2.2.4.3 绑定原语

这组原语定义了设备的上层如何往其本地绑定表中增加（提交）一个绑定记录，或从其本地绑定表中移除一个绑定记录。

只有支持绑定表或绑定表缓存的一个设备，可以处理这些原语。如果任何其它设备从其上层接收到这些原语，原语将被丢弃。

2.2.4.3.1 APSME-BIND.request

如果支持的话，本原语允许上层通过在其本地绑定表中创建一个实体，请求把两个设备绑定到一起，或绑定一个设备到一个组。

2.2.4.3.1.1 服务原语的语义

本原语的语义如下：

APSME-BIND.request	{
SrcAddr,	
SrcEndpoint,	
ClusterId,	
DstAddrMode,	
DstAddr,	
DstEndpoint	
	}

表 2.6 指明了 APSME-BIND.request 原语的参数。

表 2.6 APSME-BIND.request 参数

名称	类型	有效范围	描述
SrcAddr	IEEE Address	有效的 64 位的 IEEE 地址	绑定目录的源 IEEE 地址
SrcEndpoint	Integer	0x01 – 0xff	绑定目录的源端点
ClusterId	Integer	0x0000 – 0xffff	将要绑定到目的设备的源设备上的 cluster 标识符。

表 2.6 APSME-BIND.request 参数

DstAddrMode	Integer	0x00 – 0xff	该原语使用的目标地址寻址模式。该参数可以从下面的列表取一个非保留值： 0x00=保留 0x01= DstAddr 16 位组地址，DstEndpoint 不存在 0x02=保留 0x03= DstAddr 和 DstEndpoint 都存在，是 64 位扩展地址 0x04- 0xff=保留
DstAddr	Address	通过 DstAddrMode 参数来指定	绑定目录的目的地址
DstEndpoint	Integer	0x01 – 0xf0, 0xff	当且仅当 DstAddrMode 参数的值为 0x03 时，该参数才存在，如果存在，该参数将是绑定目录的目标端点

2.2.4.3.1.2 原语生成

本原语由上层实体生成，发给 APS 子层，目的是在一个支持绑定表的设备上发起一个绑定操作。

2.2.4.3.1.3 接收结果

当前没有加入到网络或不支持绑定表的一个设备接收到本原语后，APSME 发出 Status 参数设置为 ILLEGAL_REQUEST 的 APSME-BIND.confirm 原语。

如果支持绑定表的设备的 APS 子层从 NHLE 接收到本原语，APSME 尝试在其绑定表中直接创建一个规定的实体。如果实体可以创建，APSME 发出 Status 参数为 SUCCESS 的 APSME-BIND.confirm 原语。如果因为绑定表容量缺乏实体不能创建，APSME 发出 Status 参数为 TABLE_FULL 的 APSME-BIND.confirm 原语。

2.2.4.3.2 APSME-BIND.confirm

本原语允许上层被通知其直接或代理请求绑定两个设备，或绑定一个设备到一个组的结果。

2.2.4.3.2.1 服务原语的语义

本原语的语义如下：

APSME-BIND.confirm { Status, SrcAddr, SrcEndpoint, ClusterId, DstAddrMode, DstAddr, DstEndpoint }

表 2.7 指明了 APSME-BIND.confirm 原语的参数。

表 2.7 APSME-BIND.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS, ILLEGAL_DEVICE, ILLEGAL_REQUEST, TABLE_FULL, NOT_SUPPORTED	绑定请求结果
SrcAddr	IEEE address	有效的 64 位 IEEE 地址	绑定目录的源 IEEE 地址
SrcEndpoint	Integer	0x01 – 0xf0	绑定目录的源端点
ClusterId	Integer	0x0000 – 0xffff	将要绑定到目标设备的源设备 cluster 标识符

表 2.7 APSME-BIND.confirm 参数

DstAddrMode	Integer	0x00 – 0xff	该原语使用的源地址寻址模式。此参数可以从下面的列表中取的一个非保留值： 0x00=保留 0x01=DstAddr16 位组地址，DstEndpoint 不存在 0x02=保留 0x03= DstAddr 和 DstEndpoint 都存在，是 64 位扩展地址 0x04-0xff=保留
DstAddr	Address	通过 DstAddrMode 参数来指定	绑定目录的目标地址
DstEndpoint	Integer	0x01 – 0xff, 0xff	当且仅当 DstAddrMode 参数值为 0x03 时，该参数才存在，如果存在，该参数将是绑定目录的目标端点

2.2.4.3.2.2 原语生成

本原语由 APSME 生成，发给其 NHLE，以响应一个 APSME-BIND.request 原语。如果该请求成功，Status 参数将表示一个成功的绑定请求。否则，状态参数表示 NOT_SUPPORTED,ILLEGAL_REQUEST 或 TABLE_FULL 的一个错误代码。

2.2.4.3.2.3 接收结果

收到本原语，上层被告知其绑定请求的结果。如果绑定请求成功，状态参数设置为 SUCCESS。否则，状态参数表示错误。

2.2.4.3.3 APSME-UNBIND.request

如果支持的话，本原语允许上层通过移除其本地绑定表中的一个实体，请求取消绑定两个设备，或从组中取消绑定一个设备。

2.2.4.3.3.1 服务原语的语义

原语的语义如下：

APSME-UNBIND.request	{ SrcAddr, SrcEndpoint, ClusterId, DstAddrMode, DstAddr, DstEndpoint }
----------------------	---

表 2.8 指明了 APSME-UNBIND.request 原语的参数。

表 2.8 APSME-UNBIND.request 参数

名称	类型	有效范围	描述
SrcAddr	IEEE Address	有效的 64 位 IEEE 地址	绑定目录的源 IEEE 地址
SrcEndpoint	Integer	0x01 – 0xff	绑定目录的源端点
ClusterId	Integer	0x0000 – 0xffff	绑定到目标设备的源设备上的 cluster 标识符
DstAddrMode	Integer	0x00 — 0xff	源地址寻址模式用于该原语, 此参数可以从下面的列表中取的一个非保留值: 0x00:预留 0x01: DstAddr16 位组地址和 DstEndpoint 未指定 0x02=预留的 0x03= DstAddr64 位扩展地址和 DstEndpoint 指定 0x04-0xff=预留的
DstAddr	Address	通 过 DstAddrMode 参数指定	绑定目录的目标地址
DstEndpoint	Integer	0x01 — 0xf0, 0xff	本参数只在 DstAddrMode 参数有值 0x03 才会给定, 如果给定, 即为绑定目录的目的地址。

2.2.4.3.3.2 原语生成

本原语由上层生成，发给 APS 子层，目的是在一个支持绑定表的设备上发起一个取消绑定操作。

2.2.4.3.3.3 接收结果

当前没有加入网络或不支持绑定表的一个设备接收到本原语时，APSME 发出 Status 参数设置为 ILLEGAL_REQUEST 的 APSME-UNBIND.confirm 原语。

如果支持绑定表的设备的 APS 从 NHLE 接收到本原语，APSME 在其绑定表中寻找指明的条目。如果该条目存在，APSME 移除该条目，并发出 Status 参数设置为 SUCCESS 的 APSME-UNBIND.confirm 原语（参见 2.2.4.3.4 小节）。如果找不到该条目，APSME 发出 Status 参数设置为 INVALID_BINDING 的 APSME-UNBIND.confirm 原语。

2.2.4.3.4 APSME-UNBIND.confirm

本原语允许上层被通知其请求取消绑定两个设备，或从一个组中取消绑定一个设备的结果。

2.2.4.3.4.1 服务原语的语义

本原语的语义如下：

APSME-UNBIND.confirm	{
	Status,
	SrcAddr,
	SrcEndpoint,
	ClusterId,
	DstAddrMode,
	DstAddr,
	DstEndpoint
	}

表 2.9 指明了 APSME-UNBIND.confirm 原语的参数。

表 2.9 APSME-UNBIND.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS, ILLEGAL_DEVICE, ILLEGAL_REQUEST, INVALID_BINDING	取消绑定请求的结果
SrcAddr	IEEE address	有效的 64 位 IEEE 地址	绑定目录的源 IEEE 地址
SrcEndpoint	Integer	0x01 – 0xf0	绑定目录的源端点
ClusterId	Integer	0x0000 – 0xffff	绑定到目的设备的源设备的 Cluster 标识符
DstAddrMode	Integer	0x00 – 0xff	该原语使用的源地址寻址模式。此参数可以从下面的列表中取一个非保留值： 0x00=保留 0x01=DstAddr16 位组地址，DstEndpoint 不存在 0x02=保留 1. 0x03=DstAddr 和 DstEndpoint 都存在，是 64 位扩展地址 0x04-0xff=保留
DstAddr	Address	通过 DstAddrMode 参数来指定	绑定目录的目的地址
DstEndpoint	Integer	0x01 – 0xf0, 0xff	绑定目录的目的端点

2.2.4.3.4.2 原语生成

本原语由 APSME 生成，发给其 NHLE，以响应一个 APSME-UNBIND.request 原语。如果该请求成功，Status 参数将表示一个成功的取消绑定请求。否则，状态参数表示一个 ILLEGAL_REQUEST 或 INVALID_BINDING 的错误代码。

2.2.4.3.4.3 接收结果

在接收到本原语时，上层被通知其取消绑定请求的结果。如果取消绑定请求成功，Status 参数设置为 SUCCESS。否则，Status 参数表示错误。

2.2.4.4 信息库维护

这组原语定义了设备的上层如何读取和写入 AIB 中的属性。

2.2.4.4.1 APSME-GET.request

本原语允许上层从 AIB 中读取一个属性的值。

2.2.4.4.1.1 服务原语的语义

本原语的语义如下：

APSME-GET.request	{ AIBAttribute }
-------------------	------------------------

表 2.10 指明了本原语的参数。

表 2.10 APSME-GET.request 参数

名称	类型	有效范围	描述
AIB 属性	Integer	见表 2.24	要读取的 AIB 属性的标识符

2.2.4.4.1.2 原语生成

本原语由上层生成，发给其 APSME，目的是从 AIB 中读取一个属性。

2.2.4.4.1.3 接收结果

在接收到本原语时，APSME 尝试从其数据库中检索被请求的 AIB 属性。如果在数据库中找到 AIB 属性的标识符，APSME 发出状态值为 UNSUPPORTED_ATTRIBUTE 的 APSME-GET.confirm 原语。

如果被请求的 AIB 属性检索成功，APSME 发出状态值为 SUCCESS 的 APSME-GET.confirm 原语，它包含 AIB 属性标识符和值。

2.2.4.4.2 APSME-GET.confirm

本原语报告尝试从 AIB 中读取一个属性值的结果。

2.2.4.4.2.1 服务原语的语义

本原语的语义如下：

APSME-GET.confirm	{
	Status,
	AIBAttribute,
	AIBAttributeLength,
	AIBAttributeValue
	}

表 2.11 指明了本原语的参数。

表 2.11 APSME-GET.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS或 UNSUPPORTED_ATTR IBUTE	请求读取 AIB 属性值 的结果
AIBAttribute	Integer	见表 2.24	被读取的 AIB 属性的 标识符
AIBAttributeLen gth	Integer	0x0000 - 0xffff	返回属性值的字节长 度，以八位字节为单 位
AIBAttributeVal ue	Various	具体属性 (见表 2.24)	被读取的 AIB 属性的 值

2.2.4.4.2.2 原语生成

本原语由 APSME 生成，发给其上层，以响应一个 APSME-GET.request 原语。本原语返回一个 SUCCESS 的状态，表示请求读取一个 AIB 属性成功，或一个错误代码 UNSUPPORTED_ATTRIBUTE。这些状态值的原因在 2.2.4.4.1.3 小节中做了详细描述。

2.2.4.4.2.3 接收结果

在接收到本原语时，上层被通知其请求读取一个 AIB 属性的结果。如果请求读取一个 AIB 属性成功，Status 参数将被设置为 SUCCESS。否则，状态参数显示错误。

2.2.4.4.3 APSME-SET.request

本原语允许上层往 AIB 写入一个属性值。

2.2.4.4.3.1 服务原语的语义

本原语的语义如下：

```
APSME-SET.request  {  
  
    AIBAttribute,  
  
    AIBAttributeLength,  
  
    AIBAttributeValue  
  
}
```

表 2.12 说明了本原语的参数。

表 2.12 APSME-SET.request 参数

名称	类型	有效范围	描述
AIBAttribute	Integer	见表 2.24	要写的 AIB 属性的标识符
AIBAttributeLength	Integer	0x0000 - 0xffff	要设置的属性值的字节长度，以八位字节为单位
AIBAttributeValue	Various	具体属性 (见表 2.24)	要写的 AIB 属性的值

2.2.4.4.3.2 原语生成

本原语由上层生成，发给其 APSME，目的是往 AIB 写入一个属性值。

2.2.4.4.3.3 接收结果

在接收到本原语时，APSME 尝试在其数据库中给指明的 AIB 属性写入给定的值。如果在数据库中找到 AIBAttribute 参数指明的属性，APSME 发出状态值为 UNSUPPORTED_ATTRIBUTE 的 APSME-SET.confirm 原语。如果 AIBAttribute 参数指明的属性在给定属性的合法范围之外，APSME 发出状态值为 INVALID_PARAMETER 的 APSME-SET.confirm 原语。

如果被请求的 AIB 属性成功写入，APSME 发出状态值为 SUCCESS 的 APSME-SET.confirm 原语。

2.2.4.4.4 APSME-SET.confirm

本原语报告尝试为一个 AIB 属性写入一个值的结果。

2.2.4.4.4.1 服务原语的语义

本原语的语义如下：

APSME-SET.confirm	{
	Status,
	AIBAttribute
	}

表 2.13 说明了本原语的语义。

表 2.13 APSME-SET.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS, INVALID_PARAMETER或 UNSUPPORTED_ATTRIBUTE	写 AIB 属性请求的结果
AIBAttribute	Integer	见表 2.24	被写 AIB 属性的标识符

2.2.4.4.4.2 原语生成

本原语由 APSME 生成，发给其上层，以响应一个 APSME-SET.request 原语。本原语返回一个状态值 SUCCESS，表示该请求值被写入指明的 AIB 属性，或一个 INVALID_PARAMETER 或 UNSUPPORTED_ATTRIBUTE 的错误代码。这些状态值的原因在 2.2.4.4.3.3 小节中做了详细描述。

2.2.4.4.4.3 接收结果

在接收到本原语时，上层被通知其请求写入一个 AIB 属性值的结果。如果请求值写入指定的 AIB 属性，Status 参数将被设置为 SUCCESS。否则，Status 参数显示错误。

2.2.4.5 组管理

这组原语允许上层通过增加和移除组表中的条目，为当前设备上的端点管理组成员。

2.2.4.5.1 APSME-ADD-GROUP.request

本原语允许上层为某个特定端点请求往某个特定的组增加组成员。

2.2.4.5.1.1 服务原语的语义

本原语的语义如下：

APSME-ADD-GROUP.request	{ GroupAddress, Endpoint }
-------------------------	-------------------------------------

表 2.14 说明了本原语的参数。

表 2.14 APSME-ADD-GROUP.request 参数

名称	类型	有效范围	描述
GroupAddress	16 位组地址	0x0000 - 0xffff	将要添加的 16 位组地址
Endpoint	Integer	0x01 - 0xf0	将要添加的特定组的端点

2.2.4.5.1.2 原语生成

本原语由上层生成，当它希望给一个端点的某个组增加成员时，这样要送到该组的帧稍后可以被传送到该端点。

2.2.4.5.1.3 接收结果

如果在接收到本原语时，发现 GroupAddress 参数在合法范围之外，那么 APSME 将给上层发出 Status 值为 INVALID_PARAMETER 的 APSME-ADD-GROUP.confirm 原语。同样，如果 Endpoint 参数值为 0x00，要不然列举了不在当前设备上的端点，APSME 将发出 Status 值为 INVALID_PARAMETER 的 APSME-ADD-GROUP.confirm 原语。

如上文所述，在检查了参数之后，APSME 将检查组表，查看是否已经存在一个包含 GroupAddress 和 Endpoint 参数给定值的条目。如果这样的条目存在于表中，那么 APSME 将给上层发出 Status 值为 SUCCESS 的 APSME-ADD-GROUP.confirm 原语。如果没有这样的条目，且表中还有另外条目的空间，那么 APSME 将给组表增加一个带有 GroupAddress 和 Endpoint 参数给定值的新条目。增加条目之后，且如果 NWK 层维护了一个组表，那么 APSME 通过发出 NLME-SET.request 原语，确保相应的 NWK 组表中的 *nwkGroupIDTable* 属性和 APS 子层组表所含的组地址列表一致。一旦这两个表一致，APSME 将给上层发出 Status 值为 SUCCESS 的 APSME-ADD-GROUP.confirm 原语。如果没有给定 GroupAddress 和 Endpoint 所示的条目，但是组表中没有另外条目的空间，那么 APSME 将给上层发出 Status 值为 TABLE_FULL 的 APSME-ADD-GROUP.confirm 原语。

2.2.4.5.2 APSME-ADD-GROUP.confirm

本原语允许上层被通知其请求给一个端点增加一个组的结果。

2.2.4.5.2.1 服务原语的语义

服务原语的语义如下：

APSME-ADD-GROUP.confirm	{ Status, GroupAddress, Endpoint }
-------------------------	--

表 2.15 说明了本原语的参数

表 2.15 APSME-ADD-GROUP.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS, INVALID_PARAMETER 或 TABLE_FULL	添加一个组的请求的状态
GroupAddress	16 位组地址	0x0000 - 0xffff	将要被添加的 16 位组地址
Endpoint	Integer	0x01 - 0xf0	将要被添加的特定组的端点

2.2.4.5.2.2 原语生成

本原语由 APSME 生成，发给上层，以响应一个 APSME-ADD-GROUP.request 原语。如果 APSME-ADD-GROUP.request 成功，那么 Status 参数值将是 SUCCESS。如果 APSME-ADD-GROUP.request 原语的参数中有无效的值，那么 Status 参数值将被设置为 INVALID_PARAMETER。如果 APSME 尝试增加一个组表条目，但是表中没有另外条目的空间，那么 Status 值将为 TABLE_FULL。

2.2.4.5.2.3 接收结果

在接收到本原语时，上层被通知其请求增加一个组的状态。Status 参数值如上文所述。

2.2.4.5.3 APSME-REMOVE-GROUP.request

本原语允许上层请求移除某个特定端点的某个组中的组成员。

2.2.4.5.3.1 服务原语的语义

服务原语的语义如下：

APSME-REMOVE-GROUP.request	{ GroupAddress, Endpoint }
----------------------------	-------------------------------------

表 2.16 说明了本原语的语义。

表 2.16 APSME-REMOVE-GROUP.request 参数

名称	类型	有效范围	描述
GroupAddress	16 位组地址	0x0000 - 0xffff7	将要被移除的组的 16 位地址
Endpoint	Integer	0x01 - 0xf0	将要移除的特定组的端点

2.2.4.5.3.2 原语生成

本原语由上层生成，当它希望从一个端点上移除某个组的成员，这样要送到该组的帧不再传送到该端点。

2.2.4.5.3.3 接收结果

如果在接收到本原语时，发现 GroupAddress 参数在合法范围之外，那么 APSME 将给上层发出 Status 值为 INVALID_PARAMETER 的 APSME-REMOVE-GROUP.confirm 原语。同样，如果 Endpoint 参数值为 0x00，或者列举了不在当前设备上的端点，APSME 将发出 Status 为 INVALID_PARAMETER 的 APSME-REMOVE-GROUP.confirm 原语。

如上文所述，在检查了参数之后，APSME 将检查组表，查看是否已经存在一个包含 GroupAddress 和 Endpoint 参数给定值的条目。如果这样的条目存在于表中，条目将被移除。如果 NWK 层维护一个组表，那么 APSME 通过发出 NLME-SET.request 原语，保证 NWK 组表中的 *nwkGroupIDTable* 属性和 APS 子层组表所含的组地址列表一致。一旦这两个表一致，APSME 将给上层发出 Status 值为 SUCCESS 的 APSME-REMOVE-GROUP.confirm 原语。如果没有这样的条目，APSME 将给上层发出 Status 值为 INVALID_GROUP 的 APSME-REMOVE-GROUP.confirm 原语。

2.2.4.5.4 APSME-REMOVE-GROUP.confirm

本原语允许上层被通知其请求从一个端点移除一个组的结果。

2.2.4.5.4.1 服务原语的语义

服务原语的语义如下：

APSME-REMOVE-GROUP.confirm	{
	Status,
	GroupAddress,
	Endpoint
	}

表 2.17 说明了本原语的参数。

表 2.17 APSME-REMOVE-GROUP.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS或 INVALID_PARAMETER	移除一个组的请求的状态
GroupAddress	16 位组地址	0x0000 - 0xffff	将要被移除的组的 16 位地址
Endpoint	Integer	0x01 - 0xf0	从组中被移除的端点

2.2.4.5.4.2 原语生成

本原语由 APSME 生成，发给上层，以响应一个 APMSE-REMOVE-GROUP.request 原语。如果 APSME-REMOVE-GROUP.request 成功，Status 参数值将是 SUCCESS。如果因为不存在包含 GroupAddress 和 Endpoint 参数给定值的一个条目，APMSE-REMOVE-GROUP.request 不成功，那么状态值将是 INVALID_GROUP。如果 APMSE-REMOVE-GROUP.request 原语的参数之一有无效的值，那么 Status 值将是 INVALID_PARAMETER。

2.2.4.5.4.3 接收结果

在接收到本原语时，上层被通知其请求移除一个组的结果。Status 参数值如上文所述。

2.2.4.5.5 APSME-REMOVE-ALL-GROUPS.request

本原语由上层生成，当它希望从一个端点移除所有组的成员时，这样不再有送到那些组的帧被传送到该端点。

2.2.4.5.5.1 服务原语的语义

服务原语的语义如下:

```

APSME-REMOVE-ALL-GROUPS.request {
    Endpoint
}

```

表 2.18 说明了本原语的参数。

表 2.18 APSME-REMOVE-ALL-GROUPS.request 参数

名称	类型	有效范围	描述
Endpoint	Integer	0x01 - 0xf0	被移除的特定组中的端点

2.2.4.5.2 原语生成

本原语由上层生成，当它希望从一个端点移除所有组的成员时，这样不再有送到那些组的帧被传送到该端点。

2.2.4.5.3 接收结果

如果在接收到本原语时，Endpoint 参数值为 0x00，要不然列举了不再当前设备上的端点，APSME 将发出 Status 值为 INVALID_PARAMETER 的 APSME-REMOVE-ALL-GROUPS.confirm 原语。

如上文所述，在检查了 Endpoint 参数之后，APSME 将从组表中移除与该端点相关的所有条目。APSME 通过发出 NLME-SET.request 原语，保证 NWK 组表中的 *nwkGroupIDTable* 属性和 APS 子层组表所含的组地址列表一致。一旦这两个表一致，APSME 给上层发出 Status 值为 SUCCESS 的 APSME-REMOVE-ALL-GROUPS.confirm 原语。

2.2.4.5.6 APSME-REMOVE-ALL-GROUPS.confirm

本原语允许上层被通知其请求从一个端点移除所有组的结果。

2.2.4.5.6.1 服务原语的语义

服务原语的语义如下：

APSME-REMOVE-ALL-GROUPS.confirm	{ Status, Endpoint }
---------------------------------	-------------------------------

表 2.19 说明了本原语的参数。

表 2.19 APSME-REMOVE-ALL-GROUPS.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS, INVALID_PARAMETER 或 TABLE_FULL	移除所有组的请求的状态
Endpoint	Integer	0x01 - 0xf0	从所有组中被移除的端点

2.2.4.5.6.2 原语生成

本原语由 APSME 生成，发给上层，以响应一个 APSME-REMOVE-ALL-GROUPS.request 原语。如果 APSME-REMOVE-ALL-GROUPS.request 成功，那么 Status 参数值将是 SUCCESS。如果 APSME-REMOVE-ALL-GROUPS.request 的 Endpoint 参数有不合法的值，那么 Status 值将是 INVALID_PARAMETER。

2.2.4.5.6.3 接收结果

在接收到本原语时，上层被通知其请求从一个端点移除所有组的状态。Status 参数值如上文所述。

2.2.5 帧格式

本小节描述了 APS 帧（APDU）的格式。每个 APS 帧包括如下基本组成部分：

- 一个 APS 头，它包括帧控制和寻址信息。
- 一个 APS 负载，长度可变，它包括特定帧类型的信息。

APS 子层的帧被描述为有特定顺序的一系列域。本小节中所有的帧格式按照它们在 NWK 层传输的顺序描述，从左至右，最左边的位首先传输。每个域中的位从 0 开始编码（最左边和最低）到 k-1（最右边和最高），这样域的长度就是 k 位。长于一个八位字节的域被送往 NWK 层，按照从编号最低的八位字节到编号最高的八位字节的顺序。

传输时，所有标记为保留的域都设置为零。接收时，必须检查本规范版本中所有标记为保留的域是否等于零。如果这样的一个保留域不等于零，就不再对该帧进行进一步处理，且该帧必须被丢弃。

2.2.5.1 通用的 APDU 帧格式

APS 帧格式包括一个 APS 头和一个 APS 负载。APS 头的域以固定顺序出现，但是，地址域可能不会包括在所有的帧中。通用的 APS 帧应该如图 2.2 所示的格式编排。

字节：1	0/1	0/2	0/2	0/2	0/1	1	0/变量	变量
帧控制	目标端点	组地址	Cluster 标识符	Profile 标识符	源端点	APS 计数器	扩展头	帧 负载
	寻址域							
APS 头								APS 负载

图 2.2 通用 APS 帧格式

2.2.5.1.1 帧控制域

帧控制域长度为 8 位，包括定义帧类型的信息，寻址域和其它控制标志。帧控制域应该如图 2.3 所示的格式编排。

位:0-1	2-3	4	5	6	7
帧类型	传送模式	Ack.格式 ^a	安全	Ack.请求	扩展头存在

a. CCB #813

图 2.3 帧控制域的格式

2.2.5.1.1.1 帧类型子域

帧类型子域长度为两位，应该设置为表 2.20 所列的非保留值之一。

表 2.20 帧类型子域值

帧类型值 $b_1 b_0$	帧类型名称
00	数据

表 2.20 帧类型子域值

01	命令
10	确认
11	保留

2.2.5.1.1.2 传送模式子域

传送模式子域长度为两位，应该设置为表 2.21 所列的非保留值之一。

表 2.21 传送模式子域的值

传送模式值 $b_1 b_0$	传送模式名称
00	正常单播传送
01	间接寻址
10	广播
11	组寻址

如果值是 0b00，帧将被传送到接收设备上的一个给定的端点。

如果值是 0b10，信息是广播的。在这种情况下，信息将送到 3.6.5 小节定义的选定广播地址的所有设备。目标端点域设置为 0x01-0xf0（对于广播到特定端点）之间的一个值，或 0xff（对于广播到所有活动端点）。

如果值是 0b11，那么使用的是组寻址，该帧将只传送到 APS 头组地址域指定的组中表示组成员的设备端点。注意源设备的其它端点可能是输出的帧要去的组的成员。该帧应该被传送到组的任何成员，包括源设备上是指定组的成员的其它端点。

nwkUseMulticast 设置为 TRUE 的设备不能把输出帧的传送模式设置为 0b11。在这种情况下，输出帧的传送模式必须设置为 0b10（广播），使用目标地址模式设置为组寻址的一个 NLDEDATA.request 发送该帧。

2.2.5.1.1.3 Ack 格式域

这个位表示目标端点、cluster 标识符、profile 标识符和源端点域在确认帧中是否存在。这个位对于数据帧确认设置为 0，对于 APS 命令帧确认¹设置为 1。

2.2.5.1.1.4 安全子域

安全服务供应商（参见第 4 章）管理安全子域。

2.2.5.1.1.5 确认请求子域

确认请求子域长度为 1 位，指明了接收者在接收该帧时，当前传输是否需要给发起者发送一个确认帧。如果该子域设置为 1，接收者在确定该帧有效之后，给发起者构造并发送一个确认帧。如果该子域设置为 0，接收者不给发起者发送一个确认帧。

对于所有广播或多播帧，这个子域设置为 0。

2.2.5.1.1.6 扩展头存在

扩展头存在子域长度一位，规定扩展头是否包含在此帧内。如果该子域设置为 1，那么扩展头应该包含在此帧内。否则，不应该包含在此帧内。

2.2.5.1.2 目标端点域

目标端点域长度为 8 位，指明了最终接收该帧的端点。仅在帧控制域的传送模式子域设置为 0b00（正常单播传送），0b01（帧控制域的间接地址模式子域也设置为 0 时，使用间接传送），或 0b10（广播传送）时，帧中才包含该域。在广播传送的情况下，帧将被传送到规定在 0x01-0xf0 范围内的目标端点，或如果规定为 0xff，传送到所有活动端点。

如果目标端点值为 0x00，就把帧送到 ZigBee 设备对象（ZDO），驻扎在每个设备上。目标端点值为 0x01-0xf0，就把帧送到运行在该端点上的一个应用程序。目标端点值为 0xff，就把帧送到除过端点 0x00 的所有活动端点。所有其它端点（0xf1-0xfe）保留。

2.2.5.1.3 组地址域

组地址域长度为 16 位，仅在帧控制域的传送模式子域值为 0b11 时才存在。在这种情况下，目标端点不存在。如果帧的 APS 头包含一个组地址域，帧将传送到所有端点，其中设备的组表包含该端点和组地址域指明的组之间的一个联系。

1. CCB #813

nwkUseMulticast 设置为 TRUE 的设备不能设置输出帧的组地址域。

2.2.5.1.4 Cluster 标识符域

Cluster 标识符域长度为 16 位，指明了与该帧有关的 cluster 的标识符，以及哪个应该用于过滤和解释每个传送帧的设备上的信息。仅对于数据帧或确认帧这个域存在。

2.2.5.1.5 Profile 标识符域

Profile 标识符长度为 2 个字节，指明了帧要传输到的 ZigBee profile 标识符，应在每个传送帧的设备上的信息过滤期间使用。这个域仅对于数据帧或确认帧存在。

2.2.5.1.6 源端点域

源端点域长度为 8 位，指明了帧的初始发起者所在的端点。源端点值 0x00 表示帧从 ZigBee 设备对象（ZDO）发出，驻留在每个设备上。源端点值 0x01-0xf0 表示帧从运行在该端点的一个应用程序发出。所有其它端点（0xf1-0xff）保留。

2.2.5.1.7 APS 计数器

该域长度为 8 位，用于 2.2.8.3.2 小节中的描述，防止接收重复的帧。该值应该每次新的传输之后加一。

2.2.5.1.8 扩展头子帧

扩展头子帧包含更多子域，格式如图2.4。

字节：1	0/1	0/1
扩展帧控制	块数	ACK bitfield

图 2.4 扩展头子帧的格式

2.2.5.1.8.1 扩展帧控制域

扩展帧控制域 8 位长，包含定义使用分段的信息。扩展帧控制域的格式如图 2.5。

位：0-1	2-7
分段	保留

图 2.5 扩展帧控制域的格式

分段子域 2 位长，应该设定为表 2.22 中列出的一个非保留值。

表 2.22 分段子域的值

分段值 $b_1 b_0$	描述
00	传输不分段
01	帧是分段传输的第一段
10	帧是分段传输的部分但不是第一部分
11	保留

2.2.5.1.8.2 块数

块数域一字节长，用于分段控制，如下：如果分段子域设置为指示传输不分段，那么块数域不包含在子帧内。如果分段子域设置为 01，那么块数域应该包含在子帧内且应该指示分段传输的块数。如果分段子域设置为 10，那么块数域应该包含在子帧内且应该指示当前帧传输哪个块的编号表示，第二段取值 0x01，第三段取值 0x02 等等。

2.2.5.1.8.3 Ack Bitfield

Ack Bitfield 域一字节长，用于 2.2.8.4.5.2 小节所述的 APS 确认，指示一个分段的 ASDU 的哪一块已经被成功接收。本域只在帧类型子域指示一个确认且分段子域指示分段传输时才存在。

2.2.5.1.9 帧负载域

帧负载域长度可变，包含指明不同帧类型的信息。

2.2.5.2 不同帧类型的格式

定义了三种帧类型：数据，APS 命令和确认。

这些帧类型将在以下小节中一一讨论。

2.2.5.2.1 数据帧格式

数据帧应按图 2.6 所示的格式编排。

字节: 1	0/1	0/2	2	2	1	1	0/可变	可变
帧控制	目标端点	组地址	Cluster 标识符	Profile 标识符	源端点	APS 计数器	扩展头	帧负载
	寻址域							
APS 头								APS 负载

图 2.6 数据帧格式

数据帧域的顺序应遵循图 2.2 所示的通用 APS 帧的顺序。

2.2.5.2.1.1 数据帧 APS 头域

数据帧的 APS 头域包含帧控制，cluster 标识符，Profile 标识符，源端点和 APS 计数器域。根据帧控制域传送模式和扩展头存在子域的值，目标端点、组地址和扩展头域应包含在一个数据帧中。

在帧控制域中，帧类型子域应包含指明一个数据帧的值，如表 2.20 所示。所有其它子域应根据数据帧的预定用途合适的设置。

2.2.5.2.1.2 数据负载域

对于一个输出的数据帧，数据负载域应该包含部分或所有上层请求 APS 数据服务传输的八位字节序列。对于一个输入的数据帧，数据负载域应该包含所有或部分 APS 数据服务已经接收的，以及要被传送到上层的八位字节序列。

2.2.5.2.2 APS 命令帧格式

APS 命令帧格式应该按照图 2.7 所示的格式编排。

字节: 1	1	1	可变
帧控制	APS 计数器	APS 命令标识符	APS 命令负载
APS 头		APS 负载	

图 2.7 APS 命令帧格式

APS 命令帧域的顺序应该和图 2.2 所示的通用 APS 帧的顺序一致。

2.2.5.2.2.1 APS 命令帧的 APS 头域

APS 命令帧的 APS 头域应该包含帧控制和 APS 计数器域。在这个规范版本中，APS 命令帧不能分段，且扩展头域不存在。

在帧控制域中，帧类型子域应该包含表明一个 APS 命令帧的值，如表 2.20 所示。APS 命令负载应该根据 APS 命令帧的预定用途设置为合适的值。

2.2.5.2.2.2 APS 命令帧标识符域

APS 命令帧标识符域指明了使用的 APS 命令。

2.2.5.2.2.3 APS 命令负载域

APS 命令帧的 APS 命令负载域应该包含 APS 命令本身。

2.2.5.2.3 确认帧格式

确认帧格式应该按照图 2.8 所示的格式编排。

字节: 1	0/1	0/2	0/2	0/1	1	0/可变
帧控制	目标端点	cluster 标识符	Profile 标识符	源端点	APS 计数器	扩展头
APS 头						

图 2.8 确认帧格式

确认帧域的顺序应该和图 2.2 所示的通用 APS 帧的顺序一致。

2.2.5.2.3.1 确认帧的 APS 头域

如果帧控制域的 ack 格式域没有设置，目标端点、cluster 标识符、profile 标识符和源端点应该存在。这对于数据帧确认²不设置。源和目标端点域都应该包括在确认帧中。扩展头域应根据帧控制域的扩展头存在子域的值，包含在一个数据帧中。

在帧控制域中，帧类型子域应该包括指明一个确认帧的值，如表 2.20 所示。扩展头存在子域包含和确认帧同样的值。所有其它子域应该根据确认帧的预定用途设置合适的值。

如果帧控制域的 ack 格式域设置了，帧就是一个 APS 命令帧确认，且不能设置目标端点、cluster 标识符、profile 标识符和源端点域³。或者，如果一个 APS 数据帧被确认，源端点域应该映射被确认的帧目标端点域的值。同样，目标端点域应该映射被确认的帧源端点域的值。

APS 计数器域应该包含和该帧同样的值，而该帧是原来帧的一个确认帧。

如果扩展头存在，扩展帧控制域的分段子域应包含和确认帧同样的值。如果这个帧使用分段，那么块数和 ack bitfield 域应存在。存在的情况下，如果这是一个分段传输的第一个帧，块数域应包含值零，否则应包含和确认帧同样的值。

2.2.6 命令帧

本规范没有定义命令帧。关于 APS 命令帧和相关安全原语的详细描述参见 4.4.9 小节。

2. CCB #813
3. CCB #813

2.2.7 常量和 PIB 属性

2.2.7.1 APS 常量

定义了 APS 子层特征的常量列在表 2.23 中。

表 2.23 APS 子层常量

常量	描述	值
apscMaxDescriptorSize	一个非复杂的描述符包含的字节的最大数	64
apscMaxFrameRetries	传输失败后允许重传的最大次数	3
apscAckWaitDuration	等待确认传输一帧的最大秒数	$0.05 * (2 * nwkcMaxDepth) +$ (安全加密/解密延时), 其中(安全加密/解密延时) = 0.1 (采取每个加密或解密周期为 0.05)
apscMinDuplicateRejectionTableSize	APS 重复拒绝表所需的最小值	1
apscMaxWindowSize	分段参数-可以一次激活的未确认帧的最大数 (见 2.2.8.4.5 小节)	由栈 Profile 设定 (支持 1-8)
apscInterframeDelay	分段参数-发送一个分段传输的 2 个块之间的标准延迟 (见 2.2.8.4.5 小节)	由栈 Profile 设定
apscMinHeaderOverhead	APS 子层到 ASDU 增加的最小字节数	0x0C

2.2.7.2 APS 信息库

APS 信息库包含管理设备的 APS 层所要求的属性。AIB 的属性列在表 2.24 中。相关安全的 AIB 属性列在 4.4.10 小节中。

表 2.24 APS IB 属性

属性	标识符	类型	值域	描述	默认值
apsBindingTable	0xc1	Set	可变的	设备绑定表目录的当前集合 (见 2.2.8.2.1 小节)	空集
<i>apsDesignatedCoordinator</i>	0xc2	Boolean	TRUE 或 FALSE	如果在启动时设备成为 ZigBee 协调器, 为 TRUE, 否则为 FALSE	FALSE
<i>apsChannelMask</i>	0xc3	IEEE 802.15.4 信道掩码	对于 PHY 的任何有效掩码	用于这个设备网络操作的允许信道的掩码	所有信道
<i>apsUseExtendedPANID</i>	0xc4	64位扩展地址	0x00000000 to 0xffffffffffff	形成或加入网络的 64 位地址	0x0000000000000000
apsGroupTable	0xc5	Set	可变的	组表目录的当前集合 (见表 2.25)	空集
<i>apsNonmemberRadius</i>	0xc6	Integer	0x00-0x07	当使用 NWK 层多播时 NonmemberRadius 参数使用的值	2
<i>apsPermissionsConfiguration</i>	0xc7	见表格 2.23	可变的	允许配置项目的当前集合	空集
<i>apsUseInsecureJoin</i>	0xc8	Boolean	TRUE 或 FALSE	控制在启动时不安全加入的一个标志	TRUE
<i>apsInterframeDelay</i>	0xc9	Integer	0x00-0xff, (可由栈 Profile 限制)	分段参数-以微秒为单位, 发送一个分段传输的 2 个块之间的标准延迟 (见 2.2.8.4.5 小节)	由栈 Profile 设置

表 2.24 APS IB 属性（续表）

<i>apsLastChannelEnergy</i>	0xc a	Integer	0x00-0xff	信道改变之前，在先前的信道上执行的信道能量扫描的能量测量（符合[B1]）	空集
<i>apsLastChannelFailureRate</i>	xcb	Integer	0-100 (十进制)	信道改变之前，之前信道传输网络传输失败的最新百分比（传输失败对总传输尝试次数的百分比）	空集
<i>apsChannelTimer</i>	0xc c	Integer	1-24(十进制)	一个倒计时计时器（以小时记）指示下一个允许的频率捷变信道改变的时间。NULL值指示频道以前未被改变。	空集

表 2.25 组表条目格式

组 ID	端点列表
16 位组地址	属于该组成员的这个设备的端点列表

2.2.8 功能描述

2.2.8.1 永久数据

要求 APS 在永久存储器中维护尽量少的数据。这组数据应该能经受住断电、设备复位或其它处理事件。下面的数据应该保存在 APS 永久存储器中：

- *apsBindingTable* （如果设备支持）
- *apsDesignatedCoordinator* （如果设备支持）
- *ApsChannelMask*

- *apsUseExtendedPANID*
- *apsUseInsecureJoin*
- *apsGroupTable* （如果设备支持）
- 绑定表缓存（如果设备被指定为一个主要或备份绑定表缓存，见 2.4.2.4 小节）
- 发现缓存（如果设备被指定为一个主要发现缓存，见 2.4.2.1 小节）
- 设备上每个活动端点的节点描述符，电源描述符以及简单描述符
- 网络管理地址⁴

这些数据成为永久数据的方法不在本规范之内。

2.2.8.2 绑定

APS 可以维护一个绑定表，它允许 ZigBee 设备为来自给定源端点并带有一个给定 cluster ID 的帧，建立一个指定的目的地。每个指定的目标代表具体设备上的一个具体端点，或一个组地址。

2.2.8.2.1 绑定表执行

指定为包含绑定表的一个设备必须能够支持一个执行相关的具体长度的绑定表。绑定表应该执行以下映射：

$$(a_s, e_s, c_s) = \{(a_{d1}, e_{d1}), (a_{d2}, e_{d2}) \dots (a_{dn}, e_{dn})\}$$

其中：

a_s	= 作为绑定连接源的设备地址。
e_s	= 作为绑定连接源的设备的端点标识符。
c_s	= Cluster 标识符用于绑定连接。
a_{di}	= 与绑定连接有关的第 i 个目的地址或者目的组地址。
e_{di}	= 与绑定连接有关的第 i 个可选目的端点标识符。注意当 a_{di} 是一个设备地址时， e_{di} 才能存在。

2.2.8.2.2 绑定

APSME-BIND.request 或 APSME-UNBIND.request 原语启动创建或移除一个绑定连接的程序。只有一个支持绑定表缓存的设备，或一个希望存储源绑定的设备，可以启动该程序。如果这个程序由其它类型的设备发起，那么 APSME 发出状态参数设置为 ILLEGAL_REQUEST 的 APSME-BIND.confirm 或 APSME-UNBIND.confirm 原语。

当启动该程序，APSME 应该首先为源和目的地取出绑定连接的地址和端点。如果 DstAddrMode 参数值为 0x01，表示组寻址，那么只有源地址按照上面描述的处理。16 位组地址直接用作一个目标地址，且在这种情况下，不存在目标端点。有了这个信息，APSME 应该根据绑定或取消绑定程序是否分别启动，创建一个新的条目或从其绑定表中移除相应的条目。

如果请求一个绑定操作，APSME 应该在绑定表中创建一个新的条目。如果有空间的话，设备应该只在绑定表中创建一个新的条目。如果绑定表没有空间，APSME 就发出 Status 参数设置为 TABLE_FULL 的 APSME-BIND.confirm 原语。

如果请求一个取消绑定操作，APSME 应该查询其绑定表，寻找匹配发起请求所含信息的一个现有绑定。如果找不到一个条目，APSME 应该终止程序，并通知 NHLE 该无效绑定。这通过发出 Status 参数设置为 INVALID_BINDING 的 APSME-UNBIND.confirm 原语完成。如果找到了一个条目，APSME 应该在绑定表中移除该条目。

如果绑定连接成功创建或移除，APSME 应该通知 NHLE 直接绑定尝试的结果，以及程序的成功。这通过发出 APSME-BIND.confirm 或 APSME-UNBIND.confirm 原语完成，分别带有绑定结果，且状态参数设置为 SUCCESS。

一个成功的直接绑定程序在图 2.9 所示的 MSC 中说明。

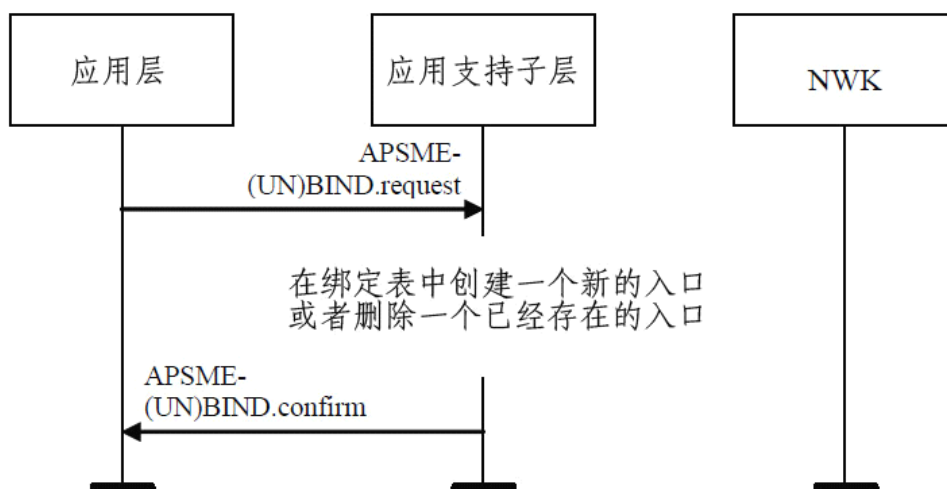


图 2.9 支持绑定表设备上的绑定

2.2.8.3 组寻址

APS 子层应该维护一个组表，它允许端点与组联系在一起，允许组寻址的帧有选择地传输到那些在表中与特定一个组相联系的端点。

APS 子层组表的组地址列表必须和 NWK 层组表的组 ID 列表(存储在 *nwkGroupIDTable* 属性中)一致。

2.2.8.3.1 组表

为了本次讨论，组表应该被看作组和端点之间的一组联系，如下：

$$\{(g_1 - ep_{11}, ep_{12} \dots ep_{1n}), (g_2 - ep_{21}, ep_{22} \dots ep_{2m}) \dots (g_i - ep_{i1}, ep_{i2} \dots ep_{ik})\}$$

其中：

g_i = 表中列出的第 i 组
 ep_{ij} = 与第 i 组相关的第 j 个端点

只要能够表示上面描述的联系，本规范的执行者可以自由地以任何方便高效的方式执行组表。

2.2.8.4 传输，接收和确认

本小节描述了传输、接收和确认的基本程序。

2.2.8.4.1 传输

只有是当前网络一部分的那些设备可以从 APS 子层发送帧。如果任何其它设备接收到一个传输帧的请求，它应该丢弃该帧，并通知发起层这个错误。状态为 CHANNEL_ACCESS_FAILURE 的 APSDE-DATA.confirm 原语表示传输帧的尝试因为信道忙失败。

APS 子层处理或生成的所有帧应该根据 2.2.5.1 小节规定的通用帧格式构造，使用 NWK 层数据服务传输。

使用传送模式 0b00（正常单播）和 0b10（广播）的传输必须包含源端点和目标端点域。传送模式子域值为 0b11 的组寻址传输，应该包含一个源端点域和组地址域，但是没有目标端点域。注意源设备的其它端点是合法的组成员，组地址帧可能的目的地。

对于绑定表存储在源设备的所有设备，源设备的 APSDE 确定绑定表条目是否包含一个单播目标设备地址或一个目标组地址。在绑定表条目包含一个单播目标设备地址，且这个目标设备地址是源设备本身的情况下，APSDE 给上层发出一个 APSDE-DATA.indication 原语，且不传输一个帧。否则，APSDE 将该帧传输给对应于绑定表条目指明的目标地址的 16 位 NWK 地址，且帧控制域的传送模式子域设置为 0b00。在绑定表条目包含一个目标组地址，且 *nwkUseMulticast* 是 FALSE 的情况下，帧控制域的传送模式子域值必须为 0b11，目标组地址必须放在 APS 头中，且目标端点被忽略。然后使用 NLDED-ATA.request 原语广播该帧，并使用广播地址 0xffffd。在绑定表条目包含一个目标组地址，且 *nwkUseMulticast* 是 TRUE 的情况下，帧控制域的传送模式子域值必须为 0b10，且目标端点值必须为 0xff。然后使用 NLDED-ATA.request 原语单播该帧，并使用绑定表条目提供的组地址。

如果要求安全，该帧应该按照 4.4 节描述的进行处理。

如果要求分段，且这个帧允许，那么该帧应该按照 2.2.8.4.5 节描述的进行处理。

当构造了帧并准备传输，它应该被传递给带有一个合适目标和源地址的 NWK 数据服务。另外，APS 层应该保证网络层的路由发现功能开启。通过给 NWK 层发出 NLDE-DATA.request 原语发起 APDU 传输，传输的结果通过 NLDE-DATA.confirm 原语返回。

2.2.8.4.2 接收和拒绝

APS 子层应该可以通过 NWK 层数据服务过滤到达的帧，且只列出 NHLE 感兴趣的那些帧。

如果 APSDE 接收到一个加密的帧，它应该按照 4.4 节的描述对帧进行处理，来移除安全。

如果 APSDE 收到一个包含目标端点域的帧，那么 APSDE 应该把它直接传递给目标端点提供的 NHLE，除非它是一个不完整的分段传输的一部分，或它被确定为先前传递帧的重复。经过了同一个不完整的分段传输和重复帧的检测，如果目标端点设置为广播端点 (0xff)，且收到的 NLDE-DATA.indication 原语的 DstAddrMode 参数不是 0x01，那么 APSDE 把帧送 NHLE 所支持的全无保留的端点。

如果设备的 APSDE 收到一个传输，其帧控制域的传送模式子域设置为 0b11，表示组寻址，它应该把该帧发给设备组表中与 APS 头的组地址域所含的 16 位组地址相联系的每个端点。同样，如果设备的 APSDE 收到一个 NLDE-DATA.request 原语，且 DstAddrMode 参数值为 0x01，也表示组寻址，它应该把该帧发给设备在组表中与 DstAddress 参数值给定的 16 位组地址相联系的每个端点。在这两种情况下，都为与给定组地址相联系的每个端点查询组表，并给上层发出 APSDE-DATA.indication 原语，且 DstEndpoint 参数值等于关联端点数。所有发出的原语中，APSDE-DATA.indication 原语的所有其它参数应该保持不变。

APSDE 应该保持一个重复拒绝表，至少包含源地址，APS 计数器和时间信息，这样根据本规范传输和多次接收的帧被确定为重复，只传输给 NHLE 一次。该表的大小应该至少是 apscMinDuplicateRejectionTableSize。

2.2.8.4.3 确认的使用

一个数据或 APS 命令帧发送时，应该为该帧设置合适的确认请求子域。一个确认帧发送时，应该总是把确认请求子域设置为 0。同样，任何广播或多播的帧发送时应该把确认请求子域设置为 0。

2.2.8.4.3.1 没有确认

一个帧被其预定接收者接收时，其确认请求（AR）子域设置为 0，就不应该有确认。发出设备应该假定帧的传输成功。图 2.10 展示了从发出者到接收者传输一个没有要求确认的数据帧的情景。在这种情况下，发出者传输 AR 子域等于 0 的数据帧。

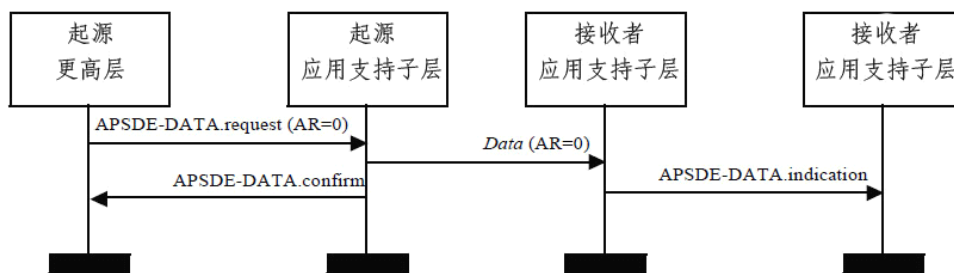


图 2.10 无确认的成功的数据传输

2.2.8.3.3.2 确认

一个帧被其预定接收者接收时，其确认请求（AR）子域设置为 1，应该返回确认。如果预定接收者正确接收该帧，它应该生成并发送一个确认帧给待确认帧的发出者。

当 APS 子层确定帧是有效的，确认帧的传输就应该开始了。

图 2.11 展示了从发出者到接收者传输一个带有确认的数据帧的情景。在这种情况下，发出者通过传输 AR 子域设置为 1 的数据帧，指示接收者它需要一个确认。

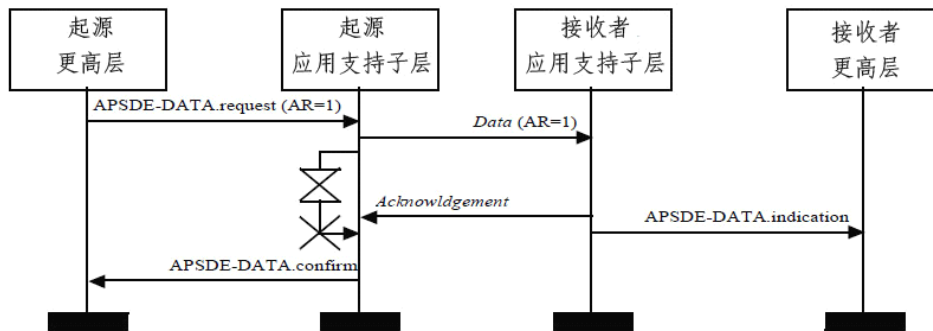


图 2.11 带确认的成功的数据传输

2.2.8.4.4 重传

一个设备发送确认请求子域设置为 0 的帧，应该假定传输成功接收，并应该因此不再执行重传程序。

一个设备发送确认请求子域设置为 1 的帧，为了接收相应的确认帧，应该等待最多 *apscAckWaitDuration* 秒。

如果在 *apscAckWaitDuration* 秒内收到一个确认帧，它包含和原始帧相同的 cluster 标识符和 APS 计数器，且源端点等于原始帧要传输到的目标端点，传输应该视为成功，设备不再采取进一步行动。如果在 *apscAckWaitDuration* 秒内没有收到一个确认，或在 *apscAckWaitDuration* 秒内收到了确认，但是包含一个意外的 cluster 标识符或 APS 计数器，或源端点不等于原始帧要传输到的目标端点，设备应该断定一次传输尝试失败。

如果一次传输尝试失败，设备应该重复传输帧的程序，等待确认，直到 *apscMaxFrameRetries* 次数的最大值。如果在 *apscMaxFrameRetries* 次重传后仍没有收到一个确认，APS 子层应该假定传输失败并且通知上层失败。

加密帧的重传应该使用和原始帧相同的帧计数器。

2.2.8.4.5 分段的传输

当一个 ASDU 太大而不能用一个 MAC 数据帧传输，请求一个有确认的单播，且该帧允许分段，ASDU 将被分成许多小字节的字符串，这里将其称为“块”。每个块分别在一个帧里传输。

一个“传输窗口”用来安排一个有序的事务。窗口的大小由栈 Profile 设置，可以设置为高达 8 块。下面的协议安排，在窗口可以移动之前，必须接收和确认一个传输窗口中的所有块。当传输窗口中的所有块已经被成功接收之后，就发送一个确认，或根据下面的协议，请求传输一个或者多个未接收的块。

不使用 APS 确认的事务不能分段。单播和广播传输不允许使用分段。

2.2.8.4.5.1 传输

一个分段传输的所有块应该有同样的 APS 计数器值。扩展帧头子帧应该包括在帧中。扩展帧控制域的分段子域的第一个块必须设置为 0b01，一个分段传输的所有随后的块设置为 0b10。块数域应该指示第一块传输的总块数，第二个块取值 0x01，此后每个块都应该增加。

应该维护一个传输窗口，最初覆盖块 0 到(apscMaxWindowSize-1)，或者如果这个更少就是总块数。

如果需要安全，那么每个帧都应该独立地处理，见第 4 章所述。传输每个块后，APS 应该启动一个计时器。如果在当前传输窗口中有更多未确认的块要发送，那么在 apscInterframeDelay 毫秒的延误之后，下一个块将会被传送到 NWK 数据服务。否则计时器将设置为 apscAckWaitDuration 秒。

应该维护一个 retryCounter 参数，并在每个新事务中设置为零。如果 apscAckWaitDuration 计时器到期，那么具有最少未确认块数的块将被重新传送到 NWK 数据服务，且 retryCounter 参数将被增加。如果 retryCounter 参数达到值 apscMaxFrameRetries，事务视为失败，给 NHLE 返回一个 APSDE-DATA.confirm 原语，状态值为 NO_ACK。

在收到一个确认帧，且 APS 计数器、块数和寻址域具有匹配的值，输出块就按照下节所述被确认。如果至少有一个之前未确认的块被确认，那么计时器应该停止，retryCounter 参数将复位。如果当前传输窗口中所有的块都被确认，那么传输窗口应该增加 apscMaxWindowSize 个。如果所有的块都被传输和确认，那么事务完成，并给 NHLE 返回一个 APSDE-DATA.confirm 原语，状态值为 SUCCESS。否则，具有最少未确认块数的块将被传送到 NWK 数据服务。

2.2.8.4.5.2 接收，拒绝和确认

如果分段使能传输所需的域不存在，该帧将会被拒绝。而且，所有参数超出本协议范围的帧也将被拒绝。

如果一个输入分段事务已经在处理过程中，但是寻址和 APS 计数器域不匹配收到的帧，那么收到的帧可以选择被拒绝，或者作为一个进一步事务独立地处理。

如果没有事务正在处理，且接收到了一个分段的帧，那么就应该尝试重组。最初的接收窗口将从 0 到 ($\text{apscMaxWindowSize}-1$)。

如果一个事务发起时 APS 计数器和源地址域的值匹配之前收到的一个事务，那么新事务将作为重复被拒绝。

如果接收窗口在 ($\text{apscAckWaitDuration} + \text{apscAckWaitDuration} * \text{apscMaxFrameRetries}$) 的时间内没有向前移动，事务就被视为失败。接收者会发送一个确认给发送者，带有错过的块⁵。

如果当前接收窗口的所有块已经被接收，且收到的一个块的块数大于当前接收窗口，那么当前接收窗口将会增加 apsMaxWindowSize 块。

如果接收到最后一个块，或者如果收到的一个块的块数值等于或高于窗口中之前未确认块数的最大值，那么就产生一个确认。

一旦事务中所有块都已经被接收，APS 会发出一个 APSDE-DATA.indication 原语，包括重组的信息，此事务应该视为完成。为了方便最终确认的任一重传，建议持续 $\text{apscAckWaitDuration}$ 秒的时间。块数域应该包括当前接收窗口中最小块数的值，使用数值 0 作为第一个块的值。

当产生后，确认根据 2.2.5.2.3 小节指明的确认帧格式编排。APS 计数器域应该反映被确认帧的对应域的值。

5. CCB #817

如果当前接收窗口中的第一个分段已经被正确接收，ack bitfield 的第一位应该设置为 1，否则设置为 0。类似地，随后的位应该设置为对应当前接收窗口中随后的分段的值。如果 apsMaxWindowSize 小于 8，那么余下的位设置为 1。

这个过程在下图中说明。在图 2.12 中，传输是立即成功的（这些例子假定 apscMaxWindowSize 取值为 3）。

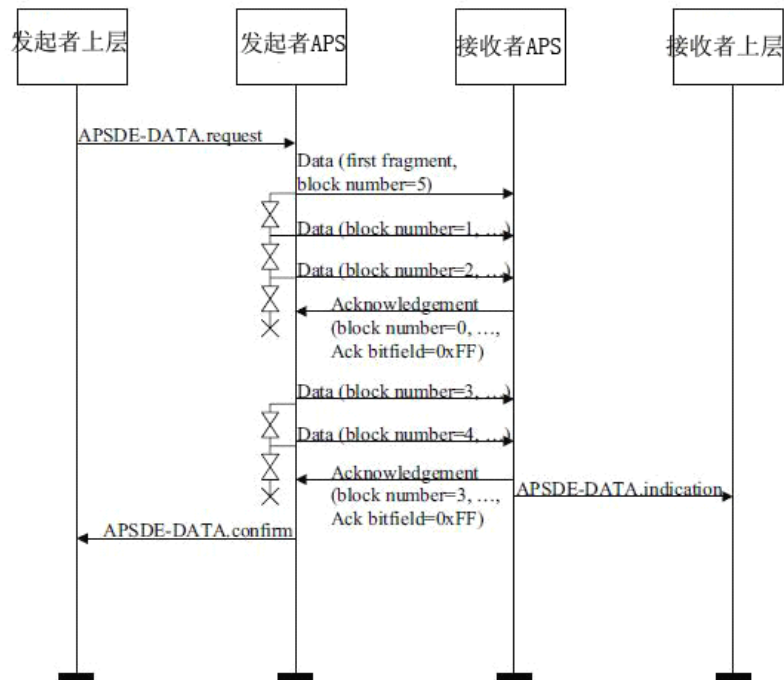


图 2.12 成功的分段数据传输

在图 2.13 中，通过网络传输时丢失了一个帧，它又被重新传输了。

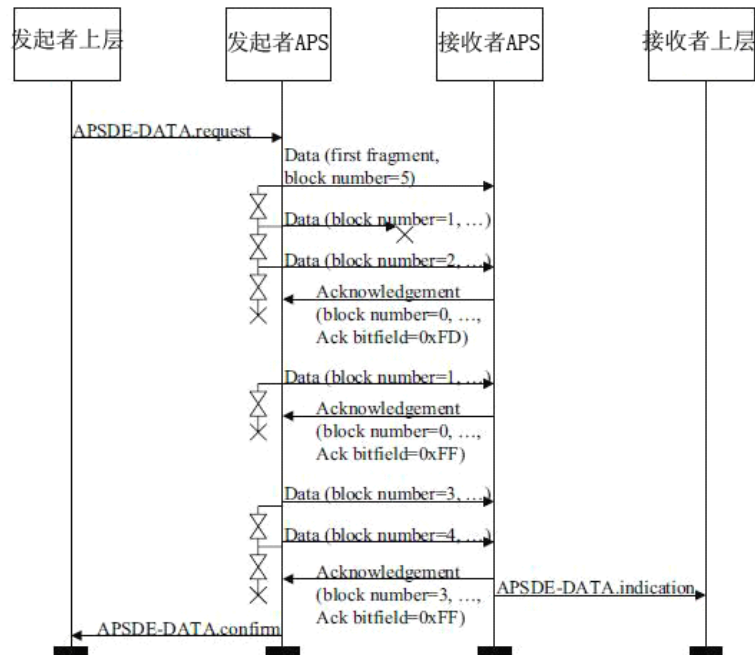


图 2.13 一次重传的分段数据传输

在图 2.14 中，网络中丢失了多个帧，包括窗口中含有最大块数的那个帧。在这种情况下需要稍多的传输量，但是源后退给了系统一个恢复的机会，ASDU 被成功地传输。

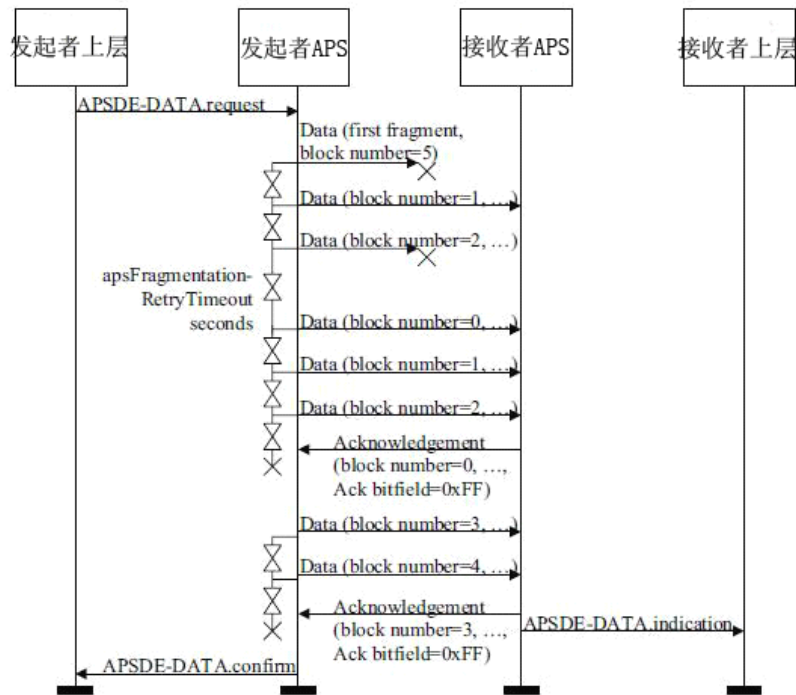


图 2.14 多次重传的分段数据传输

2.2.9 应用支持子层状态值

应用支持子层（APS）确认原语通常包括报告适用确认的请求状态的一个参数。表 2.26 给出了 APS 子层状态参数的值。

表 2.26 APS 子层状态值

名称	值	描述
SUCCESS	0x00	一个请求被成功执行
ASDU_TOO_LONG	0xa0	因为 ASDU 太大且不支持分段，一个传输请求失败
DEFRAG_DEFERRED	0xa1	一个收到的分段帧在当前不能重组
DEFRAG_UNSUPPORTED	0xa2	因为设备不支持重组，一个收到的分段帧不能重组
ILLEGAL_REQUEST	0xa3	一个参数值超出范围

表 2.26 APS 子层状态值（续表）

名称	值	描述
INVALID_BINDING	0xa4	因为请求的绑定连接不存在于绑定表中，一个 APSME-UNBIND.request 失败
INVALID_GROUP	0xa5	发出了组标识符不存在于组表中的一个 APSME-REMOVE-GROUP.request
INVALID_PARAMETER	0xa6	一个参数值无效或超出范围
NO_ACK	0xa7	因为没有收到确认，请求被确认的一个 APSDE-DATA.request 传输失败
NO_BOUND_DEVICE	0xa8	因为没有设备绑定到这个设备，目标寻址模式设置为 0x00 的一个 APSDE-DATA.request 失败
NO_SHORT_ADDRESS	0xa9	因为地址映射表中没有找到相应的短地址，目标寻址模式设置为 0x03 的一个 APSDE-DATA.request 失败
NOT_SUPPORTED	0xaa	因为设备不支持绑定表，目标寻址模式设置为 0x00 的一个 APSDE-DATA.request 失败
SECURED_LINK_KEY	0xab	收到一个使用连接密钥加密的 ASDU
SECURED_NWK_KEY	0xac	收到一个使用网络密钥加密的 ASDU
SECURITY_FAIL	0xad	相应的安全处理期间，请求加密的一个 APSDE-DATA.request 导致一个错误
TABLE_FULL	0xae	当绑定表或组表分别为满时，发出一个 APSME-BIND.request 或 APSME.ADDGROUP.request
UNSECURED	0xaf	收到一个没有任何加密的 ASDU
UNSUPPORTED_ATTRIBUTE	0xb0	发出带有一个未知的属性标识符的一个 APSME-GET.request 或 APSME.SET.request

2.3 ZigBee 应用构架

2.3.1 创建一个 ZigBee Profile

一个 ZigBee 网络设备之间通信的关键是一个关于 profile 的协议。

一种 profile 的例子是家庭自动化。这个 ZigBee profile 允许一系列设备类型交换控制信息，以形成一个无线家庭自动化应用。这些设备的目的是交换已知信息来实现控制，比如开关灯，给照明控制器发送一个照明传感测量，或如果一个位置传感器探测到移动时发送一个警报信息。

另一种类型 profile 的例子是定义了 ZigBee 设备之间普通行为的设备 profile。为了说明这一点，无线网络依靠自主设备加入网络的能力，在网络内发现其它设备和设备上的服务。设备发现和服务发现是设备 profile 支持的功能。

2.3.1.1 从 ZigBee 联盟获得一个 Profile 标识符

ZigBee 定义了两类不同的 profile：制造商定义的和公共的。这些类型的精确定义和标准是 ZigBee 联盟内部和本文件范围以外的一个行政问题。为了实施本技术规范，只有 profile 标识符的标准是唯一的。为此，每个 profile 的努力必须开始于请求 ZigBee 联盟分配一个 profile 标识符。一旦获得 profile 标识符，该 profile 标识符允许 profile 设计者做以下定义：

- 设备描述
- Cluster 标识符

profile 标识符的应用的市场空间是 ZigBee 联盟发放一个 profile 标识符的一个关键标准。profile 需要覆盖一个足够广泛的设备范围，以允许设备之间兼容，但不过于广泛，导致缺乏 Cluster 标识符来描述他们的接口。相反的，profile 不能定义地过窄而导致不同 profile 标识符的来描述许多设备，导致 profile 标识符地址空间的浪费，以及在描述设备如何配置接口时的兼容性问题。ZigBee 联盟内的政策团体将建立 profile 如何定义的标准，以帮助请求者调整其 profile 标识符请求。

2.3.1.2 定义设备描述和 Cluster

profile 标识符是 ZigBee 协议内部主要的枚举功能。每个唯一的 profile 标识符定义一个设备描述和 cluster 标识符的相关枚举。例如，对于 profile 标识符“1”，存在一批 16 位值描述的设备描述（这意味着每个 profile 内有 65,536 个可能的设备描述），以及一批 16 位值描述的 cluster 标识符（这意味着每个 profile 内有 65,536 个可能的 cluster 标识符）。每个 cluster 标识符还支持一批 16 位值描述的属性。这样，每个 profile 标识符有多达 65,536 个 cluster 标识符，每个 cluster 标识符包含多达 65,536 个属性。profile 开发者有责任在其分配的 profile 标识符内定义和分配设备描述，cluster 标识符和属性。注意设备描述，cluster 标识符和属性标识符的定义必须小心，以保证高效创建简单描述符和交换信息时的简化处理。

对于 ZigBee 联盟定义的公共的 profile 标识符，已经创建了一个 cluster 图书馆，提供了一个公共的定义和 cluster 及其属性的枚举。cluster 图书馆的目的是在应用 profile 范围内支持重复使用 cluster 和属性定义。依照惯例，当公共的 profile 使用 cluster 图书馆，它们将共享一个公共的 cluster 枚举和定义，以及属性标识符。

设备描述和 cluster 标识符的处理必须知道 profile 标识符。在任何信息交给设备之前，ZigBee 协议假定已经使用服务发现来确定设备和端点支持的 profile。同样，绑定程序假定类似的服务发现和 profile 匹配已经发生，因为产生的结果匹配提取给源地址，源端点，cluster 标识符，目标地址和目标端点。

2.3.1.3 在端点上使用 Profile

单个的 ZigBee 设备可以包含支持许多 profile，提供定义在那些 profile 内的各种 cluster 标识符的子集，还可以支持多设备描述。这个功能使用设备内部的一个地址等级定义如下：

- 设备：单个无线电支持的整个设备，带有唯一的 IEEE 和 NWK 地址。
- 端点：这是一个 8 位域，描述一个无线电支持的不同应用。端点 0x00 用于寻找设备 profile 地址，每个 ZigBee 设备必须使用。端点 0xff 用于寻找所有活动端点的地址（广播端点），端点 0xf1-0xfe 保留。因此，一个物理的 ZigBee 无线电可以支持多达 240 个在端点 0x01-0xf0 上的应用。

它是一个应用程序决定在一个设备端点上如何部署应用，以及公布哪个端点。唯一的要求是为每个端点创建的简单描述符，且那些描述符可用于服务发现。

2.3.1.4 建立服务发现

一旦设备被创建以支持具体的 profile，并符合在那些 profile 内部设备描述使用的 cluster 标识符，就可以部署应用了。为此，每个应用被分配一个不同的端点，且每个使用描述符进行描述。它是通过服务发现开启的 ZigBee 设备 profile 描述的简单描述符和其它服务发现机制，支持设备绑定和方便互补设备之间应用通信完成的。

其中的一个要点是，服务发现是基于 profile 标识符，输入 cluster 标识符列表和输出 cluster 标识符列表（明显没有设备描述）进行的。设备描述只简单地是一个指明强制和可选的 cluster 标识符的转换，它在指明 profile 类型的设备内支持。另外，期望设备描述枚举可以在 PDA 内使用或其它辅助绑定设备以提供设备能力的外部描述。

2.3.1.5 混合标准和专有 Profile

例如，一个 ZigBee 设备可以被开发为 ZigBee 公共 profile 标识符“XX”。如果一个制造商想要部署一个支持公共 profile “XX”的 ZigBee 设备，并提供厂商具体的扩展，如果 profile “XX”的定义范围内制造商扩展，这些扩展将被增加在制造商执行的公共 profile 上。或者，如果不支持制造商扩展，或 profile “XX”不支持期望的制造商扩展的类型，制造商可以在一个单独的制造商定义的 profile 标识符（在同一物理设备内一个单独的端点上宣布）上部署扩展。在这两种情况下，支持 profile 标识符“XX”但是不包含制造商扩展的设备，将只能宣布支持公共标识符 profile “XX”的基本功能，且不能响应或创建使用制造商扩展的信息。

2.3.1.6 启用向后兼容

在之前的例子中，使用 ZigBee 公共的 profile 标识符“XX”创建一个设备。如果 ZigBee 联盟要在后来更新这个公共 profile，以增加新的功能，如果通过定义 profile 支持这类扩展，修改将被直接纳入新的公共 profile 标识符 “XX”，或被引进到一个新的公共 profile 中，带有一个新的 profile 标识符（叫做“XY”）。假定公共 profile “XX”不支持扩展，通过使新设备宣布同时支持 profile 标识符“XX”和 profile 标识符“XY”，只用 profile 标识符 “XX”制造的设备仍然能和后来新制造的设备兼容。以这样的方式，新设备可以使用 profile 标识符“XX”与老设备通信，但是，也能使用 profile 标识符“XY”与同一个应用的新设备通信。ZigBee 内的服务发现功能使得网络中的设备可以决定支持的层次。

目标是 ZigBee 联盟既为制造商提供公共 profile 的扩展，也为公共 profile 提供未来的增强功能。只要有可能，这个目标包括维护同一个 profile 标识符内的那些扩展和增强功能。无论 profile 可扩展性的目标是否是可实现的，本节说明了 ZigBee 允许部署的制造商扩展范围内 profile 定义的功能，以及增强功能。对于制造商扩展和功能增强，Profile 可扩展性这一主题超出了本文件范围，由联盟的其它文件解决。

2.3.2 ZigBee 描述符

ZigBee 设备使用描述符数据结构描述自身。这些描述符包含的实际数据定义在不同的设备描述中。有五种描述符：节点，节点电源，简单，复杂和用户，显示在表 2.27 中。

表 2.27 ZigBee 描述符

描述符名称	状态	描述
节点	M	节点的类型和能力
节点电源	M	节点电源特性
简单	M	包含在节点内的设备描述
复杂	O	进一步了解设备描述
用户	O	用户自定义描述符

2.3.2.1 描述符的传输

节点，节点电源，简单和用户描述符应该按照它们在表中各自出现的顺序传输，即表顶部的域首先传输，表底部的域最后传输。每个不同的域应该遵循 1.2.1.4 小节指定的传输顺序。

每个描述符应该小于或者等于 `apscMaxDescriptorSize`，除非有条款规定能够传输不强制使用分段的发现信息。

对于简单描述符（见 2.3.2.5），存在允许描述符扩展超出 `apscMaxDescriptorSize`（见 2.4.3.1.21 和 2.4.4.1.20）的传输原语。当使用扩展传输原语，标准传输原语（见 2.4.3.1.5 和 2.4.4.1.5）要求传输一个省略的简单描述符，且设备的节点描述符指示可以使用扩展传输原语（见 2.3.2.3.12）。

复杂描述符应该按照图 2.15 说明的格式进行编排和传输。

字节: 1	可变的	...	可变的
域计数	域 1	...	域 n

图 2.15 复杂描述符的格式

复杂描述符所含的每个域应该按照图 2.16 所示进行编排。

字节: 1	可变的
压缩 XML 标记	域数据

图 2.16 一个单独的复杂描述符域格式

2.3.2.1.1 域计数字段

域计数字段长度为一个八位字节，指明了描述符所含域的个数。每个域的格式按照图 2.16 进行编排。

2.3.2.1.1.1 压缩 XML 标记域

压缩 XML 标记域长度为一个字节，指明了当前域的 XML 标记。复杂描述符的压缩 XML 标记域列在表 2.40 中。

2.3.2.1.1.2 域数据域

域数据域长度可变，包含当前域的具体信息，如压缩 XML 标记域所指。

2.3.2.2 通过描述符发现

描述符信息被 ZDO 管理实体设备和服务发现查询，使用送到端点 0 的 ZigBee 设备 profile 请求原语。关于发现操作的详细信息，参见 2.4.2.1 小节。信息通过 ZigBee 设备 profile 指示原语返回。

节点、节点电源、复杂和用户描述符适用于完整的节点。简单描述符必须指定用于定义在节点中的每个端点。如果一个节点描述符包含多个分单元，这些将在不同的端点上，这些端点的具体描述符通过在 ZigBee 设备 profile 原语中包含相关端点号读取。

2.3.2.3 节点描述符

节点描述符包含关于 ZigBee 节点功能的信息，对每个节点都是强制的。在一个节点中应该只有一个节点描述符。

节点描述符的域以它们传输的顺序展示在表 2.28 中。

表 2.28 节点描述符的域

域名称	长度(位)
逻辑类型	3
可使用复杂描述符	1
可使用的用户描述符	1
保留	3
APS 标志	3
频段	5
MAC 功能标志	8
制造商代码	16
最大缓冲区大小	8
最大输入传输大小	16
服务掩码	16
最大输出传输大小	16
描述符功能域	8

2.3.2.3.1 逻辑类型域

节点描述符的逻辑类型域长度为 3 位，指明了 ZigBee 节点的设备类型。逻辑类型域应该设置为表 2.29 所列的非保留值之一。

表 2.29 逻辑类型域的值

逻辑类型值 $b_2 b_1 b_0$	描述
000	ZigBee 协调器
001	ZigBee 路由器
010	ZigBee 终端设备
011-111	保留

2.3.2.3.2 可使用复杂描述符域

节点描述符的可使用复杂描述符域长度为 1 位，指明了该设备是否可以使用一个复杂描述符。如果该域设置为 1，可以使用复杂描述符。如果该域设置为 0，不可以使用复杂描述符。

2.3.2.3.3 可使用用户描述符域

节点描述符的可使用用户描述符域长度为 1 位，指明了该设备是否可以使用一个用户复杂描述符。如果该域设置为 1，可以使用用户描述符。如果该域设置为 0，不可以使用用户描述符。

2.3.2.3.4 APS 标志域

节点描述符的 APS 标志域长度为 3 位，指明了节点应用支持子层的功能。

2.3.2.3.5 频段域

频段域长度为 5 位，指明了节点使用的基本 IEEE802.15.4 无线电支持的频段。对每个基本 IEEE802.15.4 无线电支持的频段，频段域的相应位应该被设置为 1，如表 2.30 所示。所有其它位应该设置为 0。

表 2.30 频段域的值

频段域的位编号	支持频段
0	868–868.6MHz
1	保留
2	902–928MHz
3	2400–2483.5 MHz
4	保留

2.3.2.3.6 MAC 功能标志域

MAC 功能标志域长度为 8 位，指明了节点功能，如 IEEE802.15.4 MAC 子层【B1】所要求。MAC 功能标志域应该按照图 2.17 所示的格式进行编排。

位：0	1	2	3	4-5	6	7
备用 PAN 协调器	设备类型	电源来源	空闲时接收器开启	保留	安全能力	分配地址

图 2.17 MAC 功能标志域的格式

备用 PAN 协调器子域长度为 1 位，如果该节点可以成为一个 PAN 协调器的话，应该设置为 1。否则备用 PAN 协调器子域应该设置为 0。

设备类型子域长度为 1 位，如果该节点是一个全功能设备（FFD），应该设置为 1。否则设备类型子域应该设置为 0，表示一个简化功能设备（RFD）。

电源来源子域长度为 1 位，如果当前电源来源是主电源，应该设置为 1。否则电源来源子域应该设置为 0。该信息来自节点电源描述符的节点当前电源来源域。

空闲时接收器开启子域长度为 1 位，如果设备在空闲期间没有关闭其接收器以节省电源，应该设置为 1。否则空闲时接收器开启子域应该设置为 0（也可参见 2.3.2.4 小节）。

安全功能子域长度为 1 位，如果设备可以使用【B1】规定的安全组合发送和接收加密帧，应该设置为 1。否则安全功能子域应该设置为 0。

分配地址子域长度为 1 位，应该设置为 0 或 1。⁶

2.3.2.3.7 制造商代码域

节点描述符的制造商代码域长度为 16 位，指明了一个制造商代码，由 ZigBee 联盟分配，把制造商和设备联系起来。

2.3.2.3.8 最大缓冲区大小域

节点描述符的最大缓冲区域长度为 8 位，有效值范围是 0x00-0x7f。这个域指明了该节点的网络子层数据单元（NSDU）的最大大小，以字节为单位。这是由应用支持子层传递给应用或来自于应用的数据或命令，在进行任何分段或重组之前的最大大小。

该域可以用于一个高级别的网络管理指示。

2.3.2.3.9 最大输入传输大小域

最大输入传输大小域长度为 16 位，有效值范围是 0x0000-0x7fff。这个域指明了在一个信息传输内可以传输给该节点的应用子层数据单元（ASDU）的最大大小，以字节为单位。通过使用分段，该值可以超过节点最大缓冲区域的值（参见 2.3.2.3.8 小节）。

2.3.2.3.10 服务掩码域

节点描述符的服务掩码域长度为 16 位，位的设置表示该节点的系统服务能力。它能使系统其它节点发现特定系统的服务。位的设置定义在表 2.31 中。

表 2.31 服务掩码位分配

位号码	分配
0	主要信托中心
1	备份信托中心
2	主要绑定表缓存
3	备份绑定表缓存
4	主要发现缓存
5	备份发现缓存
6	网络管理器
7-15	保留

2.3.2.3.11 最大输出传输大小域

节点描述符的最大输出传输大小域有 16 位长，有效范围是 0x0000-0x7fff。本域说明了可以从本节点以单个信息传输的应用子层数据单元（ASDU）的最大大小，以字节为单位。通过使用分段，这个值可以超过节点最大缓冲域的值（见 2.3.2.3.8 小节）。

2.3.2.3.12 描述符能力域

节点描述符的描述符能力域有 8 字节长，位的设置表明了该节点的描述符能力。它能使系统上其它节点发现描述符的特定功能。位的设置在表 2.32 中定义。

表 2.32 描述符能力位分配

位号码	分配
0	扩展活动端点表可用
1	扩展简单描述符表可用
2-7	保留

2.3.2.4 节点电源描述符

节点电源描述符动态地指示节点的电源状态，对每个节点都是强制的。一个节点应该只有一个节点电源描述符。

节点电源描述符的域按其传输的顺序展示在表 2.33 中。

表 2.33 节点电源描述符的域

域名称	长度(位)
当前电源模式	4
可用电源来源	4
当前电源来源	4
当前电源来源的电量	4



2.3.2.4.1 当前电源模式域

节点电源描述符的当前电源模式域长度为 4 位，指明了节点当前的睡眠/省电模式。当前电源模式域应该设置为表 2.34 所列的非保留值之一。

表 2.34 当前电源模式域的值

当前电源模式值 $b_3b_2b_1b_0$	描述
0000	接收器与节点描述符的空闲时接收器开启子域同步
0001	接收机按照节点电源描述符的定义，周期性的打开
0010	当被激发时接收器打开，例如用户按下一个按钮
0011-1111	保留

2.3.2.4.2 可用电源来源域

节点电源描述符的可用电源来源域长度为 4 位，指明了该节点的可用电源来源。对该节点支持的每个电源，可用电源来源域相应的位应该设置为 1，如表 2.35 所列。所有其它位应该设置为 0。

表 2.35 可用电源来源域的值

可用的电源来源域的位编号	支持电源来源
0	常用(主要)电源
1	充电电池
2	一次性电池
3	保留

2.3.2.4.3 当前电源来源域

节点电源描述符的当前电源来源域长度为 4 位，指明了该节点当前使用的电源来源。对于选择的当前电源，当前电源来源域的相应位应该设置为 1，如表 2.36 所示。所有其它位应该设置为 0。

表 2.36 当前电源来源域的值

当前电源来源位号 码	当前电源来源
0	常用（主）电源
1	充电电池
2	一次性电池
3	保留

2.3.2.4.4 当前电源来源电量域

节点电源描述符的当前电源来源电量域长度为 4 位，指明了负荷电源来源的电量。当前电源来源电量域应该设置为表 2.37 所列的非保留值之一。

表 2.37 当前电源来源电量域的值

当前电源来源电量域 $b_3b_2b_1b_0$	负荷电量
0000	临界的
0100	33%
1000	66%
1100	100%
所有其他值	保留

2.3.2.5 简单描述符

简单描述符包含该节点所含每个端点的具体信息。简单描述符对节点所列的每个端点都是强制的。

简单描述符的域按其传输的顺序展示在表 2.38 中。因为该描述符需要无线传输，简单描述符的总长度应该小于或等于 *apsMaxDescriptorSize*。

表 2.38 简单描述符的域

域名称	长度 (位)
端点	8
应用 profile 标识符	16
应用设备标识符	16
应用设备版本	4
保留	4
应用输入 cluster 计数器	8
应用输入 cluster 列表	16*i(i 是应用输入 cluster 计数的值)
应用输出 cluster 计数器	8
应用输出 cluster 列表	16*o (o 是应用输出 cluster 计数的值)

2.3.2.5.1 端点域

简单描述符的端点域长度为 8 位，指明了该描述符指向的节点内的端点，应用程序只能使用 1-240 的端点。

2.3.2.5.2 应用 profile 标识符域

简单描述符的应用 profile 标识符域长度为 16 位，指明了该端点支持的 profile。Profile 标识符应该从 ZigBee 联盟获取。

2.3.2.5.3 应用设备标识符域

简单描述符的应用设备标识符域长度为 16 位，指明了该端点支持的设备描述。设备描述标识符应该从 ZigBee 联盟获取。

2.3.2.5.4 应用设备版本域

简单描述符的应用设备版本域长度为 4 位，指明了该端点支持的设备描述版本。应用设备版本域应该设置为表 2.39 所列的非保留值之一。

表 2.39 应用设备版本域的值

应用设备版本值 $b_3b_2b_1b_0$	描述
0001–1111	具体值由本描述符的应用 profile 标识符指明的应用 profile 设置。默认是 0000，除非由应用 profile 定义。

2.3.2.5.5 应用输入 Cluster 计数域

简单描述符的应用输入 cluster 计数域长度为 8 位，指明了该端点支持的输入 Cluster 的个数，它将出现在应用输入 Cluster 列表域中。如果该域的值是零，应该不包括应用输入 Cluster 列表域。

2.3.2.5.6 应用输入 Cluster 列表

简单描述符的应用输入 cluster 列表长度为 $16*i$ ， i 是应用输入 Cluster 计数域的值。这个域指明了该端点支持的输入 Cluster 的列表，在服务发现和绑定程序期间使用。

应用输入 Cluster 列表域应该仅当应用输入 Cluster 计数域的值大于零时包含。

2.3.2.5.7 应用输出 Cluster 计数域

简单描述符的应用输出 cluster 计数域长度为 8 位，指明了该端点支持的输出 Cluster 的个数，它将出现在应用输出 Cluster 列表域中。如果该域的值是零，应该不包括应用输出 Cluster 列表域。

2.3.2.5.8 应用输出 Cluster 列表

简单描述符的应用输出 cluster 列表长度为 $16*o$ ， o 是应用输出 Cluster 计数域的值。指明了该端点支持的输出 Cluster 的列表，在服务发现和绑定程序期间使用。

应用输出 cluster 列表域应该仅当应用输出 Cluster 计数域的值大于零时包含。

2.3.2.6 复杂描述符

复杂描述符包含该节点所含的每个设备描述的扩展信息。复杂描述符的使用是可选的。

因为该描述符内数据的扩展性和复杂性，它使用压缩 XML 标记，以 XML 的形式表示。描述符的每个域展示在表 2.40 中，因此可以按照任何顺序传输。因为该描述符需要无线传输，复杂描述符的总长度应该小于或等于 *apscMaxDescriptorSize*。

表 2.40 复杂描述符的域

域名	XML 标记	压缩 XML 标记值 X_1X_0	数据类型
保留	-	00	-
语言和字符集	<languageChar>	01	见 2.3.2.6.1 小节
制造商名称	<manufacturerName>	02	字符串
模板名称	<modelName>	03	字符串
序列号	<serialNumber>	04	字符串
设备 URL	<deviceURL>	05	字符串
图标	<icon>	06	字节字符串
图标 URL	<outliner>	07	字符串
保留	-	08 – ff	-

2.3.2.6.1 语言和字符集域

语言和字符集域长度为 3 个字节，指明了复杂描述符中字符串使用的语言和字符集。语言和字符集域的格式在图 2.18 中说明。

字节：2	1
ISO 639-1 语言代码	字符集标识符

图 2.18 语言和字符集域的格式

ISO 639-1 语言代码子域长度为 2 个字节，指明了字符串使用的语言，如【B5】所定义。

字符集标识符子域长度为 1 个字节，指明了字符集中字符使用的编码。该子域应该设置为表 2.41 所列的非保留值之一。

表 2.41 字符集标识符子域的值

字符集标识符值	每个字符的位数	描述
0x00	8	ISO 646, ASCII 字符集。每个字符装入到一个字节的低 7 位，其中字节的最高位设置为 0（见[B6]）。
0x01 – 0xff	—	保留

如果语言和字符集没有指定，语言应该默认为英语（语言代码=“EN”），且字符集为 ISO 646。

2.3.2.6.2 制造商名称域

制造商名称域长度是可变的，包含一个字符串，表示设备制造商的名称。

2.3.2.6.3 模板名称域

模板名称域长度是可变的，包含一个字符串，表示设备的模板制造商的名称。

2.3.2.6.4 序列号域

序列号域长度是可变的，包含一个字符串，表示设备的制造商序列号。

2.3.2.6.5 设备 URL 域

设备 URL 域长度是可变的，包含一个字符串，表示通过该 URL 可以获得设备更多相关信息。

2.3.2.6.6 图标域

图标域长度是可变的，包含一个字节字符串，携带电脑、网关或 PDA 上表示设备的图标数据。图标的格式应该是一个 32 乘以 32 像素的 PNG 图像。

2.3.2.6.7 图标 URL 域

图标 URL 域长度是可变的，包含一个字符串，表示通过该 URL 可以获得该设备的图标。

2.3.2.7 用户描述符

用户描述符包含允许用户使用一个用户友好的字符串，比如“卧室 TV”或“楼梯灯”来识别设备的信息。用户描述符的使用是可选的。该描述符只包含一个域，它使用 ASCII 字符集，最多应该包含 16 个字符。

用户描述符的域按照其传输的顺序展示在表 2.42 中。

表 2.42 用户描述符的域

域名	长度(字节)
用户描述	16

2.3.3 功能描述

2.3.3.1 接收和拒绝

应用构架层应该可以通过 APS 子层数据服务过滤到达的帧，只列出每个活动端点上的应用程序感兴趣的那些帧。

应用构架层通过 APSDE-DATA.indication 原语从 APS 子层接收数据，是针对一个特定的端点（DstEndpoint 参数）和一个特定的 profile（ProfileId 参数）。

如果应用构架层收到一个针对一个不活动端点的帧，该帧应该被丢弃。否则，应用构架层应该确定指定的 profile 标识符是否和指定端点上的 profile 标识符相匹配。如果 profile 标识符不匹配，应用构架层应该拒绝该帧。否则，应用构架层应该把收到帧的负载传递给指定端点上的应用程序。

2.4 ZigBee 设备 Profile

2.4.1 范围

ZigBee 应用层规范描述了常用 ZigBee 设备功能，比如绑定、设备发现和服务发现是如何在 ZigBee 设备对象内部实现的。ZigBee 设备 Profile 通过定义 cluster 能够像任何的 ZigBee profile 一样运行。不像应用程序特定的 profile，ZigBee 设备 Profile 内的 cluster 定义了所有 ZigBee 设备支持的功能。和任何 profile 文件一样，本文件详细叙述了强制和/或可选的 cluster。

2.4.2 设备 Profile 概述

设备 Profile 支持 ZigBee 协议内部四种关键的中间设备通信功能。这些功能在以下的章节介绍：

- 设备和设备发现概述
- 终端设备绑定概述
- 绑定和取消绑定概述
- 绑定表管理概述
- 网络管理概述

2.4.2.1 设备和服务发现概述

设备和服务发现是一个分布式的操作，即个别设备或指定的发现缓存设备响应发现请求。“重要设备地址”域使得响应可以来自于设备本身或一个发现缓存设备。在某些发现缓存设备和“重要设备地址”设备都响应的情况下，应该使用来自于“重要设备地址”的响应。

设备和服务发现存在以下功能：

- **设备发现：**为一个设备提供确定 PAN 上其它设备身份的功能。64 位 IEEE 地址和 16 位网络地址都支持设备发现。

- 设备发现信息可以用于以下两种方式之一：

——**广播寻址：**网络中的所有设备应该根据逻辑设备类型和匹配标准做出响应。ZigBee 终端设备将刚好响应他们自身的地址，ZigBee 协调器和 ZigBee 路由器及其相关设备应该根据请求类型响应，以其地址作为第一个条目，然后是其相关设备的地址。响应设备应该使用单播响应的 APS 确认服务。

——**单播寻址：**只有指定的设备响应。一个 ZigBee 终端设备应该只以其地址响应。ZigBee 终端设备仅响应他们自身的地址，一个 ZigBee 协调器或路由器应该以其自己的地址和每个相关子设备的地址回复。所含的相关子设备允许请求者确定指定设备的网络基本拓扑结构。

- **服务发现：** 为一个设备提供了确定 PAN 上其它设备所提供的服务的功能。

- 服务发现信息可以用于以下两种方式之一：

——**广播寻址：** 由于可以返回的信息量，只有个别设备或主要发现缓存会以请求中建立的匹配标准响应。如果主要发现缓存为请求的 NWKAddrOfInterest 保持缓冲发现信息，只在这种情况下它会响应。响应设备也要使用单播响应的 APS 确认服务。

——**单播寻址：** 只有指定设备会响应。在一个 ZigBee 协调器或 ZigBee 路由器的情况下，这些设备会为睡眠的相关设备缓存服务发现信息，并代表它们响应。

- 以下查询类型支持服务发现：

——**活动端点：** 该命令允许一个查询设备确定活动的端点。一个活动端点是带有一个支持单个 profile 的应用程序的端点，由一个简单描述符描述。该命令必须是单播寻址。

——**匹配简单描述符：** 该命令允许查询设备提供一个 Profile ID（以及可选的输入和/或输出 Cluster ID），并请求返回匹配所提供标准的目标设备的一个端点的身份。该命令可以广播给所有 RxOnWhenIdle= TRUE 的设备，或单播寻址。对于广播寻址请求，响应设备应该使用单播响应的 APS 确认服务。

——**简单描述符：** 该命令允许一个查询设备为所提供的端点返回简单描述符。该命令必须是单播寻址。

——**节点描述符：** 该命令允许一个查询设备从指定设备返回节点描述符。该命令必须是单播寻址。

——**电源描述符：** 该命令允许一个查询设备从指定设备返回电源描述符。该命令必须是单播寻址。

——**复杂描述符**：该可选命令允许一个查询设备从指定设备返回复杂描述符。该命令必须是单播寻址。

——**用户描述符**：该可选命令允许一个查询设备从指定设备返回用户描述符。该命令必须是单播寻址。

2.4.2.2 终端设备绑定概述

终端设备绑定存在以下功能：

- **终端设备绑定**：

- 当使用了用户干预来确定命令/控制设备对，为一个应用程序提供了支持一个绑定的简化方法的功能。典型用法是为了安装的目的，要求一个用户在两个设备上按下按钮。第二次使用此机制允许用户移除绑定表条目。

2.4.2.3 绑定和取消绑定概述

直接设定绑定表条目存在以下功能：

- **绑定**：提供了创建一个绑定表条目的功能，其中绑定表可以把控制信息映射到其预定目的地。
- **取消绑定**：提供了移除绑定表条目的功能。

2.4.2.4 绑定表管理概述

绑定表管理存在以下功能：

- 登记执行源绑定的设备：
- 为一个源设备提供了指示其主要绑定表缓存来保持自己的绑定表的功能。
- 以另一个设备代替本设备，无论在绑定表中任何地方都可以：
- 提供了以另一个设备代替本设备的功能，通过在所有情况下替换其绑定表中的地址。
- 备份一个绑定表条目：
- 为一个主要绑定表缓存提供了给备份绑定表缓存发送一个新创建条目的详细信息的功能（在接收到一个绑定请求之后）。
- 移除一个备份绑定表条目：
- 为一个主要绑定表缓存提供了请求从备份绑定表缓存中移除某个条目的功能（在接收到一个取消绑定请求之后）。



- 备份整个绑定表：
 - 为一个主要绑定表缓存提供了请求备份其整个绑定表的功能，使用备份绑定表缓存。
- 恢复整个绑定表：
 - 为一个主要绑定表缓存提供了请求恢复其整个绑定表的功能，使用备份绑定表缓存。
- 备份主要绑定表缓存：
 - 为一个主要绑定表缓存提供了请求备份其整个源设备地址表的功能（包含任何含有其自身绑定表的源设备的地址）
- 恢复主要绑定表缓存：
 - 为一个主要绑定表缓存提供了请求恢复其整个源设备地址表的功能（包含任何含有其自身绑定表的源设备的地址）

2.4.2.5 网络管理概述

网络管理存在以下功能：

- 提供了从设备中检索管理信息的功能，包括：
 - 网络发现结果
 - 到相邻节点的链路质量
 - 路由表内容
 - 绑定表内容
 - 发现缓存内容
 - 能量探测扫描结果
- 提供了设置管理信息控制的功能，包括：
 - 网络离开
 - 网络直接加入
 - 允许加入
 - 网络更新和错误通知

2.4.2.6 设备 Profile 的设备描述

ZigBee 设备 Profile 使用了一个 ZigBee 设备描述。每个指定为强制的 cluster 会在所有 ZigBee 设备中存在。对于一些信息的响应行为根据逻辑设备类型而异。对于可选 cluster 的支持不依赖于逻辑设备类型。

2.4.2.7 配置和角色

设备 Profile 假定了一个客户端/服务器拓扑。一个发出设备发现、服务发现、绑定或网络管理请求的设备，通过一个客户端角色完成。一个服务于这些请求和响应的设备，通过一个服务器角色完成。在可以同时提供客户端和服务器角色的给定设备中，客户端和服务器角色是不排斥的。

因为许多客户端请求和服务器响应是公共的，且应用对象可以使用，而不是 ZigBee 设备对象，应用构架层的头的传输序列号应该在客户端请求及其相关服务器响应中是相同的。

设备 Profile 以两种配置之一描述了设备：

- **客户端：**一个客户端通过设备 Profile 信息给服务器发出请求。
- **服务器：**一个服务器发出响应给发起设备 Profile 信息的客户端。

2.4.2.8 ZDP 命令的传输

所有 ZDP 命令应该通过 APS 数据服务传输，并应该根据图 2.19 所示的 ZDP 帧结构来编排。

字节数：1	可变的
事务序列号	事务数据

图 2.19 ZDP 帧格式

2.4.2.8.1 事务序列号域

事务序列号域长度为 8 位，指明了 ZDP 事务的一个标识号码，这样一个响应命令帧可以和该请求帧联系起来。应用对象自身会维护一个 8 位的计数器，它被复制到该域中，每发送一个命令就加一。当值达到 0xff 时，下一个命令会重启值为 0x00 的计数器。

如果一个设备发送一个要求响应的 ZDP 请求命令，目标设备应该以相关的 ZDP 响应命令响应，并包括原始请求命令所含的事务序列号。

务序列号域可以被一个控制设备使用，它可以发出多个命令，以便它可以匹配到来的响应和相关命令。

2.4.2.8.2 事务数据域

事务数据域长度是可变的，包含各个 ZDP 事务的数据。该域的格式和长度取决于要传输的命令，如 2.4.3 和 2.4.4 小节所定义。

2.4.3 客户端服务

设备 Profile 客户端服务支持从客户端到服务器传输设备和服务发现请求，终端设备绑定请求，绑定请求，取消绑定请求，以及网络管理请求。另外，客户端服务支持从服务器接收这些请求的响应。

2.4.3.1 设备和服务发现客户端服务

表 2.43 列出了设备 Profile、设备和服务发现客户端服务支持的命令。这些命令将在以下分节一一讨论。

表 2.43 设备和服务发现的客户服务命令

设备和服务发现的客户端服务	客户端传输	服务器处理
NWK_addr_req	O	M
IEEE_addr_req	O	M
Node_Desc_req	O	M
Power_Desc_req	O	M
Simple_Desc_req	O	M
Active_EP_req	O	M
Match_Desc_req	O	M
Complex_Desc_req	O	O
User_Desc_req	O	O
Discovery_Cache_req	O	O
Device_annce	O	M
User_Desc_set	O	O
System_Server_Discover_req	O	O

表 2.43 设备和服务发现的客户服务命令

设备和服务发现的客户端服务	客户端传输	服务器处理
Discovery_store_req	O	O
Node_Desc_store_req	O	O
Power_Desc_store_req	O	O
Active_EP_store_req	O	O
Simple_Desc_store_req	O	O
Remove_node_cache_req	O	O
Find_node_cache_req	O	O
Extended_Simple_Desc_req	O	O
Extended_Active_EP_req	O	O

2.4.3.1.1 NWK_addr_req

NWK_addr_req 命令（ClusterID=0x0000）应该按图 2.20 所示的格式编排。

字节数：8	1	1
IEEEAddress	RequestType	StartIndex

图 2.20 NWK_addr_req 命令格式

表 2.44 指明了 NWK_addr_req 命令帧的域。

表 2.44 NWK_addr_req 命令的域

名称	类型	有效范围	描述
IEEEAddr	IEEE Address	有效 64 位 IEEE 地址	通过远程设备配置的 IEEE 地址
RequestType	Integer	0x00-0xff	这个命令的请求类型： 0x00 – 单个设备响应 0x01 – 扩展响应 0x02-0xFF – 保留
StartIndex	Integer	0x00-0xff	如果该命令的请求类型是扩展响应，StartIndex 为相关设备列表的被请求的元素提供起始索引。

2.4.3.1.1.1 原语生成

NWK_addr_req 从一个希望根据其已知 IEEE 地址查询远程设备 16 位地址的本地设备生成。该命令的目标地址应该单播或广播到所有的 macRxOnWhenIdle = TRUE 的设备。

2.4.3.1.1.2 接收结果

在接收时，一个远程设备应该比较 IEEEAddr 和其本地 IEEE 地址，或其本地发现缓存中保存的任何 IEEE 地址。如果没有匹配且请求是单播的，应该生成一个 NWK_addr_resp 命令，返回给本地设备，状态域设置为 DEVICE_NOT_FOUND，IEEEAddrRemoteDev 域设置为请求的 IEEE 地址；NWKAddrRemoteDev 域设置为这个设备的 NWK 地址；且 NumAssocDev、StartIndex 和 NWKAddrAssocDevList 域不能包含在帧中。如果没有匹配且命令作为一个广播被接收，该请求必须被丢弃且不产生响应。

如果在所含的 IEEEAddr 和远程设备的 IEEE 地址或发现缓冲中保存的一个 IEEE 地址之间，探测到一个匹配，RequestType 应该用于创建一个响应。如果 RequestType 是保留值之一，应该生成一个 NWK_addr_resp 命令，其中 Status 域设置为 INV_REQUESTTYPE；IEEEAddrRemoteDev 域设置为请求的 IEEE 地址，NWKAddrRemoteDev 域设置为与请求的 IEEE 地址相对应的网络地址；且 NumAssocDev、StartIndex 和 NWKAddrAssocDevList 域不能包含在帧中。

如果 RequestType 是单个设备响应，应生成一个 NWK_addr_resp 命令，返回给本地设备，状态域设置为 SUCCESS，IEEEAddrRemoteDev 域设置为请求的 IEEE 地址；NWKAddrRemoteDev 域设置为所发现的设备的 NWK 地址；且 NumAssocDev、StartIndex 和 NWKAddrAssocDevList 域不能包含在帧中。

如果 RequestType 是扩展响应，且远程设备是带有相关设备的 ZigBee 协调器或路由器，应生成一个 NWK_addr_resp 命令，返回给本地设备，状态域设置为 SUCCESS，IEEEAddrRemoteDev 域设置为设备本身的 IEEE 地址，且 NWKAddrRemoteDev 域设置为设备本身的 NWK 地址。远程设备还应该为其相关设备在 NWKAddrAssocDevList 域中提供一个所有 16 位 NWK 地址的列表，以 StartIndex 条目开头，并继续整个条目，直到达到 APS 数据包长度的最大值。然后它设置 NumAssocDev 域为 NWKAddrAssocDevList 域中条目的个数。

2.4.3.1.2 IEEE_addr_req

IEEE_addr_req 命令 (ClusterID=0x0001) 应该按图 2.21 所示的格式编排。

字节: 2	1	1
NWKAddrOfInterest	RequestType	StartIndex

图 2.21 IEEE_addr_req_Command 帧格式

表 2.45 指明了 IEEE_addr_req 命令帧的域。

表 2.45 IEEE_addr_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	设备类型	16 位网络地址	用于 IEEE 地址映射的网络地址
RequestType	Integer	0x00-0xff	这个命令的请求类型: 0x00 – 单个设备响应 0x01 – 扩展响应 0x02-0xff – 保留
StartIndex	Integer	0x00-0xff	如果该命令的请求类型是扩展响应, StartIndex 为相关设备列表的被请求的元素提供起始索引。

2.4.3.1.2.1 原语生成

IEEE_addr_req 从一个希望根据其已知 16 位地址查询远程设备 64 位 IEEE 地址的本地设备生成。该命令的目标地址是单播的。

2.4.3.1.2.2 接收结果

在接收该命令时, 一个远程设备应比较 NWKAddrOfInterest 和它的本地 NWK 地址, 或它的本地发现缓存所含的任何 NWK 地址。如果没有匹配, 生成一个 IEEE_addr_rsp 命令, 返回给本地设备, 状态域设置为 DEVICE_NOT_FOUND; IEEEAddrRemoteDev 域设置为这个设备的 IEEE 地址; NWKAddrRemoteDev 域设置为请求的 NWK 地址; 且 NumAssocDev、StartIndex 和 NWKAddrAssocDevList 域不能包含在帧中。

如果在 NWKAddrOfInterest 所含的地址和远程设备的 NWK 地址, 或发现缓存所含的一个地址之间检测到一个匹配, RequestType 应用于创建一个响应。如果 RequestType 是保留值之一, 应该生成一个 IEEE_addr_rsp 命令, 返回给本地设备, 状态域设置为 INV_REQUESTTYPE, IEEEAddrRemoteDev 域设置为这个设备的 IEEE 地址, NWKAddrRemoteDev 域设置为该设备的网络地址, 且 NumAssocDev、StartIndex 和 NWKAddrAssocDevList 域不能包含在帧中。

如果 RequestType 是单个设备响应，应该生成一个 IEEE_addr_resp 命令，返回给本地设备，状态域设置为 SUCCESS，IEEEAddrRemoteDev 域设置为所发现设备的 IEEE 地址，NWKAddrRemoteDev 域设置为请求的 NWK 地址，且 NumAssocDev、StartIndex 和 NWKAddrAssocDevList 域不能包含在帧中。

如果 RequestType 指明是一个扩展的响应，且远程设备是带有相关设备的 ZigBee 协调器或路由器，应该生成一个 IEEE_addr_resp 命令，返回给本地设备，状态域设置为 SUCCESS，IEEEAddrRemoteDev 域设置为设备本身的 IEEE 地址，NWKAddrRemoteDev 域设置为设备本身的 NWK 地址。远程设备还应该提供 NWKAddrAssocDevList 域中所有 16 位网络地址的一个列表，以 StartIndex 条目开头，并继续整个条目，直到达到 APS 数据包长度的最大值。然后它设置 NumAssocDev 域为 NWKAddrAssocDevList 域的条目个数。

2.4.3.1.3 Node_Desc_req

Node_Desc_req 命令（ClusterID=0x0002）应该按图 2.22 所示的格式编排。

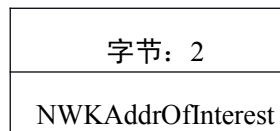


图 2.22 Node_Desc_req 命令帧的格式

表 2.46 指明了 Node_Desc_req 命令帧的域。

表 2.46 Node_Desc_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址

2.4.3.1.3.1 原语生成

Node_Desc_req 命令从一个希望查询远程设备的节点描述符的本地设备生成。该命令应该单播给远程设备本身或一个包含远程设备发现信息的备用设备。

本地设备应该使用表 2.46 所示的格式生成 Node_Desc_req 命令。NWKAddrOfInterest 域应该包含所需的节点描述符所在的远程设备的网络地址。

2.4.3.1.3.2 接收结果

在接收该命令时，接收设备应该处理命令，并根据 2.4.4.1.3.1 小节所做的描述，生成一个 Node_Desc_rsp 命令作为响应。

2.4.3.1.4 Power_Desc_req

Power_Desc_req 命令（ClusterID=0x0003）应该按图 2.23 所示的格式编排。

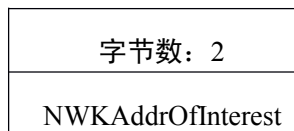


图 2.23 Power_Desc_req 命令帧格式

表 2.47 指明了 Power_Desc_req 命令帧的域。

表 2.47 Power_Desc_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址

2.4.3.1.4.1 原语生成

Power_Desc_req 命令从一个希望查询远程设备电源描述符的本地设备生成。该命令应该单播给远程设备自身或一个包含远程设备发现信息的备用设备。

本地设备应该使用表 2.47 所示的格式生成 Power_Desc_req 命令。NWKAddrOfInterest 域应该包含所需的电源描述符所在的远程设备的网络地址。

2.4.3.1.4.2 接收结果

在接收该命令时，接收设备应该处理命令，并根据 2.4.4.1.4.1 小节所做的描述生成一个 Power_Desc_rsp 命令作为响应。

2.4.3.1.5 Simple_Desc_req

Simple_Desc_req 命令（ClusterID=0x0004）应该按图 2.24 所示的格式编排。

字节数: 2	1
NWKAddrOfInterest	EndPoint

图 2.24 Simple_Desc_req 命令帧的格式

表 2.48 指明了 Simple_Desc_req 命令帧的域。

表 2.48 Simple_Desc_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址
endpoint	8 bits	1-240	目的端点

2.4.3.1.5.1 原语生成

Simple_Desc_req 命令从一个希望查询某个指定端点的远程设备简单描述符的本地设备生成。该命令应该单播给远程设备本身或一个包含远程设备发现信息的备用设备。

本地设备应该使用表 2.48 所示的格式生成 Simple_Desc_req 命令。NWKAddrOfInterest 域应该包含被查询的简单描述符所在的远程设备的网络地址，且端点域应该包含获得所需简单描述符的端点标识符。

2.4.3.1.5.2 接收结果

在接收该命令时，接收设备会处理命令，并根据 2.4.4.1.5.1 小节所做的描述生成一个 Simple_Desc_rsp 命令作为响应。

2.4.3.1.6 Active_EP_req

Active_EP_req 命令（ClusterID=0x0005）应该按图 2.25 所示的格式编排。

字节数: 2
NWKAddrOfInterest

图 2.25 Active_EP_req 命令帧的格式

表 2.49 指明了 Active_EP_req 命令帧的域。

表 2.49 Active_EP_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址

2.4.3.1.6.1 原语生成

Active_EP_req 命令从一个希望查询带有简单描述符的远程设备端点列表的本地设备生成。该命令应该单播给远程设备本身或一个包含远程设备发现信息的备用设备。

本地设备应该使用表 2.49 所示的格式生成 Active_EP_req 命令。NWKAddrOfInterest 域应该包含所需的活动端点列表所在的远程设备的网络地址。

2.4.3.1.6.2 接收结果

在接收该命令时，接收设备应该处理命令，并根据 2.4.4.1.6.1 所做的描述生成一个 Active_EP_rsp 命令作为响应。

2.4.3.1.7 Match_Desc_req

Match_Desc_req 命令（ClusterID=0x0006）应该按图 2.26 所示的格式编排。

字节数：2	2	1	可变的	1	可变的
NWKAddrOfInterest	ProfileID	NumInClusters	InClusterList	NumOutClusters	OutClusterList

图 2.26 Match_Desc_req 命令帧的格式

表 2.50 指明了 Match_Desc_req 命令帧的域。

表 2.50 Match_Desc_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址
ProfileID	Integer	0x0000-0xffff	在目的地配置的 Profile ID
NumInClusters	Integer	0x00-0xff	在 InClusterList 中提供匹配功能的输入 Cluster 的个数
InClusterList	2 bytes * NumInClusters		用于匹配的输入 ClusterID 列表；InClusterList 是通过远程设备配置的理想列表(InClusterList 的元素是本地设备提供的输出 Cluster)
NumOutClusters	Integer	0x00-0xff	在 OutClusterList 内提供匹配的输出的 Cluster 的个数
OutClusterList	2 bytes * NumOutClusters		用于匹配的输出的 ClusterID 列表；OutClusterList 是通过远程设备匹配的期望列表(OutClusterList 的元素是本地设备提供的输入 Cluster)

2.4.3.1.7.1 原语生成

Match_Desc_req 命令从一个希望找到支持给定简单描述符匹配标准的远程设备的本地设备生成。该命令应该广播给所有 RxOnWhenIdle=TRUE 的设备或单播。如果命令是单播的，它会发给远程设备本身或一个包含远程设备发现信息的备用设备。

本地设备应该使用表 2.50 所示的格式生成 Match_Desc_req 命令。如果命令要广播，或需要匹配所在的远程设备的网络地址，NWKAddrOfInterest 域应该包含广播给所有 RxOnWhenIdle=TRUE 设备的网络地址（0xffffd）。

剩余的域应该包含简单描述符匹配请求所需的标准。ProfileID 域应该包含被寻找的匹配所在 profile 的标识符。

NumInClusters 域应该包含 InClusterList 中元素的个数。如果该域的值 0，应该不包含 InClusterList 域。如果 NumInClusters 域的值不等于 0，InClusterList 域应该包含被寻找的匹配的输入 Cluster 标识符的列表。

NumOutClusters 域应该包含 OutClusterList 域中元素的个数。如果该域的值 0，应该不包含 OutClusterList 域。如果 NumOutClusters 域的值不等于 0，OutClusterList 域应该包含被寻找的匹配的输入 Cluster 标识符的列表。

2.4.3.1.7.2 接收结果

在接收该命令时，接收设备会处理命令，并根据 2.4.4.1.7.1 小节所做的描述生成一个 Match_Desc_rsp 命令作为响应。

2.4.3.1.8 Complex_Desc_req

Complex_Desc_req 命令（ClusterID=0x0010）应该按照图 2.27 所示的格式编排。

字节数: 2
NWKAddrOfInterest

图 2.27 Complex_Desc_req 命令帧的格式

表 2.51 指明了 Complex_Desc_req 命令帧的域。

表 2.51 Complex_Desc_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址

2.4.3.1.8.1 原语生成

Complex_Desc_req 命令从一个希望查询远程设备复杂描述符的本地设备生成。该命令应该单播给远程设备本身或一个包含远程设备发现信息的备用设备。

本地设备应该使用表 2.51 所示的格式生成 Complex_Desc_req 命令。NWKAddrOfInterest

域应该包含所需的复杂描述符所在的远程设备的网络地址。

2.4.3.1.8.2 接收结果

在接收该命令时，接收设备会处理命令，并根据 2.4.4.1.8.1 小节所做的描述生成一个 Complex_Desc_rsp 命令作为响应。

2.4.3.1.9 User_Desc_req

User_Desc_req (ClusterID=0x0011) 命令应该按图 2.28 所示的格式编排。

字节数: 2
NWKAddrOfInterest

图 2.28 User_Desc_req 命令帧的格式

表 2.52 指明了 User_Desc_req 命令帧的域。

表 2.52 User_Desc_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址

2.4.3.1.9.1 原语生成

User_Desc_req 命令从一个希望查询远程设备用户描述符的本地设备生成。该命令应该单播给远程设备本身或一个包含远程设备发现信息的备用设备。

本地设备应该使用表 2.52 所示的格式生成 User_Desc_req 命令。NWKAddrOfInterest 域应该包含所需的用户描述符所在的远程设备的网络地址。

2.4.3.1.9.2 接收结果

在接收该命令时，接收设备应该处理命令，并根据 2.4.4.1.9.1 小节所做的描述生成一个 User_Desc_rsp 命令作为响应。

2.4.3.1.10 Discovery_Cache_req

Discovery_Cache_req 命令（ClusterID=0x0012）应该按图 2.29 所示的格式编排。

字节数: 2	8
NWKAddr	IEEEAddr

图 2.29 Discovery_Cache_req 命令帧的格式

表 2.53 指明了 Discovery_Cache_req 命令帧的参数。

表 2.53 Discovery_Cache_req 命令的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	本地设备网络地址
IEEEAddr	Device Address	64 位 IEEE 地址	本地设备 IEEE 地址

2.4.3.1.10.1 原语生成

提供 Discovery_Cache_req，以使网络中的设备寻找网络中的一个主要发现缓存设备。本原语的目标地址广播给所有的 macRxOnWhenIdle=TRUE 的设备。

2.4.3.1.10.2 接收结果

在接收时，如果远程设备不支持 Discovery_Cache_req，请求应该被丢掉，不再做进一步处理。如果支持 Discovery_Cache_req，远程设备应该创建一个单播的 Discovery_Cache_rsp 信息给 Discovery_Cache_req 指定的源设备，并包含一个 SUCCESS 状态。

2.4.3.1.11 Device_annce

Device_annce 命令（ClusterID=0x0013）应该按图 2.30 所示的格式编排。

字节数: 2	8	1
NWKAddr	IEEEAddr	Capability

图 2.30 Device_annce 命令帧格式

表 2.54 指明 Device_annce 命令帧的域。

表 2.54 Device_annce 命令的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	本地设备网络地址
IEEEAddr	Device Address	64 位 IEEE 地址	本地设备 IEEE 地址
Capability	Bitmap	见图 2.17	本地设备的功能

2.4.3.1.11.1 原语生成

提供 Device_annce，以使网络中的 ZigBee 终端设备通知其它 ZigBee 设备，该终端设备已经加入或重新加入网络，确定了终端设备的 64 位 IEEE 地址和新的 16 位 NWK 地址，并告知远程设备 ZigBee 终端设备的功能。这个命令对于所有的 ZigBee 终端设备加入或重新加入时激发。这个命令还在 ZigBee 路由器作为 NWK 地址冲突解决方案的一部分加入或重新加入时激发。本原语的目标地址广播给所有 macRxOnWhenIdle =TRUE 的设备。

2.4.3.1.11.2 接收结果

在接收时，远程设备应该在信息中使用 IEEEAddr，以找到远程设备中任何绑定表条目的一个匹配。如果探测到一个匹配，远程设备应该更新 NIB 属性 *nwkAddressMap* 为与收到的 IEEEAddr 相对应的更新后的 NWKAddr。

远程设备应该在信息中使用 IEEEAddr，以找到远程设备中存有的 16 位网络地址一个匹配。如果探测到一个设备，IEEE 地址不是接收到的 IEEEAddr 地址域指示的地址，那么本条目应该标记为没有一个已知的有效 16 位网络地址。

2.4.3.1.12 User_Desc_set

User_Desc_set 命令（ClusterID=0x0014）应该按图 2.31 所示的格式编排。

字节数: 2	1	可变的
NWKAddrOfInterest	Length	UserDescriptor

图 2.31 User_Desc_set 命令帧格式

表 2.55 指明了 User_Desc_set 命令帧的域。

表 2.55 User_Desc_set 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址
Length	Integer	0x00-0x10	用户描述符的字节长度
UserDescription	User Descriptor		用户描述配置；如果在此输入的 ASCII 字符串长度小于 16 个字符，应补充空格字符（0x20），使总长度达到 16 个字符。代码为 0x00-0x1f 的字符是不允许的。

2.4.3.1.12.1 原语生成

User_Desc_set 命令从一个希望配置远程设备用户描述符的本地设备生成。该命令应该单播给远程设备自身或包含远程设备发现信息的备用设备。

本地设备应该使用表 2.55 所示的格式生成 User_Desc_set 命令。NWKAddrOfInterest 域应该包含要配置的用户描述符所在的远程设备的网络地址，且 UserDescription 域应该包含要配置到用户描述符的 ASCII 字符串。ASCII 代码编号 0x00 到 0x1f 不允许包含在这个字符串中。

2.4.3.1.12.2 接收结果

在接收该命令时，接收设备应该处理命令，并根据 2.4.4.1.11.1 小节所述的生成一个 User_Desc_conf 命令作为响应。

2.4.3.1.13 System_Server_Discovery_req

System_Server_Discovery_req 命令 (ClusterID=0x0015) 应该按图 2.32 所示的格式编排。

字节数: 2
ServerMask

图 2.32 System_Server_Discovery_req 命令帧格式

表 2.56 指明了 System_Server_Discovery_req 命令帧的域。

表 2.56 System_Server_Discovery_req 命令的域

名称	类型	有效范围	描述
ServerMask	Bitmap	16 位	见表 2.31 中关于位的分配

2.4.3.1.13.1 原语生成

System_Server_Discovery_req 从一个希望发现某个系统服务或 ServerMask 参数指明的服务器的本地设备生成。该请求的目标地址广播给所有 macRxOnWhenIdle=TRUE 的设备。

2.4.3.1.13.2 接收结果

在接收时, 远程设备应该在自己的节点描述符中比较 ServerMask 参数和 Server Mask 域。如果找不到匹配位, 就不采取行动。如果找到任何匹配位, 远程设备应该使用单播传输 (带有请求确认) 给发出者发送一个 System_Server_Discovery_rsp, 并显示匹配位。

2.4.3.1.14 Discovery_store_req

Discovery_store_req 命令 (ClusterID=0x0016) 应该按图 2.33 所示的格式编排。

字节数: 2	8	1	1	1	1	可变的
NWKAddr	IEEEAddr	NodeDesc-Size	PowerDesc-Size	ActiveEPS-ize	SimpleDesc-Count	SimpleDesc-SizeList

图 2.33 Discovery_Store_req 命令帧的域

表 2.57 指明了 Discovery_store_req 命令帧的域。

表 2.57 Discovery_store_req 命令的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	本地设备网络地址
IEEEAddr	Device Address	64 位 IEEE 地址	本地设备 IEEE 地址
NodeDescSize	Integer	0x00-0xff	本地设备节点描述符字节大小
PowerDescSize	Integer	0x00 - 0xff	本地设备电源描述符的字节大小
ActiveEPSize	Integer	0x00 - 0xff	本地设备 Active_EP_rsp 中 ActiveEPCount 和 ActiveIPList 域的字节大小
SimpleDescCount	Integer	0x00 - 0xff	本地设备支持的简单描述符的个数(应该 ActiveEPSize 有相同的值)
SimpleDescSizeList	Array of bytes		SimpleDescCount 长度的字节列表, 其中每个代表本地设备上每个活动端点的简单标识符的字节大小。

2.4.3.1.14.1 原语生成

提供 Discovery_store_req 以使网络中的 ZigBee 终端设备请求在一个主要发现缓存设备中存储其发现缓存信息。该请求包含本地设备要求的存储空间的数量。

本请求的目标寻址是单播。

2.4.3.1.14.2 接收结果

在接收时, 远程设备应该确定它是否是一个主要发现缓存设备。如果它不是一个主要发现缓存设备, 远程设备应该返回一个 NOT_SUPPORTED 的状态。其次, 远程设备应该确定它是否可以存储请求发现缓存的大小, 它取决于 NWKAddr 和 IEEEAddr 的大小相加, 再加上 NodeDescSize, PowerDescSize, ActiveEPSize 和 SimpleDescSizeList 的大小。如果有足够的空间, 本地设备应该提供一个 SUCCESS 状态。否则, 本地设备应该返回 INSUFFICIENT_SPACE。如果返回的是一个 SUCCESS 状态, 远程设备应该为装载本地设备的发现信息保留要求的存储空间。另外, 如果本地设备提供了一个 IEEEAddr, 它匹配一个之前的存储条目, 但是 NWKAddr 和之前的条目不同, 远程设备应该删除之前的条目, 发现缓存信息选择新登记的数据。

2.4.3.1.15 Node_Desc_store_req

Node_Desc_store_req 命令（ClusterID=0x0017）应该按图 2.34 所示的格式编排。

字节数: 2	8	可变的
NWKAddr	IEEEAddr	NodeDescriptor

图 2.34 Node_Desc_store_req 命令帧的格式

表 2.58 指明了 Node_Desc_store_req 命令帧的域。

表 2.58 Node_Desc_store_req 命令的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	本地设备网络地址
IEEEAddr	Device Address	64 位 IEEE 地址	本地设备 IEEE 地址
NodeDescriptor	Node Descriptor		见 2.3.2.3 小节的节点描述符的格式

2.4.3.1.15.1 原语生成

提供 Node_Desc_store_req，以使网络中的 ZigBee 终端设备请求在一个主要发现缓存设备中存储其节点描述符，其中之前已经收到给同一个主要发现缓存设备的 Discovery_store_req 的 SUCCESS 状态。该请求包含本地设备希望缓存的一个节点描述符。

2.4.3.1.15.2 接收结果

在接收时，远程设备会确定它是否是一个主要发现缓存设备。如果它不是一个主要发现缓存设备，远程设备会返回一个 NOT_SUPPORTED 状态。其次，远程设备会确定它之前是否为本地设备处理了一个 Discovery_store_req，并返回一个 SUCCESS 状态。如果之前没有处理过一个 SUCCESS 状态的 Discovery_store_req，远程设备应该返回 NOT_PERMITTED。接下来，远程设备会确定是否有足够的空间来为本地设备存储节点描述符。如果没有，远程设备会返回 INSUFFICIENT_SPACE。最后，远程设备应该为本地设备存储节点描述符，并返回 SUCCESS。如果该请求返回一个 SUCCESS 状态，且该请求所指的 NWKAddr 和 IEEEAddr 已经保存在主要发现缓存中，该请求的描述符应该重写之前保存的条目。

2.4.3.1.16 Power_Desc_store_req

Power_Desc_store_req 命令（ClusterID=0x0018）应该按图 2.35 所示的格式编排。

字节数: 2	8	可变的
NWKAddr	IEEEAddr	PowerDescriptor

图 2.35 Power_Desc_store_req 命令帧的格式

表 2.59 指明了 Power_Desc_store_req 命令帧的域。

表 2.59 Power_Desc_store_req 命令的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	本地设备网络地址
IEEEAddr	Device Address	64 位地址	本地设备 IEEE 地址
PowerDescriptor	Power Descriptor		见 2.3.2.4 小节中电源描述符格式；仅当状态域等于 SUCCESS，这个域才包含在帧中

2.4.3.1.16.1 原语生成

提供 Power_Desc_store_req，以使网络中的 ZigBee 终端设备请求在一个主要发现缓存设备中存储其电源描述符，其中之前已经收到给同一个主要发现缓存设备的 Discovery_store_req 的 SUCCESS 状态。该请求包含本地设备希望缓存的电源描述符。

2.4.3.1.16.2 接收结果

在接收时，远程设备应该确定它是否是一个主要发现缓存设备。如果它不是一个主要发现缓存设备，远程设备应该返回一个 NOT_SUPPORTED 状态。其次，远程设备应该确定它之前是否为本地设备处理了一个 Discovery_store_req，并返回一个 SUCCESS 状态。如果之前没有处理过一个 SUCCESS 状态的 Discovery_store_req，远程设备应该返回 NOT_PERMITTED。接下来，远程设备应该确定是否有足够的空间来为本地设备存储电源描述符。如果没有，远程设备应该返回 INSUFFICIENT_SPACE。最后，远程设备应该为本地设备存储电源描述符，并返回 SUCCESS。如果该请求返回一个 SUCCESS 状态，且该请求所指的 NWKAddr 和 IEEEAddr 已经保存在主要发现缓存中，该请求的描述符应该重写之前保存的条目。

2.4.3.1.17 Active_EP_store_req

Active_EP_store_req 命令（ClusterID=0x0019）应该按图 2.36 的格式编排。

字节数: 2	8	1	可变的
NWKAddr	IEEEAddr	ActiveEPCount	ActiveEPList

图 2.36 Active_EP_store_req 命令帧的域

表 2.60 指明了 Active_EP_store_req 命令帧的格式。

表 2.60 Active_EP_store_req 命令的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	本地设备网络地址
IEEEAddr	Device Address	64 位 IEEE 地址	本地设备 IEEE 地址
ActiveEPCount	Integer	0x00-0xff	本地设备上活动端点的个数
ActiveEPList			字节列表，每个代表一个 8 位端点编号

2.4.3.1.17.1 原语生成

提供 Active_EP_store_req，以使网络中的 ZigBee 终端设备请求在一个主要发现缓存中存储其活动端点列表，其中之前已经收到给同一个主要发现缓存设备的 Discovery_store_req 的 SUCCESS 状态。该请求包含本地设备希望缓存的活动端点的数目，以及端点列表本身。

2.4.3.1.17.2 接收结果

在接收时，远程设备应该确定它是否是一个主要发现缓存设备。如果它不是一个主要发现缓存设备，远程设备应该返回一个 NOT_SUPPORTED 状态。其次，远程设备应该确定它之前是否为本地设备处理了一个 Discovery_store_req，且返回一个 SUCCESS 状态。如果之前没有处理过一个 SUCCESS 状态的 Discovery_store_req，远程设备应该返回 NOT_PERMITTED。接下来，远程设备应该确定是否有足够的空间来为本地设备存储活动端点数目和列表。如果没有，远程设备应该返回 INSUFFICIENT_SPACE。最后，远程设备应该为本地设备存储活动端点数目和列表，并返回 SUCCESS。如果该请求返回一个 SUCCESS 状态，且该请求所指的 NWKAddr 和 IEEEAddr 已经保存在主要发现缓存中，该请求的描述符应该重写之前保存的条目。

2.4.3.1.18 Simple_Desc_store_req

Simple_Desc_store_req 命令（ClusterID=0x001a）应该按图 2.37 所示的格式编排。

字节数: 2	8	1	可变的
NWKAddr	IEEEAddr	Length	SimpleDescriptor

图 2.37 Simple_Desc_store_req 命令帧的格式

表 2.61 指明了 Simple_Desc_store_req 命令帧的域。

表 2.61 Simple_Desc_store_req 命令的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	本地设备网络地址
IEEEAddr	Device Address	64 位 IEEE 地址	本地设备 IEEE 地址
Length	Device Address	0x00 - 0xff	简单描述符遵循的字节长度
SimpleDescriptor	Simple Descriptor		见 2.3.2.5 小节中简单描述符的格式

2.4.3.1.18.1 原语生成

提供 Simple_Desc_store_req，以使网络中的 ZigBee 终端设备请求在一个主要发现缓存设备中存储其简单描述符列表，其中之前已经收到给同一个主要发现缓存设备的 Discovery_store_req 的 SUCCESS 状态。注意本地设备每个活动端点的每个简单描述符必须通过该命令开启缓存发现，分别装载在主要发现缓存设备中。该请求包含本地设备希望缓存的简单描述符的长度，以及简单描述符本身。端点是简单描述符内部的一个域，通过远程设备为本地设备管理发现缓存信息访问。

2.4.3.1.18.2 接收结果

在接收时，远程设备应该确定它是否是一个主要发现缓存设备。如果它不是一个主要发现缓存设备，远程设备应该返回一个 NOT_SUPPORTED 状态。其次，远程设备应该确定它之前是否为本地设备处理了一个 Discovery_store_req，且返回一个 SUCCESS 状态。如果之前没有处理过一个 SUCCESS 状态的 Discovery_store_req，远程设备应该返回 NOT_PERMITTED。接下来，远程设备应该确定是否有足够的空间来为本地设备存储简单描述符。如果没有，远程设备应该返回 INSUFFICIENT_SPACE。最后，远程设备应该为本地设备存储简单描述符，并返回 SUCCESS。如果该请求返回一个 SUCCESS 状态，且该请求所指的 NWKAddr 和 IEEEAddr 已经保存在主要发现缓存中，该请求的描述符应该重写之前保存的条目。

2.4.3.1.19 Remove_node_cache_req

Remove_node_cache_req 命令（ClusterID=0x001b）应该按图 2.38 所示的格式编排。

字节数: 2	8
NWKAddr	IEEEAddr

图 2.38 Remove_node_cache_req 命令帧的格式

表 2.62 指明了 Remove_node_cache_req 命令帧的域。

表 2.62 Remove_node_cache_req 命令的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	重要设备的网络地址
IEEEAddr	Device Address	64 位 IEEE 地址	重要设备的 IEEE 地址

2.4.3.1.19.1 原语生成

提供 Remove_node_cache_req, 以使网络中的 ZigBee 设备请求为一个指定的 ZigBee 终端设备从主要发现缓存设备中删除发现缓存信息。成功的 Remove_node_cache_req 结果是撤销之前成功的 Discovery_store_req, 另外在主要发现缓存设备上删除为指定 ZigBee 终端设备存储的任何缓存信息。

2.4.3.1.19.2 接收结果

在接收时, 远程设备应该确定它是否是一个主要发现缓存设备。如果它不是一个主要发现缓存设备, 远程设备应该返回一个 NOT_SUPPORTED 状态。其次, 远程设备应该确定它之前是否为本地设备处理了一个 Discovery_store_req, 且返回一个 SUCCESS 状态。如果之前没有处理过一个 SUCCESS 状态的 Discovery_store_req, 远程设备应该返回 DEVICE_NOT_FOUND。最后, 远程设备会为主要设备删除缓存发现信息, 并给本地设备返回 SUCCESS。

2.4.3.1.20 Find_node_cache_req

Find_node_cache_req 命令(ClusterID=0x001c)应该按图 2.39 所示的格式编排。

字节数: 2	8
NWKAddr	IEEEAddr

图 2.39 Find_node_cache 命令帧的格式

表 2.63 指明了 Find_node_cache_req 命令帧的域。

表 2.63 Find_node_cache 命令帧的域

名称	类型	有效范围	描述
NWKAddr	Device Address	16 位网络地址	重要设备的网络地址
IEEEAddr	Device Address	64 位 IEEE 地址	重要设备的 IEEE 地址

2.4.3.1.20.1 原语生成

提供 Find_node_cache_req, 以使网络中的 ZigBee 设备给所有 macRxOnWhenIdle=TRUE 的设备广播一个请求, 即根据请求参数所指, 找到网络上存有主要设备发现信息的设备。成功的 Find_node_cache_req 结果是有了主要发现缓存设备, 存有主要设备的发现信息, 并单播一个 Find_node_cache_rsp 返回给本地设备。注意只有满足这一标准的设备, 像 NWK_addr_req, 应该响应 Find_node_cache_req 发出的请求。

2.4.3.1.20.2 接收结果

在接收时，远程设备应该确定它是否是重要设备或一个主要发现缓存设备，如果是的话，还要看它是否含有重要设备的发现缓存信息。如果它不是重要设备或是一个主要发现缓存设备，以及不持有重要设备的发现缓存信息，远程设备应该停止处理该请求，不提供响应。如果远程设备是重要设备，或是一个主要发现缓存设备，且如果设备持有指定重要设备的发现信息，远程设备应该返回重要设备的 NWKAddr 和 IEEEAddr。

2.4.3.1.21 Extended_Simple_Desc_req

Extended_Simple_Desc_req 命令（ClusterID=0x001d）应该按图 2.40 所示的格式编排。

字节: 2	1	1
NWKAddrOfInterest	EndPoint	StartIndex

图 2.40 Extended_Simple_Desc_req 命令帧格式

表 2.64 指明了 Extended_Simple_Desc_req 命令帧的域。

表 2.64 Extended_Simple_Desc_req 命令帧的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址
Endpoint	8 位	1-240	目标端点
StartIndex	8 位	0x00-0xff	响应的 cluster 列表的起始索引, 由应用输入 cluster 列表和应用输出 cluster 列表的一个有序列表表示

2.4.3.1.21.1 原语产生

Extended_Simple_Desc_req 命令从希望查询特定端点远程设备的简单描述符的本地设备产生。本命令应该单播到远程设备本身，或者包含远程设备发现信息的备用设备。Extended_Simple_Desc_req 的目的是用于使用大量多于 Simple_Desc_req 可以描述的应用输入或输出 cluster 的设备。

本地设备应使用表 2.64 所示的格式产生 Extended_Simple_Desc_req 命令。NWKAddrOfInterest 域应包含所需的简单描述符所在的远程设备的网络地址，端点域应包含端点标识符，用以获得所需的简单描述符。StartIndex 是所产生的响应的应用输入 cluster 列表和应用输出 cluster 列表的序列中被请求的第一个条目。

2.4.3.1.2.2 接收结果

在收到该命令后，接受设备应该处理该命令，根据 2.4.4.1.20.1 小节产生 Extended_Simple_Desc_rsp 命令作为响应。

Extended_Simple_Desc_rsp 的结果应该包括表 2.108 描述的元素，以及应用输入 cluster 和应用输出 cluster 列表的可选集合，以条目 StartIndex 开始，继续到整个条目，直到到达 APS 数据包长度的最大值，带有一个 SUCCESS 状态。

2.4.3.1.2.2 Extended_Active_EP_req

Extended_Active_EP_req 命令（ClusterID=0x001e）应该按图 2.41 所示的格式编排。

字节: 2	1
NWKAddrOfInterest	StartIndex

图 2.41 Extended_Active_EP_req 命令帧格式

表 2.65 指明了 Extended_Active_EP_req 命令帧的域。

表 2.65 Extended_Active_EP_req 命令的域

名称	类型	有效范围	描述
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址
StartIndex	8 位	0x00-0xff	响应的活动端点列表中的起始索引

2.4.3.1.22.1 原语产生

Extended_Active_EP_req 命令从希望获得带有简单描述符的远程设备的端点列表的本地设备产生。本命令应该单播到远程设备本身，或者包含远程设备发现信息的备用设备。

Extended_Active_EP_req 用于支持多于一个 Active_EP_req 可以返回的活动端点的设备。

本地设备应该使用表 2.65 的格式产生 Extended_Active_EP_req 命令。NWKAddrOfInterest 域包含所需的活动端点列表所在的远程设备的网络地址。应该设置 StartIndex 域以启用活动端点检索那些列表超过单个 ASDU 的大小，且不支持分段的设备。

2.4.3.1.22.2 接收结果

在收到该命令后，接受设备应该处理该命令，根据 2.4.4.1.21.1 小节产生 Extended_Active_EP_rsp 作为响应。

Extended_Active_EP_rsp 的结果应该包括表 2.109 所述的元素，以及远程设备活动端点列表的可选集合，以条目 StartIndex 开始，继续到整个条目，直到达到 APS 数据包长度的最大值，或者应用输入和输出 cluster 列表耗尽，同时带有一个 SUCCESS 状态。

2.4.3.2 终端设备绑定，绑定，取消绑定和绑定管理客户端服务原语

表 2.66 列出了设备 Profile 支持的原语：终端设备绑定，绑定和取消绑定客户端服务。这些命令将在下面的分节中一一讨论。

表 2.66 终端设备绑定，绑定，取消绑定和绑定管理客户端服务命令

终端设备绑定, 绑定和取消绑定客户端服务	客户端传输	服务器处理
End_Device_Bind_req	O	O
Bind_req	O	O
Unbind_req	O	O
Bind_Register_req	O	O
Replace_Device_req	O	O

表 2.66 终端设备绑定, 绑定, 取消绑定和绑定管理客户端服务命令

Store_Bkup_Bind_Entry_req	O	O
Remove_Bkup_Bind_Entry_req	O	O
Backup_Bind_Table_req	O	O
Recover_Bind_Table_req	O	O
Backup_Source_Bind_req	O	O
Recover_Source_Bind_req	O	O

2.4.3.2.1 End_Device_Bind_req

End_Device_Bind_req 命令 (ClusterID=0x0020) 应该按图 2.42 所示的格式编排。

字节数: 2	8	1	2	1	可变的	1	可变的
Binding Target	SrcIEEE Address	Src Endpoint	Profile ID	Num InClusters	InCluster List	Num OutClusters	OutCluster List

图 2.42 End_Device_Bind_req 命令帧的格式

表 2.67 指明了 End_Device_Bind_req 命令帧的域。

表 2.67 End_Device_Bind_req 命令的域

名称	类型	有效范围	描述
BindingTarget	Device Address	16位网络地址	用于绑定的目标地址, 这既可以是主要缓存设备, 也可以是本地设备的短地址 ^a
SrcIEEEAddress	IEEE Address	一个有效的64位 IEEE网络地址	产生请求的设备的IEEE地址
SrcEndpoint	8 bits	1-240	产生请求的设备上的端点
ProfileID	Integer	0x0000-0xffff	在ZigBee的协调器预先配置的超时值之内, ZigBee协调器收到的两个 End_Device_Bind_req之间匹配的 ProfileID
NumInClusters	Integer	0x00-0xff	在InClusterList里提供给终端设备绑定的输入Cluster的个数。

表 2.67 End_Device_Bind_req 命令帧的域(续表)

名称	类型	有效范围	描述
InClusterList	2 bytes * NumInClusters		用于匹配的ClusterID输入列表。 InClusterList是ZigBee协调器和远程设备输出Cluster匹配的一个理想列表。 (InClusterList的元素由本地设备的输入Cluster提供)
NumOutClusters	Integer	0x00-0xff	在 OutClusterList内提供给匹配功能的输出cluster的个数。
OutClusterList	2 bytes * NumOutClusters		用于匹配的ClusterID输出列表。 OutClusterList是ZigBee协调器和远程设备输入Cluster匹配的一个理想列表。 (OutClusterList的元素由本地设备的输出Cluster提供)

2.4.3.2.1.1 原语生成

End_Device_Bind_req 从一个本地设备生成，希望执行到一个远程设备的终端设备绑定。End_Device_Bind_req 的生成典型根据一些用户行为，像按下一个按钮。该命令的目标地址应该单播，且目标地址应该是 ZigBee 协调器。

2.4.3.2.1.2 接收结果

在接收该命令时，ZigBee 协调器应该首先检查所提供的端点是否在指定的范围内。如果所提供的端点不在指定的范围内，ZigBee 协调器应该返回状态为 INVALID_EP 的一个 End_Device_Bind_rsp。

如果所提供的端点在指定的范围内，ZigBee 协调器应该为一个预先配置的超时时间保留 End_Device_Bind_req，等待第二个 End_Device_Bind_req。如果第二个请求在超时时间内没有出现，ZigBee 协调器应该生成一个 TIMEOUT 状态，并随着 End_Device_Bind_rsp 返回给发出的本地设备。假定第二个 End_Device_Bind_req 在超时时间内收到，它应该根据 ProfileID， InClusterList 以及 OutClusterList 和第一个请求匹配。

如果检测到没有 ProfileID 匹配，或如果 ProfileID 匹配但 InClusterList 或 OutClusterList 元素不匹配，应该通过到每个设备的 End_Device_Bind_rsp 给本地设备提供一个 NO_MATCH 的状态。如果有一个 ProfileID 匹配，且检测到至少一个输入或输出 clusterID，应该给生成 End_Device_Bind_req 的每个本地设备发出一个状态为 SUCCESS 的 End_Device_Bind_rsp。

为了方便行动的切换，ZigBee 协调器应该给绑定目标发出一个 Unbind_req 命令，指明任何一个匹配的 ClusterID 值。如果返回的状态是 NO_ENTRY，ZigBee 协调器应该给每个匹配的 ClusterID 值发出一个 Bind_req 命令。否则 ZigBee 协调器应该断定绑定记录应该被删除，并应该为任何进一步匹配的 ClusterID 值发出一个 Unbind_req 命令。

初始的 Unbind_req 和其后包含匹配的 cluster 的任何 Bind_req 或 Unbind_req，应该送到发起设备指明的绑定目标之一。绑定目标根据每个匹配 cluster 各自选择，如发起设备选择的绑定目标有作为输出的 cluster。SrcAddress 域应该包含同一个发起设备的 64 位 IEEE 地址，SrcEndp 域包含其端点。DstAddress 包含发起设备的 64 位 IEEE 地址，在其输入 cluster 列表中有匹配 cluster，DstEndp 域包含其端点。

2.4.3.2.2 Bind_req

Bind_req 命令（ClusterID=0x0021）应该按图 2.43 所示的格式编排。

字节数: 8	1	2	1	2/8	0/1
SrcAddress	SrcEndp	ClusterID	DstAddrMode	DstAddress	DstEndp

图 2.43 Bind_req 命令帧的格式

表 2.68 指明了 Bind_req 命令帧的域。

表 2.68 Bind_req 命令的域

名称	类型	有效范围	描述
SrcAddress	IEEE Address	有效的64位 IEEE地址	源IEEE地址
SrcEndp	Integer	0x01-0xf0	绑定目录的源端点
ClusterID	Integer	0x0000-0xffff	绑定到目的地的源设备上的cluster标识符

表 2.68 Bind_req 命令的域（续表）

名称	类型	有效范围	描述
DstAddrMode	Integer	0x00-0xff	这个命令使用的目的地址寻址模式。这个域可以从下面列表中取一个非保留值： 0x00=保留 0x01=DstAddress 16位组地址，DstEndp不存在 0x02=保留 0x03= DstAddress和DstEndp 都存在，是64位扩展地址 0x04-0xff=保留
DstAddress	Address	通过 DstAddrMode域 来定义	绑定目录的目的地址
DstEndp	Integer	0x01-0xf0	仅当DstAddrMode域的值为0x03，该域才存在。如果存在，该域将会成为绑定目录的端点。

2.4.3.2.2.1 原语生成

Bind_req 从一个本地设备生成，希望为参数所含的源和目标地址创建一个绑定表条目。该命令的目标地址只能单播，目标地址是主要绑定表缓存或是 SrcAddress 本身。绑定管理器是可选支持源设备的（除非设备也是 ZigBee 协调器），这样如果不支持的话，设备应该给 Bind_req 发出一个 NOT_SUPPORTED 状态。

2.4.3.2.2.2 接收结果

在接收时，如果支持绑定管理器，一个远程设备（一个主要绑定表缓存或 SrcAddress 指定的设备）应该根据 Bind_req 提供的参数创建一个绑定表条目。如果远程设备是一个主要绑定表缓存，需要下面附加的程序。首先，主要缓存应该为 SrcAddress 中的设备检查持有自己的源绑定的设备的表，如果找到的话，给具有同一条目的设备发出另一个 Bind_req。第二，主要缓存检查是否有一个备份绑定表缓存，如果有的话，发出一个 Store_Bkup_Binding_Entry_req 命令来备份新的条目。如果该条目已经被绑定管理器创建，那么远程设备应该以 SUCCESS 响应；否则，远程设备以 NOT_SUPPORTED 响应。



本命令必须经过 4.6.3.8 小节定义的一个允许检查。如果失败，远程设备不能执行本命令，以 NOT_AUTHORIZED 响应。

2.4.3.2.3 Unbind_req

Unbind_req 命令（ClusterID=0x0022）应该按图 2.44 的格式编排。

字节数: 8	1	2	1	2/8	0/1
SrcAddress	SrcEndp	ClusterID	DstAddrMode	DstAddress	DstEndp

图 2.44 Unbind_req 命令帧的格式

表 2.69 指明了 Unbind_req 命令帧的域。

表 2.69 Unbind_req 命令的域

名称	类型	有效范围	描述
SrcAddress	IEEE Address	一个有效的64位IEEE地址	源IEEE地址
SrcEndp	Integer	0x01-0xf0	绑定目录的源端点
ClusterID	Integer	0x0000-0xffff	要绑定到目标的源设备上的cluster标识符
DstAddrMode	Integer	0x00-0xff	该命令使用的目的地址寻址模式。这个域可以在下面的列表中取一个非保留值： 0x00=保留 0x01=DstAddress 16位组地址, DstEndp不存在 0x02=保留 0x03=DstAddress和DstEndp都存在，是64位扩展地址 0x04-0xff=保留
DstAddress	Address	通过DstAddrMode域来定义	绑定目录的目的地址
DstEndp	Integer	0x01-0xf0	只有当DstAddrMode域的值为0x03，该域才存在，如果存在，该域将是绑定目录的目的端点。

2.4.3.2.3.1 原语生成

Unbind_req 从一个本地设备生成，希望为参数所含的源和目标地址删除一个绑定表条目。该命令的目标地址只能是单播的，且目标地址必须是主要绑定表缓存或 SrcAddress。

2.4.3.2.3.2 接收结果

远程设备应该评估是否支持该请求。如果不支持该请求，返回一个 NOT_SUPPORTED 状态。如果支持该请求，远程设备（一个主要绑定表缓存或 SrcAddress）根据 Unbind_req 提供的参数删除一个绑定表条目。如果远程设备是一个主要绑定表缓存需要以下附加的程序。首先，主要缓存检查它存有自己的源绑定的设备的表，寻找 SrcAddress 中的设备，如果找到，发出另一个 Unbind_req 给具有相同条目的那个设备。第二，主要缓存检查是否有一个备份绑定表缓存，如果有的话，发出一个 Remove_Bkup_Bind_Entry_req 命令来删除这个条目的备份。如果参数所含的 SrvAddress, SrcEndp, ClusterID, DstAddress, DstEndp 的绑定表条目不存在，远程设备应该以 NO_ENTRY 响应。否则，远程设备应删除指定的绑定表条目，并以 SUCCESS 响应。

本命令必须经过 4.6.3.8 小节定义的一个允许检查。如果失败，远程设备不能执行本命令，以 NOT_AUTHORIZED 响应。

2.4.3.2.4 Bind_Register_req

Bind_Register_req 命令（ClusterID=0x0023）应该按图 2.45 所示的格式编排。

字节数: 8
NodeAddress

图 2.45 Bind_Register_req 命令帧的格式

表 2.70 指明了 Bind_Register_req 命令帧的域。

表 2.70 Bind_Register_req 命令的域

名称	类型	有效范围	描述
NodeAddress	IEEE地址	一个有效的64位IEEE地址	想持有它自己的绑定表的节点的地址

2.4.3.2.4.1 原语生成

Bind_Register_req 从一个本地设备生成，发给一个主要绑定表缓存设备，以注册希望持有自己的绑定表条目的本地设备。该请求的目标地址模式是单播的。

2.4.3.2.4.2 接收结果

如果远程设备不是一个主要绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。否则，主要绑定表缓存应该把参数给定的 NodeAddress 增加到源设备的表中，其中源设备选择存储自己的绑定表。如果失败，返回一个 TABLE_FULL 状态。否则返回一个 SUCCESS 状态。如果 NodeAddress 的一个条目已经存在于源设备的表中，行为将和新增加相同。源设备应该在发出该命令之前清除其源绑定表，以避免同步问题。在成功的情况下，源地址为 NodeAddress 的绑定表中一个已存在的绑定条目将被送给请求设备，列入其源绑定表。参见 Bind_Register_rsp 获得进一步信息。随后写入到绑定列表的绑定条目将导致使用 Bind_req，把副本写入源设备。

2.4.3.2.5 Replace_Device_req

Replace_Device_req 命令（ClusterID=0x0024）应按图 2.46 所示的格式编排。

字节数: 8	1	8	1
OldAddress	OldEndpoint	NewAddress	NewEndpoint

图 2.46 Replace_Device_req 命令帧的格式

表 2.71 指明了 Replace_Device_req 命令帧的域。

表 2.71 Replace_Device_req 命令的域

名称	类型	有效范围	描述
OldAddress	IEEE Address	有效的64位IEEE	将要被替换的节点的地址
OldEndpoint	Integer	0x01 - 0xf0	将要被替换的端点
NewAddress	IEEE Address	有效的64位IEEE	更换地址
NewEndpoint	Integer	0x01 - 0xf0	更换端点

2.4.3.2.5.1 原语生成

Replace_Device_req 是为诸如调试工具的某个设备使用，发送给一个主要绑定表缓存设备，以改变所有匹配指定 OldAddress 和 OldEndpoint 绑定表的条目。注意 OldEndpoint = 0 有特殊意义，意味着只需要匹配地址。绑定表中的端点不改变，因此 NewEndpoint 被忽略。程序改变了所有源地址和 OldAddress 相同，且如果 OldEndpoint 非零，源端点和 OldEndpoint 相同的绑定表条目。它还改变了所有目标地址和 OldAddress 相同，且如果 OldAddress 非零，目标端点和 OldEndpoint 相同的绑定表条目。该命令的目标地址模式是单播的。

2.4.3.2.5.2 接收结果

如果远程设备不是一个主要绑定表缓存，它返回一个 NOT_SUPPORTED 状态。主要绑定表缓存应检查它的 OldAddress 是否非零，如果是的话，在其绑定表中查询源地址的条目和源端点，或目标地址和目标端点，它们与 OldAddress 和 OldEndpoint 的设置是相同的。它应该改变这些条目，变成 NewAddress 和 NewEndpoint。在 OldEndpoint 是零的情况下，主要绑定表缓存应该在其绑定表中查询源地址或目标地址匹配 OldAddress 的条目。它会改变这些条目，变成 NewAddress，但端点值不变，且忽略 NewEndpoint。然后它会返回一个 SUCCESS 响应。主要绑定表缓存也应负责把改变通知给注册为持有自己源绑定表的那些受影响的设备。如果目标地址被改变，源地址出现在选择存储自己绑定表的源设备列表中，这对每个改变的绑定表条目将是必要的。在每个这种情况下，修订的绑定表条目将使用一个 Unbind_req 命令送给源设备，随后对于新条目，使用 Bind_req 命令。在绑定条目的源地址已经改变的情况下，主要绑定表缓存必须给旧的源设备发送一个 Unbind_req 命令，如果它是一个源绑定设备，并给新的源绑定设备发送一个 Bind_req 命令，如果它是一个源绑定设备。对于旧条目，主要绑定表缓存也要通过 Remove_bkup_binding_entry_req 命令更新备份绑定表缓存，对于改变过的条目，使用 Store_bkup_binding_entry_req。

2.4.3.2.6 Store_Bkup_Bind_Entry_req

Store_Bkup_Bind_Entry_req 命令（ClusterID=0x0025）应按图 2.47 所示和格式编排。

字节数: 8	1	2	1	2/8	0/1
SrcAddress	SrcEndp	ClusterID	DstAddrMode	DstAddress	DstEndp ^a

图 2.47 Store_Bkup_Bind_Entry_req 命令帧的格式

表 2.72 指明了 Store_Bkup_Bind_Entry_req 命令帧的格式。

表 2.72 Store_Bkup_Bind_Entry_req 命令的域

名称	类型	有效范围	描述
SrcAddress	IEEE Address	有效的64位IEEE地址	源IEEE地址
SrcEndpoint	Integer	0x01 - 0xf0	绑定目录的源端点
ClusterId	Integer	0x0000 - 0xffff	绑定到目的地的源设备上 cluster 的标识符
DstAddrMode	Integer	0x00-0xff	这个命令使用的目的地址寻址模式。这个域可以从下面列表中取一个非保留值： 0x00 = 保留 0x01 = DstAddress 16位组地址，DstEndp不存在 0x02 = 保留 0x03 = DstAddress和DstEndp都存在，是64位扩展地址 0x04 - 0xff = 保留
DstAddress	Address	通过DstAddrMode域来定义	绑定表的目的地地址
DstEndp	Integer	0x01-0xf0	只有当DstAddrMode域的值为0x03，该域才存在。如果存在，该域将成为绑定目录的目的端点。

2.4.3.2.6.1 原语生成

Store_Bkup_Bind_Entry_req 从一个本地主要绑定表缓存生成，发给一个远程备份绑定表缓存设备，以请求备份存储的条目。它将在主要绑定表缓存已经创建一个新的绑定表条目时生成。该请求的目标寻址模式是单播的。

2.4.3.2.6.2 接收结果

如果远程设备不是一个备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果它是备份绑定表缓存，它应从之前的发现保持主要绑定表缓存的身份。如果 Store_Bkup_Bind_Entry 参数的内容匹配绑定表缓存中一个已存在的条目，那么远程设备返回 SUCCESS。否则，备份绑定表缓存应把绑定条目增加到其绑定表中，并返回 SUCCESS 状态。如果没有空间了，应返回 TABLE_FULL 状态。

2.4.3.2.7 Remove_Bkup_Bind_Entry_req

Remove_Bkup_Bind_Entry_req 命令（ClusterID=0x0026）应按图 2.48 所示的格式编排。

字节数: 8	1	2	1	2/8	0/1
SrcAddress	SrcEndp	ClusterID	DstAddrMode	DstAddress	DstEndp

图 2.48 Remove_Bkup_Bind_Entry_req 命令帧格式

表 2.73 指明了 Remove_Bkup_Bind_Entry_req 命令帧的域。

表 2.73 Remove_Bkup_Bind_Entry_req 命令的域

名称	类型	有效范围	描述
SrcAddress	IEEE Address	有效的64位 IEEE地址	源IEEE地址
SrcEndpoint	Integer	0x01 - 0xf0	绑定目录的IEEE地址
ClusterId	Integer	0x0000 - 0xffff	绑定到目的地的设备上cluster标识符

表 2.73 Remove_Bkup_Bind_Entry_req 命令的域（续表）

名称	类型	有效范围	描述
DstAddrMode	Integer	0x00-0xff	用于该命令的目的地址寻址模式。该域可以从下面列表中取一个非保留值： 0x00 = 保留 0x01 = DstAddress 16位组地址，DstEndp 不存在 0x02 = 保留 0x03 = DstAddress和DstEndp都存在，是64位扩展地址 0x04 – 0xff = 保留
DstAddress	Address	通过 DstAddrMode 域来定义的	绑定目录的目的地址
DstEndp	Integer	0x01-0xf0	仅当DstAddrMode域的值位0x03,该域才存在。如果存在，该域将成为绑定目录的目的端点。

2.4.3.2.7.1 原语生成

Remove_Bkup_Bind_Entry_req 从一个本地主要绑定表缓存生成，发给一个远程备份绑定表缓存，以请求删除备份存储的条目。它在一个绑定表条目已经被主要绑定表缓存取消绑定时生成。该请求的目标寻址模式是单播的。

2.4.3.2.7.2 接收结果

如果远程设备不是一个备份绑定表缓存，它应该返回一个 NOT_SUPPORTED 状态。如果它是一个备份绑定表缓存，它应该从之前的发现保持主要绑定表缓存的身份。如果它不承认发送设备作为绑定表缓存，它应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应在其绑定表中查询和所提供的参数相对应的条目。如果找不到条目，返回 NO_ENTRY 状态。否则，它应删除该条目，并返回 SUCCESS 状态。

2.4.3.2.8 Backup_Bind_Table_req

Backup_Bind_Table_req 命令（ClusterID=0x0027）应按图 2.49 所示的格式编排。

字节数: 2	2	2	可变的
BindingTableEntries	StartIndex	BindingTableListCount	BindingTableList

图 2.49 Backup_Bind_Table_req 命令帧的格式

表 2.74 指明了 Backup_Bind_Table_req 命令帧的格式。

表 2.74 Backup_Bind_Table_req 命令的域

命令	类型	有效范围	描述
BindingTableEntries	Integer	0x0000 - 0xffff	在主要绑定表缓存设备上的绑定目录表的总数
StartIndex	Integer	0x0000 - 0xffff	绑定表目录的开始索引
BindingTableListCount	Integer	0x0000 - 0xffff	包含在BindingTableList内的绑定目录表的个数
BindingTableList	List of binding descriptors	列表应该包含 BindingTableListCount 给出的元素个数	描述符列表，以StartIndex元素开始，继续到主要绑定表缓存设备的绑定表的元素 BindingTableListCount（详见表2.130）。

2.4.3.2.8.1 原语生成

Backup_Bind_Table_req 从一个本地主要绑定表缓存生成，发给远程备份绑定表缓存设备，以请求备份存储其整个绑定表。该请求的目标寻址模式是单播的。

2.4.3.2.8.2 接收结果

如果远程设备不是一个备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果它是一个绑定表缓存，它应从之前的发现保持主要绑定表缓存的身份。如果它不承认发送设备作为主要绑定表缓存，应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应在其绑定表中重写绑定条目，以 StartIndex 开头，继续到 BindingTableListCount 条目。如果超出了该表的大小，应填充尽可能多的条目，并返回一个 TABLE_FULL 状态。否则，应返回一个 SUCCESS 状态。该请求在写入的最后一个条目有效地截断该表到结束。该表新的大小在响应中返回，将等于 StartIndex + BindingTableListCount，除非当达到表的最大值返回 TABLE_FULL。

2.4.3.2.9 Recover_Bind_Table_req

Recover_Bind_Table_req 命令（ClusterID=0x0028）应按图 2.50 所示的格式编排。

字节数: 2
StartIndex

图 2.50 Recover_Bind_Table_req 命令帧的域

表 2.75 指明了 Recover_Bind_Table_req 命令帧的域。

表 2.75 Recover_Bind_Table_req 命令的域

名称	类型	有效范围	描述
StartIndex	Integer	0x0000 - 0xffff	绑定表被请求的元素的起始索引

2.4.3.2.9.1 原语生成

Recover_Bind_Table_req 从一个本地主要绑定表缓存生成，当它想要完整地恢复绑定表时，发给一个远程备份绑定表缓存设备。该请求的目标寻址模式是单播的。

2.4.3.2.9.2 接收结果

如果远程设备不是备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果它不承认发送设备作为主要绑定表缓存，返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应从其备份中准备一个绑定表条目的列表，以 StartIndex 开始。它将给 Recover_Bind_Table_rsp 命令填入尽可能多的条目，并返回一个 SUCCESS 状态。

2.4.3.2.10 Backup_Source_Bind_req

Backup_Source_Bind_req 命令（ClusterID=0x0029）应按图 2.51 所示的格式编排。

字节数: 2	2	2	可变的
SourceTableEntries	StartIndex	SourceTableListCount	SourceTableList

图 2.51 Backup_Source_Bind_req 命令帧的域

表 2.76 指明了 Backup_Source_Bind_req 命令帧的域。

表 2.76 Backup_Source_Bind_req 命令的域

名称	类型	有效范围	描述
SourceTableEntries	Integer	0x0000 - 0xffff	在主要绑定表缓存设备上的源表目录总数
StartIndex	Integer	0x0000 - 0xffff	在SourceTableList中的绑定表目录的起始索引
SourceTableListCount	Integer	0x0000 - 0xffff	包括在SourceTableList内的源表目录数
SourceTableList	List of IEEE Addresses	列表应该包括 SourceTableListCount 给出的元素个数	地址列表，以StartIndex元素开始，继续到主要绑定表缓存设备的源表的元素 SourceTableListCount

2.4.3.2.10.1 原语生成

Backup_Source_Bind_req 从一个本地主要绑定表缓存生成，发给一个远程备份绑定表缓存设备，以请求备份存储其整个源表。该请求的目标寻址模式是单播的。

2.4.3.2.10.2 接收结果

如果一个远程设备不是备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果它不承认发送设备作为主要绑定表缓存，应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应在其备份源表中重写源条目，以 StartIndex 开始，继续到 SourceTableListCount 条目。如果超过了该表大小，应返回一个 TABLE_FULL 状态。否则应返回一个 SUCCESS 状态。该命令总是在条目数量等于其最大值或 SourceTableEntries，选择其中较小值，然后截断备份表。

2.4.3.2.11 Recover_Source_Bind_req

Recover_Source_Bind_req 命令（ClusterID=0x002a）应按图 2.52 所示的格式编排。

字节数: 2
StartIndex

图 2.52 Recover_Source_Bind_req 命令帧的格式

表 2.77 指明了 Recover_Source_Bind_req 命令帧的域。

表 2.77 Recover_Source_Bind_req 命令的域

名称	类型	有效范围	描述
StartIndex	Integer	0x0000 - 0xffff	绑定表被请求的元素的起始索引

2.4.3.2.11.1 原语生成

Recover_Source_Bind_req 从一个本地主要绑定表缓存生成，当它想要完整地恢复源绑定表时，发给远程备份绑定表缓存设备。该请求的目标寻址模式是单播的。

2.4.3.2.11.2 接收结果

如果远程设备不是备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果它不承认发送设备作为主要绑定表缓存，应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应从其备份中准备一个源绑定表条目的列表，以 StartIndex 开始。它将给 Recover_Source_Bind_rsp 命令填充尽可能多的条目，并返回一个 SUCCESS 状态。

2.4.3.3 网络管理客户端服务

表 2.78 列出了设备 Profile 支持的命令：网络管理客户端服务。这些原语将在以下分节一一讨论。

表 2.78 网络管理客户端服务命令

网络管理客户端服务	客户端传输	服务器处理
Mgmt_NWK_Disc_req	O	O
Mgmt_Lqi_req	O	O
Mgmt_Rtg_req	O	O
Mgmt_Bind_req	O	O
Mgmt_Leave_req	O	O
Mgmt_Direct_Join_req	O	O
Mgmt_Permit_Joining_req	O	M
Mgmt_Cache_req	O	O
Mgmt_NWK_Update_req	O	O

2.4.3.3.1 Mgmt_NWK_Disc_req

Mgmt_NWK_Disc_req 命令 (ClusterID=0x0030) 应按图 2.53 所示的格式编排。

字节数: 4	1	1
ScanChannels	ScanDuration	StartIndex

图 2.53 Mgmt_NWK_Disc_req 命令帧格式

表 2.79 指明了 Mgmt_NWK_Disc_req 命令帧的域。

表 2.79 Mgmt_NWK_Disc_req 命令的域

名称	类型	有效范围	描述
ScanChannels	Bitmap	32位域	见 (3.3.2.1小节) 中NLME-NETWORKDISCOVERY.request ScanChannels 参数的详细描述
ScanDuration	Integer	0x00-0x0e	用来计算每个通道扫描花费的时间长度的一个值。每个通道扫描花费时间的标志 ($aBaseSuperframeDuration * (2n + 1)$)。N是 ScanDuration参数的值。更多信息请查阅MAC 子层。(见[B1])
StartIndex	Integer	0x00-0xff	开始报告给Mgmt_NWK_Disc_rsp的NLME-NETWORKDISCOVERY.confirm NetworkList 的结果中的起始索引

2.4.3.3.1.1 原语生成

Mgmt_NWK_Disc_req 从一个本地设备生成，请求远程设备执行一个扫描，以报告本地设备附近的网络。该请求的目标寻址模式是单播的。

2.4.3.3.1.2 接收结果

远程设备应使用 Mgmt_NWK_Disc_req 命令提供的 ScanChannels 和 ScanDuration 参数，执行一个 NLME-NETWORK-DISCOVERY.request。扫描的结果应通过 Mgmt_NWK_Disc_rsp 命令报告给本地设备。

如果远程设备不支持该命令，Mgmt_NWK_Disc_rsp 提供的返回状态应该是 NOT_SUPPORTED。如果扫描成功，Mgmt_NWK_Disc_rsp 命令应包含一个 SUCCESS 状态，且应该报告扫描的结果，以 NetworkList 的 StartIndex 元素开始。如果扫描不成功，Mgmt_NWK_Disc_rsp 命令应包含 NLME-NETWORK-DISCOVERY.confirm 原语报告的错误代码。

2.4.3.3.2 Mgmt_Lqi_req

Mgmt_Lqi_req 命令 (ClusterID=0x0031) 应按图 2.54 所示的格式编排。

字节数: 1
StartIndex

图 2.54 Mgmt_Lqi_req 命令帧的格式

表 2.80 指明了 Mgmt_Lqi_req 命令帧的域。

表 2.80 Mgmt_Lqi_req 命令的域

名称	类型	有效范围	描述
StartIndex	Integer	0x00-0xff	邻居表被请求的事件的起始索引

Mgmt_Lqi_re

2.4.3.3.2.1 原语生成 q 从一个本地设备生成, 希望获得远程设备的一个相邻列表, 以及各个邻居的相关 LQI 值。该命令的目标寻址应该只是单播的, 且目标地址必须是一个 ZigBee 协调器或 ZigBee 路由器。

2.4.3.3.2.2 接收结果

在接收时, 一个远程设备 (ZigBee 路由器或 ZigBee 协调器) 应通过 NLME-GET.request 原语 (为了 *nwkNeighborTable* 属性) 检索相邻表的条目和相关的 LQI 值, 并通过 Mgmt_Lqi_rsp 命令报告相邻表的结果 (通过 NLME-GET.confirm 原语获得)。

如果远程设备不支持该命令, Mgmt_Lqi_rsp 提供的返回状态应是 NOT_SUPPORTED。如果相邻表成功获得, Mgmt_Lqi_rsp 命令应包含一个 SUCCESS 状态, 且应报告相邻表, 以列表枚举的元素 StartIndex 开始。如果相邻表没有成功获得, Mgmt_Lqi_rsp 命令应包含 NLME-GET.confirm 原语报告的错误代码。

2.4.3.3.3 Mgmt_Rtg_req

Mgmt_Rtg_req 命令（ClusterID=0x0032）应按图 2.55 所示的格式编排。

字节数: 1
StartIndex

图 2.55 Mgmt_Rtg_req 命令帧的格式

表 2.81 指明了 Mgmt_Rtg_req 命令帧的域。

表 2.81 Mgmt_Rtg_req 命令的域

名称	类型	有效范围	描述
StartIndex	Integer	0x00-0xff	路由表被请求的元素的起始索引

2.4.3.3.3.1 原语生成

Mgmt_Rtg_req 从一个本地设备生成，希望从远程设备检索路由表的内容。该命令的目标寻址应只是单播的，且目标地址必须是 ZigBee 路由器或 ZigBee 协调器。

2.4.3.3.3.2 接收结果

在接收时，一个远程设备（ZigBee 协调器或 ZigBee 路由器）应通过 NLME-GET.request 原语（为了 *nwkRouteTable* 属性）从 NWK 层检索路由表的条目，并通过 Mgmt_Rtg_rsp 命令报告路由表的结果（通过 NLME-GET.confirm 原语获得）。

如果远程设备不支持该可选的管理请求，它应返回一个 NOT_SUPPORTED 状态。如果路由表成功获得，Mgmt_Rtg_req 命令应包含一个 SUCCESS 状态，并报告路由表，以列表中枚举的元素 StartIndex 开始。如果路由表没有成功获得，Mgmt_Rtg_rsp 命令应包含 NLME-GET.confirm 原语报告的错误代码。

2.4.3.3.4 Mgmt_Bind_req

Mgmt_Bind_req 命令（ClusterID=0x0033）应按图 2.56 所示的格式编排。

字节数: 1
StartIndex

图 2.56 Mgmt_Bind_req 命令帧的格式

表 2.82 指明了 Mgmt_Bind_req 命令帧的域。

表 2.82 Mgmt_Bind_req 命令的域

名称	类型	有效范围	描述
StartIndex	Integer	0x00-0xff	绑定表被请求的元素的起始索引

2.4.3.3.4.1 原语生成

Mgmt_Bind_req 从一个本地设备生成，希望从远程设备检索绑定表的内容。该命令的目标寻址应只是单播的，且目标地址必须是一个主要绑定表缓存或持有自己的绑定表的源设备。

2.4.3.3.4.2 接收结果

在接收时，一个远程设备应通过 APSME-GET.request 原语（为了 *apsBindingTable* 属性）从 APS 子层中检索绑定表的条目，并通过 Mgmt_Bind_rsp 命令报告绑定表的结果（通过 APSME-GET.confirm 原语获得）。

如果远程设备不支持该可选的管理请求，它应返回一个 NOT_SUPPORTED 状态。如果绑定表成功获得，Mgmt_Bind_rsp 命令应包含一个 SUCCESS 状态，且应报告绑定表，以列表中枚举的元素 StartIndex 开始。如果绑定表没有成功获得，Mgmt_Bind_rsp 命令应包含 APSME-GET.confirm 原语报告的错误代码。

2.4.3.3.5 Mgmt_Leave_req

Mgmt_Leave_req 命令（ClusterID=0x0034）应按图 2.57 所示的格式编排。

位数: 64	6	1	1
Device Address	Reserved	Remove Children	Rejoin

图 2.57 Mgmt_Leave_req 命令帧的格式

表 2.83 指明了 Mgmt_Leave_req 命令帧的域。

表 2.83 Mgmt_Leave_req 命令的域

名称	类型	有效范围	描述
DeviceAddresses	Device Address	扩展64位，IEEE地址	见（3.2.2.16小节）的NLME-LEAVE.request内的DeviceAddress参数的详细描述。对于为NULL的DeviceAddress，应使用0x0000000000000000。
Remove Children	Bit	0或1	如果设备被要求离开网络，且如果有的话，也被要求移除它的子设备时，该域的值1。否则该域的值0。
Rejoin	Bit	0或1	如果被要求离开的设备，它的当前父节点被请求重新加入网络，该域的值1，否则，该域的值0。

a. CCB #849

2.4.3.3.5.1 原语生成

Mgmt_Leave_req 从一个本地设备生成，请求一个远程设备离开网络或请求另一个设备离开网络。Mgmt_Leave_req 由一个管理应用程序生成，并把该请求发给一个远程设备，它使用 Mgmt_Leave_req 提供的参数执行 NLME-LEAVE.request。

2.4.3.3.5.2 接收结果

在接收时，远程设备应使用 Mgmt_Leave_req 命令提供的参数发出 NLME-LEAVE.request 原语。尝试离开的结果应通过 Mgmt_Leave_rsp 命令报告给本地设备。

如果远程设备不支持该可选的管理请求，它应返回一个 NOT_SUPPORTED 状态。如果尝试离开执行成功，Mgmt_Leave_rsp 命令应包含一个 SUCCESS 状态。如果尝试离开没有执行成功，Mgmt_Leave_rsp 命令应包含 NLME-LEAVE.confirm 原语报告的错误代码。

该命令必须经过 4.6.3.8 小节定义的一个允许检查。如果失败，远程设备不能执行这命令，以 NOT_AUTHORIZED 响应。

2.4.3.3.6 Mgmt_Direct_Join_req

Mgmt_Direct_Join_req 命令 (ClusterID=0x0035) 应按图 2.58 所示的格式编排。

字节数: 8	1
Device Address	Capability Information

图 2.58 Mgmt_Direct_Join_req 命令帧的格式

表 2.84 指明了 Mgmt_Direct_Join_req 命令帧的域。

表 2.84 Mgmt_Direct_Join_req 命令的域

名称	类型	有效范围	描述
DeviceAddress	Device Address	扩展64位，IEEE地址	见3.2.2.14小节的NLME-JOIN.request的DeviceAddress参数的详细描述
CapabilityInformation	Bitmap	见表3.47	被直接加入的设备操作能力

2.4.3.3.6.1 原语生成

Mgmt_Direct_Join_req 从一个本地设备生成，请求一个远程设备允许 DeviceAddress 指定的一个设备直接加入网络。Mgmt_Direct_Join_req 由一个管理应用程序生成，并把该请求发给一个远程设备，它使用 Mgmt_Direct_Join_req 提供的参数执行 NLME-DIRECT-JOIN.request。

2.4.3.3.6.2 接收结果

在接收时，远程设备应使用 Mgmt_Direct_Join_req 命令提供的 DeviceAddress 和 CapabilityInformation 参数发出 NLME-DIRECT-JOIN.request 原语。直接尝试加入的结果应通过 Mgmt_Direct_Join_rsp 命令报告给本地设备。

如果远程设备不支持该可选的管理请求，它应返回一个 NOT_SUPPORTED 状态。如果直接尝试加入执行成功，Mgmt_Direct_Join_rsp 命令应包含一个 SUCCESS 状态。如果直接尝试加入没有执行成功，Mgmt_Direct_Join_rsp 命令应包含 NLME-DIRECT-JOIN.confirm 原语报告的错误代码。

本命令必须经过 4.6.3.8 小节定义的一个允许检查。如果失败，远程设备不能执行本命令，以 NOT_AUTHORIZED 响应。

2.4.3.3.7 Mgmt_Permit_Joining_req

Mgmt_Permit_Joining_req 命令（ClusterID=0x0036）应按图 2.59 所示的格式编排。

字节数：1	1
PermitDuration	TC_Significance

图 2.59 Mgmt_Permit_Joining_req 命令帧的格式

表 2.85 指明了 Mgmt_Permit_Joining_req 命令帧的域。

表 2.85 Mgmt_Permit_Joining_req 命令的域

名称	类型	有效范围	描述
PermitDuration	Integer	0x00 - 0xff	见3.2.2.5小节的NLMEPERMIT-JOINING.request的PermitDuration参数的详细描述
TC_Significance	Boolean Integer	0x00 - 0x01	如果设置为0x01并且远程设备是信任中心，这个命令将按照下面小节所述影响信任中心的认证政策；如果设置为0x00，对信任中心没有影响

2.4.3.3.7.1 原语生成

Mgmt_Permit_Joining_req 从一个本地设备生成，请求一个远程设备或设备允许或禁止连接。Mgmt_Permit_Joining_req 由一个管理应用程序或调试工具生成，并把该请求发给一个（或多个）远程设备，它使用 Mgmt_Permit_Joining_req 提供的 PermitDuration 参数执行 NLME-PERMIT-JOINING.request。另外，如果远程设备是信任中心，且 TC_Significance 设置为 1，信任中心的认证策略将受到影响。寻址可以是单播或广播给所有的路由器和协调器。

2.4.3.3.7.2 接收结果

在接收时,远程设备应使用 Mgmt_Permit_Joining_req 命令提供的 PermitDuration 参数发出 NLME-PERMIT-JOINING.request 原语。如果 PermitDuration 参数不等于零或 0xFF, 且参数是若干秒, 则允许加入, 直到倒计时至 0, 这个时间过去之后, 就不允许加入。如果 PermitDuration 参数设置为 0, 则禁止加入。如果设置为 0xFF, 则无限期地允许加入, 或直到接收到另一个 Mgmt_Permit_Joining_req。如果收到第二个 Mgmt_Permit_Joining_req, 而前一个仍在倒计时, 它将取代前一个请求。另外, 如果远程设备是信任中心, 且 TC_Significance 为 1, 信任中心认证将受到影响。在这种情况下, 如果 PermitDuration 设置为一个非零值, 信任中心认证被允许, 这样当信任中心收到一个 NLME-JOIN.indication 或一个 APSME-UPDATE-DEVICE.indication, 表示一个新的设备已经加入, 它将对它进行验证, 并发出一个适当的安全密钥。或者, 如果 PermitDuration 设置为 0, 如果收到一个 APSME-UPDATE-DEVICE.indication, 表示一个新的设备已经加入, 信任中心的验证功能将被禁止; 然后信任中心应发出一个 APSME-REMOVE-DEVICE.request 来拒绝这个新设备。注意 TC_Significance 标志和信任中心行为将服从协议栈 profile 施加的安全策略。尤其是信任中心的这个标志可以配置为不采取行动, 除非它已经被送往特别受信任的设备, 比如信任中心已知的配置工具。同样, 主要命令功能可以在某些协议栈 profile 下禁止, 除非发送设备可以被验证为一个已知的配置设备。如果该命令是不允许的, 应发送一个 INVALID_REQUEST 响应 (除非该命令是广播给所有路由器和协调器的, 这种情况下不应发送响应)。如果 Mgmt_Permit_Joining_req 原语作为单播信息被接收, NLME-PERMIT-JOINING.request 的结果应通过 Mgmt_Permit_Joining_rsp 命令报告回本地设备。如果该命令作为广播信息被接收, 则不应送回响应。

本命令必须经过 4.6.3.8 小节定义的一个允许检查。如果失败, 远程设备不能执行本命令, 以 NOT_AUTHORIZED 响应。如果 Mgmt_Permit_Joining_req 作为一个单播接收, 则不予响应。

2.4.3.3.8 Mgmt_Cache_req

Mgmt_Cache_req 命令 (ClusterID=0x0037) 应按图 2.60 所示的格式编排。

字节数 : 1
StartIndex

图 2.60 Mgmt_Cache_req 命令帧的域

表 2.86 指明了 Mgmt_Cache_req 命令帧的域。

表 2.86 Mgmt_Cache_req 命令的域

名称	类型	有效范围	描述
StartIndex	Integer	0x00 - 0xff	发现缓存表被请求的元素的起始索引

2.4.3.3.8.1 原语生成

提供 Mgmt_Cache_req, 以使网络中的 ZigBee 设备检索注册在一个主要发现缓存设备中的 ZigBee 终端设备列表。该原语的目标寻址应是单播的。

2.4.3.3.8.2 接收结果

在接收时, 远程设备应确定它是否是一个主要发现缓存, 或是否支持这个可选的请求原语。如果它不是一个主要发现缓存设备, 或不支持 Mgmt_Cache_req 原语, 远程设备应返回一个 NOT_SUPPORTED 状态。如果远程设备是一个主要发现缓存, 且支持 Mgmt_Cache_req, 远程设备应给本地设备返回 SUCCESS, 以及每个注册的 ZigBee 终端设备的发现缓存列表, 由 NWKAddr 和 IEEEAddr 组成。

2.4.3.3.9 Mgmt_NWK_Update_req

Mgmt_NWK_Update_req 命令(ClusterID=0x0038)格式如图 2.61。

字节: 4	1	0/1	0/1	0/2
ScanChannels	ScanDuration	ScanCount	<i>nwkUpdateId</i>	<i>nwkManagerAddr</i>

图 2.61 Mgmt_NWK_Update_req 命令帧的域

表 2.87 指定了 Mgmt_NWK_Update_req 命令帧的域。

表 2.87 Mgmt_NWK_Update_req 命令帧的域

名称	类型	有效范围	描述
ScanChannels	Bitmap	32 位域	详见（3.2.2.1 小节）NLME-ED-SCAN.request 的 ScanChannels 参数
ScanDuration	Integer	0x00-0x05 or 0xff	用来计算扫描每个信道所花时间的一个值。每个信道扫描所花的时间是(aBaseSuperframeDuration * (2n + 1))符号数，n是 ScanDuration参数的值。MAC层扫描更多信息见[B1]。 如果ScanDuration值为0xfe，这是更换信道的一个请求。 如果ScanDuration值为0xff，这是改变 <code>apsChannelMask</code> 和 <code>nwkManagerAddr</code> 参数的一个请求。
ScanCount	Integer	0x00 - 0x05	本域表示将被实施或者报告的能量扫描次数。 仅当 ScanDuration 在 0x00 到 0x05 之间，本域才存在。
<code>nwkUpdateId</code>	Integer	0x00 - 0xFF	这个请求所含的 <code>nwkUpdateId</code> 的值。本值在发送信息之前由网络信道管理器设定。 仅当 ScanDuration 等于 0xfe 或 0xff 时，本域才存在。
<code>nwkManagerAddr</code>	Device Address	16 位网络地址	仅当 ScanDuration 等于 0xff 时，本域才存在。如果存在，指示具有网络管理器位的设备的网络地址在它的节点描述符中设置。

2.4.3.3.9.1 原语生成

提供该命令，以允许更新网络的配置参数，或在本地运行环境下请求网络设备的信息。本原语目标寻址应单播或广播到所有 `macRxOnWhenIdle = TRUE` 的设备。

2.4.3.3.9.2 接收结果

在接收时，远程设备应该从 `ScanDuration` 参数的内容决定这个请求是否是 `apsChannelMask` 和 `nwkManagerAddr` 参数的更新，一个信道更新命令，或扫描信道并报告结果的一个请求。

如果 `ScanDuration` 参数等于 `0xfe`，该命令提供一个新的活动信道作为在 `ChannelMask` 中的单个信道，在此情况下 APS IB 没有更新，但是更新信道的程序已经完成。

如果 `ScanDuration` 参数等于 `0xff`，该命令提供一组新 `apsChannelMask`，带有一个新的 `nwkManagerAddr`。远程设备应该保存 `apsChannelMask` 到 APS IB 中，`nwkManagerAddr` 到 NIB 中，且不调用一个 NLME-ED-SCAN.request。

如果这个命令是单播，且 `ScanDuration` 设定为 `0xfe` 或者 `0xff`，远程设备不能响应。在这种情况下网络管理器会请求一个 APS 确认。

如果 `ScanDuration` 参数等于 `0x00` 到 `0x05`，本命令目标寻址是单播，那么本命令被解释为通过调用 NLME-ED-SCAN.request，使用 `ScanDuration` 和 `ScanCount` 参数，扫描 `ChannelMask` 中描述的信道的请求。如果远程设备不支持分段传输，由此产生的结果会超过 APDU，远程设备应该在尽可能多适合一个 APDU 的被请求信道上，执行能量检测扫描，在响应参数中突出实际扫描的信道列表。如果 `ScanCount` 请求多个扫描，每个扫描应该报告为一个单独的结果。远程设备会采用一个使用请求参数的能量扫描，由分段所述的限制进行修改，用一个 `Mgmt_NWK_Update_notify` 把结果提供给请求设备，状态为 SUCCESS。

否则如果 `ScanDuration` 等于 `0x06` 到 `0xfd`，本命令目标寻址为单播，那么远程设备会返回状态值 INVALID_REQUEST。

如果本命令目标寻址不是单播，那么远程设备不能传输一个响应。

2.4.4 服务器服务

设备 Profile 服务器服务支持设备和服务发现请求，终端设备绑定请求，绑定请求，取消绑定请求和网络管理请求的程序。另外，服务器服务支持传输这些响应返回到请求设备。

对于发给服务器的所有的广播地址请求（任意一种广播地址类型），如果不支持该命令，服务器应丢掉该数据包。对于任何广播寻址的客户端请求，错误状态不应单播返回给本地设备，包括但不限于服务器不支持的请求。

对于发给服务器的所有的单播地址请求，如果不支持该命令，服务器应制定一个只包含响应 Cluster ID 和状态域的响应数据包。响应 Cluster ID 应通过取请求的 Cluster ID，以及设置高阶位来创建。状态域应设置为 NOT_SUPPORTED。由此产生的响应应该单播给请求的客户端。

2.4.4.1 设备和服务发现服务器

表 2.88 列出了设备和服务发现服务器服务设备 profile 支持的命令。这些命令将在以下分节一一讨论。为了接收 Device_Annce 命令，服务器应为请求所提供的 IEEE 和 16 位 NWK 地址检查所有内部接口。对于本地设备所有的 IEEE 地址，应该替换 Device_Annce 所提供的相应的 NWK 地址。对于任何其它本地设备的 NWK 地址，相应的条目应标记为没有一个已知的有效 16 位 NWK 地址。服务器不应给 Device_Annce 提供响应。

表 2.88 设备和服务发现服务器服务原语

设备和服务发现服务器服务	服务器处理
NWK_addr_rsp	M
IEEE_addr_rsp	M
Node_Desc_rsp	M
Power_Desc_rsp	M
Simple_Desc_rsp	M
Active_EP_rsp	M
Match_Desc_rsp	M

表 2.88 设备和服务发现服务器服务原语（续表）

设备和服务发现服务器服务	服务器处理
Complex_Desc_rsp	O
User_Desc_rsp	O
User_Desc_conf	O
System_Server_Discovery_rsp	M
Discovery_store_rsp	O
Node_Desc_store_rsp	O
Power_Desc_store_rsp	O
Active_EP_store_rsp	O
Simple_Desc_store_rsp	O
Remove_node_cache_rsp	O
Find_node_cache_rsp	O
Extended_Simple_Desc_rsp	O
Extended_Active_EP_rsp	O

2.4.4.1.1 NWK_addr_rsp

NWK_addr_rsp 命令（ClusterID=0x8000）应按图 2.62 所示的格式编排。

字节数:1	8	2	0/1	0/1	可变的
Status	IEEEAddr RemoteDev	NWKAddr RemoteDev	Num AssocDev	StartIndex	NWKAddr AssocDevList

图 2.62 NWK_addr_rsp 命令帧的格式

表 2.89 指明了 NWK_addr_rsp 命令帧的域。

表 2.89 NWK_addr_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INV_REQUESTTYPE或 DEVICE_NOT_FOUND	NWK_addr_req 命令 的状态
IEEEAddrRemoteDev	Device Address	扩展的64位，IEEE地址	远程设备的64位地址

表 2.89 NWK_addr_rsp 命令的域（续表）

名称	类型	有效范围	描述
NWKAddrRemoteDev	Device Addresses	16位，网络地址	远程设备的16位地址
NumAssocDev	Integer	0x00-0xff	对16位短地址进行计数。 如果 请求的RequestType 是扩展响应，并且远程设备上没有相关设备，该域应该被设置为0。 如果发生错误或请求的RequestType是单个设备响应，该域不应该包含在帧中
StartIndex	Integer	0x00-0xff	这个报告的相关设备列表里的起始索引。 如果请求的RequestType是扩展请求，并且远程设备上没有相关设备，该域不应该包括在这个帧中。 如果发生错误或请求的RequestType是单个设备响应，该域不应该包括在这个帧中。

表 2.89 NWK_addr_rsp 命令的域（续表）

名称	类型	有效范围	描述
NWKAddrAssocDevList	设备地址列表	NumAssocDev1 6位短地址列表， 每个值的范围为 0x0000 0x-ffff	16位地址列表，一个对应于远程设备的每个相关设备；该域所含的16位网络地址个数指定在NumAssocDev域中。 如果请求的RequestType是扩展响应，并且远程设备没有相关设备，该域不应该包括在这个帧中。 如果发生错误或请求的RequestType是单个设备响应，该域不应该包括在这个帧中。

2.4.4.1.1.1 原语生成

NWK_addr_rsp 从一个远程设备生成，以响应一个 NWK_addr_req，后者查询远程设备的 NWK 地址或存储在一个本地发现缓存中的 NWK 地址（详细描述见 2.4.3.1.1.2 小节）。该命令的目标寻址是单播的。

2.4.4.1.1.2 接收结果

在接收 NWK_addr_rsp 时，接收者被告知其尝试从一个 IEEE 地址发现一个 NWK 地址的状态，或被告知一个错误。如果收到状态为 SUCCESS 的 NWK_addr_rsp 命令，根据原始 NWK_Addr_req 命令指明的 RequestType，该命令剩余的域包含合适的发现信息。否则，状态域表示错误，且不能包括 NumAssocDev、StartIndex 和 NWKAddrAssocDevList 域。

2.4.4.1.2 IEEE_addr_rsp

IEEE_addr_rsp 命令（ClusterID=0x8001）应按图 2.63 所示的格式编排。

字节数: 1	8	2	0/1	0/1	可变的
Status	IEEEAddr RemoteDev	NWKAddr RemoteDev	Num AssocDev	StartIndex	NWKAddr AssocDevList

图 2.63 IEEE_addr_rs 命令帧的格式

表 2.90 指明了 IEEE_addr_rsp 命令帧的域。

表 2.90 IEEE_addr_rsp 参数

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INV_REQUESTTYPE或 DEVICE_NOT_FOUND	IEEE_addr_req命令 的状态
IEEEAddrRemoteDev	Device Address	扩展的64位,IEEE地址	远程设备的64位地址
NWKAddrRemoteDev	Device Address	16位, 网络地址	远程设备的16位地址
NumAssocDev	Integer	0x00-0xff	计数16位短地址 的个数。 如果请求的 RequestType是扩展 响应, 并且远程设备 没有相关设备, 该域 应该被设置为0。 如果发生错误或 请求的RequestType 是单个设备响应, 该 域不应该包含在帧 中。

表 2.90 IEEE_addr_rsp 参数(续表)

名称	类型	有效范围	描述
StartIndex	Integer	0x00-0xff	这个报告的相关设备列表的起始索引。 如果请求的RequestType是扩展响应，并且远程设备上没有相关设备，该域不应该包含在帧中。 如果发生错误或请求的RequestType是单个设备响应，该域不应该包含在帧中。
NWKAddrAssocDevList	Device Address List	NumAssocDev 16位短地址列表，每个值的范围为 0x0000 0x-ffff	16位地址列表，一个对应于远程设备的每个相关设备；该域所含的16位网络地址个数由NumAssocDev域指定。 如果请求的RequestType是扩展响应，并且远程设备上没有相关设备，该域不应该包含在帧中。 如果发生错误或请求的RequestType是单个设备响应，该域不应该包含在帧中。

2.4.4.1.2.1 原语生成

IEEE_addr_rsp 从远程设备生成，以响应询问远程设备的 IEEE 地址，或存储在一个本地发现缓存中的 64 位 IEEE 地址的一个 IEEE_addr_req（详细描述见 2.4.3.1.2.2 小节）。该命令的目标寻址应是单播的。

2.4.4.1.2.2 接收结果

在接收 IEEE_addr_rsp 命令时,接收者被告知其尝试从一个 NWK 地址发现一个 IEEE 地址的状态,或被告知一个错误。如果收到状态为 SUCCESS 的 IEEE_addr_rsp 命令,根据原始 IEEE_Addr_req 命令指明的 RequestType,该命令的剩余域包含合适的发现信息。否则,状态域表示错误,且不能包括 NumAssocDev、StartIndex 和 NWKAddrAssocDevList 域。

2.4.4.1.3 Node_Desc_rsp

Node_Desc_rsp 命令 (ClusterID=0x8002) 应按如 2.64 所示的格式编排。

字节数: 1	2	见 2.3.2.4小节
Status	NWKAddr OfInterest	Node Descriptor

图 2.64 Node_Desc_rsp 命令帧的格式

表 2.91 指明了 Node_Desc_rsp 命令帧的域。

表 2.91 Node_Desc_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, DEVICE_NOT_FOUND ,INV_REQUESTTYPE 或NO_DESCRIPTOR	Node_Desc_req命令的状态
NWKAddr OfInterest	Device Address	16位网络地址	请求的网络地址
Node Descriptor	Node Descriptor		见2.3.2.3小节中节点描述符的 格式。 仅当状态域等于SUCCESS,该 域才包含在帧中

2.4.4.1.3.1 原语生成

Node_Desc_rsp 由一个远程设备生成,以响应一个发给该远程设备的 Node_Desc_req。该命令应单播给 Node_Desc_req 命令的发起者。

远程设备应使用表 2.91 所示的格式生成 Node_Desc_rsp 命令。NWKAddrOfInterest 域应匹配原始 Node_Desc_req 命令指定的值。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，它应把 Status 域设置为 SUCCESS，且在 NodeDescriptor 域中包含其节点描述符（参见 2.3.2.3 小节）。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，它应设置 Status 域为 INV_REQUESTTYPE，不包含 NodeDescriptor 域。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或路由器，它应确定 NWKAddrOfInterest 域匹配的网络地址是否是其子节点之一。如果 NWKAddrOfInterest 域不匹配远程设备子节点之一的网络地址，它应设置 Status 域为 DEVICE_NOT_FOUND，且不包含 NodeDescriptor 域。如果 NWKAddrOfInterest 域匹配远程设备子节点之一的网络地址，它应确定该设备的一个节点描述符是否可以使用。如果 NWKAddrOfInterest 域指明的子节点的一个节点描述符不可以使用，远程设备应设置 Status 域为 NO_DESCRIPTOR，且不包含 NodeDescriptor 域。如果 NWKAddrOfInterest 域指明的子节点的一个节点描述符可以使用，远程设备应设置 Status 域为 SUCCESS，且在 NodeDescriptor 域中包含匹配子节点设备的节点描述符（参见 2.3.2.3 小节）。

2.4.4.1.3.2 接收结果

在接收 Node_Desc_rsp 命令时，接收者被告知原始的 Node_Desc_req 命令所指的远程设备的节点描述符，或被告知一个错误。如果收到一个 Status 为 SUCCESS 的 Node_Desc_rsp 命令，NodeDescriptor 域应包含被请求的节点描述符。否则，Status 域指示错误，且不应包含 NodeDescriptor 域。

2.4.4.1.4 Power_Desc_rsp

Power_Desc_rsp 命令（ClusterID=0x8003）应按图 2.65 所示的格式编排。

位： 1	2	可变的
Status	NWKAddr OfInterest	Power Descriptor

图 2.65 Power_Desc_rsp 命令帧的格式

表 2.92 指明了 Power_Desc_rsp 命令帧的域。

表 2.92 Power_Desc_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, DEVICE_NOT_FOUND, INV_REQUESTTYPE或 NO_DESCRIPTOR	Power_Desc_req命令的状态
NWKAddrOfInterest	Device Address	16位网络地址	请求的网络地址
PowerDescriptor	Power Descriptor		见2.3.2.4小节中节点电源标识符的格式。 仅当状态域等于 SUCCESS, 该域才包含在 帧中。

2.4.4.1.4.1 原语生成

Power_Desc_rsp 由一个远程设备生成，以响应一个发给该远程设备的 Power_Desc_req。该命令应单播给 Power_Desc_req 命令的发起者。

远程设备应使用表 2.92 所示的格式生成 Power_Desc_rsp 命令。NWKAddrOfInterest 域应匹配原始的 Power_Desc_req 命令指明的值。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，它应设置 Status 域为 SUCCESS，且在 PowerDescriptor 域中包含其电源描述符（参见 2.3.2.4 小节）。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，它应设置 Status 域为 INV_REQUESTTYPE，且不包含 PowerDescriptor 域。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或路由器，它应确定 NWKAddrOfInterest 域匹配的网络地址是否是其子节点之一。如果 NWKAddrOfInterest 域不匹配远程设备子节点之一的网络地址，它应设置 Status 域为 DEVICE_NOT_FOUND，且不包含 PowerDescriptor 域。如果 NWKAddrOfInterest 域匹配远程设备子节点之一的网络地址，它应确定该设备的一个电源描述符是否可以使用。如果 NWKAddrOfInterest 域指明的子节点的一个电源描述符不可以使用，远程设备应设置 Status 域为 NO_DESCRIPTOR，且不包含 PowerDescriptor 域。如果 NWKAddrOfInterest 域指明的子节点的一个电源描述符可以使用，远程设备应设置 Status 域为 SUCCESS，且在 PowerDescriptor 域中包含匹配子节点设备的电源描述符（参见 2.3.2.4 小节）。

2.4.4.1.4.2 接收结果

在接收 Power_Desc_rsp 命令时，接收者被告知原始的 Power_Desc_req 命令所指的远程设备的电源描述符，或被告知一个错误。如果收到一个 Status 值为 SUCCESS 的 Power_Desc_rsp 命令，PowerDescriptor 域应包含被请求的电源描述符。否则，Status 域指示错误，且不应包含 PowerDescriptor 域。

2.4.4.1.5 Simple_Desc_rsp

Simple_Desc_rsp 命令（ClusterID=0x8004）应按图 2.66 所示的格式编排。

位: 1	2	1	可变的
Status	NWKAddr OfInterest	Length	Simple Descriptor

图 2.66 Simple_Desc_rsp 命令帧的格式

表 2.93 指明了 Simple_Desc_rsp 命令帧的域。

表 2.93 Simple_Desc_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INVALID_EP, NOT_ACTIVE, DEVICE_NOT_FOUND, INV_REQUESTTYPE或 NO_DESCRIPTOR	Simple_Desc_req命令的状态
NWKAddrOfInterest	Device Address	16位网络地址	请求的网络地址
Length	Integer	0x00-0xff	简单描述符遵循的字节的长度
SimpleDescriptor	Simple Descriptor		见2.3.2.5小节中简单描述符的格式。 仅当状态域等于SUCCESS, 该域才包含在帧中

2.4.4.1.5.1 原语生成

Simple_Desc_rsp 由一个远程设备生成，以响应一个发给该远程设备的 Simple_Desc_req。该命令应单播给 Simple_Desc_req 命令的发起者。

远程设备应使用表 2.93 所示的格式生成 Simple_Desc_rsp 命令。NWKAddrOfInterest 域应匹配原始的 Simple_Desc_req 命令指明的值。如果原始 Simple_Desc_req 命令指明的端点域不在表 2.48 指定的正确范围内，远程设备应设置 Status 域为 INVALID_EP，设置 Length 域为 0，且不包含 SimpleDescriptor 域。

如果 NWKAddrOfInterest 域匹配远程设备的网络地址，它应确定端点域是否指明了设备上一个活动端点的标识符。如果端点域和一个活动端点对应，远程设备应设置 Status 域为 SUCCESS，设置 Length 域为该端点简单描述符的长度，且在 SimpleDescriptor 域中包含该端点的简单描述符（参见 2.3.2.5 小节）。如果端点域和一个活动端点不对应，远程设备应设置 Status 域为 NOT_ACTIVE，设置 Length 域为 0，且不包含 SimpleDescriptor 域。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，它应设置 Status 域为 INV_REQUESTTYPE，设置 Length 域为 0，且不包含 SimpleDescriptor 域。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或路由器，它应确定 NWKAddrOfInterest 域匹配的网络地址是否是其子节点之一。如果 NWKAddrOfInterest 域不匹配远程设备子节点之一的网络地址，它应设置 Status 域为 DEVICE_NOT_FOUND，设置 Length 域为 0，且不包含 SimpleDescriptor 域。

如果 NWKAddrOfInterest 域匹配远程设备子节点之一的网络地址，它应确定该设备以及被请求端点的一个简单描述符是否可以使用。如果 NWKAddrOfInterest 域所指子节点的被请求端点的一个简单描述符不可以使用，远程设备应设置 Status 域为 NO_DESCRIPTOR，设置 Length 域为 0，且不包含 SimpleDescriptor 域。如果 NWKAddrOfInterest 域所指子节点的被请求端点的一个简单描述符可以使用，远程设备应设置 Status 域为 SUCCESS，设置 Length 域为该端点上简单描述符的长度，且在 SimpleDescriptor 域中包含匹配子节点设备的端点的简单描述符（参见 2.3.2.5 小节）。

2.4.4.1.5.2 接收结果

在接收 Simple_Desc_rsp 命令时,接收者被告知原始 Simple_Desc_req 命令所指的远程设备端点的简单描述符,或被告知一个错误。如果收到一个 Status 值为 SUCCESS 的 Simple_Desc_rsp 命令, SimpleDescriptor 域应包含被请求的简单描述符。否则,Status 域指示错误,且不应包含 SimpleDescriptor 域。

2.4.4.1.6 Active_EP_rsp

Active_EP_rsp 命令 (ClusterID=0x8005) 应按图 2.67 所示的格式编排。

字节: 1	2	1	可变的
Status	NWKAddr OfInterest	ActiveEP Count	ActiveEP List

图 2.67 Active_EP_rsp 命令帧的格式

表 2.94 指明了 Active_EP_rsp 命令帧的域。

表 2.94 Active_EP_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, DEVICE_NOT_FOUND, INV_REQUESTTYPE 或NO_DESCRIPTOR	Active_EP_req命令的状态
NWKAddrOfInterest	Device Address	16位网络地址	请求的网络地址
ActiveEPCount	Integer	0x00-0xff	远程设备上活动端点的个数
ActiveEPList			字节的列表,每个代表一个8位端点

2.4.4.1.6.1 原语生成

Active_EP_rsp 由一个远程设备生成,以响应一个发给该远程设备的 Active_EP_req。该命令应单播给 Active_EP_req 命令的发起者。

远程设备应使用表 2.94 所示的格式生成 Active_EP_rsp 命令。NWKAddrOfInterest 域应匹配原始 Active_EP_req 命令指定的值。如果 NWKAddrOfInterest 域匹配远程设备的网络地址,它应设置 Status 域为 SUCCESS,设置 ActiveEPCount 域为该设备上活动端点的个数,且在 ActiveEPList 域中包含该设备活动端点的所有标识符的一个升序列表。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，它应设置 Status 域为 INV_REQUESTTYPE，设置 ActiveEPList 域为 0，且不包含 ActiveEPList 域。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或路由器，它应确定 NWKAddrOfInterest 域是否匹配保存在一个发现缓存中的某设备的网络地址。如果 NWKAddrOfInterest 域不匹配保存在一个发现缓存中的设备的网络地址，它应设置 Status 域为 DEVICE_NOT_FOUND，设置 ActiveEPList 域为 0，且不包含 ActiveEPList 域。如果 NWKAddrOfInterest 匹配保存在远程设备一个发现缓存中的某设备的网络地址，它应确定该设备是否有任何活动的端点。如果和 ActiveEP 请求对应的发现信息还没有装载到发现缓存中，远程设备应设置 Status 域为 NO_DESCRIPTOR，设置 ActiveEPCount 域为 0，且不包含 ActiveEPList 域。如果被缓存的设备没有活动端点，远程设备应设置 Status 域为 SUCCESS，设置 ActiveEPCount 域为 0，且不包含 ActiveEPList 域。如果被缓存的设备有活动端点，远程设备应设置 Status 域为 SUCCESS，设置 ActiveEPCount 域为该设备活动端点的个数，且在 ActiveEPList 域中包含该设备活动端点的所有标识符的一个升序列表。

2.4.4.1.6.2 接收结果

在接收 Active_EP_rsp 时，接收者被告知原始 Active_EP_req 命令所指的设备的活动端点，或被告知一个错误。如果收到一个 Status 值为 SUCCESS 的 Active_EP_rsp 命令，ActiveEPCount 域表示 ActiveEPList 域中的条目个数。否则，Status 域表示错误，且不应包含 ActiveEPList 域。

2.4.4.1.7 Match_Desc_rsp

Match_Desc_rsp 命令（ClusterID=0x8006）应按图 2.68 所示的格式编排。

字节: 1	2	1	可变的
Status	NWKAddr OfInterest	Match Length	Match List

图 2.68 Match_Desc_rsp 命令帧的格式

表 2.95 指明了 Match_Desc_rsp 命令帧的域。

表 2.95 Match_Desc_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, DEVICE_NOT_FOUND, INV_REQUESTTYPE或 NO_DESCRIPTOR	Match_Desc_req命令的状态
NWKAddrOfInterest	设备地址	16位网络地址	请求的网络地址
MatchLength	Integer	0x00-0xff	匹配请求标准的远程设备的 端点的个数
MatchList			字节的列表，每个代表一个8 位的端点

2.4.4.1.7.1 原语生成

Match_Desc_rsp 由一个远程设备生成，以响应一个广播的或直接给该远程设备的 Match_Desc_req。该命令应单播给 Match_Desc_req 命令的发起者。

远程设备应使用表 2.95 所示的格式生成 Match_Desc_rsp 命令。如果原始 Match_Desc_req 的 NWKAddrOfInterest 域等于所有 macRxOnWhenIdle = TRUE 设备的广播网络地址 (0xffffd)，远程设备应使用匹配标准，如下所述，它指定在原始 Match_Desc_req 命令中，发给其每个简单描述符。如果远程设备是一个协调器或路由器，它也应使用匹配标准，如下所述，发给每个简单描述符，它可以从其每个子节点获得。

如果原始 Match_Desc_req 的 NWKAddrOfInterest 域不等于所有 macRxOnWhenIdle = TRUE 设备的广播网络地址 (0xffffd)，远程设备应把 NWKAddrOfInterest 域设置为和原始 Match_Desc_req 命令指定的网络地址相同的值。

如果 NWKAddrOfInterest 域匹配远程设备的网络地址，它应使用匹配标准，如下所述，它指定在原始 Match_Desc_req 命令中，发给其每个简单描述符。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，它应设置 Status 域为 INV_REQUESTTYPE，设置 MatchLength 域为 0，且不包含 MatchList 域。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或路由器，它应确定 NWKAddrOfInterest 域匹配的网络地址是否为其子节点之一。如果 NWKAddrOfInterest 域不匹配远程设备子节点之一的网络地址，它应设置 Status 域为 DEVICE_NOT_FOUND，设置 MatchLength 域为 0，且不包含 MatchList 域。

如果 NWKAddrOfInterest 域匹配远程设备子节点之一的网络地址，它应确定该设备的任何简单描述符是否可以使用。如果 NWKAddrOfInterest 域指定的子节点没有可以使用的简单描述符，远程设备应设置 Status 域为 NO_DESCRIPTOR，设置 MatchLength 域为 0，且不包含 MatchList 域。如果 NWKAddrOfInterest 域指定的子节点有可以使用的简单描述符，远程设备应使用匹配标准，如下所述，它指定在原始 Match_Desc_req 命令中，发给所有这些简单描述符。

远程设备应为每个简单描述符使用匹配标准（参见 2.3.2.5 小节），如下。远程设备应首先检查 ProfileID 域是否匹配简单描述符的应用 profile 标识符域。如果 profile 标识符不匹配，远程设备应假定匹配不成功，不再执行进一步的匹配。

如果 profile 标识符匹配，远程设备应确定匹配标准是否包含一个输入 cluster 列表（NumInClusters 域不等于 0）。如果匹配标准包含一个输入 cluster 列表，远程设备应至少检查 InClusterList 域中所列的 cluster 标识符之一，匹配简单描述符的应用输入 cluster 列表域中的 cluster 标识符之一。如果找到至少一个输入 cluster 匹配，远程设备应假定匹配成功，注意简单描述符指的是端点标识符，不再执行进一步的匹配。

如果远程设备不能找到输入 cluster 的任何匹配，它应确定匹配标准是否包含一个输出 cluster 列表（NumOutClusters 域不等于 0）。如果匹配标准包含一个输出 cluster 列表，远程设备应至少检查 OutClusterList 域中所列的 cluster 标识符之一，匹配简单描述符的应用输出 cluster 列表域中的 cluster 标识符之一。如果找到至少一个输出 cluster 匹配，远程设备应假定匹配成功，注意简单描述符指的是端点标识符。如果远程设备不能找到输出 cluster 的任何匹配，它应假定不成功。

如果以上程序产生一个或多个匹配，远程设备应为每个匹配设备（包括它本身）构造一个单独的 Match_Desc_rsp 命令。对于每个响应，Status 域应设置为 SUCCESS，NWKAddrOfInterest 域应设置为匹配设备的地址，MatchLength 域应设置为符合匹配设备标准的简单描述符的个数，MatchList 域应包含一个端点的升序列表，它们是符合匹配设备标准的一个简单描述符所在的端点。

2.4.4.1.7.2 接收结果

在接收 Match_Desc_rsp 命令时，接收者被告知原始 Match_Desc_req 命令所指的匹配标准查询的结果，或被告知一个错误。如果收到一个 Status 值为 SUCCESS 的 Match_Desc_rsp 命令，MatchList 域应包括含有匹配该标准的简单描述符的端点列表。否则，Status 域指示错误，且不应包含 MatchList 域。

2.4.4.1.8 Complex_Desc_rsp

Complex_Desc_rsp 命令（ClusterID=0x8010）应按图 2.69 所示的格式编排。

字节数: 1	2	1	可变的
Status	NWKAddr OfInterest	Length	Complex Descriptor

图 2.69 Complex_Desc_rsp 命令帧的格式

表 2.96 指明了 Complex_Desc_rsp 命令帧的域。

表 2.96 Complex_Desc_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, DEVICE_NOT_FOUND, INV_REQUESTTYPE或 NO_DESCRIPTOR	Complex_Desc_req命令的状态
NWKAddrOfInterest	Device Address	16位网路地址	请求的网络地址
Length	Integer	0x00-0xff	ComplexDescriptor域的字节长度
ComplexDescriptor	Complex Descriptor		见2.3.2.6小节复杂描述符的格式。仅当状态域等于SUCCESS，该域才包含在帧中。

2.4.4.1.8.1 原语生成

Complex_Desc_rsp 从一个远程设备生成，以响应一个发给远程设备的 Complex_Desc_req。该命令应单播给 Complex_Desc_req 命令的发起者。

远程设备应使用表 2.96 所示的格式生成 Complex_Desc_rsp 命令。NWKAddrOfInterest 域应匹配原始 Complex_Desc_req 命令指定的值。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，但是不存在一个复杂描述符，它应设置 Status 域为 NOT_SUPPORTED，设置 Length 域为 0，且不包含 ComplexDescriptor 域。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，且存在一个复杂描述符，它应设置 Status 域为 SUCCESS，设置 Length 域为复杂描述符的长度，且在 ComplexDescriptor 域中包含其复杂描述符（参见 2.3.2.6 小节）。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，它应设置 Status 域为 INV_REQUESTTYPE，设置 Length 域为 0，且不包含 ComplexDescriptor 域。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或路由器，它应确定 NWKAddrOfInterest 域匹配的网络地址是否是其子节点之一。如果 NWKAddrOfInterest 域不匹配远程设备子节点之一的网络地址，它应设置 Status 域为 DEVICE_NOT_FOUND，设置 Length 域为 0，且不包含 ComplexDescriptor 域。如果 NWKAddrOfInterest 域匹配远程设备子节点之一的网络地址，它应确定该设备的一个复杂描述符是否可以使用。如果 NWKAddrOfInterest 域指定的子节点的一个复杂描述符不可以使用，远程设备应设置 Status 域为 NO_DESCRIPTOR，设置 Length 域为 0，且不包含 ComplexDescriptor 域。如果 NWKAddrOfInterest 域指定的子节点的一个复杂描述符可以使用，远程设备应设置 Status 域为 SUCCESS，设置 Length 域为该设备复杂描述符的长度，且在 ComplexDescriptor 域中包含匹配子节点设备的复杂描述符（参见 2.3.2.6 小节）。

2.4.4.1.8.2 接收结果

在接收 Complex_Desc_rsp 命令时，接收者被告知原始 Complex_Desc_req 命令所指的远程设备的复杂描述符，或被告知一个错误。如果收到一个 Status 值为 SUCCESS 的 Complex_Desc_rsp 命令，ComplexDescriptor 域应包含被请求的复杂描述符。否则，Status 域指示错误，且不应包含 ComplexDescriptor 域。

2.4.4.1.9 User_Desc_rsp

User_Desc_rsp 命令（ClusterID=0x8011）应按图 2.70 所示的格式编排。

字节: 1	2	1	可变的
Status	NWKAddr OfInterest	Length	User Descriptor

图 2.70 User_Desc_rsp 命令帧的格式

表 2.97 指明了 User_Desc_rsp 命令帧的域。

表 2.97 User_Desc_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, DEVICE_NOT_FOUND, INV_REQUESTTYPE或 NO_DESCRIPTOR	User_Desc_req命令的状态
NWKAddrOfInterest	设备地址	16位网络地址	请求的网络地址
Length	Integer	0x00-0x10	UserDescriptor域的字节长度
UserDescriptor	用户描述符		见2.3.2.7小节的用户描述符格式。仅当状态域等于SUCCESS，该域才包含在帧中。

2.4.4.1.9.1 原语生成

User_Desc_rsp 从一个远程设备生成，以响应一个发给该远程设备的 User_Desc_req，该命令应单播给 User_Desc_req 命令的发起者。

远程设备应使用表 2.97 所示的格式生成 User_Desc_rsp 命令。NWKAddrOfInterest 域应匹配原始 User_Desc_req 命令指定的值。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，但是不存在一个用户描述符，它应设置 Status 域为 NO_DESCRIPTOR，设置 Length 域为 0，且不包含 UserDescriptor 域。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，且存在一个用户描述符，它应设置 Status 域为 SUCCESS，设置 Length 域为用户描述符的长度，且在 UserDescriptor 域中包含其用户描述符（参见 2.3.2.7 小节）。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，它应设置 Status 域为 INV_REQUESTTYPE，设置 Length 域为 0，且不包含 UserDescriptor 域。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或路由器，它应确定 NWKAddrOfInterest 域匹配的网络地址是否是其子节点之一。如果 NWKAddrOfInterest 域不匹配远程设备子节点之一的网络地址，它应设置 Status 域为 DEVICE_NOT_FOUND，设置 Length 域为 0，且不包含 UserDescriptor 域。如果 NWKAddrOfInterest 域匹配远程设备子节点之一的网络地址，它应确定该设备的一个用户描述符是否可以使用。如果 NWKAddrOfInterest 域指定的子节点的一个用户描述符不可以使用，远程设备应设置 Status 域为 NO_DESCRIPTOR，设置 Length 域为 0，且不包含 UserDescriptor 域。如果 NWKAddrOfInterest 域指定的子节点的一个用户描述符可以使用，远程设备应设置 Status 域为 SUCCESS，设置 Length 域为该设备用户描述符的长度，且在 UserDescriptor 域中包含匹配子节点设备的用户描述符（参见 2.3.2.7 小节）。

2.4.4.1.9.2 接收结果

在接收 User_Desc_rsp 命令时，接收者被告知原始 User_Desc_req 命令所指的远程设备的用户描述符，或被告知一个错误。如果收到一个 Status 值为 SUCCESS 的 User_Desc_rsp 命令，UserDescriptor 域应包含被请求的用户描述符。否则，Status 域指示错误，且不应包含 UserDescriptor 域。

2.4.4.1.10 System_Server_Discovery_rsp

System_Server_Discovery_rsp 命令（ClusterID=0x8015）应按图 2.74 所示的格式编排。

字节: 1	2
Status	ServerMask

图 2.71 System_Server_Discovery_rsp 命令帧

表 2.98 指明了 System_Server_Discovery_rsp 命令帧的域。

表 2.98 System_Server_Discovery_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS	System_Server_Discovery_rsp命令的状态
ServerMask	Integer	Bitmap	见表2.31中位的分配

2.4.4.1.10.1 原语生成

System_Server_Discovery_rsp 从一个远程设备生成，在收到一个 System_Server_Discovery_req 原语时，如果参数匹配其节点描述符的服务器掩码域时生成。如果没有匹配，System_Server_Discovery_req 应被忽略，不作出响应。匹配是通过掩盖 System_Server_Discovery_req 的 ServerMask 参数执行的，在节点描述符中有服务器掩码域。该命令应单播给发送 System_Server_Discovery_req 的设备，且在 TxOptions 中设置了确认请求。ServerMask 参数包含该请求中匹配节点描述符的服务器掩码的参数位。

2.4.4.1.10.2 接收结果

请求设备被告知这个设备具有请求设备寻找的若干系统服务器功能。

如果在 System_Server_Discovery_rsp 中设置了网络管理器位，那么远程设备的 NWK 地址应设置在 *nwkManagerAddr*NIB 中。

2.4.4.1.11 User_Desc_conf

User_Desc_conf 命令（ClusterID=0x8014）应按图 2.72 所示的格式编排。

字节数: 1	2
Status	NWKAddr OfInterest

图 2.72 User_Desc_conf 命令帧的格式

表 2.99 指明了 User_Desc_conf 命令帧的域。

表 2.99 User_Desc_conf 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, DEVICE_NOT_FOUND, INV_REQUESTTYPE或 NO_DESCRIPTOR	User_Desc_set命令的状态
NWKAddrOfInterest	设备地址	任何16位网络地址	用户描述符尝试设置的设备的网络地址

2.4.4.1.11.1 原语生成

User_Desc_conf 由一个远程设备生成，以响应一个发给该远程设备的 User_Desc_set。该命令应单播给 User_Desc_set 命令的发起者。

远程设备应使用表 2.99 所示的格式生成 User_Desc_conf 命令。NWKAddrOfInterest 域应匹配原始 User_Desc_set 命令指定的值。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，但是不存在一个用户描述符，它应设置 Status 域为 NOT_SUPPORTED。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，且存在一个用户描述符，它应设置 Status 域为 SUCCESS，且设置用户描述符为原始 User_Desc_set 命令指定的 ASCII 字符串。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，它应设置 Status 域为 INV_REQUESTTYPE。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或路由器，它应确定 NWKAddrOfInterest 域匹配的网络地址是否是其子节点之一。如果 NWKAddrOfInterest 域不匹配远程设备子节点之一的网络地址，它应设置 Status 域为 DEVICE_NOT_FOUND。如果 NWKAddrOfInterest 域匹配远程设备子节点之一的网络地址，它应确定该设备的一个用户描述符是否可以使用。如果 NWKAddrOfInterest 域指定的子节点的一个用户描述符不可以使用，远程设备应设置 Status 域为 NO_DESCRIPTOR。如果 NWKAddrOfInterest 域指定的子节点的一个用户描述符可以使用，远程设备应设置 Status 域为 SUCCESS，且把用户描述符配置为原始 User_Desc_set 命令指定的 ASCII 字符串。

2.4.4.1.11.2 接收结果

本地设备被告知其尝试在一个远程设备上配置用户描述符的结果。

2.4.4.1.12 Discovery_Cache_rsp

Discovery_Cache_rsp 命令（ClusterID=0x8012）应按图 2.73 所示的格式编排。

字节数: 1
Status

图 2.73 Discovery_Cache_rsp 命令帧的格式

表 2.100 指明了 Discovery_Cache_rsp 命令帧的域。

表 2.100 Discovery_Cache_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS	Discovery_Cache_req命令的状态

2.4.4.1.12.1 原语生成

Discovery_Cache_rsp 由一个收到 Discovery_Cache_req 的主要发现缓存设备生成。不是主要发现缓存设备的远程设备（在其节点描述符中指定）不能响应 Discovery_Cache_req 命令。

2.4.4.1.12.2 接收结果

在接收 Discovery_Cache_rsp 时，本地设备确定是否返回了一个 SUCCESS 状态。如果从原始 Discovery_Cache_req 命令没有返回 Discovery_Cache_rsp 信息，那么本地设备会扩大发送给本地主要发现缓存设备的请求的范围，超过之前请求提供的范围。如果返回一个 SUCCESS 状态，本地设备应使用 Discovery_Store_req，针对提供响应的远程设备，以确定是否有足够的发现缓存存储空间。

2.4.4.1.13 Discovery_store_rsp

Discovery_store_rsp 命令（ClusterID=0x8016）应按图 2.74 所示的格式编排。

字节数: 1
Status

图 2.74 Discovery_store_rsp 命令帧的格式

表 2.101 指明了 Discovery_store_rsp 命令帧的域。

表 2.101 Discovery_store_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INSUFFICIENT_SPACE 或NOT_SUPPORTED	Discovery_store_req命令的状态

2.4.4.1.13.1 原语生成

提供 Discovery_store_rsp，以把一个主要发现缓存设备的请求的状态通知给一个本地设备。该响应应包含一个状态代码，以通知本地设备该请求是否成功（主要缓存设备有空间存储本地设备的发现缓存），是否支持该请求（意味着远程设备不是一个主要发现缓存设备）或是否有足够的空间。

2.4.4.1.13.2 接收结果

在接收时，本地设备应确定响应状态是否表示了远程设备不是一个主要发现缓存设备，如 NOT_SUPPORTED 状态所指示。如果返回一个 NOT_SUPPORTED 状态，本地设备应处理任何从其它远程设备来的其它 Discovery_store_rsp 设备，或重新执行 Discovery_Cache_req 以确定另一个主要发现缓存设备的地址（如果它再次响应 Discovery_Cache_req，消除以 NOT_SUPPORTED 响应的远程设备的地址）。如果返回一个 INSUFFICIENT_SPACE 状态，本地设备也应处理任何其它 Discovery_store_rsp，且如果没有响应表示 SUCCESS，则重新执行 Discovery_Cache_req（范围域应扩大以包含更多的远程设备）。如果返回一个 SUCCESS 状态，本地设备应通过 Node_Desc_store_req，Power_Desc_store_req，Active_EP_store_req 和 Simple_Desc_store_req，把其发现缓存信息装载到远程设备。

2.4.4.1.14 Node_Desc_store_rsp

Node_Desc_store_rsp 命令（ClusterID=0x8017）应按图 2.75 所示的格式编排。

字节数: 1
Status

图 2.75 Node_Desc_store_rsp 命令帧的格式

表 2.102 指明了 Node_Desc_store_rsp 命令帧的域。

表 2.102 Node_Desc_store_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INSUFFICIENT_SPACE 或NOT_SUPPORTED	Node_store_rsp命令的状态

2.4.4.1.14.1 原语生成

提供 Node_Desc_store_rsp，以把一个主要发现缓存设备的请求的状态通知给一个本地设备。该响应应包含一个状态代码，以通知本地设备该请求是否成功（主要缓存设备有空间存储本地设备的发现缓存数据），是否支持该请求（意味着远程设备不是一个主要发现缓存设备）或是否有足够的空间。

2.4.4.1.14.2 接收结果

在接收时，本地设备应确定响应状态是否表示了远程设备不是一个主要缓存设备，如 NOT_SUPPORTED 状态所表示。如果返回一个 NOT_SUPPORTED 状态，本地设备应重新执行主要发现缓存设备的发现。如果返回一个 NOT_PERMITTED 状态，本地设备必须首先发出一个带有返回的 SUCCESS 状态的 Discovery_store_req。如果返回一个 INSUFFICIENT_SPACE 状态，本地设备也应给发送给远程设备一个 Remove_node_cache_req。如果返回一个 SUCCESS 状态，本地设备应通过 Power_Desc_store_req，Active_EP_store_req 和 Simple_Desc_store_req，继续把其剩余的发现缓存信息装载到远程设备。

2.4.4.1.15 Power_Desc_store_rsp

Power_Desc_store_rsp 命令 (ClusterID=0x8018) 应按图 2.76 所示的格式编排。

字节数: 1	8	可变的
Status	IEEEAddr	PowerDescriptor

图 2.76 Power_Desc_store_rsp 命令帧的格式

表 2.103 指明了 Power_Desc_store_rsp 命令帧的域。

表 2.103 Power_Desc_store_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INSUFFICIENT_SPACE 或NOT_SUPPORTED	Power_store_rsp命令的状态

2.4.4.1.15.1 原语生成

提供 Power_Desc_store_rsp, 以把一个主要发现缓存设备的请求的状态通知给一个本地设备。该响应包含一个状态代码, 以通知本地设备该请求是否成功 (主要缓存设备有空间存储本地设备的发现缓存数据), 是否支持该请求 (意味着远程设备不是一个主要发现缓存设备) 或是否有足够的空间。

2.4.4.1.15.2 接收结果

在接收时, 本地设备应确定响应状态是否表示了远程设备不是一个主要缓存设备, 如 NOT_SUPPORTED 状态所表示。如果返回一个 NOT_SUPPORTED 状态, 本地设备应在主要发现缓存设备上重新执行发现。如果返回一个 NOT_PERMITTED 状态, 本地设备必须首先发出一个带有返回的 SUCCESS 状态的 Discovery_store_req。如果返回一个 INSUFFICIENT_SPACE 状态, 本地设备应停止发现信息的装载, 发出一个 Remove_node_cache_req (引用本地设备), 并停止尝试装载发现信息到远程设备。

如果返回一个 SUCCESS 状态, 本地设备应通过 Active_EP_store_req 和 Simple_Desc_store_req, 继续把其剩余的发现缓存信息装载到远程设备。

2.4.4.1.16 Active_EP_store_rsp

Active_EP_store_rsp 命令（ClusterID=0x8019）应按图 2.77 所示的格式编排。

字节数：1
Status

图 2.77 Active_EP_store_rsp 命令帧的格式

表 2.104 指明了 Active_EP_store_rsp 命令帧的域。

表 2.104 Active_EP_store_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INSUFFICIENT_SPACE 或NOT_SUPPORTED	Active_EP_store_rsp 命令的状态

2.4.4.1.16.1 原语生成

提供 Active_EP_store_rsp，以把一个主要发现缓存设备的请求的状态通知给一个本地设备。该响应应包含一个状态代码，以通知本地设备该请求是否成功（主要缓存设备有空间存储本地设备的发现缓存数据），是否支持该请求（意味着远程设备不是一个主要发现缓存设备）或是否有足够的空间。

2.4.4.1.16.2 接收结果

在接收时，本地设备应确定响应状态是否表示了远程设备不是一个主要缓存设备，如 NOT_SUPPORTED 状态所表示。如果返回一个 NOT_SUPPORTED 状态，本地设备应在主要发现缓存设备上重新执行发现。如果返回一个 NOT_PERMITTED 状态，本地设备必须首先发出一个带有返回的 SUCCESS 状态的 Discovery_store_req。如果返回一个 INSUFFICIENT_SPACE 状态，本地设备应停止发现信息的装载，发出一个 Remove_node_cache_req（引用本地设备），并停止尝试装载发现信息到远程设备。如果返回一个 SUCCESS 状态，本地设备应通过 Simple_Desc_store_req，继续把其剩余的发现缓存信息装载到远程设备。

2.4.4.1.17 Simple_Desc_store_rsp

Simple_Desc_store_rsp 命令 (ClusterID=0x801a) 应按图 2.78 所示的格式编排。

字节数: 1
Status

图 2.78 Simple_Desc_store_rsp 命令帧的格式

表 2.105 指明了 Simple_Desc_store_rsp 命令帧的域。

表 2.105 Simple_Desc_store_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INSUFFICIENT_SPACE 或NOT_SUPPORTED	Simple_desc_store_rsp 命令的状态

2.4.4.1.17.1 原语生成

提供 Simple_Desc_store_rsp, 以把一个主要发现缓存设备的请求的状态通知给一个本地设备。该响应包含一个状态代码, 以通知本地设备该请求是否成功 (主要缓存设备有空间存储本地设备的发现缓存数据), 是否支持该请求 (意味着远程设备不是一个主要发现缓存设备) 或是否有足够的空间。

2.4.4.1.17.2 接收结果

在接收时, 本地设备应确定响应状态是否表示了远程设备不是一个主要缓存设备, 如 NOT_SUPPORTED 状态所表示。如果返回一个 NOT_SUPPORTED 状态, 本地设备应在主要发现缓存设备上重新执行发现。如果返回一个 NOT_PERMITTED 状态, 本地设备必须首先发出一个带有返回的 SUCCESS 状态的 Discovery_store_req。如果返回一个 INSUFFICIENT_SPACE 状态, 本地设备应停止发现信息的装载, 发出一个 Remove_node_cache_req (引用本地设备), 并停止尝试装载发现信息到远程设备。如果返回一个 SUCCESS 状态, 本地设备应通过到本地设备的其它端点的 Simple_Desc_store_req, 继续把其剩余的发现缓存信息装载到远程设备。

2.4.4.1.18 Remove_node_cache_rsp

Remove_node_cache_rsp 命令（ClusterID=0x801b）应按图 2.79 所示的格式编排。

字节数: 1
Status

图 2.79 Remove_node_cache_rsp 命令帧的格式

表 2.106 指明了 Remove_node_cache_rsp 命令帧的域。

表 2.106 Remove_node_cache_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, DEVICE_NOT_FOUND 或NOT_SUPPORTED	Remove_node_cache_rsp命令的状态

2.4.4.1.18.1 原语生成

提供 Remove_node_cache_rsp，以把一个主要发现缓存设备的请求的状态通知给一个本地设备。该响应包含一个状态节点，以通知本地设备该请求是否成功（主要缓存设备已经从指定的重要设备中删除了发现缓存数据）或是否支持该请求（意味着远程设备不是一个主要发现缓存设备）。

2.4.4.1.18.2 接收结果

在接收时，本地设备应确定响应状态是否表示了远程设备不是一个主要缓存设备，如 NOT_SUPPORTED 状态所表示。如果返回一个 NOT_SUPPORTED 状态，本地设备应重新执行 Find_node_cache_req，以找到持有指定的重要设备的发现缓存信息的主要发现缓存设备。当找到了持有指定的重要设备的发现缓存信息的主要发现缓存设备，本地设备应重复 Remove_node_cache_req，以成功删除发现信息。如果收到一个 DEVICE_NOT_FOUND 状态，这表示远程设备是主要发现缓存，但不持有该请求所指的 NWKAddr 和 IEEEAddr 的发现信息。本地设备应使用设备发现命令 NWK_Addr_req 和 IEEE_Addr_req，以确定 NWKAddr 和 IEEEAddr 的正确值。如果返回一个 SUCCESS 状态，本地设备为该请求指定的重要设备成功删除了发现缓存信息。

2.4.4.1.19 Find_node_cache_rsp

Find_node_cache_rsp 命令（ClusterID=0x801c）应按图 2.80 所示的格式编排。

字节数: 2	2	8
CacheNWKAddress	NWKAddr	IEEEAddr

图 2.80 Find_node_cache_rsp 命令帧的格式

表 2.107 指明了 Find_node_cache_rsp 命令帧的域。

表 2.107 Find_node_cache_rsp 命令的域

名称	类型	有效范围	描述
CacheNWKAddr	Device Address	16位网络地址	带有发现信息的主要发现缓存设备的网络地址（或者直接响应了请求的重要设备）
NWKAddr	Device Address	16位网络地址	重要设备的网络地址
IEEEAddr	Device Address	64位IEEE地址	重要设备的IEEE地址

2.4.4.1.19.1 原语生成

提供 Find_node_cache_rsp，以通知一个本地设备，根据请求提供的给定 NWKAddr 和 IEEEAddr 域，成功发现了该主要发现缓存设备，或意味着重要设备能够响应发现请求。Find_node_cache_rsp 应只能由一个主要发现缓存设备生成，且它持有请求中的 NWKAddr 和 IEEEAddr 的发现信息，或重要设备本身，且所有其它远程设备不能提供响应。

2.4.4.1.19.2 接收结果

在接收时，本地设备应使用 CacheNWKAddr 作为远程设备地址，为了以后和响应中的 NWKAddr 和 IEEEAddr 有关的发现请求。

2.4.4.1.20 Extended_Simple_Desc_rsp

Extended_Simple_Desc_rsp 命令 (ClusterID=0x801c) 应按图 2.81 所示的格式编排。

字节: 1	2	1	1	1	1	可变
Status	NWKAddr OfInterest	Endpoint	AppInput ClusterCount	AppOutput ClusterCount	StartIndex	AppCluster List

图 2.81 Extended_Simple_Desc_rsp 命令帧的格式

表 2.108 指明了 Extended_Simple_Desc_rsp 命令帧的域。

表 2.108 Extended_Simple_Desc_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INVALID_EP, NOT_ACTIVE, DEVICE_NOT_FOUND, INV_REQUESTTYPE or NO_DESCRIPTOR	Extended_Simple_Desc_req 命令的状态
NWKAddrOfInterest	Device Address	16 位 NWK 地址	请求的网络地址
Endpoint	8 位	1-240	目标上的端点
AppInputClusterCou nt	8 位	0x00-0xff	本端点上简单描述符的应用 输入 cluster 的总数

表 2.108 Extended_Simple_Desc_rsp 命令的域（续表）

名称	类型	有效范围	描述
AppOutputClusterCount	8 位	0x00-0xff	本端点上简单描述符的应用输出 cluster 的总数
StartIndex	8 位	0x00-0xff	响应的 AppClusterList 中的起始索引， AppClusterList 由这个端点的简单描述符的应用输入 cluster 列表和应用输出 cluster 列表的一个有序列表表示。
AppClusterList			AppInputClusterList 和 AppOutputClusterList 的一个串联的、有序列表，以简单描述符的 StartIndex 开始。仅当该状态域等于 SUCCESS 时，该域才能包括在帧中。

2.4.4.1.20.1 原语产生

Extended_Simple_Desc_rsp 由远程设备响应 Extended_Simple_Desc_req 产生，本命令应单播给 Extended_Simple_Desc_req 命令的发起者。

远程设备使用如表 2.108 的格式产生 Extended_Simple_Desc_rsp 命令。NWKAddrOfInterest 域应该匹配原始 Extended_Simple_Desc_req 命令指定的值。如果 Extended_Simple_Desc_req 命令端点域不在表 2.48 的正确范围之内，远程设备应该设置 Status 为 INVALID_EP，Endpoint 和 StartIndex 设置为请求的各自的值，且不包含在 AppClusterList 域内。

如果 NWKAddrOfInterest 域匹配远程设备的网络地址，它应该决定端点域是否指明设备上活动端点的识别符。如果端点域和活动端点对应，远程设备应该设置 Status 成 SUCCESS，设置 AppClusterList 域为从简单描述符获得的串联的 AppInputClusterList 和 AppOutputClusterList 字节序列（表 2.38），并提供 AppClusterList 的值作为响应。注意这取决于请求中 StartIndex 的值，AppClusterList 的值可能为空（例如，StartIndex 在串联的 AppInputClusterList 和 AppOutputClusterList 字节序列给出后开始）。如果端点域和一个活动端点不对应，远程设备应该设置 Status 域为 NOT_ACTIVE，StartIndex 域设置为请求提供的值，且不包含在 AppClusterList 内。

2.4.4.1.20.2 接收结果

在接收到 Extended_Simple_Desc_rsp 命令之后，接收者被通知原始 Extended_Simple_Desc_req 命令指定的远程设备的端点的被请求的 AppClusterList，或被通知一个错误。如果收到 Status 为 SUCCESS 的 Extended_Simple_Desc_rsp 命令，AppClusterList 域应包含应用输入 cluster 列表和应用输出 cluster 列表被请求的部分，以 StartIndex 开始。否则，状态域显示错误，不应包含 AppClusterList 域。

2.4.4.1.21 Extended_Active_EP_rsp

Extended_Active_EP_rsp 命令（ClusterID=0x801e）应按图 2.82 所示的格式编排。

字节: 1	2	1	1	可变
Status	NWKAddr OfInterest	ActiveEP Count	StartIndex	ActiveEP List

图 2.82 Extended_Active_EP_rsp 命令帧格式

表 2.109 指明了 Extended_Active_EP_rsp 命令帧的域。

表 2.109 Extended_Active_EP_rsp 命令帧的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, DEVICE_NOT_FOUND , INV_REQUESTTYPE 或 NO_DESCRIPTOR	Extended_Active_EP_req 命令的状态
NWKAddrOfInterest	Device Address	16 位网络地址	请求的网络地址

表 2.109 Extended_Active_EP_rsp 命令帧的域（续表）

名称	类型	有效范围	描述
ActiveEPCount	Integer	0x00-0xff	远程设备上活动端点计数
StartIndex	Integer	0x00-0xff	该报告的活动端点的起始索引
ActiveEPList			字节列表，每个代表一个 8 位端点。列表以 StartIndex 开始，持续直到余下的活动端点列出，或者整个端点条目耗尽 ASDU 的大小。

2.4.4.1.21.1 原语产生

Extended_Active_EP_rsp 由一个远程设备产生，以响应发给远程设备的一个 Extended_Active_EP_req。本命令应单播给 Extended_Active_EP_req 命令的发起者。

远程设备使用如表 2.109 的格式产生 Extended_Active_EP_rsp 命令。NWKAddrOfInterest 域应该匹配原始 Extended_Active_EP_req 命令指定的值。如果 NWKAddrOfInterest 域匹配远程设备的网络地址，远程设备应该设置 Status 为 SUCCESS，ActiveEPCount 域设置为设备上活动端点数，且包括活动端点所有标识符的一个升序列表，以 StartIndex 开始，在设备上的 ActiveEPList 域，持续直到余下的活动端点列出，或者整个端点条目耗尽 ASDU 的大小。

如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个终端设备，远程设备应该设置 Status 为 INV_REQUESTTYPE，设置 ActiveEPCount 域为 0，且不包含在 ActiveEPList 内。如果 NWKAddrOfInterest 域不匹配远程设备的网络地址，且它是一个协调器或者路由器，它应该确定 NWKAddrOfInterest 域是否匹配发现缓存中存储的一个设备的网络地址。如果 NWKAddrOfInterest 域不匹配发现缓存中存储的一个设备的网络地址，应该设置 Status 域为 DEVICE_NOT_FOUND，设置 ActiveEPCount 域为 0，且不包含 ActiveEPList 域。如果 NWKAddrOfInterest 域匹配远程设备上发现缓存中存储的一个设备的网络地址，它应确定设备是否有任何活动端点。如果和 ActiveEP 请求对应的发现信息尚未被装载到发现缓存，远程设备应该设置 Status 域为 NO_DESCRIPTOR，设置 ActiveEPCount 域为 0，且不包含 ActiveEPList 域。如果被缓存的设备没有活动端点，远程设备应该设置 Status 域为 SUCCESS，ActiveEPCount 域设置为 0，且不包含 ActiveEPList 域。如果被缓存的设备有活动端点，远程设备应该设置 Status 域为 SUCCESS，ActiveEPCount 域设置为设备活动端点数，且包含一个活动端点所有标识符的升序列表，以 StartIndex 开始，在设备上的 ActiveEPList 域内。

2.4.4.1.21.2 接收结果

在接收到 Extended_Active_EP_rsp 命令后，接收者被告知原始 Extended_Simple_Desc_req 命令指定的远程设备的活动端点，或被告知一个错误。如果收到 Status 为 SUCCESS 的 Extended_Simple_Desc_rsp 命令，ActiveEPCount 域应指示 ActiveEPList 域的条目数。否则，状态域显示错误，且不应包含 AppClusterList 域。请求设备需要多次使用具有不同 StartIndex 值的 Extended_Active_EP_req，以从远程设备上接收完整的 ActiveEPList。

2.4.4.2 终端设备绑定，绑定，取消绑定管理服务服务器服务

表 2.110 列出了设备 Profile 支持的命令：终端设备绑定，绑定和取消绑定服务器服务。这些原语将在以下分节中一一讨论。

表 2.110 终端设备绑定，取消绑定和绑定管理服务服务器服务原语

终端设备绑定，取消绑定和绑定管理服务服务器服务	服务器处理
End_Device_Bind_rsp	O
Bind_rsp	O
Unbind_rsp	O
Bind_Register_rsp	O
Replace_Device_rsp	O
Store_Bkup_Bind_Entry_rsp	O
Remove_Bkup_Bind_Entry_rsp	O
Backup_Bind_Table_rsp	O

表 2.110 终端设备绑定，取消绑定和绑定管理服务器服务原语

终端设备绑定，取消绑定和绑定管理服务器服务	服务器处理
Recover_Bind_Table_rsp	O
Backup_Source_Bind_rsp	O
Recover_Source_Bind_rsp	O

2.4.4.2.1 End_Device_Bind_rsp

End_Device_Bind_rsp 命令（ClusterID=0x8020）应按图 2.83 所示的格式编排。

字节数: 1
Status

图 2.83 End_Device_Bind_rsp 命令帧的格式

表 2.111 指明了 End_Device_Bind_rsp 命令帧的域。

表 2.111 End_Device_Bind_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, INVALID_EP, TIMEOUT或 NO_MATCH	End_Device_Bind_req命令的状态

2.4.4.2.1.1 原语生成

End_Device_Bind_rsp 由 ZigBee 协调器生成，以响应一个 End_Device_Bind_req，包含该请求的状态。该命令应单播给和绑定尝试有关的每个设备，使用确认数据服务。

NOT_SUPPORTED 的状态表示该请求发给了一个不是 ZigBee 协调器的设备，或不支持终端设备绑定的 ZigBee 协调器。否则，End_Device_Bind_req 的处理按照下面所述执行，包括 End_Device_Bind_rsp 的传输。

2.4.4.2.1.2 接收结果

当收到一个 End_Device_Bind_req 时，如之前的章节所述，确定是否保证了一个 NOT_SUPPORTED 状态。假定该设备是 ZigBee 协调器，应检查所提供的端点，确定它是否在指定的范围内。如果不是，应返回一个 INVALID_EP 状态。如果所提供的端点在指定范围内，且如果是第一个被提交评价的 End_Device_Bind_req，它应被存储，且启动一个计时器，在预先配置的超时时间值之后过期。这个超时时间值应是 ZigBee 协调器上的一个配置项。如果该计时器在收到第二个 End_Device_Bind_req 之前过期，返回一个 TIMEOUT 状态。否则，如果在超时窗口内收到第二个 End_Device_Bind_req，则比较这两个 End_Device_Bind_req 以找到匹配。状态 NO_MATCH 表示这两个 End_Device_Bind_req 为了匹配而被评估，但是 ProfileID 参数不匹配，或 ProfileID 参数匹配，但 InClusterList 和 OutClusterList 的任何元素都不匹配。状态 SUCCESS 表示检测到一个匹配，且有一个由此产生的 Bind_req 将在随后发给提供 End_Device_Bind_req 的本地设备，它带有 OutClusterList 的匹配元素。

2.4.4.2.2 Bind_rsp

Bind_rsp 命令 (ClusterID=0x8021) 应按图 2.84 所示的格式编排。

字节数:1
Status

图 2.84 Bind_rsp 命令帧的格式

表 2.112 指明了 Bind_rsp 命令帧的域。

表 2.112 Bind_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, INVALID_EP或 TABLE_FULL	Bind_req命令 的状态

2.4.4.2.2.1 原语生成

生成 Bind_rsp, 以响应一个 Bind_req。如果 Bind_req 被处理, 且在远程设备上提交了绑定表条目, 返回一个 SUCCESS 状态。如果远程设备不是一个主要绑定表缓存或 SrcAddress, 返回一个 NOT_SUPPORTED 状态。应检查所提供的端点, 以确定它是否在指定的范围内。如果不是, 应返回一个 INVALID_EP 状态。如果远程设备是主要绑定表缓存或 SrcAddress, 但没有请求所需的绑定表源, 返回一个 TABLE_FULL 状态。

2.4.4.2.2.2 接收结果

在接收时，对该请求执行错误检查，如之前的章节所述。假定状态是 SUCCESS，Bind_req 的参数通过 APSME-BIND.request 原语，被送入远程设备的绑定表中。

2.4.4.2.3 Unbind_rsp

Unbind_rsp 命令（ClusterID=0x8022）应按图 2.85 所示的格式编排。

字节数: 1
Status

图 2.85 Unbind_rsp 命令帧的格式

表 2.113 指明了 Unbind_rsp 命令帧的域。

表 2.113 Unbind_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, INVALID_EP或 NO_ENTRY	Unbind_req命令 的状态

2.4.4.2.3.1 原语生成

生成 Unbind_rsp，以响应一个 Unbind_req。如果 Unbind_req 被处理，且从远程设备中删除了相应的绑定表条目，返回一个 SUCCESS 状态。如果远程设备不是 ZigBee 协调器或 SrcAddress，返回一个 NOT_SUPPORTED 状态。应检查所提供的端点，以确定它是否在指定的范围内。如果不是，应返回一个 INVALID_EP 状态。如果远程设备是 ZigBee 协调器或 SrcAddress，但是没有和收到的请求的参数相对应的一个绑定表条目，返回一个 NO_ENTRY 状态。

2.4.4.2.3.2 接收结果

在接收时，对该响应执行错误检查。如果状态是 SUCCESS，设备已经为 Unbind_req 中指定的参数成功删除了绑定表条目。

2.4.4.2.4 Bind_Register_rsp

Bind_Register_rsp 命令（ClusterID=0x8023）应按图 2.86 所示的格式编排。

字节数:1	2	2	可变的
Status	BindingTableEntries	BindingTableListCount	BindingTableList

图 2.86 Bind_Register_rsp 命令帧的格式

表 2.114 指明了 Bind_Register_rsp 命令帧的域。

表 2.114 Bind_Register_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, TABLE_FULL	Bind_Register_reg命令的状态
BindingTable Entries	Integer	0x0000 - 0ffff	请求设备通过主要绑定缓存存储的绑定表目录数
BindingTable ListCount	Integer	0x0000 - 0xffff	包含在这个响应中的源绑定表目录数
BindingTable List	List of source binding descriptors	这个列表应该包含 BindingTableListCount 给出的事件数	源绑定列表

2.4.4.2.4.1 原语生成

Bind_Register_rsp 从一个主要绑定表缓存设备生成，以响应一个 Bind_Register_req，且包含该请求的状态。该命令应单播给请求的设备。

如果收到 Bind_Register_req 的设备不是一个主要绑定表缓存，返回一个 NOT_SUPPORTED 状态。如果选择存储自己的绑定表条目设备的列表满了，返回一个 TABLE_FULL 状态。在这些错误的情况下，BindingTableEntries 和 BindingTableListCount 应为零，且 BindingTableList 应为空。SUCCESS 状态表示请求的设备已经被成功登记。

在成功的情况下，主要绑定表缓存设备应在其缓存中查询已存在的条目，其中源地址和 Bind_Register_req 命令所提供的相同。这样的条目的个数在响应的 BindingTableEntries 中给定。这些条目用于生成 BindingTableList，个数可多达响应可以包含的最大值。条目的实际个数在响应的 BindingTableListCount 中给定，如果太大可能小于 BindingTableEntries。在这种情况下（可以预计这种情况很少见）主要绑定表缓存设备应使用 Bind_req 命令把其余的条目发送给请求设备。

2.4.4.2.4.2 接收结果

请求设备被告知其尝试登记的结果。如果成功，它应把响应中的源绑定表条目存储到其源绑定表中。

2.4.4.2.5 Replace_Device_rsp

Replace_Device_rsp 命令（ClusterID=0x8024）应按图 2.87 所示的格式编排。

字节数:1
Status

图 2.87 Replace_Device_rsp 命令帧的格式

表 2.115 指明了 Replace_Device_rsp 命令帧的域。

表 2.115 Replace_Device_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	NOT_SUPPORTED, INV_REQUESTTYPE	Replace_Device_req 命令的状态

2.4.4.2.5.1 原语生成

Replace_Device_rsp 从一个主要绑定表缓存设备生成，以响应一个 Replace_Device_req，包含请求的状态。该命令应单播给请求的设备。如果收到 Replace_Device_req 的设备不是一个主要绑定表缓存，返回一个 NOT_SUPPORTED 状态。主要绑定表缓存应在其绑定表中查询源地址和源端点，或目标地址和目标端点匹配 OldAddress 和 OldEndpoint 的条目，如 Replace_Device_req 的文字所描述。它应改变这些条目，变成 NewAddress 和可能的 NewEndpoint。然后它应返回一个 SUCCESS 的响应。

2.4.4.2.5.2 接收结果

请求设备被告知其 Replace_Device_req 命令的状态。

2.4.4.2.6 Store_Bkup_Bind_Entry_rsp

Store_Bkup_Bind_Entry_rsp 命令 (ClusterID=0x8025) 应按图 2.88 所示的格式编排。

字节数:1
Status

图 2.88 Store_Bkup_Bind_Entry_rsp 命令帧的格式

表 2.116 指明了 Store_Bkup_Bind_Entry_rsp 命令帧的域。

表 2.116 Store_Bkup_Bind_Entry_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, INV_REQUESTTYPE. TABLE_FULL	Store_Bkup_Bind_Entry_rsp命令的状态

2.4.4.2.6.1 原语生成

Store_Bkup_Bind_Entry_rsp 从一个备份绑定表缓存设备生成，以响应一个主要绑定表缓存的一个 Store_Bkup_Bind_Entry_req，包含该请求的状态。该命令应单播给请求设备。如果远程设备不是一个备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果该请求的发起者不被承认为一个绑定表缓存，它应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应把绑定条目增加到其绑定表中，并返回一个 SUCCESS 状态。如果没有空间，它应返回一个 TABLE_FULL 状态。

2.4.4.2.6.2 接收结果

请求设备被告知其尝试存储一个绑定条目的状态。

2.4.4.2.7 Remove_Bkup_Bind_Entry_rsp

Remove_Bkup_Bind_Entry_rsp 命令 (ClusterID=0x8026) 应按图 2.89 所示的格式编排。

字节数:1
Status

图 2.89 Remove_Bkup_Bind_Entry_rsp 命令帧的格式

表 2.117 指明了 Remove_Bkup_Bind_Entry_rsp 命令帧的域。

表 2.117 Remove_Bkup_Bind_Entry_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, INV_REQUESTTYPE, NO_ENTRY	Remove_Bkup_Bind_Entry_rsp命令的状态

2.4.4.2.7.2 原语生成

Remove_Bkup_Bind_Entry_rsp 从一个备份绑定表缓存设备生成，以响应主要绑定表缓存的一个 Remove_Bkup_Bind_Entry_req，包含该请求的状态。该命令应单播给请求的设备。如果远程设备不是一个备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果该请求的发起者不被承认为一个主要绑定表缓存，它应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应从其绑定表中删除绑定条目，并返回一个 SUCCESS 状态。如果找不到该条目，它应返回一个 NO_ENTRY 状态。

2.4.4.2.7.2 接收结果

请求设备被告知其尝试从备份缓存中删除一个绑定条目的状态。

2.4.4.2.8 Backup_Bind_Table_rsp

Backup_Bind_Table_rsp 命令（ClusterID=0x8027）应按图 2.90 所示的格式编排。

字节数: 1	2
Status	EntryCount

图 2.90 Backup_Bind_Table_rsp 命令帧的格式

表 2.118 指明了 Backup_Bind_Table_rsp 命令帧的域。

表 2.118 Backup_Bind_Table_rsp 命令的域

名称	型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, TABLE_FULL, INV_REQUESTTYPE	Backup_Bind_Table_rsp 命令的状态
EntryCount	Integer	0x0000 - 0xFFFF	在备份绑定表中的目录 个数

2.4.4.2.8.1 原语生成

Backup_Bind_Table_rsp 从一个备份绑定表缓存生成，以响应一个主要绑定表缓存的一个 Backup_Bind_Table_req，包含该请求的状态。该命令应单播给请求的设备。如果远程设备不是一个备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果该请求的发起者不被承认为一个主要绑定表缓存，它应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应在其绑定表中重写绑定条目，以 StartIndex 开始，继续直到 BindingTableListCount 条目。如果这超过了该表的大小，它应填充尽可能的多的条目，并返回一个 TABLE_FULL 状态，且 EntryCount 参数会是表中条目的个数。否则它应返回一个 SUCCESS 状态，且 EntryCount 会等于 StartIndex +BindingTableListCount，来自 Backup_Bind_Table_req。

2.4.4.2.8.2 接收结果

请求设备被告知其尝试存储一个绑定表的状态。

2.4.4.2.9 Recover_Bind_Table_rsp

Recover_Bind_Table_rsp 命令（ClusterID=0x8028）应按图 2.91 所示的格式编排。

字节数: 1	2	2	2	可变的
Status	BindingTableEntries	StartIndex	BindingTableListCount	BindingTableList

图 2.91 Backup_Bind_Table_rsp 命令帧的格式

表 2.119 指明了 Recover_Bind_Table_rsp 命令帧的域。

表 2.119 Recover_Bind_Table_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, INV_REQUESTTYPE, NO_ENTRY	Recover_Bind_Table_rsp命令的状态
BindingTable Entries	Integer	0x0000 - 0xffff	在备份绑定缓存中绑定表目录的总数
StartIndex	Integer	0x0000 - 0xffff	绑定表中开始报告给绑定表列表的起始索引
BindingTable ListCount	Integer	0x0000 - 0xffff	包括在BindingTableList里的绑定目录数
BindingTable List	Integer	列表应该包含 BindingTableListCount 给出的元素个数	一个描述符列表，以StartIndex元素开始并且持续到备份绑定表缓存的BindingTableListCount元素。

2.4.4.2.9.1 原语生成

Recover_Bind_Table_rsp 从一个备份绑定表缓存生成，以响应一个主要绑定表缓存的一个 Recover_Bind_Table_req，包含该请求的状态。该命令应单播给请求的设备。如果响应设备不是一个备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果该请求的发起者不被承认为一个主要绑定表缓存，它应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应从其备份中准备一个绑定表条目的列表，以 StartIndex 开始。它将把尽可能多的条目装入一个 Recover_Bind_Table_rsp 命令，并返回一个 SUCCESS 状态。如果 StartIndex 多于绑定表中的条目数量，返回一个 NO_ENTRY 状态。对于一个成功的响应，BindingTableEntries 是备份绑定表中条目的总数，BindingTableListCount 是要在响应中返回的条目个数。

2.4.4.2.9.2 接收结果

请求设备被告知其尝试恢复一个绑定表的状态。

2.4.4.2.10 Backup_Source_Bind_rsp

Backup_Source_Bind_rsp 命令（ClusterID=0x8029）应按图 2.92 所示的格式编排。

字节数: 1
Status

图 2.92 Backup_Source_Bind_rsp 命令帧格式

表 2.120 指明了 Backup_Source_Bind_rsp 命令帧的域。

表 2.120 Backup_Source_Bind_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, TABLE_FULL, INV_REQUESTTYPE	Backup_Source_Bin d_rsp命令的状态

2.4.4.2.10.1 原语生成

Backup_Source_Bind_rsp 从一个备份绑定表缓存生成，以响应一个主要绑定表缓存的一个 Backup_Source_Bind_req，包含该请求的状态。该命令应单播给请求的设备。如果远程设备不是一个备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果该请求的发起者不被承认为一个主要绑定表缓存，它应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应重写其备份源绑定表，以 StartIndex 开始，继续直到 BindingTableListCount 条目。如果这超过了该表的大小，它应返回一个 TABLE_FULL 状态。否则，它应返回一个 SUCCESS 状态。

2.4.4.2.10.2 接收结果

请求设备被告知其尝试备份源绑定表的状态。

2.4.4.2.11 Recover_Source_Bind_rsp

Recover_Source_Bind_rsp 命令（ClusterID=0x802a）应按图 2.93 所示的格式编排。

字节数: 1	2	2	2	可变的
Status	SourceTableEntries	StartIndex	SourceTableListCount	SourceTableList

图 2.93 Recover_Source_Bind_rsp 命令帧的格式

表 2.121 指明了 Recover_Source_Bind_rsp 命令帧的域。

表 2.121 Recover_Source_Bind_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, NOT_SUPPORTED, TABLE_FULL, INV_REQUESTTYPE	Recover_Source_Bind_rsp命令的状态
SourceTableEntries	Integer	0x0000 - 0xffff	在备份绑定缓存中源表目录的总数
StartIndex	Integer	0x0000 - 0xffff	在源表中开始报告给源表列表的起始索引
SourceTableListCount	Integer	0x0000 - 0xffff	包含在SourceTableList中的源表目录数
SourceTableList	List of source descriptors	列表应该包含通过SourceTableListCount给的元素个数	一个描述符列表，以StartIndex元素开始，并且继续到备份源表缓存中的SourceTableListCount元素（包括IEEE地址）。

2.4.4.2.11.1 原语生成

Recover_Source_Bind_rsp 从一个备份绑定表缓存生成，以响应一个主要绑定表缓存的一个 Recover_Source_Bind_req，包含该请求的状态。该命令应单播给请求的设备。如果响应设备不是一个备份绑定表缓存，它应返回一个 NOT_SUPPORTED 状态。如果该请求的发起者不被承认为一个主要绑定表缓存，它应返回一个 INV_REQUESTTYPE 状态。否则，备份绑定表缓存应从其备份中准备一个绑定表条目的列表，以 StartIndex 开始。它将给一个 Recover_Source_Bind_rsp 命令装入尽可能多的条目，并返回一个 SUCCESS 状态。如果 StartIndex 多于绑定表中的条目数量，返回一个 NO_ENTRY 状态。对于一个成功的响应，SourceTableEntries 是备份源表中的条目总数，SourceTableListCount 是要在响应中返回的条目的个数。

2.4.4.2.11.2 接收结果

请求设备被告知其尝试恢复一个源绑定表的状态。

2.4.4.3 网络管理服务服务器服务

表 2.122 列出了设备 Profile 支持的命令：网络管理服务服务器服务。这些命令将在以下分节中一一讨论。

表 2.122 网络管理服务服务器服务命令

网络管理服务服务器服务命令	服务器处理
Mgmt_NWK_Disc_rsp	O
Mgmt_Lqi_rsp	O
Mgmt_Rtg_rsp	O
Mgmt_Bind_rsp	O
Mgmt_Leave_rsp	O
Mgmt_Direct_Join_rsp	O
Mgmt_Permit_Joining_rsp	M
Mgmt_Cache_rsp	O
Mgmt_NWK_Update_notify	O

2.4.4.3.1 Mgmt_NWK_Disc_rsp

Mgmt_NWK_Disc_rsp 命令 (ClusterID=0x8030) 应按图 2.94 所示的格式编排。

字节数: 1	1	1	1	可变的
Status	Network Count	Start Index	NetworkList Count	NetworkList

图 2.94 Mgmt_NWK_Disc_rsp 命令帧的格式

表 2.123 指明了 Mgmt_NWK_Disc_rsp 命令帧的域。

表 2.123 Mgmt_NWK_Disc_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	从NLME-NETWORKDISCOVERY.Request原语返回的 NOT_SUPPORTED 或者任何状态的代码	Mgmt_NWK_Disc_req命令的状态

表 2.123 Mgmt_NWK_Disc_rsp 命令的域

NetworkCount	Integer	0x00-0xff	通过 NLMENETWORKDISCOVERY.con firm.报告的网络总数
StartIndex	Integer	0x00-0xff	开始为这个响应报告的 NLMENETWORKDISCOVERY.con firm的NetworkList中的起始点
NetworkListCount	Integer	0x00-0xff	在这个响应里报告的网络列表描述 符的个数
NetworkList	List of Network Descriptors	列表应该包含通过 NetworkListCount参 数给出的元素的个 数	描述符列表，每个表示发现的一个 网络，以StartIndex元素开始并且持 续到NetworkListCount，通过the NLMENETWORKDISCOVERY.con firm 原语返回的元素。每个条目的 格式如表2.124。

表 2.124 NetworkList 记录格式

名称	大小(为)	有效范围	描述
ExtendedPanID	64	64位个域网标识符	发现网络的64位扩展个域网标识 符
LogicalChannel	8	选自PHY支持的可用的 逻辑信道(见[B1])	网络占有的目前逻辑信道
StackProfile	4	0x0-0xf	ZigBee栈profile标识符，代表在发 现网络中使用的栈profile
ZigBeeVersion	4	0x0-0xf	在发现网络中使用的ZigBee协议 的版本
BeaconOrder	4	0x0-0xf	这指定了MAC 子层信标通过网 络上一个特定设备传输的频率， 关于MAC 子层信标命令的讨论 请见[B1]

表 2.124 NetworkList 记录格式

SuperframeOrder	4	0x0-0xf	对于信标导向的网络，例如信标命令 <15，这指定超帧的活动周期的长度。MAC 子层超帧的命令的讨论见 [B1]。
PermitJoining	1	TRUE或FALSE	值为TRUE表示至少有一个ZigBee路由器在网络上目前允许加入，例如它的网络已经发出NLME-PERMIT-JOINING原语和时间限制，如果有，还没有到期。
Reserved	7		每个位都应该设置为0

2.4.4.3.1.1 原语生成

生成 Mgmt_NWK_Disc_rsp，以响应一个 Mgmt_NWK_Disc_req。如果不支持该管理命令，应返回一个 NOT_SUPPORTED 状态，且 Status 域之后的所有参数域都应被忽略。否则，远程设备应执行以下程序。

在接收时，已经证实了支持 Mgmt_NWK_Disc_req 之后，远程设备应使用 Mgmt_NWK_Disc_req 命令提供的 ScanChannels 和 ScanDuration 参数，发出一个 NLME-NETWORK-DISCOVERY.request 原语。在接收 NLME-NETWORK-DISCOVERY.confirm 原语时，远程设备应通过 Mgmt_NWK_Disc_rsp 命令报告结果，以 StartIndex 元素开始。NetworkList 域应包含整个 NetworkList 记录，按表 2.124 所示的格式排列，直到 MSDU 大小的限制，即达到 *aMaxMACFrameSize*（参见【B1】）。报告的结果个数应在 NetworkListCount 中设置。

2.4.4.3.1.2 接收结果

本地设备被告知其尝试执行一个远程网络发现的结果。



2.4.4.3.2 Mgmt_Lqi_rsp

Mgmt_Lqi_rsp 命令 (ClusterID=0x8031) 应按图 2.95 所示的格式编排。

字节数: 1	1	1	1	可变的
Status	NeighborTable Entries	Start Index	NeighborTable ListCount	NeighborTable List

图 2.95 Mgmt_Lqi_rsp 命令帧的格式

表 2.125 指明了 Mgmt_Lqi_rsp 命令帧的域。

表 2.125 Mgmt_Lqi_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	从NLMEGET. confirm 原语返回 的 NOT_SUPPORTED 或者任何状态代码	Mgmt_Lqi_req命令的 状态
NeighborTableEntries	Integer	0x00-0x02	远程设备的邻居表目 录的总数
StartIndex	Integer	0x00-0xff	邻居表中开始报告给 NeighborTableList. 的 起始索引
NeighborTableListCount	Integer	0x00-0xff	包括在 NeighborTableList内 的邻居表目录数
NeighborTableList	List of Neighbor Descriptors	这个列表应该包含 通过 NeighborTableListC ount给出的元素的 个数	描述符列表, 以 StartIndex开始并且持 续到远程设备的邻居 表的 NeighborTableListCou nt的元素, 包括设备地 址和相关LQI。(详见表 2.126)

表 2.126 NeighborTableList 记录格式

名称	大小(位)	有效范围	描述
Extended PAN Id	64	64位个域网标识符	邻居设备的64位扩展个域网标识符
Extended address	64	扩展64位, IEEE地址	每个设备唯一的64位IEEE地址。如果这个值在请求的时候不知道, 该域应该设置为0xffffffffffffff
Network address	16	网络地址	邻居设备的16位网络地址
Device type	2	0x0-0x3	邻居设备的类型: 0x0 = ZigBee协调器 0x1 = ZigBee 路由器 0x2 = ZigBee 终端设备 0x3 = 未知
RxOnWhenIdle	2	0x0-0x2	表示在CAP闲置期间邻居设备的接收器是否启用: 0x0 = 接收器是关闭的 0x1 = 接收器是打开的 0x2 = 未知
Relationship	3	0x0-0x4	当前设备和邻居设备的关系: 0x0 = 邻居设备是父设备 0x1 = 邻居设备是子设备 0x2 = 邻居设备是兄弟设备 0x3 = 以上都不是 0x4 =之前的子设备
Reserved	1		该保留位应该设置为0
Permit joining	2	0x0-0x2	表示邻居设备是否接受加入请求: 0x0 = 邻居设备不接受加入请求 0x1 =邻居设备接受加入请求 0x2 =未知

表 2.126 NeighborTableList 记录格式（续表）

名称	大小(位)	有效范围	描述
Reserved	6		每个保留位应该设置为0
Depth	8	0x00- <i>nwkMaxDepth</i>	邻居设备的树的深度。值0x00代表设备是网络的ZigBee协调器。
LQI	8	0x00-0xff	来自这个设备的RF传输的估计链路质量。见[B1]中关于如何计算它的讨论

2.4.4.3.2.1 原语生成

生成 Mgmt_Lqi_rsp，以响应一个 Mgmt_Lqi_req。如果不支持该管理命令，应返回一个 NOT_SUPPORTED 状态，且 Status 域之后的所有参数域都应被忽略。否则，远程设备应执行以下程序。

在接收时，已经证实了支持 Mgmt_Lqi_req 之后，远程设备应执行一个 NLME-GET.request（为了 *nwkNeighborTable* 属性）并处理由此产生的相邻表（通过 NLME-GET.confirm 原语获得），以创建 Mgmt_Lqi_rsp 命令。成功获得了 *nwkNeighborTable*，但是 NeighborTableList 记录（参见表 2.126）中的一个或多个域不被支持（因为它们是可选的），Mgmt_Lqi_rsp 应返回一个 NOT_SUPPORTED 状态，且 Status 域之后的所有参数域都应被忽略。否则，Mgmt_Lqi_rsp 命令应包含与 NLME-GET.confirm 原语所含相同的状态，且如果这不是 SUCCESS，Status 域之后的所有参数域都应被忽略。

从 *nwkNeighborTable* 属性，相邻表应该被访问，以 StartIndex 指定的索引开始，应被移动到 Mgmt_Lqi_rsp 命令的 NeighborTableList 域。从相邻表所报告的条目应是这些，以 StartIndex 开始，且包含整个 NeighborTableList 记录（参见表 2.126），直到 MSDU 大小的限制，即达到 *aMaxMACFrameSize*（参见[B1]）。在 Mgmt_Lqi_Rsp 命令内，NeighborTableEntries 域应表示远程设备内相邻表条目的总数。NeighborTableListCount 参数应是 Mgmt_Lqi_rsp 命令中的 NeighborTableList 域所报告的条目个数。

扩展地址，设备类型，RxOnWhenIdle 和允许加入域的值是“未知”的，应在值无法使用时被返回。

2.4.4.3.2 接收结果

本地设备被告知其尝试获得相邻表的结果。

2.4.4.3.3 Mgmt_Rtg_rsp

Mgmt_Rtg_rsp 命令（ClusterID=0x8032）应按图 2.96 所示的格式编排。

字节数: 1	1	1	1	可变的
Status	RoutingTable Entries	Start Index	RoutingTable ListCount	RoutingTable List

图 2.96 Mgmt_Rtg_rsp 命令帧的格式

表 2.127 指明了 Mgmt_Rtg_rsp 命令帧的域。

表 2.127 Mgmt_Rtg_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	从NLMEGET.confirm 原语返回的 NOT_SUPPORTED或者任何状态代码	Mgmt_Rtg_req命令的状态
RoutingTableEntries	Integer	0x00-0xff	在远程设备表中路由表目录的总数
StartIndex	Integer	0x00-0xff	路由表中开始报告给 RoutingTableList的起始索引
RoutingTableListCount	Integer	0x00-0xff	包含在RoutingTableList内的路由表个数
RoutingTableList	List of Routing Descriptors	列表应该包含通过 RoutingTableListCount 给出的事件编号	一个描述符列表，以 StartIndex元素开始并且持续到远程设备的路由表的 RoutingTableListCount的元素。（详见表2.128）

表 2.128 RoutingTableList 记录格式

名称	大小(位)	有效范围	描述
Destination address	16	这条路由的16位网络地址	目的地址
Status	3	路由状态	0x0=ACTIVE. 0x1=DISCOVERY_UNDERWAY. 0x2=DISCOVERY_FAILED. 0x3=INACTIVE. 0x4-0x7=RESERVED.
Memory Constrained	1		一个标志，表示设备否是一个限制内存的集中器。
Many-to-one	1		一个标志，表示目标是发出一个多对一请求的集中器。
Route record required	1		一个标志，表示一个路由记录命令帧必须在下一个数据包之前送到目标。
Reserved	2		
Next-hop address	16	到目标的下一跳的16位网络地址	下一跳地址

CCB Comment # 250

2.4.4.3.3.1 原语生成

生成 Mgmt_Rtg_rsp，以响应一个 Mgmt_Rtg_req。如果不支持该管理命令，应返回一个 NOT_SUPPORTED 状态，且 Status 域之后的所有参数域都应被忽略。否则，远程设备应执行以下程序。

在接收时，已经证实了支持 Mgmt_Rtg_req 之后，远程设备应执行一个 NLME-GET.request（为了 *nwkRouteTable* 属性），并处理由此产生的 NLME-GET.confirm（包含 *nwkRouteTable* 属性），以创建 Mgmt_Rtg_rsp 命令。Mgmt_Rtg_rsp 命令应包含与 NLME-GET.confirm 原语所含相同的状态，且如果这不是 SUCCESS，Status 域之后的所有参数域都应被忽略。

从 *nwkRouteTable* 属性, 路由表应被访问, 以 *StartIndex* 指定的索引开始, 应被移动到 *Mgmt_Rtg_rsp* 命令的 *RoutingTableList* 域。从路由表所报告的条目应是这些, 以 *StartIndex* 开始, 且包含整个 *RoutingTableList* 记录 (参见表 2.127), 直到 MSDU 大小的限制, 即达到 *aMaxMACFrameSize* (参见【B1】)。在 *Mgmt_Rtg_rsp* 命令内, *RoutingTableEntries* 域应表示远程设备内路由表条目的总数。*RoutingTableListCount* 域应是 *Mgmt_Rtg_req* 命令中的 *RoutingTableList* 域所报告的条目个数。

2.4.4.3.2 接收结果

本地设备被告知其尝试获得路由表的结果。

2.4.4.3.4 Mgmt_Bind_rsp

Mgmt_Bind_rsp 命令 (*ClusterID*=0x8033) 应按图 2.97 所示的格式编排。

字节数: 1	1	1	1	可变的
Status	BindingTable Entries	Start Index	BindingTable ListCount	BindingTable List

图 2.89 Mgmt_Bind_rsp 命令帧的格式

表 2.129 指明了 *Mgmt_Bind_rsp* 命令帧的域。

表 2.129 Mgmt_Bind_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	从 APSMEGET. Confirm 原语返回的 NOT_SUPPORTED 或 者任何状态代码	<i>Mgmt_Bind_req</i> 命令的状态
BindingTableEntries	Integer	0x00-0xff	在远程设备中绑定表目录的 总数

表 2.129 Mgmt_Bind_rsp 命令的域

名称	类型	有效范围	描述
StartIndex	Integer	0x00-0xff	绑定表中开始报告给BindingTableList的起始索引
BindingTableListCount	Integer	0x00-0xff	包括在BindingTableList内的绑定表目录个数
BindingTableList	List of Binding Descriptors	该列表应该包含通过BindingTableListCount给出的元素个数	描述符列表,以StartIndex元素开始并且持续到远程设备的绑定表元素的BindingTableListCount (详见表2.130)

表 2.130 BindingTableList 记录格式

名称	大小(位)	有效范围	描述
SrcAddr	64	有效的64位IEEE地址	绑定目录的源IEEE地址
SrcEndpoint	8	0x01 - 0xff	绑定目录的源端点
ClusterId	16	0x0000 - 0xffff	要绑定到目的设备的源设备上的cluster标识符

表 2.130 BindingTableList 记录格式（续表）

名称	大小(位)	有效范围	描述
DstAddrMode	8	0x00 - 0xff	目的地址寻址模式。该域可以从下面列表中取一个非保留值： 0x00 = 保留 0x01 = DstAddr16位组地址，DstEndpoint不存在 0x02 = 保留 0x03 = DstAddr and DstEndp 都存在，是64位扩展地址 0x04 – 0xff = 保留
DstAddr	16/64	通过 DstAddrMode 域定义	绑定目录的目的地址
DstEndpoint	0/8	0x01 - 0xff	只有当DstAddrMode域的值为0x0时，该域才存在。如果存在，该域将是绑定目录的目的端点

2.4.4.3.4.1 原语生成

生成 Mgmt_Bind_rsp，以响应一个 Mgmt_Bind_req。如果不支持该管理命令，应返回一个 NOT_SUPPORTED 状态，且 Status 域之后的所有参数域都应被忽略。否则，远程设备应执行以下程序。

在接收时，已经证实了支持 Mgmt_Bind_req 之后，远程设备应执行一个 APSME-GET.request（为了 *apsBindingTable* 属性），并处理由此产生的 APSME-GET.confirm（包含 *apsBindingTable* 属性），以创建 Mgmt_Bind_rsp 命令。Mgmt_Bind_rsp 命令应包含与 APSME-GET.confirm 原语所含相同的状态，且如果这不是 SUCCESS，Status 域之后的所有参数域都应被忽略。

从 *apsBindingTable* 属性，绑定表应被访问，以 StartIndex 指定的索引开始，应被移动到 Mgmt_Bind_rsMgmt_Rtg_rsp 命令的 BindingTableList 域。从绑定表所报告的条目应是这些，以 StartIndex 开始，且包含整个 BindingTableList 记录（参见表 2.130），直到 MSDU 大小的限制，即达到 *aMaxMACFrameSize*（参见[B1]）。在 Mgmt_Bind_Rsp 命令内，BindingTableEntries 域应表示远程设备内绑定表条目的总数。BindingTableListCount 域应是 Mgmt_Bind_req 命令中的 BindingTableList 域所报告的条目个数。

2.4.4.3.3.2 接收结果

本地设备被告知它尝试获得绑定表的结果。

2.4.4.3.5 Mgmt_Leave_rsp

Mgmt_Leave_rsp 命令（ClusterID=0x8034）应按图 2.98 所示的格式编排。

字节数: 1
Status

图 2.98 Mgmt_Leave_rsp 命令帧的格式

表 2.131 指明了 Mgmt_Leave_rsp 命令帧的域。

表 2.131 Mgmt_Leave_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	从NLMELEAVE.confirm 原语返回的NOT_SUPPORTED或者任何状态代码	Mgmt_Leave_req命令的状态

2.4.4.3.5.1 原语生成

生成 Mgmt_Leave_rsp，以响应一个 Mgmt_Leave_req。如果不支持该管理命令，应返回一个 NOT_SUPPORTED 状态，且 Status 域之后的所有参数域都应被忽略。否则，远程设备应执行以下程序。

在接收时，已经证实了支持 Mgmt_Leave_req 之后，远程设备应执行 NLME-LEAVE.request 从当前连接的网络中分离。Mgmt_Leave_rsp 应包含与 NLME-LEAVE.confirm 原语所含相同的状态。

一旦一个设备已经分离，它可以执行预编程逻辑来执行 NLME-NETWORK-DISCOVERY 和 NLME-JOIN，以加入/重新加入一个网络。

2.4.4.3.5.2 接收结果

本地设备被告知其尝试使一个远程设备离开网络的结果。

2.4.4.3.6 Mgmt_Direct_Join_rsp

Mgmt_Direct_Join_rsp (ClusterID=0x8035) 应按图 2.99 所示的格式编排。

字节数: 1
Status

图 2.99 Mgmt_Direct_Join_rsp 命令帧的格式

表 2.132 指明了 Mgmt_Direct_Join_rsp 命令帧的域。

表 2.132 Mgmt_Direct_Join_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	从NLME-DIRECT-JOIN.confirm原语返回的NOT_SUPPORTED或者任何状态代码	Mgmt_Direct_Join_req命令的状态

2.4.4.3.6.1 原语生成

生成 Mgmt_Direct_Join_rsp, 以响应一个 Mgmt_Direct_Join_req。如果不支持该管理命令, 应返回一个 NOT_SUPPORTED 状态。否则, 远程设备应执行以下程序。

在接收时, 已经证实了支持 Mgmt_Direct_Join_req 之后, 远程设备应执行 NLME-DIRECT-JOIN.request, 以把 Mgmt_Direct_Join_req 所含的 DeviceAddress 直接连接到网络。Mgmt_Direct_Join_rsp 应包含与 NLME-DIRECT-JOIN.confirm 原语所含相同的状态。

2.4.4.3.6.2 接收结果

在接收时, 已经证实了支持 Mgmt_Direct_Join_req 之后, 远程设备应执行 NLME-DIRECT-JOIN.request 以把 Mgmt_Direct_Join_req 所含的 DeviceAddress 直接连接到网络。

2.4.4.3.7 Mgmt_Permit_Joining_rsp

Mgmt_Permit_Joining_rsp命令（ClusterID=0x8036）应按图2.100所示的格式编排。

字节数: 1
Status

图 2.100 Mgmt_Permit_Joining_rsp命令帧的格式

表2.133指明了Mgmt_Permit_Joining_rsp命令帧的域。

表 2.133 Mgmt_Permit_Joining_rsp命令的域

名称	类型	有效范围	描述
Status	Integer	从NLME-PERMITJOINING. Confirm原语返回的SUCCESS, INVALID_REQUEST或者任何状 态代码	Mgmt_Permit_Joining_rsp 命令的状态

2.4.4.3.7.1 原语生成

生成Mgmt_Permit_Joining_rsp，以响应一个单播的Mgmt_Permit_Joining_req。在下面的描述中，注意如果Mgmt_Permit_Joining_req作为一个广播给所有路由器的信息被接收，就不应该发送响应。如果请求的设备不允许这个管理命令，应返回一个INVALID_REQUEST状态。在接收时，已经证实了支持Mgmt_Permit_Joining_req，远程设备应执行NLME-PERMIT-JOINING.request。Mgmt_Permit-Joining_rsp应包含与NLME-PERMIT-JOINING.confirm原语所含相同的状态。

2.4.4.3.7.2 接收结果

Mgmt_Permit_Joining_req 命令的状态被通知给请求者。



2.4.4.3.8 Mgmt_Cache_rsp

Mgmt_Cache_rsp 命令（ClusterID=0x8037）应按图 2.101 所示的格式编排。

字节数: 1	1	1	1	可变的
Status	DiscoveryCacheEntries	StartIndex	DiscoveryCacheList Count	DiscoveryCacheList

图 2.101 Mgmt_Cache_rsp 命令帧的格式

表 2.134 指明了 Mgmt_Cache_rsp 命令帧的域。

表 2.134 Mgmt_Cache_rsp 命令的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS或 NOT_SUPPORTED	Mgmt_Cache_rsp命令的状态
DiscoveryCacheEntries	Integer	0x00 - 0xff	DiscoveryCacheEntries
StartIndex	Integer	0x00 - 0xff	StartIndex
DiscoveryCacheListC ount	Integer	0x00 - 0xff	列表应该包含通过 DiscoveryCacheListCount参数给出的 元素数
DiscoveryCacheList	Integer	DiscoveryCache描述 符列表	一个描述符列表, 每个表示一个已注 册的发现缓存设备, 以StartIndex元 素开始并且持续到 DiscoveryCacheListCount, 已注册在 主要发现缓存中的设备。每个目录的 的格式如表2.135。

表 2.135 DiscoveryCacheList 记录格式

名称	大小	有效范围	描述
Extended Address	64	扩展的64位IEEE 地址	缓存设备的64位IEEE地 址
Network Address	16	网络地址	缓存设备的16位网络地 址

2.4.4.3.8.1 原语生成

生成 Mgmt_Cache_rsp, 以响应一个 Mgmt_Cache_req。如果不支持该管理命令, 或远程设备不是一个主要缓存设备, 应返回一个 NOT_SUPPORTED 状态, 且 Status 域之后的所有参数域都应被忽略。否则, 远程设备应执行以下程序。在接收 Mgmt_Cache_req 时, 且已经证实了支持 Mgmt_Cache_req, 远程设备应使用该主要发现缓存设备的发现缓存, 访问一个已登记 ZigBee 终端设备的内部维护列表。所报告的条目应是这些, 以 StartIndex 开始, 包括整个 DiscoveryCacheList 记录 (参见表 2.138), 直到 MSDU 大小的限制, 即达到 *aMaxMACFrameSize* (参见【B1】)。在 Mgmt_Cache_rsp 命令内, DiscoveryCacheListEntries 域应表示远程设备内已登记条目的总数。参数 DiscoveryCacheListCount 应是 Mgmt_Cache_rsp 命令中的 DiscoveryCacheList 域所报告的条目的个数。

2.4.4.3.8.2 接收结果

本地设备被告知其尝试获得发现缓存列表的结果。

2.4.4.3.9 Mgmt_NWK_Update_notify

Mgmt_NWK_Update_notify 命令 (ClusterID=0x8038) 应按图 2.102 所示的格式编排。

字节: 1	4	2	2	1	可变的
Status	ScannedChannels	TotalTransmissions	TransmissionFailures	ScannedChannelsListCount	EnergyValues

图 2.102 Mgmt_NWK_Update_notify 命令帧的格式

表 2.136 指明了 Mgmt_NWK_Update_notify 命令帧的域。

表 2.136 Mgmt_NWK_Update_notify 命令帧的域

名称	类型	有效范围	描述
Status	Integer	SUCCESS, INVALID_REQUEST 或 NOT_SUPPORTED	Mgmt_NWK_Update_notify 命令的状态
ScannedChannels	Bitmap	0x00000000 - 0xffffffff	请求所扫描的信道列表
TotalTransmissions	Integer	0x0000 -0xffff	设备报告的总传输次数
TransmissionFailure s	Integer	x0000 -0xffff	设备报告的总传输失败次数
ScannedChannelsLi stCount	Integer	0x00 - 0xff	包含 EnergyValues 参数所含的记录个数的列表
EnergyValues	Integer	ED 值的列表, 每个范围可以是 0x00 - 0xff	这个信道上执行的一个能量检测的结果, 符合[B1]

2.4.4.3.9.1 原语生成

提供 Mgmt_NWK_Update_notify, 以使 ZigBee 设备向一个网络管理器报告本地信道的情况。所扫描的信道列表是扫描信道的报告, 随后是一个记录列表, 每个表示一个扫描的信道, 每个记录包含一个字节, 表示扫描期间测量的能量级别, 或如果这个信道上有很多的干扰, 就是 0xff。

当发送这个原语以响应一个 Mgmt_NWK_Update_req 命令, 状态域应表示请求的状态。当未经请求发送时状态域应设置为 SUCCESS。

2.4.4.3.9.2 接收结果

本地设备被告知发送设备或其尝试更新网络配置参数的本地信道的情况。

2.4.5 ZDP 枚举描述

本小节解释了 ZDP 使用的枚举的含义。表 2.137 展示了 ZDP 枚举值的描述。

表 2.137 ZDP 枚举描述

枚举类型	值	描述
SUCCESS	0x00	请求的操作或传输成功完成
-	0x01-0x7f	保留
INV_REQUESTTYPE	0x80	所提供的请求类型是无效的。
DEVICE_NOT_FOUND	0x81	不存在子描述符到父描述符的请求设备
INVALID_EP	0x82	提供端点等于0x00,或者在0xf1和0xff之间
NOT_ACTIVE	0x83	请求端点不能用一个简单描述符来描述
NOT_SUPPORTED	0x84	所请求的可选功能不提供给目标设备
TIMEOUT	0x85	请求操作发生超时
NO_MATCH	0x86	终端设备绑定请求失败原因在于未能匹配任何合适的cluster。
-	0x87	保留
NO_ENTRY	0x88	取消绑定请求不成功,原因在于协调器或者源设备在它的绑定表中没有取消绑定的目录
NO_DESCRIPTOR	0x89	子描述符不能跟随一个发现请求到父描述符
INSUFFICIENT_SPACE	0x8a	该设备不具备的存储空间,以支持请求的操作
NOT_PERMITTED	0x8b	设备不在一个合适的状态去支持请求操作
TABLE_FULL	0x8c	设备没有表空间去支持操作
NOT_AUTHORIZED	0x8d	目标的允许配置表指明请求未经这个设备授权。
-	0x8e-0xff	保留

2.4.6 一致性

当声明符合本 Profile 时，应该以特定方式支持本 Profile 指明为强制的所有功能（强制处理）。这也适用于可选的和有条件的功能，对它们的支持是指明的，服从作为 ZigBee 认证程序部分的验证。

2.5 ZigBee 设备对象（ZDO）

2.5.1 范围

本节描述了在一个 ZigBee 应用支持子层(2.2 小节)和 ZigBee 网络层(第 3 章)上执行一个 ZigBee 设备对象应用程序所需的概念，结构和原语。

ZigBee 设备对象是一个应用程序，使用了网络和应用支持子层原语来执行 ZigBee 终端设备，ZigBee 路由器和 ZigBee 协调器。

ZigBee 设备对象 Profile 使用了 Cluster 来描述其原语。ZigBee 设备 Profile Clusters 不使用属性，类似于一个信息传输协议中的信息。在 ZigBee 设备 Profile 内使用 Cluster 标识符来枚举 ZigBee 设备对象内使用的信息。

ZigBee 设备对象还使用了配置属性。这些属性不是任何 Cluster 的元素。ZigBee 设备对象内的配置属性是由应用程序或协议栈 profile 设置的配置参数。配置属性也不与 ZigBee 设备 profile 有关，尽管配置属性和 ZigBee 设备 profile 都被 ZigBee 设备对象使用。

2.5.2 设备对象描述

ZigBee 设备对象是一个应用程序解决方案，驻扎在 ZigBee 协议栈结构内的应用层（APL）和应用支持子层（APS）上面，如图 1.1 所示。

ZigBee 设备对象负责以下功能：

- 初始化应用支持子层（APS），网络层（NWK），安全服务提供商（SSP）和任何其它 ZigBee 设备层次，而不是驻留在端点 1-240 的端点应用程序。

- 组装来自于端点应用程序的配置信息，以确定和执行以下分节描述的功能。

2.5.2.1 主要发现缓存设备操作

主要发现缓存设备通过设备的配置和在节点描述符中的宣布来指定。主要发现缓存设备作为一个状态机器运行，和希望使用主要发现缓存的服务的客户端有关。以下状态和操作，如图 2.103 所述，应被主要发现缓存设备支持：

- 未发现:
 - 客户端使用找到节点缓存请求，广播给所有的 `macRxOnWhenIdle=TRUE` 的设备，以确定是否有一个本地设备的已有的发现缓存条目。如果一个发现缓存设备响应了请求，本地设备可以更新发现信息，转换到已注册状态。
 - 客户端使用范围有限的信息 `System Server Discovery` 请求，广播给所有 `macRxOnWhenIdle=TRUE` 的设备，在请求给定的半径内寻找一个主要发现缓存设备。
- 发现:
 - 客户端使用单播的发现缓存请求，发给发现缓存设备，包含它希望存储的发现缓存信息的大小。发现缓存设备将响应一个 `SUCCESS`、`INSUFFICIENT_SPACE` 或 `NOT_SUPPORTED`。
- 注册:
 - 当客户端从发现缓存设备，从之前的一个发现缓存请求收到一个 `SUCCESS` 状态，或找到节点缓存请求找到一个预先存在的发现缓存条目，就达到该状态。客户端现在必须使用节点描述符存储请求，电源描述符存储请求，活动终端存储请求和简单描述符存储请求装载其发现信息，以使主要发现缓存设备在其代表上完全响应。
- 未登记:
 - 客户端（或任何其它设备）可以请求取消登记。删除节点缓存请求将从主要发现缓存设备中删除该设备。主要缓存设备为所有其支持的已登记客户端响应设备和服务发现请求。找到节点缓存请求被想要为一个给定的兴趣设备找到设备和服务发现位置的客户端使用。注意如果发现信息由设备本身持有，设备也必须响应，以确定自己是存放发现信息的设备。关于主要发现缓存设备的状态机器处理参见图 2.103。

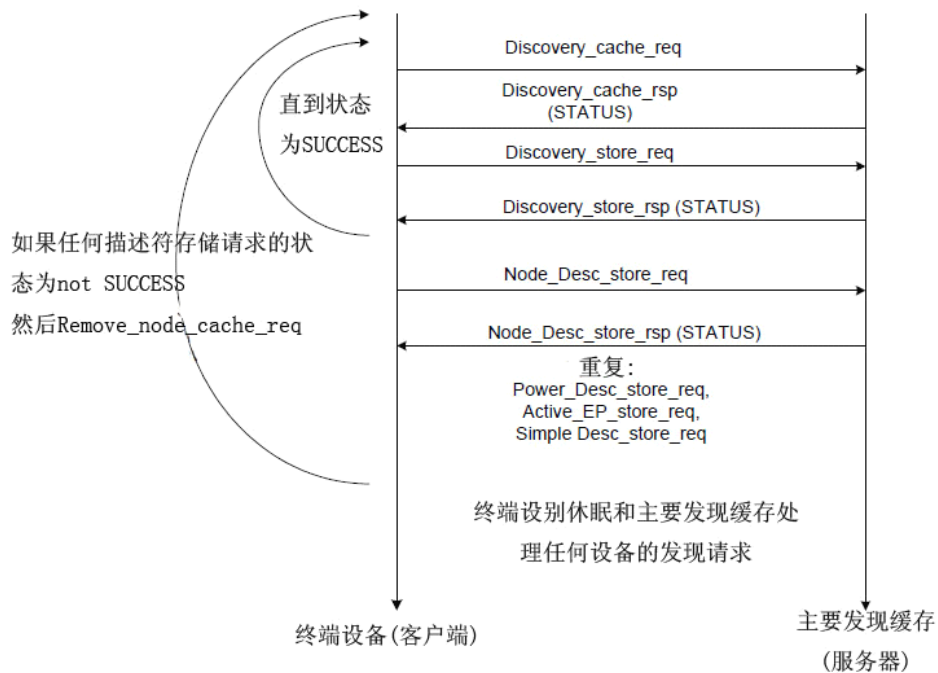


图 2.103 主要发现缓存状态设备

2.5.2.2 设备和服务发现

该功能应在一个 PAN 内支持设备和服务发现。另外，对于 ZigBee 设备类型，该功能应执行以下步骤：

- 在使用了睡眠 ZigBee 终端设备的每个网络内，一些 ZigBee 路由器（或 ZigBee 协调器）必须被指定为主要发现缓存设备，被它们的节点描述符描述。这些主要缓存设备自身是可以发现的，并提供服务器服务，以代表睡眠的 ZigBee 终端设备装载和存储发现信息。另外，主要缓存设备代表睡眠的 ZigBee 终端设备响应发现请求。每个主要发现缓存设备应是一个 ZigBee 路由器或 ZigBee 协调器。
- 对于 Config_Node_Power 指明的打算睡眠的 ZigBee 终端设备，设备和服务发现应管理装载和存储 NWK 地址，IEEE 地址，活动端点，简单描述符，节点描述符和电源描述符到一个 ZigBee 终端设备选定的主要发现缓存设备中，以允许设备和服务发现在这些睡眠设备上运行。

- 对于指定为主要发现缓存设备的 ZigBee 协调器和 ZigBee 路由器，该功能应该代表已经登记的睡眠 ZigBee 终端设备响应发现请求，并装载其发现信息。
- 对于所有的 ZigBee 设备，设备和服务发现应该支持来自于其它设备的设备和服务发现请求，并允许从其本地应用程序对象生成请求。注意设备和服务发现服务可以由代表其它 ZigBee 终端设备的主要发现缓存设备提供。在主要发现缓存设备是该请求的目标的情况下，应在请求和/或响应中填充 NWKAddrOfInterest 或兴趣域的设备，以把该请求的目标和是发现目标的设备区分开来。应该支持以下发现功能：
 - 设备发现:
 - 根据一个 ZigBee 协调器或 ZigBee 路由器的 IEEE 地址的一个单播查询，应该返回被请求的设备的 IEEE 地址，另外可选择地返回所有相关设备的 NWK 地址。
 - 根据一个 ZigBee 终端设备的 IEEE 地址的单播查询，应返回被请求设备的 IEEE 地址。
 - 根据提供的一个 IEEE 地址，一个 ZigBee 协调器或 ZigBee 路由器的 NWK 地址的广播查询（任何广播地址类型），应返回被请求设备的 NWK 地址，另外可选择地返回所有相关设备的 NWK 地址。
 - 根据提供的一个 IEEE 地址，一个 ZigBee 终端设备的 NWK 地址的广播查询（任何广播地址类型），应返回被请求设备的 NWK 地址。响应的设备应为广播查询的单播响应使用 APS 确认服务。
 - 服务发现:根据以下输入，应提供相应的响应:
 - NWK 地址加上活动终端查询类型——指定设备应返回驻扎在该设备上的所有应用程序的终端个数。如果活动端点的列表超过 ASDU 大小，服务器设备又不支持分段，还能提供查询类型的一个扩展版本，以通过多个请求返回完整的列表。

- NWK 地址或广播地址(任何广播地址类型)加上服务匹配, 包含 Profile ID, 和可选择的输入和输出 Cluster——指定设备把 Profile ID 和所有活动端点匹配, 以确定一个匹配。如果没有指定输入和输出 Cluster, 返回匹配该请求的端点。如果请求中提供了输入和/或输出 Cluster, 那些以及响应中提供的匹配, 与提供匹配的设备端点的列表的任何匹配都是匹配的。响应的设备对于发给广播查询的单播的响应, 应使用 APS 确认服务。在应用程序 profile 希望枚举输入 Cluster 和其由此产生的具有同一 Cluster 标识符的响应输出 Cluster 的情况下, 应用程序 profile 应该只列出目的是服务发现的简单描述符内的那些输入 Cluster。
- NWK 地址加上节点描述符或电源描述符查询类型——指定设备应为该设备返回节点或电源描述符。
- NWK 地址, 端点号加上简单描述符查询类型——指定地址应返回与设备该端点相关联的简单描述符。如果输入和/或输出 cluster 的列表超过 ASDU 能够在一个数据包中返回简单描述符的容量, 还能提供查询类型的一个扩展版本, 以通过多个请求返回完整的列表。
- 可选地, NWK 地址加上复杂或用户描述符查询类型——如果支持的话, 指定地址应返回该设备的复杂或用户描述符。

2.5.2.3 安全管理

该功能确定是否启用或禁用安全功能, 以及如果启用的话, 执行以下功能:

- 建立密钥
- 传输密钥
- 请求密钥
- 更新设备
- 删除设备
- 转换密钥
- 验证设备

安全管理功能在安全服务规范（第 4 章）介绍。安全管理，由 APSME 原语实施，由 ZDO 调用，执行以下步骤：

- 联系信任中心以获得本设备和信任中心之间的主要密钥（如果本设备是信任中心，信任中心的主要密钥已经预先配置，则此步骤省略）。此步骤使用了传输密钥原语。
- 建立一个信任中心的连接密钥。本步骤使用了 APSME-Establish-Key 原语（住宅模式中忽略）。
- 使用和信任中心加密的通信，从信任中心获得 NWK 密钥（商业模式）。或者，来自信任中心的 NWK 密钥可以使用未加密的通信从信任中心发送（住宅模式）。本步骤使用了 APSME-TRANSPORT-KEY 原语。
- 根据需要，建立网络中被确定为信息目的地的具体设备的连接密钥和主要密钥（商业模式）。这些步骤使用了 APSME-ESTABLISH-KEY、APSME-TRANSPORT-KEY 和/或 APSME-REQUEST-KEY 原语。
- 使用 APSME-DEVICE-UPDATE 原语，通知加入该网络的任何设备的信任中心。本功能只在设备是一个 ZigBee 路由器时执行。
- 使用 APSME-REQUEST-KEY 原语，允许设备从信任中心获得密钥。
- 使用 APSME-REMOVE-DEVICE 原语，允许信任中心从网络中删除设备。
- 使用 APSME-SWITCH-KEY 原语，允许信任中心转换活动网络密钥。
- 允许网络中的设备使用 APSME-AUTHENTICATE 原语验证其它设备。

2.5.2.4 网络管理

该功能应根据确定的配置设置，通过一个程式化的应用程序或在安装期间，执行 ZigBee 协调器、ZigBee 路由器或 ZigBee 终端设备的逻辑设备类型。如果设备类型是一个 ZigBee 路由器或 ZigBee 终端设备，该功能应提供选择一个已存在的 PAN 来加入的能力，并且如果网络通信丢失，执行允许设备重新加入的程序的能力。如果设备类型是一个 ZigBee 协调器或 ZigBee 路由器，该功能应提供选择一个未使用的信道来创建一个新的 PAN 的能力。注意有可能部署没有一个设备被预先指定为 ZigBee 协调器的一个网络，而且第一个全功能设备（FFD）作为 ZigBee 协调器被激活。以下描述覆盖了网络管理的处理步骤：

- 允许规定网络扫描程序的一个信道列表。默认为在选定的运行频带上指明使用所有的信道。
- 管理网络扫描程序，以确定相邻网络，并确定其 ZigBee 协调器和路由器。
- 允许选择一个信道以启动一个 PAN（ZigBee 协调器）或选择一个已存在的 PAN 来加入（ZigBee 路由器或 ZigBee 终端设备）。
- 支持孤点和扩展程序来重新加入网络，包括支持 intra_PAN 的可移植性。
- 可以支持直接加入。对于 ZigBee 协调器和 ZigBee 路由器，可以支持直接加入的一个本地版本，以使得设备通过孤程序或重新加入程序加入。
- 可以支持允许外部网络管理的管理实体。
- 检测和报告干扰以支持变化的网络信道。
- 管理网络干扰报告，且如果某个节点被确定为整个 PAN 的网络管理器，如果初始信道上存在干扰，为网络操作选择一个新信道。

2.5.2.5 绑定管理

绑定管理执行以下功能：

- 建立绑定表的源大小。这个源的大小是通过一个程式化的应用程序，或通过安装期间定义的一个配置参数来确定的。
- 为从 APS 绑定表中增加或删除实体处理绑定请求。
- 支持来自于外部应用程序的绑定和取消绑定命令，比如那些可以存放在 PDA 上的命令，以支持辅助绑定。绑定和取消绑定命令应通过 ZigBee 设备 Profile 来支持（参见 2.4 节）。
- 对于 ZigBee 协调器，支持终端设备绑定，即允许在按钮按下或其它手动方式的基础上的绑定。
- 允许源设备把自身可以存储自己的绑定表的能力注册到一个主要绑定表缓存中。
- 允许配置工具交换一个设备到提到另一个设备的所有绑定表条目中。
- 允许主要绑定表缓存备份和恢复各自的绑定表条目，或整个绑定表，或者存储自己绑定表的源设备表。

2.5.2.6 节点管理

对于 ZigBee 协调器和 ZigBee 路由器，节点管理功能执行以下功能：

- 允许远程管理命令，来执行网络发现
- 提供远程管理命令，来检索路由表
- 提供远程管理命令，来检索绑定表
- 提供一个远程管理命令，来使设备离开网络或指示另一个设备离开网络
- 提供一个远程管理命令，来为远程设备的邻居检索 LQI
- 提供一个远程管理命令，来允许或禁止在某个路由器上加入，或一般地通过信任中心允许或禁止加入。

2.5.2.7 组管理器

组管理器执行以下功能：

- 提供在应用程序控制下，把本地设备内应用对象纳入组的功能。
- 提供在应用程序控制下，把本地设备内应用对象移出组成员的功能。

2.5.3 层接口描述

不像驻扎在端点 1-240 的应用程序的其它设备描述符，除了 APSDE-SAP，ZigBee 设备对象 (ZDO) 的接口通过 APSME-SAP 到 APS。在端点 0 上的 ZDO 通信使用了 APSDE-SAP，通过像和所有其它应用程序一样的 Profile。ZDO 所使用的 Profile 是 ZigBee 设备 Profile（参见 2.4 节）。ZDO 帧不能分段。

在任何发送的 ZigBee 设备 Profile 请求帧中，ZigBee 设备对象使用端点 0 作为源和目标端点，且在任何收到的响应帧中，也期望使用端点 0 作为源和目标端点。

2.5.4 系统使用

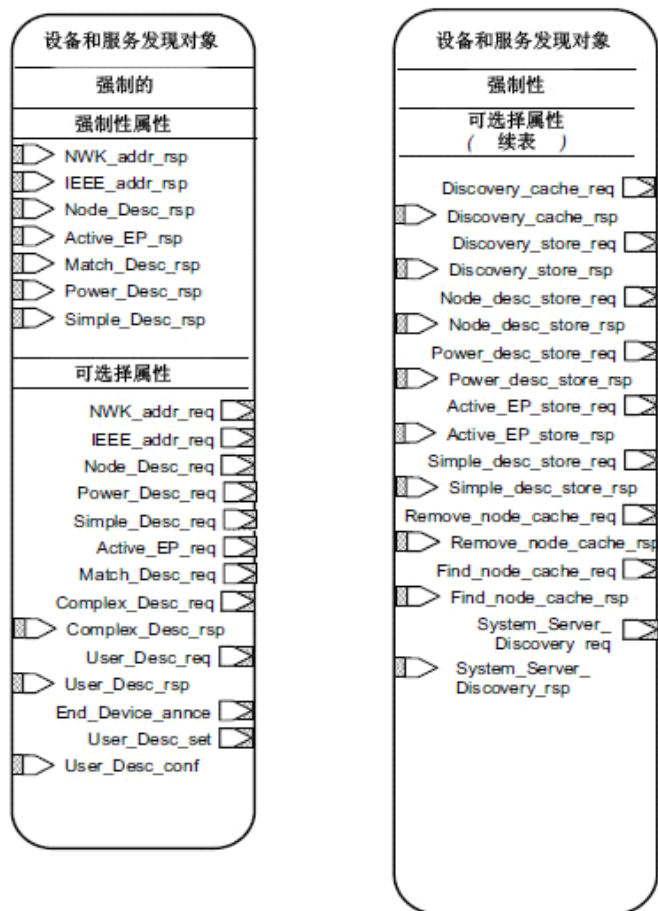


图 2.104 系统使用 ZigBee 设备对象的详细情况

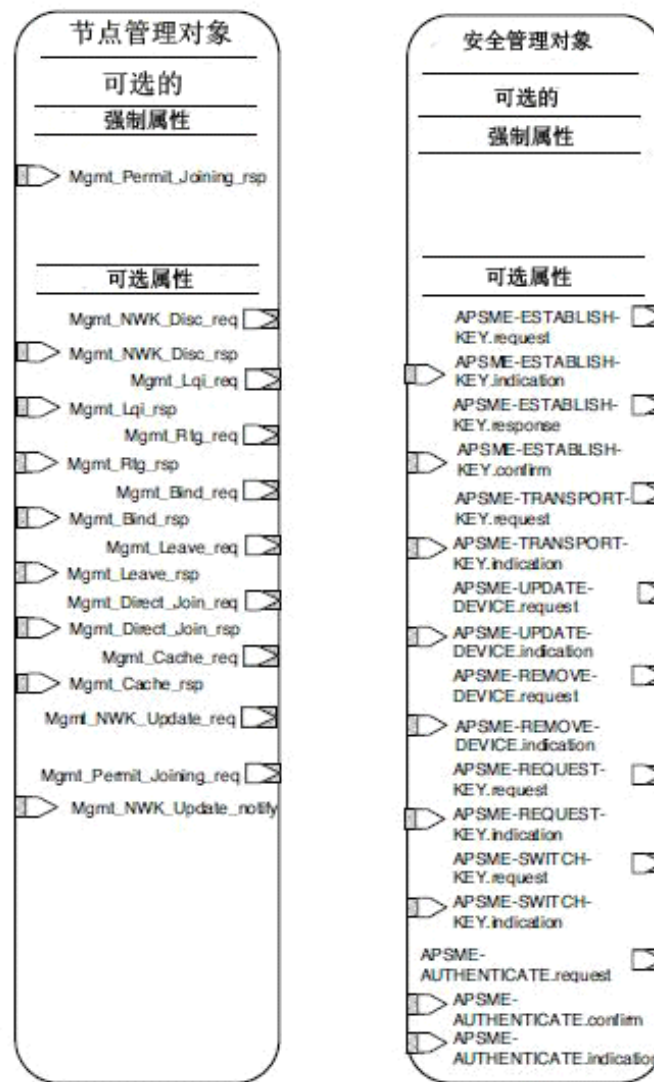


图 2.105 系统使用 ZigBee 设备对象的详细情况-节点管理对象和安全管理对象

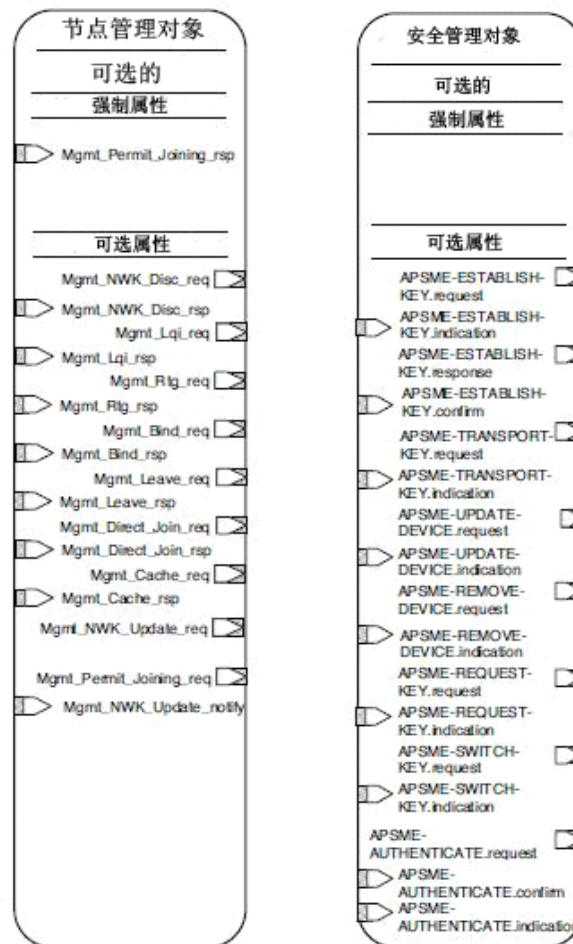


图 2.106 系统使用 ZigBee 设备对象的详细情况-绑定管理对象和网络管理对象

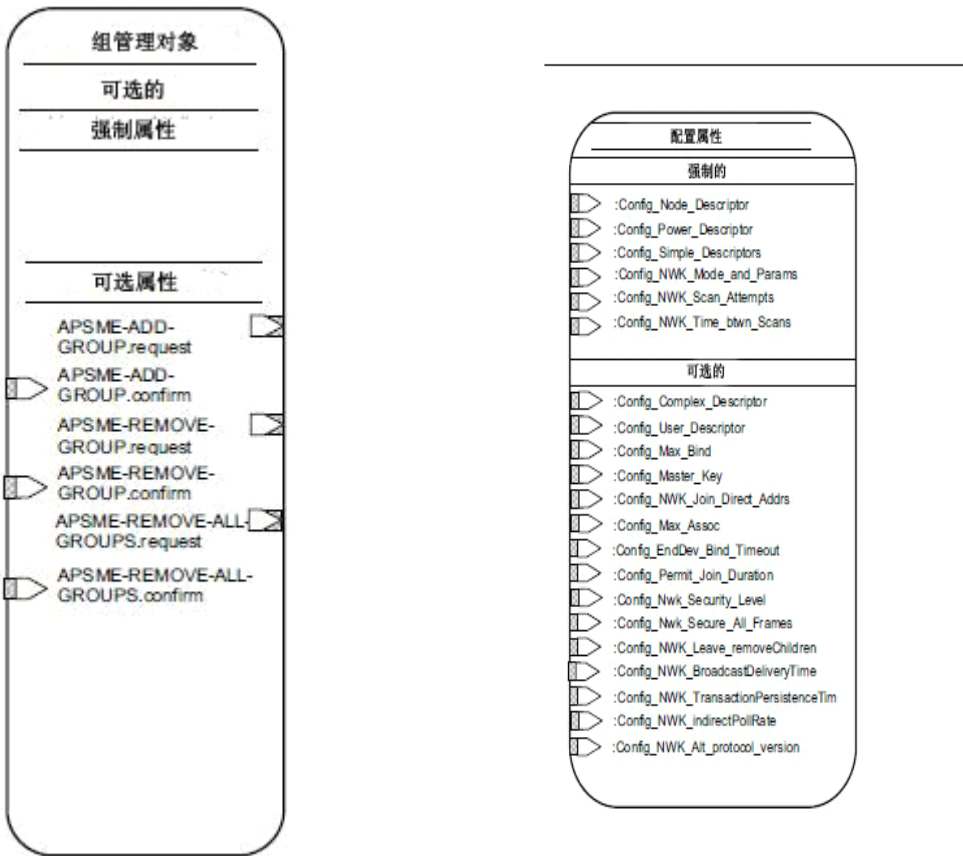


图 2.107 系统使用 ZigBee 设备对象的详细情况-组管理对象和配置属性

2.5.5 对象定义和行为

2.5.5.1 对象概述

ZigBee 设备对象包含六种对象:

- 设备和服务发现
- 网络管理器
- 绑定管理器
- 安全管理器
- 节点管理器
- 组管理器

表 2.138 描述了这些 ZigBee 设备对象。

表 2.138 ZigBee 设备对象

对象		描述
名称	状态	
Device_and_Service_Discovery	M	处理设备和服务的发现。
:Network_Manager	M	处理网络的活动，如网络发现，离开/加入一个网络，设 网络连接，并建立一个网络。
:Binding_Manager	O	处理终端设备绑定，绑定和解除绑定活动。
:Security_Manager	O	处理安全服务，如密钥载入中，密钥的建立，密钥传输 和验证。
:Node_Manager	O	处理管理职能。
:Group Manager	O	处理组管理

2.5.5.2 可选的和强制的对象和属性

列为强制的对象应出现在所有的 ZigBee 设备中。但是，对于某些 ZigBee 逻辑类型，对所有的 ZigBee 设备列为可选的对象，在具体逻辑设备类型中可能是强制的。例如，在 Network_Manager 对象中的 NLME-NETWORK-FORMATION.request 是一个强制的对象，且是一个可选的属性，但对于 ZigBee 协调器逻辑设备类型该属性是必须的。各个设备对象节的简介一节将详细介绍逻辑设备类型支持的对象和属性需求。

2.5.5.3 安全密钥使用

ZigBee 设备对象可以为 ZigBee 设备 Profile 原语创建的数据包使用安全功能。这些在端点 0 上使用 APSDE 的应用程序数据包，应使用 APSDE 安全服务供应商接口，像所有其它应用程序对象。

2.5.5.4 公共的和私有的方法

对设备上任何端点应用程序都可以使用的方法叫做公共的方法。私有的方法仅对在端点 0 上的设备应用程序可以使用，且最终应用程序不可以使用（运行在端点 1 到 240 上）。

2.5.5.5 状态机的功能描述

2.5.5.5.1 ZigBee 协调器

2.5.5.5.1.1 初始化

执行时应把表 2.139 所示的有关启动的 IB 参数的值设置为反映期望的设备启动行为。特别是，IB *apsDesignatedCoordinator* 属性必须设置为 TRUE。如果设备执行多个 ZigBee 协议版本或栈 profile 的选项，应为每个选项选择一个值，并因此设置 *nwkProtocolVersion* 和 *nwkStackProfile*。另外，应做出安排，以提供配置元素来描述每个活动端点和应用以及活动端点列表的节点描述符，电源描述符，简单描述符。这些配置应体现

在:Config_Node_Descriptor, :Config_Power_Descriptor 和:Config_Simple_Descriptors 中。如果:Config_Node_Descriptor 配置对象表明，本设备是一个主要发现缓存设备，该设备应配置为处理服务器命令，用于和送给主要发现缓存的请求有关的 ZigBee 设备 profile，并应根据 2.5.2.1 小节所提供的状态机描述进行操作。

如果支持，应做出安排，来为复杂描述符，用户描述符，绑定条目最大个数和主要密钥提供配置元素。这些元素应体现在:Config_Complex_Descriptor, :Config_User_Descriptor, :Config_Max_Bind 和:Config_Master_Key 中。

要作为一个 ZigBee 协调器启动，设备应用程序必须执行 2.5.5.5.6.2 小节所述的启动程序，启动参数的设置如上所述。这会影响 3.6.1.1 小节所述的网络形成程序的执行。设备应用程序必须根据设备使用的栈 Profile 的约定建立的值，设置 *nwkSecurityLevel*、*nwkAllFresh* 和 *nwkSecureAllFramesNIB* 属性。设备应用程序必须通过 NLME-NETWORK-FORMATION.confirm 检查返回状态以证实成功创建了 PAN。:Config_Permit_Join_Duration 应根据使用 NLME-PERMIT-JOINING.request 所提供的默认参数值来设置。另外，*nwkNetworkBroadcastDeliveryTime* 和 *nwkTransactionPersistenceTime* 网络信息模块参数（见 3.6.2 小节）应分别设置

为:Config_NWK_BroadcastDeliveryTime 和:Config_NWK_TransactionPersistenceTime（见 2.5.6 小节）。

应做出安排，以保证在 ZigBee 设备对象完成初始化状态，并转为正常运行状态之前，在 EP1 到 EP240 上的最终应用程序的 APS 原语调用返回适当的错误状态值。

2.5.5.1.1.2 正常运行状态

在本状态中，ZigBee 协调器应通过为列表中所含的每个地址发出一个 NLME-DIRECT-JOIN.request，处理在:Config_NWK_Join_Direct_Addrs 中的直接加入地址列表。直接加入地址的处理应使用:Config_Max_Assoc 参数，来评估是否成功处理了在:Config_NWK_Join_Direct_Addrs 中的一个直接加入地址。

ZigBee 协调器应根据配置选项:Config_Permit_Join_Duration 和:Config_Max_Assoc, 允许其它设备加入网络。当一个新设备加入网络，应通过 NLME-JOIN.indication 来通知设备应用程序。如果设备被纳入到 PAN 中，ZigBee 协调器应通过 SUCCESS 状态的 NLME-JOIN.confirm 来表示。

ZigBee 协调器应响应任何设备发现或服务发现操作，请求其自身的设备，且如果它被指定为一个主要发现缓存设备时，也应代表存储在发现信息中的已登记设备响应。设备应用程序也应保证绑定条目的个数不超过:Config_Max_Bind 属性。

ZigBee 协调器应支持 NLME-PERMIT-JOINING.request 和 NLME-PERMIT-JOINING.confirm，以允许应用程序控制网络加入处理。

ZigBee 协调器应支持 NLME-LEAVE.request 和 NLME-LEAVE.indication，使用:Config_NWK_Leave_removeChildren 属性，在适当的情况下，允许在应用程序控制下删除相关设备。导致删除相关设备的条件可能包括缺少安全证书，通过一个享有特权的应用程序或异常检测来删除该设备。当请求一个 ZigBee 终端设备离开网络时，NLME-LEAVE.request 原语的 ReuseAddress 参数值应为 TRUE，表示 NWK 层可以把之前离开设备使用的地址，给另一个随后加入的终端设备。

ZigBee 协调器应维护一个当前相关设备的列表，并帮助支持孤扫描和重新加入程序，以使得之前的相关设备重新加入网络。ZigBee 协调器通过 NLME-DIRECT-JOIN.request 和 NLME-DIRECT-JOIN.confirm，可以支持设备被直接纳入网络中的功能。该功能应允许 ZigBee IEEE 地址的列表提供给 ZigBee 协调器，以及之前相关设备包含的那些地址。带有那些地址的 ZigBee 设备应该可以通过孤点或重新加入程序直接加入网络，而不是直接联系。

ZigBee 协调器应支持 3.2.2.30 小节的 NLME-NWK-STATUS.indication 和每个这些通知的处理。

ZigBee 协调器应处理来自于 ZigBee 路由器和 ZigBee 终端设备的 End_Device_Bind_req。在收到一个 End_Device_Bind_req 时，ZigBee 协调器应使用属性中的:Config_EndDev_Bind_Timeout 的值，并等待第二个 End_Device_Bind_req。如果在超时时间内第二个指示到达，ZigBee 协调器应在两个指示中匹配 Profile ID。如果两个指示中的 Profile ID 不匹配，通过 End_Device_Bind_rsp 给每个设备返回一个适当的错误状态。如果 Profile ID 匹配，ZigBee 协调器应在这两个指示中匹配 AppInClusterLists 和 AppOutClusterLists。第一个指示的 AppInClusterLists 中的 Cluster ID 匹配第二个指示的 AppOutClusterLists 中的 Cluster ID，应保存在一个列表中，以便列入 End_Dev_Bind_rsp 中。

ZigBee 协调器应处理来自于其它 ZigBee 设备的 Device_annce 信息。在收到一个 *nwkUseTreeRouting* 为 TRUE 的 Device_annce 时，ZigBee 协调器应为 PAN 中的设备检查持有 64 位 IEEE 地址的所有内部表，寻找 Device_annce 信息所提供的地址的一个匹配。如果检测到一个匹配，ZigBee 协调器应更新其 APS 信息模块地址映射条目，把相应的匹配 64 位 IEEE 地址，映射为 Device_annce 所含的更新后的 16 位 NWK 地址。在收到一个 *nwkUseTreeRouting* 为 FALSE 的 Device_annce 时，ZigBee 协调器使用 3.6.9 小节所述的地址冲突解决程序。

ZigBee 协调器可以在应用程序控制下从其它应用程序对象生成 APSME-AUTHENTICATE.request 并可以处理和响应来自其它设备的 APSME-AUTHENTICATE.indication。ZigBee 协调器必须提供 APSME-AUTHENTICATE.confirm 给请求已经被处理的那些应用程序对象。

2.5.5.5.1.3 信任中心操作

当网络启用安全功能，地址 *apsTrustCenterAddress* 指明的网络设备必须像信任中心一样运行。

信任中心操作定义在 4.6.2 小节中。

2.5.5.5.2 ZigBee 路由器

2.5.5.5.2.1 初始化

执行时应把表 2.139 所示的有关启动的 IB 参数的值设置为反映期望的设备启动行为。特别是，IB *apsDesignatedCoordinator* 属性必须设置为 FALSE。如果:Config_Node_Descriptor 配置对象指示本设备是一个主要发现缓存设备，设备应配置为处理服务器命令，用于和送给主要发现缓存的请求有关的 ZigBee 设备 Profile，并应根据 2.5.2.1 小节所提供的状态机描述进行操作。

如果支持，应做出安排，来为复杂描述符，用户描述符，绑定条目的最大值和主要密钥提供配置元素。这些元素应体现在 Config_Complex_Descriptor，:Config_User_Descriptor，:Config_Max_Bind 和:Config_Master_Key 中。

要作为一个 ZigBee 协调器启动，设备应用程序必须执行 2.5.5.5.6.2 小节所述的启动程序，启动参数的设置如上所述。这会影响 3.6.1.4.2 小节所述的网络重新加入程序，或 3.6.1.4.1 小节所述的通过 MAC 连接网络加入的整个程序的执行。NLME-NETWORK-DISCOVERY.request 程序应被执行:Config_NWK_Scan_Attempts，每次间隔时间是:Config_NWK_Time_btwn_Scans。重复 NLME-NETWORK-DISCOVERY.request 的目的是给 NWK 层提供一个更准确的相邻列表和相关链接质量指示。选择 PAN 的算法的规定留给 profile 描述，可以包括使用 Extended PAN ID、网络的操作模式、确定 PAN 上 ZigBee 路由器或协调器的身份、PAN 上从 ZigBee 协调器到 ZigBee 路由器的深度、ZigBee 路由器或协调器的容量、路由消耗，或协议版本号（这些参数由 NLME-NETWORK-DISCOVERY.confirm 和信标负载提供）。

ZigBee 路由器在应用程序控制下，且如果设备支持向后兼容，它可以使用当前的协议版本号加入网络，或使用一个之前的协议版本号加入网络。一个 ZigBee PAN 应包括只使用一个协议版本号的设备（在同一个 PAN 内，使用不同协议版本号的设备和帧格式的网络是不允许的）。一个可选的配置属性:Config_NWK_alt_protocol_version 提供了设备可以选择使用的协议版本号，而不是当前协议版本号。一旦 ZigBee 路由器选择了一个 PAN 和一个具体的协议版本号，它应使用其 *nwkProtocolVersion* 指明的协议版本号。另外，然后 ZigBee 路由器应遵守使用该协议版本号的 ZigBee 规范版本提供的所有帧格式和处理规则。

:Config_Permit_Join_Duration 必须根据使用 NLME-PERMIT-JOINING.request 提供的默认参数值设置。路由器必须支持 NLME-START-ROUTER.request 和 NLME-START-ROUTER.confirm, 以在它加入的 PAN 内开始作为一个路由器运行。另外, *nwkNetworkBroadcastDeliveryTime* 和 *nwkTransactionPersistenceTime* 网络形成块参数 (见 3.6.2 小节) 必须分别用:Config_NWK_BroadcastDeliveryTime 和:Config_NWK_TransactionPersistenceTime 设置 (见 2.5.6 小节)。

应做出安排, 以保证在 ZigBee 设备对象完成初始化状态, 并转为正常运行状态之前, 在 EP1 到 EP240 上的最终应用程序的 APS 原语调用返回适当的错误状态值。

如果网络启用安全功能, 设备应等待信任中心对它进行验证, 为了成功获取 NWK 密钥时, 以开始作为一个路由器在网络中运行。信任中心详细操作见 4.6.2 小节。

设备应用程序应设置 *nwkSecurityLevel* 和 *nwkSecureAllFrames* NIB 属性为网络中使用的值, 且使用 NLME-START-ROUTER.req 开始作为路由器运行。

2.5.5.5.2.2 正常运行状态

在本状态中, ZigBee 路由器应根据配置选项:Config_Permit_Join_Duration 和:Config_Max_Assoc, 允许其它设备加入网络。当一个新设备加入网络, 应通过 NLME-JOIN.indication 属性通知设备应用程序。如果设备被纳入到 PAN 中, ZigBee 路由器应通过 SUCCESS 状态的 NLME-JOIN.confirm 来表示。如果网络启用了安全功能, 设备应用程序应通过 APSME-UPDATE-DEVICE.request 通知信任中心。

在收到一个参数 IsParent 设置为 FALSE 的 NLME-JOIN.indication 原语时, 表示本设备不是父节点的孤指示从 NWK 层通知给 ZDO。它的处理机制在 2.5.5.5.4 中描述。

ZigBee 路由器应响应任何设备发现或服务发现操作, 请求其自身的设备, 且如果它被指定为一个主要发现缓存设备, 也应代表存储发现信息的已登记设备响应。设备应用程序还应保证绑定条目的个数不超过:Config_Max_Bind 属性。

如果在商业模式下支持安全功能，ZigBee 路由器应支持 Config_Master_Key，并应在连接密钥的密钥建立程序中使用主要密钥。在列出一个远程目标地址，需要安全通信时，ZigBee 路由器应支持 APSME-ESTABLISH-KEY.request 来为远程设备建立一个连接密钥，且 APSME-ESTABLISH-KEY.indication 表示该请求到目的地，且应支持 APSME-ESTABLISH-KEY.confirm 和 APSME-ESTABLISH-KEY.response 来完成连接密钥的密钥建立。ZigBee 路由器应提供为请求安全通信的已知目的地存储连接密钥的能力，并应管理连接密钥的密钥存储。ZigBee 路由器应支持 APSME-TRANSPORT-KEY.indication 以从信任中心接收密钥。

如果商业或住宅模式支持安全操作，ZigBee 路由器应通过 APSME-REQUEST-KEY.request 请求信任中心更新其 NWK 密钥。

ZigBee 路由器应支持 NLME-PERMIT-JOINING.request 和 NLME-PERMIT-JOINING.confirm，以允许应用程序控制网络加入程序。

ZigBee 路由器应支持 3.2.2.30 小节的 NLME-NWK-STATUS.indication 以及每个这些通知的处理。

ZigBee 路由器应支持 NLME-LEAVE.request 和 NLME-LEAVE.confirm，使用 Config_NWK_Leave_removeChildren 属性，在适当的情况下，且在应用程序控制下，允许删除相关设备。导致删除相关设备的条件可以包括缺少安全证书，通过一个享有特权的应用程序或异常检测删除设备。当请求一个 ZigBee 终端设备离开网络时，NLME-LEAVE.request 原语的 ReuseAddress 参数值应为 TRUE，表示 NWK 层可以把之前离开设备使用的地址给随后加入网络的另一个终端设备。

ZigBee 路由器应处理来自于其它 ZigBee 设备的 Device_annce 信息。在收到一个 *nwkUseTreeRouting* 为 TRUE 的 Device_annce 时，ZigBee 路由器应为 PAN 中的设备检查持有 64 位 IEEE 地址的所有内部表，寻找 End_Device_annce 信息所提供的地址的一个匹配。最少，应检查 ZigBee 路由器持有的绑定表。如果检测到一个匹配，ZigBee 路由器应更新其 APS 信息模块地址映射条目，把相应的匹配 64 位 IEEE 地址，映射为 Device_annce 所含的更新后的 16 位 NWK 地址。在收到一个 *nwkUseTreeRouting* 为 FALSE 的 Device_annce 时，ZigBee 路由器使用 3.6.9 小节详细描述冲突解决程序。

ZigBee 路由器应维护一个当前相关设备的列表，并帮助支持孤扫描和重新加入程序，以使得之前的相关设备重新加入网络。

ZigBee 路由器可以在应用程序控制下从其它应用程序对象生成 APSME-AUTHENTICATE.request。并可以处理和响应来自其它设备的 APSME-AUTHENTICATE.indication。ZigBee 路由器必须提供 APSME-AUTHENTICATE.confirm 给请求已经被处理的那些应用程序对象。

ZigBee 路由器可以确定它丢失了与已经加入的网络的联系。在这种情况下，路由器建立一个主动扫描以找到该网络。如果多次找到网络，路由器在信标负载的 *nwkUpdateId* 值最近的时候尝试重新加入。

2.5.5.5.3 绑定表缓存操作

任何路由器（包括协调器）可以指定为一个主要绑定表缓存或一个备份绑定表缓存。

它应响应 System_Server_Discovery_req 原语，以使得其它设备发现它，并使用它的功能。

一个主要绑定表缓存应维护一个绑定表，和一个已登记设备的表，来缓存自己绑定表。

一个主要绑定表缓存应响应 Bind_Register_req 和 Replace_Device_req 原语，如 2.4.3.2 节所述。

如果一个备份绑定表缓存可以使用，一个主要绑定表缓存应使用附加的绑定管理原语来备份和恢复其绑定表和其源绑定设备表。

一个备份绑定表缓存应为一个或多个主要绑定表缓存维护绑定表和已注册绑定设备表的一个备份。它应支持绑定管理原语来备份和恢复这些表。

2.5.5.5.4 支持 Intra-PAN 可移植性的操作

2.5.5.5.4.1 概述

本节所描述的操作由 ZigBee 协调器和 ZigBee 路由器设备实施，以支持 Intra-PAN 的可移植性。主要步骤总结如下：

- 发现问题——移动设备的 ZDO 通过 NLME-NWK-STATUS.indication 原语，被告知一个失败确认，并确定一个问题。

- 实施 NWK 层重新加入程序——移动 ZED 的 ZDO 使用 NLME-JOIN.request 原语，或通过一个加密的或未加密的重新加入程序发起本程序。本程序密切反映了 MAC 连接程序。注意 ZigBee 路由器如果发现它们不再和信任中心有联系，也应周期性地实施本程序。
- 安全验证——描述了加密和解密协议步骤，以保证孤设备应该被接受。（参见 2.5.5.5.4.2 节）
- 通知网络其它设备——如果设备改变父设备，必须完成 3.6.1.9 小节地址冲突探测的步骤。这些行动也必须通知原来的父设备，终端设备已经更换了父设备。⁷

这些步骤详细描述在下面小节。ZED 加密重新加入机制和 ZED 不加密重新加入机制分别如图 2.108 和图 2.109 所示。ZR 不加密重新加入机制如图 2.110。注意在加密和不加密重新加入中 NWK 和 SEC 节（见 3.2.2.11, 3.2.2.12, 3.2.2.13, 3.6.1.4 和 4.6.3 小节）是这些步骤的权威文档。本节的图表只是为了图解。

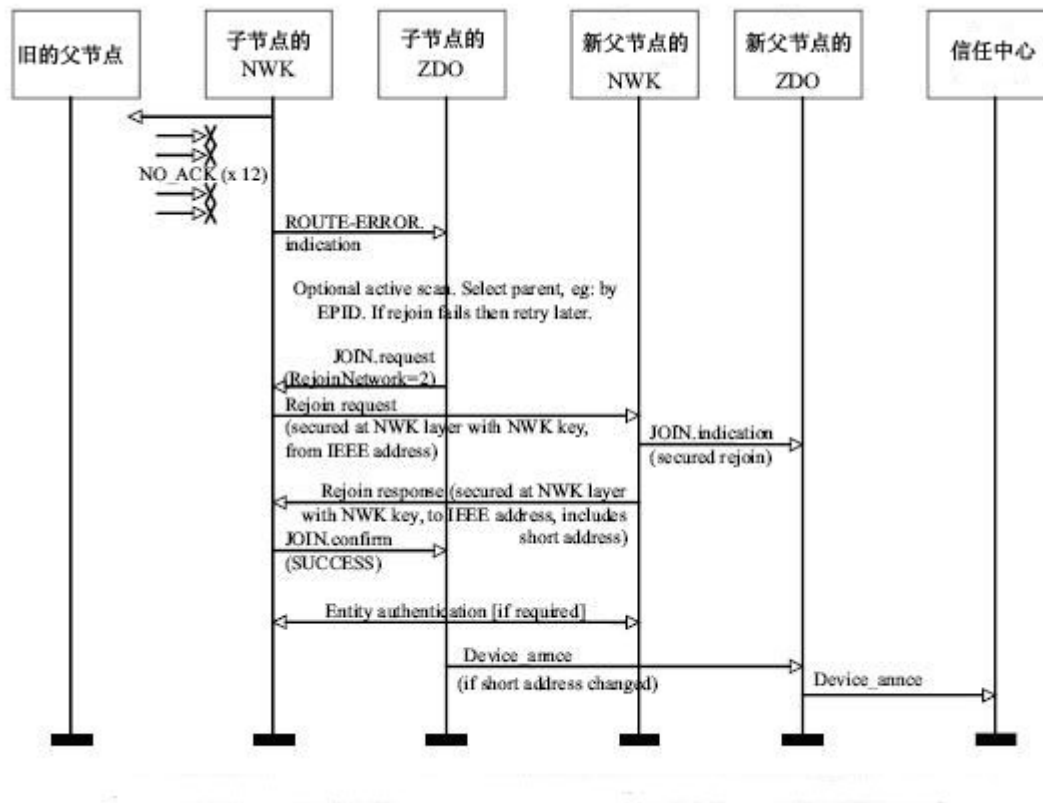


图 2.108 便携信息序列图：ZED 加密重新加入

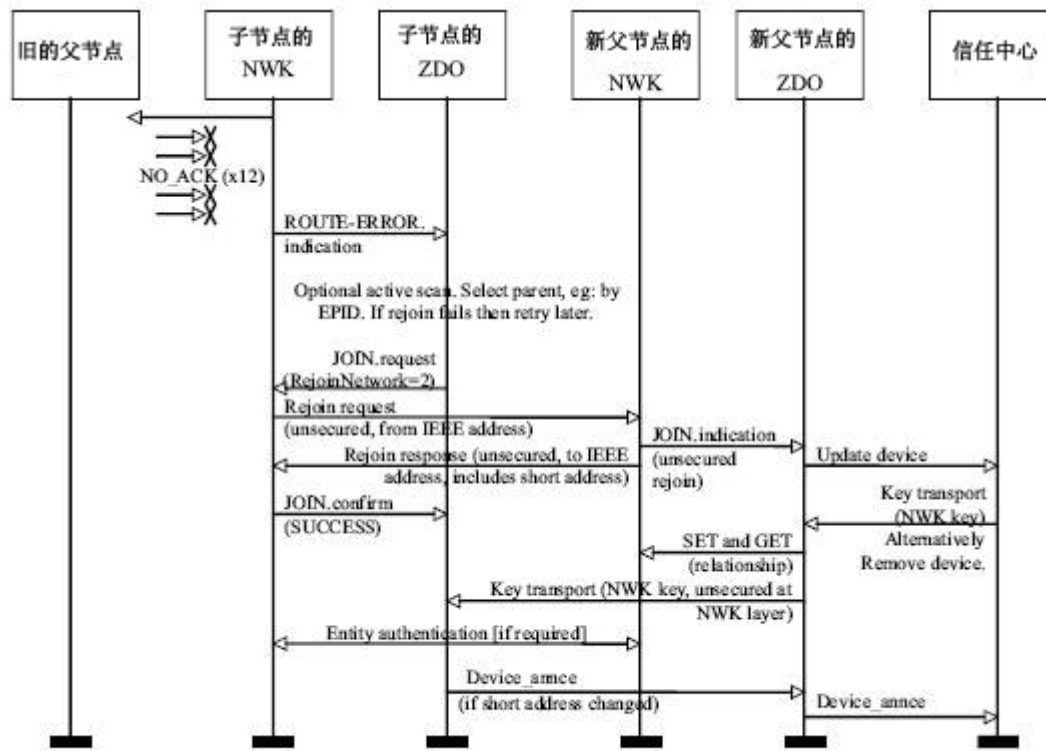


图 2.109 便携信息序列图：ZED 未加密重新加入

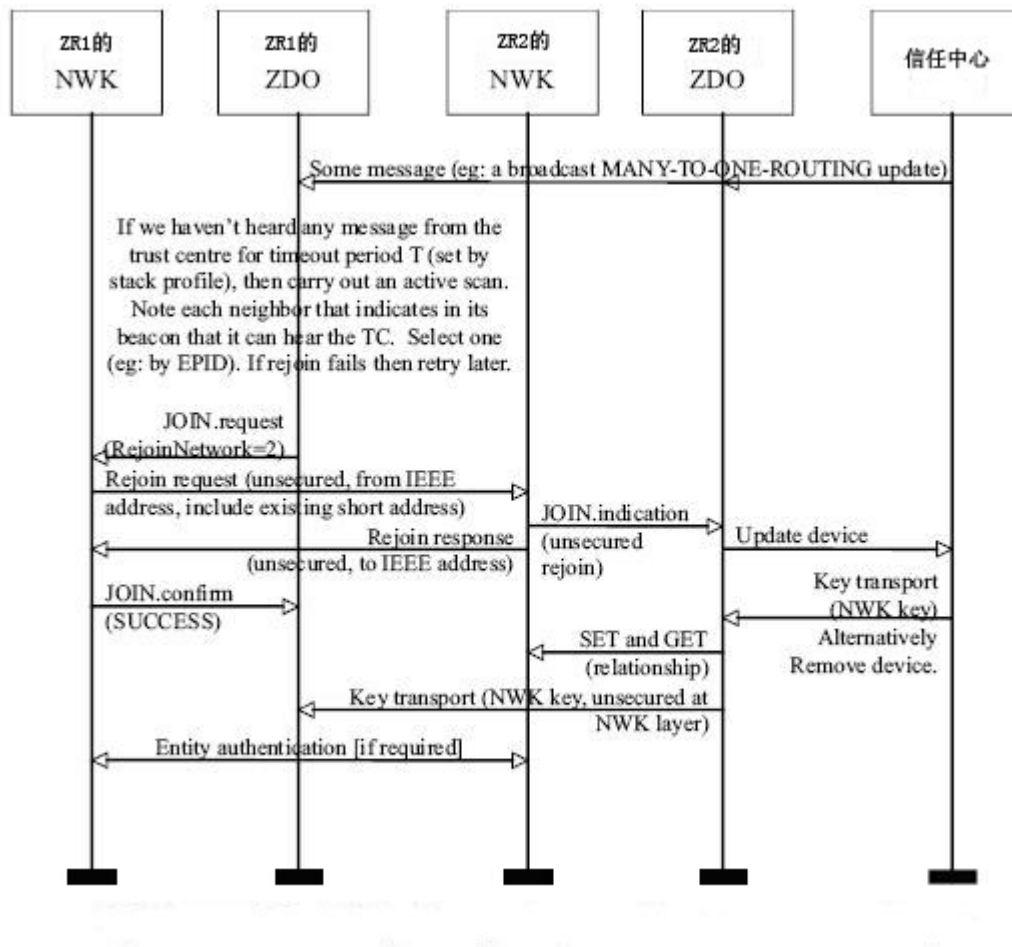


图 2.110 便携信息序列图：ZR 未加密重新加入

2.5.5.4.2 安全验证操作的描述

对于 MAC 连接,当 NLME 发出一个 NLME-JOIN.indication 原语时,一个 ZigBee 协调器或 ZigBee 路由器设备被告知一个重新加入的设备。这应该和连接指示的处理方式相同,除了一个加密重新加入的更新设备和密钥传输步骤。

直到下面描述的验证步骤已经实施,完整的网络操作是不允许的。

一个新(重新)加入的节点和其新的父节点应采取措施,来证实确实允许它在这个网络上。设想有两种情况:

- 一个或其它设备不是根据本规范执行的,并且不应该加入。这里描述的措施允许双方撤销在这种情况下的加入。

- 一个或其它设备是一个妥协/黑装置。在安全功能启用的情况下，4.6.3.6 小节的措施是另外适用的，以便撤销一个未经授权的加入。

该验证使用已有的命令实施。下面的 2.5.5.5.4.3 小节描述了一个给新的父节点的 `Device_annce` 命令的传输。新的父节点应检查这个或其它一些信息格式正确，且包括孤设备相应的地址域。如果安全功能启用，那么本命令应以网络密钥加密，且新的父节点应证实所有的安全程序已经正确实施。如果所有的这些检查成功，那么孤设备应加入网络，否则它应不在此时加入网络。正常情况下，从一个没有加入到网络的设备发来的信息不应转发给整个网络，并且命令不应被执行。相应的，孤设备只有从新的父节点收到至少一个正确格式的 ZigBee 信息时，才能加入网络。如果安全启用，这个信息必须必须用网络密钥加密，且所有的安全处理必须正确实施。如果信息不能在协议中交换，那么孤设备不应在此时加入网络。

2.5.5.5.4.3 通知网络其它设备的操作描述

如果 ZigBee 终端设备使用重新加入的孤程序重新加入一个新的父节点，它必须完成 3.6.1.9⁸ 小节所述的地址冲突程序。在收到一个 `Device_annce` 时，所有设备应检查其持有 PAN 内设备 64 位 IEEE 地址的内部表，寻找 `Device_annce` 所提供的地址的一个匹配。如果检测到一个匹配，设备应更新对应匹配 64 位 IEEE 地址的 NIB 属性 `nwkAddressMap`，以映射 `Device_annce` 所含的更新后的 16 位 NWK 地址。所有设备应使用 NLME-SET 和 NLME-GET 原语来更新 NWK NIB 的 `nwkNeighborTable`。这个 ZED 之前的父节点通过修改 `nwkNeighborTable` 的关系域为 0x04，删除作为其子节点之一的 ZED，这个状态导致一个状态代码为“Target Device Unavailable”的 NLME-NWKSTATUS.indication 原语（见 3.2.2.30 小节）。如果 `nwkUseTreeRouting` 是 TRUE，没有提供地址冲突检测，按照 intra-PAN 的可移植性，不允许父设备删除设备或任何其它操作，来重新发布用于一个不同 IEEE 地址的子节点的一个短地址。或者，如果 `nwkUseTreeRouting` 是 FALSE，提供了地址冲突检测，但是设备一般在 intra-PAN 的可移植性程序期间保存自己已有的 NWK 地址。而且，如果 NWK 地址在 intra-PAN 的可移植性期间改变，ZDO 应安排任何 IEEE 地址映射到短地址，它们对于在本设备上运行的应用程序是已知的。该行为是强制的，但是实现的机制在本规范的范围之外。

2.5.5.5.5 ZigBee 终端设备

2.5.5.5.5.1 初始化

执行时应把表 2.139 所示的有关启动的 IB 参数的值设置为反映期望的设备启动行为。特别是，IB *apsDesignatedCoordinator* 属性必须设置为 FALSE。

如果支持，应做出安排，来为复杂描述符，用户描述符，绑定条目的最大个数和主要密钥提供配置元素。这些元素应体现在:Config_Complex_Descriptor, :Config_User_Descriptor, :Config_Max_Bind 和:Config_Master_Key 中。如果设备应用程序设置 NLME-JOINRxOnWhenIdle 参数为 FALSE，终端设备应使用 2.5.2.1 小节所述的程序，来发现一个主要发现缓存设备，登记它并成功装载其设备和服务发现信息。为了方便装载发现信息到主要发现缓存设备，本地设备可以暂时增加到它的父节点的轮询率。在登记任何主要发现缓存设备之前，终端设备应使用发现节点缓存请求，来保证它之前没有登记在任何其它主要发现缓存设备中。如果一个服务器响应指示终端设备之前已经登记，终端设备应在该主要发现缓存设备中更新其发现缓存信息，或应从之前的登记删除其发现缓存信息，并创建一个新的登记。

要作为一个 ZigBee 终端设备启动，设备应用程序必须执行 2.5.5.5.6.2 小节所述的启动程序，启动参数的设置如上所述。这会影响 3.6.1.4.2 小节所述的网络重新加入程序，或 3.6.1.4.1 小节所述的通过 MAC 连接网络加入的整个程序的执行。NLME-NETWORK-DISCOVERY.request 应执行 :Config_NWK_Scan_Attempts，每次间隔时间是 :Config_NWK_Time_btwn_Scans。重复 NLME-NETWORK-DISCOVERY.request 的目的是为 NWK 层提供一个更准确的相邻表和相关链路质量指示。选择 PAN 的算法的规定留给 profile 描述，可以包括使用 Extended PAN ID、网络的操作模式、确定 PAN 上 ZigBee 路由器或协调器的身份、PAN 上从 ZigBee 协调器到 ZigBee 路由器的深度、ZigBee 路由器或协调器的容量、路由消耗，或协议版本号（这些参数由 NLME-NETWORK-DISCOVERY.confirm 和信标负载提供）。

ZigBee 终端设备在应用程序控制下，且如果设备支持向后兼容，它可以使用当前的协议版本号加入网络，或使用一个之前的协议版本号加入网络。一个 ZigBee PAN 应包括只使用一个协议版本号的设备（在同一个 PAN 内，使用不同协议版本号的设备和帧格式的网络是不允许的）。一个可选的配置属性:Config_NWK_alt_protocol_version 提供了设备可以选择使用的协议版本号，而不是当前协议版本号。一旦 ZigBee 终端设备选择了一个 PAN 和一个具体的协议版本号，它应使用协议版本号作为它的 *nwkProtocolVersion*。另外，然后 ZigBee 终端设备应遵守使用该协议版本号的 ZigBee 规范版本提供的所有帧格式和处理规则。

如果设备应用程序设置 NLME-JOIN RxOnWhenIdle 参数为 FALSE，:Config_NWK_indirectPollRate 应用于确定间接信息请求的轮询率。:Config_NWK_indirectPollRate 应根据该设备支持的应用程序 profile 建立的值来设置。一旦间接通信请求的轮询发起，如果检测到和父节点的通信失败，由间接信息请求:Config_Parent_Link_Threshold_Retry 连续尝试的失败次数来确定，设备应用程序应使用网络重新加入程序。

一旦终端设备成功加入网络，设备应发出一个 Device_annce，提供其 64 位 IEEE 地址和 16 位 NWK 地址。

应做出安排，在 ZigBee 设备对象完成初始化和转到正常运行状态之前，保证来自于 EP1 到 EP240 的最终应用程序的 APS 原语调用返回适当的错误状态值。

如果网络启用安全功能，设备等待信任中心对它进行验证，为了成功获取 NWK 密钥以作为一个终端设备在网络中开始运行。信任中心的操作详见 4.6.2 小节。

2.5.5.5.2 正常运行状态

如果设备应用程序设置 NLME-JOIN RxOnWhenIdle 参数为 FALSE，:Config_NWK_indirectPollRate 应用于为在正常运行状态下的间接传输轮询父节点。当收到一个分段的信息，设备可以暂时增加它的轮询率，保证至少每过 macTransactionPersistenceTime 秒轮询它的父节点。

ZigBee 终端设备应响应任何设备发现和服务发现操作，请求其自身的设备，使用 2.5.4 小节所述的属性。

如果在商业模式下启用安全功能，ZigBee 终端设备应支持:Config_Master_Key，且应在连接密钥的密钥建立程序中使用主要密钥。在列出一个远程目标地址，需要安全通信时，ZigBee 终端设备应支持 APSME-ESTABLISH-KEY.request 来为远程设备建立一个连接密钥，并支持 APSME-ESTABLISH-KEY.indication 来表示发给目的地的请求，并应支持 APSME-ESTABLISH-KEY.confirm 和 APSME-ESTABLISH-KEY.response 来完成连接密钥的密钥建立。ZigBee 终端设备应提供为需要安全通信的已知目的地存储连接密钥的能力，并应管理连接密钥的密钥存储。ZigBee 终端设备应支持 APSME-TRANSPORT-KEY.indication 从信任中心接收密钥。

如果在商业模式下启用安全，ZigBee 终端设备应请求信任中心通过 APSME-REQUEST-KEY.request 更新其 NWK 密钥。

ZigBee 终端设备应处理来自于其它 ZigBee 终端设备的 Device_annce 信息。在收到一个 *nwkUseTreeRouting* 为 TRUE 的 Device_annce 时，ZigBee 终端设备应为 PAN 中的设备检查持有 64 位 IEEE 地址的所有内部表，寻找 Device_annce 信息所提供的地址的一个匹配。如果检测到一个匹配，ZigBee 终端设备应更新对应匹配 64 位 IEEE 地址的 NIB *nwkAddressMap*，以映射 Device_annce 所含的更新后的 16 位 NWK 地址。⁹

ZigBee 终端设备应处理 NWK 层送来的 NLME-NEW-STATUS.indication。如果错误代码等于 0x09（父节点连接失败），ZED 将更新其 ZDO 中维护的失败计数器。如果失败计数器的值比:Config_Parent_Link_Retry_Threshold 属性小，ZED 可以根据 ZED 的应用程序，决定发出进一步的命令来尝试与父节点通信。如果失败计数器的值超过:Config_Parent_Link_Retry_Threshold 属性，那么 ZED 将准备启动重新加入程序。注意执行者可以选择使用更准确的时间窗口方案来确定一次连接失败。

9. CCB #842

重新加入程序非常密切的反映了 MAC 连接程序，但是允许一个设备重新加入一个不接受新连接的父节点。ZDO 可以使用 NLME-NETWORK-DISCOVERY.request 原语来检测潜在的替代父节点，并为了使恢复延迟和可靠性最优化，应根据该设备信标的以下信息选择一个合适的新的父节点：

- PAN ID
- EPID (扩展的 PAN ID)
- 信道
- 信号强度
- 潜在的父节点是否表示它当前可以与其信任中心通信
- 该设备是否最近加入这个父节点，或这个网络失败

一旦一个潜在的父节点已经选定，ZDO 应发出一个 NLME-JOIN.request 原语，RejoinNetwork 设置为 0x02。

重新加入程序的启动时间由最后一次 NLME-JOIN.request 原语发来的时间，和:Config_Rejoin_Interval 属性确定。仅在当前的和之前的 NLME-JOIN.request 之间的发送间隔时间长于:Config_Rejoin_Interval 时，才能发送一个新的 NLME-JOIN.request 原语。如果执行了一定数量的重试（或某个时间段已经过去）但是仍没有成功，应用程序可能希望逐渐增加:Config_Rejoin_Interval。:Config_Rejoin_Interval 应不超过:Config_Max_Rejoin_Interval。每次成功收到一个 NLME-JOIN.confirm，ZDO 应重置其失败计数器为 0，:Config_Rejoin_Interval 属性为其初始值。默认初始值的选择和增加重新加入间隔的算法应由应用程序确定，在本规范的范围之外。

如果 ZigBee 终端设备使用重新加入程序重新加入一个新的父节点，它应该完成 3.6.1.9 小节说明的地址冲突程序。¹⁰

ZigBee 终端设备在应用程序控制下可以从其它应用程序对象中产生 APSME-AUTHENTICATE.requests，并处理和响应来自其它设备的 APSME-AUTHENTICATE.indications。ZigBee 终端设备应提供 APSME-AUTHENTICATE.confirms 给那些请求已经被处理的应用程序对象。

2.5.5.5.6 调试应用的支持

领域中的 ZigBee 设备需要调试，它取决于开发者提供执行这些调试的应用。有一个风险是不同厂商的应用工作情况不同，因此降低了同一网络上来自不同厂商的 ZigBee 设备准确无误操作的能力。作为本问题解决方案的一部分，本小节列出了 ZigBee 设备一组通用的配置参数，并概述了设备在启动时使用的一般程序。本解决方案的其它关键组成部分是一组通用的调试协议和程序，这些在本文件范围之外。

10. CCB #833

2.5.5.5.6.1 配置参数

2.5.5.5.6.2 小节中概述的启动程序的目的是这样一种方式，通过一贯使用这一程序，设备可以经过调试的所有阶段加入到合适的 ZigBee 网络，能够发送和接收应用数据的传输。后期的调试，包括绑定和组成员的调试在下面 2.5.5.5.6.3 小节简要讨论。使用系统参数的程序见表格 2.139。

表 2.139 启动参数

名称	参考	评论
nwkExtendedPANID	表 3.43	这是设备要加入网络的扩展 PANID，如果值为 0x0000000000000000，那么设备没有连接到网络。
apsDesignatedCoordinator	表 2.24	这是布尔标志，指示设备启动时是否假定必须成为一个 ZigBee 协调器
apsChannelMask	表 2.24	这是一个掩码，包含允许的信道，设备可以尝试去形成或加入一个网络。
apsUseExtendedPANI	表 2.24	加入或形成的网络的 64 位标识符
apsUseInsecureJoin	表 2.24	布尔标志，默认为 TRUE，指示启动时能否使用未加密加入

2.5.5.5.6.2 启动程序

启动程序使用 2.5.5.5.6.1 小节中的参数去执行设备上 ZigBee 网络设施的受控制启动。在设备重启时，程序必须运行，但也可以在应用程序控制下按照开发者的意愿运行。

当设备启动，应该检查 *nwkExtendedPANID* 的值。如果 *nwkExtendedPANID* 有一个非零值，那么设备应该假设它有网络运行所需的所有网络参数。注意设备应该假定信道标识符在它当前网络参数中存在，但如果 *nwkExtendedPANID* 没有找到，可能需要扫描 *ChannelMask*。为了在电源失败和处理器复位期间这个工作能有效进行，*nwkExtendedPANID* 应该放在非易失性存储器中。¹¹

如果该设备发现它没有连接到网络，那么它应该检查 *apsDesignatedCoordinator* 的值。如果此参数的值为 TRUE，那么设备应遵循启动一个网络的程序，见 3.6.1.4.1 小节，并应该使用 *apsChannelMask* 作为 NLME-NETWORK-FORMATION.request 原语 ScanChannels 参数的值，并设置 *nwkExtendedPANID* 的值为 *apsUseExtendedPANID* 给定的值，如果 *apsUseExtendedPANID* 有一个非零值。

如果设备不被指定为协调器，且 *apsUseExtendedPANID* 有一个非零值，设备应该尝试重新加入 *apsUseExtendedPANID* 指定的网络。为此，应该使用 NLME-JOIN.request，ExtendedPANID 参数等于 *apsUseExtendedPANID* 的值，该原语的 ScanChannels 参数等于 *apsChannelMask* 配置参数的值。NLME-JOIN.request 原语的 RejoinNetwork 参数应该值为 0x02 指示重新加入。

如果网络重新加入的尝试失败，AIB *apsUseInsecureJoin* 属性的值为 TRUE，设备应该遵从 3.6.1.4.1 小节的程序来加入一个网络，在 ScanChannels 掩码需要的地方使用 *apsChannelMask*。如果 *apsUseExtendedPANID* 有一个非零值，设备应该加入指定的网络，如果发现网络不可用，程序失败。如果 *apsUseExtendedPANID* 等于 0x0000000000000000，那么设备应该加入最好的可用网络。

2.5.5.5.6.3 进一步调试

一旦一个设备连接在网络上，可以和网络上其它设备以一种加密的方式进行通信，就可以进行其它的调试。其它适用于调试的项目见表 2.140。

表 2.140 更多调试参数

名称	参考	评论
apsBindingTable	表 2.24	本设备的绑定表。绑定提供了一种分离的关注，应用可以操作时无需为它们发出的帧管理接收者地址信息。这个信息可以在调试时输入，设备的主应用程序甚至觉察不到。
nwkGroupIDTable	表 3.43	调试应用程序可以通过访问这个表，管理设备及其端点的组成员。
nwkSecurityMaterialSet	表 4.2	该集合包含网络密钥材料，调试应用程序应该可以访问。
apsDeviceKeyPairSet	表 4.28	包含在密钥对的集合中，对于一个设备是信任中心主密钥，调试应用程序应该可以访问。
apsTrustCenterAddress	表 4.28	信任中心的 IEEE 地址
nwkShortAddress	表 3.43	只要可能产生地址冲突，调试应用可以设置网络的短地址，地址冲突解决的结果见 3.6.1.9 小节

2.5.5.6 设备和服务发现

设备和服务发现功能支持：

- 设备发现
- 服务发现

设备管理使用 ZigBee 设备 Profile（参见 2.4 节）执行以上功能。

2.5.5.6.1 设备和服务发现内可选的和强制的属性

设备和服务发现对象内所有请求的属性对于所有 ZigBee 逻辑设备类型是可选的。表 2.141 列为强制的响应对于所有 ZigBee 逻辑设备类型是强制的，列为可选的响应对于所有的 ZigBee 逻辑设备类型是可选的。关于这些属性的任何描述参见 2.4 节。

表 2.141 设备和服务发现属性

属性	M/O	类型
NWK_addr_req	O	Public
NWK_addr_rsp	M	Public
IEEE_addr_req	O	Public
IEEE_addr_rsp	M	Public
Node_Desc_req	O	Public
Node_Desc_rsp	M	Public
Power_Desc_req	O	Public
Power_Desc_rsp	M	Public
Simple_Desc_req	O	Public
Simple_Desc_rsp	M	Public
Active_EP_req	O	Public
Active_EP_rsp	M	Public
Match_Desc_req	O	Public
Match_Desc_rsp	M	Public
Complex_Desc_req	O	Public
Complex_Desc_rsp	O	Public
User_Desc_req	O	Public
User_Desc_rsp	O	Public
Device_annce	M	Public
User_Desc_set	O	Public
User_Desc_conf	O	Public
System_Server_Discovery_req	O	Public
System_Server_Discovery_rsp	O	Public
Discovery_Cache_req	O	Public

表 2.141 设备和服务发现属性(续表)

属性	M/O	类型
Discovery_Cache_rsp	O	Public
Discovery_store_req	O	Public
Discovery_store_rsp	O	Public
Node_Desc_store_req	O	Public
Node_Desc_store_rsp	O	Public
Power_Desc_store_req	O	Public
Power_Desc_store_rsp	O	Public
Active_EP_store_req	O	Public
Active_EP_store_rsp	O	Public
Simple_Desc_store_req	O	Public
Simple_Desc_store_rsp	O	Public
Remove_node_cache_req	O	Public
Remove_node_cache_rsp	O	Public
Find_node_cache_req	O	Public
Find_node_cache_rsp	O	Public

2.5.5.7 安全管理

安全管理确定是否启用或禁用安全功能，以及如果启用的话，应执行以下步骤：

- 建立密钥
- 传输密钥
- 验证

2.5.5.7.1 安全管理内可选的和强制的属性

安全管理本身对于所有 ZigBee 设备类型是可选的对象。如果安全管理列出，所有请求和响应对应于所有 ZigBee 设备类型是强制的。如果安全管理没有列出，安全管理的属性不在任何 ZigBee 逻辑设备类型中列出。关于表 2.142 所列的原语的任何描述参见 2.4 节。

表 2.142 安全管理属性

属性	M/O	类型
APSME-ESTABLISH-KEY.request	O	Public
APSME-ESTABLISH-KEY.response	O	Public
APSME-ESTABLISH-KEY.indication	O	Public
APSME-ESTABLISH-KEY.confirm	O	Public
APSME-TRANSPORT-KEY.request	O	Public
APSME-TRANSPORT-KEY.indication	O	Public
APSME-UPDATE-DEVICE.request	O	Public
APSME-UPDATE-DEVICE.indication	O	Public
APSME-REMOVE-DEVICE.request	O	Public
APSME-REMOVE-DEVICE.indication	O	Public
APSME-REQUEST-KEY.request	O	Public
APSME-REQUEST-KEY.indication	O	Public
APSME-SWITCH-KEY.request	O	Public
APSME-SWITCH-KEY.indication	O	Public
APSME-AUTHENTICATE.request	O	Private
APSME-AUTHENTICATE.indication	O	Private
APSME-AUTHENTICATE.confirm	O	Private

2.5.5.8 绑定管理

绑定管理功能支持：

- 终端设备绑定
- 绑定和取消绑定

一旦指令到达支持绑定表的 ZigBee 协调器或路由器，或者支持绑定表的终端设备，绑定管理使用 ZigBee 设备 Profile 命令和 APSME-SAP 原语执行以上功能，来提交/删除绑定表条目。

2.5.5.8.1 绑定管理内可选的和强制的属性

绑定管理对于所有 ZigBee 设备类型是可选的对象。

如果绑定管理列出，所有请求对于所有 ZigBee 逻辑设备类型是可选的。另外，ZigBee 协调器应支持响应，或 ZigBee 协调器，以及那些设备中持有的绑定表条目的源地址相对应的 ZigBee 终端设备应支持响应。

如果绑定管理没有列出，对于所有 ZigBee 逻辑设备类型不应支持所有请求和所有响应。表 2.143 总结了绑定管理属性。

表 2.143 绑定管理属性

属性	M/O	类型
End_Device_Bind_req	O	Public
End_Device_Bind_rsp	O	Public
Bind_req	O	Public
Bind_rsp	O	Public
Unbind_req	O	Public
Unbind_rsp	O	Public
Bind_Register_req	O	Public
Bind_Register_rsp	O	Public
Replace_Device_req	O	Public
Replace_Device_rsp	O	Public
Store_Bkup_Bind_Entry_req	O	Public
Store_Bkup_Bind_Entry_rsp	O	Public
Remove_Bkup_Bind_req	O	Public
Remove_Bkup_Bind_rsp	O	Public
Backup_Bind_Table_req	O	Public
Backup_Bind_Table_rsp	O	Public
Recover_Bind_Table_req	O	Public
Recover_Bind_Table_rsp	O	Public
Backup_Source_Bind_req	O	Public
Backup_Source_Bind_rsp	O	Public
Recover_Source_Bind_req	O	Public
Recover_Source_Bind_rsp	O	Public
APSME-BIND.request	O	Private

表 2.143 绑定管理属性

属性	M/O	类型
APSME-BIND.confirm	O	Private
APSME-UNBIND.request	O	Private
APSME-UNBIND.confirm	O	Private

2.5.5.9 网络管理

网络管理功能支持：

- 网络发现
- 网络形成
- 允许/禁止连接
- 连接和断开
- 路由发现
- 网络复位
- 无线设备接收器状态启用/禁用
- 获得和设置网络管理信息模块数据
- 检测和报告干扰
- 如果特殊模式被确定为整个 PAN 的网络管理器,接受网络干扰报告并改变网络信道

网络管理使用 NLME-SAP 原语（参见第 3 章）执行以上功能。

2.5.5.9.1 网络管理内可选的和强制的属性

网络管理对象对于所有 ZigBee 设备类型是强制的。

网络发现，获得和设置属性（请求和确认）对于所有 ZigBee 逻辑设备类型是强制的。

如果 ZigBee 逻辑设备类型是 ZigBee 协调器,应支持 NWK 形成请求和确认,NWK 离开请求,NWK 离开指示,NWK 离开确认,NWK 加入指示,NWK 允许加入请求,以及 NWK 允许加入确认。可能支持 NWK 直接加入请求,NWK 直接加入确认,NWK 路由发现请求和 NWK 路由发现确认。应不支持 NWK 加入请求和 NWK 加入响应。

如果 ZigBee 逻辑设备类型是 ZigBee 路由器, 应不支持 NWK 形成请求和确认。另外, 应支持 NWK 启动路由器请求, NWK 启动路由器确认, NWK 加入请求, NWK 加入确认, NWK 加入指示, NWK 离开请求, NWK 离开确认, NWK 离开指示, NWK 允许加入请求和 NWK 允许加入确认。可能支持 NWK 直接加入请求, NWK 直接加入确认, NWK 路由器发现请求和 NWK 路由器发现确认。

如果 ZigBee 逻辑设备类型是 ZigBee 终端设备, 应不支持 NWK 形成请求和确认以及 NWK 启动路由器请求和确认。另外, 应不支持 NWK 加入指示, NWK 离开指示和 NWK 允许加入请求。应支持 NWK 加入请求, NWK 加入确认, NWK 离开请求, NWK 离开确认。

对于所有的 ZigBee 逻辑设备类型, NWK 同步请求, 指示和确认以及 NWK 复位请求和确认, 以及 NWK 路由发现请求和确认应是可选的。表 2.144 总结了网络管理属性。关于表 2.144 所列的任何原语的描述参见第 3 章。

对于所有的 ZigBee 逻辑设备类型, 应支持 NWK 网络状态指示的接收, 但是在规范的本版本中不需要采取行动。

表 2.144 网络管理属性

属性	M/O	类型
NLME-GET.request	M	Private
NLME-GET.confirm	M	Private
NLME-SET.request	M	Private
NLME-SET.confirm	M	Private
NLME-NETWORK-DISCOVERY.request	M	Public
NLME-NETWORK-DISCOVERY.confirm	M	Public
NLME-NETWORK-FORMATION.request	O	Private
NLME-NETWORK-FORMATION.confirm	O	Private
NLME-START-ROUTER.request	O	Private
NLME-START-ROUTER.confirm	O	Private
NLME-JOIN.request	O	Private
NLME-JOIN.confirm	O	Private
NLME_JOIN.indication	O	Private
NLME-PERMIT-JOINING	O	Public
NLME-PERMIT-JOINING.confirm	O	Public

表 2.144 网络管理属性（续表）

属性	M/O	类型
NLME-DIRECT-JOIN.request	O	Public
NLME-DIRECT-JOIN.confirm	O	Public
NLME_LEAVE.request	M	Public
NLME_LEAVE.confirm	M	Public
NLME_LEAVE.indication	M	Public
NLME-RESET.request	O	Private
NLME-RESET.confirm	O	Private
NLME-SYNC.request	O	Public
NLME-SYNC.indication	O	Public
NLME-SYNC.confirm	O	Public
NLME-NWK-STATUS.indication	M	Private
NLME-ROUTE-DISCOVERY.request	O	Public
NLME-ROUTE-DISCOVERY.confirm	O	Private
NLME-ED-SCAN.request	O	Private
NLME-ED-SCAN.confirm	O	Private
NLME-START-BACKOFF.request	O	Private

网络中一个设备可以成为网络信道管理器。网络信道管理器操作见附录 E 描述。所有其它设备负责追踪信息传输失败和报告干扰，和附录 E 一致。

2.5.5.10 节点管理

节点管理支持请求和响应管理功能的能力。这些管理功能只提供了外部的设备可见性，关于接收请求的设备操作状态。

2.5.5.10.1 节点管理内的可选的和强制的属性

节点管理对于所有的 ZigBee 设备类型是可选的对象。如果节点管理对象列出，节点管理内所有请求和响应属性也是可选的。表 2.145 总结了节点管理属性。关于这些属性的描述参见 2.4 节。

表 2.145 节点管理属性

属性	M/O	类型
Mgmt_NWK_Disc_req	O	Public
Mgmt_NWK_Disc_rsp	O	Public
Mgmt_Lqi_req	O	Public
Mgmt_Lqi_rsp	O	Public
Mgmt_Rtg_req	O	Public
Mgmt_Rtg_rsp	O	Public
Mgmt_Bind_req	O	Public
Mgmt_Bind_rsp	O	Public
Mgmt_Leave_req	O	Public
Mgmt_Leave_rsp	O	Public
Mgmt_Direct_Join_req	O	Public
Mgmt_Direct_Join_rsp	O	Public
Mgmt_Permit_Joining_req	O	Public
Mgmt_Permit_Joining_rsp	O	Public
Mgmt_Cache_req	O	Public
Mgmt_Cache_rsp	O	Public
Mgmt_NWK_Update_req	O	private
Mgmt_NWK_Update_notif y	O	private

2.5.5.11 组管理器

组管理器提供把应用程序对象纳入组，或者把应用程序对象移出组的功能。组管理功能只在本地设备的应用程序对象上操作。管理其它设备上的组的机制不在本文件的范围之内。

2.5.5.11.1 组管理内可选的和强制性的属性

组管理对于所有的 ZigBee 设备类型是一个可选的对象。如果组管理对象列出，组管理内所有的请求和响应属性都是可选的。表 2.146 总结了组管理的属性。这些属性的描述见 2.4 节。

表 2.146 组管理属性

属性	M/O	类型
APSME-ADD-GROUP.request	O	Public
APSME-ADD-GROUP.confirm	O	Public
APSME-REMOVE-GROUP.request	O	Public
APSME-REMOVE-GROUP.confirm	O	Public
APSME-REMOVE-ALLGROUPS.request	O	Public
APSME-REMOVE-ALL GROUPS.confirm	O	Public

2.5.6 配置属性

该属性用来表示用于一个设备配置属性的最少的强制的和/或可选的属性。

表 2.147 配置属性

属性	M/O	类型
:Config_Node_Descriptor	M	Public
:Config_Power_Descriptor	M	Public
:Config_Simple_Descriptors	M	Public
:Config_NWK_Mode_and_Params	M	Public
:Config_NWK_Scan_Attempts	M	Private
:Config_NWK_Time_btwn_Scans	M	Private
:Config_Complex_Descriptor	O	Public
:Config_User_Descriptor	O	Public
:Config_Max_Bind	O	Private
:Config_Master_Key	O	Private
:Config_EndDev_Bind_Timeout	O	Private
:Config_Permit_Join_Duration	O	Public
:Config_NWK_Security_Level	O	Private
:Config_NWK_Secure_All_Frames	O	Private
:Config_NWK_Leave_removeChildren	O	Private
:Config_NWK_BroadcastDeliveryTime	O	Private

表 2.147 配置属性（续表）

属性	M/O	类型
:Config_NWK_TransactionPersistenceTime	O	Private
:Config_NWK_indirectPollRate	O	Private
:Config_Max_Assoc	O	Private
:Config_NWK_Join_Direct_Addrs	O	Public
:Config_Parent_Link_Retry_Threshold	O	Public
:Config_Rejoin_Interval	O	Public
:Config_Max_Rejoin_Interval	O	Public

2.5.6.1 配置属性定义

表 2.148 配置属性定义

属性	描述	当更新时
:Config_Node_Descriptor	这个设备节点描述符的内容(见2.3.2.3小节)	: Config_Node_Descriptor或者在应用程序首次加载时创建，或者设备在网络中开始操作时使用调试工具时之前初始化时创建。它用于服务发现来描述外部查询设备的节点的功能
:Config_Power_Descriptor	这个设备电源描述符的内容(见2.3.2.4小节)	: Config_Power_Descriptor或者在应用程序首次加载时创建，或者设备在网络中开始操作时使用调试工具时之前初始化时创建。它用于服务发现来描述外部查询设备的电源的功能
:Config_Simple_Descriptors	这个设备每个活动端点的简单描述符的内容(见2.3.2.5小节)	: Config_Simple_Descriptors在应用程序首次加载时创建并且被视为“只读”。简单描述符用于描述外部查询设备的接口功能

表 2.148 配置属性定义（续表）

属性	描述	当更新时
:Config_NWK_Scan_Attempts	整型值，表示NWK层确定连接哪个ZigBee协调器或路由器之前，尝试扫描的次数(见2.5.5.5小节)。 该属性有一个默认值5并且有效值在1到255之间	:Config_NWK_Scan_Attempts在ZDO中使用，来调用NLME-NETWORKDISCOVERY.request 原语指明的次数（为路由器和终端设备）。
:Config_NWK_Time_btwn_Scans	整型值，代表通过 :Config_NWK_Scan_Attempts描述的每次网络发现之间的时间延时（单位毫秒）(见2.5.5.5小节)。 属性有一个默认值100 (毫秒) 并且有效范围在 1 到65535 (毫秒)之间	:Config_NWK_Time_btwn_Scans在 ZDO中使用，提供NLME-NETWORKDISCOVERY.request 尝试之间的一个延时
:Config_Complex_Descriptor	（可选）该设备复杂描述符的内容（见2.3.2.6小节）	:Config_Complex_Descriptor当应用程序首次加载或者当设备在网络中开始操作之前使用调试工具初始化被创建，它用于服务发现，为外部查询设备提供描述扩展设备功能。
:Config_User_Descriptor	（可选）该设备用户描述符的内容（见2.3.2.7小节）	:Config_User_Descriptor当应用程序首次加载或者当设备在网络中开始操作之前使用调试工具初始化被创建。它用于服务发现，为该设备到外部查询设备提供一个可描述的字符串

表 2.148 配置属性定义（续表）

属性	描述	当更新时
:Config_Max_Bind	常数，描述绑定目录允许的最大个数	:Config_Max_Bind 是该设备绑定表目录支持的最大数
:Config_Master_Key	如果设备启用安全，使用的主要密钥(见第4章)。	:Config_Master_Key当前当应用程序第一次加载时出现或者当设备在网络中开始操作时，用调试工具初始化。如果设备支持和启用安全，它用于安全操作。
:Config_EndDev_Bind_Timeout	终端设备绑定使用的超时值（见2.4.3.2小节）	:Config_EndDev_Bind_Timeout只被ZigBee协调器使用，并且用于决定终端设备绑定请求是否在超时窗口内已经收到
:Config_Permit_Join_Duration	允许加入持续值，通过NLME-PERMITJOINING.request原语来设置(第3章)。	:Config_Permit_Join_Duration默认值是0x00，然而，该值可以根据profile不同的需要来确定。
:Config_NWK_Security_Level	网络的安全级别(见第3章)。	该属性只用于信任中心，并且用于设置网络安全级别
:Config_NWK_Secure_All_Frames	所有网络帧是否都应该被加密(见第3章)。	该属性只用于信息中心，并且用于确定网络层安全是否适用于网络中的所有帧
:Config_NWK_Leave_removeChildren	设置策略，即如果设备通过NLME-LEAVE被要求离开网络，子设备是否被移除(见第3章)。	设置该参数的策略在栈Profile使用中能找到
:Config_NWK_BroadcastDeliveryTime	见表3.57	该配置属性的值在栈Profile中被确定

表 2.148 配置属性定义（续表）

属性	描述	当更新时
:Config_NWK_Transaction PersistenceTime	见表 3.44。 该属性对ZigBee 协调器和ZigBee 路由器是强制的, 并且不用于ZigBee 终端设备。	该配置属性的值在栈Profile中被确定
:Config_NWK_Alt_protocol_version	设置协议版本号列表, 不是当前协议版本号, 设备可以选择在加入的个域网中使用它。 该属性仅适用于ZigBee路由器或者终端设备;列表中的协议版本号必须是早期的ZigBee规范版本。	:Config_NWK_Alt_protocol_version 允许ZigBee路由器和ZigBee终端设备加入发现使用早期的ZigBee规范版本的网络; 因为该参数是可选的, 当只要求当前ZigBee规范时, 设备也可以忽略该属性; 如果需要只含在当前规范中的某种功能, 或设备限制了代码大小, 该属性被忽略。
:Config_NWK_indirectPoll Rate	设置从父设备请求直接传输信息的设备的轮询率, 单位毫秒	该属性值通过部署在设备上的应用程序profile来确定
:Config_Max_Assoc	设置到父路由器或者协调器的最大允许关联数, 可以是路由器,终端设备, 或两者皆可。	该属性值通过设备使用的栈profile来确定。注意对于一些栈profile, 可以分别给允许路由器关联最大个数和允许终端设备关联最大数提供一个值。

表 2.148 配置属性定义（续表）

属性	描述	当更新时
:Config_NWK_Join_Direct_Addrs	包括以下域: DeviceAddress - 直接加入设备的64位IEEE地址 CapabilityInformation-直接加入设备的操作能力 MasterKey –如果启用安全, 用于新设备的密钥对描述符的主要密钥 (见表4.37) 详见3.2.2.14小节	:Config_NWK_Join_Direct_Addrs允许ZigBee协调器或路由器用直接加入地址列表进行预配置
:Config_Parent_Link_Retry_Threshold	父连接重试阈值的内容 (见2.5.5.5.5.2小节)	:Config_Parent_Link_Retry_Threshold在应用程序首次加载时, 或者使用调试工具初始化时创建。它用于ZED决定在重新加入进程初始化之前它应该重新连接父路由器的次数
:Config_Rejoin_Interval	重新加入间隔的内容 (见2.5.5.5.5.2小节)	:Config_Rejoin_Interval在应用程序首次加载时创建, 或者用调试工具初始化时创建。它用于ZED决定多久应该重新启动进程
:Config_MAX_Rejoin_Interval	最大重新加入间隔的内容 2.5.5.5.5.2)	:Config_MAX_Rejoin_Interval在应用程序加载时创建, 或者调试程序初始化时创建。它用于ZED设置重新加入程序期间:Config_Rejoin_Interval允许的最大值。

版权声明

郑州新双恒信息技术有限公司拥有本译文的版权。本译文免费供大家一起学习交流。译文出现的错误及误差，您可以通过以下方式联系本公司，我们将非常感谢。欢迎在网络上转载本译文，但必须保证本译文的完整，否则本公司不承担任何后果。如果你在转载的时候，需要对本译文进行修改或重新编辑，请征得本公司的同意。谢谢！

网址：<http://www.zigbee-sh.cn>

QQ 群：83028739



蜂舞™
FENGWU

第三章

网络规范

3.1 概述

3.1.1 网络（NWK）层概述

网络层必须提供功能，以保证 IEEE802.15.4-2003 MAC 子层的正确操作，并为应用层提供一个合适的服务接口。要和应用层通信，网络层的概念包括两个服务实体，提供必要的功能。这些服务实体是数据服务和管理服务。NWK 层数据实体（NLDE）通过其相关的 SAP，NLDE-SAP，提供了数据传输服务，而 NLME-SAP 提供了管理服务。NIME 使用 NLDE 来获得它的一些管理任务，且它还维护一个管理对象的数据库，叫做网络信息库（NIB）。

3.1.1.1 网络层数据实体（NLDE）

NLDE 应提供一个数据服务，以允许一个应用程序在两个或多个设备之间传输应用协议数据单元（APDU）。设备本身必须位于同一个网络。

NLDE 将提供以下服务：

- 生成网络级别的 PDU（NPDU）：NLDE 应该可以通过增加一个合适的协议头，从一个应用支持子层的 PDU，生成一个 NPDU。
- 拓扑指定的路由：NLDE 应该可以传输一个 NPDU 给一个合适的设备，它是通信的最终目的地或是通信链中朝向最终目的地的下一步。

- 安全：保证传输的真实性和保密性的能力。

3.1.1.2 网络层管理实体（NLME）

NLME 应提供一个管理服务，以允许一个应用程序与协议栈相互作用。

NLME 应提供以下服务：

- 配置一个新设备：为所需的操作充分配置协议栈的功能。配置选项包括开始一个作为一个 ZigBee 协调器的操作，或加入一个已存在的网络。
- 开始一个网络：建立一个新的网络的功能。
- 加入、重新加入和离开一个网络：加入、重新加入或离开一个网络的功能，以及为一个 ZigBee 协调器或 ZigBee 路由器请求一个设备离开网络的功能。
- 寻址：ZigBee 协调器和路由器给新加入网络的设备分配地址的能力。
- 邻居发现：发现，记录和报告信息关于设备的单跳邻居的能力。
- 路由发现：发现并记录通过网络的路径的功能，即信息可以有效地传送。
- 接收控制：一个设备控制何时接收者是激活的，以及激活多长时间，从而使 MAC 子层同步或直接接收。
- 路由：这个是使用不同路由机制的能力，例如单播，广播，多播或者多对一，在网络中高效交换数据。

3.2 服务规范

图 3.1 描述了 NWK 层的组件和接口。

NWK 层提供了两个服务，通过两个服务访问点（SAP）访问。这些是 NWK 数据服务，通过 NWK 层数据实体 SAP(NLDE-SAP)访问和 NWK 管理实体，通过 NWK 层管理实体 SAP(NLME-SAP)访问。这两个服务通过 MCPS-SAP 和 MLME-SAP 接口，提供了应用层和 MAC 子层之间的接口（参见[B1]）。除了这些外部接口，在 NLME 和 NLDE 之间还有一个隐藏的接口，允许 NLME 使用 NWK 数据服务。

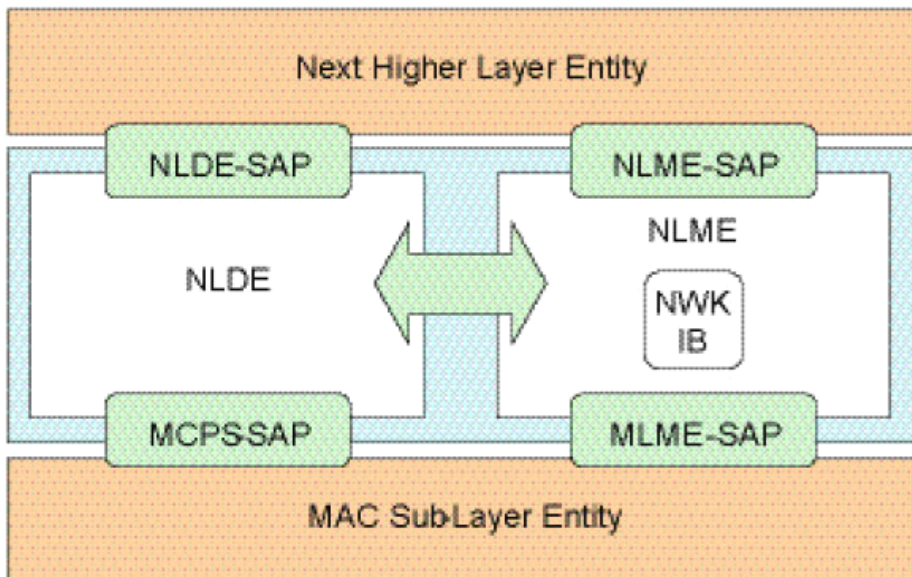


图 3.1 网络层参考模型

3.2.1 NWK 数据服务

NEK 层数据实体 SAP（NLDE-SAP）支持在对等的应用程序实体之间传输应用协议数据单元（APDU）。

表 3.1 列出了 NLDE-SAP 支持的原语以及讨论这些原语的章节。

表 3.1 NLDE-SAP 原语

NLDE-SAP原语	请求	确认	指示
NLDE-DATA	3.2.1.1	3.2.1.2	3.2.1.3

3.2.1.1 NLDE-DATA.request

本原语请求传输一个数据 PDU（NSDU），从本地 APS 子层实体到一个或多个对等的 APS 子层实体。

3.2.1.1.1 服务原语的语义

本原语的语义如下：

NLDE-DATA.request	{	DstAddrMode, DstAddr, NsduLength, Nsdu, NsduHandle, Radius, NonmemberRadius, DiscoverRoute, SecurityEnable 	}
-------------------	---	--	---

表 3.2 指明了 NLDE-DATA.request 原语的参数。

表 3.2 NLDE-DATA.request 参数

名称	类型	有效范围	描述
DstAddrMode	Integer	0x01 or 0x02	通过DstAddr提供的目的地址类型；这可能有以下两个值中的一个： 0x01=16位组播地址 0x02=设备的16位网络地址或者一个16位广播地址
DstAddr	16-bit Address	0x0000-0xFFFF	目的地址
NsduLength	Integer	< aMaxMACFrameSize-nwkcMinHeaderOverhead	组成转移NSDU的字节数。考虑到ZigBee网络层不使用扩展的寻址模式，这已被IEEE802.15.4规范中的aMaxMACFrameSize限定所修改。这使得头闲置的部分被用于有效载荷。参见小节D.4。
Nsdu	Set of Octets	-	组成转移NSDU的套接字
NsduHandle	Integer	0x00 – 0xff	与NSDU相联系的句柄是通过网络层实体传输的

表 3.2 NLDE-DATA.request 参数（续表）

名称	类型	有效范围	描述
Radius	Unsigned Integer	0x00 – 0xff	间隔, 跳数, 帧允许通过网络传输
NonmemberRadius	Integer	0x00 – 0x07	间隔, 跳数, 组播帧将通过不是组成员的节点传送; 0x07 被视为无穷
DiscoverRoute	Integer	0x00 – 0x01	DiscoverRoute参数可能用于控制传输该帧的路由发现操作(见子章节 3.7.3.4): 0x00 =禁用路由发现 0x01 =启用路由发现
SecurityEnable	Boolean	TRUE or FALSE	SecurityEnable参数可用于启用当前帧网络层安全处理, 如果安全级别在 NIB中被指定为0,意思为没有安全, 然后这个参数将会被忽略;否则, TRUE 表示安全处理通过应用安全级别来指定, FALSE 表示没有合适的处理

3.2.1.1.2 原语生成

每当一个数据 PDU (NSDU) 要被传输给一个对等的 APS 子层实体时, 本原语由一个本地 APS 子层实体生成。

3.2.1.1.3 接收结果

如果本原语被一个当前没有连接的设备接收, NWK 层将发出一个状态为 INVALID_REQUEST 的 NLDE-DATA.confirm 原语。

在接收本原语时, NLDE 首先构造一个 NPDU, 以传输所提供的 NSDU。如果在处理期间, NLDE 在传输 NSDU 之前发出了 NLDEDATA.confirm 原语, 则中止所有进一步的处理。在构造新的 NPDU 时, NWK 头的目标地址域将被设置为 DstAddr 参数提供的值, 且源地址域值将为 MAC PIB 中的 *macShortAddress* 属性的值。NWK 头的帧控制域的发现路由域将被设置为 DiscoverRoute 参数提供的值。如果所提供的 Radius 参数值不为零, 那么 NWK 头的半径域就设置为 Radius 参数的值。如果 Radius 参数值为零, 那么 NWK 头的半径域将被设置为 NWK IB 的 *nwkMaxDepth* 属性值的两倍。NWK 层将为 3.6.2.1 小节所述的帧生成一个序列号。且帧的 NWK 头的序列号域设置为这个序列号的值。NWK 头的多播标志域将根据 DstAddrMode 参数的值设置。如果 DstAddrMode 参数值为 0x01, NWK 头将包括一个多播控制域, 其中的域将设置如下:

- 如果该节点是 DstAddr 参数所指定的组的一个成员, 多播模式域将设置为 0x01
- 否则, 多播模式域将设置为 0x00
- 非成员半径和最大非成员半径域将设置为 NonmemberRadius 参数的值

一旦构造了 NPDU, 如果 NSDU 是单播的, 使用 3.6.3.3 小节所述的程序为它寻找路由;

如果是广播的，使用 3.6.5 小节所述的程序寻找路由；如果是多播的，使用 3.6.6.2 小节所述的程序寻找路由。当寻找路由程序指定要传输 NSDU，这通过发出 MCPSDATA.request 原语来完成，且 SrcAddrMode 和 DstAddrMode 参数设置为 0x02，表示使用 16 位的网络地址。SrcPANId 和 DstPANId 参数应设置为 MAC PIB 的 macShortAddr 的值。DstAddr 参数的值是下一跳的地址，由寻找路由程序确定。如果信息是单播的，TxOptions 参数的位 b0 应设置为 1，表示需要一个确认。在接收 MCPSDATA.confirm 原语时，NLDE 发出 NLDE-DATA.confirm 原语，它的状态值等于从 MAC 子层收到的状态值。

如果 NIB 属性 *nwkSecurityLevel* 值为非零，且 SecurityEnable 参数值为 TRUE，那么在按照 4.3 节所述的进行传输之前，为该帧应用 NWK 层的安全处理。否则，不为该帧在 NWK 层执行安全处理。如果执行了安全处理，且它因任何原因失败，那么帧被丢弃，且 NLDE 发出 NLDE-DATA.confirm 原语，它的状态参数值等于安全组合返回的状态值。

3.2.1.2 NLDE-DATA.confirm

本原语报告请求从一个本地 APS 子层实体到一个对等 APS 子层实体，传输一个数据 PDU (NSDU) 的结果。

3.2.1.2.1 服务原语的语义

本原语的语义如下：

NLDE-DATA.confirm	{
	Status
	NsduHandle,
	TxTime
	}

表 3.3 指明了 NLDE-DATA.confirm 原语的参数。

表 3.3 NLDE-DATA.confirm 参数

名称	类型	有效范围	描述
Status	Status	从securitysuite 或者 MCPSDATA.confirm 原语返回的 INVALID_REQUEST,MAX_FRM_COUNTER,NO_KEY,BAD_CCM_OUTPUT,ROUTE_ERROR,BT_TABLE_FULL,FRAME_NOT_BUFFERED 或者任何状态值（见[B1]）	相应的状态请求
NsduHandle	Integer	0x00 – 0xff	处理相关被确认的NSDU
TxTime	Integer	执行相关	基于本地时钟传输包的时间指示。时间应该基于在规定执行中每个传输包的同一点。该值只有在 <i>nwkTimeStamp</i> 设置为 TRUE 之后才能提供。

3.2.1.2.2 原语生成

本原语由一个本地 NLDE 生成，以响应收到一个 NLDE-DATA.request 原语。

状态域将反映相应的请求的状态，如 3.2.1.1.3 小节所描述。

3.2.1.2.3 接收结果

在接收本原语时，发起设备的 APS 子层被告知其请求传输的结果。如果传输尝试成功，状态参数将设置为 SUCCESS。否则，状态参数将指示错误。

3.2.1.3 NLDE-DATA.indication

本原语表示从 NWK 层到本地 APS 子层实体传输一个数据 PDU（NSDU）。

3.2.1.3.1 服务原语的语义

本原语的语义如下：

NLDE-DATA.indication	{
	DstAddrMode,
	DstAddr,
	SrcAddr,
	NsduLength,
	Nsdu,
	LinkQuality
	RxTime
	SecurityUse
	}

表 3.4 指明了 NLDE-DATA.indication 原语的参数。

表 3.4 NLDE-DATA.indication 参数

名称	类型	有效范围	描述
DstAddrMode	Integer	0x01 或 0x02	通过DstAddr参数提供的目的地址类型。这可能有以下两个值中其中一个: 0x01=16位组播地址 0x02=设备的16位网络地址或者16位广播地址
DstAddr	16-bit Address	0x0000-0xFFFF	NSDU发送的目的地址
SrcAddr	16-bit Device address	除了广播地址的任何无效的设备地址	来自NSDU起源的单个设备地址

表 3.4 NLDE-DATA.indication 参数（续表）

名称	类型	有效范围	描述
NsduLength	Integer	$\leq \text{aMaxMACFrameSize} - (\text{nwkMinHeaderOverhead} + \text{nwkMinHeaderOverhead})$	组成表示NSDU的字节数。考虑到 ZigBee 网络层不使用扩展的寻址模式，这已被 IEEE802.15.4 规范中的 aMaxMACFrameSize 限定所修改。这使得头闲置的部分被用于有效载荷。参见小节 D.4。
Nsdu	Set of octets	–	组成表示NSDU的字节组合
LinkQuality	Integer	0x00 – 0xff	链接质量指示通过在 MAC 上接收到的帧时作为 MCPS-DATA.indication 原语的一个参数传递(见[B1])
RxTime	Integer	执行具体	基于本地时钟接受数据包的时间指示。时间应该基于在给定执行中每个接受的数据包的同一点。本值只有在 <i>nwkTimeStamp</i> 设置为 TRUE 时才会提供。
SecurityUse	Boolean	TRUE 或 FALSE	指示是否接收的数据帧使用安全，如果安全适用于接收帧，本值设置为 TRUE，如果接收帧未加密则为 FALSE。

3.2.1.3.2 原语生成

从本地 MAC 子层实体收到一个适当寻址的数据帧时，本原语由 NLDE 生成，发给 APS 子层。

3.2.1.3.3 接收结果

在接收本原语时，APS 子层被告知数据到达该设备。

3.2.2 NWK 管理服务

NWK 层管理实体 SAP (NLME-SAP) 为其上层和 NLME 之间传输管理命令提供接口。表 3.5 列出了 NLME 通过 NLME-SAP 接口支持的原语，以及包含这些原语详细信息的章节。

表 3.5 通过 NLME-SAP 访问的原语概述

名称	在这个规范中的子章节编号			
	请求	指示	相应	确认
NLME-NETWORK-DISCOVERY	3.2.2.1			3.2.2.2
NLME-NETWORK-FORMATION	3.2.2.3			3.2.2.4
NLME-PERMIT-JOINING	3.2.2.5			3.2.2.6
NLME-START-ROUTER	3.2.2.7			3.2.2.8
NLME-ED-SCAN	3.2.2.9			3.2.2.10
NLME-JOIN	3.2.2.11	3.2.2.12		3.2.2.13
NLME-DIRECT-JOIN	3.2.2.14			3.2.2.15
NLME-LEAVE	3.2.2.16	3.2.2.17		3.2.2.18
NLME-RESET	3.2.2.19			3.2.2.20
NLME-SYNC	3.2.2.22			3.2.2.24
NLME-SYNC-LOSS		3.2.2.23		
NLME-GET	3.2.2.26			3.2.2.27
NLME-SET	3.2.2.28			3.2.2.29
NLME-NWK-STATUS		3.2.2.30		
NLME-ROUTE-DISCOVERY	3.2.2.31			3.2.2.32 ^a

a. CCB#822

3.2.2.1 NLME-NETWORK-DISCOVERY.request

该原语支持网络层上层应用该原语来发现在 POS 范围内正在运行的网络。

3.2.2.1.1 服务原语的语义

本原语的语义如下：

NLME-NETWORK-DISCOVERY.request	{
	ScanChannels,
	ScanDuration
	}

表 3.6 指明了 NLME-NETWORKDISCOVERY.request 原语的参数。

表 3.6 NLME-NETWORK-DISCOVERY.request 参数

名称	类型	有效范围	描述
ScanChannels	Bitmap	32位域	最高有效的五位 (b27,...,b31) 是保留位;最低有效的27位 (b0, b1,... b26) 代表要扫描的27有效通道 (1 = scan, 0 =do not scan) 中的每一个(见[B1])
ScanDuration	Integer	0x00 – 0x0e	用于计算扫描每个通道花费的时间长度的值扫描每个通道花费的时间 ($aBaseSuperframeDuration * (2n + 1)$), n是ScanDuration 参数的值; MAC 子层扫描上有更多的信息(见 [B1])

3.2.2.1.2 原语生成

本原语由一个 ZigBee 设备的上层生成，发给其 NLME 来请求发现在设备个人操作空间 (POS) 内运行的网络。

3.2.2.1.3 接收结果

在接收本原语时，NWK 层将通过在 ScanDuration 参数指定的期间内，扫描 ScanChannels 参数指定的信道，执行一个主动扫描，尝试发现在设备 POS 内运行的网络。该扫描是通过 MLMESCAN.request 原语执行的。

在收到 MLME-SCAN.confirm 原语时，NLME 发出 NLME-NETWORK-DISCOVERY.confirm 原语，包含关于发现的网络的信息，且状态值等于 MLME-SCAN.confirm 返回的状态值。

3.2.2.2 NLME-NETWORK-DISCOVERY.confirm

本原语报告一个网络发现操作的结果。

3.2.2.2.1 服务原语的语义

本原语的语义如下：

```
NLME-NETWORK-DISCOVERY.confirm    {  
                                     Status  
                                     NetworkCount,  
                                     NetworkDescriptor,  
                                     }
```

表 3.7 指明了 NLME-NETWORK-DISCOVERY.confirm 原语的参数。

表 3.7 NLME-NETWORK-DISCOVERY.confirm 参数

名称	类型	有效范围	描述
Status	Status	MLME-SCAN.confirm原语返回的任何状态值	见[B1]。
NetworkCount	Integer	0x00 -0xff	给出了通过搜索发现的网络的个数。
NetworkDescriptor	网络描述符的列表	列表包含通过 NetworkCount参数给出的事件数	描述符列表，每个网络发现，表3.8给出了一个各项目的内容的详细描述

表 3.8 给出了 NetworkDescriptor 参数的一个网络描述符的内容的详细解释。

表 3.8 网络描述符信息域

名称	类型	有效范围	描述
ExtendedPanId	Integer	0x0000000000000001 - 0xfffffffffffffffe	网络的64位个域网标识符
LogicalChannel	Integer	选自通过物理层提供的可用的逻辑通道 (见[B1])	目前占用网络的逻辑信道
StackProfile	Integer	0x00 – 0x0f	发现网络中用于表示栈协议的 Zigbee栈协议标识符

表 3.8 网络描述符信息域（续表）

名称	类型	有效范围	描述
ZigBeeVersion	Integer	0x00 – 0x0f	在发现网络中使用的zigbee协议的版本
BeaconOrder	Integer	0x00 – 0x0f	指定通过网络中给定的设备多久传输MAC子层信标;MAC子层信标命令的讨论(见[B1])
SuperframeOrder	Integer	0x00 – 0x0f	信标指示网络,就是说, 信标命令<15, 这代表超帧的活动周期的长度; MAC子层超帧命令的讨论(见[B1])
PermitJoining	Boolean	TRUE或FALSE	在当前允许加入的网络中 TRUE值代表至少一个ZigBee路由器; 就是说, 它的NWK已经发出一个NLME-PERMIT-JOINING原语并且如果规定时间限制, 尚未过期
RouterCapacity	Boolean	TRUE或FALSE	值设置为true,如果设备能够从路由器设备接受加入请求, 否则设置为false
EndDeviceCapacity	Boolean	TRUE或FALSE	值设置为true,如果设备能够从路由器设备接受加入请求, 否则设置为false

3.2.2.2.2 原语生成

本原语由 NLME 生成, 发给其执行发现任务的上层, 该任务由一个 NLME-NETWORKDISCOVERY.request 原语发起。

3.2.2.2.3 接收结果

其上层接收到该原语后, 就可得到网络的搜索结果。

3.2.2.3 NLME-NETWORK-FORMATION.request

本原语允许上层请求设备启动一个新的 ZigBee 网络, 以它本身作为协调器, 并随后修改其超帧的配置。

3.2.2.3.1 服务原语的语义

本原语的语义如下：

```

NLME-NETWORK-FORMATION.request    {
    ScanChannels,
    ScanDuration,
    BeaconOrder,
    SuperframeOrder,
    BatteryLifeExtension
}
```

表 3.9 指明了 NLME-NETWORK-FORMATION.request 原语的参数。

表 3.9 NLME-NETWORK-FORMATION.request 参数

名称	类型	有效范围	描述
ScanChannels	Bitmap	32位域	最高有效五位(b27,...,b31) 是保留位. 最低有效27位 (b0, b1,... b26) 代表在准备启动网络中的(1=scan(扫描), 0=do not scan(不扫描)) 将要被扫描的27个有效通道中的每一个(见[B1])
ScanDuration	Integer	0x00 – 0x0e	用于计算扫描每个信道花费时间的长度的值; 扫描每个信道花费的时间是 $(aBaseSuperframeDuration * (2n + 1))$, n 是 ScanDuration参数的值 (见[B1])
BeaconOrder	Integer	0x00 – 0x0f	更高层想形成的网络信标命令
SuperframeOrder	Integer	0x00 – 0x0f	更高层想形成的网络超帧命令
BatteryLifeExtension	Boolean	TRUE or FALSE	如果该值为 TRUE, NLME将请求ZigBee协调器启动支持电池寿命延长模式; 如果该值为FALSE, NLME将请求 ZigBee协调器启动不支持电池寿命延长模式

3.2.2.3.2 原语生成

本原语由一个具有 ZigBee 协调器功能的设备的上层生成，发给其 NLME 来请求初始化自身为一个新网络的 ZigBee 协调器，并随后修改其超帧的配置。

3.2.2.3.3 接收结果

一个不能成为 ZigBee 协调器的设备收到本原语时，NLME 发出状态参数设置为 INVALID_REQUEST 的 NLME-NETWORK-FORMATION.confirm 原语。

如果设备被初始化为一个 ZigBee 协调器，NLME 请求 MAC 子层首先执行一个能量检测扫描，然后在指定的信道上执行一个主动扫描。为此，NLME 给 MAC 子层发出 MLME-SCAN.request 原语，它的 ScanType 参数设置为表示一个能量检测扫描，然后再次发出原语，ScanType 参数设置为表示一个主动扫描。在执行主动扫描之后，从 MAC 子层收到 MLME-SCAN.confirm 原语时，NLME 选择一个合适的信道。NWK 层将挑选一个不与任何运行在选定信道上的已知网络冲突的 PAN 标识符。一旦找到合适的信道和 PAN 标识符，NLME 将选择 0x0000 作为 16 位短 MAC 地址，并通知 MAC 子层。为此，NLME 给 MAC 子层发出 MLME-SET.request 原语，以设置 MAC PIB 属性 *macShortAddress*。如果 NIB 属性 *nwkExtendedPANId* 等于 0x0000000000000000，这个属性应初始化为 MAC 常量 *aExtendedAddress* 的值。如果找不到合适的信道或 PAN 标识符，NLME 发出状态参数设置为 STARTUP_FAILURE 的 NLME-NETWORK-FORMATION.confirm 原语。

如果只是在上层请求中只提供一个信道，NLME 不需要在开始网络之前进行能量扫描。主动扫描仍然执行，保证选择一个 PAN 标识符，和已知的正在操作的网络并不冲突。

接着，NLME 给 MAC 子层发出 MLME-START.request 原语。MLME-START.request 原语的 PANCoordinator 参数设置为 TRUE。BeaconOrder、SuperframeOrder 和 BatteryLifeExtension 参数将和 NLME-NETWORK-FORMATION.request 原语给定的值相同。如果发出该原语用于初始化一个新的超帧，MLME-START.request 原语中的 CoordRealignment 参数设置为 FALSE。如果发出该原语用于修改任何的 PAN 配置属性，MLME-START.request 原语中的 CoordRealignment 参数设置为 TRUE。在接收相关的 MLME-START.confirm 原语时，NLME 给上层发出 NLME-NETWORK-FORMATION.confirm 原语，它的状态值是 MLME-START.confirm 原语返回的状态值。

3.2.2.4 NLME-NETWORK-FORMATION.confirm

本原语报告请求在一个网络中初始化一个 ZigBee 协调器的结果。

3.2.2.4.1 服务原语的语义

本原语的语义如下：

NLME-NETWORK-FORMATION.confirm	{
	Status
	}

表 3.10 指明了 NLME-NETWORKFORMATION.confirm 原语的参数。

表 3.10 NLME-NETWORK-FORMATION.confirm 参数

名称	类型	有效范围	描述
Status	Status	从MLME-START.confirm原语返回的 INVALID_REQUEST,STARTUP_FAI LURE或任何状态值	试图去初始化ZigBee协调器或者 请求更改为超帧配置的结果

3.2.2.4.2 原语生成

本原语由 NLME 生成, 发给其上层, 以响应一个 NLME-NETWORK-FORMATION.request 原语。本原语返回一个 INVALID_REQUEST、STARTUP_FAILURE 的状态值, 或 MLME-START.confirm 原语返回的任何状态值。这些值可以返回的条件如 3.2.2.3.3 小节所述。

3.2.2.4.3 接收结果

在接收本原语时, 上层被告知其请求初始化设备为一个 ZigBee 协调器, 或请求修改超帧配置的结果。如果 NLME 成功, 状态参数将被设置为 SUCCESS。否则, 状态参数指示错误。

3.2.2.5 NLME-PERMIT-JOINING.request

当可以接受设备加入其网络时, 本原语允许一个 ZigBee 协调器或路由器的上层设置其 MAC 子层的连接允许标志, 并保持一段固定的时间。

3.2.2.5.1 服务原语的语义

服务原语的语义如下：

NLME-PERMIT-JOINING.request	{
	PermitDuration
	}

3.11 指明了 NLME-PERMIT-JOINING.request 原语的参数。

表 3.11 NLME-PERMIT-JOINING.request 参数

名称	类型	有效范围	描述
PermitDuration	Integer	0x00 – 0xff	在 ZigBee 协调器或者路由器之间将允许关联的以秒为单位的时间长度; 0x00 和 0xff 代表允许是禁用或者启用的，没有一个特定的时间限制

3.2.2.5.2 原语生成

每当想要允许或禁止新设备加入网络时，本原语由一个 ZigBee 协调器或路由器的上层生成，发给其 NLME。

3.2.2.5.3 接收结果

只允许一个 ZigBee 协调器或路由器的上层发出本原语。一个 ZigBee 终端设备的 NWK 层收到本原语时，NLME-PERMIT-JOINING.confirm 原语返回一个 INVALID_REQUEST 状态。

在收到本原语，且 PermitDuration 参数设置为 0x00 时，NLME 通过给 MAC 子层发出 MLME-SET.request 原语，设置 MAC PIB 属性 *macAssociationPermit* 为 FALSE。一旦收到 MLMESET.confirm 原语，NLME 发出 NLME-PERMITJOINING.confirm 原语，状态值等于从 MAC 子层收到的状态值。

在收到本原语，且 PermitDuration 参数设置为 0xff 时，NLME 通过给 MAC 子层发出 MLME-SET.request 原语，设置 MAC PIB 属性 *macAssociationPermit* 为 TRUE。一旦收到 MLMESET.confirm 原语，NLME 发出 NLME-PERMITJOINING.confirm 原语，状态值等于从 MAC 子层收到的状态值。

在收到本原语,且PermitDuration参数设置不是0x00或0xff的任何值时,NLME 设置 MAC PIB 属性 *macAssociationPermit* 为 TRUE,如上所述。接着收到 MLME-SET.confirm 原语之后,NLME 启动一个计时器,在 PermitDuration 秒之后过期。一旦设置了计时器,NLME 发出 NLMEPERMIT-JOINING.confirm 原语,状态值等于 MAC 子层收到的状态值。计时器过期时,NLME 通过发出 MLME-SET.request 原语,设置 *macAssociationPermit* 为 FALSE。

上层每次发出的 NLME-PERMIT-JOINING.request 原语会替代之前所有的请求。

3.2.2.6 NLME-PERMIT-JOINING.confirm

本原语允许一个 ZigBee 协调器或路由器的上层被告知其请求允许接受设备加入网络的结果。

3.2.2.6.1 服务原语的语义

本原语的语义如下:

NLME-PERMIT-JOINING.confirm	{ Status }
-----------------------------	------------------

表 3.12 指明了 NLME-PERMIT-JOINING.confirm 原语的参数。

表 3.12 NLME-PERMIT-JOINING.confirm 参数

名称	类型	有效范围	描述
Status	Status	从the MLME-SET.confirm原语返回的 INVALID_REQUEST或者任何状态(见[B1])	相应请求的状态

3.2.2.6.2 原语生成

本原语由一个 ZigBee 协调器或路由器的启动 NLME 生成,发给其上层,以响应一个 NLME-PERMITJOINING.request。状态参数表示从 MAC 子层收到的状态,或发出一个 INVALID_REQUEST 的错误代码。这些状态的原因描述在 3.2.2.5 小节中。

3.2.2.6.3 接收结果

在接收本原语时,启动设备的上层被告知其请求允许设备加入网络的结果。

3.2.2.7 NLME-START-ROUTER.request

这组原语允许上层 ZigBee 路由器，开始从事期望作为一个 ZigBee 路由器的活动，包括为数据帧寻找路由，路由发现和从其它设备接收加入网络的请求。

3.2.2.7.1 服务原语的语义

本原语的语义如下：

NLME-START-ROUTER.request	{ BeaconOrder, SuperframeOrder, BatteryLifeExtension }
---------------------------	--

表 3.13 指明了 NLME-START-ROUTER.request 的参数。

表 3.13 NLME-START-ROUTER.request 参数

名称	类型	有效范围	描述
BeaconOrder	Integer	0x00 – 0x0f	更高层想形成的网络信标命令
SuperframeOrder	Integer	0x00 – 0x0f	更高层想形成的网络超帧命令
BatteryLifeExtension	Boolean	TRUE or FALSE	如果该值为 TRUE, NLME将请求ZigBee路由器启动支持电池寿命延长模式；如果该值为FALSE, NLME将请求ZigBee路由器启动不支持电池寿命延长模式；

3.2.2.7.2 原语生成

本原语由一个新设备的上层生成，发给其 NLME 来请求把自身初始化为一个 ZigBee 路由器。它还可以发给一个已经作为一个 ZigBee 路由器运行的设备的 NLME，来调整其超帧的配置。

3.2.2.7.3 接收结果

还没有作为一个路由器加入一个 ZigBee 网络的设备收到本原语，NLME 发出 NLME-START-ROUTER.confirm 原语，状态参数设置为 INVALID_REQUEST。

已经作为一个路由器加入一个 ZigBee 网络的设备的 NLME 收到本原语，NLME 发出 MLME-START.request 原语给 MAC 子层。MLME-START.request 原语的 BeaconOrder、SuperframeOrder 和 BatteryLifeExtension 参数值为 NLME-START-ROUTER.request 相应的参数值。如果发出原语是为了首次作为一个路由器启动，MLME-START.request 原语的 CoordRealignment 参数设置为 FALSE。如果发出原语用来修改任何的 PAN 配置属性，MLME-START.request 原语的 CoordRealignment 参数设置为 TRUE。

在收到相关的 MLME-START.confirm 原语时，NLME 给上层发出 NLME-START-ROUTER.confirm 原语，状态值是 MLME-START.confirm 原语返回的状态值。仅当 MLME-START.confirm 原语返回的状态值是 SUCCESS，设备可以开始从事期望作为一个 ZigBee 路由器的活动，包括为数据帧寻找路由，路由发现，路由修复和从其它设备接收加入网络的请求。否则，设备被明确禁止从事这些活动。

3.2.2.8 NLME-START-ROUTER.confirm

本原语报告请求初始化或修改一个 ZigBee 路由器的结果。

3.2.2.8.1 服务原语的语义

本原语的语义如下：

NLME-START-ROUTER.confirm	{
	Status
	}

表 3.14 指明了 NLME-START-ROUTER.confirm 的参数。

表 3.14 NLME-START-ROUTER.confirm 参数

名称	类型	有效范围	描述
Status	Status	从MLMESTART.confirm 原语返回的INVALID_REQUEST或者任何状态值	试图初始化Zigbee路由器的结果

3.2.2.8.2 原语生成

本原语由一个 NLME 生成，发给其上层，以响应一个 NLME-START-ROUTER.request 原语。本原语返回一个 INVALID_REQUEST 的状态值，或 MLME-START.confirm 原语返回的任何状态值。这些值可以返回的条件描述在 3.2.2.7.3 小节。

3.2.2.8.3 接收结果

在接收本原语时，上层被告知其请求初始化或修改一个 ZigBee 路由器的超帧配置的结果。上层被告知自身的请求去初始化一个 ZigBee 路由器的结果。如果 NLME 成功，状态参数将设置为 SUCCESS。否则，状态参数指示错误。

3.2.2.9 NLME-ED-SCAN.request

该原语允许设备上层要求一个能量检测去评估本地的信道。

3.2.2.9.1 服务原语的语义

本原语的语义如下：

NLME-ED-SCAN.request	{
	ScanChannels,
	ScanDuration,
	}

表 3.15 指明了服务原语的参数。

表 3.15 NLME-ED-SCAN.request 参数

名称	类型	有效范围	描述
ScanChannels	Bitmap	32-bit field	5个最重要位 (b27,..., b31) 被保留. 27个最不重要位 (b0, b1,... b26) 指示扫描那个信道。 (1=scan, 0=do not scan) 27个中每个信道 (see [B1]).
ScanDuration	Integer	0x00-0x0e	用来计算扫描每个信道时间的值，时间值为 $(aBaseSuperframeDuration * (2n + 1))$ 其中 n 是 ScanDuration 参数 [B1].

3.2.2.9.2 原语生成

设备上层产生这个原语要求执行一个信道的能量扫描。

3.2.2.9.3 接收结果

在当前连接到网络的设备接收到本原语之后，设备会暂时停止网络上的操作执行一次能量扫描。为了此次操作，NLME 会发布 NLME-SCAN.request 原语到 MAC 子层，它的 ScanType 参数设置为表示一个能量检测扫描，ScanChannels 和 ScanDuration 来自 NLME 请求。

3.2.2.10 NLME-ED-SCAN.confirm

本原语提供给上层能量扫描的结果。

3.2.2.10.1 服务原语的语义

本原语的语义如下：

```

NLME-ED-SCAN.confirm      {
                                Status
                                UnscannedChannels,
                                EnergyDetectList,
                                }
  
```

表 3.16 指明了服务原语的参数

表 3.16 NLME-ED-SCAN.confirm

名称	类型	有效范围	描述
Status	Status	SUCCESS, 或者从 MAC 上获得的有效 码	请求的状态
UnscannedChannels	Bitmap	32 bit field	5个最重要位 (b27,..., b31) 被保留. 27个最不重要位 (b0, b1,... b26) 指示扫描那个信道。 (1=scan, 0=do not scan) 27个中每个信道 (见 [B1]). 注意这是[B1]UnscannedChannel 表.
EnergyDetectList	List of integers	0x00-0xff for each integer	能量测量表和 [B1]一致, 每个信道一个。

3.2.2.10.2 原语生成

这个原语由 ZigBee 设备的 NLME 响应 NLME-ED-SCAN.request 产生。状态指示了从 MAC 子层 MLME-SCAN.confirm 原语收到的状态。UnscannedChannels 指示了未扫描的信道 (0= 已经扫描的信道)。EnergyDetectList 包含了 ED 扫描结果(整数列表, 0x00 - 0xff), 即符合 IEEE 802.15.4-2003 的已经扫描的信道。

3.2.2.10.3 接收结果

在收了本原语之后, 上层被通知 ED 扫描结果。

3.2.2.11 NLME-JOIN.request

该原语允许设备上层通过该原语以直接或间接方式请求连接网络，或者当设备为孤点设备时，请求重新连接网络。或者在一个网络中为设备改变操作的信道。

3.2.2.11.1 服务原语的语义

本原语的语义如下：

NLME-JOIN.request	{	
		ExtendedPANId,
		RejoinNetwork,
		ScanChannels,
		ScanDuration,
		CapabilityInformation,
		SecurityEnable
	}	

表 3.17 指明了 NLME-JOIN.request 原语的参数。

表 3.17 NLME-JOIN.request 参数

名称	类型	有效范围	描述
ExtendedPANId	Integer	0x0000000000 000001–0xffff ffffffffffffe	加入网络的64位个域网标识符
RejoinNetwork	Integer	0x00 – 0x03	该参数控制加入网络的方法：如果设备通过关联请求加入网络，参数是0x00；如果设备用孤立程序直接加入或重新加入网络，参数是0x01；如果设备使用网络重新加入程序，参数是0x02；如果设备要把操作的网络信道改为ScanChannels参数指明的信道，参数是0x03。
ScanChannels	Bitmap	32位域	最高五位(b27,..., b31)是保留位。最低的27位位(b0, b1,... b26) 代表27个有效信道哪个信道将要被扫描（1=扫描，0=不扫描）(见[B1])
ScanDuration	Integer	0x00-0x0e	用于计算扫描每个通道花费的时间长度。扫描每个信道花费的时间是以 $(aBaseSuperframeDuration * (2n + 1))$ 为标志， n 是ScanDuration参数的值 [B1]
CapabilityInformation	Bitmap	见表格3.47	设备被直接加入后的操作能力。
SecurityEnable	Boolean	TRUE 或 FALSE	如果RejoinNetwork值为0x02，且这是TRUE，设备会安全重新加入。否则设置为FALSE。

3.2.2.11.2 原语生成

一个设备的上层生成本原语，以请求：

- 使用 MAC 子层连接程序加入一个网络

- 使用孤节点程序加入或直接加入一个网络
- 使用 NWK 重新加入程序加入或重新加入一个网络
- 为加入网络的设备切换操作频道

3.2.2.11.3 接收结果

一个当前已经加入了一个网络的设备收到本原语，且 RejoinNetwork 参数设置为 0x00，NLME 发出一个 NLMEJOIN.confirm 原语，状态参数设置为 INVALID_REQUEST。

一个当前没有加入一个网络的设备收到本原语，且 RejoinNetwork 参数等于 0x00，设备尝试加入 64 位 ExtendedPANId 参数指定的网络。参见 3.6.1.4.1.1。

如果设备收到本原语，且 RejoinNetwork 参数等于 0x01，那么设备尝试使用孤程序加入或重新加入网络，参见 3.6.1.4.3.2。

如果设备收到本原语，且 RejoinNetwork 参数等于 0x02，设备尝试加入 ExtendedPANId 参数给出的 64 位扩展 PAN ID 指定的网络。加入程序参见 3.6.1.4.2。

一旦设备成功加入一个网络，*nwkExtendedPANId* NIB 属性设置为加入的网络的扩展 PAN 标识符。

如果设备收到本原语，且 RejoinNetwork 参数等于 0x03，设备支持设置 *phyCurrentChannel* 的值，设备尝试去更换操作信道到 *ScanChannels* 中的信道。如果 *ScanChannels* 中提供了不止一个频道，NLME 发布了一个 NLME-JOIN.confirm 原语，Status 参数设置为 INVALID_REQUEST。如果信道改变成功，设备会发出一个 NLME-JOIN.confirm 原语，Status 参数设置为 SUCCESS。如果设备不支持直接设置 *phyCurrentChannel*，那么 NLME 发出一个 NLME-JOIN.confirm 原语，Status 参数设置为 UNSUPPORTED_ATTRIBUTE。¹²

3.2.2.12 NLME-JOIN.indication

本原语允许一个 ZigBee 协调器或 ZigBee 路由器的上层得到通知，当一个新设备通过连接，或使用 3.6.1.4.3 小节所述的 NWK 重新加入程序重新加入，成功加入它的网络。

12. CCB #820

3.2.2.12.1 服务原语的语义

本原语的语义如下：

NLME-JOIN.indication	{ NetworkAddress, ExtendedAddress, CapabilityInformation, RejoinNetwork SecureJoin }
----------------------	--

表 3.18 指明了 NLME-JOIN.indication 原语的参数。

表 3.18 NLME-JOIN.indication 参数

名称	类型	有效范围	描述
NetworkAddress	Network address	0x0001 – 0xffff7	已经被加入到网络的实体的网络地址
ExtendedAddress	64-bit IEEE address	任何 64-bit, IEEE 地址	已经被加入到网络的实体的64位 IEEE地址
CapabilityInformation	Bitmap	See [B1]	指定的加入设备的操作能力
RejoinNetwork	Integer	0x00 - 0x02	RejoinNetwork参数指示加入网络的方法。设备通过连接加入参数为0x00，设备通过孤程序直接加入或重新加入参数为0x01，设备通过网络重新加入参数为0x02。 ^a
SecureRejoin	Boolean	TRUE 或 FALSE	如果重新加入是以一种安全方式执行该参数为TRUE。否则，该参数为FALSE

a. CCB #821

3.2.2.12.2 原语生成

在使用图 3.35 所示的 MAC 连接程序，成功增加一个新设备到网络，或使用图 3.40 所示的 NWK 重新加入程序，允许一个设备重新加入网络时，本原语由一个 ZigBee 协调器或路由器的 NLME 生成，发给其上层。

3.2.2.12.3 接收结果

在接收本原语时，一个 ZigBee 协调器或路由器的上层被告知一个新设备已经加入到它的网络。

3.2.2.13 NLME-JOIN.confirm

本原语允许上层被告知其请求加入一个网络的结果。

3.2.2.13.1 服务原语的语义

本原语的语义如下：

NLME-JOIN.confirm	{
	Status
	NetworkAddress,
	ExtendedPANID,
	ActiveChannel
	}

表 3.19 指明了 NLME-JOIN.confirm 原语的参数。

表 3.19 NLME-JOIN.confirm 参数

名称	类型	有效范围	描述
Status	Status	INVALID_REQUEST, NOT_PERMITTED,NO_NETWORKS 或者 MLMEASSOCIATE. Confirm 和 MLMESCAN. Confirm 原语返回的任何状态值	相应请求的状态
NetworkAddress	Integer	0x0001 –0xfff7 and 0xffff	分配给设备的 16 位网络地址；如果加入未成功，参数值等于 0xffff。
ExtendedPANID	Integer	0x0000000000000001 . 0xfffffffffffffffe	设备现在已经是网络的一个成员的 64 位扩展 PAN 标识符
ActiveChannel	Integer	PHY 支持的任何信道个数，见 [B1]	PHY PIBphyCurrentChannel 的值等于加入的现网络。

3.2.2.13.2 原语生成

本原语由发起 NLME 生成，发给其上层，以响应一个 NLME-JOIN.request 原语。如果请求成功，状态参数值应为 SUCCESS。否则，状态参数表示一个错误代码 INVALID_REQUEST, NOT_PERMITTED, NO_NETWORKS 或 MLME-ASSOCIATE.confirm 原语或 MLME-SCAN.confirm 原语返回的任何状态值。这些状态值的原因完整描述在 3.2.2.11.3 小节。

3.2.2.13.3 接收结果

在接收本原语时，发起设备的上层被告知其使用 MAC 子层连接程序，请求加入一个网络，或使用 MAC 子层孤节点程序直接加入，或一旦它成为孤节点，重新加入一个网络的结果。

3.2.2.14 NLME-DIRECT-JOIN.request

本可选的原语允许一个 ZigBee 协调器或路由器的上层请求直接纳入另一个设备到它的网络。

3.2.2.14.1 服务原语的语义

本可选原语的语义如下：

NLME-DIRECT-JOIN.request	{	DeviceAddress,
		CapabilityInformation
	}	

表 3.20 指明了 NLME-DIRECT-JOIN.request 原语的参数。

表 3.20 NLME-DIRECT-JOIN.request 参数

名称	类型	有效范围	描述
DeviceAddress	64-bit IEEE address	任何64位IEEE地址	被直接加入的设备的IEEE地址
CapabilityInformation	Bitmap	见表3.47	被直接加入设备的操作能力

3.2.2.14.2 原语生成

一个 ZigBee 协调器或路由器的上层生成本原语，以直接增加一个新设备到它的网络。这个过程不需要任何无线传输。

3.2.2.14.3 接收结果

在接收本原语时，NLME 应尝试把 DeviceAddress 参数指定的设备增加到其邻居表中。CapabilityInformation 参数将包含要纳入的设备一个描述。执行本规范的设备中的备用 PAN 协调器位设置为 0。如果设备是一个 ZigBee 路由器，设备类型设置为 1，或如果它是一个终端设备，设置为 0。如果设备从备用当前电源接收电力，电源来源位设置为 1，否则设置为 0。如果设备在空闲期间没有禁用其接收器，空闲时接收器开启位设置为 1，否则设置为 0。如果设备能够安全运行，安全功能位设置为 1，否则设置为 0。

如果 NLME 成功把设备增加到其邻居表中，NLME 发出状态为 SUCCESS 的 NLME-DIRECT-JOIN.confirm 原语。如果 NLME 发现被请求的设备已经出现在其邻居表中，NLME 发出状态为 ALREADY_PRESENT 的 NLME-DIRECT-JOIN.confirm 原语。如果设备列表中没有增加一个新设备的可用空间，NLME 发出状态为 NEIGHBOR_TABLE_FULL 的 NLME-DIRECT-JOIN.confirm 原语。

3.2.2.15 NLME-DIRECT-JOIN.confirm

本原语允许一个 ZigBee 协调器或路由器的上层被告知其请求直接纳入另一个设备到其网络的结果。

3.2.2.15.1 服务原语的语义

本原语的语义如下：

NLME-DIRECT-JOIN.confirm	{
	Status
	DeviceAddress,
	}

表 3.21 指明了 NLME-DIRECT-JOIN.confirm 原语的参数。

表 3.21 NLME-DIRECT-JOIN.confirm 参数

名称	类型	有效范围	描述
Status	Status	SUCCESS,ALREADY_PRESENT,NEIGHBOR_TABLE_FULL	相关请求的状态
DeviceAddress	64-bit IEEE address	任何64位IEEE地址	这是一个确认64位IEEE地址的请求

3.2.2.15.2 原语生成

本原语由发起 NLME 生成, 发给其上层, 以响应一个 NLME-DIRECT-JOIN.request 原语。如果请求成功, 状态参数表示一次成功的加入尝试。否则, 状态参数表示一个 ALREADY_PRESENT 或 NEIGHBOR_TABLE_FULL 的错误代码。这些状态值的原因完整描述在 3.2.2.14.3 小节中。

3.2.2.15.3 接收结果

在接收本原语时, 发起设备的上层被告知其请求直接纳入另一个设备到一个网络的结果。

3.2.2.16 NLME-LEAVE.request

本原语允许上层请求它或另一个设备离开网络。

3.2.2.16.1 服务原语的语义

本原语的语义如下:

NLME-LEAVE.request	{	DeviceAddress,
		RemoveChildren,
		Rejoin,
		}

表 3.22 指明了 NLME-LEAVE.request 原语的参数。

表 3.22 NLME-LEAVE.request 参数

名称	类型	有效范围	描述
DeviceAddress	Device address	任何64位 IEEE地址	从网络中被移除的实体的64位IEEE地址, 如果设备要从网络中移除自己, 为NULL
RemoveChildren	Boolean	TRUE或 FALSE	若有的话, 如果设备被要求离开网络, 其也被移除它的子设备, 参数的值为TRUE, 否则其值为FALSE。
Rejoin	Boolean	TRUE或 FALSE	如果设备被要求离开当前父设备而重新加入网络, 该参数值为TRUE, 否则, 该参数值为FALSE。

3.2.2.16.2 原语生成

一个设备的上层生成本原语, 以请求离开网络。一个 ZigBee 协调器或路由器的上层也可以生成本原语从网络中移除一个设备。

3.2.2.16.3 接收结果

一个当前没有被纳入一个网络的设备的 NLME 收到本原语时, NLME 发出状态值为 INVALID_REQUEST 的 NLME-LEAVE.confirm 原语。一个当前已经被纳入一个网络的设备的 NLME 收到本原语时, 且 DeviceAddress 参数等于 NULL, RemoveChildren 参数等于 FALSE, NLME 将使用 3.6.1.10.1 小节描述的程序从网络中移除设备本身。然后 NLME 将清除其路由表和路由发现表, 并给 MAC 子层发出一个 MLME-RESET.request 原语。NLME 还将把与其之前的父节点相对应的邻居表条目中的关系域设置为 0x03, 表示没有关系。如果收到 NLME-LEAVE.request 原语, 且 DeviceAddress 参数等于 NULL, RemoveChildren 参数等于 TRUE, 那么 NLME 将尝试移除设备的子节点, 如 3.6.1.10.3 小节所描述。

一个 ZigBee 协调器或 ZigBee 路由器收到本原语, 且 DeviceAddress 参数不等于 NULL, NLME 确定指定的设备是否是一个子节点。如果被请求的设备不存在, NLME 发出状态值为 UNKNOWN_DEVICE 的 NLME-LEAVE.confirm 原语。如果被请求的设备存在, NLME 将尝试使用 3.6.1.10.3 小节所述的程序从网络中移除它。如果 RemoveChildren 参数等于 TRUE, 那么设备也将被请求移除其子节点。在移除之后, NLME 将发出 NLME-LEAVE.confirm 原语, 它的 DeviceAddress 等于被移除设备的 64 位 IEEE 地址, 状态参数等于 MCPS-DATA.confirm 原语所提供的状态。如果对应离开设备的邻居表条目的关系域值为 0x01, 那么就被改为 0x04 表示之前的子节点。如果值为 0x05, 表示该子节点尚未经过验证, 它就被改为 0x03, 表示没有关系。

3.2.2.17 NLME-LEAVE.indication

如果 ZigBee 设备已经离开网络，或一个相邻设备已经离开网络，本原语允许一个 ZigBee 设备的上层收到通知。

3.2.2.17.1 服务原语的语义

本原语的语义如下：

NLME-LEAVE.indication	{ DeviceAddress, Rejoin }
-----------------------	--

表 3.23 指明了 NLME-LEAVE.indication 原语的参数。

表 3.23 NLME-LEAVE.indication 参数

名称	类型	有效范围	描述
DeviceAddresses	64-bit IEEE address	任何64位IEEE地址	从网络中移除自己的实体的64位IEEE地址或者通过它的父设备发送离开网络的原语为 NULL
Rejoin	Boolean	TRUE或FALSE	如果该设备被要求从请求重新加入网络的父设备分开，这个参数的值为TRUE。否则，该参数的值为FALSE.

3.2.2.17.2 原语生成

在收到一个关于其 PAN 上一个设备的广播离开命令时，本原语由一个 ZigBee 协调器或 ZigBee 路由器的 NLME 生成，发给其上层。它还由一个 ZigBee 协调器或终端设备生成，发给其上层，来表示它已经被其相关路由器或 ZigBee 协调器从网络中成功移除。

3.2.2.17.3 接收结果

在接收本原语时，一个 ZigBee 协调器或 ZigBee 路由器的上层被告知，之前在网络上的一个设备已经离开。该原语还可以通知一个 ZigBee 路由器或终端设备的上层，它已经被其相关 ZigBee 路由器或 ZigBee 协调器从网络中移除。在这种情况下，Rejoin 参数的值指示上层父节点设备上的对等实体是否希望已被移除的设备重新加入同一个网络。

3.2.2.18 NLME-LEAVE.confirm

本原语允许发起设备的上层告知其请求自身或另一个设备离开网络的结果。

3.2.2.18.1 服务原语的语义

本原语的语义如下：

NLME-LEAVE.confirm	{
	Status
	DeviceAddress,
	}

表 3.24 指明了 NLME-LEAVE.confirm 原语的参数。

表 3.24 NLME-LEAVE.confirm 参数

名称	类型	有效范围	描述
Status	Status	SUCCESS, INVALID_REQUEST, UNKNOWN_DEVICE 或者通过 MCPSDATA.confirm 原语返回的任何状态	相应请求的状态
DeviceAddress	64-bit IEEE address	任何 64 位 IEEE 地址	在确认请求中的 64 位的 IEEE 地址或者如果该设备请求从网络中移除自己则为 null

3.2.2.18.2 原语生成

本原语由发起 NIME 生成，发给其上层，以响应一个 NLME-LEAVE.request 原语。如果请求成功，状态参数表示一个成功的离开尝试。否则，状态参数表示一个 INVALID_REQUEST, UNKNOWN_DEVICE 的错误代码，或 MCPS-DATA.confirm 原语提供的一个状态。这些状态值的原因的完整描述在 3.2.2.16.3 小节中。

3.2.2.18.3 接收结果

在接收本原语时,发起设备的上层被告知其请求自身或一个子节点设备离开网络的结果。

3.2.2.19 NLME-RESET.request

本原语允许上层请求 NWK 层执行一个复位操作。

3.2.2.19.1 服务原语的语义

本原语的语义如下:

NLME-RESET.request	{ WarmStart }
--------------------	---------------------

表 3.25 指明了本原语的参数

表 3.25 NLME-RESET.request 参数

名称	类型	有效范围	描述
WarmStart	Boolean	TRUE或FALSE	如果请求复位到所有协议栈值成为初始默认值,参数有值为 FALSE, 如果该值为 TRUE, 设备希望根据 NIB设置在调用之前恢复操作。

3.2.2.19.2 原语生成

本原语由上层生成,发给其 NLME 来请求复位 NWK 层到其初始条件,或者在原语发布之前根据其当前 NIB 值恢复操作。

3.2.2.19.3 接收结果

在接收本原语时, 如果 WarmStart 参数值为 FALSE, NLME 给 MAC 子层发出 SetDefaultPIB 参数设置为 TRUE 的 MLME-RESET.request 原语。在收到相应的 MLME-RESET.confirm 原语时, NWK 层通过清除所有的内部变量, 路由表和路由发现表条目, 以及把所有的 NIB 属性设置为其默认值来复位自身。一旦 NWK 层复位, NLME 发出 NLME-RESET.confirm, 如果 MAC 子层成功复位, 状态参数设置为 SUCCESS, 否则设置为 DISABLE_TRX_FAILURE。

在接收本原语时，且 WarmStart 参数设置为 TRUE，网络层不能改变任何 NIB 值，而应恢复正常的网络操作，认为自己成为 NIB 指定的网络的一部分。路由表值和邻居表值应该被清除。网络和 MAC 层属性值预先载入的方法留给执行者。

如果本原语发给当前已经加入一个网络的一个设备的 NLME，任何使用 NLME-LEAVE.request 原语请求断开连接的企图都由上层进行优先判断。

3.2.2.20 NLME-RESET.confirm

本原语允许上层被告知其请求复位 NWK 层的结果。

3.2.2.20.1 服务原语的语义

本原语的语义如下：

NLME-RESET.confirm	{
	Status
	}

表 3.26 指明了本原语的语义。

表 3.26 NLME-RESET.confirm 参数

名称	类型	有效范围	描述
Status	Status	从MLMERESET.confirm 原语返回的任何状态值	复位操作的结果

3.2.2.20.2 原语生成

本原语由 NLME 生成，发给其上层，以响应一个 NLME-RESET.request 原语。如果请求成功，状态参数值为 SUCCESS。否则，状态参数表示一个 DISABLE_TRX_FAILURE 的错误代码。这些状态值的原因的完整描述在 3.2.2.19.3 小节中。

3.2.2.20.3 接收结果

在接收本原语时，上层被告知其请求复位 NWK 层的结果。

3.2.2.21 网络层复位信息顺序图

图 3.2 说明了复位 NWK 层所需信息的顺序。

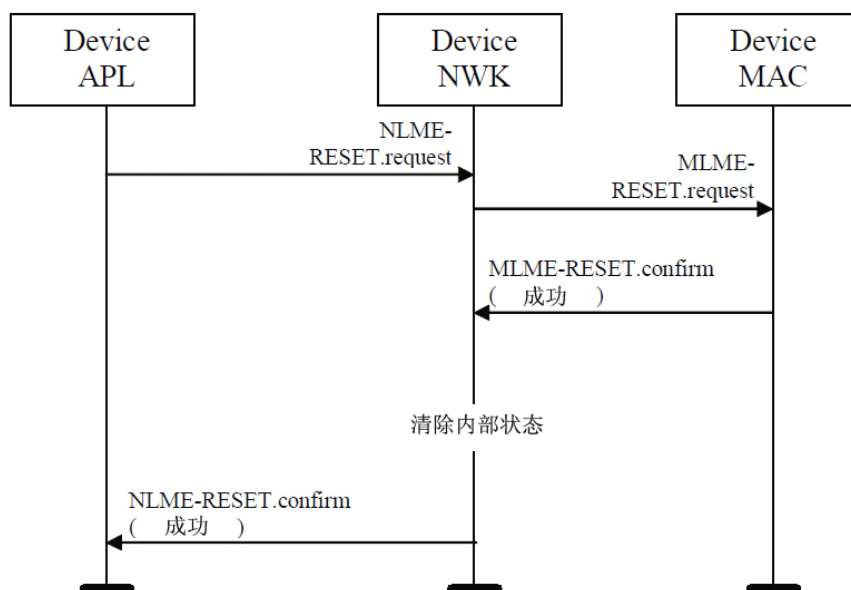


图 3.2 复位网络层信息序列图

3.2.2.22 NLME-SYNC.request

设备应用层使用该原语与 ZigBee 协调器或路由器进行同步，或从它的 ZigBee 协调器或路由器中得到数据。

3.2.2.22.1 服务原语的语义

本原语的语义如下：

NLME-SYNC.request	{
	Track
	}

表 3.27 指明了本原语的参数。

表 3.27 NLME-SYNC.request 参数

名称	类型	有效范围	描述
Track	Boolean	TRUE或FALSE	是否应该与未来信标保持同步

3.2.2.22.2 原语生成

每当上层希望实现同步或在其 ZigBee 协调器或路由器中检查是否存在未处理的数据时，生成本原语。

3.2.2.22.3 接收结果

如果 TRACK 参数设置为 FALSE，且设备运行在一个没有启用信标的网络上，NLME 给 MAC 子层发出 MLME-POLL.request 原语。在收到相应的 MLME-POLL.confirm 原语时，NLME 发出 NLME-SYNC.confirm 原语，状态参数设置为 MLME-POLL.confirm 报告的值。

如果 TRACK 参数设置为 FALSE，且设备运行在一个启用信标的网络上，NLME 首先通过发出 MLME-SET.request 原语，在 MAC 子层中设置 *macAutoRequest* PIB 属性。然后它发出 TrackBeacon 参数设置为 FALSE 的 MLME-SYNC.request 原语。然后 NLME 发出状态参数设置为 SUCCESS 的 NLME-SYNC.confirm 原语。

如果 TRACK 参数设置为 TRUE，且设备运行在一个没有启用信标的网络上，NLME 发出状态参数设置为 INVALID_PARAMETER 的 NLME-SYNC.confirm 原语。

如果 TRACK 参数设置为 TRUE，且设备运行在一个启用信标的网络上，NLME 首先通过发出 MLME-SET.request 原语，在 MAC 子层中设置 *macAutoRequest* PIB 属性。然后它发出 TrackBeacon 参数设置为 TRUE 的 MLME-SYNC.request 原语。然后 NLME 发出状态参数设置为 SUCCESS 的 NLME-SYNC.confirm 原语。

3.2.2.23 NLME-SYNC-LOSS.indication

该原语向上层通告 MAC 子层丢失网络同步信号。

3.2.2.23.1 服务原语的语义

本原语的语义如下：

NLME-SYNC.indication	{
	}

本原语没有参数。

3.2.2.23.2 原语生成

通过 LossReason 设置为 BEACON_LOST 的 MLME-SYNC-LOSS.indication 原语，从 MAC 子层收到一个丢失同步的通知之后，生成本原语。该原语跟随着 NLME-SYNC.request 原语之后，发送到网络层管理实体。

3.2.2.23.3 接收结果

上层被告知丢失了同步信标。

3.2.2.24 NLME-SYNC.confirm

本原语允许上层被告知其请求同步或从其 ZigBee 协调器或路由器中提取数据的结果。

3.2.2.24.1 服务原语的语义

本原语的语义如下：

NLME-SYNC.confirm	{
	Status
	}

表 3.28 指明了本原语的参数。

表 3.28 NLME-SYNC.confirm 参数

名称	类型	有效范围	描述
Status	Status	从MLME_POLL.confirm原语返回的 SUCCESS, SYNC_FAILURE, INVALID_PARAMETER或者任何状 态值	请求同步的ZigBee协 调器或路由器的结果

3.2.2.24.2 原语生成

本原语由发起 NLME 生成，发给其上层，以响应一个 NLME-SYNC.request 原语。如果请求成功，状态参数表示一个同步尝试。否则，状态参数表示一个错误代码。

这些状态值的原因的完整描述在 3.2.2.22.2 小节。

3.2.2.24.3 接收结果

在接收本原语时，上层被告知其请求同步或从其 ZigBee 协调器或路由器中提取数据的结果。如果 NLME 成功，状态参数将设置为 SUCCESS。否则，状态参数表示错误。

3.2.2.25 同步的信息顺序图

图 3.3 和图 3.4 说明了一个设备成功与一个 ZigBee 协调器或 ZigBee 路由器同步所需信息的顺序。图 3.3 说明了一个非信标网络的情况，图 3.4 说明了一个信标网络的情况。

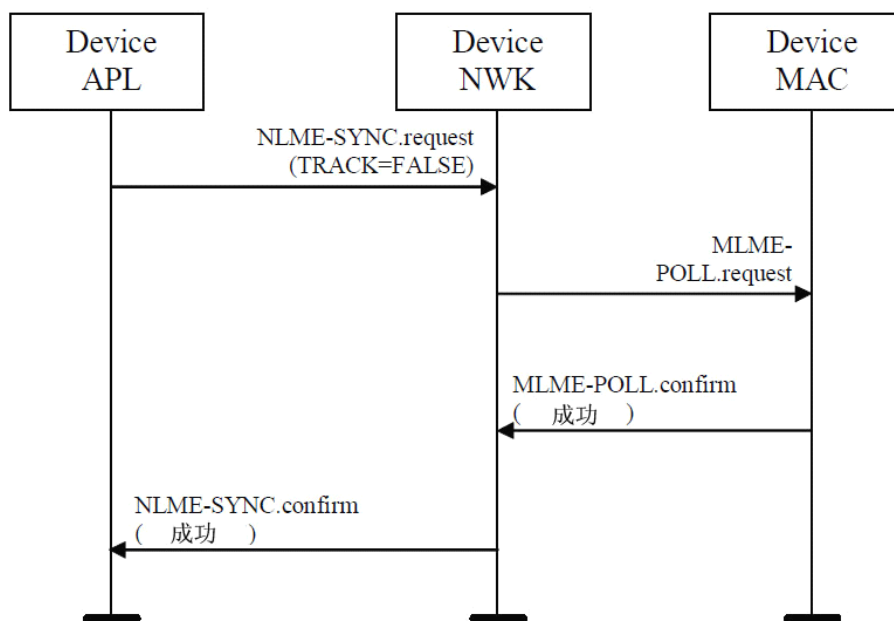


图 3.3 非信标网络同步的信息序列图

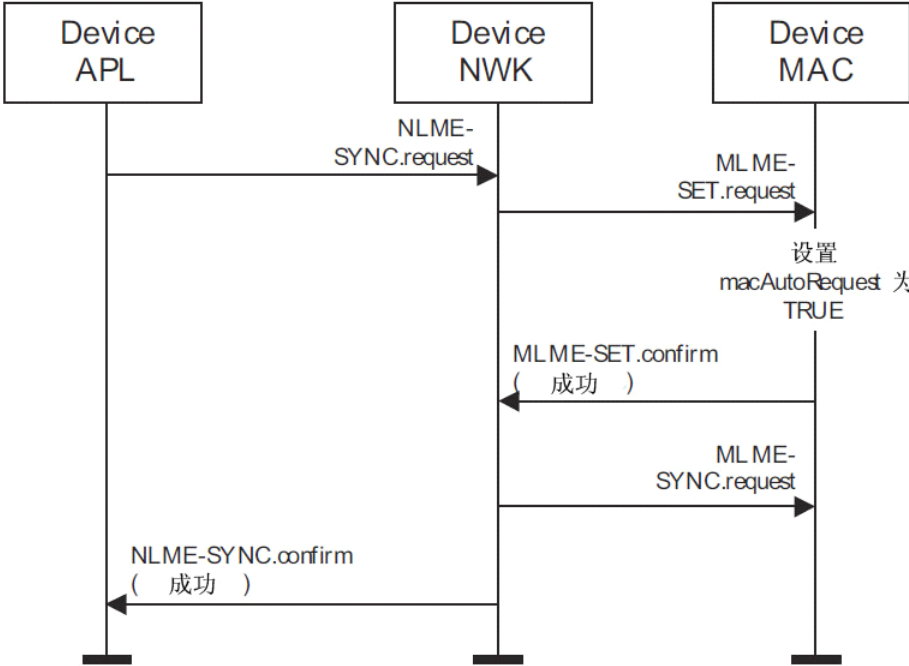


图 3.4 信标网络的信息序列图

3.2.2.26 NLME-GET.request

本原语允许上层从 NIB 中读取一个属性的值。

3.2.2.26.1 服务原语的语义

本原语的语义如下：

NLME-GET.request	{
	NIBAttribute
	}

表 3.29 指明了本原语的参数。

表 3.29 NLME-GET.request 参数

名称	类型	有效范围	描述
NIBAttribute	Integer	见表3.44	读取NIB属性的标识符

3.2.2.26.2 原语生成

本原语由上层生成，发给其 NLME，以从 NIB 中读取一个属性。

3.2.2.26.3 接收结果

在接收本原语时，NLME 尝试从其数据库中检索被请求的 NIB 属性。如果在数据库中没有找到 NIB 属性的标识符，NLME 发出状态值为 UNSUPPORTED_ATTRIBUTE 的 NLME-GET.confirm 原语。

如果成功检索到被请求的 NIB 属性，NLME 发出状态为 SUCCESS 的 NLME-GET.confirm 原语，并含有 NIB 属性标识符和值。

3.2.2.27 NLME-GET.confirm

本原语报告尝试从 NIB 中读取一个属性值的结果。

3.2.2.27.1 服务原语的语义

本原语的语义如下：

NLME-GET.confirm	{
	Status,
	NIBAttribute,
	NIBAttributeLength,
	NIBAttributeValue
	}

表 3.30 指明了本原语的参数。

表 3.30 NLME-GET.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS或 UNSUPPORTED_ATTRIBUTE	读取NIB属性值的 请求的结果
NIBAttribute	Integer	见表3.44	读取NIB属性的 标识符
NIBAttributeLength	Integer	0x0000 – 0xffff	返回的属性值的 字节的长度
NIBAttributeValue	Various	具体属性(见表3.44)	被读取的NIB属 性的值

3.2.2.27.2 原语生成

本原语由 NLME 生成，发给其上层，以响应一个 NLME-GET.request 原语。本原语返回一个 SUCCESS 状态，表示请求读取一个 NIB 属性成功，或一个错误代码 UNSUPPORTED_ATTRIBUTE。这些状态值的原因的完整描述在 3.2.2.26.3 小节中。

3.2.2.27.3 接收结果

在接收本原语时，上层被告知其请求读取一个 NIB 属性的结果。如果请求读取一个 NIB 属性成功，状态参数将设置为 SUCCESS，否则，状态参数表示错误。

3.2.2.28 NLME-SET.request

本原语允许上层往 NIB 中写入一个属性的值。

3.2.2.28.1 服务原语的语义

本原语的语义如下：

```
NLME-SET.request      {  
                        NIBAttribute,  
                        NIBAttributeLength,  
                        NIBAttributeValue  
                        }  
                        
```

表 3.31 指明了本原语的参数。

表 3.31 NLME-SET.request 参数

名称	类型	有效范围	描述
NIBAttribute	Integer	见表3.44	将要写的NIB属性的标识符
NIBAttributeLength	Integer	0x0000 – 0xffff	设置属性值的字节的长度
NIBAttributeValue	Various	具体属性(见表3.44)	被写的NIB属性的值

3.2.2.28.2 原语生成

本原语由上层生成，发给其 NLME，以在 NIB 中写入一个属性值。

3.2.2.28.3 接收结果

在接收本原语时, NLME 尝试把给定值写入在其数据库中指定的 NIB。如果 NIBAttribute 参数指定了一个数据库中找不到的属性, NLME 发出状态为 UNSUPPORTED_ATTRIBUTE 的 NLMESET.confirm 原语。如果 NIBAttribute 参数指定了一个在给定属性的有效值之外的值, NLME 发出状态为 INVALID_PARAMETER 的 NLME-SET.confirm 原语。

如果被请求的 NIB 属性成功写入, NLME 发出状态为 SUCCESS 的 NLME-SET.confirm 原语。

3.2.2.29 NLME-SET.confirm

本原语报告尝试给一个 NIB 属性写入一个值的结果。

3.2.2.29.1 服务原语的语义

本原语的语义如下:

NLME-SET.confirm	{
	Status,
	NIBAttribute
	}

表 3.32 指明了本原语的参数。

表 3.32 NLME-SET.confirm 参数

名称	类型	有效范围	描述
Status	Enumeration	SUCCESS, INVALID_PARAMETER或 UNSUPPORTED_ATTRIBUTE	写NIB属性请求的结果
NIBAttribute	Integer	见表3.44	被写的NIB属性的标识符

3.2.2.29.2 原语生成

本原语由 NLME 生成, 发给其上层, 以响应一个 NLME-SET.request 原语。本原语返回一个 SUCCESS 状态, 表示被请求的值写入到指定的 NIB 属性, 或一个错误代码 INVALID_PARAMETER 或 UNSUPPORTED_ATTRIBUTE。这些状态值的原因的完整描述在 3.2.2.28.3.小节。

3.2.2.29.3 接收结果

在接收本原语时，上层被告知其请求写入一个 NIB 属性的值的结果。如果被请求的值写入到指定的 NIB 属性，状态参数将设置为 SUCCESS。否则，状态参数表示错误。

3.2.2.30 NLME-NWK-STATUS.indication

本原语允许上层被告知通过网络的失败。

3.2.2.30.1 服务原语的语义

本原语的语义如下：

```
NLME-ROUTE-ERROR.indication    {  
                                   Status  
                                   NetworkAddr,  
                                   }
```

表 3.33 指明了本原语的参数。

表 3.33 NLME-ROUTE-ERROR.indication 参数

名称	类型	有效范围	描述
Status	Status	任何网络状态代码(见表3.42)	与失败有关的错误代码
NetworkAddr	Integer	0x0000 – 0xFFFF7	与状态信息有关的设备的16位网络地址

3.2.2.30.2 原语生成

当下列情况其中一个发生时，本原语由一个设备的 NWK 层生成，传递给上层：

- 设备没有成功发现或修复一条到 NetworkAddr 参数给定的目标地址的路由
- 由于表 3.42 给定的原因之一，该设备的 NWK 层没有成功发送到一个帧到其最终设备子节点，即 NetworkAddr 参数给定的 16 位网络地址
- NWK 层收到一个送往该设备的路由错误命令帧。在这种情况下，NetworkAddr 和 Status 参数的值将反映目标地址的值和命令帧的错误代码域。

3.2.2.30.3 接收结果

上层被告知一次与确定设备的通信失败。

3.2.2.31 NLME-ROUTE-DISCOVERY.request

本原语允许上层发起路由发现。

3.2.2.31.1 服务原语的语义

本原语的语义如下：

NLME-ROUTE-DISCOVERY.request	{
	DstAddrMode,
	DstAddr,
	Radius
	NoRouteCache
	}

表 3.34 指明了本原语的参数。

表 3.34 NLME-ROUTE-DISCOVERY.request 参数

名称	类型	有效范围	描述
DstAddrMode	Integer	0x00 – 0x02	参数指明了提供目的地址的种类。 DstAddrMode参数可能是下列三个值中的一个： 0x00 =没有目的地址 0x01 = 16位网络组播地址 0x02 =单个设备16位网络地址
DstAddr	16-bit NWK Address	任何网络地址或者组播地址	路由发现的目的地。 如果DstAddrMode 参数值为0x00 那么将不提供 DstAddr. 这代表路由发现将是一个设备发送发现命令作为目标的多对一的发现。 如果 DstAddrMode 参数值为0x01,代表组播路由发现那么目的地应该是一个组播的16位网络地址。如果 DstAddrMode 参数的值为0x02, 这就代表单播路由发现。 DstAddr是将被发现的设备的16位网络地址
Radius	Integer	0x00 – 0xff	此可选参数描述了路由请求通过网络传输的跳数。
NoRouteCache	Boolean	TRUE或 FALSE	当DstAddrMode值为零的情况下，指示多对一路由发现，本标志决定NWK是否应该建立一个路由记录表。TRUE=不应该建立路由记录表 FALSE=建立一个路由记录表

3.2.2.31.2 原语生成

本原语由一个 ZigBee 协调器或路由器的上层生成，发给其 NLME 来请求发起路由发现。

3.2.2.31.3 接收结果

一个 ZigBee 终端设备的 NLME 收到本原语时，NLME 必须给上层发出状态值为 INVALID_REQUEST 的 NLME-ROUTE-DISCOVERY.confirm 原语。

NLME 收到本原语，且 DstAddrMode 参数不等于 0x00，DstAddr 参数等于一个广播地址时，NLME 必须给上层发出状态值为 INVALID_REQUEST 的 NLME-ROUTE-DISCOVERY.confirm 原语。

一个 ZigBee 路由器或 ZigBee 协调器的 NLME 收到本原语，且没有寻找路由功能，DstAddrMode 参数等于 0x01 或 0x02 时，NLME 将给上层发出状态值为 ROUTE_ERROR 的 NLME-ROUTED-DISCOVERY.confirm，NetworkStatusCode 值为 0x04 表示没有路由功能。

一个 ZigBee 路由器或 ZigBee 协调器收到本原语，且具有寻找路由功能，DstAddrMode 参数等于 0x02 时，NLME 必须在当前设备和 DstAddr 参数给定的 16 位 NWK 地址的网络设备之间，发起一个单播路由的发现。发起一个单播路由的发现的程序在 3.6.3.5.1 小节做了概述。

一个 ZigBee 路由器或 ZigBee 协调器收到本原语，且具有寻找路由功能，DstAddrMode 参数等于 0x01 时，NLME 必须首先检查设备是否是 DstAddr 参数指明的多播组的一个成员。即它将检查在 NIB 属性 nwkGroupIDTable 中是否有一个条目对应于目标地址。如果设备是一个多播组的成员，那么 NLME 将立即发出状态值为 SUCCESS 的 NLME-ROUTE-DISCOVERY.confirm 原语，并停止进一步处理 NLME-ROUTEDISCOVERY.request 原语。如果设备不是一个多播组的成员，NLME 将在当前设备和 DstAddr 参数确定的多播组之间，发起一个单播路由的发现。

一个 ZigBee 路由器或 ZigBee 协调器收到本原语，且 DstAddrMode 参数等于 0x00 时，NLME 必须发起多对一的路由发现。发起多对一的路由发现的程序在 3.6.3.5.1 小节做了概述。

在以上描述的三种实际的路由发现中，NLME 将通过使用 MAC 子层的 MCPS-DATA.request 原语，尝试传输一个路由发现命令帧来发起路由发现。如果提供了一个值给可选的 Radius 参数，该值将被放在向外发送的帧的 NWK 头的半径域中。如果没有提供一个值，那么 NWK 头的半径域将设置为 NWK IB 的 nwkMaxDepth 属性值的两倍，用于数据帧传输的情况。如果 MAC 子层因为任何原因传输路由请求命令帧失败，NLME 将给上层发出 ROUTE-DISCOVERY.confirm 原语，状态参数值等于 MCPS-DATA.confirm 返回的值。如果路由发现命令帧成功发送，且如果 DstAddrMode 参数值为 0x00，表示多对一的路由发现，NLME 将立即发出值为 SUCCESS 的 ROUTE-DISCOVERY.confirm 原语。否则，NLME 将等待直到收到一个路由回复命令帧，否则路由操作超时，如 3.6.3.5 小节所述。如果在路由操作超时之前收到一个路由回复命令帧，NLME 将向上层发出状态值为 SUCCESS 的 NLME-ROUTE-DISCOVERY.confirm 原语，状态值为 SUCCESS。如果操作超时，它将发出状态为 ROUTE_ERROR 的 NLME_ROUTE-DISCOVERY.confirm 原语，NetworkStatusCode 值反映失败的原因，见表 3.42。

3.2.2.32 NLME-ROUTE-DISCOVERY.confirm

本原语通知上层关于尝试发起路由发现的结果。

3.2.2.32.1 服务原语的语义

本原语的语义如下：

NLME-ROUTE-DISCOVERY.confirm	{
	Status
	NetworkStatusCode
	}

表 3.35 指明了 NLME-ROUTEDISCOVERY.confirm 原语的参数。

表 3.35 NLME-ROUTE-DISCOVERY.confirm 参数

名称	类型	有效范围	描述
Status	状态值	从MCPS-DATA.confirm 原语返回的 INVALID_REQUEST,NO_ROUTING_CAPACITY,ROUTE_DISCOVERY_FAILED或者任何状态值	初始化路由发现的状态
NetworkStatusCode	状态码	见表格3.42	当Status 参数值为 ROUTE_ERROR,本码给出本错误的更多信息，否则应该被忽略。

3.2.2.32.2 原语生成

本原语由 NLME 生成，传递给上层，作为尝试发起路由发现的结果。

3.2.2.32.3 接收结果

上层被告知其尝试发起路由发现的状态。状态参数可能的值和生成的环境描述在

3.2.2.31.3 小节中。¹³

3.3 帧格式

本小节指明了 NWK 帧（NPDU）的格式。每个 NWK 帧包括以下基本的组件：

- 一个 NWK 头，包括帧控制，寻址和顺序信息
- 一个 NWK 负载，长度可变，包含指定帧类型的信息

NWK 层的帧被描述为有一个具体顺序的一系列域。本小节中的所有帧按照它们由 MAC 子层传输的顺序说明，从左至右，最左边的位首先传输。每个域中的位从 0 开始编号（最左边和最低位）直到 k-1（最右边和最高位），则该域的长度为 k 位。长于一个字节的域发送给 MAC 子层，按照低位字节到高位字节的顺序。

3.3.1 通用 NPDU 帧格式

NWK 帧格式包括一个 NWK 头和一个 NWK 负载。NWK 头的域以一个固定顺序出现。但是，多播控制域仅在多播标志值为 1 的时候出现。NWK 帧必须按照图 3.5 所示的格式编排。

字节数：2	2	2	1	1	0/8	0/8	0/1	可变的	可变的
帧控制	目的地址	源地址	半径域	序列号	目的 IEEE 地址	源 IEEE 地址	多播传送控制	源路由子帧	帧负载
NWK 头									负载

图 3.5 通用网络帧格式

13. CCB #822

3.3.1.1 帧控制域

帧控制域长度为 16 位，包含定义帧类型，寻址，顺序域和其它控制标志的信息。帧控制域按图 3.6 所示的格式编排。

位数: 0-1	2-5	6-7	8	9	10	11	12	13-15
帧类型	协议版本	发现路由	多播标记	安全	源路由	目的 IEEE 地址	源 IEEE 地址	保留

图 3.6 帧控制域

该表展示了 NWK 数据帧的允许的帧控制子域的配置，注意下面列出的所有帧的帧类型子域都等于 00，表示数据和协议版本子域反映执行的 ZigBee 协议版本。

表格 3.36 允许的帧控制子域配置

数据传输方法	发现路由	多播	安全	目的 IEEE 地址	源 IEEE 地址
单播	00 或者 01	0	0 或者 1	0 或者 1	0 或者 1
广播	00	0	0 或者 1	0	0 或者 1
多播	00	1	0 或者 1	0	0 或者 1
源路由	00	0	0 或者 1	0 或者 1	0 或者 1

3.3.1.1.1 帧类型子域

帧类型子域长度为 2 位，必须设置为表 3.37 所列的非保留值之一。

表 3.37 帧类型子域的值

帧类型值 b_1b_0	帧类型名称
00	数据
01	网络命令
10 – 11	保留

3.3.1.1.2 协议版本子域

协议版本子域长度为 4 位，必须设置为反映使用的 ZigBee NWK 协议版本的一个数字。一个设备使用的协议版本必须可以作为 NWK 常量 *nwkProtocolVersion* 的值。

3.3.1.1.3 发现路由子域

发现路由子域可以用于为该帧的传输控制路由发现操作（参见 3.6.3.5 小节）。

表 3.38 发现路由子域的值

发现路由域的值	域含义
0x00	禁止路由发现
0x01	允许路由发现
0x02, 0x03	保留

对于 NWK 层命令帧，发现路由子域必须设置为 0x00，表示禁止路由发现。

3.3.1.1.4 多播标志子域

多播标志子域长度为 1 位，如果该帧是一个单播或广播，帧值为 0，如果它是一个多播帧值为 1。仅当多播标志值为 1，NWK 头的多播控制域才会出现。

3.3.1.1.5 安全子域

当且仅当该帧启用 NWK 安全操作时，安全子域值为 1。如果在另一个层为该帧执行了安全功能，或完全禁用，它的值必须为 0。

3.3.1.1.6 源路由子域

当且仅当一个源路由子帧出现在 NWK 头中时，源路由子域值为 1。如果源路由子帧没有出现，源路由子域值为 0。

3.3.1.1.7 目标 IEEE 地址子域

当且仅当网络头包含目的地完整的 IEEE 地址时，源路由子域值为 1。

3.3.1.1.8 源 IEEE 地址子域

当且仅当网络头包含源设备完整的 IEEE 地址时，源 IEEE 地址子域值为 1。

3.3.1.2 目标地址域

目标地址域必须总是出现，且长度必须是 2 个字节。如果帧控制域的多播标志子域值为 0，目标地址域不应持有目标设备或广播地址的 16 位网络地址（参见表 3.54）。如果多播标志子域值为 1，目标地址域必须持有目标多播组的 16 位组 ID。注意一个设备的网络地址必须设置为 MAC PIB 属性 *macShortAddress* 的值。

3.3.1.3 源地址域

源地址域必须总是出现。它的长度将总是 2 个字节，并必须持有帧中源设备的网络地址。注意一个设备的网络地址必须设置为 MAC PIB 属性的 *macShortAddress* 的值。

3.3.1.4 半径域

半径域必须总是出现。它的长度将是 1 个字节，指明一个传输的半径范围。该域必须被每个接收设备减 1。

3.3.1.5 序列号域

序列号域出现在每个帧中，长度为 1 个字节。序列号的值将在每次传输新的帧后加 1。源地址的值和帧的序列号域，作为一对，可以在序列号的 1 个字节长度范围的限制之内，用于唯一确定一个帧。关于使用序列号域更多的详细信息，参见 3.6.2 小节。

3.3.1.6 目标 IEEE 地址域

如果目标 IEEE 地址域存在的话，包括 NWK 头目标地址域所含的 16 位网络地址对应的 64 位 IEEE 地址。在收到包含 64 位 IEEE 地址的帧后，应该检查 *nwkAddressMap* 和邻居表的内容，必要的时候更新。3.6.1.9.2 小节描述了检测地址冲突时采取的行动。如果 16 位网络地址是个广播或者多播地址的话，那么目标 IEEE 地址域就不能出现。

3.3.1.7 源 IEEE 地址域

如果源 IEEE 地址域存在的话，包括 NWK 头源地址域所含的 16 位网络地址对应的 64 位 IEEE 地址。在收到包含 64 位 IEEE 地址的帧后，应该检查 *nwkAddressMap* 和邻居表的内容，必要的时候更新。3.6.1.9.2 小节描述了检测地址冲突时采取的行动。

3.3.1.8 多播控制域

多播控制子域长度为 1 个字节，仅当多播标志子域值为 1 时出现。如图 3.7 所示，它分为三个子域。

位：0-1	2-4	5-7
Multicast mode	NonmemberRadi us	MaxNonmemberRadi us

图 3.7 多播控制域的格式

3.3.1.8.1 多播模式子域

多播模式子域表示帧是否使用成员或非成员模式传输。成员模式用于在目标组成员的设备之间宣传多播。非成员模式用于从一个不是多播组成员的设备到一个属于多播组部分的设备，传输一个多播帧。

表 3.39 多播模式子域的值

组播模式域的值	域含义
0x0	非成员模式
0x1	成员模式
0x2	保留
0x3	保留

3.3.1.8.2 NonmemberRadius 子域

当一个成员模式多播被不属于目标组成员的设备中继时，NonmemberRadius 子域表示中继的范围。属于目标组成员的接收设备将设置该域为 MaxNonmemberRadius 子域的值。如果 NonmemberRadius 域值为 0，不属于目标组成员的接收设备将丢弃该帧，如果 NonmemberRadius 域值在 0x01 和 0x06 的范围之间，将递减该域。值 0x07 表示一个无限范围，且不再递减。

NonmemberRadius 子域的值将不会超越 MaxNonmemberRadius 子域的值。

3.3.1.8.3 MaxNonmemberRadius 子域

这个帧的 NonmemberRadius 域的最大值。

3.3.1.9 源路由子帧域

源路由子帧域仅当帧控制域的源路由子域值为 1 时出现。如图 3.8 所示，它分为三个子域。

字节数: 1	1	可变的
中继计数	中继索引	中继列表

图 3.8 源路由子帧格式

3.3.1.9.1 中继计数子域

中继计数子域表示源路由子帧的中继列表子域所含的中继次数。

3.3.1.9.2 中继索引

中继索引子域表示中继列表子域中数据包将被传输到的下一个中继的索引。该域由数据包的发起者初始化为 0，且被每个接收中继加 1。

3.3.1.9.3 中继列表子域

中继列表子域包括一个中继地址的列表，最接近目标地址的中继先被列出。最接近发起者的中继最后被列出。

3.3.1.10 帧负载域

帧负载域长度可变，包含指定不同帧类型的信息。

3.3.2 不同帧类型的格式

定义了两种 NWK 帧类型：数据帧和 NWK 命令帧。这些帧类型将在以下小节一一讨论。

3.3.2.1 数据帧格式

数据帧按图 3.9 所示的格式编排。

字节数: 2	可变的	可变的
帧控制	路由域	数据负载
NWK 头		网络负载

图 3.9 数据帧格式

数据帧的域的顺序必须遵守图 3.5 所示的通用 NWK 帧格式的顺序。

3.3.2.1.1 数据帧 Nwk 头域

NWK 头域必须包含帧控制域, 并根据需要结合一个适当的路由域。

在帧控制域中, 帧类型子域必须包含表示一个数据帧的值, 如表 3.37 所示。所有其它子域必须根据数据帧的预定用途设置。

路由域必须根据帧控制域的设置, 适当地结合一个地址域和广播域 (参见图 3.6)。

3.3.2.1.2 数据负载域

一个数据帧的数据负载域必须包括上层请求 NWK 层传输的字节序列。

3.3.2.2 NWK 命令帧格式

NWK 命令帧格式必须按图 3.10 所示的格式编排。

字节数: 2	可变的	1	可变的
帧控制	路由域	网络命令标识符	网络命令负载
网络头		网络负载	

图 3.10 网络命令帧的格式

NWK 命令帧的域的顺序必须遵守图 3.5 所示的通用 NWK 帧的顺序。

3.3.2.2.1 NWK 命令帧的 NWK 头域

NWK 命令帧的 NWK 头域必须包含帧控制域, 并根据需要结合的一个合适的路由域。

在帧控制域中, 帧类型子域必须包括表示一个 NWK 命令帧的值, 如表 3.37 所示。所有其它子域必须根据 NWK 命令帧的预定用途设置。

路由域必须根据帧控制域的设置，并根据需要适当地结合一个地址域和广播域。

3.3.2.2.2 NWK 命令标识符域

NWK 命令标识符域表示使用的 NWK 命令。该域必须设置为表 3.40 所列的非保留值之一。

3.3.2.2.3 NWK 命令负载域

一个 NWK 命令帧的 NWK 命令负载域必须包含 NWK 命令本身。

3.4 命令帧

NWK 层定义的命令帧列在表 3.40 中。以下小节详细介绍了 NLME 如何为传输构造不同的命令。

对于这样的每个命令，以下条件适用于 NWK 头域，除非各个命令中有特殊规定。

- NWK 帧控制域的帧类型子域应该设置为指示该帧是一个 NWK 命令帧。
- NWK 帧头中的发现路由子域应该设置为抑制路由发现（见表 3.38）。
- NWK 帧头中的源地址域应该设置为发起设备的地址。

表 3.40 网络命令帧

命令帧标识符	命令名称	子章节
0x01	路由请求	3.4.1
0x02	路由回复	3.4.2
0x03	网络状态	3.4.3
0x04	离开	3.4.4
0x05	路由记录	3.4.5
0x06	重新加入请求	3.4.6

表 3.40 网络命令帧（续表）

0x07	重新加入响应	3.4.7
0x08	连接状态	3.4.8
0x09	网络报告	3.4.9
0x0a	网路更新	3.4.10
0x0b – 0xff	保留	—

3.4.1 路由请求命令

路由请求命令允许一个设备请求在无线设备范围内的其它设备参与查找一个特定的目标设备，并在网络内建立一个状态，这将在未来允许信息更容易、经济地通过路由送到目标地址。路由请求命令的负载必须按照图 3.11 所示的格式编排。

字节数: 1	1	2	1	0/8
命令选项	路由请求标识符	目的地址	路由成本	目的 IEEE 地址
网络命令负载				

图 3.11 路由请求命令帧的格式

3.4.1.1 MAC 数据服务要求

为了使用 MAC 数据服务传输该命令，IEEE 802.15.4-2003 【B1】指定，以下信息必须包含在 MAC 帧的头：

- 目标 PAN 标识符必须设置为发送路由请求命令的设备的 PAN 标识符
- 目标地址必须设置为广播地址 0xffff
- 源地址和 PAN 标识符必须设置为发送路由请求命令的设备的地址和 PAN 标识符，该设备不一定是命令的发起设备。
- 由于任何从 NWK 层发出的加密的帧必须使用 NWK 层安全功能，帧控制域必须设置为指定帧是一个 MAC 数据帧的值，且 MAC 安全功能启用；因为帧是广播的，不能指定确认请求
- 寻址模式和 intra-PAN 标志必须设置以支持这里描述的寻址域

3.4.1.2 NWK 头域

为了使路由请求到达它的目的地以正确完成路由发现程序，必须提供下列信息：

- NWK 头的目标地址域必须设置为所有设备广播的地址 *macRxOnWhenIdle* = TRUE。（见表格 3.54）。
- 帧控制域的源 IEEE 地址子域应该设置为 1，NWK 头的 IEEE 地址域必须存在且必须包含帧发起者的 64 位 IEEE 地址。

3.4.1.3 NWK 负载域

NWK 帧负载包含一个命令标识符域，一个命令选项域，路由请求标识符域，预定目的地的地址，一个最新的路径消耗汇总，和目标 IEEE 地址。

命令帧标识符应该包括表示它是一个路由请求命令帧的值。

3.4.1.3.1 命令选项域

8 位的命令选项域的格式如图 3.12 所示。

位：0-2	3-4	5	6	7
保留	多对一	目的 IEEE 地址	多播	保留

图 3.12 路由请求命令选项域

3.4.1.3.1.1 多对一

多对一域应该包含表格 3.41 中的一个非保留值。

表 3.41 多对一域值

值	描述
0	路由请求不是一个多对一路由请求
1	路由请求是一个多对一路由请求，发送者支持一个路由记录表
2	路由请求是一个多对一路由请求，发送者不支持一个路由记录表
3	保留

3.4.1.3.1.2 目标 IEEE 地址

目标 IEEE 地址域是一个仅有 1 位的域。当且仅当命令帧包含目标 IEEE 地址时，它的值为 1。如果已知的话目标 IEEE 地址域应该被加上。¹⁴

3.4.1.3.1.3 多播子域

多播子域是一个仅有 1 位的域。当且仅当命令帧是发给一个路由器或一个多播组的请求时，它的值为 1，在这种情况下目标地址域包含期望组的组 ID。

3.4.1.3.2 路由请求标识符

路由请求标识符是一个 8 位的序列号，用于路由请求，一个特定设备的 NWK 层每次发出一个路由请求，它就增加 1。

3.4.1.3.3 目标地址

目标地址长度必须为 2 个字节，表示路由请求命令帧的预定目标地址。

3.4.1.3.4 路径消耗

路径消耗域长度为 8 个字节，当一个路由请求命令帧在网络中移动时，用于累加路由消耗信息（参见 3.6.3.5.2 小节）。

3.4.1.3.5 目标 IEEE 地址

目标 IEEE 地址应该是 8 字节长，代表路由请求命令帧的目标 IEEE 地址。只有在命令帧选项域的目标 IEEE 地址子域值为 1 时才会存在。

3.4.2 路由回复命令

路由回复命令允许一个路由请求命令指定的目标设备通知路由请求的发出者，该请求已经被接收。它还允许 ZigBee 路由器沿着路由请求采取的路径来建立状态信息，这将使得源设备发送给目标设备的帧更有效地传输。路由回复命令必须按图 3.13 所示的格式编排。

字节数: 1	1	2	2	1	0/8	0/8
命令选项	路由请求标识符	发起者地址	响应者地址	路由成本	发起者 IEEE 地址	响应者 IEEE 地址
NWK 命令负载						

图 3.13 路由回复命令格式

3.4.2.1 MAC 数据服务要求

为了使用 MAC 数据服务传输这个命令，IEEE 802.15.4-2003[B1]指定，以下信息必须包含在 MAC 帧的头：

- 目标 MAC 地址和 PAN 标识符必须分别设置为回到相应路由请求命令帧的发起者的路径第一跳的网络地址和 PAN 标识符。目标 PAN 标识符必须和发起者的 PAN 标识符相同。
- 源 MAC 地址和 PAN 标识符必须设置为发出路由请求命令的设备的地址和 PAN 标识符，它可以是或可以不是发起命令的设备。
- 由于从 NWK 层发出的加密的帧必须使用 NWK 层安全功能，帧控制域必须设置为指定该帧是一个 MAC 数据帧的值，且 MAC 安全功能启用。传输选项必须设置为请求确认。寻址模式和 intra-PAN 标志必须设置以支持这里描述的寻址域。

3.4.2.2 NWK 头域

为了使这个路由回复到达其目的地，且路由发现程序能正确完成，必须提供以下信息：

- NWK 头中的源地址应该设置为传送帧设备的 16 位网络地址。
- NWK 头的目标地址域必须设置为回到相应路由请求的发出者的路径的第一跳的网络地址。
- 因为这是一个 NWK 层命令帧，帧控制域的源 IEEE 地址子域应该设置为 1，NWK 头中的源 IEEE 地址应该指定，应该包含帧发起者的 64 位 IEEE 地址。帧控制域的目的 IEEE 地址子域应该设置为 1，NWK 头中的目的 IEEE 地址应该指定，应该包含相应路由请求发起者第一跳的 64 位 IEEE 地址。

3.4.2.3 NWK 负载域

NWK 帧的负载包含一个命令标识符域，一个命令选项域，路由请求标识符，发出者和响应者地址和一个最新的路径消耗汇总。

命令帧标识符必须包含表示一个路由回复命令帧的值。

3.4.2.3.1 命令选项域

8 位的命令选项域的格式如图 3.14 所示。

位: 0-3	4	5	6	7
保留	发起者 IEEE地 址	响应者 IEEE地址	多播	保留

图 3.14 路由回复命令选项域

3.4.2.3.1.1 发起者 IEEE 地址

发起者 IEEE 地址子域是仅有 1 位的域。当且仅当发起者 IEEE 地址包含在负载中时，它的值为 1。当 nwkUniqueAddr 是 FALSE，这个位应该设置。

3.4.2.3.1.2 响应者 IEEE 地址

响应者 IEEE 地址子域是仅有 1 位的域。当且仅当响应者 IEEE 地址包含在负载中，它的值为 1。当 nwkUniqueAddr 是 FALSE，这个位应该设置，并且多播子域应该设置为 0。

3.4.2.3.1.3 多播子域

多播子域是一个 1 位的域。当且仅当命令帧用于回复一个路由器的请求，发给一个多播组时，它的值为 1。在这种情况下响应者的地址域包含期望组的组 ID。

3.4.2.3.2 路由请求标识符

路由请求标识符是该帧所回复的路由请求的 8 位序列号。

3.4.2.3.3 发起者地址

发起者地址域必须长为 2 个字节，且必须包含该帧所回复的路由请求命令帧发出者的 16 位网络地址。

3.4.2.3.4 响应者地址

响应者地址域必须长为 2 个字节，且必须总是和相应的路由请求命令帧的目标地址域的值相同。

3.4.2.3.5 路径消耗

路径消耗域用于合计路由回复命令帧在网络中传输的链路消耗（参见 3.6.3.5.3 小节）。

3.4.2.3.6 发起者 IEEE 地址

发起者 IEEE 地址域应该是 8 字节长，应该包含路由请求命令帧发起者的 64 位扩展地址，该帧是个回复。命令选项域的响应者 IEEE 地址子域值为 1 时，这个域才能出现。

3.4.2.3.7 响应者 IEEE 地址

响应者 IEEE 地址域应该是 8 字节长，应该包含路由请求命令帧发起者的 64 位扩展目的地址，该帧是个回复。命令选项域的响应者 IEEE 地址子域值为 1 时，这个域才能给出。

3.4.3 网络状态命令

设备使用网络状态命令报告从特定设备的 NWK 层到网络中其他设备的对等 NWK 层实体产生的错误和其他情况。NWK 状态命令还可以用来诊断网络问题，例如地址冲突。一个网络状态命令的负载必须按图 3.15 所示的格式编排。

字节数: 1	2
状态码	目的地址
NWK 命令负载	

图 3.15 网络状态命令帧的格式

3.4.3.1 MAC 数据服务要求

为了使用 MAC 数据服务传输这个命令，IEEE 802.15.4-2003[B1]指定，必须提供以下信息：

- 目标 MAC 地址和 PAN 标识符必须分别设置为返回面临转发失败的数据帧的源的路径第一跳的地址和 PAN 标识符。
- 源 MAC 地址和 PAN 标识符必须设置为发送路由错误命令的设备的地址和 PAN 标识符。
- 由于 NWK 层发出的任何加密的帧必须使用 NWK 层安全功能，帧控制域必须指明该帧是一个 MAC 数据帧，且启用 MAC 安全功能。如果目的地 MAC 地址是广播地址 0xffff，传输选项不能被设置为请求确认。
- 寻址模式和 intra-PAN 标志必须设置为支持这里描述的寻址域。

3.4.3.2 NWK 头域

网络状态命令可以是单播或广播的。NWK 头域必须设置如下：

- 源地址域总是设置为发起命令帧的设备的 16 位网络地址。
- 帧控制域的源 IEEE 地址子域设置为 1，NWK 头的源 IEEE 地址域应存在，包含帧的发起者的 64 位 IEEE 地址。
- 当为了响应一个路由错误发送，NWK 头的目标地址域设置为和面临转发失败的数据帧源地址相同的值。
- 当且仅当网络状态命令帧不是广播的，帧控制域的目标 IEEE 地址子域值为 1，如果 IEEE 地址已知，NWK 头的目标 IEEE 地址域应存在，包含对应于目标地址域 16 位网络地址的 64 位 IEEE 地址。

3.4.3.3 NWK 负载域

路由错误命令帧的 NWK 帧负载包括一个命令帧标识符域，一个错误代码域和一个目标地址域，如下所述。命令帧标识符必须设置，以指明是表 3.40 定义的路由错误命令帧。

3.4.3.3.1 状态代码

状态代码必须设置为表 3.42 所示的非保留值之一。

表 3.42 网络状态命令帧的状态代码

值	状态代码
0x00	无有效路由
0x01	树连接失败
0x02	非树连接失败
0x03	低电池电量
0x04	无路由能力
0x05	无间接能力
0x06	间接传送到期
0x07	目标设备不可用
0x08	目标地址未分配
0x09	父连接失败
0x0a	无效路由
0x0b	源路由失败
0x0c	多对一路由失败
0x0d	地址冲突
0x0e	认证地址
0x0f	PAN识别符更新
0x10	网络地址更新
0x11	坏帧计算器 ^a
0x12	坏密钥序列号 ^b
0x13–0xff	保留

a. CCB #765

b. CCB #673

这些状态代码可用于一个路由状态命令帧的状态代码域的值和NLME-NWK-STATUS.indication原语的状态参数的值。每个简单的解释如下：

- **无路由可用：**已经尝试了路由发现和/或修复，但是没有发现到预定目标地址的路由。

- **树连接失败：**因为尝试沿着树为该帧寻找路由失败，发生路由失败。
- **非树连接失败：**失败的发生不是因为尝试沿着树寻找路由。
- **低电池电量：**因为中继设备正在电池电力不足的情况下运行，帧没有被中继。
- **无路由能力：**因为中继设备没有路由能力，发生失败。
- **无间接能力：**因为尝试为一个睡眠的终端设备子节点缓冲一个帧且中继设备没有缓冲能力可以使用，发生失败。
- **间接传输到期：**代表一个终端睡眠设备子节点被缓冲的帧因为超时被丢弃。
- **目标设备不可用：**中继设备的一个终端设备子节点因为某种原因不可用。
- **目标地址未分配：**目标地址是中继设备一个不存在的终端设备子节点的帧。
- **父节点连接失败：**因为到设备的父节点 RF 连接失败，发生此次失败。
- **无效的路由：**目标地址域确定的多播路由应该是有效的。
- **源路由失败：**源路由失败，可能表示源路由的连接之中的一个连接失败。
- **多对一路由失败：**一个多对一的路由请求建立一条路由失败。
- **地址冲突：**目标地址域中的地址已经被决定给两个或两个以上设备使用。
- **认证地址：**源设备已有 IEEE 地址在来源 IEEE 地址域中，如果目的地的 IEEE 地址字段存在，它的值包含的是预计的目的地 IEEE 地址。
- **PAN 识别符更新：**已经更新后设备的可操作的 PAN 识别符
- **网络地址更新：**已经更新后设备的
- 网络地址

- **坏帧计数器：**接受帧的计数器有值小于或等于 *nwkSecurityMaterialSet*.¹⁵
- **坏密钥序列号：**接受帧的序列号不匹配 *nwkActiveKeySeqNumber*.¹⁶

3.4.3.3.2 目标地址

目标地址域长度为 2 个字节，当且仅当网络状态帧为了响应路由失败而发送时才存在。在此情况下，必须包含面临转发失败的数据帧的目标地址。

3.4.4 离开命令

离开命令由 NLME 使用，来通知网络中的其它设备一个设备正要离开网络，或请求一个设备离开网络。离开命令的负载必须按图 3.16 所示的格式编排。

1
命令选项
NWK命令负载

图 3.16 离开命令帧格式

3.4.4.1 MAC 数据服务要求

为了使用 MAC 数据服务传输该命令，IEEE802.15.4-2003[B1]指定，必须提供以下信息：

- 目标 MAC 地址和 PAN 标识符必须分别设置为帧要被送到的相邻设备的地址和 PAN 标识符，或者如果 NWK 头也包括一个广播地址，发送到 MAC 广播地址 0xffff。
- 源 MAC 地址和 PAN 标识符必须设置为发送离开命令的设备的地址和 PAN 标识符。
- 由于 NWK 层发出的任何加密的帧必须使用 NWK 层安全功能，帧控制域必须设置为指明该帧是一个 MAC 数据帧，且启用 MAC 安全功能。必须有一个确认。

15. CCB #765

16. CCB #765

- 寻址模式和 intra-PAN 标志必须设置为支持这里描述的寻址域。

3.4.4.2 NWK 头域

离开命令帧的 NWK 头域应该设置如下：

- 帧控制域的源 IEEE 地址子域必须设置为 1，NWK 头的源地址域必须存在，且应该包含帧发起者的 64 位 IEEE 地址。
- 如果命令选项域的请求子域设置为 1，那么 NWK 头的目标地址域应该设置为要求离开的子设备的网络地址。
- 如果请求子域设置为 0，NWK 头的目标地址域应该设置为 0xfffd，这样指令就被 *macRxOnWhenIdle* 等于 TRUE 的设备接收。
- 当且仅当命令帧不是广播的，帧控制域的目标 IEEE 地址子域设置为 1，目标 IEEE 地址域必须设置为被请求离开的设备的 IEEE 地址。
- 半径域必须设置为 1。

3.4.4.3 NWK 负载域

离开命令帧的 NWK 负载包含一个命令帧标识符域和一个命令选项域。命令帧标识符域必须设置为指明它是表 3.40 所述的离开命令帧。

3.4.4.3.1. 命令选项域

8 位的命令选项域的格式如图 3.17 所示。

位: 0-4	5	6	7
保留	重新加入	请求	移除子节点

图 3.17 离开命令选项域

3.4.4.3.1.1 Rejoin 子域

Rejoin 子域是 1 位，位置在第 5 位。如果该子域的值 1，从其当前父节点离开的设备将重新加入网络。如果该子域的值 0，设备将不再重新加入网络。

3.4.4.3.1.2 Request 子域

Request 子域是 1 位域，位置在第 6 位。如果该子域的值为 1，那么离开命令帧请求另一个设备离开网络。如果该子域的值为 0，那么离开命令帧表示发送设备打算离开网络。

3.4.4.3.1.3 Remove Children 子域

Remove Children 子域是 1 位，位置在第 7 位。如果该子域的值为 1，那么离开网络的设备的子节点也将被移除。如果子域有一个值为 0，那么离开网络的设备的子节点不被移除。

3.4.5 路由记录命令

路由记录命令允许把一个单播数据包在通过网络时采取的路由记录在命令负载中，并发送给目标设备。路由记录命令的负载必须按图 3.18 所示的格式编排。

字节数: 1	可变的
中继计数	中继列表
网络命令负载	

图 3.18 路由记录命令格式

3.4.5.1 MAC 数据服务要求

为了使用 MAC 数据服务传输该命令，IEEE802.15.4-2003[B1]指定，必须提供以下信息:

- 目标 MAC 地址和 PAN 标识符必须分别设置为帧要被送到的相邻设备的地址和 PAN 标识符。
- 源 MAC 地址和 PAN 标识符必须设置为发送路由记录命令的设备的地址和 PAN 标识符。
- 由于 NWK 层发出的任何加密的帧必须使用 NWK 层安全功能，帧控制域必须设置为指明该帧是一个 MAC 数据帧，且启用 MAC 安全功能。必须需要一个确认。
- 寻址模式和 intra-PAN 标志必须设置为支持这里描述的寻址域。

3.4.5.2 NWK 头域

路由记录命令帧的 NWK 头域应该设置如下：

- 如果路由记录被作为上一层 NLDE-DATA.request 原语的结果启动，源地址域应该设置为帧发起者的 16 位网络地址。如果路由记录作为代表一个终端子设备中继数据帧的结果启动，源地址域应该包含终端子设备的 16 位网络地址。
- 帧控制域的源 IEEE 地址子域应该设置为 1，NWK 头的源 IEEE 地址域必须存在，且应该包含在源地址子域内的 16 位网络地址对应的 64 位 IEEE 地址。
- NWK 头的目的地址域应该设置为集中器的设备的 16 位网络地址，即帧的目的地。
- 帧控制域的目的 IEEE 地址子域应该设置为 1，目的 IEEE 地址域应该设置为集中器设备的 IEEE 地址，即帧的目的地，如果地址已知。
- 帧控制域的源路由子域必须设置为 0。

3.4.5.3 NWK 负载

NWK 帧负载包含一个命令标识符域，一个中继计数器域，和一个中继列表域。命令帧标识符必须包含表示它是一个路由记录命令帧的值。

3.4.5.3.1 Relay Count 域

本域包含路由记录命令的中继列表域的中继次数。如果路由记录作为来自上层的 NLDE-DATA.request 原语的结果启动，中继计数域初始化为 0。如果路由记录作为代表一个终端子设备中继数据帧的结果启动，中继计数域初始化为 1。在任何一种情况下，它被每个接受到的中继递增。

3.4.5.3.2 中继列表域

中继列表域是一个列表，包含中继该数据包的节点的 16 位网络地址。如果路由记录作为代表一个终端子设备中继数据帧的结果启动，发起设备将会用自己的 16 位网络地址初始化本域。接收的中继节点在转发数据包之前附加其网络地址到列表中。

3.4.6 重新加入请求命令

重新加入请求命令允许一个设备重新加入它的网络。这通常用来响应一次通信失败，比如当一个终端设备不能再与其原始父节点通信的时候。

字节数: 1
能力信息
NWK命令负载

图 3.19 重新加入请求命令帧格式

3.4.6.1 MAC 数据服务要求

为了使用 MAC 数据服务传输该命令，IEEE802.15.4-2003[B1]指定，必须提供以下信息：

- 目标 MAC 地址和 PAN 标识符必须分别设置为未来的父节点的地址和 PAN 标识符。
- 源 MAC 地址和 PAN 标识符必须设置为传输重新加入命令帧的设备之前的短地址和 PAN 标识符。
- 传输选项必须设置为需要确认。
- 寻址模式和 intra-PAN 标志必须设置为支持这里描述的寻址域。

3.4.6.2 NWK 头域

重新加入请求命令帧的 NWK 头域应该设置如下：

- NWK 头的源地址域应该设置为发出请求的设备的 16 位网络地址。
- 帧控制域的源 IEEE 地址子域应该设置为 1，源 IEEE 地址域应该设置为发送请求设备的 IEEE 地址。
- NWK 头的目的地址域应该设置为预期的父设备的 16 位网络地址。
- 帧控制域的目的 IEEE 地址子域应该设置为 1，目的 IEEE 地址域应该设置为预期的父设备的 IEEE 地址，如果地址已知。
- 半径域应该设置为 1。

3.4.6.3 NWK 负载域

NWK 帧负载包含一个命令标识符域和一个能力信息域。命令帧标识符必须包含表示它是一个重新加入命令帧的值。

3.4.6.3.1 能力信息域

这个 1 字节的域的格式和【B1】的关联请求命令中的能力信息域相同，也描述在表 3.47 中。

3.4.7 重新加入响应命令

重新加入响应命令由一个设备发送，通知一个子节点其短地址和重新加入状态。

字节数: 2	1
网络地址	重新加入状态
NWK命令负载	

图 3.20 重新加入响应命令帧的格式

3.4.7.1 MAC 数据服务要求

为了使用 MAC 数据服务传输该命令，【B1】指定，必须提供以下信息：

- 目标 MAC 地址和 PAN 标识符必须分别设置为请求重新加入网络的设备之前的网络地址和 PAN 标识符。
- 源 MAC 地址和 PAN 标识符必须设置为接收并处理重新加入命令帧的设备的网络地址和 PAN 标识符。
- 必须需要一个确认。
- 寻址模式和 intra-PAN 标志必须设置为支持这里描述的寻址域。如果重新加入请求命令的“Capability Information”字节的“Receiver on when idle”位等于 0x00，TXOptions 将请求使用“间接传输”。否则将使用“直接传输”。

3.4.7.2 NWK 头域

重新加入响应命令帧的 NWK 头域应该设置如下：

- 源地址域应该设置为发出响应的设备的 6 位网络地址。
- 帧控制域的源 IEEE 地址子域应该设置为 1，源 IEEE 地址域应该存在，且包含发送响应的父设备的 64 位 IEEE 地址。
- NWK 头的目的地址域应该设置为重新加入设备的当前网络地址，即发送加入响应的设备，此帧为响应。
- 帧控制域的目的 IEEE 地址子域应该设置为 1，NWK 头的目的 IEEE 地址域应该存在，且应该包含子节点设备的 64 位 IEEE 地址，它是重新加入请求命令的源，此帧为响应。
- NWK 层应该把重新加入响应命令的安全和收到的重新加入请求命令帧的（此帧为响应）安全设置为同一个安全等级。

3.4.7.3 NWK 负载域

3.4.7.3.1 网络地址域

如果重新加入成功，这个 2 字节的域包含分配给重新加入设备的新的短地址。如果重新加入不成功，这包含广播地址（0xffffd）。

3.4.7.3.2 重新加入状态域

这个 1 字节的域必须包含[B1]指定的非保留的关联状态值之一。

3.4.8 链路状态命令

链路状态命令允许附近的路由器可以相互通信自己的输入链路消耗，见 3.6.3.4 小节。链路状态帧通过单跳广播传输。没有重试。

3.4.8.1 MAC 数据服务要求

为了使用 MAC 数据服务传输该命令，IEEE 802.15.4-2003 【B1】指定，以下信息必须包含在 MAC 帧的头：

- 目标 PAN 标识符必须设置为发送链路状态命令的设备的 PAN 标识符。
- 目标地址必须设置为广播地址 0xffff。
- 源 MAC 地址和 PAN 标识符必须设置为发送链路状态命令的设备的地址和 PAN 标识符。
- 由于任何从 NWK 层发出的加密的帧必须使用 NWK 层安全功能，帧控制域必须设置为指定帧是一个 MAC 数据帧的值，且 MAC 安全功能启用；因为帧是广播的，不能指定请求确认。
- 寻址模式和 intra-PAN 标志必须设置以支持这里描述的寻址域。

3.4.8.2 NWK 头域

链路状态命令帧的 NWK 头域应该设置如下：

- 帧控制域的源地址域，源 IEEE 地址子域，NWK 头的源 IEEE 地址应该按照一般的 NWK 命令帧设置。
- NWK 头的目标地址应该设置为仅路由器广播地址（见表 3.54）。
- 帧控制域的目标 IEEE 地址子域应该是 0，NWK 头的目标 IEEE 地址不应存在。
- 半径域应该设置为 1。

3.4.8.3 NWK 负载域

链路状态命令的 NWK 负载域格式见图 3.21。

字节：1	可变
命令选项	链路状态表
NWK 命令负载	

图 3.21 链路状态格式

3.4.8.3.1 命令选项域

8 字节命令选项域的格式如图 3.22。

位：0-4	5	6	7
条目数	第一帧	最后帧	保留

图 3.22 链路状态选项域

命令选项域的条目数子域指明了链路状态表中的链路状态条目数。如果这是发送者链路状态的第一帧，第一帧子域设置为 1，如果这是发送者链路状态的最后帧，最后帧子域设置为 1。如果发送者链路状态正好为一帧，第一帧和最后帧位都应该设置为 1。

3.4.8.3.2 链路状态表域

链路状态表的条目格式如图 3.23。

字节：2	1
邻居网络地址	链路状态

图 3.23 链路状态条目

链路状态条目按网络地址以递增顺序排序。如果所有的路由器邻居不适合一个帧，应发送多个帧。当发送多个帧时，帧 N 的链路状态列表的最后网络地址等于帧 N+1 的链路状态列表的第一个网络地址。

每个链路状态条目包含一个路由器邻居的网络地址，首先是最低字节，其次是链路状态字节。输入消耗域包含设备估计的邻居的链路消耗，这是 1 至 7 之间的一个值。输出消耗域包括来自邻居表的输出消耗域的值。

链路状态条目的链路状态域的格式如下：

位：0-2	3	4-6	7
输入消耗	保留	输出消耗	保留

3.4.9 网络报告命令

网络报告命令允许设备报告网络事件给 NIB 属性 *nwkManagerAddr* 所含地址指定的设备。这类事件是无线设备信道条件和 PAN ID 冲突。网络报告命令的负载格式如图 3.24。

字节：1	8	可变
命令选项（见图 3.25）	EPID	报告信息
NWK 命令负载		

图 3.24 网络报告命令帧格式

3.4.9.1 MAC 数据服务要求

为了使用 MAC 数据服务传输该命令，IEEE 802.15.4-2003【B1】指定，以下信息必须包含在 MAC 帧的头：

- 目标 PAN 标识符必须设置为发送网络报告命令的设备的 PAN 标识符。
- 目标地址必须设置为路由表条目中下一跳地址域的值，其中该条目的目标地址域的值和 NIB 属性 *nwkManagerAddr* 的值相同。如果没有这样的路由表条目，那么 NWK 会尝试路由发现，见 3.6.3.5 小节。
- 源 MAC 地址和 PAN 标识符必须设置为发送网络报告命令的设备的地址和 PAN 标识符，该设备不一定是发送命令的源设备。
- 由于任何从 NWK 层发出的加密的帧必须使用 NWK 层安全功能，帧控制域必须设置为指定帧是一个 MAC 数据帧的值，且 MAC 安全功能禁止；传输选项应该指定要求请求确认。

3.4.9.2 NWK 头域

网络报告命令帧的 NWK 头域应该设置如下：

- 帧控制域的源地址域，源 IEEE 地址子域和 NWK 头域的源 IEEE 地址子域应该按照一般的 NWK 命令帧设置。
- NWK 头的目标地址域应该设置为 NIB 属性 *nwkManagerAddr* 所含的 16 位网络地址。
- 帧控制域的源 IEEE 地址子域的值应该为 1，NWK 头的源 IEEE 地址域应该存在，包含 NIB 中 *nwkManagerAddr* 属性所含的 16 位网络地址对应的 64 位 IEEE 地址。

3.4.9.3 NWK 负载域

NWK 帧负载包含一个命令标识符域，一个命令选项域，EPID 域和一个报告信息负载。命令帧标识符应该包括指示这是一个网络报告命令帧的值。

3.4.9.3.1 命令选项域

8 位的命令选项域的格式如图 3.25 所示。

位：0-4	5-7
报告信息计数	报告命令标识符（见图 3.26）

图 3.25 网络报告命令选项域

3.4.9.3.1.1 报告信息计数子域

报告信息计数子域包含一个整数，指示报告信息域所含的记录个数。记录大小取决于报告命令标识符的值。

3.4.9.3.1.2 报告命令标识符子域

报告命令标识符子域包含一个整数，指示报告信息命令的类型。表 3.26 包含可以插入本域的值。

报告命令标识符值	报告类型
0x00	PAN 标识符冲突
0x01 - 0x07	保留

图 3.26 报告命令命令标识符子域¹⁷

3.4.9.3.2 EPID 域

EPID 域包含 64 位 EPID，识别报告设备是哪个网络的成员。

17. CCB #823

3.4.9.3.3 报告信息

报告信息域提供要报告的信息，本域格式取决于报告命令标识符子域的值。

3.4.9.3.3.1 PAN 标识符冲突报告

如果报告命令标识符子域指示有一个 PAN 标识符冲突，那么报告信息域格式如图 3.27。

字节: 2	2	2
第一个 PAN ID		第 n 个 PAN ID

图 3.27 PAN 标识符冲突报告

PAN 标识符冲突报告应该由运行在报告设备附近区域的 16 位 PAN 标识符列表组成。PAN ID 冲突报告中的 PAN 标识符个数应该等于命令选项域的报告信息计数子域的值。

3.4.10 ¹⁸ 网络更新命令

网络更新命令允许 NIB 中 *nwkManagerAddr* 属性指定的设备把配置信息的改变广播给网络中所有设备。例如，广播网络正准备改变它的短 PAN 标识符。

网络更新命令的负载格式如图 3.28。

字节: 1	8	1	可变
命令选项(见图 3.25)	EPID	更新 ID	更新信息
NWK 命令负载			

图 3.28 网络更新命令帧的格式

3.4.10.1 MAC 数据服务要求

为了使用【B1】指定的 MAC 数据服务传输该命令，以下信息必须包含在 MAC 帧的头:

¹⁸. CCB #823

- 为了使命令帧到达没有收到这个更新命令的网络设备，目标 PAN 标识符必须设置为 ZigBee 协调器的旧 PAN 标识符。目标地址必须根据 3.6.5 小节所述的广播传输的程序设置。
- 源 MAC 地址和 PAN 标识符必须设置为发送网络报告命令的设备的网络地址和旧 PAN 标识符，该设备不一定是发起命令的源设备。
- 由于 NWK 层发起的任何加密帧都使用 NWK 层安全服务，帧控制域必须设置为指明帧是一个 MAC 数据帧的值，且 MAC 安全禁用。

3.4.10.2 NWK 头域

网络更新命令的 NWK 头域设置如下：

- 帧控制域的源地址域，源 IEEE 地址子域和 NWK 头域的源 IEEE 地址子域应该按照一般的 NWK 命令帧设置。
- NWK 头域的目标地址应该设置为广播地址 0xffff。
- 帧控制域的目标 IEEE 地址子域值应该为 0，NWK 头域的目标 IEEE 地址子域不应存在。

3.4.10.3 NWK 负载域

NWK 帧负载包含一个命令标识符域，一个命令选项域，一个 EPID 域和一个更新信息可变域。

命令帧标识符应该包括指示这是一个网络更新命令帧的值。

3.4.10.3.1 命令选项域

8 位的命令选项域的格式如图 3.29 所示。

位：0-4	5-7
更新信息计数	更新命令标识符（见图 3.30）

图 3.29 网络更新命令选项域

3.4.10.3.1.1 更新信息计数子域

更新信息计数子域包含一个整数，指示更新信息域的记录个数。记录大小取决于更新命令标识符子域的值。

3.4.10.3.1.2 更新命令标识符子域

更新命令标识符子域包含一个整数，指示更新信息命令的类型。图 3.30 包含可以插入本域的值：

更新命令标识符值	报告类型
0x00	PAN 标识符更新
0x01 - 0x07	保留 ^a

图 3.30 更新命令识别符子域

3.4.10.3.2 EPID 域

EPID 域包含 64 位 EPID，识别要被更新的网络。

3.4.10.3.3 更新 Id 域

更新 Id 域反映发送此帧的设备的 *mwkUpdateId* 属性的当前值。

3.4.10.3.4 更新信息

更新信息域提供要更新的信息，本域的格式取决于更新命令标识符子域的值。

3.4.10.3.4.1 PAN 标识符更新

如果更新命令标识符子域指示有一个 PAN 标识符更新，那么更新信息域的格式如图 3.31。

字节: 2
新的 PAN ID

图 3.31 PAN 标识符更新

PAN 标识符更新由一个 16 位 PAN 标识符组成，它是这个网络使用的新 PAN 识别符。更新信息计数子域应等于 1，因为更新信息域只含有一个 PAN 识别符。

3.5 常量和 NIB 属性¹⁹

3.5.1 NWK 常量

定义 NWK 层特点的常量列在表 3.43 中。

表 3.43 网络层常量

常量	描述	值
<i>nwkcCoordinatorCapable</i>	一个布尔标志，表明该设备是否能成为 ZigBee 的协调器; 0x00 代表不能成为协调器，0x01 代表设备能成为协调器	独立于配置
<i>nwkcDefaultSecurityLevel</i>	使用默认安全级别(见第4章)	定义在栈 profile 中
<i>nwkcDiscoveryRetryLimit</i>	路由发现重试的最大次数	0x03
<i>nwkcMinHeaderOverhead</i>	通过网络层到 NSDU 添加的最小字节数	0x08
<i>nwkcProtocolVersion</i>	设备上的 ZigBee 网络协议的版本	0x02

19. CCB #824

表 3.43 网络层常量（续表）

常量	描述	值
<i>nwkcWaitBeforeValidation</i>	在接收路由回复和发送有效路由信息之间，多播路由请求发送者持续时间（单位：毫秒）	0x500
<i>nwkcRouteDiscoveryTime</i>	路由发现到期之前持续的时间，以毫秒为单位。	0x2710
<i>nwkcMaxBroadcastJitter</i>	以毫秒为单位的最大的广播抖动测量时间。	0x40
<i>nwkcInitialRREQRetries</i>	首次广播传输的路由请求命令帧重试的次数	0x03
<i>nwkcRREQRetries</i>	广播传输路由请求命令帧通过中间的 ZigBee 路由器或者 ZigBee 协调器中继重传的次数	0x02
<i>nwkcRREQRetryInterval</i>	在广播路由请求命令帧之间重试的毫秒数	0xfe
<i>nwkcMinRREQJitter</i>	路由请求命令帧广播转发的最小抖动，以2毫秒插槽为单位	0x01
<i>nwkcMaxRREQJitter</i>	路由请求命令帧广播转发的最大抖动，以2毫秒插槽为单位	0x40
<i>nwkcMACFrameOverhead</i>	ZigBee网络层使用MAC头大小	0x0b

3.5.2 NWK 信息库

NWK 信息库（NIB）包括管理一个设备 NWK 层所需的属性。这些属性的每一个都可以分别使用 NLME-GET.request 和 NLME-SET.request 原语读或写，在只读栏包含一个值为 Yes 的属性除外。在此情况下，属性值可以使用 NLME-GET.request 原语读，但不能使用 NLME-SET.request 原语设置。一般地，这些只读属性通过其他机制设置。例如，*nwkSequenceNumber* 属性根据 3.6.2.1 小节进行设置，网络层每次发送一个帧就增加。NIB 的属性列在表 3.44 中。

表 3.44 NIB 属性

属性	ID	类型	只读	范围	描述	默认
<i>nwkSequenceNumber</i>	0x81	Integer	Yes	0x00 – 0xff	一个序列号用于标识离开帧（见3.6.2小节）	范围内的随机值
<i>nwkPassiveAckTimeout</i>	0x82	Integer	No	0x0000 – 0x2710	父设备和所有子设备转发广播信息允许的最大时间延时以秒为单位(被动确认超时)	定义在栈 profile中
<i>nwkMaxBroadcastRetries</i>	0x83	Integer	No	0x00 – 0x5	广播传输失败后允许重传的最大数	0x03
<i>nwkMaxChildren</i>	0x84	Integer	No	0x00 – 0xff	设备在它的当前网络允许拥有的子节点数。注意当 <i>nwkAddrAlloc</i> 值为 0x02 时，表示随即分配地址，这个属性的值是独立于执行程序的。	定义在栈 profile中
<i>nwkMaxDepth</i>	0x85	Integer	Yes	0x00 – 0xff	设备可以有的深度	定义在栈 profile中
<i>nwkMaxRouters</i>	0x86	Integer	No	0x01- 0xff	允许任何一个设备有几个路由器作为子节点；ZigBee 协调器确定网络中所有设备的这个值。如果 <i>nwkAddrAlloc</i> 是 0x02，不使用这个值。	定义在栈 profile中
<i>nwkNeighborTable</i>	0x87	Set	No	可变的	设备上邻居表目录的当前设置(见表3.48)	空集

表 3.44 NIB 属性(续表)

属性	ID	类型	只读	范围	描述	默认
<i>nwkNetworkBroadcastDeliveryTime</i>	0x88	Integer	No	0– 0xff	需要围绕整个网络的广播信息的延时,以秒为单位。这个值是根据其他NIB属性计算的。公式见3.5.2.1小节。	NA
<i>nwkReportConstantCost</i>	0x89	Integer	No	0x00-0x01	如果设为0,网络层应使用MAC层报告的LQI值计算来自所有邻居节点的链路消耗;否则,它应报告一个恒定的值。	0x00
<i>nwkRouteDiscoveryRetriesPermitted</i>	0x8a	Integer	No	0x00-x03	在失败路由请求之后允许重试的最大次数	nwkcDiscoveryRetryLimit
<i>nwkRouteTable</i>	0x8b	Set	No	可变的	设备路由表目录的当前设置 (见表 3.51)	空集
<i>nwkSymLink</i>	0x8c	Boolean	No	TRUE或FALSE	目前路由对称设置: TRUE意思是路由被看作由对称链路组成。路由发现期间创建了向前和向后的路由,他们是相同的。 FALSE意思是路由不被看作由对称链路组成。路由发现期间只存储向前的链路。	FALSE
<i>nwkCapabilityInformation</i>	0x8f	Bit vector	Yes	见表3.47	包含网络连接期间建立的设备能力信息	0x00

表 3.44 NIB 属性(续表)

属性	ID	类型	只读	范围	描述	默认
<i>nwkAddrAlloc</i>	0x90	Integer	No	0x00-0x02	一个值，决定分配地址使用的方法： 0x00 = 使用分布式地址分配 0x01 = 保留 0x02 = 使用随机地址分配	0x00
<i>nwkUseTreeRouting</i>	0x91	Boolean	No	TRUE 或 FALSE	一个标志，决定网络层是否应该承担使用分层路由的能力： TRUE = 承担分层路由的能力 FALSE = 绝不使用分层路由	TRUE
<i>nwkManagerAddr</i>	0x92	Integer	No	0x0000 - 0xffff7	指定网络信道管理器功能地址 ^a	0x0000
<i>nwkMaxSourceRoute</i>	0x93	Integer	No	0x00 - 0xff	源路由跳的最大数	0x0c
<i>nwkUpdateId</i>	0x94	Integer	No	0x00 - 0xFF	值指示操作节点网络设置的快照	0x00
<i>nwkTransactionPersistenceTime</i>	0x95	Integer	No	0x0000 - 0xffff	通过协调器存储并且通过它的信标指示的最大时间(在超帧周期里)。该属性反映了MAC PIB属性 <i>macTransactionPersistenceTime</i> 的值(见[B1])和更高层所做的任何修改也将反映在MAC PIB属性里。	0x01f4

表 3.44 NIB 属性(续表)

属性	ID	类型	只读	范围	描述	默认
<i>nwkNetworkAddress</i>	0x96	Integer	No	0x0000 - 0xffff7	设备用于和个域网通信的16位地址 该属性反映了MAC PIB属性 <i>macShortAddress</i> 的值 (见[B1])和更高层所做的任何修改也将反映在MAC PIB属性里。	0xffff
<i>nwkStackProfile</i>	0x97	Integer	No	0x00-0x0f	该设备使用的ZigBee栈profile标识符	
<i>nwkBroadcastTransaction Table</i>	0x98	Set	Yes	-	设备中广播事务表的当前设置 (见表3.55)	空集
<i>nwkGroupIDTable</i>	0x99	Set	No	Variable	组标识符的集合, 范围0x0000 - 0xffff,本设备是该组一个成员。	空集
<i>nwkExtendedPANID</i>	0x9a	64-bit extended address	No	0x0000000000000000-0xfffffffffffffffe	设备所在的个域网的扩展个域网标识符。 0x0000000000000000意思为扩展个域网标识符是未知的。	0x0000000000000000
<i>nwkUseMulticast</i>	0x9b	Boolean	No	TRUE 或 FALSE	一个标志, 决定多播信息发生的层。 TRUE=多播发生在网络层 FALSE=多播发生在APS层并使用APS头	TRUE
<i>nwkRouteRecordTable</i>	0x9c	Set	No	Variable	路由记录表 (见表3.45)	空集

表 3.44 NIB 属性(续表)

属性	ID	类型	只读	范围	描述	默认
<i>nwkIsConcentrator</i>	0x9d	Boolean	No	TRUE或 FALSE	一个标志，决定设备是否是集中器。 TRUE=设备是集中器 FALSE=设备不是集中器	FALSE
<i>nwkConcentratorRadius</i>	0x9e	Integer	No	0x00 - 0xff	集中器路由发现的跳计数半径	0x0000
<i>nwkConcentratorDiscoveryTime</i>	0x9f	Integer	No	0x00 - 0xff	在集中器路由发现之间的时间，以秒为单位。如果设置为0x0000，在启动时发现，只能由上层使用。	0x0000
<i>nwkSecurityLevel</i>	0xa0		No		第四章定义的安全属性	
<i>nwkSecurityMaterialSet</i>	0xa1		No		第四章定义的安全属性	
<i>nwkActiveKeySeqNumber</i>	0xa2		No		第四章定义的安全属性	
<i>nwkAllFresh</i>	0xa3		No		第四章定义的安全属性	
<i>nwkSecureAllFrames</i>	0xa5		No		第四章定义的安全属性	
<i>nwkLinkStatusPeriod</i>	0xa6	Integer	No	0x00 - 0xff	链路状态命令帧之间的时间，以秒为单位	0x0f
<i>nwkRouterAgeLimit</i>	0xa7	Integer	No	0x00 - 0xff	复位链路消耗为零之前错过的链路状态命令帧数	3

表 3.44 NIB 属性(续表)

属性	ID	类型	只读	范围	描述	默认
<i>nwkUniqueAddr</i>	0xa8	Boolean	No	TRUE或 FALSE	一个标志，决定网络层是否应该检测和校正冲突的地址。 TRUE=假定地址是唯一的 FALSE=地址可能不唯一	TRUE
<i>nwkAddressMap</i>	0xa9	Set	No	可变的	64位IEEE到16位网络地址映射图的当前集合（见表格3.46）	空集
<i>nwkTimeStamp</i>	0x8C ^b	Boolean	No	TRUE或 FALSE	一个标志，决定在输入和输出数据包中是否提供时间戳指示。 TRUE=提供时间指示 FALSE=不提供时间指示	FALSE
<i>nwkPANId</i>	0x80 ^c	16-bit PAN ID	No	0x0000 - 0xffff	这个NIB属性应该一直和 <i>macPANId</i> 有一样的值。	0xffff
<i>nwkTxTotal</i>	0x8D ^d	Integer	No	0x0000 - 0xffff	这个设备上NWK层单播传输的次数。 每次NWK层通过调用MAC子层MCPS-DATA.request原语传送一个单播帧，它应增加此计数器。如果对此属性执行一个NLME-SET.request原语，或如果属性 <i>nwkTxTotal</i> 回到0xffff，NWK层都应把包含在邻居表中的每个传输失败域复位为0x00。	0

a. CCB #819

b. CCB #830

- c. CCB #830
 d. CCB #830

表 3.45 路由记录表条目格式

域名称	域类型	有效范围	参考
Network Address	Integer	0x0000-0xfff7	本路由记录的目标网络地址
Relay Count	Integer	0x0000 - 0xffff	从集中器到目标的中继节点计数
Path	网络地址集合		代表从集中器到目标路由的网络地址

表 3.46 网络地址映射图

64 位 IEEE 地址	16 位网络地址
一个有效的 64 位 IEEE 地址 或如果未知为空值	0x0000 - 0xfff7

3.5.2.1 广播传送时间

广播传输的总传送时间，即广播被传送到网络中每个设备需要的时间，按照下列公式计算：

$$\begin{aligned}
 nwkBroadcastDeliveryTime = & 2 * nwkMaxDepth * \\
 & ((0.05 + (nwkMaxBroadcastJitter / 2)) + \\
 & nwkPassiveAckTimeout * nwkBroadcastRetries / \\
 & 1000)
 \end{aligned}$$

3.6 功能描述

3.6.1 网络和设备维护

所有的 ZigBee 设备必须提供以下功能：

- 加入一个网络
- 离开一个网络
- 重新加入一个网络

ZigBee 协调器和路由器必须提供以下另外的功能：

- 允许设备使用以下方法加入网络：
 - 来自于 MAC 的连接指示
 - 来自于应用层的明确的加入请求
 - 重新加入请求
- 允许设备使用以下方法离开网络：
 - 网络离开命令帧
 - 来自于应用程序的明确的离开请求
- 参与分配逻辑网络地址
- 维护一个相邻设备的列表

ZigBee 协调器必须提供建立一个新网络的功能。ZigBee 路由器和终端设备在一个网络中应提供便携支持。

3.6.1.1 建立一个新网络

建立一个新网络的程序通过使用 NLME-NETWORK-FORMATION.request 原语发起。只有 *nwkcCoordinatorCapable* 常量值为 0x01 的设备和当前没有加入一个网络的设备才可以尝试建立一个新网络。如果这个程序在任何其它设备上发起，NLME 将终止该程序，并通知上层这一不合法的请求。这通过发出状态参数设置为 INVALID_REQUEST 的 NLME-NETWORKFORMATION.confirm 原语实现。

当发起这个程序，NLME 必须首先请求 MAC 子层在一组指定的信道，或如 PHY 层（见 [B1]）所规定的默认的全部可用信道上执行一个能量检测扫描，以寻找可能的干扰。通过给 MAC 子层发出 MLME-SCAN.request 原语来发起一个信道扫描，ScanType 参数设置为能量检测扫描。通信的结果通过 MLME-SCAN.confirm 原语送回。如果仅指明了一个信道，就不需要这个扫描。

在收到一个成功的能量检测扫描结果时，NLME 必须根据递增的能量测量把信道排序，并丢弃那些能量级别超出可接受的级别的信道。一个可接受能量级别的选择留给执行程序。然后 NLME 必须通过发出 MLME-SCAN.request 原语执行一个主动扫描，ScanType 参数设置为主动扫描，且 ChannelList 设置为可接受信道的列表，以寻找其它的 ZigBee 设备。为了确定建立一个新网络最好的信道，NLME 必须审查返回的 PAN 描述符列表，找到现有网络数目最小的第一个信道，优先选择一个没有检测到网络的信道。

如果没有找到合适的信道，NLME 必须终止该程序，并通知上层这次启动的失败。这通过发出状态参数设置为 STARTUP_FAILURE 的 NLME-NETWORK-FORMATION.confirm 原语实现。

如果找到了一个合适的信道，NLME 必须为新网络选择一个 PAN 标识符。为此设备必须选择一个小于 0xffff 的、在已选的信道上还未被使用的随机 PAN 标识符。一旦 NLME 做了选择，它必须通过发出 MLME-SET.request 原语，设置 MAC 子层的 *macPANID* 属性为这个值。

如果没有唯一的 PAN 标识符可以选择，NLME 必须终止该程序，并通过发出状态参数设置为 STARTUP_FAILURE 的 NLME-NETWORK-FORMATION.confirm 原语，通知上层这次启动的失败。

一旦选定了一个 PAN 标识符，NLME 必须选择一个等于 0x0000 的 16 位网络地址，并设置 MAC 子层的 *macShortAddress* NIB 属性等于选定的网络地址。

一旦选定了一个网络地址，NLME 必须检查 NIB 的 *nwkExtendedPANId* 属性的值。如果该值是 0x0000000000000000，这个属性被初始化为 MAC 常量 *aExtendedAddress* 的值。

一旦检查了 *nwkExtendedPANId* 的值，NLME 必须通过给 MAC 子层发出 MLME-START.request 原语，开始运行新的 PAN。MLME-START.request 原语的参数必须根据 NLME-NETWORK-FORMATION.request 传递的那些参数值来设置，即信道扫描的结果和选定的 PAN 标识符。PAN 启动的状态通过 MLME-START.confirm 原语返回。

在收到 PAN 启动状态时，NLME 必须通知上层其请求初始化 ZigBee 协调器的状态。这通过发出 NLME-NETWORK-FORMATION.confirm 原语来实现，状态参数设置为从 MAC 子层返回的 MLME-START.confirm 原语的状态值。

成功启动一个新网络的程序的信息顺序图（MSC）如图 3.32 所示。

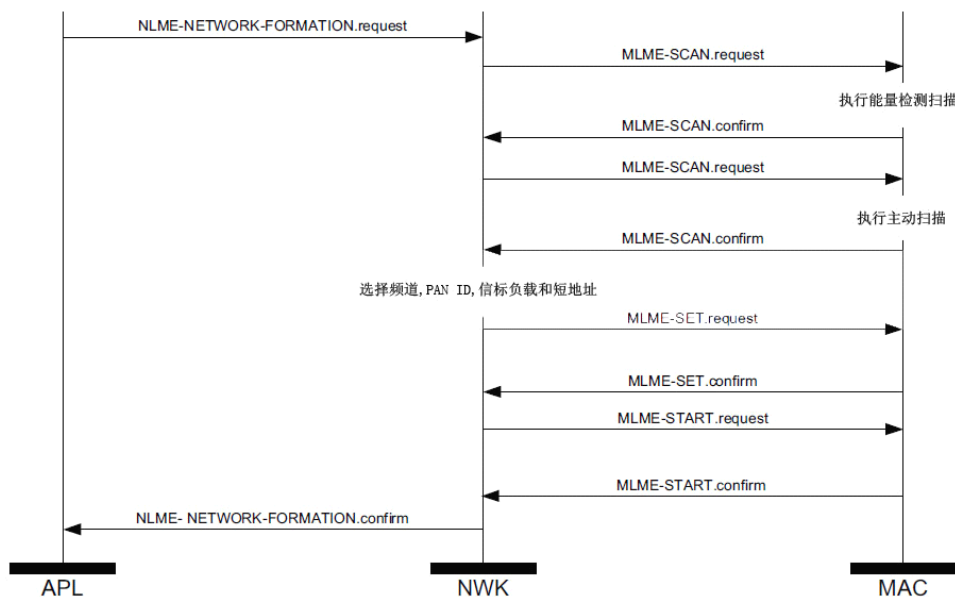


图 3.32 建立一个新网络

3.6.1.2 允许设备加入一个网络

允许设备加入一个网络的程序通过 NLME-PERMIT-JOINING.request 原语发起。只有 ZigBee 协调器或一个 ZigBee 路由器设备可以尝试允许设备加入网络。

当发起这个程序，且 PermitDuration 参数设置为 0x00，NLME 必须设置 MAC 子层的 *macAssociationPermit* 属性为 FALSE。通过发出 MLME-SET.request 原语来发起设置一个 MAC 子层属性。

当发起这个程序，且 PermitDuration 参数设置为 0x01 和 0xfe 之间的一个值，NLME 必须设置 MAC 子层的 *macAssociationPermit* 属性为 TRUE。然后 NLME 必须启动一个计时器，在指定的持续时间后过期。在这个计时器到期后，NLME 必须设置 MAC 子层的 *macAssociationPermit* 属性为 FALSE。

当发起这个程序，且 PermitDuration 参数设置为 0xff，NLME 必须设置 MAC 子层的 *macAssociationPermit* 属性为 TRUE，表示一个无限期的时间，除非发出另一个 NLME-PERMITJOINING.request 原语。

允许设备加入一个网络的程序在图 3.33 所示的 MSC 中说明。

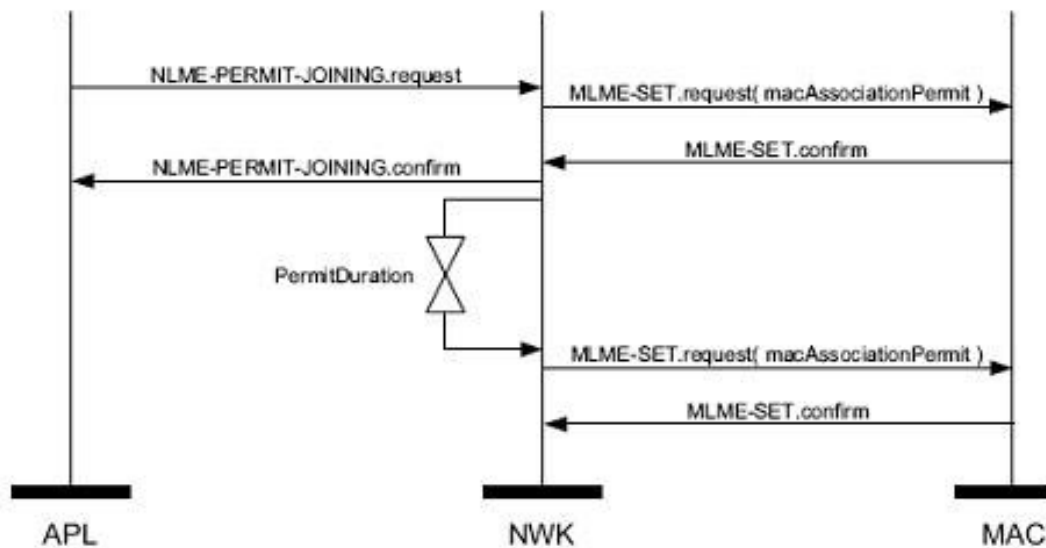


图 3.33 允许设备加入网络

3.6.1.3 网络发现

如果有的话，网络层使更高层发现哪个网络在一个设备的 POS 中是可操作的。

网络发现的程序通过发出 NLME-NETWORK-DISCOVERY.request 原语发起。ScanChannels 参数设置为指示网络扫描哪个信道，ScanDuration 参数设置为指示扫描每个信道花费的时间。在收到本原语时，NWK 层会发布 MLME-SCAN.request 原语，请求 MAC 子层执行一个主动扫描。

扫描期间收到的每个负载长度非零的信标帧，都会导致从扫描设备的 MAC 子层发出 MLME-BEACON-NOTIFY.indication 原语到它的 NLME。本原语包括例如信标设备的寻址信息，它是否是允许连接和信标负载这样的信息。（完整的参数列表见[B1]）。扫描设备的 NLME 应该检查信标负载中的协议 ID 域，证实它是否匹配 ZigBee 协议标识符。如果不匹配，信标被忽略。否则，设备会从每个收到的信标复制相关的信息（信标负载结构见图 3.49）到它的邻居表（邻居表条目内容见表格 3.48）。

一旦 MAC 子层通过发出 NLME-SCAN.confirm 原语到 NLME，表明扫描完成，NWK 层应该发出 NLME-NETWORK-DISCOVERY.confirm 原语，包含每个侦听网络的描述。每个网络描述包含 ZigBee 版本、栈 profile、扩展 PAN Id、PAN Id、逻辑频道和是否允许加入的信息（见表格 3.8）。

3.6.1.4 加入一个网络

为了保证讨论，当一个设备在网络中有资格允许新设备加入时。一个父子关系就形成了。加入时，新设备变为子节点，而第一个设备变为父节点。

3.6.1.4.1 通过连接加入一个网络

本小节指明了一个设备（子节点）如果选择使用 MAC 层提供的基本连接功能加入一个网络必须遵守的程序，以及一个 ZigBee 协调器或路由器（父节点）从 MAC 层接收 MLME-ASSOCIATE.request 原语必须遵守的程序。

3.6.1.4.1.1 子节点程序

使用 MAC 层连接程序加入一个网络应该用 3.6.1.3 小节描述的网络发现进行。在收到 NLME-NETWORK-DISCOVERY.confirm 原语之后，上层应该选择一个已发现网络去加入或者重做一遍网络发现。一旦选择了一个网络，然后它就发布一个 NLME-JOIN.request 原语，RejoinNetwork 参数设置为 0x00, JoinAsRouter 参数设置为表示设备是否想作为一个路由器设备加入。

只有那些还没加入到网络的设备可以启动加入程序。如果任何其它设备发起了这个程序，NLME 必须终止该程序，并通过发出状态参数设置为 INVALID_REQUEST 的 NLME-JOIN.confirm 原语通知上层这一非法的请求。

对于一个还未加入一个网络的设备，NLME-JOIN.request 原语必须引起 NWK 层为一个合适的父节点设备查询其相邻表。即一个合适的父节点设备满足下列条件：

- 设备属于 ExtendedPANId 参数定义的网络。
- 设备对加入请求开放，并宣传正确设备类型的功能。
- 按照 3.6.3.1 小节所述的计算，从这个设备收到的帧的链路质量产生的链路消耗至少为 3。
- 如果相邻表条目包含这个设备的一个潜在父节点域，这个域值必须是 1，表示该设备是一个潜在父节点。
- 设备必须有最新的 ID，确定最新 ID 需要考虑最新 ID 可能会回到 0。特别是设备的信标负载中的最新 ID 必须大于或等于——再次，补偿回到——NIB 的 *nwkUpdateId* 属性。

如果相邻表不包含合适的父节点设备，NLME 必须以状态参数为 NOT_PERMITTED 的一个 NLME-JOIN.confirm 原语响应。如果相邻表有不止一个可以作为一个合适父节点的设备，必须选择从 ZigBee 协调器深度最小的设备。如果不止一个设备具有最小的深度，NWK 层从它们中间自由选择。

一旦确定了一个合适的父节点，NLME 必须给 MAC 子层发出一个 MLME-ASSOCIATE.request 原语。MLME-ASSOCIATE.request 原语的 LogicalChannel 参数必须设置为从相邻表条目中找到的对应潜在父节点的协调器的地址。CapabilityInformation 参数的比特域是表 3.47 所示的值，能力信息存储为 *nwkCapabilityInformation* NIB 属性（见表 3.44）的值。如果不止一个设备满足这些要求，那么加入设备可以选择网络深度最小的父节点。

表 3.47 能力信息比特域

位	名称	描述
0	备用 PAN 协调器	执行本规范时该域值总是 0。
1	设备类型	如果加入设备是 ZigBee 路由器，该域为 1。如果设备是 ZigBee 终端设备，或者是作为终端设备加入的具有路由器功能的设备，值为 0。
2	电源	该域设置为传递给 NLME-JOIN-request 原语的 PowerSource 参数最低位的值。值是： 0x01 = 主电源供电的设备 0x00 = 其它电源
3	空闲时接收器开启	该域设置为传递给 NLME-JOIN-request 原语的 RxOnWhenIdle 参数最低位的值。 0x01 = 设备空闲时，接收器使能 0x00 = 设备空闲时，接收器可能禁止
4-5	保留	执行本规范时该域总是 0。
6	安全功能	如果设备运行在标准的安全模式下该域为 0，如果设备运行在高级安全模式下，为 1。注意这优先于[B1] ^a 中指定的默认含义。
7	分配地址	执行本规范时该域为 1，表示加入设备必须被分配一个 16 位网络地址，除了设备以安全模式方式，第一次使用 NWK 重新加入命令加入网络时，自己选择地址。在这种情况下，值为 0。

a. CCB #788

否则, NLME 发出 NLME-JOIN.confirm, 状态参数设置为 MLME-ASSOCIATE.confirm 原语返回的状态参数值。

如果 RejoinNetwork 参数是 0x00, JoinAsRouter 参数设置为 TRUE, 设备在网络中就像一个 ZigBee 路由器一样运行。如果 JoinAsRouter 参数是 FALSE, 那么它作为一个终端设备加入, 且不参与路由。

MLME-ASSOCIATE.request 原语(见第 2 章)的寻址参数必须设置为设备从邻居表中选择的寻址信息。连接的状态通过 MLME-ASSOCIATE.confirm 原语被送回 NLME。

如果尝试加入不成功, NWK 层必须从 MAC 子层接收一个状态参数表示错误的 MLME-ASSOCIATE.confirm 原语。如果状态参数表示一部分邻近设备拒绝允许加入(即 PAN 容量达到最大或 PAN 拒绝访问), 那么尝试加入的设备把相应的相邻表条目的 Potential parent 位设置为 0, 表示一个加入尝试失败。设置 Potential parent 位为 0 以确保 NWK 层不会发出另一个请求来连接同一个相邻设备。每次发出一个 MLME-SCAN.request 原语, 相邻表中的每个条目的 Potential parent 位必须设置为 1。

如果潜在父节点不允许新的路由器连接并且加入设备设置 JoinAsRouter 参数为 TRUE(例如, 与设备相连接的路由器可能已经达到最大个数 *nwkMaxRouters*), 那么一个加入请求也可能不成功。在这种情况下, NLME-JOIN.confirm 原语将指示一个 NOT_PERMITTED 状态。在这种情况下子节点设备的应用程序可以希望尝试更换作为一个终端设备再次加入, 通过发出另一个 JoinAsRouter 参数设置为 FALSE 的 NLME-JOIN.request。

如果尝试加入不成功, NLME 必须尝试从相邻表中找到另一个合适的父节点。如果找不到这样的设备, NLME 必须发出 NLME-JOIN.confirm 原语, 状态参数设置为 MLME-ASSOCIATE.confirm 原语返回的值。

如果尝试加入不成功, 且有第二个相邻设备可以作为一个合适的父节点, NWK 层必须为第二个设备发起 MAC 子层连接程序。NWK 层必须重复这个程序直到它成功加入 PAN, 或用完它加入 PAN 的选择。

如果设备不能成功加入上层指定的 PAN, NLME 必须通过发出 NLME-JOIN.confirm 原语终止程序, 状态参数设置为收到的最后一个 MLME-ASSOCIATE.confirm 原语返回的值。在这种情况下, 设备不能收到一个有效的逻辑地址, 也不能在网络上传输。

如果尝试加入成功，NWK发出状态值为SUCCESS的NLME-JOINconfirm原语。在这种情况下，NWK层收到的MLME-ASSOCIATE.confirm原语必须包含一个16位的逻辑地址，在该网络中是唯一的，子节点可以在未来的传输中使用。然后NWK层必须在相应的相邻表条目中设置Relationship域，来表示该邻居是它的父节点。到这个时候，父节点必须把每个新设备增加到其相邻表中。此外，NWK层将在NIB中更新*nwkNetworkAddress*、*nwkUpdateId*和*nwkPANId*的值。

如果设备尝试加入一个安全的网络且它是一个路由器，它将需要在传输信标之前等待，直到其父节点对它进行认证。因此设备必须等待从上层发出一个 NLME-START-ROUTER.request 原语。在接收这个原语时，如果它是一个路由器 NLME 必须发出一个 MLME-START.request 原语。如果 NLME-STARTROUTER.request 原语在一个终端设备上发出，NWK 层必须发出一个状态值设置为 INVALID_REQUEST 的 NLME-START-ROUTER.confirm 原语。

一旦设备成功加入网络，如果它是一个路由器，且上层已经发出了一个 NLME-START-ROUTER.request，NWK 层必须给其 MAC 子层发出 MLME-START.request 原语。PANId、LogicalChannel、BeaconOrder 和 SuperframeOrder 参数必须设置为等于其父节点相邻表条目中相应的值。网络深度的设置至少要比父节点网络深度大 1，除非父节点网络深度值为 0x0f，即信标负载的 4 位设备深度域的最大值。在这种情况下，网络深度也设置为 0x0f。PANCoordinator 和 CoordRealignment 参数都必须设置为 FALSE。在接收 MLME-START.confirm 原语时，NWK 层必须发出一个具有相同状态值的 NLME-START-ROUTER.confirm 原语。

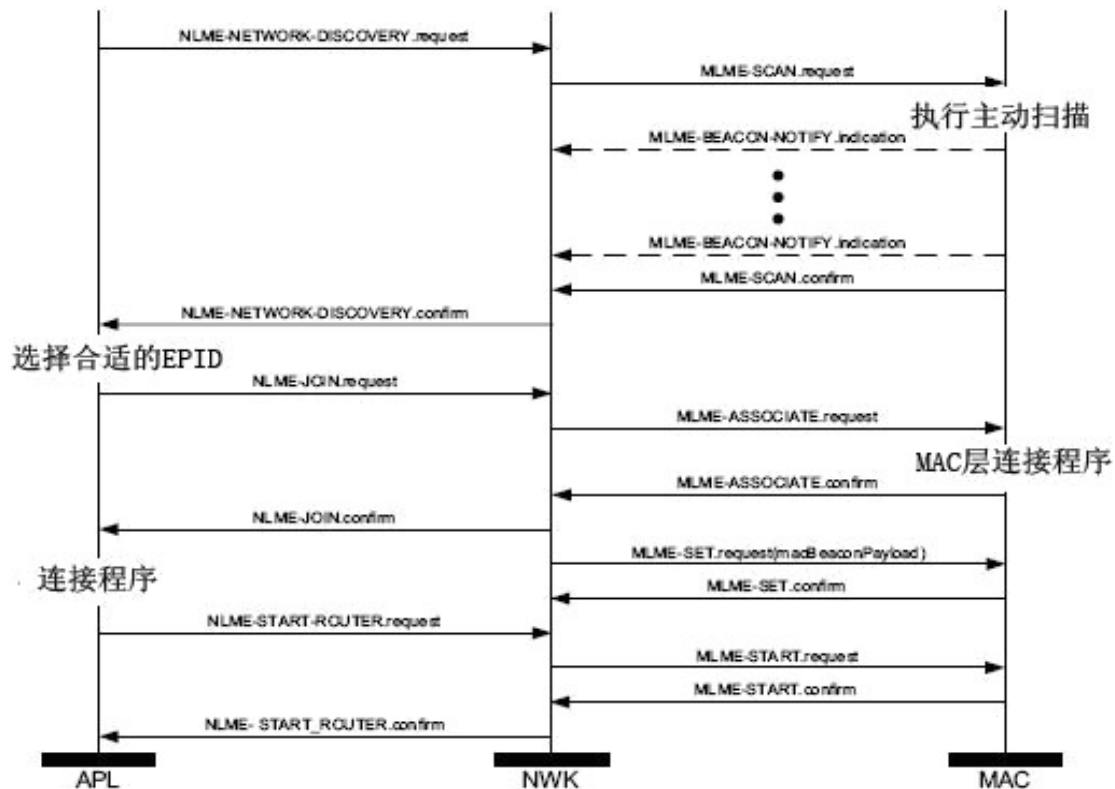


图 3.34 通过连接加入网络的程序

3.6.1.4.1.2 父节点程序

对于一个 ZigBee 协调器或路由器，它使用 MAC 子层连接程序加入一个设备到其网络的程序，是通过来自 MAC 子层的 `MLME-ASSOCIATE.indication` 原语发起。只有是一个 ZigBee 协调器或一个 ZigBee 路由器以及允许设备加入网络的那些设备可以发起这个程序。如果这个程序在任何其它设备上发起，NLME 必须终止该程序。

当发起本程序，一个潜在父节点的 NLME 必须首先确定希望加入的设备是否已经存在于它的网络上。为此，为了确定是否可以找到 64 位，扩展地址的一个匹配，NLME 必须查询其相邻表。如果找到了一个扩展地址匹配，NLME 检查所提供的 `DeviceCapabilities` 是否匹配记录在邻居表中的设备类型。如果设备类型也匹配 NLME，那么它就获得相应的 16 位网络地址，并发出一个连接响应给 MAC 子层。如果没有找到设备类型匹配，NLME 移除它的邻居表中设备的所有记录，并重新启动 `MLME-ASSOCIATION.indication` 的程序。如果没有找到扩展地址匹配，如果可能的话，NLME 为新设备分配一个 16 位网络地址。关于地址分配机制的解释参见 3.6.1.6 小节和 3.6.1.7 小节。

如果潜在父节点没有空间接受更多的子节点，NLME 必须终止该程序并在随后给 MAC 子层的 MLME-ASSOCIATE.response 原语中指明这个事实。本原语的状态参数必须指明 PAN 的容量已经达到最大值。

如果加入请求得到批准，父节点的 NLME 必须使用所提供的设备信息，在其相邻表中为子节点创建一个新的条目，并在随后给 MAC 子层的 MLME-ASSOCIATE.response 原语中指示一个成功的连接。如果 *nwkSecurityLevel* 值是 0x00，新的邻居表条目的关系域设置为值 0x01，表示邻居是一个子节点；否则，设置为 0x05，表示一个未认证的子节点。传输给子节点的响应状态通过 MLME-COMM-STATUS.indication 原语返回给网络层。

如果传输不成功（即 MLME-COMM-STATUS.indication 原语所含的状态参数不等于 SUCCESS），NLME 必须终止该程序。如果传输成功，NLME 必须通过发出 NLME-JOIN.indication 原语，通知上层一个子节点刚刚加入网络。

成功加入一个设备到网络的程序见图 3.35 所示的 MSC。

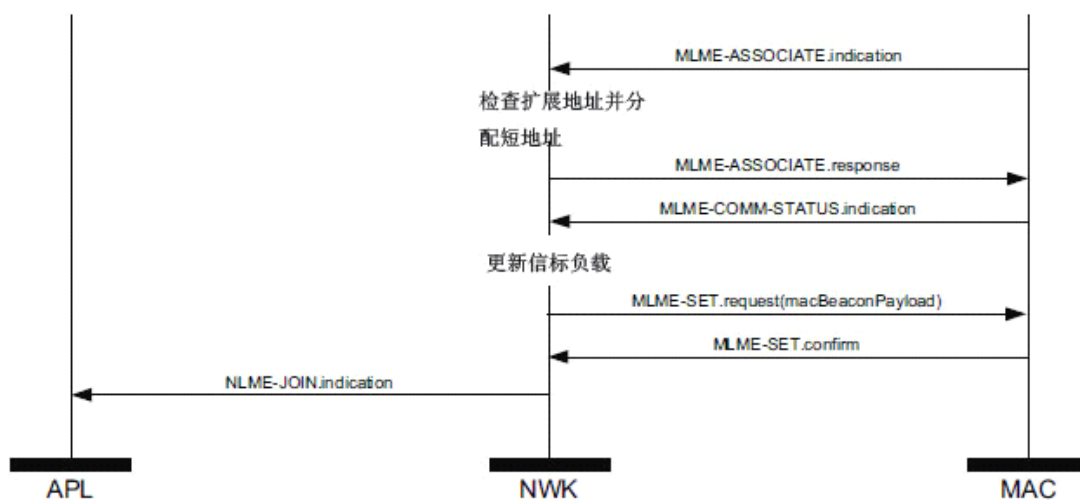


图 3.35 处理一个加入请求的程序

3.6.1.4.2 使用 NWK 重新加入命令加入或重新加入一个网络

失去与网络所有联系的设备，比如一个不能再与其父节点成功通信的 ZED，可以使用 NWK 层重新加入请求和 NWK 重新加入响应命令重新加入网络。重新加入程序和上节描述的连接程序一样，除了 MAC 连接程序被重新加入请求和重新加入响应命令的交流所代替，且因为 NWK 命令使用了 NWK 安全功能，所以不执行认证步骤。使用这些命令而不是 MAC 程序允许一个设备重新加入一个当前不允许新设备加入的网络。

首次加入网络的设备还可以使用以下小节介绍的本程序的一个派生程序。

3.6.1.4.2.1 子节点程序

使用 NWK 重新加入程序重新加入一个网络的程序必须通过发出 NLME-JOIN.request 原语发起，如图 3.36 所示，且 RejoinNetwork 参数设置为 0x02，ExtendedPANId 参数设置为要重新加入的网络的 ExtendedPANId。如果设备想要作为一个路由器加入，CapabilityInformation 参数的设备类型域设置为 1，否则置 0。

设置 ScanChannels 参数，表示为找到这个网络要扫描哪个信道，ScanDuration 参数设置为表示扫描每个信道花费的时间。

在接收本原语时，NWK 层必须发出一个 MLME-SCAN.request 原语，请求 MAC 子层执行一个主动扫描。

扫描期间收到的每个负载长度不为零的信标帧必须引起发出 MLME-BEACON-NOTIFY.indication 原语，从扫描设备的 MAC 子层发给其 NLME。扫描设备的 NLME 必须检查信标负载所含的 ExtendedPANId，看它否是一个正确的值。如果不是，信标被忽略。否则，设备必须从每个收到的信标中复制相关信息（关于信标负载的结构参见图 3.49）到其相邻表中（关于相邻表条目的内容参见表 3.48 和表 3.49）。

一旦 MAC 子层通过给 NLME 发出 MLME-SCAN.confirm 原语，指示扫描完成，NWK 层必须为一个合适的父节点设备查询其相邻表。一个合适的父节点设备必须公布 JoinAsRouter 参数中的请求类型的设备能力，并有一个最多为 3 的链路消耗（参见 3.6.3.1 小节）。如果相邻表不包含合适的父节点设备，NLME 必须响应一个状态参数为 NOT_PERMITTED 的 NLME-JOIN.confirm。如果相邻表有不止一个可以作为一个合适父节点的设备，必须选择从 ZigBee 协调器深度最小的设备。

一旦确定了一个合适的父节点，NLME 必须构造一个 NWK 重新加入请求命令帧。NWK 头的目标地址域值必须等于从邻居表中所选的父节点候选人的 16 位网络地址。NWK 头的源地址域必须设置为 NIB 属性 *nwkNetworkAddress* 的值。源 IEEE 地址域和目标 IEEE 地址域都必须出现在 NWK 头中。如果设备是首次加入这个网络，且它的 NIB 属性 *nwkNetworkAddress* 的值是 0xffff，表示它目前没有加入这个网络，设备必须为自己选择一个 16 位网络地址，并设置 *nwkNetworkAddress* 属性为这个值。地址应根据 3.6.1.7 小节所述的程序随机选择。在这种情况下，无论何时 NIB 属性 *nwkAddrAlloc* 的值都为 0x02，表示随机寻址，命令负载的能力信息域的分配地址子域应设置为 0，表示自己选择地址。

使用 MAC 数据服务成功传输了重新加入请求命令后，网络层将载入一个倒计时的计数器，值为 *aResponseWaitTime* (【B1】)。如果这个计时器在收到一个重新加入响应命令帧之前到期，那么重新加入不成功。如果 *CapabilityInformation* 参数的 *receiver on when idle* 域等于 1，设备应发出一个 MLME-POLL.request 给潜在父节点，来检索重新加入响应命令。如果 *receiver on when idle* 域等于 0，就不需要轮询。²⁰

收到一个重新加入响应命令帧时，在上述程序之后或任何其它时间，设备必须检查命令帧负载中的重新加入设备的 IEEE 地址域和父节点设备的 IEEE 地址域。如果重新加入设备的 IEEE 地址域的值不等于接收设备的 IEEE 地址域，或如果父节点设备的 IEEE 地址域的值不等于一个重新加入请求命令帧送到的最近潜在父节点的 IEEE 地址（或在一个未经同意的重新加入响应下的当前的父节点），那么重新加入响应命令帧必须被丢弃，不再进一步处理。

如果重新加入响应命令帧的重新加入状态域表示一部分附近设备拒绝允许重新加入（即 PAN 容量达到最大值或 PAN 拒绝访问），那么尝试加入的设备必须在相应的相邻表条目中设置 *potential parent* 位为 0，表示一个失败的加入尝试。设置 *potential parent* 位为 0 以保证 NWK 层不会给同一相邻设备发出另一个请求来重新加入。如果尝试加入不成功，NLME 必须尝试从相邻设备中找到另一个合适的父节点。如果找不到这样的设备，NLME 必须发出状态参数设置为 NOT_PERMITTED 的 NLME-JOIN.confirm 原语。

如果尝试加入不成功，且有第二个相邻设备可以作为一个合适的父节点，NWK 层必须为第二个设备发起 NWK 重新加入程序。NWK 层必须重复该程序直到它成功的重新加入 PAN，或用完了它加入 PAN 的选择。如果设备不能成功的重新加入上层指定的 PAN，NLME 必须通过发出状态参数设置为 NOT_PERMITTED 的 NLME-JOIN.confirm 原语终止该程序。在这种情况下，设备不能收到一个有效的逻辑地址，也不能在网络上传输。如果尝试重新加入网络成功，NWK 层收到的 NWK 重新加入响应命令必须包含一个 16 位逻辑地址，在该网络中是唯一的，子节点可以在未来的传输中使用。注意这个地址可以和存储在 NIB 属性 *nwkNetworkAddress* 中的设备的当前 16 位网络地址一样。然后 NWK 层必须在相应的相邻表条目中设置关系域，表示该邻居是其父节点。到这个时候，父节点必须把每个新设备增加到其相邻表中。此外，如果必要的话，NWK 层将在 NIB 中更新 *nwkShortAddress*、*nwkUpdateId* 和 *nwkPANId* 的值。

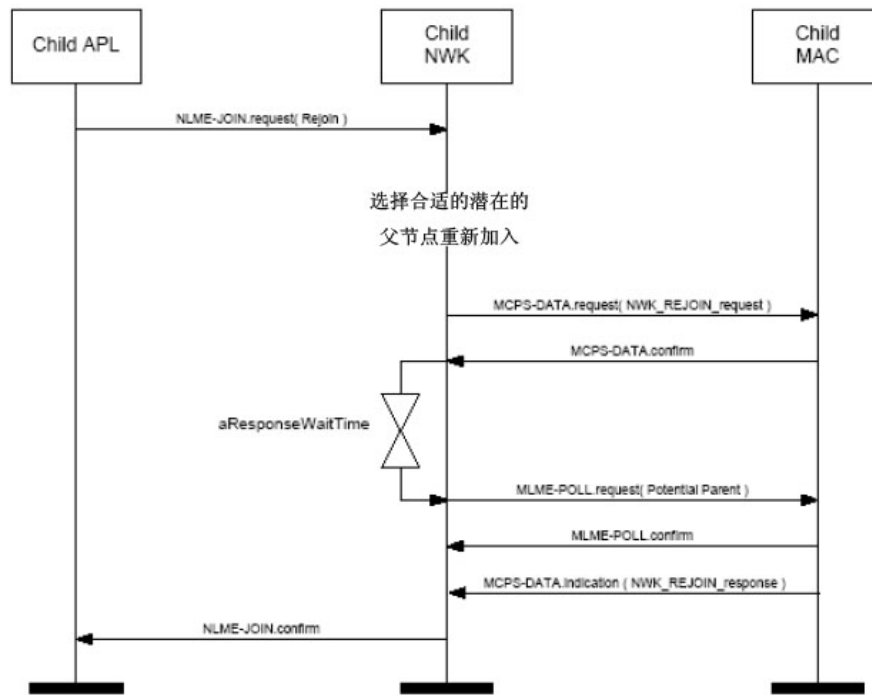


图 3.36 子节点重新加入程序

3.6.1.4.2.2 父节点程序

对于一个 ZigBee 协调器或路由器，它使用 NWK 重新加入程序把一个设备重新纳入其网络的程序，是通过 MAC 数据服务发出一个 NWK 层重新加入命令帧发起的。只有是一个 ZigBee 协调器或一个 ZigBee 路由器的那些设备可以发起该程序。如果该程序在任何其它设备上发起，NLME 必须终止该程序。当发起该程序，一个潜在父节点的 NLME 必须首先确定它是否已经知道了请求设备的信息。为此，NLME 必须查询其相邻表，以确定是否可以找到 64 位，扩展地址的一个匹配。如果找到了一个扩展地址匹配，NLME 检查所提供的 DeviceCapabilities 是否匹配邻居表中记录的设备类型。如果设备类型匹配，NLME 认为加入尝试成功，并使用在其相邻表中找到的 16 位网络地址作为加入设备的网络地址。如果没有找到一个设备类型匹配，NLME 在它的邻居表中移除设备的所有记录，并重新启动 NWK 重新加入命令。

如果潜在父节点没有空间接受加入设备，NLME 应终止该程序，并在随后的重新加入响应命令中指明这一事实。这个命令的状态参数必须表示 PAN 容量达到最大值。

如果重新加入请求被批准，父节点的 NLME 使用所提供的设备信息，在它的邻居表中为这个子节点创建一个新的条目，或如果这样一个条目已经存在的话，修改已有的条目，并通过回复请求设备一个 NWK 重新加入响应命令，表示一个成功的重新加入。如果 NIB 属性 *nwkAddrAlloc* 的值为 0x00，表示树寻址，NLME 应为加入设备新分配一个 16 位网络地址。关于地址分配机制的解释见 3.6.1.6 和 3.6.1.7 小节。

如果 NIB 属性 *nwkAddrAlloc* 的值不是 0x00，重新加入请求命令帧负载的能力信息域的分配地址子域的值可以是 0，表示自己指派或预先存在的网络地址。在这种情况下，对于所有的 NWK 命令帧，应该检查 NWK 头的源地址域的 16 位网络地址，结合 NWK 头的源 IEEE 地址域的 64 位 IEEE 地址，防止 3.6.1.9 小节所述的地址冲突。如果发现一个地址冲突，就为加入设备选择一个新的、不冲突的地址，并放在出去的重新加入响应命令帧的命令帧负载的网络地址域中。否则，输入重新加入请求命令帧的源地址域的内容应放在出去的重新加入响应命令帧的命令帧负载的网络地址域中。

然后 NLME 必须通过发出 NLME-JOIN.indication 原语，通知上层一个子节点刚刚重新加入了网络。把一个设备成功地重新纳入一个网络的程序在图 3.37 所示的 MSC 中说明。

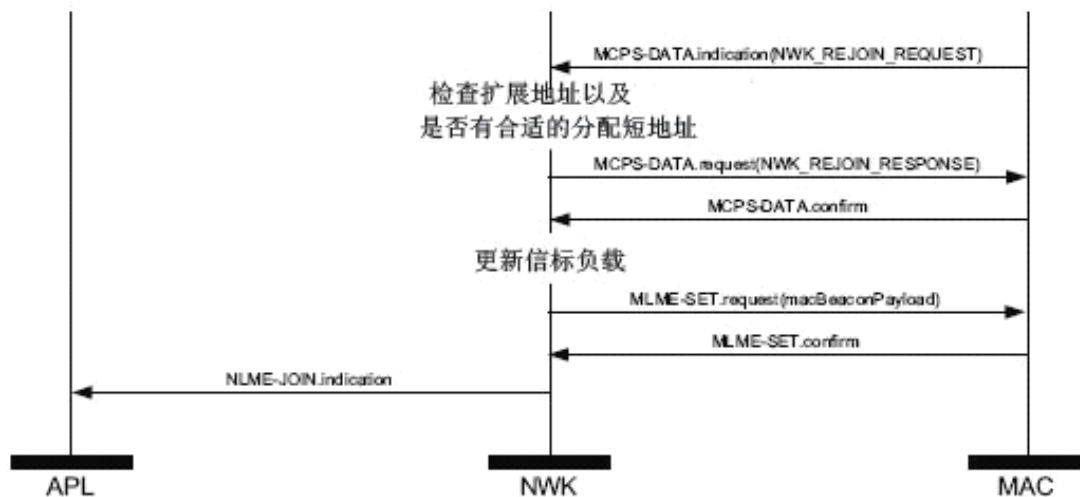


图 3.37 父节点重新加入程序

3.6.1.4.3 直接加入一个网络

本小节指明了一个设备如何被一个之前指定的父节点设备（ZigBee 协调器或路由器）直接增加到一个网络。在这种情况下，父节点设备预先配置了子节点设备的 64 位地址。以下文字描述了预先知道这个地址是如何用来建立父子关系的。

对于一个 ZigBee 协调器或路由器，把一个设备直接纳入其网络的程序是通过发出 `NLME-DIRECT-JOIN.request` 原语发起的，且 `DeviceAddress` 参数设置为要被纳入网络的设备的地址。只有是一个 ZigBee 协调器或一个 ZigBee 路由器的那些设备可以发起该程序。如果这个程序在任何其它设备上发起，NLME 必须终止该程序，并通过发出状态参数设置为 `INVALID_REQUEST` 的 `NLME-DIRECT-JOIN.confirm` 原语，通知上层这一非法的请求。

当发起该程序，父节点的 NLME 必须首先确定指定设备是否已经存在于它的网络中。为此，NLME 必须查询其相邻表，以确定是否可以找到 64 位、扩展地址的一个匹配。如果找到一个匹配，NLME 必须终止该程序，并通过发出状态参数设置为 `ALREADY_PRESENT` 的 `NLME-DIRECT-JOIN.confirm` 原语，通知上层该设备已经出现在设备列表中。

如果没有找到匹配，如果可能的话，NLME 必须为新设备分配一个 16 位网络地址以及一个新的邻居表条目。地址分配机制的解释见 3.6.1.6 和 3.6.1.7 小节。如果父节点设备的邻居表没有空间了 NLME 应终止该程序，并通过发出状态参数设置为 `NEIGHBOR_TABLE_FULL` 的 `NLME-DIRECT-JOIN.confirm` 原语，通知上层没有可用的容量。如果有可用的容量，NLME 必须通过发出状态参数设置为 `SUCCESS` 的 `NLME-DIRECT-JOIN.confirm` 原语，通知上层设备已经加入了网络。

一旦父节点把子节点增加到它的网络，还需要为子节点建立和父节点的联系，以完成父子关系的建立。子节点必须通过发出 3.6.1.4.3.1 小节所描述的孤程序满足这一要求。

一个支持直接加入的父节点必须遵循图 3.38 所示的程序，以成功地直接纳入一个设备到网络。该程序不需要任何无线传输。

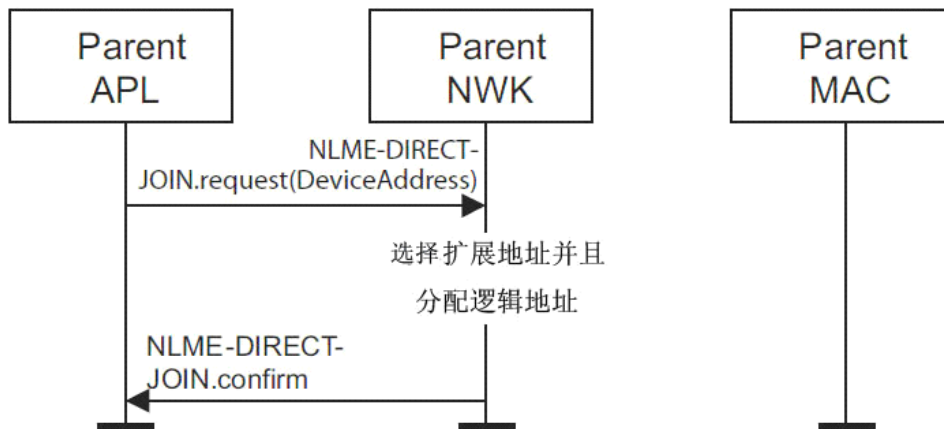


图 3.38 直接加入一个设备到网络

3.6.1.4.3.1 通过孤程序加入或重新加入一个网络

本小节指明了如何由一个已经直接加入一个网络的设备（通过孤程序加入），或之前加入了一个网络，但与其父节点失去联系的设备（通过孤程序重新加入）发起孤程序。

一个已经被直接增加到一个网络的设备必须发起孤程序以完成建立和其父节点的关系。设备上的应用程序将确定是否发起该程序，并且如果发起的话，将在上电时通知网络层。

对于一个之前加入了一个网络的设备，如果其 NLME 从它的 MAC 子层重复收到通信失败的通知，它可以选择发起孤程序。

3.6.1.4.3.2 子节点程序

可选的通过孤程序加入由一个设备使用 NLME-JOIN.request 原语发起，RejoinNetwork 参数设置为 0x01。

当发起本程序，NLME 必须首先请求 MAC 子层在 ScanChannels 参数给定的一组信道上执行一个孤扫描。通过给 MAC 子层发出 MLME-SCAN.request 原语发起一个孤扫描，结果通过 MLME-SCAN.confirm 原语返回 NLME。

如果子节点找到了它的父节点，孤扫描成功，NLME 必须通过发出状态参数设置为 SUCCESS 的 NLME-JOIN.confirm 原语，通知上层其请求加入或重新加入网络成功。

注意如果子节点设备是第一次加入，或如果子节点设备之前已经加入网络但没有保留 3.6.8 小节所述的树深度信息，如果不采取措施恢复这一信息它可能无法在网络中正确运行，该措施在本规范的范围之外。

如果孤扫描不成功（父节点没有找到），NLME 必须终止该程序，并通知上层没有找到网络。这通过发出状态参数设置为 NO_NETWORKS 的 NLME-JOIN.confirm 原语实现。

一个子节点通过孤程序成功加入或重新加入一个网络的程序在图 3.39 的 MSC 中说明。

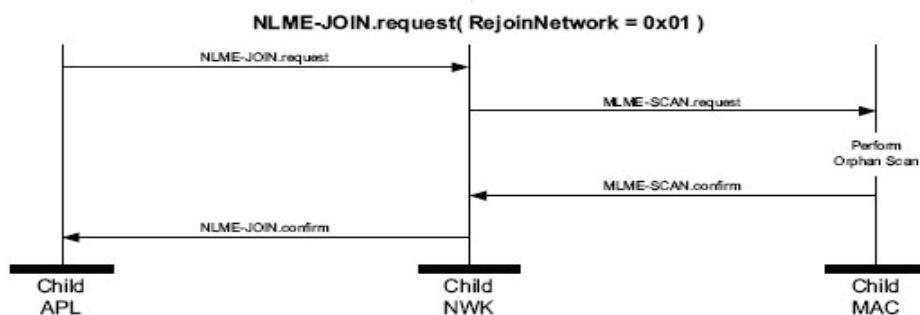


图 3.39 通过孤加入或重新加入网络的子程序

3.6.1.4.3.3 父节点程序

当一个设备从 MAC 子层收到 MLME-ORPHAN.indication 原语，它被告知出现了一个孤设备。只有是 ZigBee 协调器或 ZigBee 路由器（即设备具有作为父节点的能力）的设备可以发起该程序。如果该程序在任何其它设备上发出，NLME 必须终止该程序。

当发起该程序，NLME 必须首先确定孤设备是否是它的子节点。这通过比较孤设备的扩展地址和其相邻表中记录的子节点的地址来完成。如果找到一个匹配(孤设备是它的子节点)，NLME 必须取得相应的 16 位网络地址，并把包含在其随后给 MAC 子层的孤响应中。发给 MAC 子层的孤响应通过发出 MLME-ORPHAN.response 原语发起，且传输的状态通过 MLME-COMM-STATUS.indication 原语返回 NLME。

如果没有找到地址匹配（孤设备不是它的子节点），必须终止程序，且不通知上层。

一个父节点加入或重新加入其孤子节点到其网络的程序在图 3.40 所示的 MSC 中说明。

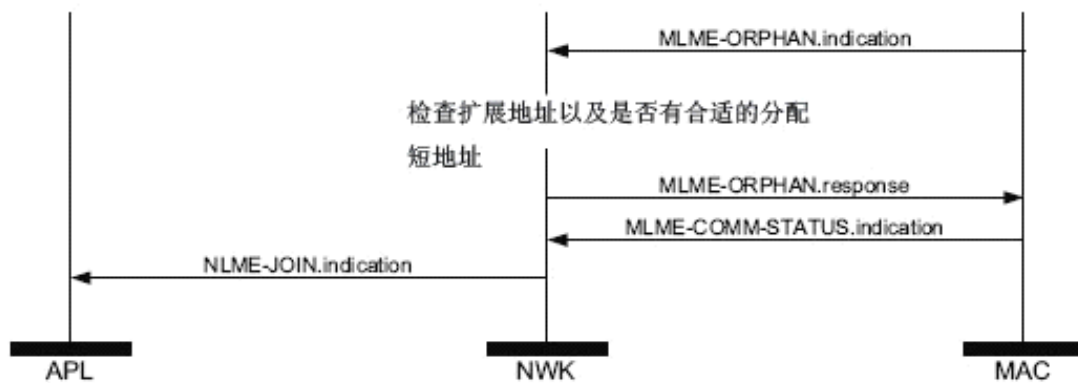


图 3.40 通过孤加入或重新加入一个设备到网络的父程序

3.6.1.5 邻居表

一个设备的邻居表必须包含传输范围内每个设备的信息，取决于一些执行相关的限制。

邻居表在两种情况下是有用的。首先，它在网络发现或重新加入期间使用，以存储 RF 接收范围内关于路由器的信息，这些路由器是候选父节点设备。第二，在设备加入一个网络之后，它用于存储该网络中有关相邻设备的关系和链路状态信息。一个设备每次从相应的邻居收到任何帧，就必须更新一个表的条目。

向外消耗域含有邻居所测量的链路消耗。值从邻居收到的命令帧的最近链路状态获得。值 0 表示这个设备没有收到链路状态命令。

年龄域说明自从收到上个状态命令帧，经过 *nwkLinkStatusPeriod* 间隔的时间，最大值高达 *nwkRouterAgeLimit*。

常用网络操作中使用的强制和可选的数据列在表 3.48 中。

表 3.48 邻居表目录格式

域名称	域类型	有效范围	描述
Extended address	Integer	扩展64位IEEE地址	每个设备唯一的64位IEEE地址；如果邻居是一个父设备或者设备的子节点该域应该被指定
Network address	Network address	0x0000 – 0xffff	邻居设备的16位网络地址 该域应该在每个邻居表目录中指定
Device type	Integer	0x00 – 0x02	邻居设备的类型： 0x00 = ZigBee协调器 0x01 = ZigBee路由器 0x02 = ZigBee终端设备 该域应该在每个邻居表目录中指定
RxOnWhenIdle	Boolean	TRUE或FALSE	代表邻居表在CAP的空闲部分时是可用的。 TRUE = 接收器是打开的 FALSE =接收器是关闭的 该域应该指定记录ZigBee路由器或者ZigBee协调器的父节点或子节点的目录

表 3.48 邻居表目录格式（续表）

域名称	域类型	有效范围	描述
Relationship	Integer	0x00 – 0x03	当前设备和邻居设备间的关系: 0x00=邻居设备是父节点 0x01=邻居设备是子节点 0x02=邻居设备是同属节点 0x03=以上都不是 0x04=前一个子节点 该域应该在每个邻居表目录中指定
Transmit Failure	Integer	0x00 – 0xff	一个值代表前一个传输到设备是否成功; 值越大则表明越失败。在每个邻居表中都存在该子域。
LQI	Integer	0x00 – 0xff	来自设备的RF传输估计连接质量见3.6.3.1小节的如何计算的讨论。 该域应该在每个邻居表目录中指定。
Outgoing Cost	Integer	0x00 - 0xff	由邻居测量到的向外链路的消耗。值为0表示没有向外消耗可用。 如果 <code>nwk.SymLink</code> = TRUE该域是强制的。

表 3.48 邻居表目录格式（续表）

域名称	域类型	有效范围	描述
Age	Integer	0x00 - 0xff	自从收到一个链路状态命令，经过的时间间隔 <i>nwkLinkStatusPeriod</i> 。 如果 <i>nwkSymLink</i> = TRUE 该域是强制的。
Incoming beacon timestamp	Integer	0x000000-0xffffffff	从邻居接收的最后信标帧，时间，符号。该值等于信标帧被接收时的时间戳，就像 IEEE 802.15.4-2003[B1]描述的那样该域是可选的。
Beacon transmission time offset	Integer	0x000000-0xffffffff	在邻居的信标和它的父节点的信标之间时间差以符号形式传输。该时间差去计算邻居设备的父节点时可能扣除相应的传入时间戳。该域是可选的。

如上所述，网络发现和重新加入期间可能用到的信息见表 3.49。所有显示为可选的域在 NLME 选定一个网络加入之后就不再保留。同样的，和不是所选网络成员的设备相对应的相邻表条目必须被丢弃。

表 3.49 更多的邻居表域

域名称	域类型	有效范围	描述
Extended PAN ID	Integer	0x0000000000000001 - 0xfffffffffffffffe	设备属于网络的64位唯一标识符
Logical channel	Integer	从PHY支持的可变逻辑信道中选择。	逻辑通道上的网络操作。
Depth	Integer	0x00 - 0xff	邻居设备的树深度

表 3.49 更多的邻居表域（续表）

域名称	域类型	有效范围	描述
Beacon order	Integer	0x00 – 0x0f	设备的 IEEE 802.15.4 信标命令
Permit joining	Boolean	TRUE 或 FALSE	表明该设备是否接受加入请求。 TRUE = 设备允许加入请求 FALSE = 设备不允许加入请求
Potential parent	Integer	0x00 – 0x01	表明是否设备作为一个潜在的父设备已排除。 0x00 代表设备不是潜在的父节点 0x01 代表设备是潜在的父节点

3.6.1.6 分布式地址分配机制

NIB 属性 *nwkAddrAlloc* 的默认值为 0x00，使用一个分布式地址方案分配网络地址，它设计为给每个潜在父节点提供一个有限的网络地址子块。这些地址在一个特定的网络内是唯一的，由一个父节点给它的子节点。ZigBee 协调器确定在其网络内允许的任何设备子节点的最大个数。这些子节点中，最大值为 *nwkMaxRouters* 的设备可以是路由器功能设备。其余设备必须保留为终端设备。每个设备有一个相关的深度，表示只使用父子链路传输的一个帧到达 ZigBee 协调器，必须经过的最少跳数。ZigBee 协调器本身的深度是 0，而其子节点深度是 1。多跳网络最大深度大于 1。ZigBee 协调器还确定了网络的最大深度。

根据一个父节点可以有的子节点最大个数的值 *nwkMaxChildren (Cm)*，网络的最大深度 *nwkMaxDepth (Lm)*，和一个父节点可以有的路由器子节点的最大个数 *nwkMaxRouters (Rm)*，我们可以计算出函数 *Cskip(d)* 的值，对于一个给定的网络深度 *d*，基本上是从该深度到其路由器功能的子节点设备，每个父节点分配的地址子块的大小，如下：

$$Cskip(d) = \begin{cases} 1 + Cm \cdot (Lm - d - 1), & \text{if } Rm = 1 \\ \frac{1 + Cm - Rm - Cm \cdot Rm^{Lm-d-1}}{1 - Rm}, & \text{否则} \end{cases}$$

为了实施这里的讨论，如果一个设备 $Cskip(d)$ 的值为 0，那么它不能再接受子节点，且必须被看做一个 ZigBee 终端设备。设备的 NLME 必须设置 MAC 子层信标负载的 End device Capacity 和 Router Capacity 子域为 0。

一个 $Cskip(d)$ 值大于 0 的父节点设备必须接收子节点设备，并根据子节点设备是否具有路由器功能给它们分配不同的地址。

必须使用 $Cskip(d)$ 的值作为一个偏移地址，把网络地址分配给具有路由器功能的子节点设备。一个父节点给它的第一个路由功能子节点设备分配一个地址 1，大于它自己的地址。随后给路由器功能子节点设备分配的地址根据 $Cskip(d)$ 各自都不相同。最多只能分配 $nwkMaxRouters$ 个这样的地址。

给终端设备分配的网络地址，依次在第 n 个地址的方式， A_n ，根据下列公式：

$$A_n = A_{parent} + Cskip(d) * Rm + n$$

其中 $d(1 < n < (Cm - Rm))$ ， A_{parent} 表示父节点的地址。

例如在一个网络中， $nwkMaxChildren=8$ ， $nwkMaxRouters=4$ 和 $nwkMaxDepth=3$ ， $Cskip(d)$ 计算出的值列在表 3.50 中。图 3.41 解释了网络的例子。

表 3.50 每个给出网络深度的寻址偏置值的例子

网络深度, d	偏置值, $Cskip(d)$
0	31
1	7
2	1
3	0

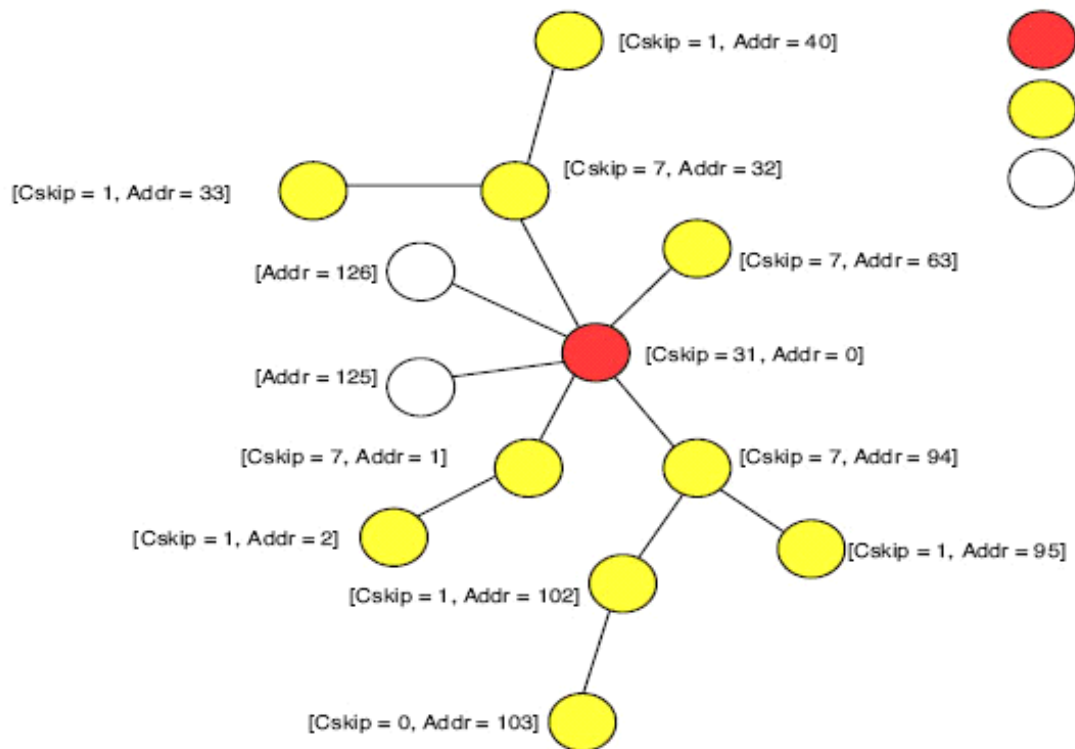


图 3.41 网络中的地址分配实例

因为一个地址子块不能在设备之间共享，有可能一个父节点用完了它的地址列表，而第二个父节点的地址还未使用。一个没有可用地址的父节点通过设置 MAC 子层信标负载的 End Device Capacity 和 Router Capacity 子域为 0，禁止一个新设备加入网络。

在这种情况下，新设备必须找到另一个父节点。如果在新设备的传输范围内没有其它设备可用，设备不能加入网络，除非它物理位置移动或有其它一些变化。

3.6.1.7 随机地址分配机制

当 NIB 属性 *nwkAddrAlloc* 值为 0x02，地址就随机选择。在这种情况下 *nwkMaxRouter* 的值无关。随机地址分配应符合参考[B12]所述的 NIST 测试机制。当一个设备使用 MAC 连接加入网络，它的父节点选择一个尚未出现在父节点 NIB 的任何条目中的随机地址。随机寻址机制下，一旦设备已经被分配一个地址，就没有放弃这个地址，应予以保留，除非它收到一个指令表明它的地址与网络中另一个设备冲突。此外，随机寻址机制下，设备可以自己分配随机地址并保留它们，就像使用重新加入命令帧（见 3.6.1.4.2 小节）加入网络的情况一样。没有父节点的 ZigBee 协调器的地址总是 0x0000。

3.6.1.8 安装和处理

应当清楚地看到, *nwkMaxDepth* 大致确定了在网络内从树的根到最远的终端设备的“距离”。原则上, *nwkMaxDepth* 也决定了整个网络的直径。特别是, 对于一个理想的网络布局, 其中 ZigBee 协调器位于网络的中心如图 3.41 所示, 网络直径必须是 $2 * nwkMaxDepth$ 。在实际上, 应用程序驱动的安置决定和部署顺序可能导致一个更小的直径。在这种情况下, *nwkMaxDepth* 提供了网络直径一个下界, 而 $2 * nwkMaxDepth$ 提供了上界。

最后, 当 *nwkUseTreeAddrAlloc* 值为 0x00 时, 由于树没有动态平衡的事实, 某些安装脚本是可能存在的, 如按照线性方式分布的网络设备, 可能在远远没有到达真正的网络容量时, 其网络地址已经全部分配。

随机地址分配机制下, *nwkMaxDepth* 和跨过网络的跳数有关。在使用随机地址分配的网络中这不是一个受控制的值。

3.6.1.9 地址冲突

当同一个网络中的两个设备的 *nwkNetworkAddress* 具有相同值时, 发生地址冲突。为了防止所有这类冲突, 例如使用树地址分配, 禁止重新使用已分配地址, 并非总是可行的。本节描述了如何检测地址冲突并纠正。如果 NIB 属性 *nwkUniqueAddr* 是 FALSE, 地址冲突检测应使能。

注意路由信息使用的网络地址是在路由发现过程中进行验证的。现在 *device_annce* 也用于验证地址。验证仅适用于发现或 *device_annce* 期间所列的设备、链路和信息。验证可以在其它时间完成, 比如给一个邻居直接发送一个单播之前, 发送一个状态代码为 0x0e 的网络状态命令, 表示地址验证。

如果一个设备收到一个广播数据帧, 接收后发现地址冲突, 如 3.6.1.9.2 小节所述, 在采取下面 3.6.1.9.3 小节²¹所述的解决方案之前, 它不能像往常一样转发这个帧, 而应丢弃。

21. CCB #869

3.6.1.9.1 获取地址信息

NWK 层从输入信息获取地址信息，包括 NWK 命令信息和数据信息。数据信息的地址信息被添加到 NIB 中的网络地址映射表，然后传递到 NWK 层。

通过添加目标 IEEE 地址和源 IEEE 地址域其中之一，或两个都添加到信息的 NWK 帧，就增加了检测地址冲突的功能。当 *nwkUniqueAddr* 是 FALSE，所有 NWK 命令信息都必须包含源 IEEE 地址，且如果源设备知道目标 IEEE 地址的话，还要包含目标 IEEE 地址。

当 *nwkUniqueAddr* 是 FALSE，路由请求命令必须在 Sender IEEE 地址域中包含发送设备的 IEEE 地址。这能确保设备知道它们的邻居的 IEEE 地址。

3.6.1.9.2 检测地址冲突

加入一个网络之后，或由于冲突修改地址之后，设备在发送信息之前必须发送一个 *device_annnc*，或发起一个路由发现。

收到 NWK 头包含 64 位 IEEE 地址的一个帧时，必须检查 NIB 属性 *nwkAddressMap* 和邻居表的内容是否一致。

如果输入帧的 NWK 头的目标地址域等于 NIB 属性 *nwkNetworkAddress* 的值，那么如果出现的话，NWK 层检查目标 IEEE 地址域和 *aExtendedAddress* 的值。如果这两个 IEEE 地址不一样，那么就检测到 *nwkNetworkAddress* 上的一个本地地址冲突。

如果邻居表或地址映射条目位于 64 位地址是空 IEEE 地址 (0x00....00) 的表，可以更新表中的 64 位地址。但是，如果 64 位地址不是空 IEEE 地址，且不对应收到的 16 位地址，设备就在网络中其它地方检测到一个冲突。

当收到创建一个新的 BTR 的一个广播帧，如果 NWK 头的源地址域等于 NIB 属性 *nwkNetworkAddress*，那么就检测到 *nwkNetworkAddress*²² 上的一个本地地址冲突。

地址冲突的解决方案如 3.6.1.9.3 小节所述。

3.6.1.9.3 解决地址冲突

如果一个 ZigBee 协调器或路由器确定一个不属于自己的地址有多个用户,它应广播一个网络状态命令通知网络,状态代码是 0x0d 表示地址冲突,且目标地址域是违规的地址。网络状态命令应广播给 0xFFFD,即所有 *macRxOnWhenIdle*= TRUE 的设备。设备通过 *nwkMaxBroadcastJitter* 界定的一个随机抖动数,延迟发起这个广播。如果这个延迟期间收到了一个具有同样负载的网络状态,设备就取消自己的广播。

如果一个设备确定自己的网络地址有一个或多个其它用户,或如果设备收到一个网络状态命令,状态代码是 0x0d,表示网络冲突,且目标地址域是它自己的地址,那么设备就获取一个新的地址。

如果在一个 ZigBee 终端设备上检测到冲突,或 *nwkAddrAlloc* 不等于随机地址分配,那么设备执行一个重新加入以获得一个新的地址。否则,需要一个新地址的设备随机选择一个新的地址,避免所有 NIB 条目中出现的地址。

如果一个父节点设备检测到或被告知一个子节点有地址冲突,父节点就为子节点选择一个新的地址,并发送一个未经同意的重新加入响应命令帧来通知子节点这个新的地址。要通知上层改变了一个地址,应发送一个状态是“Network Address Update”的 NLME-NWK-STATUS.indication,且新的网络地址作为 ShortAddr 参数²³的值。

3.6.1.10 离开一个网络

本小节指明了一个设备从网络中移除自身,和一个设备的父节点请求移除它的方法。在这两种情况下,被移除设备的子节点,如果有的话,也可能被清除。

3.6.1.10.1 设备发起从网络中移除自身的方法

本小节描述了一个设备如何发起从网络中移除自身,以响应上层的一个 NLME-LEAVE.request 原语,如图 3.42 所示。

23. CCB #814

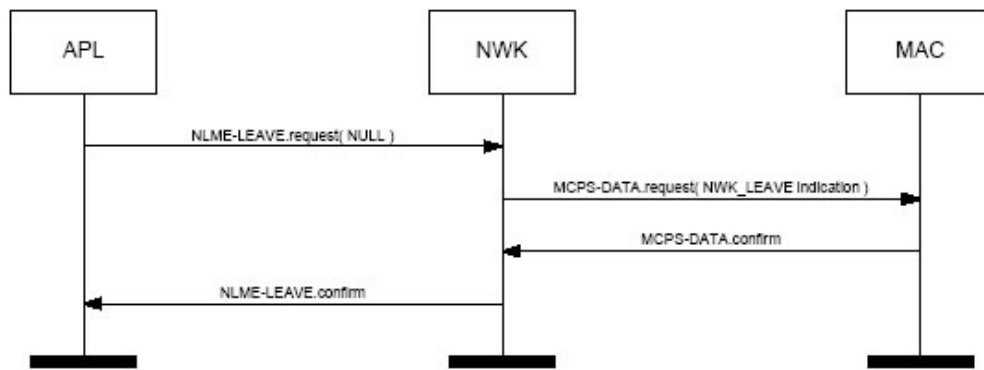


图 3.42 发起离开程序

当一个 ZigBee 路由器或 ZigBee 协调器的 NWK 层收到 DeviceAddress 参数等于 NULL (表示设备要移除自身) 的 NLME-LEAVE.request 原语, 设备必须使用 MCPS-DATA.request 原语发送一个离开命令帧, DstAddr 参数设置为 0xffff, 表示一个 MAC 广播。离开命令帧的命令选项域的请求子域必须设置为 0。离开命令帧的命令选项域的移除子域的值必须反映 NLME-LEAVE.request 原语的 RemoveChildren 参数的值, 且离开命令的 Rejoin 子域的值必须反映 NLME-LEAVE.request 原语的 Rejoin 参数的值。在传输离开命令帧之后, 它必须给上层发出一个 NLME-LEAVE.confirm 原语, 其中 DeviceAddress 参数等于 NULL。如果离开命令帧成功传输, 状态参数必须是 SUCCESS。否则, NLME-LEAVE.confirm 原语的状态参数值必须和 MCPS-DATA.confirm 原语返回的状态参数值相同。

如果接收 NLME-LEAVE.request 原语的设备是一个 ZigBee 终端设备, 那么设备必须使用 MCPS-DATA.request 原语发送一个离开命令帧, 且 DstAddr 参数设置为其父节点设备的 16 位网络地址, 表示一个 MAC 单播。离开命令帧的命令选项域的请求和移除子域都必须设置为 0。在传输离开命令帧之后, 它应设置 NIB 属性 *nwkExtendedPANId* 为 0x0000000000000000, 并给上层发出一个 NWK-LEAVE.confirm 原语, 且 DeviceAddress 参数等于 NULL。如果离开命令帧成功传输, 状态参数必须是 SUCCESS。否则, NLME-LEAVE.confirm 原语的状态参数值必须和 MCPS-DATA.confirm 原语返回的状态参数值相同。

3.6.1.10.2 设备从网络中移除其子节点的方法

本小节描述了一个设备如何发起从网络中移除它的其中一个子节点设备，以响应从上层收到一个 NLME-LEAVE.request 原语，如图 3.43 所示。

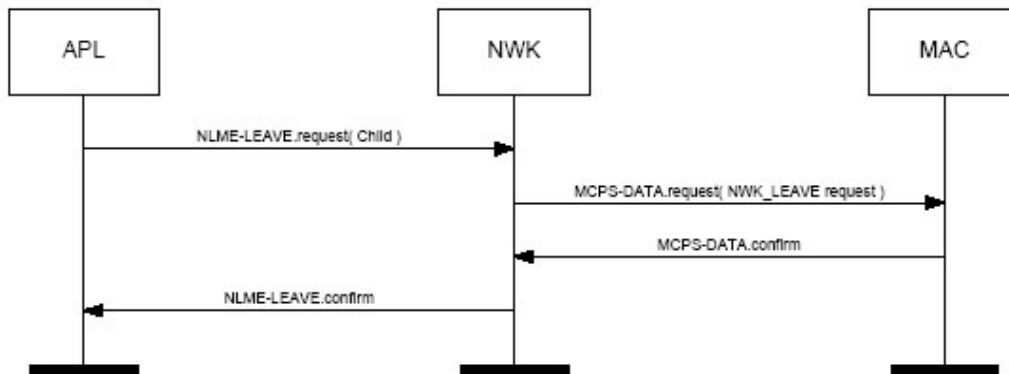


图 3.43 设备移除它的子节点的程序

一个 ZigBee 协调器或 ZigBee 路由器的 NWK 层收到本原语，且 DeviceAddress 参数等于一个子节点设备的 64 位 IEEE 地址时，如果对应于子节点设备的邻居表条目的关系域值不是 0x05，表示该子节点未经验证，设备必须使用 MCPS-DATA.request 原语，发送一个网络离开命令帧，其中 DstAddr 参数设置为该子节点设备的 16 位网络地址。离开命令帧的命令选项的请求子域值必须为 1，表示一个离开网络的请求。离开命令帧的命令选项的移除子节点子域的值必须反映 NLME-LEAVE.request 原语的 RemoveChildren 参数的值，且离开命令的 Rejoin 子域必须反映 NLME-LEAVE.request 原语的 Rejoin 参数的值。

如果对应于被移除设备的邻居表条目的关系域值为 0x05，表示它是一个未经验证的子节点，设备就不能发送一个网络离开命令帧。

然后 NWK 层必须发出 NLME-LEAVE.confirm 原语，且 DeviceAddress 参数设置为要被移除的子节点设备的 64 位 IEEE 地址。如果没有发送离开命令帧，即子节点未经验证的情况下，NLME-LEAVE.confirm 原语的状态参数值为 SUCCESS。否则，NLME-LEAVE.confirm 的状态参数值必须和 MCPS-DATA.confirm 原语返回的状态参数值一样。

子节点设备被移除之后，父节点的 NWK 层应修改其邻居表，和关于子节点设备的任何其它内部数据结构，以表示该设备不再在网络上。在一个子节点设备已经被移除之后，上层寻找和给它传输帧都是错误的。

如果一个未经验证的子节点在被验证之前设备从网络移除，那么之前该设备使用的地址被要求保留，可以分配给随后加入网络的其它设备。

ZigBee 终端设备没有子节点设备可以移除，且不能接收 DeviceAddress 参数非空的 NLME-LEAVE.request 原语。

3.6.1.10.3 接收离开命令帧

NWK 层通过 MCPS-DATA.indication 原语接收离开命令帧时，如图 3.44 所示，设备必须检查命令帧的命令选项的请求子域的值。如果请求子域值为 0，那么 NWK 层必须给上层发出 NLME-LEAVE.indication 原语，且设备地址参数等于离开命令帧的离开设备 IEEE 地址子域所含的值。设备还要修改其邻居表，和关于离开设备的任何其它内部数据结构，以表示它不再在网络上。在一个设备已经离开网络之后，上层寻找和给它传输帧都是错误的。

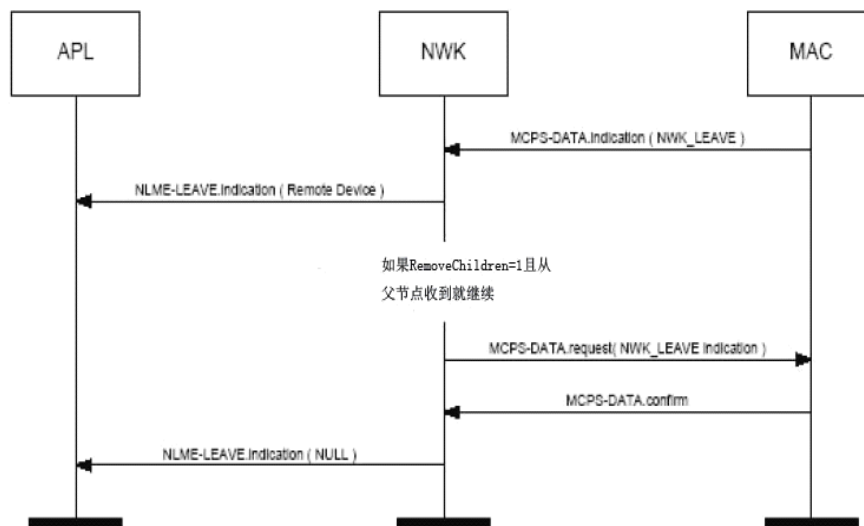


图 3.44 接收离开命令

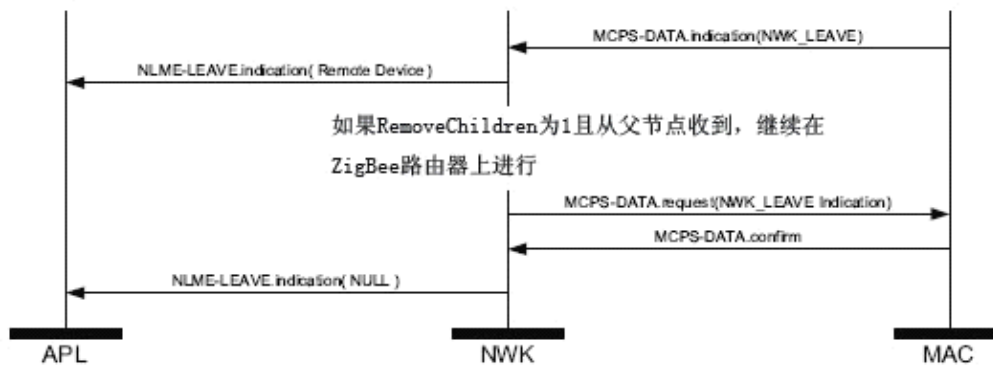


图 3.45 ZED 接收离开命令

如果一个 ZigBee 路由器的 NWK 层收到如上所述的一个离开命令帧时，交付该命令帧的 MCPS-DATA.indication 的 SrcAddr 参数是接收者的父节点的 16 位网络地址，且发现命令选项域的请求子域的值 1，或发现命令选项域的移除子域子域的值 1，那么接收者必须使用 MCPS-DATA.request 原语发送一个离开命令帧，DstAddr 参数设置为 0xffff，表示一个 MAC 广播。离开命令帧的命令选项域的请求子域必须设置为 0。

要发送的离开命令帧的命令选项域的移除子域子域的值必须反映输入离开命令帧相同域的值。在传输一个离开命令帧之后，它应设置 NIB 属性 *nwkExtendedPANId* 的值为 0x0000000000000000，并给上层发出一个 NLME-LEAVE.indication 原语，且 DeviceAddress 参数等于 NULL。

如果请求子域值为 1，离开命令帧的源是接收者的父节点以外的一个设备，帧必须被立即且默默地丢弃。

如果一个 ZigBee 终端设备收到了如上所述的一个离开命令帧，且交付该命令帧的 MCPS-DATA.indication 的 SrcAddr 参数是接收者的父节点的 16 位网络地址，它应设置 NIB 属性 *nwkExtendedPANId* 的值为 0x0000000000000000，并给上层发出一个 NLME-LEAVE.indication 原语，且 DeviceAddress 参数等于 NULL。

NWK 层可以使用重试技术，如 3.6.5 小节所述，来增强离开程序的可靠性，但是这些机制超出了本规范的范围。

3.6.1.11 改变 ZigBee 协调器配置

如果一个 ZigBee 协调器设备的上层希望改变网络的配置,它必须请求 MAC 子层在其 PIB 中发起改变。ZigBee 协调器配置由以下项目组成:

- 设备是否希望成为 ZigBee 父节点
- MAC 超帧的信标顺序
- MAC 超帧的超帧顺序
- 是否使用电池寿命延长模式

对 ZigBee 协调器的改变通过给 NLME 发出 NLME-NETWORK-FORMATION.request 原语发起。尝试的状态通过 NLME-NETWORK-FORMATION.confirm 原语返回。

关于这种改变对 MAC 子层的影响的更多详细信息,参见 IEEE 802.15.4-2003[B1]。

3.6.1.12 复位一个设备

一个设备的 NWK 层必须在初始上电之后,在尝试加入一个新网络之前,设备尝试离开不打算重新加入的网络之后立即复位。这个程序不能在任何其它时间发起。通过给 NLME 发出 NLME-RESET.request 原语发起一个复位,尝试的状态通过 NLME-RESET.confirm 原语返回。复位程序必须清除设备的路由表条目。

一些设备可以把 NWK 层数量存储在非易失存储器中,在一个复位之后恢复它们。NLME-RESET.request 的 WarmStart 参数也可用于这个目的。当 *nwkAddrAlloc* 等于 0x00,设备总是在加入或重新加入时从它的父节点得到一个网络地址。新的网络地址可以和其旧的网络地址不同。在这种情况下,任何与已经复位的设备通信的设备必须使用更高层的协议和程序重新发现该设备。当 *nwkAddrAlloc* 等于 0x02,设备重新加入一个网络时可以使用同样的地址,因此复位时不能丢弃它的地址,除非它不打算重新加入同一个网络。

3.6.1.13 管理 PANId 冲突

由于 16 位 PANID 不是唯一的,就有可能出现一个 PANId 冲突。下一节解释了——通过使用网络报告和网络更新命令帧——如何更新一个网络的 PANId。

3.6.1.13.1 检测一个 PANID 冲突

任何运行在一个网络上的设备，接收到一个 MLME-BEACON-NOTIFY.indication，其中信标负载的 PAN 标识符匹配它自己的 PAN 标识符，但是信标负载所含的 EPID 值不出现，或不等于 *nwkExtendedPANID*，就应视为检测到一个 PAN 标识符冲突。

检测到一个 PAN 标识符冲突的节点应构造一个网络报告命令帧，类型是 PAN Identifier Conflict，发送给 *nwkManagerAddr* 属性给出的地址确定的设备。报告信息域含有本地附近区域使用的所有 16 位 PAN 标识符的一个列表。如何创建这个列表在本规范的范围之外，但是建议发出 ACTIVE 类型的 MLME-SCAN.request 之后就创建。

3.6.1.13.2 接收一个网络报告命令帧

NIB 参数 *nwkManagerAddr* 所含的 16 位网络地址确定的设备是接收网络报告命令帧的设备，命令帧类型是 PAN identifier conflict。

接收网络报告命令帧时，指定的网络层功能管理器²⁴应为网络选择一个新的 16 位 PAN 标识符。新的 PAN 标识符是随机选择的，但是需要进行检查以确保所选的 PAN 标识符尚未用于本地附近区域，而且不含在网络报告命令帧的报告信息域中。

一旦一个新的 PAN 标识符已经选定，指定的网络层功能管理器²⁴首先增加 NIB 属性 *nwkUpdateId*（如果必要的话回到 0），然后构造一个网络更新命令帧，类型是 PAN identifier update。更新信息域设置为新的 PAN 标识符的值。

它发送出这个命令帧之后，指定的网络功能层管理器²⁶启动一个定时器，值等于 *nwkNetworkBroadcastDeliveryTime* 秒。当定时器到期，ZigBee 协调器通过重发出带有新的 PANID 的 MLME-START.request，把它当前的 PAN ID 修改为新选择的 PAN ID。

传输网络更新命令帧时，指定的网络层功能管理器²⁷应创建一个 NLME-NWK-STATUS.indication 原语，且 NetworkAddr 参数设置为 0，状态参数设置为 PAN Identifier Update。

24. CCB #819

25. CCB #819

26. CCB #819

27. CCB #819

3.6.1.13.3 接收一个网络更新命令帧

收到类型为 PAN identifier update 的一个网络更新命令帧时，设备启动一个定时器，值等于 *nwkNetworkBroadcastDeliveryTime* 秒。当定时器到期，设备把它当前的 PAN 标识符修改为 Update Information 域所含的值。

传输网络更新命令帧时，设备创建一个 NLME-NWK-STATUS.indication 原语，NetworkAddr 参数设置为 0，状态参数设置为 PAN Identifier Update。

从 NIB 属性 *nwkManagerAddr* 确定的设备收到一个网络更新命令，update id 域所含的值必须存储在 NIB 属性 *nwkUpdateId* 中。信标负载也必须更新。

3.6.2 传输和接收

3.6.2.1 传输

只有那些当前互相连接的设备可以从 NWK 层发送数据帧。如果一个没有连接的设备收到一个传输一个帧的请求，它必须丢弃该帧，并通过发出一个状态为 INVALID_REQUEST 的 NLDE-DATA.confirm 原语，通知上层这一错误。

所有在 NWK 层内处理或生成的帧必须根据图 3.5 指明的通用帧格式构造，并使用 MAC 子层数据服务传输。

除了源地址和目标地址域，所有的 NWK 层传输必须包括一个半径域和一个序列号域。对于一个更高层发出的数据帧，半径域的值可以使用 NLDE-DATA.request 原语的 Radius 参数来提供。如果没有提供一个值，那么 NWK 头的半径域必须设置为 NWK IB 的 *nwkMaxDepth* 属性值的两倍（参见 3.5 节）。每个设备的 NWK 层必须保持一个序列号，初始化为一个随机值。序列号必须在每次 NWK 层构造一个新的 NWK 帧时增加 1，即因为上层请求传输一个新的 NWK 数据帧，或当它需要构造一个新的 NWK 层命令帧时。增加之后，序列号的值必须插入帧的 NWK 头的序列号域中。

一旦完成了一个 NPDU，如果该帧需要安全功能，必须把它传递给安全服务提供商，以根据指定的安全组合进行随后的处理（参见 4.2.2 小节）。如果 NLDE-DATA.request 的 SecurityEnable 参数等于 FALSE，则不需要安全处理。如果 *nwkSecurityLevel* 指定的 NWK 安全级别等于 0，或如果更高层发出该帧且 *nwkSecureAllFrames* NIB 属性设置为 0x00，那么帧控制域的安全子域必须总是设置为 0。

成功完成安全处理之后，安全组合把该帧返回到 NWK 层以传输。处理过的帧将附加一个正确的辅助头。如果帧的安全处理失败，且该帧是一个数据帧，该帧将通知上层 NLDE-DATA.confirm 原语的状态。如果帧的安全处理失败，且该帧是一个网络命令帧，它被丢弃，且不再进行进一步处理。

当构造了该帧且准备传输，它必须被送到 MAC 数据服务。通过给 MAC 子层发出一个 MCPS-DATA.request 原语发起一个 NPDU 传输。然后 MCPS-DATA.confirm 原语返回传输的结果。

3.6.2.2 接收和拒绝

为了接收数据，一个设备必须开启它的接收器。上层可以使用 NLME-SYNC.request 原语发起接收。在一个开启信标的网络中，NWK 层接收本原语将导致一个设备与其父节点的下一个信标同步，且可选地跟踪未来的信标。NWK 层必须通过发出一个 MLME-SYNC.request 给 MAC 子层来完成。在一个没有开启信标的网络中，NLME-SYNC.request 将导致 NWK 层使用 MLME-POLL.request 原语轮询设备的父节点。

在一个没有开启信标的网络中，一个 ZigBee 协调器或 ZigBee 路由器的 NWK 层必须保证，在最大程度的可行情况下，无论何时，接收器都是开启的状态。在一个开启信标的网络中，NWK 层必须保证，在它自己的超帧和它父节点的超帧的活跃期间，当设备不传输时接收器开启。为此 NWK 层可以使用 MAC PIB 的 *macRxOnWhenIdle* 属性。

一旦接收器开启，NWK 层将开始通过 MAC 数据服务接收帧。在每次接收帧时，NWK 头的半径域必须减 1。如果因为该值减少变成 0，在任何情况下，该帧都不能再被传输。但是它可以传递给上层，或以由 NWK 层其它方式处理，它属于本规范以外的概述。以下数据帧必须使用 NLDE-DATA.indication 原语传递给上层：

- 一个广播地址的帧匹配一个广播组，且设备是其中的一个成员。
- 单播数据帧和源地址数据帧，且目标地址匹配设备的网络地址。
- 多播数据帧，其中组标识符列在 *nwkGroupIDTable* 中。

如果接收设备是一个 ZigBee 协调器或一个运行的 ZigBee 路由器，即一个已经激活了 NLME-START-ROUTER.request 原语的路由器，它必须按照以下处理数据帧：

- 广播和多播数据帧必须根据 3.6.5 和 3.6.6 小节所述的程序进行中继。
- 单播的数据帧，且目标地址不匹配设备的网络地址，必须根据 3.6.3.3 小节所述的程序进行中继。（在所有其它情况下，单播数据帧必须被立即丢弃。）
- 源路由的数据帧，且目标地址不匹配设备的网络地址，必须根据 3.6.3.3.2 小节所述的程序中继。
- 处理路由命令帧的程序概述在 3.6.3.5.2 小节中。
- 处理目标地址匹配设备的网络地址的路由回复命令帧的程序在 3.6.3.5.3 小节中。
- 目标地址和设备的网络地址不匹配的路由回复命令帧必须被立即丢弃。路由错误命令帧必须和数据帧以同样的方式处理。

NWK 层必须指明上层使用 NLDE-DATA.indication 原语进行一个数据帧的接收。

在接收一个帧时，NLDE 必须检查帧控制域安全子域的值。如果这个值不为零，NLDE 必须把该帧传递给安全服务供应商（参见 4.2.2）根据指定的安全组合以作随后的处理。如果安全子域设置为 0，NIB 的 *nwkSecurityLevel* 属性不为零，且输入帧是一个 NWK 命令帧，NLDE 必须丢弃该帧。如果安全子域设置为 0，NIB 的 *nwkSecurityLevel* 属性不为零，且输入帧是一个 NWK 数据帧，NLDE 必须检查 *nwkSecureAllFrames* NIB 属性。如果这个属性设置为 0x01，NLDE 只接受能目的地是它自己的帧，即它不需要转发给其它设备。

3.6.3 寻找路由

ZigBee 协调器和路由器必须提供以下功能：

- 代表更高层中继数据帧
- 代表其它 ZigBee 路由器中继数据帧
- 参与路由发现，为随后的数据帧建立路由
- 代表终端设备参与路由发现
- 参与路由修复
- 使用路由发现指明的 ZigBee 路径消耗指标

ZigBee 协调器或路由器可以提供以下功能：

- 维护路由表以记住可用的最佳路由
- 代表更高层启动路由发现
- 代表其它 ZigBee 路由器启动路由发现
- 启动路由修复
- 实施邻居路由

3.6.3.1 路由消耗

ZigBee 路由算法在路由发现和维护期间为路由比较使用了一个路径消耗指标。为了计算这个指标，与每个路径的链路有关的一个成本，叫做链路消耗，以及链路消耗值总结为此路径的产生整体消耗。

更正式地，如果我们定义了一个长度为 L 的路径 P ，设备组的顺序为 $[D1, D2 \dots DL]$ ，和一个链路 $[Di, Di+1]$ 作为一个长度为 2 的子路径，那么路径消耗

$$C\{P\} = \sum_{i=1}^{L-1} C\{[Di, Di+1]\}$$

其中 $C\{[Di, Di+1]\}$ 的每个指的是一个链路消耗。一个链路 l 的链路成本 $C\{l\}$ 是一个函数，值 $[0 \dots 7]$ 的间隔，定义如下：

$$C\{l\} = \begin{cases} 7, \\ \min\left(7, \text{round}\left(\frac{1}{p_l^4}\right)\right) \end{cases}$$

其中 p_l 定义为数据包在此链路上转发的可能性。

因此，执行者可以报告一个值为 7 的常量作为链路消耗，或他们可以报告一个反映 p_l 接收可能性的值——具体地，互相的可能性——它必须反过来反映需要通过使用的链路得到一个数据包每次的期望尝试次数。一个提供这两种选择的设备可以通过设置 NIB 属性 *nwkReportConstantCost* 的值为 TRUE，强制它报告一个常量链路消耗。如果 NIB 属性 *nwkSymLink* 值为 TRUE，那么 *nwkReportConstantCost* 属性值必须为 FALSE，且 NWK 层必须按照上述²⁸方式计算路由消耗。

但是还有一个问题，是如何估计或测量 p_l 。这主要是一个执行问题，执行者可以自由运用他们的构思。 p_l 可以通过一段时间内接收信标和数据帧的实际次数来评估，并注意合适的序列号来检测丢失的帧。这通常被看做接收可能性最精确的测量方法。但是，最简单的方法，且对所有情况都可用，是根据 IEEE802.15.4-2003 MAC 和 PHY 提供的每帧 LQI 的平均值形成的。即使使用了其它一些方法，最初的消耗估计必须基于平均的 LQI。一个表驱动函数可以用于映射平均 LQI 值到 $C\{l\}$ 的值。强烈建议执行者检查他们表中的硬件生产测试中获得的数据，因为不准确的消耗将妨碍 ZigBee 路由算法的运行能力。

3.6.3.2 路由表

一个 ZigBee 路由器或 ZigBee 协调器可以维护一个路由表。一个 ZigBee 路由表中必须存储的信息展示在表 3.51 中。建议从条目中收回不再使用的老化和退休路由表条目的表空间；但是这在本规范的范围之外。

表 3.51 路由表条目

字段名	大小	描述
Destination address	2 octets	16位网络地址或者该设备的组ID;如果目的设备是一个 ZigBee 路由器或者 ZigBee 协调器, 该域应该包含那个设备的实际16位地址; 如果目的设备是一个终端设备, 该域应该包含那个设备父节点的16位网络地址。
Status	3 bits	路由的状态。见表3.52中的值。
No route cache	1 bit	一个标志, 表示这个地址指示的目标没有存储源路由。
Many-to-one	1 bit	代表目的地是一个集线器发出多对一的请求的标志。
Route record required	1 bit	代表路由记录命令帧应该在发送到目的之间发送到一个数据库包的标志。
GroupID flag	1 bit	代表目的地址是一个组ID的标志。
Next-hop address	2 octets	到目的地的下一跳的16位网络地址。

表 3.52 枚举了路由状态域的值。

表 3.52 路由状态值

数值	状态
0x0	ACTIVE
0x1	DISCOVERY_UNDERWAY
0x2	DISCOVERY_FAILED
0x3	INACTIVE
0x4	VALIDATION_UNDERWAY
0x5 – 0x7	保留

本节描述了路由算法。名词“路由表能力”用于描述一种情况, 即一个设备有能力使用其路由表来建立到某个目标设备的一条路由。如果一个设备有以下条件, 就说它有路由表能力:

- 它是一个 ZigBee 协调器或 ZigBee 路由器
- 它维护一个路由表

- 它有一个空闲的路由表条目或它已经有一个对应于目标地址的路由表条目

如果一个 ZigBee 路由器或 ZigBee 协调器维护一个路由表，它也必须维护一个路由发现表，包含表 3.53 所示的信息。路由表条目能保存很长时间，是持续的，而路由发现表条目仅能持续一个路由发现操作的期限，可以重复使用。

表 3.53 路由发现表条目

字段名	大小	描述
Route request ID	1 octet	每次设备初始化请求自动增加的路由请求命令帧的序列号。
Source address	2 octets	路由请求的发起者的16位网络地址。
Sender address	2 octets	发送最低消耗路由请求命令帧对应的该实体的路由请求标识符和源地址设备的16位网络地址，该域用来确认最终路由应答命令帧应该遵循的路径。
Forward Cost	1 octet	来自源路由请求到当前设备的累计路径成本。
Residual cost	1 octet	来自当前设备到目的设备的累计路径成本。
Expiration time	2 octets	倒数计时器代表直到路由发现终止的毫秒数；初始化值是 <code>nwkRouteDiscoveryTime</code> 。

如果一个设备有以下条件，就说它有“路由发现表能力”：

- 它维护一个路由发现表
- 在它的路由发现表中有一个空闲的条目

如果一个设备既有路由表能力又有路由发现表能力，那么可以说它有“路由能力”。

路由发现期间，一个 ZigBee 路由器或 ZigBee 协调器需要维护以便于参与发现某条路由的信息，分布在一个路由表条目和一个路由发现表条目之间。一旦发现完成，只有路由表条目需要维护，以便 NWK 层沿着所发现的路由执行路由寻找。在本小节中，提到了一个路由表条目及其“对应”路由发现表条目之间的这种关系，反之亦然。这一对应的维护取决于执行者，因为表中的条目没有任何共同之处，但是值得注意的是定义一个路由发现的唯一“密钥”，是路由发现命令帧的源地址，路由请求 ID 由该设备生成，并在命令帧负载中携带。

如果一个设备有能力发起一个多对一路由请求，它还必须维护一个路由记录表（见表 3.45）。

3.6.3.3 接收一个单播帧

在接收一个来自 MAC 子层的单播帧，或来自上层的 NLDE-DATA.request 时，NWK 层根据以下程序为它寻找路由。

如果接收设备是一个 ZigBee 路由器或 ZigBee 协调器，且帧的目标地址是一个 ZigBee 终端设备，而且是接收设备的子节点，该帧必须使用 MCPS-DATA.request 原语直接送给目标地址，如 3.6.2.1 小节所述。帧还必须设置下一跳目标地址等于最终的目标地址。否则，为了随后进行的讨论，如果它是一个路由器或协调器或终端设备且 *nwkAddrAlloc* 值为 0x02，或它的父节点的网络地址，如果它是一个终端设备且 *nwkAddrAlloc* 值为 0x00，就定义 *routing address* 为这个设备的网络地址。定义一个帧的 *routing destination* 为该帧的 NWK 目标的路由地址。注意分布式地址分配使得能够从它的地址获取任何设备的路由地址。详细信息参见 3.6.1.6 小节。

一个 ZigBee 路由器或 ZigBee 协调器可以检查邻居表，寻找对应帧的路由目标的一个条目。如果有这样一个条目，设备可以使用 3.6.2.1 小节所述的 MCPS-DATA.request 原语直接路由该帧到这个目标。

一个具有路由能力的设备必须检查其路由表寻找一个对应帧的路由目的地的条目。如果有这样的一个条目，且如果该条目路由状态域的值为 ACTIVE 或 VALIDATION_UNDERWAY，设备必须使用 MCPS-DATA.request 原语中继该帧，且如果该条目的路由状态域还没有该值，设置它为 ACTIVE。如果路由表条目中的多对一域设置为 TRUE，NWK 必须按照 3.6.3.5.4 小节所述的程序确定是否必须发送一个路由记录命令帧。

当中继一个单播数据帧时，MCPS-DATA.request 原语的 SrcAddrMode 和 DstAddrMode 参数的值必须都为 0x02，表示 16 位地址。SrcPANId 和 DstPANId 参数的值必须都是中继设备 MAC PIB 属性 macPANId 提供的值。SrcAddr 参数设置为中继设备 MAC PIB 属性 *macShortAddress* 的值，且 DstAddr 参数必须是对应于路由目标的路由表条目的下一跳地址域的值。TxOptions 参数的位 *b0* 设置为 1，表示这是一个有确认的传输。

如果设备有一个路由表条目对应帧的路由目标，但是该条目路由状态域的值是 DISCOVERY_UNDERWAY，设备就通过查询它的发现表确定是否已经发起了发现。如果设备已经发起了发现，必须视为已经为该帧发起路由发现，否则，设备发起 3.6.3.5.1 小节所述的路由发现。帧可以有选择地缓冲挂起路由发现，或如果 NIB 属性 *nwkUseTreeRouting* 值为 TRUE，使用分层路由沿着树寻找路由。如果该帧沿着树寻找路由，NWK 头帧控制域的发现路由子域必须设置为 0x00。

如果设备有一个路由表条目对应帧的路由目标，但是该条目的路由状态域值为 DISCOVERY_FAILED 或 INACTIVE，设备可以使用分层路由沿着树为该帧寻找路由。条件是 NIB 属性 *nwkUseTreeRouting* 值为 TRUE。如果设备没有对应于路由目标，状态值为 ACTIVE 或 VALIDATION_UNDERWAY 的一个路由表条目，且它从上层收到该帧，它必须检查其源路由表，寻找对应路由目标的一个条目。如果找到了这样的一个条目，设备必须使用 3.6.3.3.1 小节所述的源路由传输该帧。如果设备没有对应路由目标的一个路由表条目且不是它使用源路由发起该帧，它必须检查 NWK 头帧控制域的发现路由子域。如果发现路由子域值为 0x01，设备必须发起路由发现，如 3.6.3.5.1 小节所述。如果发现路由子域值为 0，NIB 属性 *nwkUseTreeRouting* 值为 FALSE，且没有对应帧的路由目标的路由表，该帧必须被丢弃，且 NLDE 必须发出状态值为 ROUTE_ERROR 的 NLDE-DATA.confirm 原语。

一个没有路由能力的设备必须使用分层路由沿着树寻找路由，且 NIB 属性 *nwkUseTreeRouting* 的值为 TRUE。如果 NIB 属性 *nwkUseTreeRouting* 的值是 FALSE，帧就被丢弃。如果帧是因为来自当前设备的 NHL 的一个 NLDE-DATA.request 产生的，NLDE 就发出状态值为 ROUTE_ERROR 的 NLDE-DATA.confirm 原语。如果帧代表其它设备被中继，NIME 发出一个网络状态命令帧，发给帧的源设备，状态为 0x04，表示缺少路由能力。它还要给上层发出 NLME-NWK-STATUS.indication，NetworkAddr 参数等于帧的 16 位网络地址，状态参数等于 0x04，表示缺少路由能力。

对于分层路由，如果目标是设备的一个子孙，设备必须把帧送到合适的子节点。如果目标是一个子节点，且它还是一个终端设备，该帧的交付可能因为子节点设备的 *macRxOnWhenIdle* 状态失败。如果子节点的 *macRxOnWhenIdle* 设置为 FALSE，可以使用 IEEE 802.15.4-2003[B1]描述的间接传输来交付该帧。如果目标不是一个子孙，设备必须把帧送给其父节点。

网络中的其它每个设备都是 ZigBee 协调器的一个子孙，且网络中没有设备是任何 ZigBee 终端设备的子孙。对于一个地址是 A 深度是 d 的 ZigBee 路由器，如果以下逻辑表达式为真，那么地址是 D 的一个目标设备是一个子孙：

$$A < D < A + Cskip(d-1)$$

对于 $Cskip(d)$ 的定义，参见 3.6.1.6 小节。

如果确定了目标是接收设备的一个子孙，下一跳设备的地址 N 由以下公式给定：

$$N=D$$

上式是对于 ZigBee 终端设备来说，其中 $D > A + Rm \times Cskip(d)$ ，否则：

$$N = A + 1 + \left\lfloor \frac{D - (A + 1)}{Cskip(d)} \right\rfloor \times Cskip(d)$$

如果一个 ZigBee 路由器或 ZigBee 协调器的 NWK 层因为任何原因传递一个单播或多播数据帧失败，路由器或协调器必须尽最大努力报告这次失败。交付一个 NLME-NWK-STATUS 失败不报告。这个报告可以采取两种形式之一。如果失败的数据帧因为上层的请求被中继，那么 NWK 层必须给上层发出一个带有错误的 NLDE-DATA.confirm。该原语 NetworkAddr 参数的值必须是帧的预定目的地。如果帧是代表另一个设备被中继，那么中继设备必须发送一个网络状态命令帧，送回数据帧的源。网络状态命令帧的目标地址域必须采取失败数据帧的目标地址域。

在另外一种情况下，可能报告的失败原因在表 3.42 中。

3.6.3.3.1 发起一个源路由数据帧

如果从上层收到一个数据帧，确定必须使用如上所述的源路由传输该帧，就从源记录表中检索源路由。

如果没有中间的中继节点，帧必须通过使用 MCPS-DATA.request 原语，直接传输给路由目标，且 DstAddr 参数值表示路由目标地址。

如果有至少一个中继节点，NWK 头帧控制域的源路由标志必须设置，且 NWK 头源路由子域必须出现。源路由子帧的中继次数子域值必须等于中继列表中的中继次数。中继索引子域值必须等于 1，小于中继次数。中继列表子域必须包含中继地址的列表，最低字节优先。接近目的地的中继必须首先列出。接近发出者的中继必须最后列出。

设备必须使用 MCPS-DATA.request 原语中继该帧。DstAddr 参数的值必须为中继列表中最后的中继地址。

3.6.3.3.2 中继一个源路由数据帧

如 3.6.3.3 小节所述，从 MAC 子层收到一个源路由数据帧时，如果源路由子帧的中继索引子域值为 0，设备应检查帧的 NWK 头的目标地址域。如果帧的 NWK 头的目标地址域等于 NIB 属性 *nwkNetworkAddress* 的值，那么帧应使用 NLDE-DATA.indication 原语被传递给上层。如果目标地址域不等于 NIB 属性 *nwkNetworkAddress* 的值，且接收设备是一个 ZigBee 路由器或 ZigBee 协调器，设备就使用 MCPS-DATA.request 原语直接中继该帧到 NWK 头目标，否则，帧被默默丢弃。

如果中继索引子域值大于 0，设备比较它的网络地址和中继列表的中继索引找到的地址。如果这两个地址不匹配，帧被丢弃，不采取进一步行动。否则，设备把中继索引子域减 1，并立即中继该帧到中继列表子域中它自己的地址前面的那个地址。

当中继一个源路由数据帧，设备的 NWK 层还应检查对应于帧的源地址的路由表条目。如果路由表条目的路由记录缓存域值为 FALSE，那么路由表条目的路由记录请求域应设置为 FALSE。

3.6.3.4 链路状态信息

无线链路可能是不对称的，即它们可以在一个方向上工作而不是其它方向。这可能导致路由回复失败，因为它们沿着路由请求所发现的链路往回传播的。

对于多对一路由和双向路由发现 (*nwkSymLink* = TRUE)，要求发现路由在两个方向上都是可靠的。要达到这个目标，路由器测量到它们的邻居的交换链路消耗是通过定期传输链路状态帧作为一个单跳广播完成的。然后反向链路消耗信息用于路由发现期间，以确保所发现的路由在两个方向都使用了高质量的链路。

3.6.3.4.1 发起一个链路状态命令帧

当加入一个网络，一个 ZigBee 路由器或协调器必须每过 *nwkLinkStatusPeriod* 秒定期发送一个链路状态命令，作为一个没有重试的单跳广播。如果需要的话可以发送地更为频繁一些。应增加随机抖动以避免与其它节点同步。链路状态命令帧的格式见 3.4.8 小节。

终端设备不发送链路状态命令帧。

3.6.3.4.2 接收一个链路状态命令帧

一个 ZigBee 路由器或协调器接收一个链路状态命令帧，对应发送设备的邻居表条目的 *age* 域就重置为 0。一个帧所涵盖的地址列表从链路状态列表中的第一个和最后一个地址，以及命令选项域的第一个帧和最后一个帧比特确定。如果接收者的网络地址在帧所涵盖的范围之外，帧就被丢弃，程序终止。如果接收者的网络地址在帧所涵盖的范围内，那么就查询链路状态列表。如果找到了接收者的地址，对应发送者的邻居表条目的输出消耗域设置为链路状态条目的输入消耗的值。如果没有找到接收者的地址，输出消耗域设置为 0。

终端设备不处理链路状态命令帧。

3.6.3.4.3 邻居表的老化

对于使用链路状态信息的设备，每过 *nwkLinkStatusPeriod* 邻居表中的路由器的年龄域就增加。如果这个值超过了 *nwkRouterAgeLimit*，邻居表条目的输出消耗域就设置为 0。换句话说，如果一个设备在 *nwkRouterAgeLimit* 内从一个路由器邻居接收链路状态信息连续失败，老的输出消耗信息就被丢弃。在这种情况下，邻居条目就被视为是陈旧的，如果必要的话就重新使用，为一个新的邻居腾出空间。终端设备不发出链路状态信息，因此从不被视为陈旧。

如果 *nwkAddrAlloc* 值为 0x00, 邻居的邻居表条目就不被视为是陈旧, 是可重复使用的。

3.6.3.5 路由发现

路由发现是网络设备通过 NWK 进行合作找到并建立路由的程序。*Unicast route discovery* 总是在某个源设备和某个目标设备之间执行的。*Multicast route discovery* 是在某个源设备和多播组之间执行的。*Many-to-one route discovery* 是由一个源设备执行的, 以在一个给定的半径内, 从所有 ZigBee 路由器和 ZigBee 协调器建立到它自身的路由。发起多对一路由发现的源设备被指定为一个集中器, 在本文件中这样命名。在整个 3.6.3.5 小节中, *a destination address* 可以是一个 16 位广播地址, 某个设备的 16 位 NWK 地址, 或一个 16 位多播地址, 也叫做一个多播组 ID。一个路由请求命令, 其中目标地址是某个设备路由地址, 路由请求选项域没有设置多播位, 它就是一个 *unicast route request*。一个路由请求命令, 其中路由请求选项域设置了多播位, 它就是一个 *multicast route request*。一个多播路由请求的目标地址域必须是一个多播组 ID。一个路由请求命令负载, 其中目标地址子域是一个广播地址 (参见表 3.54), 它就是一个 *many-to-one route request*。一个多对一路由请求的路由请求选项域的多播位必须设置为 0。

3.6.3.5.1 发起路由发现

一个设备的单播路由发现程序必须由 NWK 层在一个 ZigBee 路由器或 ZigBee 协调器上发起, 当 NWK 层从上层收到一个 NLDE-ROUTE-DISCOVERY.request 原语, 其中 DstAddrMode 参数值为 0x02; 或从上层收到一个 NLDE-DATA.request 原语, 且 DstAddrMode 参数设置为 0x02, 发现路由子域设置为 0x01, 且没有路由表条目对应目标设备的路由地址 (DstAddr 参数指定的 16 位网络地址); 或从 MAC 子层收到一个帧, 且 NWK 头目标地址域的值不是当前设备的地址, 当前设备的终端设备子节点的地址, 或一个广播地址, 以及:

- 帧控制域的发现路由子域的值为 0x01, 且
- 没有路由表条目对应路由帧的目标, 且
- 收到帧的 NWK 头的源地址域的值和当前设备的终端设备子节点之一的 16 位网络地址相同, 或
- NIB 属性 *nwkUseTreeRouting* 的值为 TRUE。

一个多播地址的路由发现程序必须由 NWK 层发起，以响应从上层收到一个 NLME-ROUTE-DISCOVERY.request 原语，其中 DstAddrMode 参数值为 0x01，或响应 3.6.6.2.2 小节指定的程序。

如果设备发起的路由发现没有路由表条目对应目标设备的路由地址，它必须建立一个路由表条目，状态等于 DISCOVERY_UNDERWAY。如果设备有一个现存的路由表条目对应目标设备的路由地址，且状态等于 ACTIVE 或 VALIDATION_UNDERWAY，该条目必须被使用，且状态域必须保持该条目的当前值。如果它有一个现存的路由表条目，且状态不是 ACTIVE，该条目必须被使用，且该条目的状态必须设置为 DISCOVERY_UNDERWAY。如果具有相同发起者和路由请求 ID 的相应的路由发现表条目尚未存在，设备还要建立一个。

每个发出一个路由请求命令帧的设备必须维护一个计数器，用于生成路由请求标识符。当一个新的路由请求命令帧创建时，路由请求计数器加 1，且值存储在设备的路由发现表的 Route request 标识符域中。路由表中和路由发现表的其它域设置为 3.6.3.2 小节所述的值。

NWK 层可以选择缓冲收到的帧，挂起路由发现或如果该帧是一个单播帧，且 NIB 属性 *nwkUseTreeRouting* 值为 TRUE，设置 NWK 头的帧控制域的发现路由子域为 0，沿着树转发该数据帧。

一旦设备创建了路由发现表和路由表条目，路由请求命令帧必须按图 3.12 描述的负载创建。不同的域的入口如下：

- 命令帧标识符域必须设置为表示命令帧是一个路由请求，参见表 3.40。
- 路由请求标识符域必须设置为存储在路由发现表条目中的值。
- 多播标志和目标地址域必须根据要被发现的路由的目标地址设置。
- 路径消耗域必须设置为 0。

一旦创建，路由请求命令帧就准备广播，使用 MCPS-DATA.request 原语传递给 MAC 子层。

当发起路由发现，广播一个路由请求命令帧时，NWK 层必须在初始的广播之后，重试广播 *nwkInitialRREQRetries* 次，由此产生一个最大 *nwkInitialRREQRetries* + 1 次的传输。重试将以 *nwkRREQRetryInterval* 毫秒的时间间隔分开。

多对一的路由发现程序必须由一个 ZigBee 路由器或协调器的 NWK 层，从上层收到一个 NLME-ROUTE-DISCOVERY.request 原语，且 DstAddrMode 参数值为 0x00 时发起。一个多对一的路由请求命令帧不能重试；但是，仍然创建一个发现表条目以在 *nwkRouteDiscoveryTime* 周期期间²⁹内提供循环检测。如果 NLME-ROUTE-DISCOVERY.request 原语的 NoRouteCache 参数是 TRUE，命令帧负载的命令选项域的多对一子域应设置为 2。否则，多对一子域设置为 1。注意在这种情况下，NWK 层应维护一个路由记录表。NWK 头的目标地址域应设置为 0xffff，即所有路由器的广播地址。广播半径设置为 *nwkConcentratorRadius* 的值。发起多对一路由发现的源设备指定为一个集中器，在本文件中这样命名，且 NIB 属性 *nwkIsConcentrator* 设置为 TRUE。如果设备的 *nwkIsConcentrator* 等于 TRUE，且 *nwkConcentratorDiscoveryTime* 是非零值，网络层就每过 *nwkConcentratorDiscoveryTime* 发出一个路由请求命令帧。

3.6.3.5.2 接收一个路由请求命令帧

收到一个路由请求命令帧时，如果设备是一个终端设备，它必须丢弃该帧。否则，它必须确定它是否有路由能力。

如果设备没有路由能力，且路由请求是一个多播的路由请求或一个多对一的路由请求，路由请求必须被丢弃，路由请求程序必须终止。

如果 *nwkAddrAlloc* 是 0x00，设备没有路由能力，且路由请求是一个单播的路由请求，设备必须检查该帧是否是沿着一个合法的路径被接收的。如果该帧从设备子节点设备之一接收，且源设备是该子节点设备的一个子孙，或如果该帧从设备父节点设备接收，且源设备不是该设备的一个子孙，路径就是合法的。如果路由请求命令帧不是沿着一个合法的路径接收，它必须被丢弃。否则，设备必须检查它是否是预定的目的地。它还必须通过比较路由请求命令帧负载的目标地址域和它的终端设备子节点，如果有的话，检查命令帧的目标是否是它的终

端设备子节点之一。如果该设备或它的终端设备子节点之一是路由请求命令帧的目标，它必须以一个路由回复命令帧回复。当以一个路由回复命令帧回复一个路由请求时，设备必须构造一个帧，帧类型域设置为0x01。路由回复的源地址必须设置为创建路由回复的设备的16位网络地址，目标地址必须设置为计算出来的下一跳地址，考虑路由请求的发出者作为一个目的地。从下一跳设备到当前设备的链路消耗必须按照3.6.3.1小节描述的进行计算，并插入路由回复命令帧的路径消耗域中。路由回复命令帧必须通过发出一个MCPS-DATA.request原语单播给下一跳设备。

如果设备不是路由请求命令帧的目的地，设备必须计算从之前传输该帧的设备的链路消耗，如 3.6.3.1 小节所述。这个值必须增加到存储在路由请求命令帧的路径消耗值上面。然后路由请求命令帧必须使用 MCPS-DATA.request 服务原语单播给目的地。这个单播传输的下一跳以同样的方式确定，就像帧是一个数据帧，地址是负载中目标地址域确定的设备。

如果设备有路由能力，且收到的请求是一个单播路由请求，设备必须通过比较路由请求命令帧负载的目标地址域和它自己的地址，检查它是否是命令帧的目标。它还要通过比较路由请求命令帧负载的目标地址域和它每个终端设备子节点的地址，如果有的话，检查命令帧的目标地址是否是它的终端设备子节点之一。如果设备或它的终端设备子节点之一是路由请求命令帧的目标，设备必须确定，是否有一个路由发现表（参见表 3.53）条目，具有相同的路由请求标识符和源地址域。如果不存在这样的条目，应该创建一个。

如果设备有路由能力，且收到的路由请求帧的路由请求命令选项域的多播子域表示一个多播路由请求，设备必须确定在 *nwkGroupIDTable* 中是否已经存在一个条目，其中组标识符域匹配帧的目标地址。如果找到一个匹配条目，设备必须确定是否存在一个路由发现表（参见表 3.53）条目，它具有相同的路由请求标识符和源地址域。如果没有这样的条目存在，应该创建一个。

对于多对一路由请求，以及常规的路由请求，如果 *nwkSymLink* 参数是 TRUE，接收一个路由请求命令帧时，查询邻居表，寻找对应发送设备的一个条目。如果没有找到这样的条目，或如果条目的输出消耗域值为 0，帧就被丢弃，路由请求程序终止。邻居的输入和输出消耗的最大值用于路径消耗的计算，而不是输入消耗。这包括转发之前路由请求帧的路径消耗域增加的值。

当创建路由发现表条目时，各个域设置为路由请求命令帧的相应的值。唯一例外的是转发消耗域，它通过使用命令帧之前的发送者来计算链路消耗，如 3.6.3.1 小节所述，并把它增加到路由请求命令帧所含的路径消耗上面。以上计算的结果存储在新创建的路由发现表条目的转发消耗域中。如果 *nwkSymLink* 属性设置为 TRUE，设备还必须创建一个路由表条目，目标地址域设置为路由请求命令帧的源地址，下一跳域设置为之前传输命令帧的设备的地址。状态域必须设置为 ACTIVE。然后设备必须给路由请求命令帧的源发出一个路由回复命令帧。对于源地址和路由请求标识符对，在设备已经有一个路由发现表条目的情况下，设备必须确定路由请求命令帧的路径消耗是否小于存储在路由发现表条目中的转发消耗。比较的完成是通过首先计算从之前发送该帧的设备的链路消耗，如 3.6.3.1 小节所述，然后把它增加到路由请求命令帧的路径消耗值上面。如果该值大于路由发现表条目中的值，帧必须被丢弃，不再需要进一步处理。否则，在路由发现表中的转发消耗和发送者地址域被更新为新的消耗和之前转发路由请求命令帧的设备的地址。

如果 *nwkSymLink* 属性设置为 TRUE 且收到的路由请求命令帧是一个单播的路由请求，设备也必须创建一个路由表条目，目标地址域设置为路由请求命令帧的源地址，下一跳域设置为之前转发该命令帧的设备的地址。状态域必须设置为 ACTIVE。然后设备必须响应一个路由回复命令帧。在这两种情况下，如果设备代表它的终端设备子节点之一响应，路由回复命令帧负载的响应者地址必须设置为等于该终端设备子节点的地址，而不是响应设备的地址。

当一个设备有路由能力，但它不是收到的路由请求命令帧的目标，它必须确定是否有一个路由发现表条目（参见表 3.53），它有相同的路由请求标识符和源地址域。如果没有这样的条目存在，必须创建一个。路由请求计时器必须设置为在 *nwkRouteDiscoveryTime* 毫秒后过期。如果存在一个路由表条目和目标路由地址相对应，且它的状态不是 ACTIVE 或 VALIDATION_UNDERWAY，状态必须设置为 DISCOVERY_UNDERWAY。如果不存在这样的条目，且该帧是一个单播的路由请求，必须创建一个条目，且它的状态设置为 DISCOVERY_UNDERWAY。

如果帧是一个多对一路由请求，设备也应创建一个路由表条目，通过设置下一跳域为之前传输该命令帧的设备的地址，使目标地址域等于路由请求命令帧的源地址。如果帧是一个多对一的路由请求（即命令帧负载的命令选项域的多对一子域的值非零），路由表条目的多对一域必须设置为 TRUE，且如果命令帧负载的命令选项域的多对一子域的值为 2，没有路由缓存标志设置为 TRUE，或如果值为 1，路由缓存标志设置为 FALSE。如果路由表条目是新的，或如果没有路由缓存标志设置为 TRUE，或如果下一跳域改变了，路由记录请求域就设置为 TRUE，否则它不变。状态域必须设置为 ACTIVE。当路由请求计时器到期，设备从路由发现表中移除路由请求条目。当这个发生时，如果它的状态域值为 DISCOVERY_UNDERWAY，且路由发现表中没有为该目标地址的路由发现创建其它的条目，对应目标路由地址的路由表条目也必须被移除。

如果路由发现表中已经存在一个条目，路由请求命令帧的路径消耗必须和路由发现表条目的转发消耗值相比较。比较的完成是通过计算从之前设备的链路消耗，如 3.6.3.1 小节所述，并把它增加到路由请求命令帧的路径消耗值上面。如果路径消耗更大，路由请求命令帧被丢弃，不再需要进一步处理。否则，路由发现表中的转发消耗和发送者地址域被更新为新的消耗和之前转发该路由请求命令帧的设备的地址。另外，路由请求命令帧的路径消耗域必须更新为比较之后计算出来的消耗。如果 *nwkSymLink* 属性设置为 TRUE，且收到的路由请求命令帧是一个单播的路由请求，设备也必须更新任一路由表条目，其中目标地址域设置为路由请求命令帧的源地址，下一跳域设置为之前传输该命令帧的设备的地址。状态域必须设置为 ACTIVE。然后设备必须使用 MCPS-DATA.request 原语广播该路由请求命令帧。

当广播一个路由请求命令帧时，NWK 层必须根据下式计算出的一个随机抖动数延迟重传：

$$2 \times R[nwkcMinRREQJitter, nwkcMaxRREQJitter]$$

其中 $R[a, b]$ 是一个随机的函数，间隔为 $[a, b]$ 。这个不稳定数的单位是毫秒。执行者可以调整这个不稳定数，以使接收到的路由消耗大的路由请求命令帧比路由消耗小的延时更大。NWK 层必须在最初的中继之后重试广播 *nwkcRREQRetries* 次，因此每次中继尝试有一个最大值 *nwkcRREQRetries* + 1 次中继。如果一个具有相同的源和路由请求标识符的帧到达，且它的路径消耗比等待重传的那个帧更低，在这种情况下，执行者可以选择丢弃等待重传的路由请求命令帧。

设备还可以把和负载中目标域的路由地址相对应的路由表条目的状态域设置为 DISCOVERY_UNDERWAY。如果不存在这样的条目，它必须被创建。

当以一个路由回复命令帧回复一个路由请求时，一个有路由发现表条目对应路由请求的源地址和路由请求标识符的设备必须构造一个命令帧，帧类型域设置为 0x01。NWK 头的源地址域必须设置为当前设备的 16 位网络地址，目标地址域必须设置为对应的路由发现表条目中的发送者地址域的值。构造路由回复的设备必须以下方式填入负载域。

- NWK 命令标识符必须设置为路由回复。
- 路由请求标识符域必须设置为和路由请求命令帧的路由请求标识符域相同的值。
- 发出者地址域必须设置为路由请求命令帧的网络头的源地址。
- 从路由发现表条目中使用对应路由请求命令帧的网络头的源地址的发送者地址域，设备必须计算 3.6.3.1 所述的链路消耗。这个链路消耗必须送入路径消耗域。

然后通过使用 MCPS-DATA.request 原语，把路由回复命令帧单播到目的地，从路由发现表中获得的发送者地址作为下一跳。

3.6.3.5.3 接收一个路由回复命令帧

在接收一个路由回复命令帧时，设备应执行以下程序。

如果接收设备没有路由能力且它的 NIB 属性 *nwkUseTreeRouting* 值为 TRUE，它必须使用树路由转发该路由回复，就像它是一个数据帧。如果接收设备没有路由能力且它的 NIB 属性 *nwkUseTreeRouting* 值为 FALSE，它必须丢弃该命令帧。在转发路由回复命令帧之前，设备必须通过计算从下一跳设备到它本身的链路消耗更新负载中的路径消耗域，如 3.6.3.1 小节所述，并把它增加到路由回复的路径消耗域的值上面。

如果接收设备有路由能力，它必须通过比较命令帧负载中发出者地址域的内容和它自己的地址，检查它是否是路由回复命令帧的目标。如果是的话，它必须查询其路由发现表，寻找对应于路由回复命令帧负载的路由请求标识符的一个条目。如果没有这样的条目，路由回复命令帧必须被丢弃，路由回复程序必须终止。如果存在一个路由发现表条目，设备必须查询其路由表，寻找一个条目目标地址域等于对应于路由回复命令帧的响应者地址的路由地址。如果没有这样的路由表条目，路由回复命令帧必须被丢弃，如果对应路由回复命令帧的路由请求标识符的一个路由发现表存在，它还必须被移除，路由回复程序必须终止。如果一个路由表条目和一个路由发现表条目存在，且如果路由表条目的状态域设置为 **DISCOVERY_UNDERWAY**，且路由表条目的 **GroupId** 标志为 **TRUE**，要不然就是 **ACTIVE**，它必须修改为 **VALIDATION_UNDERWAY**；路由表中的下一跳域必须设置为之前转发路由回复命令帧的设备。路由发现表条目的剩余消耗域必须设置为路由回复负载中的路径消耗域。

如果状态域已经设置为 **ACTIVE** 或 **VALIDATION_UNDERWAY**，设备必须比较路由回复命令帧的路径消耗和路由发现表条目中记录的剩余消耗，且如果路由回复命令帧的消耗较小的话，更新路由表条目的剩余消耗域和下一跳域。如果路由回复的路径消耗不是更小，路由回复必须被丢弃，不再采取进一步处理。

如果收到路由回复的设备不是目的地，设备必须找到路由发现表条目中对应路由命令帧负载中发出者地址和路由请求标识符的条目。如果不存在这样路由发现表条目，路由回复命令帧必须被丢弃。如果存在一个路由发现表条目，必须比较路由回复命令帧的路径消耗值和路由发现表条目的剩余消耗域。如果路由发现表条目的值小于路由回复的值，路由回复命令帧必须被丢弃。

否则，设备必须找到路由表条目，其中目标地址域等于路由回复命令帧的响应者地址相应的路由地址。如果路由发现表条目存在，但没有相应的路由表条目是错误的，路由回复命令帧必须被丢弃。路由表条目必须通过替换下一跳域为之前转发路由回复命令帧的地址，进行更新。路由发现表条目也必须通过替换剩余消耗域为路由回复命令帧的值，进行更新。

每当收到一个路由回复时导致相应的路由表条目的下一跳域被修改，且路由表条目的 GroupId 标志是 TRUE 时，设备必须设置相应的路由发现表条目的过期时间域，如果设备是路由回复的目的地，在 *nwkcWaitBeforeValidation* 毫秒后过期，如果它不是，在 *nwkcRouteDiscoveryTime* 毫秒后过期。

在更新它自己的路由条目之后，设备必须转发路由回复给目的地。在转发路由回复之前，路径消耗值必须更新。发送者必须通过查询其路由地址并提取发送者地址，找到路由回复目的地的下一跳。它必须使用这个下一跳地址来计算链路消耗，如 3.6.3.1 小节所述。这个消耗必须加到路由回复的路径消耗域上面。命令帧网络头的目标地址必须设置为下一跳地址，且帧必须使用 MCPS-DATA.request 原语单播给下一跳设备。MCPS-DATA.request 原语的 DstAddr 参数必须设置为路由发现表的下一跳地址。

如果 NIB 属性 *nwkcSymLink* 的值为 TRUE，且如果这样一个条目尚未存在，NWK 层必须在中继路由回复命令帧时也创建一个反向路由表条目。路由表条目的目标地址域的值必须对应路由回复命令帧的发出者地址域的值。状态域值必须为 ACTIVE。下一跳地址域值必须对应正被中继的路由回复命令的下一跳地址，在上段已经确定。如果反向路由表条目已经存在，如果必要的话，下一跳地址域必须被更新。

3.6.3.5.4 发起和处理一个路由记录命令帧

如果一个 ZigBee 路由器或 ZigBee 协调器的 NWK 层通过上层的 NLDE-DATA.request 发起了一个单播数据帧，且对应帧的目标地址的路由表条目的多对一域值为 TRUE，那么 NWK 层必须检查同一个路由表条目的路由记录请求域。如果路由记录请求域的值也是 TRUE，NWK 必须在发送数据帧之前单播一个路由记录命令给目标。

如果一个 ZigBee 路由器或 ZigBee 协调器的 NWK 层正在代表其终端子节点之一转发一个单播数据帧，且目标设备的路由表条目的多对一域值为 TRUE，那么设备必须在中继数据帧之前单播一个路由记录帧给目标。

有一个可选的优化，即路由器或协调器可以保持跟踪它的哪个终端设备从某个集中器设备收到了源路由数据帧，从而可以减少它代表其终端设备子节点发送给集中器的路由记录命令的数量。

收到路由记录命令的每个中继节点必须把它的网络地址附加在命令负载中，增加中继次数，并转发该信息。如果没有可用的下一跳，或如果交付给下一跳失败，或如果负载中没有足够的空间来存放网络地址，命令帧必须被丢弃，而不产生错误命令。

目的地在接收路由记录命令时，必须把路由存储在源路由表中。到信息源或中间节点的任何已有源路由必须被替换为新的路由信息。

3.6.3.6 一个路由发现表条目的过期

当创建一个路由发现表条目时，过期计时器必须设置为在 *nwkcRouteDiscoveryTime* 毫秒之后过期，对于路由表相应条目的 *GroupId* 标志为 TRUE 的那些条目，当收到一个路由回复导致下一跳被修改，且如果设备是路由回复的目的地，相应的路由发现表条目的过期时间域设置为在 *nwkcWaitBeforeValidation* 毫秒之后过期，如果它不是，设置为 *nwkcRouteDiscoveryTime* 毫秒。当计时器过期，设备必须从路由发现表中移除路由请求条目。如果设备是路由请求的发出者，且对应目标地址的路由表条目状态域值为 *VALIDATION_UNDERWAY*，那么设备必须传输一个信息以验证该路由；即一个缓冲挂起的路由发现或一个错误代码为 0x0a（验证路由）的路由错误命令。如果对应目标地址的路由表条目状态域值不是 *ACTIVE* 或 *VALIDATION_UNDERWAY*，且路由发现表中没有其它具有相同目标域值的条目，该路由表条目也必须被移除。

3.6.3.7 路由维护

一个设备 NWK 层必须为每个邻居维护一个失败计数器，这些邻居都有一个输出的链路，即它已经被请求发送数据帧。如果输出链路被归为一条失败链路，那么设备必须启动路由修复，如以下章节所述。执行者可以选择一个简单的失败计数方案来生成这个失败计数器的值，或他们可以使用一个更精确的时间窗口方案。注意重要的是不要太频繁地发起路由修复，由于修复操作可能在网络中太多，导致其它交通中断。退休链路的程序并停止保持跟踪他们的链路计数器在本规范的范围之外。

3.6.3.7.1 链路失败的情况

如果当一个设备正在使用多对一域为 TRUE 的一个路由表条目转发一个单播数据帧时，遇到一条失败的链路，必须产生一个网络状态命令帧，状态代码是 0x0c，表示多对一路由失败。网络状态命令帧的 NWK 头的目标地址域必须等于导致错误的帧的 NWK 头的目标地址域。网络状态命令帧使用 MCPS-DATA.request 原语单播给一个随机的路由器邻居。因为它是一个多对一路由，希望所有邻居都有到目的地的一个路由表条目。在接收网络状态命令帧时，如果没有目的地的路由表条目出现，或如果把网络状态命令帧交付给路由表条目的下一跳失败了，网络状态命令帧必须使用 MCPS-DATA.request 原语重新单播给一个随机的路由器邻居。NWK 头的半径计数器将限制网络状态命令帧中继的最大次数。目的地在接收网络状态命令帧时，必须使用 NLME-NWK-STATUS.indication 原语把它传递给上层。由于路由错误 NWK 层不会自动重新发现多对一路由。

如果设备正在使用正常单播路由转发一个单播帧时，遇到一个失败的链路，设备就给源设备送回一个网络状态命令帧，状态代码表示失败的原因（见表 3.42），并发出一个 NLME-NWK-STATUS.indication 给上层，状态代码表示失败的原因。

如果接收一个网络状态命令帧的路由器是命令的目标，其中命令帧负载的状态代码是 0x01 或 0x02，表示一个链路失败。如果存在的话，NWK 层将移除对应命令帧负载的目标地址的路由表条目，并使用带有同样状态代码的 NLME-NWK-STATUS.indication 把这个失败通知给上层。

如果接收一个网络状态命令帧的路由器是命令预定目标的终端设备的父节点，其中命令帧负载的状态代码域是 0x01 或 0x02，表示一个链路失败。如果存在的话，NWK 层将移除对应命令帧负载的目标地址的路由表条目。然后它像往常一样中继该帧到终端设备。

如果一个终端设备接收一个网络状态命令帧，NWK 层就使用 NLME-NWK-STATUS.indication 把这个失败通知给上层。

如果一个终端设备遇到其父节点的一条链路失败，终端设备就使用 NLME-NWK-STATUS.indication 原语通知上层，状态参数是 0x09，表示父节点链路失败（见表 3.42）。同样地，如果一个没有路由能力的 ZigBee 路由器，且 *nwkUseTreeRouting* 值为 TRUE，遇到其父节点的一条失败链路，它必须使用 NLME-NWK-STATUS.indication 原语通知上层，状态参数值为 0x09，表示父节点链路失败。

3.6.4 调度信标传输

信标调度在一个多跳拓扑中是必需的，以防止一个设备的信标帧和它相邻设备的信标帧或数据传输相冲突。当执行一个树拓扑时信标调度是必需的，但执行一个网状网络拓扑时不是必需的，因为在 ZigBee 网状网络中信标是不允许的。

3.6.4.1 调度方法

ZigBee 协调器必须为网络中每个设备确定信标顺序和超帧顺序（关于这些属性的更多信息参见[B1]）。因为多跳信标网络的目的之一是允许路由节点有机会休眠，以节省电力，信标顺序必须设置为远远大于超帧顺序。以这种方式设置属性可以在任何近邻调度每个设备超帧的活动部分，以便它们不会在一定时间重叠。换句话说，时间分成大约 (*macBeaconInterval* / *macSuperframeDuration*) 的非重叠时间槽，网络中每个设备超帧的活动部分必须占据这些非重叠时间槽之一。对于一个信标设备，图 3.46 展示了由此产生的帧结构的一个例子。

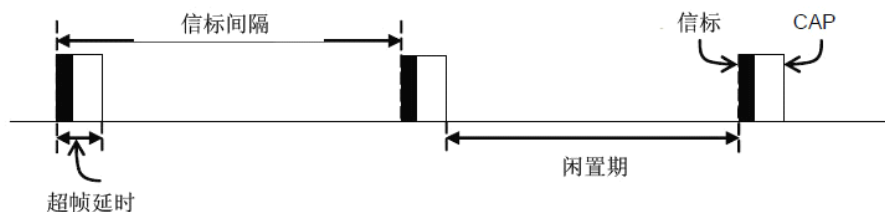


图 3.46 信标设备的典型的帧结构

一个设备的信标帧必须在它的非重叠时间槽启动的时候传输，传输时间是根据相应的父节点设备的信标传输时间来衡量的。这个时间偏移值必须包含在一个多跳信标网络每个设备的信标负载中（关于信标负载参数完整的列表，参见 3.6.7 小节）。因此收到一个信标帧的一个设备必须知道相邻设备和相邻设备父节点的信标传输时间，因为父节点的传输时间可以通过从信标帧的时间戳中减去时间偏移值来计算。接收设备必须在其相邻表中存储信标帧的本地时间戳，以及信标负载所含的偏移值。让一个设备知道何时它的父节点是活动的目的是通过减轻隐藏的节点问题，维护父子通信链路的完整性。换句话说，一个设备将永远不会在同一时间作为它的邻居的父节点传输。

在一个树网络中的通信必须使用父子链路沿着树寻找路由来完成。因为每个子节点跟踪它的父节点的信标，从一个父节点到其子节点的传输必须使用间接传输机制来完成。从一个子节点到其父节点的传输必须在父节点的 CAP 期间完成。关于通信程序的详细信息可以在 IEEE 802.15.4-2003 [B1]中找到。

一个希望加入网络的新设备必须遵守 3.6.1.4 小节所述的程序。在加入网络的步骤中，新设备必须根据在 MAC 扫描程序期间收集的信息，建立它的邻居表。使用这个信息，新设备必须为它的信标传输选择一个合适的时间和 CAP（它的超帧结构的的活动部分），这样它的超帧结构的的活动部分不会和任何邻居或任何邻居父节点的超帧活动部分重叠。如果在邻近区域没有可用的非重叠时间槽，设备不能传输信标，作为一个终端设备在网络上运行。如果有一个可用的非重叠信标，必须选择父节点和新设备信标帧之间的一个时间偏移值，且包含在新设备的信标负载中。选择信标传输时间，以避免在它的邻居和其父节点的超帧活动部分期间进行信标传输的任何算法都可以使用，只要将保证互操作性。

为了消除漂移，新设备必须跟踪它的父节点的信标，并调整它自己的信标传输时间，这样使得时间偏移值在两个不变的常量之间。因此网络中每个设备的信标帧基本上与 ZigBee 协调器的信标帧是同步的。图 3.47 说明了一个父节点和其子节点的活动超帧部分之间的关系。

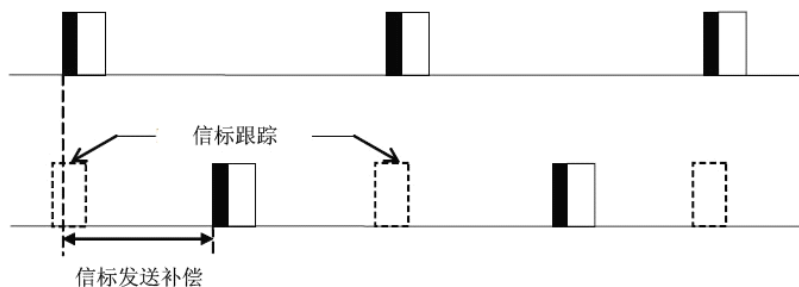


图 3.47 父子节点超帧定位关系

网络可支持的设备密度是和信标顺序的超帧顺序成反比的。每个设备较小的比例，较长的不活动时期，可以使更多的设备可以在同一邻近区域传输信标帧。建议一个树网络使用一个为 0 的超帧顺序，即当运行在 2.4 GHz 带宽时，超帧持续时间为 15.36 毫秒，信标顺序在 6 和 10 之间，即当在 2.4 GHz 带宽时，信标间隔在 0.98304 秒和 15.72864 秒之间。使用这些超帧和信标顺序值，网络中的设备典型的工作周期将在~2% 和~0.1%之间，不考虑频带。

3.6.5 广播通信

本小节指明了在一个 ZigBee 网络内，一个广播传输是如何完成的。这个机制用于广播所有的网络层数据帧。一个网络中的任何设备可以发起一个广播传输，到同一个网络中的其它一些设备。一个广播传输由本地 APS 子层实体通过设置 DstAddr 参数为表 3.54 所示的广播地址，使用 NLDE-DATA.request 原语发起，或由 NWK 层通过使用同样的这些广播地址构造一个输出的 NWK 头时发起。

表 3.54 广播地址

广播地址	目的地组
0xffff	个域网中所有设备
0xfffe	保留
0xfffd	<i>macRxOnWhenIdle</i> = TRUE

表 3.54 广播地址（续表）

广播地址	目的地组
0xfffc	所有路由器和协调器
0xfffb	仅对低功耗路由器
0xffff8 - 0xffffa	保留

为了传输一个广播 MSDU，一个 ZigBee 路由器或 ZigBee 协调器的 NWK 层给 MAC 子层发出一个 MCPS-DATA.request 原语，DstAddrMode 参数设置为 0x02（16 位网络地址），且 DstAddr 参数设置为 0xffff。对于一个 ZigBee 终端设备，广播帧的 MAC 目标地址必须等于终端设备父节点的 16 位网络地址。PANId 参数必须设置为 ZigBee 网络的 PANId。本规范不支持多个网络间的广播。广播传输不能使用 MAC 子层确认；而是使用一个消极确认机制。消极确认意味着每个 ZigBee 路由器和 ZigBee 协调器保持跟踪它的相邻设备是否成功中继了广播信息。通过设置 TxOptions 参数的确认传输标志为 FALSE 来禁用 MAC 子层确认。TxOptions 参数的所有其它标志必须根据网络配置来设置。

ZigBee 协调器、每个 ZigBee 路由器和那些 *macRxOnWhenIdle* 等于 TRUE 的 ZigBee 终端设备必须保持任何新广播事务的一个记录，新广播可以从本地发起或从一个相邻设备收到。这个记录叫做广播事务记录（BTR）并必须至少包含广播帧的序列号和源地址。广播事务记录存储在表 3.55 所示的 *nwkBroadcastTransactionTable*（BTT）中。

表 3.55 广播事务记录

域名称	大小	描述
Source Address	2 字节	广播发起者的 16 位网络地址。
Sequence Number	1 字节	发起者的广播信息的 NWK 层序列号。
Expiration Time	1 字节	倒计数的定时器，表示多少秒后这个条目过期；起始值是 <i>nwkNetworkBroadcastDeliveryTime</i> 。

当一个设备从一个相邻设备收到一个广播帧，它必须比较帧的目标地址和它的设备类型。如果目标地址不对应于表 3.54 所列的接收者设备类型，帧必须被丢弃。如果目标地址对应接收者的设备类型，设备将比较广播帧的序列号和源地址和它的 BTT 中的记录。

如果设备在它的 BTT 中有这个广播帧的一个 BTR，它可以更新 BTR，把相邻设备标记为已经中继了广播帧。然后它必须丢掉该帧。如果没有找到这样的记录，它必须在它的 BTT 中创建一个新的 BTR，并可以把相邻设备标记为已经中继了广播。然后 NWK 层必须使用 NLDE-DATA.indication 指示上层，一个新的广播帧已经被接收。如果半径域值大于 0 或如果设备不是一个 ZigBee 终端设备，它必须转发该帧。否则，它必须丢弃该帧。在转发之前，它必须等待一个随机的时间段，叫做广播抖动。这个时间段必须由 *nwkMaxBroadcastJitter* 属性的值界定。*macRxOnWhenIdle* 等于 FALSE 的 ZigBee 终端设备不能参与中继广播帧，也不需要为它们发出的广播帧维护一个 BTT。

如果在接收一个广播帧时，NWK 层发现 BTT 满了，且不包含过期条目，那么帧应被忽略。在这种情况下，帧不能被转发，也不能传给上层。

一个 ZigBee 协调器或 ZigBee 路由器，运行在一个非信标 ZigBee 网络上，一个之前的广播帧最多只能重传 *nwkMaxBroadcastRetries* 次。如果设备不支持消极确认，那么它必须把该帧转发 *nwkMaxBroadcastRetries* 次。如果设备支持消极确认，且它的任何相邻设备没有在 *nwkPassiveAckTimeout* 秒内中继广播帧，那么它必须继续转发该帧，最多可达 *nwkMaxBroadcastRetries* 次。

一个设备应在信息已创建，*nwkNetworkBroadcastDeliveryTime* 秒过去后修改一个 BTT 条目的状态。该条目应把状态修改为过期，因此如果一个收到新的广播需要空间时，可以重写该条目。

当 *macRxOnWhenIdle* MAC PIB 属性设置为 FALSE 的一个 ZigBee 路由器，收到一个广播传输，它必须使用一个不同于以上所述的程序来转发。它必须使用一个 MAC 层单播，毫不延迟地分别转发给它的每一个邻居，即 MCPS-DATA.request 原语的 DstAddr 参数设置为接收设备的地址，而不是广播地址。同样地，*macRxOnWhenIdle* MAC PIB 属性设置为 TRUE 的一个路由器或协调器，且它有 *macRxOnWhenIdle* MAC PIB 属性设置为 FALSE 的一个或多个邻居，在目标地址为 0xffff 表示广播给所有设备的情况下，除了执行之前章节说明的更常用的广播程序外，依次地像 MAC 层单播一样把广播帧转发给这些邻居的每一个。间接传输，如 IEEE 802.15.4-2003 [B1]所述，可以用于保证这些单播到达它们的目的地。

每个 ZigBee 路由器必须有能力在 NWK 层缓冲至少 1 个帧，以便于转发广播帧。

图 3.48 展示了一个设备和两个相邻设备之间的一个广播传输。

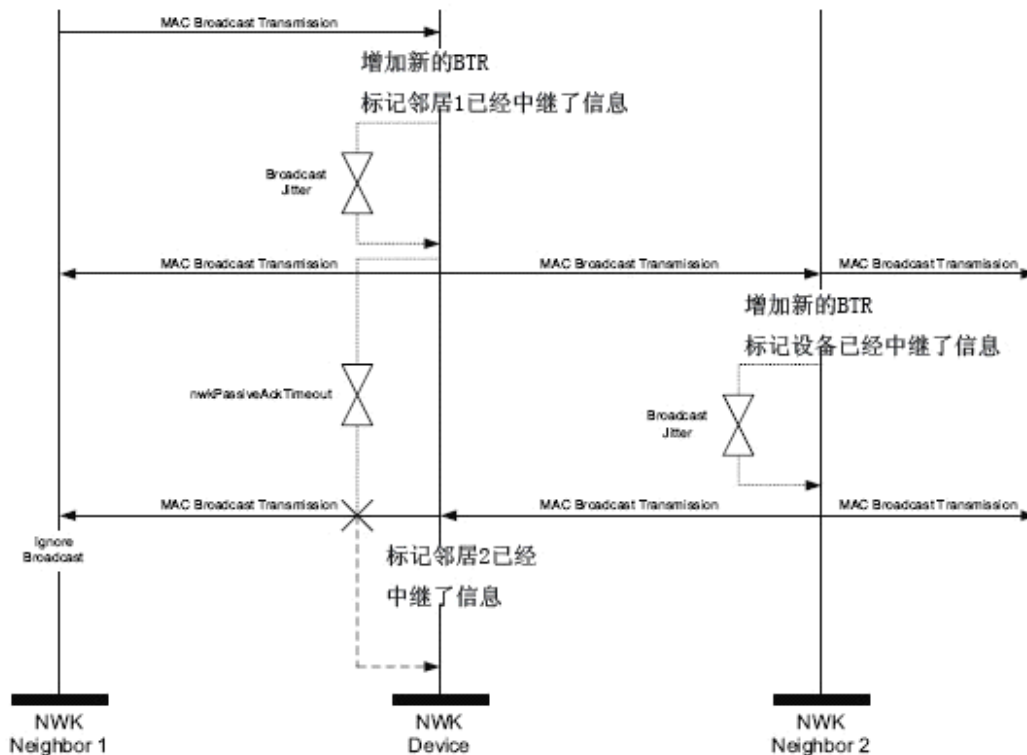


图 3.48 广播传输信息序列图

3.6.6 多播通信

本小节指明了一个 ZigBee 网络内多播传输是如何完成的。多播寻址使用 16 位多播组 ID 完成。多播组是所有已登记在同一个多播组 ID 下的节点的集合，物理上它们由不超过一个给定半径内的一跳的距离区分开，这个半径叫做 **MaxNonMemberRadius**。一个多播信息发送给一个特定的目标组，由它们的多播表中该组的 ID 所列的所有设备接收。只有数据帧是多播的——没有多播的 NWK 命令帧。

多播帧既可以由目标多播组的成员在网络中传播，也可以由非目标多播组成员在网络中传播。一个数据包可以用两种方式之一发送，这由数据包的一个模式标志指明，确定中继到下一跳的方式。如果原始信息由组的成员创建，就被视为处于“成员模式”，按广播方式中继。如果原始信息由不是组成员的设备创建，就被视为处于“非成员模式”，按单播给一个组成员的方式中继。一旦一个非成员信息到达目标组的任何成员，就立即转换为成员模式类型，不管谁下一个中继数据包。

多播帧可以由终端设备发起，但是不发送给 *macRxOnWhenIdle* 等于 FALSE 的设备。

3.6.6.1 组 ID 表

设备的 NWK 层可以维护一个组 ID 表，*nwkGroupIDTable*，可作为表 3.44 所示的 NIB 属性。如果 *nwkGroupIDTable* NIB 属性出现，那么应含有一组 16 位组标识符，其中设备是这个组的一个成员。

注意可选的 *nwkGroupIDTable* NIB 属性的功能和强制的 APS 组表（见表 2.18）有一定的重叠。如果这两个表设备都维护了，那么要使用 NWK 层多播作为接收组寻址帧的方法，必须保证 APS 组表中出现的每个 16 位组标识符也出现在 NWK 组表中。

还要注意从执行角度来看，跨越层复制组标识符列表是很浪费的，假定执行者将找到一种方法，结合 APS 和 NWK 组表，以避免浪费。

3.6.6.2 从上层接收一个多播帧

如果 NWK 层从其上层收到一个 NLDE-DATA.request，且多播控制域是 0x01，NWK 层必须确定 *nwkGroupIDTable* 中是否存在一个条目，其中组标识符域匹配帧的目标地址。如果找到一个匹配条目，NWK 层必须根据 3.6.6.2.1 小节所述的程序多播该帧。如果没有找到一个匹配条目，该帧必须使用 3.6.6.2.2 小节所述的程序，发起一个非成员模式多播。

3.6.6.2.1 发起一个成员模式多播

NWK 层必须设置多播控制域的多播模式子域为 0x01（成员模式）。如果 BTT 表满了，且不包含过期条目，信息将不被发送，NLDE 必须发出状态值为 BT_TABLE_FULL 的 NLDE-DATA.confirm 原语。如果 BTT 不满，或包含一个过期的 BTR，必须创建一个新的 BTR，其中本地节点作为源，且含有多播帧的序列号。然后信息将根据 3.6.6.3 小节最后一段所述的程序进行传输。

3.6.6.2.2 发起一个非成员模式多播

NWK 层必须设置多播控制域的多播模式子域为 0x00（非成员模式）。然后，NWK 层必须检查其路由表，寻找对应帧的目标 GroupID 的一个条目。如果有这样一个条目，NWK 层必须检查条目的状态域。如果状态是 ACTIVE，那么设备必须（重新）传输该帧。如果状态是 VALIDATION_UNDERWAY，那么状态必须改为 ACTIVE，设备必须根据 3.6.6.4 小节最后一段所述的程序传输该帧，且 NLDE 必须发出 NLDE-DATA.confirm 原语，状态值为从 MCPS-DATA.confirm 原语收到的状态值。如果没有对应帧的目标 GroupID 的一个条目，且 DiscoverRoute 参数的值为 0x00（抑制路由发现），帧必须被丢弃，NLDE 发出状态值为 ROUTE_DISCOVERY_FAILED 的 NLDE-DATA.confirm 原语。如果 DiscoverRoute 参数的值为 0x01（启用路由发现），且没有路由表条目对应帧的目标 GroupID，那么设备必须根据 3.6.3.5.1 小节所述，立即发起路由发现。帧可以有选择地缓冲等待路由发现。如果它没有被缓冲，帧必须被丢弃，NLDE 必须发出状态值为 FRAME_NOT_BUFFERED 的 NLDE-DATA.confirm 原语。

3.6.6.3 接收一个成员模式多播帧

当一个设备从一个相邻设备接收一个成员模式多播帧时，它必须比较多播帧的序列号和源地址和它 BTT 中的记录。如果设备的 BTT 中有这个多播帧的 BTR，它必须丢弃这个帧。如果没有找到记录，且 BTT 满了，并不包含过期的条目，它必须丢弃这个帧。如果没有找到记录，且 BTT 不满或包含一个过期的 BTR，它必须创建一个新的 BTR，并按以下段落所述继续处理信息。

当从一个邻居收到一个成员模式多播帧，并增加到了 BTT，然后 NWK 层必须确定 *nwkGroupIDTable* 中是否存在一个条目，其中组标识符域匹配帧的目标组 ID。如果找到一个匹配条目，信息将被传递给上层，多播控制域必须设置为 0x01（成员模式），NonmemberRadius 子域的值将设置为多播控制域的 MaxNonmemberRadius 子域的值，信息将按以下段落所述的进行传输。

如果没有找到一个匹配条目，NWK 层必须检查帧的多播 NonmemberRadius 域。如果多播 NonmemberRadius 域的值 0，信息以及新增加的 BTR 将被丢弃。否则，NonmemberRadius 子域将减少，如果它小于 0x07，帧将按以下段落所述传输。

每个成员模式多播信息必须传输 *nwkMaxBroadcastRetries* 次。对于不是在本地上发出的成员模式多播帧，最初的传输必须以一个随机的时间延迟，由 *nwkMaxBroadcastJitter* 属性的值界定。一个设备必须在某个成员模式多播信息重新转发期间，延迟 *nwkPassiveAckTimeout* 秒的时间。不像广播信息，多播信息没有消极确认。ZigBee 终端设备不能参与多播帧的中继。

要传输一个成员模式多播 MSDU，NWK 层给 MAC 子层发出一个 MCPS-DATA.request 原语，DstAddrMode 参数设置为 0x02（16 位网络地址），DstAddr 参数设置为 0xffff，这是广播的网络地址。PANId 参数必须设置为 ZigBee 网络的 PANId。成员模式多播传输不能使用 MAC 子层确认，或广播使用的消极确认。通过设置 TxOptions 参数的确认传输标志为 FALSE 来禁用 MAC 子层确认。TxOptions 参数的所有其它标志必须根据网络配置来设置。

3.6.6.4 接收一个非成员模式多播帧

当一个设备从一个相邻设备收到一个非成员模式多播帧时，那么 NWK 层必须确定 *nwkGroupIDTable* 中是否存在一个条目，它的组 ID 域匹配帧的目标组 ID。如果找到一个匹配条目，多播控制域必须设置为 0x01（成员模式），信息按照它作为一个成员模式多播接收的方式处理。如果没有找到匹配的 *nwkGroupIDTable* 条目，设备必须检查它的路由表，寻找对应帧的 GroupID 的一个条目。如果没有这样的路由表条目，信息必须被丢弃。如果有这样一个条目，NWK 层必须确定条目的状态域。如果状态是 ACTIVE，那么设备必须（重新）传输该帧。

如果状态是 `VALIDATION_UNDERWAY`，那么状态必须改为 `ACTIVE`，且设备必须（重新）传输该帧。要传输一个非成员模式多播 MSDU，NWK 层给 MAC 子层发出一个 `MCPS-DATA.request` 原语，`DstAddrMode` 参数设置为 `0x02`（16 位网络地址），且 `DstAddr` 参数设置为匹配路由表条目确定的下一跳。`PANId` 参数必须设置为 ZigBee 网络的 `PANId`。必须通过设置 `TxOptions` 参数的确认传输标志为 `TRUE` 来启用 MAC 子层确认。`TxOptions` 参数的所有其它标志必须根据网络配置来设置。

3.6.7 MAC 信标中的网络信息

本小节指明了 NWK 层如何使用一个 MAC 子层信标帧的信标负载，来把 NWK 层特定的信息传达给相邻设备。

信标负载所含的信息见表 3.56。这使得 NWK 层给执行网络发现的新设备提供了额外的信息，允许这些新设备更有效地选择一个网络和某个邻居加入。关于网络发现程序的详细描述见 3.6.1.4.1.1 小节。

表 3.56 网络层信息域

名称	类型	有效范围	描述
Protocol ID	Integer	0x00 – 0xff	该域指定了网络层使用的协议，为施行本规范应始终设置为 0，代表 ZigBee 协议；0xff 应该保留为了将来使用 ZigBee 联盟的协议
Stack profile	Integer	0x00 – 0xff	Zigbee 栈 profile 标识符
nwkcProtocolVersion	Integer	0x00 – 0xff	Zigbee 协议版本
Router capacity	Boolean	TRUE 或 FALSE	如果该设备是路由能力设备接收加入请求的能力，该值设置为 TRUE，否则设为 FALSE。

表 3.56 网络层信息域(续表)

名称	类型	有效范围	描述
Device depth	Integer	0x00 – 0x0f	该设备的树深度。0x00代表该设备是网络中的ZigBee协调器。
End device capacity	Boolean	TRUE或FALSE	如果设备是来自寻求加入网络的终端设备接收加入请求的能力，该值设置为TRUE，其他设置为FALSE。
<i>nwkExtendedPANID</i>	64-bit Extended address	0x0000000000000001–0xfffffffffffffffe	标识设备是个域网成员的全球唯一ID。默认情况下,这是格式化网络的ZigBee协调器的64位网络地址,但是其他值是允许的并且没有地址结构请求。
TxOffset	Integer	0x000000 – 0xffff	该值代表在设备的信标传输时间和它的父节点设备的信标传输时间之间的以符号形式的时间差；这可能会抵消减去来自计算父节点信标传输时间的设备的信标传输时间该参数只在当执行一个多跳信标网络时被包括。
<i>nwkUpdateId</i>	Integer	0x00 - 0xFF	该域反映NIB ^b 属性 <i>nwkUpdateId</i> 的值。

a. CCB #818

b. CCB #818

ZigBee 协调器的 NWK 层必须在网络形成以后立即更新信标负载。所有其它 ZigBee 设备必须在加入完成，以及任何时间网络配置（表 3.10 指明的任何参数）改变之后，立即更新它。信标负载使用 MLME-SET.request 原语写入到 MAC 子层 PIB。*macBeaconPayloadLength* 属性设置为信标负载的长度，表示信标负载的字节序列要写入到 *macBeaconPayload* 属性。

表示信标负载的字节序列的格式如图 3.48 所示。

位: 0-7	8-11	12-15	16-17	18	19-22	23	24-87	88-111	112-119
协议 ID	栈 profi le	nwkCP rotocol 版本	保留	路由 能力	设备深度	终端设 备能力	<i>Nwk Extended PANID</i>	Tx补偿 (可选的)	<i>nwkUpda teId^b</i>

- a. CCB #818
- b. CCB #818

图 3.49 MAC 子层信标负载的格式

3.6.8 永久性数据

在某个区域运行的设备可以手动或由维修人员编程复位,或可以由于某些原因意外复位,包括局部或网络范围停电,正常维护期间的电池更换,碰撞等等。以下信息应当在复位期间保存,以维持一个网络的运行:

- 设备的 PAN Id 和扩展 PAN Id
- 设备的 16 位网络地址
- 每个相关设备的 64 位 IEEE 地址和 16 位网络地址, 如果 *nwkAddrAlloc* 等于 0, 还有每个相关的路由器子节点。
- 对于终端设备, 父节点设备的 16 位网络地址。
- 使用的栈 profile
- 设备的深度

这些数据永久保存的方法超出了本规范的范围。

3.6.9 低功耗路由器 (LPR)

低功耗路由器定义为通过定期断电它们的无线设备, 靠电池运行多年的路由器。通过在加入阶段查看以下功能信息位域 (见表 3.47), LPR 应被视为高功率路由器 (HPR):

- 设备类型设置为 1
- 空闲时接收器开启设置为 FALSE

LPR 设备能够接收在网络中广播的网络命令帧。这可以通过为所有路由器和协调器，设置 NWK 头的目标地址为广播地址实现（见表 3.54）。

3.7 NWK 层状态值

网络（NWK）层配置原语通常包括一个报告请求状态的参数，该请求使用了确认。NWK 层状态参数的值在表 3.57 中。

表 3.57 网络层状态值

名称	值	描述
SUCCESS	0x00	已经执行成功的一个请求
INVALID_PARAMETER	0xc1	从高层发出的原语无效或者超出范围
INVALID_REQUEST	0xc2	考虑到网络层目前的状态，高层发送的请求原语无效或者不能执行
NOT_PERMITTED	0xc3	NLME-JOIN.request不被接受
STARTUP_FAILURE	0xc4	NLME-NETWORK-FORMATION.request启动网络失败
ALREADY_PRESENT	0xc5	产生NLME-DIRECT-JOIN.request原语的设备 的邻居表中已经存在有地址设备提供的 NLME-DIRECT-JOIN.request原语
SYNC_FAILURE	0xc6	使用表明NLME-SYNC.request在MAC层中已经失败
NEIGHBOR_TABLE_FULL	0xc7	NLME-JOIN-DIRECTLY.request已经失败，因为在邻居表中没有更多的空间
UNKNOWN_DEVICE	0xc8	NLME-LEAVE.request已经失败，因为产生原语的设备地址不在邻居表中的参数列表中
UNSUPPORTED_ATTRIBUTE	0xc9	NLME-GET.request或者 NLME-SET.request 以未知属性标识符的形式已发出
NO_NETWORKS	0xca	NLME-JOIN.request已发送到没有网络探测的环境中
Reserved	0xcb	

表 3.57 网络层状态值（续表）

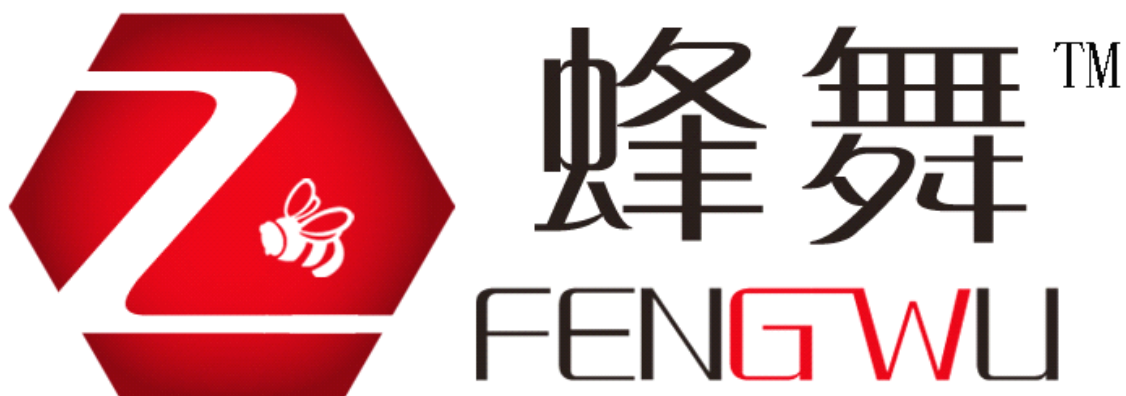
名称	值	描述
MAX_FRM_CNTR	0xcc	因为帧计数器达到最大值，所以输出帧安全处理失败
NO_KEY	0xcd	输出帧尝试安全处理且失败，因为对于处理没有有效的钥匙
BAD_CCM_OUTPUT	0xce	输出帧尝试安全处理且失败，因为安全设计产生一个错误的输出
NO_ROUTING_CAPACITY	0xcf	由于缺少路由表或者发现路由表能力，尝试发现路由失败
ROUTE_DISCOVERY_FAILED	0xd0	试图发现一个路由失败，由于缺乏路由能力
ROUTE_ERROR	0xd1	由于发送设备路由失败NLDE-DATA.request原语失败
BT_TABLE_FULL	0xd2	由于在BTT中没有足够的空间，尝试发送一个广播帧或成员模式多点传送失败
FRAME_NOT_BUFFERED	0xd3	由于没有足够的缓存，一个NLDE-DATA.request失败。 非会员模式组播帧被丢弃之前路由发现

版权声明

郑州新双恒信息技术有限公司拥有本译文的版权。本译文免费供大家一起学习交流。译文出现的错误及误差，您可以通过以下方式联系本公司，我们将非常感谢。欢迎在网络上转载本译文，但必须保证本译文的完整，否则本公司不承担任何后果。如果你在转载的时候，需要对本译文进行修改或重新编辑，请征得本公司的同意。谢谢！

网址：<http://www.zigbee-sh.cn>

QQ 群：83028739



第四章

安全服务规范

4.1 文件结构

本文件的余下部分详细描述了 ZigBee 协议栈内可以提供的各种安全服务。4.2 节给出了基本定义和参考资料。4.2.1 小节给出了安全服务的大致描述。本章讨论了总体安全架构，介绍了本架构下每个网络层提供的基本安全服务。4.2.2 到 4.2.3 小节分别描述了 ZigBee 联盟关于 NWK 层和 APS 层的安全规范。这些小节介绍了安全机制、原语，定义了用于安全目的的帧格式。4.5 节描述了 NWK 层和 APS 层共有的安全元素。4.6 节大概描述了可提供的安全功能的功能描述。最后，附录提供了执行所需要的技术资料 and 测试向量，以及 NWK 层和 APS 层所用的测试密钥机制和协议。

4.2 概述

ZigBee 提供的安全服务包括密钥建立、密钥传输、帧的保护和设备管理的方法。这些安全服务形成了 ZigBee 设备内实施安全策略的结构单元。本文件介绍了安全服务的规范 and 如何使用这些服务的功能性描述。

4.2.1 安全架构和设计

本节描述了安全架构，如果可用，本架构补充了安全规范 IEEE Std. 802.15.4 802 [B1]中已经存在的安全服务。

4.2.1.1 安全假定

ZigBee 安全架构提供的安全等级取决于对称密钥的保管，基础是采用的保护机制和执行合适的加密机制和相关安全策略。对安全架构的信任最终简化为对设置安全初值，安装密钥材料的信任和密钥材料的安全处理和储存的信任。

安全协议的执行，例如密钥的建立，是假定执行了完整的协议，没有遗漏任何步骤。假定随机数字发生器如期运作。而且，假定设备外部不能以不安全方式获得密钥。也就是说，除非密钥材料受到保护，例如在密钥传输中，一个设备不会有意地或者无意地传送它的密钥材料给其它设备。本假定的一个例外情况发生在当一个没有预先配置的设备加入网络中。在此情况下，单个密钥会未受保护地发送，因此会导致一个简短的易受攻击期，即密钥可能被任何设备获得。如果一个未经信任的设备能够获得一个密钥，这会导致一个关键的安全脆弱期。

这些假定中以下的附加说明适用于：由于网络设备的低成本，我们一般不能假定硬件是抗干扰的。因此，设备的物理访问可能会导致访问密钥材料、其它保密信息存取，以及安全软件和硬件。

由于成本限制，ZigBee 只好假定不同的应用使用逻辑上不分开相同无线设备（比如使用防火墙）。再者，从特定设备的角度看，甚至不可能去证实另一个设备上不同应用之间的加密分离（除非认证）——或者甚至在此通信协议栈的不同层之间——有没有被合适地执行。因此，我们必须假定使用相同无线设备的各自应用互相信任；这也就是说，没有加密任务分离。另外，低一些的层（比如 APS 层、NWK 层，或者 MAC 层）是任何一个应用都能完全可用的。这些假定形成设备的一个开放信任模式；通信协议栈的不同层和运行在单个设备上的所有应用互相信任。

小结：

- 提供的安全服务仅仅加密保护不同设备之间的接口。
- 相同设备上不同协议栈层之间接口的分离通过安全服务接入点的合理设计实现，是不需要加密的。

4.2.1.2 安全设计选择

一个设备的开放信任模式（4.2.1.1 小节已有表述）有深远影响。它允许在同一设备的设备不同层之间重复使用相同的密钥材料，且它允许在设备到设备的基础上实现端到端的安全，而不是在两个通信设备的两个特定层之间（或者甚至是两个应用程序之间）。

但是，我们还需要考虑的是，是否关注恶意网络设备未经允许通过网络在整个网络中传输帧的能力。

从这些意见可以得出以下架构设计选择：

- 首先，建立“最初产生帧的那一层负责最初加密它”这一原则。例如，如果一个 NWK 层命令帧需要保护，应当使用 NWK 层安全。
- 第二，如果要求保护服务不被盗窃（例如不被恶意网络设备盗窃），为所有帧使用 NWK 层安全，除了那些在一个路由器和一个新加入的设备（直到新加入的设备收到活动网络密钥）之间传递的帧。因此，只有已经加入了网络并顺利收到了活动网络密钥的设备才能将它的信息在网络上多跳之间通信。
- 第三，由于有开放信任模式，安全可以基于每个层重复使用密钥。例如，活动网络密钥可以用于加密 APS 层广播帧，或 NWK 层帧。密钥的重复使用有助于降低存储成本。
- 第四，端对端安全使能，这样只有源和目标设备可以存取它们的共享密钥。这限制了对那些含有重要信息的设备的信任要求。另外，它保证设备间的路由信息的实现可以独立于信任考虑（因此如果使用了安全，有助于注意力分离）。
- 第五，为了简化设备的互用性，一个特定网络里所有设备和一个设备里所有层使用的安全等级应该相同。如果一个应用程序的负载的安全需要高于一个给定网络提供的负载，该应用程序应该形成它自己更高安全等级的独立网络。

有几个策略，任一实际的执行都必须正确解决。应用 profile 应包括的策略有：

- 处理加密和解密数据包引起的错误状况。有些错误状况可能显示安全材料的不同步，或者指示不断攻击。
- 检测和处理计数器同步丢失和计数器溢出。
- 检测和处理密钥同步丢失。
- 如有需要，终止和周期性地更新密钥。

4.2.1.3 安全密钥

一个 ZigBee 设备网络之间的安全是基于一个“连接”密钥和一个“网络”密钥。APL 对等实体之间的单播通信是通过两个设备共享一个 128 位的连接密钥的方式加密的，而广播通信是通过网络中所有设备共享一个 128 位的网络密钥加密的。指定的接收者总是知道准确的安全安排；也就是说，接收者知道一个帧是用一个连接密钥还是一个网络密钥来加密的。

一个设备可以通过密钥传输、密钥建立，或者预安装（比如在工厂安装期间）来获得连接密钥。一个设备可以通过密钥传输，或者预安装来获得一个网络密钥。获得一个连接密钥的密钥建立技术（见 4.2.3.1 小节）基于一个“主”密钥。一个设备可以通过密钥传输，或者预安装来获得主密钥（为了建立相应的连接密钥）。最终，设备之间的安全取决于这些密钥的安全初始化和安装。

有两种不同类型的网络密钥：标准的和高安全的。类型控制了一个网络密钥如何分布；还可以控制如何初始化网络帧计数器。类型不影响信息的加密。

在一个安全的网络中有各种各样的安全服务。注意在不同的安全服务中要避免重复使用密钥，否则由于不希望的相互作用可能会导致安全漏洞。因此，这些不同的服务使用来自一个使用连接密钥的单向函数的一个密钥（在 4.5.3 小节中有说明）。使用不相关的关密钥保证了不同安全协议执行的逻辑分离。密钥载入密钥用于保护传输的主密钥和连接密钥；密钥传输密钥用于保护传输的网络密钥。活动网络密钥可以由 ZigBee NWK 和 APL 层使用。如此，所有这些层都必须能使用相同网络层和相关的输出的和输入帧计数器。连接密钥和主密钥只能用于 APS 子层。因此，APL 层应该只能使用连接密钥和主密钥。

4.2.1.4 ZigBee 安全架构

ZigBee 安全架构包括在协议栈两个层次的安全机制。NWK 层和 APS 层负责安全传输它们各自的帧。此外，APS 子层提供了建立和维护安全关系的服务。ZDO 设备对象（ZDO）管理一个设备的安全策略和安全配置。图 1.1 显示了一个完整的 ZigBee 协议栈概览。APS 层和 NWK 层提供的安全机制在本规范版本中有描述。

4.2.2 NWK 层安全

当 NWK 层产生的一个帧需要加密，或者更高层次的 NIB 中 *nwkSecureAllFrames* 属性为 TRUE，ZigBee 应该使用本规范 4.3.1 小节给出的帧保护机制，除非 NLDE-DATA.request 原语的 *SecurityEnable* 参数是 FALSE，明确指明禁止安全。NWK 层帧保护机制应该使用高级加密标准(AES) [B8]，并且使用附录 A 指定的 CCM*。适用于一个 NWK 帧的安全等级在 NIB 中 *nwkSecurityLevel* 属性中给出。上层通过建立活动和备用的网络密钥，以及决定使用那个安全等级来管理 NWK 层安全。

图 4.1 展示了一个 NWK 帧包含的安全域的一个例子。

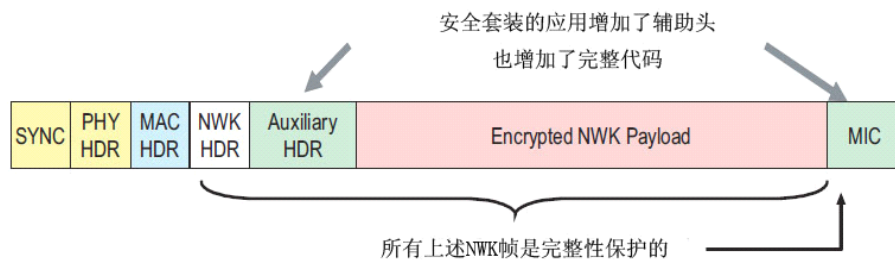


图 4.1 ZigBee 的安全帧的 NWK 层

4.2.3 APL 层安全

当 APL 层产生的一个帧需要加密，APS 子层应该处理安全。APL 层的帧保护机制在本规范的 4.4.1 小节有说明。APS 层允许帧安全基于连接密钥或者网络密钥。图 4.2 展示了一个 APL 帧包含的安全域的一个例子。APS 层还负责为应用和 ZDO 提供密钥建立、密钥传输和设备管理服务。

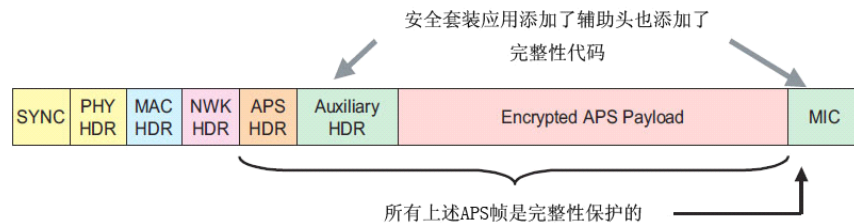


图 4.2 ZigBee 的安全帧的 APS 层

4.2.3.1 密钥建立

APS 子层的密钥建立服务提供了一个 ZigBee 设备可以得到和另一个 ZigBee 设备共享的密钥（所谓的“连接密钥”，见 4.2.1.3 小节）的机制。密钥建立涉及两个实体，一个发起设备和一个响应设备，这必须先通过信任认证步骤。

信任信息（比如一个主密钥）提供了建立一个连接密钥的起点，可以在频带内或者频带外配置。一旦配置了信任信息，一个密钥建立协议包括三个概念的步骤：

- 交换短暂数据
- 使用短暂数据来获得连接密钥
- 确认这个连接密钥被正确地计算

在对称密钥密钥建立（SKKE）协议中，一个发起设备建立与使用主密钥的一个响应设备。这个主密钥，可以在制造过程中预装，也可以由一个信任中心安装（例如，发起设备，响应设备或者作为一个信任中心的第三方设备），或者可以基于一个用户输入的数据（例如：PIN、口令或密钥）。应该支持主密钥的秘密性和权威性来获得信任基础。

4.2.3.2 密钥传输

密钥传输服务提供了加密和无加密方式传送一个密钥到另一个设备或其它设备。加密的密钥传输命令提供了一种方式来传输来自一个密钥源（例如，信任中心）的一个主密钥、连接密钥，或网络密钥到其它设备。无加密的密钥传输命令提供了一种方式在一个设备上加载初始密钥。这个命令不加密保护正在加载的密钥。在这种情况下，传输的密钥的安全可用非加密方式实现；例如，通过保证秘密性和真实性的频带外信道传输命令。

4.2.3.3 更新设备

更新设备的服务为一个设备（例如，一个路由器）提供了一种安全方式通知第二个设备（例如，一个信任中心），第三个设备已经改变了状态，必须更新（例如，该设备加入或离开了网络）。这一机制的信任中心维护一个准确的名单，列出活跃的网络设备。

4.2.3.4 移除设备

移除设备服务为一个设备（例如，一个信任中心）提供了一个安全的方式，可以通知其它设备（例如一个路由器），它的一个子节点应从网络中移除。例如，移除设备服务可以用于从一个网络中移除没有满足信任中心对网络设备安全要求的一个设备。

4.2.3.5 请求密钥

请求密钥服务为一个设备提供了一种安全的方式，请求来自于另一个设备（例如它的信任中心）的活动网络密钥，或端到端应用主密钥。

4.2.3.6 更换密钥

更换密钥服务为一个设备（例如，一个信任中心）提供了一个安全的方式，通知另一个设备它应该更换到一个不同的活动网络密钥。

4.2.3.7 实体认证

实体认证服务为一个设备提供了一个安全的方式，当同时提供基于一个共享密钥的认证时信息能和另一个设备同步。

4.2.3.8 允许配置表

允许配置表指明哪个设备有权执行某种类型的命令，并决定每种情况下是否需要连接密钥安全。

4.2.4 信任中心作用

为了安全起见，ZigBee 定义了“信任中心”的作用。信任中心是一个网络内设备可以信任的设备，为了网络和端到端应用的配置管理分发密钥。该网络所有成员只能承认一个信任中心，并且每一个安全的网络只能有一个信任中心存在。

在高安全等级中，商业应用（见 4.6.2.1 小节），一个设备可以预先装载信任中心地址和初始主密钥。或者，如果应用可以忍受一小会攻击，主密钥可以通过频带内未加密的密钥传输发送。如果没有预先装载，一个设备的信任中心默认是 ZigBee 协调器或协调器指定的一个设备。

在低安全等级中，家居应用（见 4.6.2.2 小节），一个设备使用当前网络密钥和信任中心安全通信，可以通过预先配置或者通过频带内未加密的密钥传输发送。

为了信任管理，一个设备应该接受通过未加密的密钥传输来自其信任中心的一个初始主密钥或者活动网络密钥。为了网络管理，一个设备只能从它的信任中心接受一个初始的活动网络密钥和最新的网络密钥。为了配置，一个设备只能从其信任中心接受主密钥和连接密钥，用于建立两个设备之间端到端的安全。除了初始主密钥和网络密钥，额外的连接密钥、主密钥，以及网络密钥只能在它们来自一个设备的信任中心时，通过加密的密钥传输接受。

4.3 NWK 层安全

NWK 层负责安全地传输输出帧和安全接收输入帧所需的处理步骤。上层通过建立适当的密钥和帧计数器控制安全处理操作，并建立使用哪种安全级别。

4.3.1 帧安全

安全处理输出和输入的 NWK 帧涉及的步骤分别在 4.3.1.1 和 4.3.1.2 小节中有说明。

4.3.1.1 输出帧的安全处理

如果 NWK 层有一个帧，包括一个头 *NwkHeader* 和负载 *payload*，需要安全保护并且 *nwkSecurityLevel* > 0，如果是一个 NWK 数据帧，NLDE-DATA.request 的 *SecurityEnabled* 参数值为 TRUE，应按照如下步骤实施安全：

- 1 从 NIB 属性 *nwkSecurityMaterialSet* 中获得 *nwkActiveKeySeqNumber*，使用它去检索活动网络密钥 *key*、输出帧计数器 *OutgoingFrameCounter* 和密钥序列号 *KeySeqNumber*。从 *nwkSecurityLevel* 获得安全等级。如果输出帧计数器值为 4 字节表示的整数 $2^{32}-1$ ，或如果密钥不能获得，安全处理失败，不能对本帧进行进一步安全处理。
- 2 建立辅助头 *AuxiliaryHeader*（见 4.5.1 小节）：
 - a 安全控制域应该如下设置：
 - i 安全等级子域应该设置为步骤 1 中获得的安全等级。
 - ii 密钥标识符子域应该设置为 ‘01’（即活动网络密钥）。
 - iii 扩展的临时子域应该设置为 1。
 - b 设置源地址域为本地设备的 64 位扩展地址。
 - c 帧计数器域设置为步骤 1 中的输出帧计数器读数。
 - d 密钥序列号域应该设置为步骤 1 获得的序列号。
- 3 执行 CCM*模式加密和认证操作，见附录 A，有如下实例：
 - a 参数 *M* 从表 4.38 对应于第 1 步的安全级别中获得；
 - b 位字符串 *Key* 应从第 1 步获得；
 - c 该临时 *N* 应是 13 字节字符串，使用第 a 步安全控制域、第 d 步获得的帧计数器域、第 c 步获得的源地址域构成（见 4.5.2.2 小节）；
 - d 如果安全等级需要加密，字节字符串 *a* 应该是字符串 *NwkHeader* || *AuxiliaryHeader*，字节字符串 *m* 应该是字符串 *Payload*。否则，八字节字符串 *a* 是字符串 *NwkHeader* || *AuxiliaryHeader* || *Payload*，字节字符串 *m* 应该是长度为零的字符串。

- 4 如果步骤 3 调用的 CCM*模式输出“无效”，安全处理失败，本帧不能进一步进行安全处理。
- 5 让 c 成为步骤 3 的输出结果。如果安全等级需要加密，加密的输出帧为 $NwkHeader \parallel AuxiliaryHeader \parallel c$ ，否则，加密的输出帧为 $NwkHeader \parallel AuxiliaryHeader \parallel Payload \parallel c$ 。
- 6 如果加密的输出帧大于 $aMaxMACFrameSize$ (见 IEEE Std. 802.15.4 802 [B1])，安全处理失败，本帧不能进一步进行安全处理。
- 7 第一步获得的输出帧计数器应该增加一，并且储存在网络安全材料描述符的 *OutgoingFrameCounter* 元素中，可由 NIB *nwkActiveKeySeqNumber* 引用；即这个和保护帧使用的密钥相联系的输出帧计数器值被更新。
- 8 安全控制域的安全等级子域重写为 3 位的全零字符串 ‘000’。

4.3.1.2 输入帧的安全处理

如果 NWK 层收到一个加密帧（由头 *NwkHeader*、辅助头 *AuxiliaryHeader* 和负载 *SecuredPayload* 组成），如 NWK 头帧控制域的安全子域指明，应该进行如下安全处理：

- 1 从 NIB *nwkSecurityLevel* 属性中决定安全等级。重写 3 位的 *AuxiliaryHeader* 的安全控制域的安全等级子域字符串为这个值。从辅助头 *AuxiliaryHeader* 中（见 4.5.1 小节）决定序列号 *SequenceNumber*，发送地址 *SenderAddress* 和接收帧计数器 *ReceivedFrameCount*。如果 *ReceivedFrameCount* 值为 4 字节表示的整数 $2^{32}-1$ ，安全处理会给上层一个状态为“坏帧计数器”的失败³⁰，本帧不能进一步进行安全处理。

30. CCB #765

- 2 通过匹配 *nwkSecurityMaterialSet* 和 NIB 属性 *nwkSecurityMaterialSet* 中的任何密钥的序列号, 获取合适的安全材料(包括密钥和其它属性)。如果对应于 *SenderAddress* 的邻居表节的关系域值为 0x01 (子节点), 该域必须设置为 0x05 (未经验证的子节点)。如果不能获得安全材料, 安全处理会指示上层一个状态为“帧安全失败”的失败, 本帧不能进一步进行安全处理。³¹
- 3 如果有一个输入帧计数 *FrameCount* 对应于步骤 2 获得的安全材料的 *SenderAddress*, 且如果 *ReceivedFrameCount* 小于 *FrameCount*, 安全处理会指示上层一个状态为“坏帧计数器”的失败, 本帧不能进一步进行安全处理。³²
- 4 执行 CCM*模式加密和认证检查操作, 见 A.2 小节, 有如下实例:
 - a 参数 *M* 应从表 4.38 对应于第 1 步的安全级别获得。
 - b 位字符串 *Key* 应从第 2 步获得。
 - c 临时 *N* 应是 13 字节字符串, 使用从 *AuxiliaryHeader* 中获得的安全控制域、帧计数器域和源地址域(见 4.5.2.2 小节)构成的。注意安全控制域的安全等级子域已经在步骤 1 中被重写, 现在包含了从 NIB *nwkSecurityLevel* 属性中确定的值。
 - d 解析字节字符串 *SecuredPayload* 为 *Payload₁* || *Payload₂*, 最右边的字符串 *Payload₂* 是一个 *M* 字节的字符串。如果此操作失败, 安全处理会指示上层一个状态为“帧安全失败”的失败, 本帧不能进一步进行安全处理³³。
 - e 如果安全级别需要加密, 八位字节字符串 *a* 应是字符串 *NwkHeader* || *AuxiliaryHeader*, 八位字节字符串 *c* 应是字符串 *SecuredPayload*。否则, 八位字节字符串 *a* 是字符串 *NwkHeader* || *AuxiliaryHeader* || *Payload₁*, 八位字符串 *c* 应是 *Payload₂*。
- 5 返回 CCM*操作的结果:
 - a 如果步骤 4 调用的 CCM*模式输出“无效”, 安全处理会指示上层一个状态为“帧安全失败”的失败, 本帧不能进一步进行安全处理。³⁴

31. CCB #673

32. CCB #765

33. CCB #673

34. CCB #673

- b 让 m 成为步骤 4 的输出结果。如果安全等级需要加密，设置字节字符串 $UnsecuredNwkFrame$ 为字符串 $NwkHeader \parallel m$ ，否则，设置字节字符串 $UnsecuredNwkFrame$ 为 $NwkHeader \parallel Payload$ 。
- 6 设置 $FrameCount$ 为 $(ReceivedFrameCount + 1)$ ，在 NIB 中保存 $FrameCount$ 和 $SenderAddress$ 。 $UnsecuredNwkFrame$ 现在表示未加密的收到的网络帧，安全处理成功。为了不会导致存储帧计数和地址信息超过可用内存，为 NWK 层安全所需的输入帧计数器分配的内存用 $M \times N$ 来限制，其中 M 和 N 分别表示 $nwkSecurityMaterialSet$ 和 $nwkNeighborTable$ 。
- 7 如果收到的帧的序列号属于 $nwkSecurityMaterialSet$ 中的一个新条目，那么 $nwkActiveKeySeqNumber$ 应该设置为收到的序列号。
- 8 如果在 NIB 中 $nwkNeighborTable$ 有一个条目，它的扩展地址和 $SenderAddress$ 匹配，关系域值为 0x05（未经认证的子节点），那么设置该条目的关系域值为 0x01（子节点）。

4.3.2 加密的 NPDU 帧

NWK 层的帧格式（见 3.3.1 小节）包括一个 NWK 头和 NWK 负载域。NWK 头包括帧控制和路由域。当安全适用于一个 NPDU 帧，在 NWK 帧控制域安全位应设置为 1，表示辅助帧头存在。辅助帧头的格式在 4.5.1 小节。加密的 NWK 层帧格式在图 4.3 有显示。辅助帧头位于 NWK 头和负载域之间。

字节数： 可变的	14	可变的	
原始的 NWK 头([B3], 7.1节)	辅 助 帧 头	加密负载	加密信息完整性代码(MIC)
		安全帧负载 = CCM*的输出	
完整的NWK头		加密的网络负载	

图 4.3 加密的网络层帧格式

4.3.3 安全相关的 NIB 属性

NWK PIB 包含了管理 NWK 层安全所需的属性。分别使用 NLME-GET.Request 和 NLME-SET.request 原语，每个属性可以读写。安全相关的 NWK PIB 属性在表格 4.1-4.3 中有显示。

表 4.1 NIB 安全属性

属性	标识符	类型	范围	描述	默认值
<i>nwkSecurityLevel</i>	0xa0	Octet	0x00-07	接收和发送帧的安全级别；允许的安全级别标识符列在表4.30中	0x05
<i>nwkSecurity-MaterialSet</i>	0xa1	0,1, 或2的集合。网络安全的材料描述(见表4.2)	可变的	一组网络安全性的材料描述维护主动和备用的网络密钥	-
<i>nwkActiveKeySeqNumber</i>	0xa2	Octet	0x00-0xFF	<i>nwkSecurityMaterialSet</i> 活动中的动网络密钥的序列号	0x00

表 4.1 NIB 安全属性(续表)

属性	标识符	类型	范围	描述	默认值
<i>nwkAllFresh</i>	0xa3	Boolean	TRUE FALSE	代表当内存传入帧计数溢出时传入的网络帧必须进行全新的检查	TRUE
nwkSecure All Frames	0xa5	Boolean	TRUE FALSE	这表明，如果安全应适用于接收和发出的NWK数据帧；如果设置为0x01, 安全处理应该适用于所有接收和发送的帧，除了当前设备已有的定义, 帧控制域设置为0的安全子域的数据帧;如果该属性值为 0x01 网络层不能转发拥有帧控制域设备为0的安全子域的帧；NLDE-DATA.request原语的SecurityEnable参数会覆盖该属性的设置。	TRUE

表 4.2 网络安全材料描述符的元素

名称	类型	范围	描述	默认
KeySeqNumber	Octet	0x00-0xFF	通过信任中心分配给网络密钥的序列号并且用于区分网络用于网络密钥更新目的的密钥,和传入帧安全操作	00
OutgoingFrameCounter	4字节的有序集合	0x00000000-0xFFFFFFFF	用于发送帧的原始帧计数器	0x00000000
IncomingFrameCounterSet	见表4.3中接收帧计数器描述符值的设置	可变的	接收帧计数器的值和当前设备地址的设置	空集
Key	16字节的有序集合	-	密钥的实际值	-
KeyType	Octet	0x00 - 0xff	密钥的类型。 0x01 =标准 0x05 =高级 所有其它值保留。	0x01

表 4.3 输入帧计数器描述符的元素

名称	类型	范围	描述	默认
SenderAddress	设备地址	任何有效的 64 位地址	扩展设备地址	设备具体
IncomingFrameCounter	4字节的有序集	0x00000000-0xFFFFFFFF	输入帧的计数器,用于输入的帧。	0x00000000

4.4 APS 层安全

APS 层负责安全地转发输出帧，安全地接收输入帧，以及安全地建立并管理对称密钥所需的的处理步骤。上层通过发出原语给 APS 层控制对称密钥的管理。

表 4.4 列出了密钥管理和维护可用的原语。上层也确定保护输出帧时使用哪种安全级别。

表 4.4 APS 层安全原语

APSME 安全原语	请求	确认	指示	响应	描述
APSME-ESTABLISH-KEY	4.5.2.1 小节	4.4.2.2 小节	4.4.2.3 小节	4.4.2.4 小节	使用SKKE方法的其它 ZigBee设备的建立连接密钥
APSME-TRANSPORT-KEY	4.4.3.1 小节		4.4.3.2 小节		从一个设备传输到另一个设备传输安全材料
APSME-UPDATE-DEVICE	4.4.4.1 小节		4.4.4.2 小节		当一个新设备加入时或者一个存在设备已经离开网络时通知信任中心
APSME-REMOVE-DEVICE	4.4.5.1 小节		4.4.5.2 小节		用于信任中心通知路由器，路由器的一个子设备应该从网络中移除。
APSME-REQUEST-KEY	4.4.6.1 小节		4.4.6.2 小节		用于通过设备请求信任中心发送一个应用程序主密钥或者当前网络密钥
APSME-SWITCH-KEY	4.4.7.1 小节		4.4.7.2 小节		用于通过信任中心告诉设备切换到一个新的网络密钥
APSME-AUTHENTICATE	4.4.8.1 小节	4.4.8.2 小节	4.4.8.3 小节	-	由两个设备使用，互相验证对方。

4.4.1 帧安全

安全处理输出和输入 APS 帧的详细步骤在 4.4.1.1 和 4.4.1.2 小节中有描述。

4.4.1.1 输出帧的安全处理

如果 APS 层有一个帧，由头 *ApsHeader* 和负载 *Payload* 组成，需要安全保护，且 *nwkSecurityLevel* > 0，应该适用如下安全步骤：

- 1 使用如下步骤获得安全材料和密钥标识符 *KeyIdentifier*。如果安全材料和密钥标识符不能决定，那么安全处理失败，本帧不能进一步进行安全处理。
 - a 如果该帧是一个 APSDE-DATA.request 原语的结果：
 - i 如果 TxOptions 参数规定活动 NWK 密钥需要用于加密数据帧，那么应该从 NIB 用 *nwkActiveKeySeqNumber* 获得安全材料，以从 NIB 属性 *nwkSecurityMaterialSet* 中检索活动网络密钥、输出帧计数器和序列号。*KeyIdentifier* 应该设置为 ‘01’（即活动网络密钥）。
 - ii 否则，和输出帧目标地址相关的安全材料应该从 AIB *apsDeviceKeyPairSet* 属性中获得。*KeyIdentifier* 应该设置为 ‘00’（即一个数据密钥）。
 - b 如果该帧是一个需要加密的 APS 命令的结果：
 - i 第一，尝试从 AIB *apsDeviceKeyPairSet* 属性中检索和输出帧目标地址相关的安全材料，除了传输密钥命令，*KeyIdentifier* 都应该设置为 ‘00’（即一个数据密钥）。如果是传输密钥命令，当传输的是一个网络密钥，*KeyIdentifier* 应该设置为 ‘02’（即密钥传输密钥），当传输一个应用连接密钥、应用主密钥，或者信任中心主密钥，应该设置为 ‘03’（即密钥装载密钥）。密钥传输和密钥装载密钥见 4.5.3 小节描述。
 - ii 如果第一个尝试失败，那么安全材料应该使用 *nwkActiveKeySeqNumber* 属性从 NIB 中获得，来检索活动网络密钥、输出帧计数器，且序列号从 *nwkSecurityMaterialSet* 属性中获得。*KeyIdentifier* 应该设置为 ‘01’（即活动网络密钥）。

- 2 如果 *KeyIdentifier* 等于 01（即活动网络密钥），APS 层应该先证实 NWK 层是否也申请安全。如果 NWK 层也申请安全，那么 APS 层就不能申请任何安全。APS 层可以通过证实 NIB *nwkSecureAllFrames* 属性值为 TRUE，以及 *nwkSecurityLevel* 属性值为非零，来确定 NWK 层也在申请安全。
- 3 提取从步骤 1 获得的安全材料的输出的帧计数器（且如果 *KeyIdentifier* 等于 01，密钥序列号）。如果输出帧计数器值为 4 字节代表的整数 $2^{32}-1$ ，或者如果密钥无法获得，安全处理失败，本帧不能进一步进行安全处理。
- 4 从 NIB 的 *nwkSecurityLevel* 属性中获得安全等级。
- 5 建立辅助头 *AuxiliaryHeader*（见 4.5.1 小节）。安全控制域应该如下设置：
 - a 安全等级子域应该设置为步骤 4 中获得的安全等级。
 - i 密钥标识符子域应该设置为 *KeyIdentifier*。
 - ii 扩展的临时子域应该设置为 0。
 - b 帧计数器域应该设置为步骤 3 中的帧计数器。
 - c 如果 *KeyIdentifier* 是 01，密钥序列号域应该存在，且设置为步骤 3 获得的密钥序列号。否则密钥序列号域不存在。
- 6 执行 CCM*模式加密和认证检查操作，见 A.2 小节，有如下实例：
 - a 参数 *M* 应从对应于第 3 步的安全级别的表 4.38 获得。
 - b 位字符串 *Key* 从第 1 步获得。
 - c 该临时 *N* 应是 13 字节字符串，使用第 5 步获得的安全控制域和帧计数器域，以及本地设备中获得的 64 位扩展地址（见 4.5.2.2 小节）构成。
 - d 如果安全等级需要加密，字节字符串 *a* 应该是字符串 *ApsHeader* || *AuxiliaryHeader*，字节字符串 *M* 应该是 *Payload*。否则，八位字节字符串 *a* 是字符串 *ApsHeader* || *AuxiliaryHeader* || *Payload*，字节字符串 *M* 应该是长度为零的字符串。
- 7 如果步骤 6 引用的 CCM*模式输出“无效”，安全处理失败，本帧不能进一步进行安全处理。

- 8 让 c 成为步骤 6 的输出结果。如果安全等级需要加密, 加密的输出帧为 $ApsHeader \parallel AuxiliaryHeader \parallel c$, 否则, 加密的输出帧为 $ApsHeader \parallel AuxiliaryHeader \parallel Payload \parallel c$ 。
- 9 如果 MSDU 中加密的输出帧比 $aMaxMACFrameSize$ 大 (见 IEEE Std. 802.15.4 802 [B1]), 安全处理会失败, 本帧不能进一步进行安全处理。
- 10 第 3 步的输出帧计数器应该增加, 并且储存在 NIB、AIB 和 MAC PIB 的合适位置, 和用来保护输出帧的密钥对应。
- 11 重写安全控制域的安全等级子域为 3 位的全零字符串 ‘000’。

4.4.1.2 输入帧的安全处理

如果 APS 层收到一个加密的帧 (由一个头 $ApsHeader$, 辅助头 $AuxiliaryHeader$ 和负载 $SecuredPayload$ 组成), 如 APS 头帧控制域的安全子域所指示, 它应按照如下步骤执行安全处理:

- 1 设置序列号 $SequenceNumber$, 密钥标识符 $KeyIdentifier$ 和来自辅助头 $AuxiliaryHeader$ 的帧计数器值 $ReceivedFrameCounter$ 。如果 $ReceivedFrameCounter$ 是 4 字节表示的整数 $2^{32}-1$, 安全处理会失败, 本帧不能进一步进行安全处理。
- 2 从 NIB 地址映射表中确定源地址 $SourceAddress$, 使用 APS 帧的源地址作为索引。如果源地址是不完整的或者不可用的, 安全处理会失败, 本帧不能进一步进行安全处理。
- 3 通过以下方式获得合适的安全材料。如果安全材料不能获得, 安全处理会失败, 本帧不能进一步进行安全处理。
 - a 如果 $KeyIdentifier$ 是 ‘00’ (即一个数据密钥), 和输入帧 $SourceAddress$ 相关的安全材料应该从 AIB $apsDeviceKeyPairSet$ 属性中获得。
 - b 如果 $KeyIdentifier$ 是 ‘01’ (即一个网络密钥), 安全材料应该通过匹配 $SequenceNumber$ 和 NIB 中 $nwkSecurityMaterialSet$ 属性的任一密钥的序列号获得。
 - c 如果 $KeyIdentifier$ 是 ‘02’ (即一个密钥传输密钥), 和输入帧 $SourceAddress$ 相关的安全材料应该从 AIB $apsDeviceKeyPairSet$ 属性获得; 这个操作的密钥应该从 4.5.3 小节关于密钥装载密钥描述的安全材料中获得。

- d 如果 *KeyIdentifier* 是 ‘03’ (即一个密钥装载密钥), 和输入帧 *SourceAddress* 相关的安全材料应该从 AIB *apsDeviceKeyPairSet* 属性获得。这个操作的密钥应该从 4.5.3 小节关于密钥装载密钥描述的安全材料中获得。
- 4 如果有一个输入的帧计数 *FrameCount* 和从第 3 步中获得的安全材料 *SourceAddress* 相关, 并且如果 *ReceivedFrameCount* 小于 *FrameCount*, 安全处理会失败, 本帧不能进一步进行安全处理。
- 5 设置安全等级 *SecLevel* 如下。如果 *ApsHeader* 帧控制域的帧类型子域指示了一个 APS 数据帧, 那么 *SecLevel* 应该设置为 NIB 的 *nwkSecurityLevel* 属性。否则, *SecLevel* 应该设置为 7(ENC-MIC-128)。否则, *AuxiliaryHeader* 中的安全控制域的安全等级子域值为 *SecLevel*。
- 6 执行 CCM*模式加密和认证检查操作, 见附录 A.3, 有如下实例:
 - a 参数 *M* 应从对应于第 5 步的安全级别的表 4.38 中获得。
 - b 位字符串 *Key* 从第 3 步获得。
 - c 该临时 *N* 应是 13 字节字符串, 使用 *AuxiliaryHeader* 获得的安全控制域, 帧计数器域, 和第 2 步获得的 *SourceAddress* (见 4.5.2.2 小节) 构成。
 - d 解析字节字符串 *SecuredPayload* 为 *Payload₁* || *Payload₂*, 最右边字符串 *Payload₂* 是一个 *M* 字节的字符串。如果此操作失败, 安全处理会失败, 本帧不能进一步进行安全处理。
 - e 如果安全等级需要加密, 字节字符串 *a* 应该是 *ApsHeader* || *AuxiliaryHeader*, 字节字符串 *c* 应该是 *SecuredPayload*。否则, 八位字节字符串 *a* 是字符串 *ApsHeader* || *AuxiliaryHeader* || *Payload₁*, 字节字符串 *c* 应该是字符串 *Payload₂*。
- 7 返回 CCM*操作的结果:
 - a 如果步骤 6 调用的 CCM*模式输出“无效”, 安全处理会失败, 本帧不能进一步进行安全处理。

- b 让 m 成为步骤 6 的输出结果。如果安全等级需要加密，设置字节字符串 *UnsecuredApsFrame* 为 *ApsHeader*|| m 。否则，设置字节字符串 *UnsecuredApsFrame* 为 *ApsHeader*|| *Payload*。
- 8 设置 *FrameCount* 为 (*ReceivedFrameCount*+ 1)，在第 3 步获得的合适安全材料中保存 *FrameCount* 和 *SourceAddress*。如果存储帧计数和地址信息超过这类信息分配的内存存储量，且 NIB 中 *nwkAllFresh* 属性设置为 TRUE，那么安全处理会失败，本帧不能进一步进行安全处理。否则安全处理成功。
- 9 如果收到的帧的序列号是 *nwkSecurityMaterialSet* 中的一个新条目，那么 *nwkActiveKeySeqNumber* 应该设置为收到的序列号。

4.4.2 密钥建立服务

APSME 提供允许两个设备之间可以手动建立一个连接密钥的服务。初始信任信息（例如，一个主密钥）必须在运行密钥建立协议之前安装在每个设备上（见 4.4.3 小节提供初始信任信息的机制）。

4.4.2.1 APSME-ESTABLISH-KEY.request

APSME-ESTABLISH-KEY.request 原语用来启动一个密钥建立协议。当需要安全地和另外一个设备通信需要使用这个原语。

一个设备将会作为一个发起设备，另外一个设备将会作为一个响应设备。发起设备将会启动密钥建立协议通过发出：

- APSME-ESTABLISH-KEY.request，参数表明响应设备的地址。
- 表明应该使用哪个密钥建立协议（目前是 SKKE）。

4.4.2.1.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-ESTABLISH-KEY.request	{
	ResponderAddress,
	UseParent,
	ResponderParentAddress,
	KeyEstablishmentMethod
	}

表 4.5 指明了 APSME-ESTABLISH-KEY.request 原语的参数。

表 4.5 APSME-ESTABLISH-KEY.request 参数

参数名称	类型	有效范围	描述
Responder-Address	Device Address	任何有效的64位地址	响应设备的扩展64位地址
UseParent	Boolean	TRUE FALSE	此参数代表响应设备的父节点是否应该用于转发发起者和响应设备之间的信息： TRUE: 使用父节点 FALSE: 不使用父节点
Responder-ParentAddress	Device Address	任何有效的64位地址	如果 UseParent 是TRUE,那么 ResponderParentAddress参数应该包含响应设备的父设备的扩展64位地址; 否则, 该参数不能用并且不需要设置。
KeyEstablishment-Method	Integer	0x00 - 0x03	请求密钥建立的方法应是以下之一： 0x00 = SKKE 方法 0x01-0x03=保留

4.4.2.1.2 原语生成

当发起设备请求和一个响应设备建立一个连接密钥，它的上层产生本原语。如果发起设备希望使用父响应设备联络（为了 NWK 安全），应该设置 UseParent 参数为 TRUE，并设置 ResponderParentAddress 参数为父响应设备的 64 位扩展地址。

4.4.2.1.3 接收结果

接收到 APSME-ESTABLISH_KEY.request 原语之后，KeyEstablishmentMethod 参数等于 SKKE，引起 APSME 执行 SKKE 协议，见 4.4.2.6 小节。本地的 APSME 应该作为本协议的发起设备，ResponderAddress 参数指明的 APSME 应该作为本协议的响应设备，UseParent 参数将会控制信息是否通过 ResponderParentAddress 给出的父响应设备间接发送。

4.4.2.2 APSME-ESTABLISH-KEY.confirm

本原语是在密钥建立协议成功或者失败后发给 ZDO 的。

4.4.2.2.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-ESTABLISH-KEY.confirm	{
	Address,
	Status
	}

表 4.6 指明了 APSME-ESTABLISH-KEY.confirm 参数的原语。表 4.10 描述了本原语的 Status 参数可以返回的一些代码。除了这些代码，如果发送一个协议信息，发出的 NLDE-DATA.confirm 原语 Status 参数设为一个不是 SUCCESS 的值，APSME-ESTABLISH-KEY.confirm 原语 Status 参数应该设置为从 NWK 层收到的值。

表 4.6 APSME-ESTABLISH-KEY.confirm 参数

名称	类型	有效范围	描述
Address	Device Address	任何有效的64位地址	执行密钥建立协议设备的扩展64位地址
Status	Enumeration	通过表4.10给出的值或者从NLDE-DATA.confirm原语返回的任何状态值	该参数代表密钥建立协议的最终状态

4.4.2.2.2 原语生成

在密钥建立协议完成之后，响应设备和发起设备的 APSME 都应该发出这个原语给 ZDO。

4.4.2.2.3 接收结果

如果密钥建立成功，发起设备和响应设备的 AIB 应该用最新的连接密钥更新，发起者应该能和响应者加密通信。如果密钥建立不成功，那么 AIB 不能改变。

4.4.2.3 APSME-ESTABLISH-KEY.indication

当响应者从发起者接收到一个原始的密钥建立信息(例如一个 SKKE-1 帧),它的 APSME 应该发出这个原语给它的 ZDO。

4.4.2.3.1 服务原语的语义

这个原语应该提供以下的接口:

APSME-ESTABLISH-KEY.indication	{
	InitiatorAddress,
	KeyEstablishmentMethod
	}

表 4.7 指明了 APSME-ESTABLISH-KEY.indication 原语的参数。

表 4.7 APSME-ESTABLISH-KEY.indication 参数

名称	类型	有效范围	描述
InitiatorAddress	Device Address	任何有效的64位地址	发起设备的扩展64位地址
KeyEstablishmentMethod	Integer	0x00 - 0x03	密钥建立协议请求方法应该是下列之一: 0x00 = SKKE方法 0x01-0x03=保留

4.4.2.3.2 原语生成

当从一个发起设备收到一个开始密钥建立协议(例如一个 SKKE-1 帧)的请求,且和发起者相关的一个主密钥存在于 AIB 中,响应设备的 APSME 应该发出这个原语给 ZDO。

4.4.2.3.3 接收结果

在收到 APSME-ESTABLISH-KEY.indication 原语之后,ZDO 会使用 KeyEstablishmentMethod 和 InitiatorAddress 参数去确定是否和发起者建立密钥。ZDO 会使用 APSME-ESTABLISH-KEY.response 原语响应。

4.4.2.4 APSME-ESTABLISH-KEY.response

响应设备的 ZDO 会使用 APSME-ESTABLISH-KEY.Response 原语去响应 APSME-ESTABLISH-KEY.indication 原语。ZDO 决定是否继续密钥建立或者终止。这个决定在 APSME-ESTABLISH-KEY.indication 原语 Accept 参数中指明。

4.4.2.4.1 服务原语的语义

这个原语应该提供以下的接口：

```
APSME-ESTABLISH-KEY.response      {  
                                     InitiatorAddress,  
                                     Accept  
                                     }
```

表 4.8 指明了 APSME-ESTABLISH-KEY.response 原语的参数。

表 4.8 APSME-ESTABLISH-KEY.response 参数

名称	类型	有效范围	描述
InitiatorAddress	Device Address	有效的64位地址	初始化密钥建设备的扩展64位地址
Accept	Boolean	TRUE FALSE	该参数代表响应到初始化者的请求执行一个密钥建立协议。响应应该为： TRUE = 接收 FALSE = 拒绝

4.4.2.4.2 原语生成

在发起者启动一个密钥建立协议（即收到 APSME-ESTABLISH-KEY.indication 之后）之后，APSME-ESTABLISH-KEY.response 原语应该由 ZDO 产生并且提供给 APSME。这个原语提供给响应者的 ZDO 一个机会来决定接受还是拒绝和给定的发起者建立密钥的请求。

除了这个决定的任一标准，响应者的 ZDO 还必须检查 4.6.3.8 小节详述的允许配置表。

4.4.2.4.3 接收结果

如果 Accpt 参数是 TRUE，那么响应者的 APSME 会尝试按照 KeyEstablishmentMethod 参数指明的去执行密钥建立协议。如果 KeyEstablishmentMethod 等于 SKKE，APSME 会执行 SKKE 协议，见 4.4.2.6 小节。本地的 APSME 会作为本协议的响应者，InitiatorAddress 参数指明的 APSME 会作为本协议的发起者。如果 Accept 参数是 FALSE，本地的 APSME 会终止并且清除有关未定的密钥建立协议的中间数据。

如果 TransportKeyData 参数的 UseParent 子参数以及 *nwkSecureAllFrame* NIB 属性都是 TRUE，命令帧必须嵌入到一个通道命令帧中，如 4.6.3.7 小节所述。通道命令代替传输密钥命令被发送。

4.4.2.5 数据服务信息序列图

图 4.4 为两个设备之间成功建立密钥的原语序列：

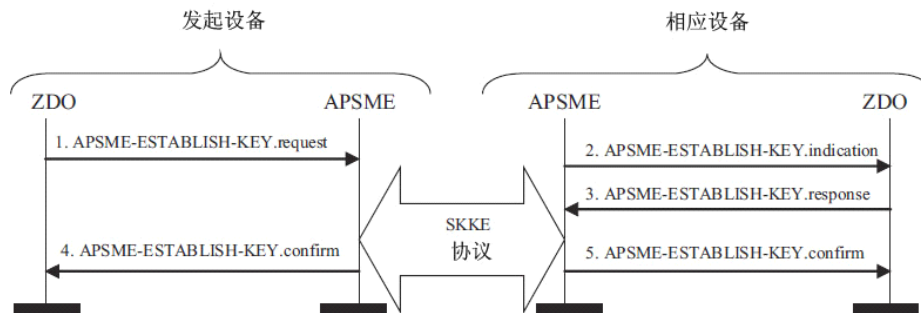


图 4.4 成功 APSME – ESTABLISH-KEY 原语的序列图

4.4.2.6 SKKE 协议

发起者和响应者的 APSME 执行对称密钥密钥协议方案，在 B.2.1 小节中有实例，B.1 中有说明。共享密钥（B.1 步骤 2 中有说明）应该是主密钥，在发起设备和响应设备之间共享，从 AIB 中 *apsDeviceKeyPairSet* 属性里合适的主密钥元素中获得。B.1 说明的方案执行期间发送的信息应该指定为表 4.9 里的帧名称。这些 SKKE 帧的格式在 4.4.9.1 小节中给出。发起设备负责发送 SKKE-1 帧和 SKKE-3 帧，响应设备负责发送 SKKE-2 帧和 SKKE-4 帧。另外，如果 APSME-ESTABLISH-KEY.request 原语的 UseParent 参数是 TRUE，父响应者（见 APSME-ESTABLISH-KEY.request 原语的 ResponderParentAddress 参数指明）应该作为一个连接并且在发起设备和响应设备之间发送信息。

在密钥建立方案期间，如果发起者或响应者设备检测到表 4.10 列出的任何错误条件，该方案将会中止，本地的 APSME 将会发出 APSME-ESTABLISH-KEY.confirm 原语，Status 参数设置为表 4.10 所指示。如果没有错误发生（即密钥协议方案输出“有效”），那么发起者和响应者将得到的密钥（即 *KeyData*）作为他们最新的共享密钥。发起者或响应者都会更新或者添加这一连接密钥到它们的 AIB 中，设置相应的输入和输出帧计数为零，发出 APSME-ESTABLISH-KEY.confirm 原语，设置 Status 参数为 SUCCESS。

表 4.9 映射帧的名字到对称密钥协商方案的密钥信息

帧名称	描述	子章节
SKKE-1	通过发起者发送在操作步骤1 (B.7.1小节)	4.4.2.6.2 小节
SKKE-2	通过响应者发送在操作步骤2 (B.7.2小节)	4.4.2.6.3 小节
SKKE-3	通过发起者发送在操作步骤8 (B.7.1小节)	4.4.2.6.4 小节
SKKE-4	通过响应者发送在操作步骤 9 (B.7.2小节)	4.4.2.6.5 小节

表 4.10 对称密钥密钥协定错误条件的状态代码映射

状态描述	状态代码	值
没有错误发生	SUCCESS	0x00
输入到密钥建立原语其中之一的一个无效参数	INVALID_PARAMETER	0x01
没有主密钥是可用的	NO_MASTER_KEY	0x02
质疑是无效的： 发起者在操作步骤3 (B.7.1小节) 响应者在操作步骤3 (B.7.2小节)	INVALID_CHALLENGE	0x03
SKG 输出无效： 发起者在操作步骤4 (B.7.1小节) 响应者在操作步骤3 (B.7.2小节)	INVALID_SKG	0x04
MAC转换输出无效： 发起者在操作步骤8 (B.7.1小节) 响应者在操作步骤10 (B.7.2小节)	INVALID_MAC	0x05
标记检查转换输出无效： 发起者在操作步骤12 (B.7.1小节) 响应者在操作步骤8 (B.7.2小节)	INVALID_KEY	0x06
无论是发起者还是响应者等待一个预期的传入信息的时间比等AIB的 <code>apsSecurityTimeOutPeriod</code> 属性要长	TIMEOUT	0x07
发起者或响应者收到的SKKE帧不按顺序	BAD_FRAME	0x08

4.4.2.6.1 产生和发送初始的 SKKE-1 帧

SKKE 协议从发起设备发送一个 SKKE-1 帧开始。SKKE-1 命令帧应该按照 4.4.9.1 小节构造。

如果 APSME-ESTABLISH-KEY.request 原语的 UseParent 参数是 FALSE，发起设备通过发送一个 SKKE-1 帧到响应设备开始协议（见 APSME-ESTABLISH-KEY.request 原语的 ResponderAddress 参数指明）。否则，发起设备会发送这个 SKKE-1 帧到父响应者设备开始协议（见 APSME-ESTABLISH-KEY.request 原语的 ResponderParentAddress 参数指明）。SKKE-1 帧在发送中使用 NLDE-DATA.request 原语，NWK 层安全设置为默认的 NWK 层安全等级。

4.4.2.6.2 接收 SKKE-1 帧

如果 SKKE-1 帧的响应者地址域不等于本地设备地址，APSME 将会执行如下步骤：

- 1 如果响应者地址域给定的设备不是本地设备的子设备，SKKE-1 帧应该被丢弃。
- 2 否则，本地设备的 APSME 将会使用 NLDE-DATA.request 原语发送 SKKE-1 帧到响应设备：
 - DestAddr 参数设置为 16 位地址，和 SKKE-1 帧的响应者地址域的 64 位地址对应；
 - DiscoverRoute 参数设置为 0x01；
 - SecurityEnable 参数设置为 FALSE。

否则，APSME 应该执行如下步骤：

如果设备没有一个主密钥对应发起者地址域：

- 1 SKKE-1 帧应该被丢弃，且 APSME-ESTABLISH-KEY.confirm 原语发送时 Status 参数设置为 NO_MASTER_KEY（见表 4.10）。APSME 应该停止处理这个 SKKE 协议。
- 2 否则，APSME 会发出一个原语 APSME-ESTABLISH-KEY.Indication, InitiatorAddress 参数设置为 SKKE-1 帧的发起者地址域，KeyEstablishmentMethod 参数设置为 0（即 SKKE 协议）。
- 3 在 APSME-ESTABLISH-KEY.indication 原语发出之后，在接收到相应的原语 APSME-ESTABLISH-KEY.response 时，APSME 应该评估接收的 APSME-ESTABLISH-KEY.response 原语的 InitiatorAddress 和 Accept 参数。如果 InitiatorAddress 参数设置为 SKKE-1 帧的发起者地址，Accept 参数设置为 FALSE，APSME 将会停止 SKKE 协议并且丢弃 SKKE-1 帧。

否则：

- 1 设备应该按 4.4.9.1 小节所述构建一个 SKKE-2 帧。如果 SKKE-1 帧的源指示同样的设备作为 SKKE-1 帧的发起者地址域。设备将会直接发送 SKKE-2 帧到发起设备，使用 NLDE-DATA.request 原语，DestAddr 参数设置为 SKKE-1 帧的源，DiscoverRoute 参数设置为 0x01，SecurityEnable 参数设置为 TRUE。否则，设备会发送 SKKE-2 帧到父设备，使用 NLDE-DATA.request 原语，DiscoverRoute 参数设置为 0x01，SecurityEnable 参数设置为 FALSE。

4.4.2.6.3 接收 SKKE-2 帧

如果SKKE-2帧的响应者地址并不等于本地设备地址，APSME应执行下列步骤：

- 1 如果响应者地址域给出的设备不是本地设备的子设备，SKKE-2帧应该被丢弃。
- 2 否则，设备会发送SKKE-2帧到发起设备，使用NLDE-DATA.request原语，NWK层安全设置为默认等级。

否则，本设备应该构建一个SKKE-3帧，如4.4.9.1小节指明。如果SKKE-2帧的源和SKKE-2帧的响应者地址域一样，该设备应将此SKKE-3帧直接发送到响应设备。否则，该设备应向响应者父设备发出SKKE-3帧。发出SKKE-3帧使用NLDE-DATA.request原语，NWK层安全设置为默认的NWK层安全等级。

4.4.2.6.4 接收 SKKE-3 帧

如果SKKE-3帧的响应者地址并不等于本地设备地址，APSME应执行下列步骤：

- 1 如果SKKE-3帧的响应者地址域给出的设备不等于本地设备地址，APSME执行以下步骤：
 - i 如果响应者地址域指定的设备不是本地设备的子节点，SKKE-3帧被丢弃。
 - ii 否则，设备会发送SKKE-3帧到响应设备，使用NLDE-DATA.request原语，NWK层安全禁用；
 - iii 否则，设备会处理SKKE-3数据域，如果协议不成功，设备应发出APSME-ESTABLISH-KEY.confirm原语，地址参数设置为发起者地址，状态参数应该适当设置；
- 2 如果从设备的角度来看协议是成功的，该设备应构建一个SKKE-4帧，如4.4.9.1小节指明。如果SKKE-3帧的源和SKKE-3帧的发起者地址域一样，该设备应将此SKKE-4帧直接发送到发起设备，使用NLDE-DATA.request原语，NWK层安全设置为默认级别。否则，该设备应发出SKKE-4帧给它的父设备，使用NLDE-DATA.request原语，NWK层安全禁用。最后，该设备应发出APSME-ESTABLISH-KEY.confirm原语，地址参数设置为发起者地址，Status参数设置为成功。

4.4.2.6.5 接收 SKKE-4 帧

如果 SKKE-4 帧的发起者地址并不等于本地设备地址， APSME 应执行下列步骤：

- 1 如果响应者地址域给出的设备不是本地设备的子设备， SKKE-4 帧应该被丢弃。
- 2 否则，本地设备的 APSME 会发送 SKKE-4 帧到发起设备，使用 NLDE-DATA.request 原语， NWK 层安全设置为默认级别。

否则， APSME 会处理 SKKE-4 帧，设备应发出 APSME-ESTABLISH-KEY.confirm 原语，地址参数设置为响应者地址，状态参数适当地设置。

4.4.3 传输密钥服务

APSME 提供允许发起者传输密钥材料到响应者的服务。可传输的不同类型的密钥材料见表 4.12 到 4.15。

4.4.3.1 APSME-TRANSPORT-KEY.request

APSME-TRANSPORT-KEY.request 原语用来传输密钥到其它设备。

4.4.3.1.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-TRANSPORT-KEY.request	{
	DestAddress,
	KeyType,
	TransportKeyData
	}

表 4.11 指明了 APSME-TRANSPORT-KEY.request 原语的参数。

表 4.11 APSME-TRANSPORT-KEY.request 参数

参数名称	类型	有效范围	描述
DestAddress	Device address	任何有效的64位地址	目的设备的扩展64位地址
KeyType	Integer	0x00 – 0x06	标识应该被传输的密钥材料的类型；见表 4.12
TransportKeyData	Variable	可变的	密钥确认和使用的参数被传输。参数的类型取决于下面KeyType参数： KeyType = 0x00见表4.13 KeyType = 0x01见表4.14 KeyType = 0x02见表4.15 KeyType = 0x03见表4.15 KeyType = 0x04见表4.13 KeyType = 0x05见表4.14

表 4.12 Transport-Key 原语的 KeyType 参数

枚举	值	描述
Trust-center master key	0x00	代表密钥是用来建立信任中心和其它设备之间的连接的一个主密钥
Standard network key	0x01	表示密钥是一个网络密钥，用于标准安全模式，可以使用密钥传输分发，或表示密钥是一个标准的网络密钥。
Application master key	0x02	代表密钥是用来建立两个设备间的连接的一个主密钥
Application link key	0x03	代表密钥是一个用于作为两个设备间基本安全的连接密钥
Trust-center link key	0x04	表示密钥是一个连接密钥，用作信任中心和其它设备之间的基本密钥。
High-security network key	0x05	表示密钥是一个网络密钥，用在高级安全模式中，只能使用密钥传输分发。

表 4.13 信任中心主密钥或连接密钥的 TransportKeyData 参数

参数名称	类型	有效范围	描述
ParentAddress	Device address	任何有效的64位地址	通过DestAddress参数给出的目的设备的父节点的扩展64位地址
Key	Set of 16 octets	可变的	信任中心密钥或连接密钥

表 4.14 网络密钥的 TransportKeyData 参数

参数名称	类型	有效范围	描述
KeySeqNumber	Octet	0x00-0xFF	通过信任中心分配的网络密钥，用于区分网络密钥的目的是更新密钥还是输入帧安全操作。
Set of 16 octets		Variable	网络密钥
UseParent	Boolean	TRUE FALSE	此参数代表目的设备的父节点是否应该用于转发密钥的目的设备： TRUE=使用父节点 FALSE=不使用父节点
ParentAddress	Device address	任何有效的64位地址	如果 UseParent 是TRUE, 那么 ParentAddress参数应该包含目的设备的父设备的扩展64位地址；否则，不使用该参数并且不需要设置。

表 4.15 应用程序主密钥或连接密钥的 TransportKeyData 参数

参数名称	类型	有效范围	描述
PartnerAddress	Device address	任何有效的 64位地址	发送主密钥设备的扩展64位地址
Initiator	Boolean	TRUE FALSE	此参数代表如果主密钥请求它的目的设备： TRUE=如果目的请求密钥 FALSE=其它
Key	Set of 16 octets	可变的	主密钥或者连接密钥(由KeyType参数指明)

4.4.3.1.2 原语生成

当发起设备需要传输密钥到响应设备时，它的 ZDO 应该产生这个原语。

4.4.3.1.3 接收结果

APSME-TRANSPORT-KEY.request 原语的接收会导致 APSME 创建一个密钥传输命令数据包（见 4.4.9.2 小节）。

如果 KeyType 参数是 0x00 或 0x04（即信任中心主密钥或连接密钥），密钥传输命令的密钥描述域应设置如下：

- 密钥子域应设置为 TransportKeyData 参数的 Key 子参数；
- 目标地址子域应设置为 DestinationAddress 参数；
- 源地址子域应设置为本地设备地址。

此命令帧应该按照 4.4.1.1 小节的规定安全保护。然后，如果安全处理成功，通过发出 NLDE-DATA.request 原语发送到 TransportKeyData 参数的 ParentAddress 子参数指定的设备。

如果 DestinationAddress 参数是全零，加密后的命令帧就被单播给设备任何子节点以及所有的 rx-off-when-idle 的子节点。这些单播一直重复直到成功，或随后收到一个 KeyType 参数等于 0x01 或 0x05 的 APSME-TRANSPORT-KEY.request 原语，或经过的周期是建议最大轮询间隔的两倍。

如果 KeyType 参数是 0x01 或 0x05（即一个网络密钥），传输密钥命令的密钥描述域应设置如下：

- 密钥子域应设置为 TransportKeyData 参数的 Key 子参数；
- 序列号子域应设置为 TransportKeyData 参数的 KeySeqNumber 子参数；
- 目标地址子域应设置为 DestinationAddress 参数；
- 源地址子域应设置为本地设备地址。

该命令帧应该按照 4.4.1.1 小节的规定安全保护，如果安全处理成功，那么通过发出 NLDE-DATA.request 原语发送到 TransportKeyData（如果 TransportKeyData 参数的 UseParent 子参数是 TRUE）参数或者 DestinationAddress 参数（如果 TransportKeyData 参数的 UseParent 子参数是 FALSE）的 ParentAddress 子参数指定的设备。

如果 TransportKeyData 参数的 UseParent 子参数为 FALSE，且 *nwkNeighborTable* 中有一个条目，它的扩展地址匹配 SenderAddress 参数，且关系域值为 0x05（未经验证的子节点），那么该节的关系域应设置为值 0x01（子节点）。

如果 KeyType 参数是 0x02 或 0x03（即应用程序主密钥或者连接密钥），传输密钥命令的密钥描述域应设置如下：

- 密钥子域应设置为 TransportKeyData 参数的 Key 子参数；
- 同伴地址子域应设置为 TransportKeyData 参数的 PartnerAddress 子参数；
- 发起者子域应该设置为 1（如果 TransportKeyData 参数的 Initiator 子参数是 TRUE）或者 0（如果 TransportKeyData 参数的 Initiator 子参数是 FALSE）。

该命令帧应该按照 4.4.1.1 小节的规定安全保护，如果安全处理成功，那么通过发出 NLDE-DATA.request 原语发送到 DestinationAddress 参数指定的设备。

4.4.3.2 APSME-TRANSPORT-KEY.indication

APSME-TRANSPORT-KEY.indication 原语用来通知 ZDO 接收到了密钥材料。

4.4.3.2.1 服务原语的语义

这个原语应该提供以下的接口：

```
APSME-TRANSPORT-KEY.indication    {  
                                     SrcAddress,  
                                     KeyType,  
                                     TransportKeyData  
                                     }
```

表 4.16 指明了 APSME-TRANSPORT-KEY.indication 原语的参数。

表 4.16 APSME-TRANSPORT-KEY.indication 参数

名称	类型	有效范围	描述
SrcAddress	Device Address	任何有效的64位地址	原始的传输密钥的设备的扩展64位地址
KeyType	Octet	0x00 – 0x06	代表传输密钥材料的类型，见表4.12
TransportKeyData	Variable	可变的	随着认证和使用参数传输的密钥，该参数取决于KeyType参数，如下： KeyType = 0x00 见表4.17 KeyType = 0x01 见表4.18 KeyType = 0x02 见表4.20 KeyType = 0x03 见表4.20 KeyType = 0x04 见表4.17 KeyType = 0x05 见表4.18

表 4.17 信任中心主密钥或连接密钥的 TransportKeyData 参数

参数名称	类型	有效范围	描述
TrustCenter-MasterKey	Set of 16 octets	可变的	信任中心主密钥

表 4.18 网络密钥的 TransportKeyData 参数

参数名称	类型	有效范围	描述
KeySeqNumber	Octet	0x00-0xFF	通过信任中心分配给网络密钥的序列号，用于区分网络密钥是更新密钥还是输入帧安全操作目的。
NetworkKey	Set of 16 octets	可变的	网络密钥

表 4.19 应用程序主密钥或连接密钥的 TransportKeyData 参数

参数名称	类型	有效范围	描述
PartnerAddress	Device address	任何有效 64 位地址	也送给这个主密钥的设备的扩展 64 位地址
Key	Set of 16 octets	可变的	主密钥或连接密钥（由 KeyType 参数指定）

4.4.3.2.2 原语产生

当 APSME 接收到一个传输密钥命令，它会产生这个原语，见 4.4.3.3 小节。

4.4.3.2.3 接收结果

在收到这一原语之后，ZDO 被通知收到密钥材料。

4.4.3.3 接收密钥传输命令

接收密钥传输命令时，APSME 会按照 4.5.1.1 小节的规定执行安全处理，然后检查密钥类型子域。

收到一个加密的传输密钥命令时，APSME 检查密钥类型子域。如果密钥类型子域设置为 0x02、0x03、0x04 或 0x05（应用程序连接密钥或主密钥、信任中心连接密钥或高安全网络密钥），且命令不是使用一个信任中心连接密钥加密的，命令就被丢弃。如果密钥类型域设置为 0x01（即标准网络密钥），且命令不是使用一个信任中心连接密钥或活动网络密钥加密的，命令就被丢弃。

如果密钥类型域设置为 0x02 或 0x03（即应用程序连接密钥或主密钥），APSME 就发出 APSME-TRANSPORT-KEY.indication 原语，SrcAddress 参数设置为密钥传输命令的源（由 NLDE-DATA.indication SrcAddress 参数指定），KeyType 参数设置为密钥类型域。TransportKeyData 参数设置如下：

- Key 子参数应定为密钥域。
- PartnerAddress 子参数设置为同伴地址子域。
- 如果发起者域为 1，Initiator 参数设置为 TRUE。否则为 0。

如果密钥类型域是 0x00、0x01、0x04 或 0x05（信任中心主密钥或者网络密钥），目标地址域等于本地设备地址，或如果密钥类型域设置为 0x01（即标准网络密钥），目标域是全零字符串，且当前网络密钥类型等于密钥类型域的值，APSME 会发出 APSME-TRANSPORT-KEY.indication 原语，SrcAddress 参数设置为密钥传输命令的源地址域，KeyType 参数设置为密钥类型域。TransportKeyData 参数设置如下：Key 子参数应定为密钥域，如果是网络密钥（即密钥类型域设置为 0x01 或 0x05），KeySeqNumber 子参数设置为序列号域。

如果密钥类型域是 0x00 或 0x01（即信任中心主密钥或者连接密钥或标准网络密钥），目标地址域不等于本地设备地址，APSME 会发送命令到目标地址域指示的地址，发出 NLDE-DATA.request 原语，安全禁用。

如果密钥类型域设置为 0x01（即标准网络密钥）且 *nwkNeighborTable* 中的条目的扩展地址匹配目标地址域的值，且关系域值为 0x05（未经验证的子节点），那么该条目的关系域应设置为值 0x01（子节点）。

收到密钥类型域设置为 0x01 的一个加密传输密钥命令时，如果目标域是全零，且源地址域设置为 *apsTrustCenterAddress* 的值，路由器就尝试单播这个传输密钥命令给任何子节点和所有 rx-off-when-idle 的子节点。路由器继续这样做直到成功，或直到随后收到密钥类型域设置为 0x01 或 0x05 的一个传输密钥命令，或直到经过两倍于建议的最大轮询间隔的周期。

在接收到未加密的密钥传输命令之后，APSME 会检查密钥类型子域，如果密钥类型域设置为 0x00（即信任中心主密钥），目标地址域等于本地设备地址，设备没有信任中心主密钥和地址（即 AIB 中的 *apsTrustCenterAddress*），那么 APSME 会发出 APSME-TRANSPORT-KEY.indication 原语。如果密钥类型域设置为 0x01 或 0x05（即网络密钥），目标地址域等于本地设备地址，且设备没有网络密钥，那么 APSME 会发出 APSME-TRANSPORT-KEY.indication 原语。如果密钥类型域设置为 0x04（即信任中心连接密钥），目标地址域等于本地地址，且设备没有信任中心连接密钥，那么 APSME 发出 APSME-TRANSPORT-KEY.indication 原语。如果发出了 APSME-TRANSPORT-KEY.indication 原语，SrcAddress 参数设置为密钥传输命令的源地址域，KeyType 参数设置为密钥类型域。TransportKeyData 参数设置如下：Key 子参数应设置为密钥域，如果是网络密钥（即密钥类型域设置 0x01 或 0x05），KeySeqNumber 子参数设置为序列号域。

4.4.4 更新设备服务

APSME 提供服务允许设备（例如一个路由器）去通知另外一个设备（例如一个信任中心），第三个设备已经改变状态（例如加入或者已经离开网络）。

4.4.4.1 APSME-UPDATE-DEVICE.request

当 ZDO 需要通知一个设备（例如一个信任中心）另外一个设备的状态需要更新（例如设备加入或者已经离开网络），它会发出这个原语。

4.4.4.1.1 服务原语的语义

这个原语应该提供以下的接口：

```
APSME-UPDATE-DEVICE.request      {  
                                   DestAddress,  
                                   DeviceAddress,  
                                   Status,  
                                   DeviceShortAddress  
                                   }
```

表 4.20 指明了 APSME-UPDATE-DEVICE.request 原语的参数。

表 4.20 APSME-UPDATE-DEVICE.request 参数

参数名称	类型	有效范围	描述
DestAddress	Device Address	任何有效64位地址	发送更新信息设备的扩展64位更新信息
DeviceAddress	Device Address	任何有效64位地址	状态正在更新的设备的扩展64位地址
Status	Integer	0x00 – 0x07	代表通过DeviceAddress参数给出的设备的更新状态： 0x00=标准设备安全加入 0x01=标准设备非安全加入 0x02=设备离开 0x03=标准设备非安全重新加入 0x04=高安全设备安全重新加入 0x05=高安全设备非安全加入 0x06=保留 0x07=高安全设备非安全重新加入
DeviceShortAddress	Network address	0x0000 - 0xffff	状态为正在更新的设备的16位网络地址

4.4.4.1.2 原语生成

当 ZDO 想传送更新设备信息到另一个设备（例如，信任中心），它（例如，在路由器或 ZigBee 协调器上）应启动 APSME-UPDATE-DEVICE.request 原语。

4.4.4.1.3 接收结果

收到 APSME-UPDATE-DEVICE.request 原语，设备应首先建立一个更新设备命令帧（见 4.4.9.3 小节）。此命令帧设备地址域设置为 DeviceAddress 参数，状态域应根据状态参数设置，该设备短地址域应设置为 DeviceShortAddress 参数。此命令帧应按照 4.4.1.1 小节安全保护的规定加以保护，然后，如果安全处理成功，通过发出 NLDE-DATA.request 原语，发送到 DestAddress 参数指定的设备。

4.4.4.2 APSME-UPDATE-DEVICE.indication

APSME 会发送原语通知 ZDO 接收到了更新设备命令帧。

4.4.4.2.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-UPDATE-DEVICE.indication	{ SrcAddress, DeviceAddress, Status, DeviceShortAddress }
--------------------------------	--

表 4.21 表明了 APSME-UPDATE-DEVICE.Indication 原语的参数。

表 4.21 APSME-UPDATE-DEVICE.indication 参数

参数名称	类型	有效范围	描述
SrcAddress	Device Address	任何有效的64位地址	原设备的更新设备命令的扩展64位地址
DeviceAddress	Device Address	任何有效的64位地址	状态正在更新的扩展的64位地址
Status	Octet	0x00 – 0x07	代表通过DeviceAddress参数给出的设备的更新状态 0x00=设备安全重新加入. 0x01=设备非安全加入. 0x02=设备离开 0x03=标准设备非安全重新加入 0x04=高安全设备安全重新加入 0x05=高安全设备非安全加入 0x06=保留 0x07=高安全设备非安全重新加入
DeviceShortAddress	Network address	0x0000 - 0xffff	正在被更新的16位网络地址

4.4.4.2.2 原语生成

当 APSME 接收到一个成功加密并认证过的更新设备命令帧，它会产生这个原语，见 4.4.1.2 小节。

4.4.4.2.3 接收结果

在收到 APSME-UPDATE-DEVICE.request 原语之后，ZDO 将被通知，DeviceAddress 参数指示的设备已经根据 Status 参数更新了状态。

4.4.5 移除设备服务

APSME 提供服务允许一个设备（例如信任中心）去通知其它设备（例如路由器）它的一个子设备应该从网络中移除。

4.4.5.1 APSME-REMOVE-DEVICE.request

一个设备的 ZDO（例如信任中心）应该发出这个原语去请求其它设备（例如路由器）移除网络中它的一个子设备。例如信任中心可以使用这个原语去移除一个未成功认证的子设备。

4.4.5.1.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-REMOVE-DEVICE.request	{
	ParentAddress,
	ChildAddress
	}

表 4.22 指明了 APSME-REMOVE-DEVICE.request 原语的参数。

表 4.22 APSME-REMOVE-DEVICE.request 参数

参数名称	类型	有效范围	描述
ParentAddress	Device Address	任何有效的64位地址	被请求移除的子设备的父设备的扩展64位地址
ChildAddress	Device Address	任何有效的64位地址	请求被移除的子设备的扩展64位地址

4.4.5.1.2 原语生成

一个设备的 ZDO（例如信任中心）应该发出 APSME-REMOVE-DEVICE.request 原语去请求一个父设备（由 ParentAddress 参数指明）移除网络中它的一个子设备（由 ChildAddress 参数指明）。

4.4.5.1.3 接收结果

在接收 APSME-REMOVE-DEVICE.request 原语之后，设备会首先创建一个移除设备命令帧（见 4.4.9.3 小节）。该命令帧的子节点地址域应该设置为 ChildAddress 参数。该命令帧按照 4.4.1.1 小节的规定执行安全保护，然后，如果安全处理成功，通过发出 NLDE-DATA.request 原语发送到 ParentAddress 参数指定的设备。

4.4.5.2 APSME-REMOVE-DEVICE.indication

APSME 发出这个原语去通知 ZDO 已经收到了移除设备命令帧。

4.4.5.2.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-REMOVE-DEVICE.indication	{
	SrcAddress,
	ChildAddress
	}

表 4.23 指定了 APSME-REMOVE-DEVICE.indication 的参数。

表 4.23 APSME-REMOVE-DEVICE.indication 参数

参数名称	类型	有效范围	描述
SrcAddress	Device Address	任何有效的64位地址	请求移除子设备的设备的扩展64位地址
ChildAddress	Device Address	任何有效的64位地址	请求移除子设备的设备的扩展64位地址

4.4.5.2.2 原语生成

当 APSME 接收到一个成功加密认证过的移除设备命令帧，会产生这个原语，见 4.4.1.2 小节。

4.4.5.2.3 接收结果

在收到 APSME-REMOVE-DEVICE.indication 原语之后，ZDO 应该被通知 SrcAddress 参数引用的设备正在请求 ChildAddress 参数指明的子设备被移除出网络。

4.4.6 请求密钥服务

APSME 提供服务允许设备从另一个设备（例如它的信任中心）请求活动网络密钥或者一个主密钥。

4.4.6.1 APSME-REQUEST-KEY.request

这个原语允许 ZDO 请求活动网络密钥或者一个新的端对端应用程序主密钥（主密钥或连接密钥）。

4.4.6.1.1 服务原语的语义

这个原语应该提供以下的接口：

```
APSME-REQUEST-KEY.request    {  
                                DestAddress,  
                                KeyType,  
                                PartnerAddress  
                                }
```

表 4.24 指定了 APSME-REQUEST-KEY.request 的参数。

表 4.24 APSME-REQUEST-KEY.request 参数

参数名称	类型	有效范围	描述
DestAddress	Device Address	任何有效的64位地址	请求密钥命令应该被发送到的设备的扩展64位地址
KeyType	Octet	0x00-0xFF	被请求的密钥类型 0x01 = 网络密钥 0x02 = 应用程序密钥 0x00 和 0x03-0xFF =保留
PartnerAddress	Device Address	任何有效的64位地址	如果KeyType 参数代表一个应用程序密钥,该参数应该代表接收到和设备请求的密钥一样的设备的扩展64位地址

4.4.6.1.2 原语生成

当设备请求活动网络密钥或者一个新的端对端应用主密钥（主密钥或连接密钥），它的 ZDO 应该产生 APSME-REQUEST-KEY.request 原语，。

4.4.6.1.3 接收结果

在收到 APSME-REQUEST-KEY.request 原语之后，设备应该先创建一个请求密钥命令帧（见 4.4.9.5 小节）。这个命令帧的密钥类型子域应该设置为和 KeyType 参数相同的值。如果 KeyType 参数是 0x02(即一个应用程序密钥)，那么该命令帧的同伴地址域应该为参数 PartnerAddress。否则，该命令帧的同伴地址域不能存在。

这个命令帧应该按照 4.4.1.1 小节的规定执行安全保护，然后，如果安全处理成功，通过发出 NLDE-DATA.request 原语发送到 DestAddress 参数指定的设备。

4.4.6.2 APSME-REQUEST-KEY.indication

APSME 会发出这个原语去通知 ZDO 已经收到了请求密钥命令帧。

4.4.6.2.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-REQUEST-KEY.indication	{
	SrcAddress,
	KeyType,
	PartnerAddress
	}

表 4.25 指定了 APSME-REQUEST-KEY.indication 原语的参数。

表 4.25 APSME-REQUEST-KEY.indication 参数

参数名称	类型	有效范围	描述
SrcAddress	Device Address	任何有效的64位地址	发送请求密钥设备的扩展64位地址
KeyType	Octet	0x00-0xFF	被请求的密钥类型： 0x01 = 网络密钥 0x02 = 应用程序密钥 0x00和0x03-0xFF = 保留
PartnerAddress	Device Address	任何有效的64位地址	如果KeyType 参数代表一个应用程序密钥，该参数应该代表接收到和设备请求一样的密钥的设备的64位地址

4.4.6.2.2 原语生成

当 APSME 收到了一个成功加密认证过的请求密钥命令帧,它会产生这个原语,见 4.4.1.2 小节。

4.4.6.2.3 接收结果

在收到 APSME-REQUEST-KEY.indication 原语之后,ZDO 应该被告知 SrcAddress 参数指明的设备正在请求一个密钥。被请求的密钥的类型应该由 KeyType 参数指示出来。如果 KeyType 参数是 0x02 (即一个应用程序密钥), PartnerAddress 参数会指示收到同样密钥的同伴设备作为请求密钥的设备 (即 SrcAddress 参数指明的设备)。

4.4.7 更换密钥服务

APSME 提供服务允许一个设备 (例如信任中心) 去通知其它设备应该更换到一个新的活动网络密钥。

4.4.7.1 APSME-SWITCH-KEY.request

这个原语允许一个设备 (例如信任中心) 去请求其它设备应该更换到一个新的活动网络密钥。

4.4.7.1.1 服务原语的语义

这个原语应该提供以下的接口:

APSME-SWITCH-KEY.request	{ DestAddress, KeySeqNumber }
--------------------------	--

表 4.26 指示了 APSME-SWITCH-KEY.request 原语的参数。

表 4.26 APSME-SWITCH-KEY.request 参数

参数名称	类型	有效范围	描述
DestAddress	Device Address	任何有效的64位地址	更换密钥命令发送到设备的扩展64位地址
KeySeqNumber	Octet	0x00-0xFF	序列号码分配给网络主要由信任中心, 用来区分网络密钥

4.4.7.1.2 原语生成

当一个设备（例如信任中心）需要去通知一个设备应该更换到一个新的活动网络密钥，它的 ZDO 应该产生 APSME-SWITCH-KEY.request 原语。

4.4.7.1.3 接收结果

在接收到 APSME-SWITCH-KEY.request 原语之后，设备应该首先创建一个更换密钥命令帧（见 4.4.9.6 小节）。该命令帧的序列号域应该设置为和 KeySeqNumber 参数的同样值。

这个命令帧应该按照 4.4.1.1 小节的规定执行安全保护，然后如果安全处理成功，通过发出 NLDE-DATA.request 原语发送到 DestAddress 参数指定的设备。

4.4.7.2 APSME-SWITCH-KEY.indication

APSME 会发出这个原语去通知 ZDO 已经收到了更换密钥命令帧。

4.4.7.2.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-SWITCH-KEY.indication	{
	SrcAddress,
	KeySeqNumber
	}

表 4.27 指定了 APSME-SWITCH-KEY.indication 原语的参数。

表 4.27 APSME-SWITCH-KEY.indication 参数

参数名称	类型	有效范围	描述
SrcAddress	Device Address	任何有效的64位地址	发送更换密钥命令的设备的扩展64位地址
KeySeqNumber	Octet	0x00-0xFF	序列号码通过信任中心分配给网络密钥，用来区分网络密钥

4.4.7.2.2 原语生成

当 APSME 收到了一个成功加密认证过的更换密钥命令帧, 它会产生这个原语, 见 4.4.1.2 小节。

4.4.7.2.3 接收结果

在接收到 APSME-SWITCH-KEY.indication 原语之后, ZDO 应该被告知 SrcAddress 参数指明的设备正在请求 KeySeqNumber 参数指明的网络密钥成为一个新的活动网络密钥。

4.4.7.3 加密的 APDU 帧

APS 层的帧格式包括 APS 头和 APS 负载域(见图 2.4)。APS 头包括帧控制和寻址域。当安全应用于一个应用协议数据单元帧, APS 帧控制域的安全位应设置为 1, 表示存在辅助帧头。辅助帧头的格式在 4.5.1 小节给出。加密的 APS 帧格式如图 4.5。辅助帧头位于 APS 头和 APS 负载域之间。

字节数: 可变的	5 或者 6	可变的	
原始的 APS 头([B7], 7.1节)	辅助帧头	加密负载	加密信息完整性代码(MIC)
		安全的范围内有效负载=CCM*输出	
完整的 APS 头		安全的 APS 负载	

图 4.5 安全的 APS 层帧格式

4.4.8 实体认证服务

APSME 提供允许两个设备之间互相认证的服务。此过程使用一个随机质疑, 它的响应基于一个预先共享的秘密来认证数据的发起者, 在此情况下, 秘密即密钥。它也允许可选的认证数据转换。

4.4.8.1 APSME-AUTHENTICATE.request

APSME-AUTHENTICATE.request 原语用来启动实体认证或者响应其它设备发起的实体认证。本原语可以在需要不使用帧安全而认证其它设备时使用。协议确认的真实性是基于两个设备共享一个预先共享的密钥。

4.4.8.1.1 服务原语的语义

这个原语应该提供以下的接口：

```

APSME-AUTHENTICATE.request      {
    PartnerAddress,
    Action,
    RandomChallenge
}
```

表 4.28 指定了 APSME-AUTHENTICATE.request 原语的参数。

表 4.28 APSME-AUTHENTICATE.request 的参数

参数名称	类型	有效范围	描述
PartnerAddress	DevAddress	任何有效的 64 位地址	扩展的，发起设备的 64 位 IEEE 地址
Action	枚举	INITIATE, RESPOND_ACCEPT, RESPOND_REJECT	表明要采取的行动，见表 4.29
RandomChallenge	16 字节的集合	-	从发起者收到的 16 字节随机质疑，本参数只有在 Action 参数等于 RESPOND_ACCEPT 时才有效。

表 4.29 行动参数列举

列举	值	描述
INITIATE	0x00	启动实体认证
RESPOND_ACCEPT	0x01	响应实体认证请求，接受
RESPOND_REJECT	0x02	响应实体认证请求，拒绝

4.4.8.1.2 原语生成

当发起者或者响应者需要启动或响应实体认证时，它的 ZDO 会产生这个原语，。如果 ZDO 响应 APSME-AUTHENTICATE.indication 原语，应该设置 RandomChallenge 为指示中相应的 RandomChallenge 参数的值。

4.4.8.1.3 接收结果

在接收到 APSME-AUTHENTICATE.request 原语之后，Action 参数设置为 INITIATE 会引起 APSME 启动相互实体认证协议，见 4.4.8.5 小节。本地的 APSME 会作为这个协议的发起者，PartnerAddress 参数指示的 APSME 会作为这个协议的响应者。

在接收到 APSME-AUTHENTICATE.request 原语之后，Action 参数设置为 RESPOND_ACCEPT 会引起 APSME 参加相互实体认证协议，见 4.4.8.5 小节。本地的 APSME 会作为这个协议的响应者，PartnerAddress 参数指示的 APSME 会作为协议的发起者。如果 Action 参数设置为 RESPOND_REJECT，实体认证不会发生。

4.4.8.2 APSME-AUTHENTICATE.confirm

在实体认证完成或失败时，本原语发给发起设备和响应设备的 ZDO。

4.4.8.1.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-AUTHENTICATE.confirm	{
	Address,
	Status
	}

表 4.30 指定了 APSME-AUTHENTICATE.confirm 原语的参数。除了这些代码，如果发送的协议信息中，NLDE-DATA.confirm 原语>Status 参数设置的值不是 SUCCESS，APSME-AUTHENTICATE.Confirm 原语>Status 参数设置为从 NWK 层收到的值。

表 4.30 APSME-AUTHENTICATE.confirm 原语的参数

参数名称	类型	有效范围	描述
Address	DevAddress	任何有效的 64 位地址	扩展的，实体认证发生设备的 64 位 IEEE 地址
Status	枚举	表 4.29 给 的 值 或 从 NLDE-DATA.confirm 原语返回的值	实体认证的最终状态

4.4.8.2.2 原语生成

在完成实体认证之后，发起者或者响应者的 APSME 会发出这个原语到它的 ZDO。

4.4.8.2.3 接收结果

在接收到 APSME-AUTHENTICATE.confirm 原语之后，发起者的 ZDO 应该被通知其请求认证响应者的结果。如果传输成功，Status 参数会设置为 SUCCESS，ZDO 会知道它和响应者共享一个密钥，且它已经从响应者收到了认证数据。否则，Status 参数会显示错误。

响应者的 ZDO 应该被通知发起者的认证数据传输的结果。如果传输成功，Status 参数会设置为 SUCCESS，ZDO 会知道它和发起者共享一个密钥，且它已经从发起者收到了认证数据。否则，Status 参数会显示错误。

4.4.8.3 APSME-AUTHENTICATE.indication

当响应者收到来自于发起者的初始认证信息，它的 APSME 发出这个原语到它的 ZDO。

4.4.8.3.1 服务原语的语义

这个原语应该提供以下的接口：

APSME-AUTHENTICATE.indication	{ InitiatorAddress, RandomChallenge, }
-------------------------------	---

表 4.31 指定了 APSME-AUTHENTICATE.indication 原语的参数。

表 4.31 APSME-AUTHENTICATE.indication 原语的参数

参数名称	类型	有效范围	描述
InitiatorAddress	DevAddress列举	任何有效的 64 位地址	扩展的，发起设备的 64 位 IEEE 地址
RandomChallenge	16 字节集合	-	从发起者上收到的 16 字节随机质疑

4.4.8.3.2 原语生成

当响应设备收到了来自于发起者的开始实体认证的请求，它的 APSME 会产生这个原语到它的 ZDO。

4.4.8.3.3 接收结果

在接收到 APSME-AUTHENTICATE.indication 原语之后，ZDO 会决定它是否希望参加实体认证，使用 InitiatorAddress 参数指定的发起者。如果这样，ZDO 会用 APSME-AUTHENTICATE.request 原语响应，设置 Action 参数为 RESPOND_ACCEPT，相应的 APSME-AUTHENTICATE.request 原语的 RandomChallenge 参数设置为 RandomChallenge 参数。如果它不希望参加发起者的实体认证，ZDO 会用 APSME-AUTHENTICATE.request 原语响应，设置 Action 参数为 RESPOND_REJECT。

4.4.8.4 数据服务信息序列图

图 4.6 说明了两个设备之间一个成功的实体认证所需的原语的序列。

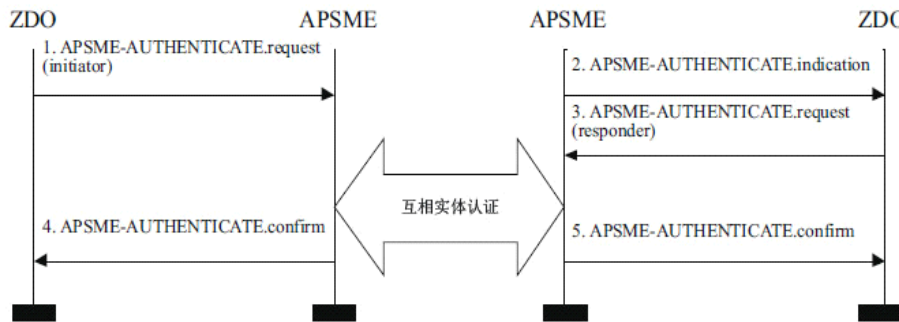


图 4.6 成功的 APSME-AUTHENTICATE 原语的信息序列

4.4.8.5 互相实体认证协议

发起者和响应者的 APSME 执行 B.8 小节指定的互相实体认证机制。在 B.8 小节指定的机制期间发送的信息应指定为表 4.32 给定的帧名称。这些实体认证帧的格式在 4.4.9.7 小节。

发起者和响应者质疑帧的 KeyType 子域必须设置为 0x00, 表示发起者和响应者之间共享的共享密钥是活动网络密钥。

B.8.2 小节的先决条件的第 2 步指明的 MacKey 必须是活动网络密钥, 在 *nwkSecurityMaterialSet* 上使用 NLME-GET.request 获得的发起者和响应者之间共享, 使用对应于 *nwkActiveKeySeqNumber* 的密钥。如果到来的发起者或响应者质疑帧的 KeySeqNumber 子域不匹配 *nwkActiveKeySeqNumber*, 或网络密钥不可用, 本地 APSME 就发出 APSME-AUTHENTICATE.confirm 原语, 状态参数设置为 NO_KEY。

B.8.2 小节指明的条件 2 以及 B.8.2 小节步骤 7 指明的 Text1, 应设置为输出的帧计数器, 与使用 A.1 小节指明的字节表示的响应者的活动网络密钥有关。使用 NLME-GET.request, 这是 *nwkSecurityMaterialSet* 条目的 OutgoingFrameCounter 条目, 它的 KeySeqNumber 条目等于 *nwkActiveKeySeqNumber*。如果帧计数器不可用, 响应者 APSME 发出 APSME-AUTHENTICATE.confirm 原语, 状态参数设置为 NO_DATA。对应响应者的到来的帧计数器, 与发起者的活动网络密钥有关, 应设置为 B.8.1 小节步骤 6 指明的 Text1。使用 NLME-SET.request, 这是 IncomingFrameCounterSet 条目的 IncomingFrameCounter 条目, 其中 SenderAddress 条目对应响应者, *nwkSecurityMaterialSet* 条目的 KeySeqNumber 条目等于 *nwkActiveKeySeqNumber*。

B.8.1 小节先决条件 1 和 B.8.1 小节步骤 4 指明的 *Text2* 应设置为输出帧计数器，与使用 A.1 小节指明的字节表示的发起者的活动网络密钥有关。使用 NLME-GET.request，这是 *nwkSecurityMaterialSet* 条目的 *OutgoingFrameCounter* 条目，它的 *KeySeqNumber* 条目等于 *nwkActiveKeySeqNumber*。如果帧计数器不可用，响应者 APSME 发出 APSME-AUTHENTICATE.confirm 原语，状态参数设置为 NO_DATA。对应发起者的输入帧计数器，与响应者的活动网络密钥有关，应设置为 B.8.2 小节步骤 3 指明的 *Text2*。使用 NLME-SET.request，这是 *IncomingFrameCounterSet* 条目的 *IncomingFrameCounter* 条目，其中 *nwkSecurityMaterialSet* 条目的 *KeySeqNumber* 条目等于 *nwkActiveKeySeqNumber*。

当发出 NLDE-DATA.request 原语以发送质疑帧或 MAC 和数据帧，SecurityEnable 参数必须总是设置为 FALSE，表示在网络层不使用安全。

当发送发起者或响应者质疑时，认证机制应通过启动一个周期为 *apsSecurityTimeoutPeriod* 的定时器进行监视。

认证机制期间，如果检测到表 4.32 所列的任何错误条件，那么机制就停止，本地 APSME 就发出 APSME-AUTHENTICATE.confirm 原语，状态参数设置为表 4.32 所指明的值。如果没有错误条件发生（即认证机制输出“有效”），那么发起者应视为可以使用所指定的密钥和响应者安全通信。发起者和响应者应发出 APSME-AUTHENTICATE.confirm 原语，状态参数设置为 SUCCESS。

表 4.32 帧名称和互相实体认证机制的映射

帧名称	描述	参考
APS_CMD_EA_INIT_CHLNG	由发起者在行动步骤 1 期间发送（B.8.1 小节）	4.4.9.7.1
APS_CMD_EA_RSP_CHLNG	由响应者在行动步骤 2 期间发送（B.8.2 小节）	4.4.9.7.2
APS_CMD_EA_INIT_MAC_DATA	由发起者在行动步骤 5 期间发送（B.8.1 小节）	4.4.9.7.3
APS_CMD_EA_RSP_MAC_DATA	由响应者在行动步骤 8 期间发送（B.8.2 小节）	4.4.9.7.4

表 4.33 互相实体认证错误条件和状态代码的映射

状态描述	状态代码	值
没有发生错误。	SUCCESS	0x00
一个无效参数输入到密钥建立原语的其中一个。	INVALID_PARAMETER	0x01
不存在认证密钥。	NO_KEY	0x02
不存在认证数据。	NO_DATA	0x03
质疑无效： 行动步骤 2 期间的发起者（B.8.1 小节） 行动步骤 1 期间的响应者（B.8.2 小节）	INVALID_CHALLENGE	0x04
MAC 传输输出无效： 行动步骤 4 期间的发起者（B.8.1 小节） 行动步骤 4 和 7 期间的响应者（B.8.2 小节）	INVALID_MAC	0x05
标记检查传输输出无效： 行动步骤 3 期间的发起者（B.8.1 小节） 行动步骤 6 期间的响应者（B.8.2 小节）	INVALID_KEY	0x06
发起者或响应者等待一个希望的到来信息的时间大于 <i>apsSecurityTimeoutPeriod</i> 。	TIMEOUT	0x07

4.4.9 命令帧

APS 层命令帧格式在本节中给出。

所有的 APS 层命令帧除非明文规定都通过加密发送。

表 4.34 给出了命令标识符值。

表 4.34 命令标识符的值

命令标识符	值
APS_CMD_SKKE_1	0x01
APS_CMD_SKKE_2	0x02
APS_CMD_SKKE_3	0x03 ^a
APS_CMD_SKKE_4	0x04
APS_CMD_TRANSPORT_KEY	0x05
APS_CMD_UPDATE_DEVICE	0x06

表 4.34 命令标识符的值（续表）

命令标识符	值
APS_CMD_REMOVE_DEVICE	0x07
APS_CMD_REQUEST_KEY	0x08
APS_CMD_SWITCH_KEY	0x09
APS_CMD_EA_INIT_CHLNG	0x0A
APS_CMD_EA_RSP_CHLNG	0x0B
APS_CMD_EA_INIT_MAC_DATA	0x0C
APS_CMD_EA_RSP_MAC_DATA	0x0D
APS_CMD_TUNNEL	0x0E

a. CCB #746

4.4.9.1 密钥建立命令

密钥建立时使用的 APS 命令帧在本节中有指示。通用 APS 帧格式的 APS 头部分的可选域不能存在。所有的密钥建立命令帧未加密发送。

通用 SKKE 命令帧格式如图 4.7。

字节数: 1	1	1	8	8	16
帧控制	APS计数器 ^a	APS命令标识符 ^b	发起者地址	响应者地址	数据
APS头 ^c			负载		

a. CCB #811

b. CCB #811

c. CCB #811

图 4.7 通用 SKKE 帧命令格式

4.4.9.1.1 命令标识符域

命令标识符域应该指示 APS 命令类型。对于 SKKE 帧，命令标识符应该指示 SKKE-1, SKKE-2, SKKE-3, 或 SKKE-4 帧，取决于帧类型（APS_CMD_SKKE_1, APS_CMD_SKKE_2, APS_CMD_SKKE_3, APS_CMD_SKKE_4, 见表格 4.34）³⁵。

35. CCB #811

4.4.9.1.2 发起者地址域

发起者地址域应该是作为密钥建立协议发起设备的 64 位扩展地址。

4.4.9.1.3 响应者地址域

响应者地址域应该是作为密钥建立协议响应设备的 64 位扩展地址。

4.4.9.1.4 数据域

数据域的内容取决于命令标识符域（即 SKKE-1，SKKE-2，SKKE-3，或 SKKE-4）。

4.4.9.1.4.1 到 4.4.9.1.4.4 小节描述了每个命令类型的数据域的内容。

4.4.9.1.4.1 SKKE-1 帧

数据域应该是 B.7.1 小节第 1 步发起者产生的质疑 QEU 的八位字节表示。

4.4.9.1.4.2 SKKE-2 帧

数据域应该是 B.7.2 小节 2 第 2 步响应者产生的质疑 QEV 的八位字节表示。

4.4.9.1.4.3 SKKE-3 帧

数据域应该是 B.7.1 小节第 1 步发起者产生的字符串 $MacTag_2$ 的八位字节表示。

4.4.9.1.4.4 SKKE-4 帧

数据域应该是 B.7.2 小节第 10 步响应者产生的字符串 $MacTag_1$ 的八位字节表示。

4.4.9.2 传输密钥命令

传输密钥命令格式如图 4.8。通用 APS 帧格式的 APS 头部分的可选域不能存在。

字节数: 1	1	1	1	可变的
帧控制	APS计数器 ^a	APS 命令标识符 ^b	密钥类型	密钥标识符
APS 头 ^c		负载		

a. CCB #811

b. CCB #811

c. CCB #811

图 4.8 Transport-Key 命令帧

4.4.9.2.1 命令标识符域

命令标识符域应该指明密钥传输 APS 命令类型(APS_CMD_TRANSPORT_KEY, 见表 4.34)。³⁶

4.4.9.2.2 密钥类型域

本域 8 位长, 描述了正在传输中的密钥类型。不同的密钥类型见表 4.12 。

4.4.9.2.3 密钥描述符域

本域长度可变, 应该包含实际(未保护)的传输密钥值, 以及任何相关标识和使用参数。该域信息取决于正在传输的密钥类型(见 4.4.9.2.2 小节密钥类型域), 应该设置为下列小节描述的格式之一。

4.4.9.2.3.1 信任中心主密钥或连接密钥描述符域

如果密钥类型域设置为 0 或 4, 密钥描述符域应该使用如图 4.9 的格式。

字节数: 16	8	8
密钥	目的地址	源地址

图 4.9 在 Transport-Key 命令中的信任中心主密钥描述符

36. CCB #811

- 密钥子域应该包含能用于和信任中心建立连接密钥的主密钥。
- 目标地址子域应该包含使用这个主密钥的设备地址。
- 源地址子域应该包含最初发送这个主密钥的设备（例如信任中心）地址。

4.4.9.2.3.2 网络密钥描述符域

如果密钥类型子域设置为 1，5 或 6，本域格式见图 4.10。

字节数: 16	1	8	8
密钥	序列号	目的地址	源地址

图 4.10 在 Transport-Key 命令中的网络密钥描述符域

- 密钥子域应该包含一个网络密钥。
- 序列号子域应该包含和网络密钥相关的序列号。
- 目标地址子域应该包含使用该网络密钥的设备地址。
- 如果网络密钥发送到一个广播地址，目标地址子域应该设置为全零字符串，在接收时应该被忽略。
- 源地址子域应该包含最初发送该网络密钥的设备（例如信任中心）地址。

4.4.9.2.3.3 应用程序主密钥和连接密钥的描述符域

如果密钥类型域设置为 2 或者 3，该域格式见图 4.11。

字节数: 16	8	1
密钥	同伴地址	发起者标志

图 4.11 在 Transport-Key 命令的应用程序主密钥标识符

- 密钥子域应该包含一个主密钥或者和同伴地址子域指明的设备共享的连接密钥。
- 同伴地址子域应该包含这个连接密钥或者主密钥发送到的其它设备的地址。

- 如果设备接收到本密钥请求的这个数据包，发起者标志子域应该设置为 1。否则本子域应该设置为 0。

4.4.9.3 更新设备命令

设备更新的 APS 命令帧在本节定义。通用 APS 帧格式的 APS 头部分的可选域不能存在。更新设备命令帧格式如图 4.12 。

字节数: 1	1	1	8	2	1
帧控制	APS计数器 ^a	命令标识符 ^b	设备地址	设备短地址	状态
APS头 ^c		负载			

a. CCB #811

b. CCB #811

c. CCB #811

图 4.12 Update-Device 命令帧的格式

4.4.9.3.1 命令标识符域

命令标识符域应该指明设备更新 APS 命令类型(APS_CMD_UPDATE_DEVICE， 见表 4.34)。³⁷

4.4.9.3.2 设备地址域

设备地址域应该为更新状态的设备的 64 位扩展地址。

4.4.9.3.3 设备短地址域

设备短地址域应该是更新状态的设备的 16 位扩展地址。

4.4.9.3.4 状态域

状态域应该被指定一个值，如表 4.20 描述的状态参数。

37. CCB #811

4.4.9.4 移除设备命令

APS 移除设备命令帧在本节中定义。通用 APS 帧格式的 APS 头部分的可选域不能存在。移除设备命令帧格式如图 4.13 。

字节数: 1	1	1	8
帧控制	APS计数器 ^a	命令标识符 ^b	子节点地址
APS 头 ^c		负载	

a. CCB #811

b. CCB #811

c. CCB #811

图 4.13 Remove-Device 命令帧的格式

4.4.9.4.1 命令标识符域

命令标识符域应该指示 APS 移除设备命令类型（APS_CMD_REMOVE_DEVICE，见表 4.34）。³⁸

4.4.9.4.2 子地址域

子地址域应该为要求移除出网络的设备的 64 位扩展地址。

4.4.9.5 请求密钥命令

设备请求密钥的 APS 命令帧在本节中定义。通用 APS 帧格式的 APS 头部分的可选域不能存在。

38. CCB #811



请求密钥命令帧格式如图 4.14 。

字节数: 1	1	1	1	0/8
帧控制	APS计数器 ^a	命令标识符 ^b	帧类型	同伴地址
APS 头 ^c		负载		

a. CCB #811

b. CCB #811

c. CCB #811

图 4.14 Request-Key 命令帧的格式

4.4.9.5.1 命令标识符域

命令标识符域应该指示 APS 请求密钥命令类型（APS_CMD_REQUEST_KEY，见表 4.34）。³⁹

4.4.9.5.2 密钥类型域

密钥类型域当网络密钥被请求时应该设置为 1，当应用程序密钥被请求时应该设置为 2，当信任中心连接密钥被请求时设置为 4。

4.4.9.5.3 同伴地址域

当密钥类型域是 2（即应用程序密钥），同伴地址域应该包含应该发送密钥的同伴设备的 64 位扩展地址。同伴设备和发起请求密钥命令的设备都会收到这个密钥。

当密钥类型是 1（即网络密钥）同伴地址域不得存在。

4.4.9.6 更换密钥命令

设备使用的更换密钥的 APS 命令帧在本节中定义。通用 APS 帧格式的 APS 头部分的可选项不能存在。

39. CCB #811

更换密钥命令帧格式如图 4.15。

字节数: 1	1	1	1
帧控制	APS计数器 ^a	APS命令标识符 ^b	序列号
APS 头 ^c		负载	

a. CCB #811

b. CCB #811

c. CCB #811

图 4.15 更换密钥命令帧格式

4.4.9.6.1 命令标识符域

命令标识符域应该指示 APS 更换密钥命令类型(APS_CMD_SWITCH_KEY, 见表 4.34)。

40

4.4.9.6.2 序列号域

序列号域应该包含确定要激活的网络密钥的序列号。

4.4.9.7 实体认证帧

实体认证中使用的 APS 命令帧在本节中描述。通用 APS 帧的 APS 头的可选域不能存在。发起者和响应者应该在各自传输中执行随机延迟以避免包传输冲突。⁴¹

40. CCB #811

41. CCB #674

4.4.9.7.1 实体认证发起者质疑帧

该帧是相互实体认证协议中发起者发给响应者的第一个帧，格式如图 4.16。

字节: 1	1	1	可变	8	8	16
帧控制	APS 计数器 ^a	APS 命令标识符 ^b	KeyInfo	发起者	响应者	质疑
APS 头 ^c		负载 ^d				

a. CCB#811

b. CCB#811

c. CCB#811

d. CCB#811

图 4.16 实体认证发起者质疑帧格式

4.4.9.7.1.1 命令标识符域

命令标识符域应该指示实体认证发起者质疑 APS 帧类型（APS_CMD_EA_INIT_CHLNG，见表格 4.34）。⁴²

4.4.9.7.1.2 KeyInfo 域

KeyInfo 域被划分为 2 个子域，如图 4.17。

字节: 1	0/1
KeyType	KeySeqNumber

图 4.17 KeyInfo 域格式

4.4.9.7.1.2.1 KeyType 子域

KeyType 子域是一个字节长，应该设置为表 4.35 的一个非保留值。

表 4.35 KeyType 子域的值

值	描述
0x00	活动的网络密钥
0x01	发起者和响应者之间共享的连接密钥
0x02-0xff	保留

42 CCB#811

4.4.9.7.1.2.2 KeySeqNumber 子域

KeySeqNumber 子域应该设置为活动网络密钥的密钥序列号。

4.4.9.7.1.3 发起者域

发起者域应该设置为作为发起设备的 64 位扩展地址。

4.4.9.7.1.4 响应者域

响应者域应该设置为作为响应设备的 64 位扩展地址。

4.4.9.7.1.5 质疑域

质疑域应为 B.8.1 小节第 1 步发起者产生的 *QEU* 质疑的字节表示。

4.4.9.7.2 实体认证响应质疑帧

此帧是响应者在相互实体认证协议中响应者发送给发起者的第一帧，格式如图 4.18。

字节: 1	1	1	可变	8	8	16
帧控制	APS 计数器 ^a	APS 命令标识符 ^b	KeyInfo	发起者	响应者	质疑
APS 头 ^c		负载 ^d				

a. CCB #811

b. CCB #811

c. CCB #811

d. CCB #811

图 4.18 实体认证响应者质疑帧格式

4.4.9.7.2.1 命令标识符域

命令标识符域应该指示实体认证响应者质疑 APS 命令类型（APS_CMD_EA_RSP_CHLNG，见表 4.34）。⁴³

43. CCB #811

4.4.9.7.2.2 KeyInfo 域

KeyInfo 域被分成两个子域如图 4.19。

字节: 1	0/1
KeyType	KeySeqNumber

图 4.19 KeyInfo 域格式

4.4.9.7.2.2.1 KeyType 子域

KeyType 域应该设置为 0x00，表示在发起者和响应者之间共享的共享密钥是活动网络密钥。

4.4.9.7.2.2.2 KeySeqNumber 子域

KeySeqNumber 子域应该设置为活动网络密钥的密钥序列号。

4.4.9.7.2.3 发起者域

发起者域应该设置为作为发起设备的 64 位扩展地址。

4.4.9.7.2.4 响应者域

响应者域应该设置为作为响应设备的 64 位扩展地址。

4.4.9.7.2.5 质疑域

质疑域应为 B.8.2 小节第 2 步响应者产生的 *QEV* 质疑的字节表示。

4.4.9.7.3 实体认证发起者 MAC 和数据帧

此帧是发起者在相互实体认证协议中发起者发送给响应者的第二个帧，格式如图 4.20。

字节: 1	1	1	16	1	4
帧控制	APS 计数器 ^a	APS 命令帧标识符 ^b	MAC	数据类型	数据
APS 头 ^c		负载 ^d			

a. CCB #811

b. CCB #811

c. CCB #811

d. CCB #811

图 4.20 实体认证发起者 MAC 和数据帧格式

4.4.9.7.3.1 命令标识符域

命令标识符域应该指示实体认证发起者 MAC 和数据 APS 命令类型 (APS_CMD_EA_RSP_DATA, 见表 4.34)。⁴⁴

4.4.9.7.3.2 MAC 域

MAC 域应是 B.8.1 小节第 4 步发起者产生的字符串 *MacTag₂* 的字节表示。

4.4.9.7.3.3 DataType 域

DataType 域应该设置为 0x00，表示活动与网络密钥相关的帧计数器。

4.4.9.7.3.4 数据域

数据域应该是字符串 *Text₂* 的字节表示，即与活动网络密钥相关的帧计数器。

44. CCB #811

4.4.9.7.4 实体认证响应者 MAC 和数据帧

此帧是响应者在相互实体认证协议中响应者发送给发起者的第二个帧，格式如图 4.21。

字节: 1	1	1	16	1	4
帧控制	APS 计数器 ^a	APS 命令帧标识符 ^b	MAC	数据类型	数据
APS 头 ^c		负载 ^d			

a. CCB #811

b. CCB #811

c. CCB #811

d. CCB #811

图 4.21 实体认证响应者 MAC 和数据帧格式

4.4.9.7.4.1 命令标识符域

命令标识符域应该指示实体认证响应者 MAC 和数据命令类型 (APS_CMD_EA_RSP_MAC_DATA, 见表 4.34)。⁴⁵

4.4.9.7.4.2 MAC 域

MAC 域应是 B.8.2 小节第 8 步响应者产生的字符串 *MacTag_I* 的字节表示。

4.4.9.7.4.3 DataType 域

DataType 域应该设置为 0x00，表示与活动网络密钥相关的帧计数器。

4.4.9.7.4.4 Data 域

数据域应该是字符串 *Text_I* 的字节表示，即与活动网络密钥相关的帧计数器。

4.4.9.8 通道命令

本节中指明了设备发送命令给缺少当前网络密钥的设备所用的 APS 命令帧。通用 APS 帧格式的 APS 头的可选域不能存在。通道密钥命令帧不加密发送。⁴⁶

45. CCB #811

46. CCB #850

通道密钥命令帧的格式如图 4.22 所示。

字节: 1	1	1	8	2	13	可变	4
帧控制	APS 计数器 ^a	APS 命令帧标识符	目标地址	通道 APS 头	通道辅助帧	通道命令	通道 APS MIC ^b
APS 头 ^c		负载 ^d					

图 4.22 通道命令帧格式

a. CCB #811

b. CCB #811

c. CCB #811

d. CCB #811

4.4.9.8.1 命令标识符域

命令标识符域应该指示通道 APS 命令帧类型 (APS_CMD_TUNNEL，见表 4.34)。⁴⁷

4.4.9.8.2 目的地址

目的地址域应该是收到通道命令设备的 64 位扩展地址。

4.4.9.8.3 通道辅助帧域

通道辅助帧域应该设置为用于加密通道命令的辅助帧（见 4.5.1 小节）。辅助帧应该指示使用连接密钥，应包括扩展临时域。

4.4.9.8.4 通道命令域

通道命令域应为要发送到目的地的 APS 命令帧。

4.4.10 安全相关的 AIB 属性

AIB 包括 APS 层安全管理所需的属性。每个属性都可以分别用 APSME-GET.request 原语读和 APSME-SET.request 原语写。在 APS PIB 中包含的安全相关的属性见表 4.36 和表 4.37。

表 4.36 AIB 安全属性

属性	标识符	类型	范围	描述	默认值
<i>apsDeviceKeyPairSet</i>	0xaa	密钥对描述符目录的设置。 见表4.37。	可变的	一组密钥对描述符包括其它设备共享的主密钥对和连接密钥对	-
<i>apsTrustCenterAddress</i>	0xab	Device address	任何有效的64位地址	代表设备的信任中心的地址	-
<i>apsSecurityTimeOutPeriod</i>	0xac	Integer	0x0000-0xFFFF	一个设备等待一个期待的安全协议帧的时间周期 (以毫秒为单位)	1000

表 4.37 Key-Pair 描述符的元素

名称	类型	范围	描述	默认值
DeviceAddress	Device address	任何有效的64位地址	代表该密钥对共享的实体的地址	-
MasterKey	Set of 16 octets	-	主密钥的实际值	-
LinkKey	Set of 16 octets	-	连接密钥的实际值	-
OutgoingFrame-Counter	Set of 4 octets	0x00000000-0xFFFFFFFF	和这个连接密钥使用的输出帧计数器	0x00000000
IncomingFrame-Counter	Set of 4 octets	0x00000000-0xFFFFFFFF	对应 <i>DeviceAddress</i> 的输入帧计数器的值	0x00000000

4.5 公用的安全元素

这一节描述与安全有关的功能，用于在一个以上 ZigBee 的层。NWK 和 APS 层应采用 4.5.1 小节指定的辅助头以及 4.5.2 小节指定的安全参数。在这一规范描述的所有帧的格式和域，按照由 NWK 层传输的顺序，从左至右，其中最左边的位首先传输。每个域的位从 0（最左边的和最低位）编号至 $k - 1$ （最右边的和最高位），域长度就为 k 位。长于一个字节的域被发送到上层，按照编号最低的字节到编号最高的字节的顺序。

4.5.1 辅助帧头格式

辅助帧头格式，如图 4.23，应该包括一个安全控制域和一个帧计数器域，可能会包含一个发送者地址域和一个密钥序列号域。

字节数: 1	4	0/8	0/1
安全控制	帧计数器	源地址	密钥序列号

图 4.23 辅助帧头格式

4.5.1.1 安全控制域

安全控制域应该包含一个安全等级，一个密钥标识符和一个扩展的临时子域，格式如图 4.24。

位: 0-2	3-4	5	6-7
安全级别	密钥标识符	扩展临时	保留

图 4.24 安全控制域格式

4.5.1.1.1 安全等级子域

安全等级标识符指示了输出帧如何加密，输入帧如何加密；它还指示了负载是否加密，帧提供了多大程度的认证，在信息完整性代码（MIC）的长度上反映出来。MIC 位长度取值 0, 32, 64 或 128，决定了 MIC 是正确的可能性的概率。安全等级的安全特性在表 4.38 中列了出来。注意安全等级标识符不表示不同的安全等级的相关强度。还要注意安全等级 0 和 4 不能用于帧安全。

表 4.38 对 NWK 和 APS 层可用的安全级别

安全级别标识	安全级别子域 (表 4.24)	安全属性	数据加密	帧完整性(MIC长度M, 字节数)
0x00	‘000’	None	OFF	NO (M = 0)
0x01	‘001’	MIC-32	OFF	YES (M=4)
0x02	‘010’	MIC-64	OFF	YES (M=8)
0x03	‘011’	MIC-128	OFF	YES (M=16)
0x04	‘100’	ENC	ON	NO (M = 0)
0x05	‘101’	ENC-MIC- 32	ON	YES (M=4)
0x06	‘110’	ENC-MIC- 64	ON	YES (M=8)
0x07	‘111’	ENC-MIC- 128	ON	YES (M=16)

4.5.1.1.2 密钥标识符子域

密钥标识符子域包含 2 位，用来识别用于保护帧的密钥。密钥标识符子域的编码见表 4.39。

表 4.39 密钥标识符子域编码

密钥标识符	密钥标识符子域 (表格4. 24)	描述
0x00	‘00’	一个数据密钥
0x01	‘01’	一个网络密钥
0x02	‘10’	一个密钥传输密钥
0x03	‘11’	一个密钥载入密钥

4.5.1.1.3 扩展临时子域

如果辅助头的发送者地址域存在的话，扩展的临时子域应该设置为 1。否则设置为 0。

4.5.1.2 计数器域

计数器域用于更新帧和阻止处理复制帧。

4.5.1.3 源地址域

源地址域只在安全控制域的扩展临时子域是 1 时才存在。当存在时，源地址域应该指示负责加密该帧的设备的扩展 64 位地址。

4.5.1.4 密钥序列号域

密钥序列号域只在安全控制域的密钥标识符子域是 1 时才存在（即网络密钥）。在存在时，密钥序列号域应该指示用于加密该帧的网络密钥的序列号。

4.5.2 安全参数

本小节指明了 CCM*安全操作的参数。

4.5.2.1 CCM*操作模式和参数

在特定安全等级上对 NWK 或 APS 帧使用安全，对应于 B.1.2 小节指明的 AES-CCM*操作模式的特定的一个实例。AES-CCM*操作模式是 AES-CCM 模式的拓展，AES-CCM 模式在 802.15.4-2003 MAC 规范中使用，提供认证和加密的功能，或者同时提供这两个功能。

该临时格式如 4.5.2.2 小节。

表格 4.38 给出了安全控制域的安全等级子域（表格 4.24），安全等级标识符，以及用于这些操作的 CCM*加密/认证属性之间的关系。

4.5.2.2 CCM*临时

用于 CCM*加密和认证传输、CCM*加密和认证检查传输的临时输入包括帧明确含有的数据，和两种设备都可以独立获得的数据。图 4.25 指明了 CCM*临时的子域的顺序和长度。正在处理的帧的临时域的安全控制和帧计数域和辅助头的安全控制和帧计数域相同（见 4.5.1 小节）。临时的源地址域应该设置为产生该帧安全保护的扩展的 64 位 IEEE 地址。当辅助头的安全控制域的扩展临时子域是 1，产生该帧安全保护的设备的扩展 64 位 IEEE 地址应该对应正在处理的帧的辅助头的源地址域（见 4.5.1 小节）。

字节数: 8	4	1
源地址	帧计数器	安全控制

图 4.25 CCM* 临时

4.5.3 密钥层次

通过 4.4.2 小节指明的密钥建立方案（或 4.4.3 小节指明的传输密钥命令）在两个（或者多个）设备间建立起来的连接密钥，用来设置相关的密钥，包括数据密钥，密钥传输密钥和密钥装载密钥。这些密钥设置如下：

- 1 密钥传输密钥。本密钥是执行 B.1.4 小节专用密钥哈希函数的结果，在连接密钥之下，输入的字符串是 1 字节字符串 ‘0x00’。
- 2 密钥装载密钥。本密钥是执行 B.1.4 小节专用密钥哈希函数的结果，在连接密钥之下，输入的字符串是 1 字节字符串 ‘0x02’。
- 3 数据密钥。本密钥等于连接密钥。

所有从连接密钥中获得的密钥应该共享相关的帧计数器。而且，ZigBee 的所有层应该共享活动网络密钥，以及相关的输出和输入帧计数器。

4.5.4 执行指南（提供信息）

本节提供了确保一个加密执行所遵循的一般指南。

4.5.4.1 随机数发生器

一个 ZigBee 设备执行密钥建立安全服务（见 4.2.3.1 小节）可能需要一个随机数发生的强大方法。例如，当连接密钥被预先安装（例如在工厂），可能不需要随机数。

所有需要随机数的情况下，随机数不可预见或有足够的熵是很关键的，这样一个攻击者不能通过耗尽搜索去确定它们。一般推荐就是随机数产生应该符合 FIPS140-2 [B13]的随机数测试。产生随机数的方法包括：

- 1 基于 ZigBee 硬件内的随机时钟和计数器的随机数；
- 2 基于随机外部事件的随机数；
- 3 在产生时从一个外部源往每个 ZigBee 设备中播种一个好的随机数。然后这个随机数可以用做一个种子去产生另外的随机数。

这些方法也可以组合使用。由于随机数产生可能被集成到 ZigBee IC，它的设计——以及因此的最终可行的任何加密/安全方案——留给 IC 生产厂家。

4.5.4.2 安全操作

为了避免攻击者可能利用的“漏洞”，安全程序被很好地执行和测试很重要。还希望安全执行不需要对每个应用重复认证。安全服务应该被安全专家执行和测试，不能被重复执行不同或者被不同的应用修改。

4.5.4.3 一致

一致应该在本规范的 profile 中定义。所选的加密协议的正确执行应该作为 ZigBee 认证过程的一部分被证实。本认证应该包括著名的价值测试：执行必须显示特定的某些参数，它会正确计算相应的结果。

4.6 功能描述

本小节提供了 ZigBee 网络中如何使用安全服务的详细描述。4.6.1 小节给出了 ZigBee 协调器的安全初始化责任的描述。4.6.2 小节给出了信任中心应用的简介。4.6.3 小节给出了详细的安全步骤。

4.6.1 ZigBee 协调器

ZigBee 协调器应该通过设置 NIB 中的 *NwkSecurityLevel* 属性来配置网络的安全等级。如果 *NwkSecurityLevel* 属性设置为 0，网络将不被加密，否则网络将被加密。

ZigBee 协调器应该通过设置 AIB 属性 *apsTrustCenterAddress* 来配置信任中心的地址。该地址的默认值是 ZigBee 协调器自己的地址，否则，ZigBee 协调器会指定另外一个备用的信任中心。

4.6.2 信任中心应用程序

信任中心应用程序在 ZigBee 网络设备中被设备信任的一个设备上运行，为了网络和端到端应用配置管理来分发密钥。信任中心应配置为标准或高级别安全模式运行，通过直接发送连接密钥（即密钥代管能力），或发送主密钥，可以用来帮助建立端到端应用程序密钥。这些密钥应该随机产生。

4.6.2.1 高级别安全模式

信任中心的高级别安全模式是专为高安全性的商业应用设计的。在这种模式下，信任中心应保持一份设备、主密钥、连接密钥和网络密钥的清单，它需要在控制和执行网络密钥更新和网络准入策略中使用。在此模式下，信任中心的网络中设备数量增多，所需的内存也增多，并且 NIB 中的 *nwkAllFresh* 属性应设置为 TRUE。它还使用 SKKE 和实体认证执行密钥建立。

4.6.2.2 标准级别安全模式

信任中心的标准级别安全模式是专为低安全性的住宅应用设计的。在这种模式下，信任中心应保持一份网络中所有设备、主密钥、连接密钥和网络密钥的清单；但是它应该维护一个标准的网络密钥，和网络准入的控制策略。在本模式下，信任中心的网络设备数量增多，所需的内存不增多，并且 NIB 中的 *nwkAllFresh* 属性应设置为 FALSE。

4.6.3 安全步骤

这一小节给出了加入一个安全的网络，认证新加入的设备，更新网络密钥，恢复网络密钥，建立端到端的应用程序密钥，离开一个安全的网络的信息序列图。

4.6.3.1 加入一个安全的网络

图4.26展示了当加入一个安全的网络时，加入设备和路由器设备通信的信息序列图实例。

运行在一个网络中的设备如果错过了一个密钥更新，也可以使用这些步骤来接收最新的网络密钥。

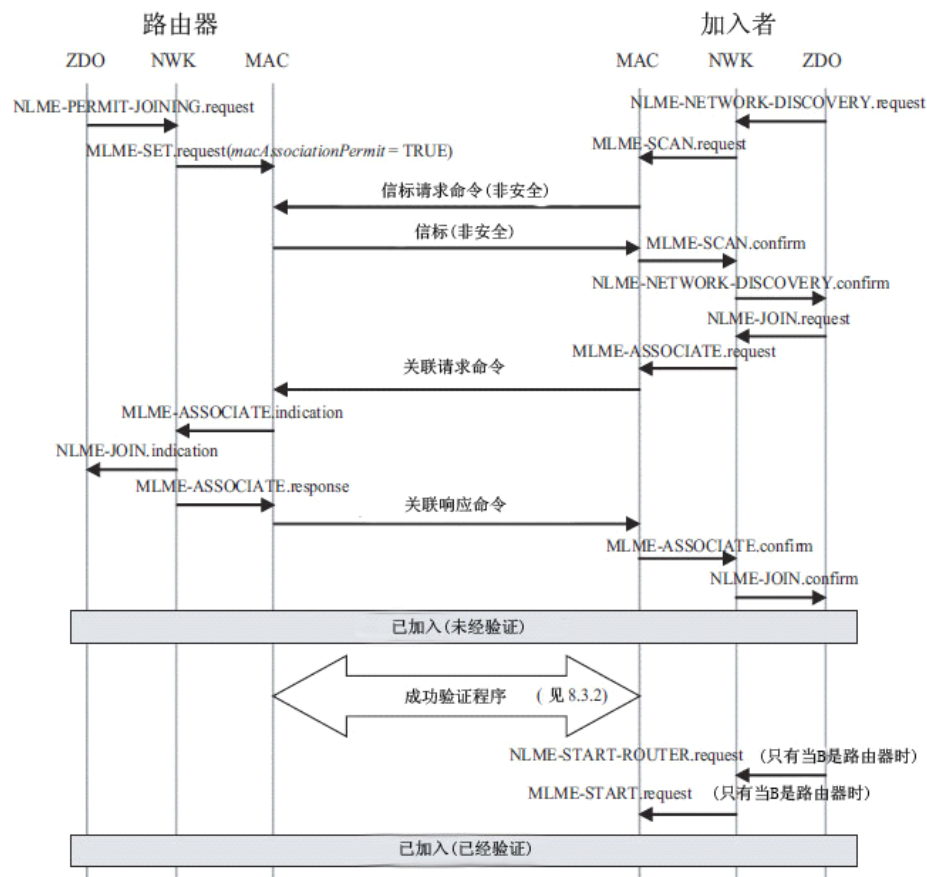


图 4.26 加入安全网络的例子

加入设备会通过发出一个 NLME-NETWORK-DISCOVERY.request 原语开始加入的程序，这个原语会引用会导致传输未加密的信标请求帧的一个 MLME-SCAN.request 原语（取决于这次扫描是主动扫描还是被动扫描）。

加入设备从附近的路由器接收信标，NWK 层会发出一个 NLME-NETWORK-DISCOVERY.confirm 原语。本原语的 NetworkList 参数会指明所有附近的 PAN。在图 4.26 当中，显示的路由器设备已经处于信标“连接允许”子域设置为“1”的状态。

加入设备应该决定加入哪个 PAN 网络，应该发出 NLME-JOIN.request 原语去加入那个 PAN 网络。如果加入者已经有本 PAN 网络的网络密钥，NLME-JOIN.request 原语的 SecurityEnable 参数应该设置为 TRUE；否则，应该设置为 FALSE。如图 4.26 所示，NLME-JOIN.request 原语导致连接请求或重新加入请求命令被发送到路由器。

在接收到连接请求 MAC 命令，路由器应该发出一个 MLME-ASSOCIATE.indication 原语。下一步，NWK 层会发出一个 NLME-JOIN.indication 原语到路由器的 ZDO。路由器现在应该知道加入设备的地址和安全功能。该路由器也会发出一个 MLME-ASSOCIATE.response 原语。本原语会引发一个连接响应命令发送到加入者。

或者，在接收到重新加入请求网络命令之后，NWK 层会发出一个 NLME-JOIN.confirm 原语给路由器的 ZDO。路由器现在应该知道加入设备的地址和安全功能，以及网络密钥是否用于加密重新加入请求命令。路由器还将发出一个重新加入响应命令给加入者。

在接收到连接响应 MAC 命令或重新加入响应网络命令，加入者发出 NLME-JOIN.confirm 原语。路由器现在向网络宣布“加入但未被认证”。应该遵守认证程序（见 4.6.3.2 小节）。

如果加入者不是一个路由器，成功完成认证程序之后，立即宣布“加入且已经被认证”。

如果加入者是一个路由器，只有通过发起路由操作成功完成认证程序之后，宣布“加入且已经被认证”。通过加入者的 ZDO 发出 NLME-START.request 原语导致 MLME-START.request 原语被发送到加入者，开始 MAC 层的路由操作。

如果路由器拒绝了加入者，它的连接响应帧或重新加入响应帧应该包含连接状态域设置为一个不是 0x00 的值，并且，在本参数到达 NLME-JOIN.confirm 原语的加入者的 ZDO 之后，加入者不能开始认证程序。

4.6.3.2 认证

一旦一个设备加入了一个安全的网络，且宣布“加入但未被认证”，它应必须按照本小节进行认证。在高安全模式下，邻居的路由器也必须按照 4.4.8 小节进行认证。

4.6.3.2.1 路由器操作

如果路由器不是信任中心，它应该在接收到 NLME-JOIN.indication 原语之后立即发出一个 APSME-UPDATE-DEVICE.request 原语，开始认证程序，其中 DestAddress 参数设置为 AIB 中的 *apsTrustCenterAddress*，DeviceAddress 参数设置为新加入设备的地址。该原语的状态应该由 NLME-JOIN.indication 原语参数根据表 4.40 设置。

表 4.40 NLME-JOIN.indication 参数到更新设备状态的映射

NLME-JOIN.indication 参数			更新设备状态	
能力信息 位 6	RejoinNetwork	SecuredJoin	状态	描述
0	TRUE	FALSE	0x00	标准安全设备加密加入
0	FALSE	FALSE	0x01	标准安全设备未加密加入
0	TRUE	TRUE	0x03	标准安全设备未加密重新加入
1	TRUE	FALSE	0x04	高安全设备未加密重新加入
1	FALSE	FALSE	0x05	高安全设备未加密加入
1	TRUE	TRUE	0x07	高安全设备未加密重新加入

如果路由器是信任中心，它应该简单的像一个信任中心运行以开始认证程序。

如果由加入设备发起，路由器通过使用 APSME-AUTHENTICATION.request 响应加入设备发起的一个实体认证来完成认证程序，其中 PartnerAddress 参数设置为加入设备的地址，Action 参数设置为响应，且 RandomChallenge 参数设置为一个新的随机值。

当相应的 NIB 中 *nwkNeighborTable* 的关系域条目的值是 0x05（未经验证的子节点），路由器不能转发信息到一个子节点设备，或代表该子节点响应 ZDO 请求或 NWK 命令请求。

4.6.3.2.2 信任中心操作

信任中心在验证程序中的作用，在收到输入的更新设备命令后激活，或收到 NLME-JOIN.indication 原语之后立即激活（如果路由器是信任中心）。信任中心的行为不同取决于至少五个因素：

- 信任中心是否决定允许新设备加入网络（例如，信任中心处于允许新设备加入的模式）；
- 信任中心是否运行在住宅或商业模式（分别见 4.6.2.1 小节和 4.6.2.2 小节）；
- 如果在标准安全模式，设备是无加密还是加密加入，以及加入设备的安全功能，正如更新设备命令的 Status 子域所指示；
- 如果在高安全模式，设备是无加密还是加密加入，加入设备的安全功能，正如更新设备命令的 Status 子域所指示，信任中心是否有一个主密钥对应新加入设备；
- NIB 中 *nwkSecureAllFrames* 属性。
- 当前网络密钥的类型。

如果在验证程序的任何时候，信任中心决定不允许新的设备加入网络（例如，一个决策或一个失败的密钥建立协议），它应采取行动，移除该设备出网络。如果信任中心不是新加入设备的路由器，它应该通过发出 APSME-REMOVE-DEVICE.request 原语，从网络中移除该设备，ParentAddress 参数设置为产生更新设备命令的路由器地址，ChildAddress 参数设置为已经加入（但未认证）的设备地址。

4.6.3.2.2.1 标准安全模式

在为了认证程序激活后，信任中心通过发出 APSME-TRANSPORT-KEY.request 原语将活动网络密钥发送到设备，DestAddress 参数设置为新加入的设备的地址，KeyType 参数设置为 0x01（即标准网络密钥）。

如果加入设备已经有网络密钥（即更新设备命令的 Status 子域是 0x00），TransportKeyData 应该作如下设置：KeySeqNumber 子参数应该设置为 0，NetworkKey 子参数应该设置为全 0，UseParent 参数应该设置为 FALSE。

否则，KeySeqNumber 子参数应设置为这个活动网络密钥的顺序计数值，NetworkKey 子参数应设置为活动网络密钥。如果信任中心是路由器，UseParent 子参数应设置为 FALSE；否则，UseParent 子参数应设置为 TRUE，ParentAddress 子参数应设置为产生更新设备命令的路由器的地址。

如果加入的设备并未预先配置一个活动网络密钥，本传输密钥原语的发出会导致活动网络密钥无加密地从路由器发送到新加入设备——这里假定安全通过非加密方式存在——例如，只发送此密钥一次，在低功耗、外部输入到路由器和加入者后立即发送，可以保证保密性和真实性。如果在 *apsSecurityTimeOutPeriod* 之内加入设备没有收到密钥，它应该复位，可以选择重新启动加入程序。

4.6.3.2.2.2 高安全模式

在为了认证程序激活后，信任中心运行在商业模式取决于设备加入网络是否预先配置有信任中心主密钥或连接密钥。

如果信任中心还没有和新加入设备共享一个主密钥或连接密钥，它应该通过发出 APSME-TRANSPORT-KEY.request 原语发给设备一个主密钥或连接密钥，DestAddress 参数设置为新加入设备的地址，KeyType 参数 0x00 或 0x04（即信任中心主密钥或连接密钥）。该 TransportKeyData 子参数应设置如下：Key 子参数应设置为合适的信任中心主密钥或连接密钥，如果信任中心是路由器，ParentAddress 子参数应设置为本地设备地址。否则，ParentAddress 子参数应设置为产生更新设备命令的路由器的地址。本传输密钥原语的发出会导致网络密钥无加密从路由器发送到新加入设备——这里假定安全通过非加密方式存在——例如，只发送此密钥一次，在低功耗，外部输入到路由器和加入者后立即发送，等等。

如果发送的是一个主密钥，信任中心将会通过发出 APSME-ESTABLISH-KEY.request 原语建立一个连接密钥，ResponderAddress 参数设置为新加入设备的地址，KeyEstablishmentMethod 设置为 0x00(即 SKKE)。另外，如果 NIB 中 *nwkSecureAllFrames* 属性是 FALSE 或者信任中心是路由器，UseParent 参数应该设置为 FALSE。否则，UseParent 参数应设置为 TRUE，ResponderParentAddress 参数应设置为产生更新设备命令的路由器的地址。

在接收到相应的 APSME-ESTABLISH-KEY.confirm 原语之后，且 Status 等于 0x00(即成功)，信任中心会发送活动网络密钥到新设备，通过发出 APSME-TRANSPORT-KEY.request 原语，DestAddress 参数设置为新加入设备的地址，KeyType 参数 0x06（即高安全网络密钥传输）。TransportKeyData 子参数应设置为如下：

- KeySeqNumber 子参数应设置为这个活动网络密钥的顺序计数值；
- NetworkKey 子参数应设置为活动网络密钥；
- 该 UseParent 子参数应设置为 FALSE。

4.6.3.2.3 加入设备操作

在成功地加入或重新加入一个安全的网络之后，加入设备应该参加本节中指定的认证程序。成功完成认证程序之后，加入设备应设置 AIB 中 *nwkSecurityLevel* 和 *nwkSecureAllFrames* 属性为路由器中信标所表明的值。

在一个安全的网络中一个已经加入和验证、且 *nwkSecureAllFrames* 等于 TRUE 的设备，将必须永远应用 NWK 层安全到输出（输入）帧，除非该帧是要到达（或源自）一个新加入的但是未被认证的子设备。如果 *nwkSecureAllFrames* 等于 FALSE，没有这种限制。

加入设备参加认证程序取决于设备的状态。有至少 4 种可能的初始状态要考虑：

- 预先配置有活动网络密钥（即标准安全模式）
- 预先配置有信任中心连接密钥和地址（即标准安全模式）
- 预先配置有信任中心主密钥和地址（即高安全模式）
- 没有预先配置（即不确定的模式——标准模式或者高安全模式）

在一个安全的网络中，如果设备在预先配置的时间内没有通过认证，它应该离开网络（见 4.6.3.3 小节）。

4.6.3.2.3.1 预先配置的网络密钥

如果加入设备只预先配置有活动网络密钥（且如果加入成功），它应该设置该密钥的输出帧计数器为零，清空该密钥的输入帧计数器设置，然后等待从信任中心接收一个虚网络密钥（全零）。在收到 APSMETRANSPORT-KEY.indication 原语，且 KeyType 参数设置为 0x01（即标准网络密钥）之后，加入的设备应该设置它的 AIB 中 *apsTrustCenterAddress* 属性为 APSME-TRANSPORTKEY.Indication 原语的 SrcAddress 参数。现在新加入的设备被视为通过了认证，应该进入标准安全模式的正常操作状态。

4.6.3.2.3.2 预先配置的信任中心连接密钥

如果加入的设备只预先配置有信任中心连接密钥和地址(即 AIB 中 *apsTrustCenterAddress* 属性), 它应等待从信任中心接收一个活动网络密钥。在收到 *KeyType* 参数设置为 0x01 (即标准网络密钥) 的 APSME-ESTABLISH-KEY.indication 原语之后, 加入设备就设置它的 AIB 属性 *apsTrustCenterAddress* 为 APSME-TRANSPORT-KEY.indication 原语的 SrcAddress 参数。现在加入设备被视为经过验证, 应进入标准安全模式的正常操作状态。

4.6.3.2.3.3 配置的信任中心主密钥

如果加入设备预先配置有信任中心主密钥和地址(即 AIB 中的 *apsTrustCenterAddress* 属性), 它应等待建立一个连接密钥, 并从信任中心接收一个活动网络密钥。因此, 在接收到 InitiatorAddress 参数设置为信任中心地址, 且 KeyEstablishmentMethod 参数设置为 SKKE 的 APSME-ESTABLISH-KEY.indication 原语时, 加入设备应响应 APSME-ESTABLISH-KEY.response 原语, 其中 InitiatorAddress 参数设置为信任中心的地址, Accept 参数设置为 TRUE。收到 Address 参数设置为信任中心地址, 状态参数设置为 0x00 (即成功) 的 APSME-ESTABLISH-KEY.confirm 原语之后, 加入设备应期望收到该活动网络密钥。如果加入设备在 *apsSecurityTimeOutPeriod* 内没有收到 InitiatorAddress 参数设置为信任中心地址, 且 KeyEstablishmentMethod 参数设置为 SKKE 的 APSME-ESTABLISH-KEY.indication 原语, 它应复位, 可以选择再次启动加入程序。

如果加入设备有一个信任中心连接密钥, 收到 SourceAddress 参数设置为信任中心地址且 KeyType 参数设置为 0x05 (即高安全网络密钥) 的 APSME-TRANSPORT-KEY.indication 原语时, 设备使用 TransportKeyData 参数中的数据来配置活动网络密钥。

认证程序应由加入设备发起一个实体认证完成，即路由器使用 APSME-AUTHENTICATION.request, PartnerAddress 参数设置为路由器地址, Action 参数设置为发起, 且 RandomChallenge 参数设置为一个新的随机值。然后路由器将响应一个 APSME-AUTHENTICATION.request, PartnerAddress 参数设置为加入设备地址, Action 参数设置为响应, 且 RandomChallenge 参数设置为一个新的随机值。

现在加入设备被视为已经认证, 进入高安全模式的正常操作状态。

4.6.3.2.3.4 未预先配置

如果加入设备没有预先配置活动网络密钥, 也没有一个信任中心主密钥或连接密钥和地址 (即 AIB 中 *apsTrustCenterAddress* 属性), 它应该等待接收一个无加密信任中心主密钥或活动网络密钥。实施者应注意, 传输一个无加密密钥代表了安全风险, 如果安全是一个问题, 应预先配置密钥——最好通过保证保密性和真实性的带外机制。如果加入设备在 *apsSecurityTimeOutPeriod* 时间内没有收到任何密钥, 它应复位, 可以再次选择开始加入程序。

在收到 APSME-TRANSPORT-KEY.indication 原语之后, 且 KeyType 参数设置为 0x01 (即标准网络密钥), 加入的设备应使 TransportKeyData 参数中的数据作为它的活动网络密钥, 应设置它的 AIB 中 *apsTrustCenterAddress* 属性为 APSME-TRANSPORT-KEY.indication 原语的 SrcAddress 参数。现在加入的设备被认为通过验证, 并应进入标准安全模式的正常操作状态。

在收到 APSME-TRANSPORT-KEY.indication 原语之后, 且 KeyType 参数设置为 0x00 (即信任中心连接密钥), 加入的设备应使 TransportKeyData 参数中的数据作为它的信任中心连接密钥, 应设置它的 AIB 中 *apsTrustCenterAddress* 属性为 SrcAddress 参数。如果 APSME-TRANSPORT-KEY.indication 原语的 SourceAddress 参数设置为信任中心的地址, 且 KeyType 参数设置为 0x01 (即标准网络密钥), 加入的设备应使用 TransportKeyData 参数中的数据来配置活动网络密钥。活动网络密钥的所有输入帧计数器和输出帧计数器都应设置为 0。现在加入的设备被认为通过验证, 并应进入标准安全模式的正常操作状态。

在收到 APSME-TRANSPORT-KEY.indication 原语之后, 且 KeyType 参数设置为 0x00 (即信任中心主密钥), 加入的设备应使 TransportKeyData 参数中的数据作为它的信任中心主密钥, 设置它的 AIB 中 *apsTrustCenterAddress* 属性为 SrcAddress 参数。

下一步，在收到 APSME-ESTABLISH-KEY.indication 原语之后，且 InitiatorAddress 参数设置为信任中心地址，KeyEstablishmentMethod 参数设置为 SKKE，加入的设备应该响应 APSME-ESTABLISH-KEY.response 原语，InitiatorAddress 参数设置为信任中心地址，Accept 参数设置为 TRUE。在收到 APSME-ESTABLISH-KEY.confirm 原语之后，且 Address 参数设置为信任中心地址，Status 参数设置为 0x00（即成功），加入的设备应该期望接收到活动网络密钥。如果 APSME-TRANSPORT-KEY.indication 原语的 SourceAddress 参数设置为信任中心地址，且 KeyType 参数设置为 0x0（即高安全网络密钥），加入的设备应使用 TransportKeyData 参数中的数据去配置活动网络密钥。认证程序通过使用 APSME-AUTHENTICATION.request 由加入设备发起一个和路由器的实体认证完成，其中 PartnerAddress 参数设置为路由器地址，Action 参数设置为发起，RandomChallenge 参数设置为一个新的随机值。然后路由器响应一个 APSME-AUTHENTICATION.request，PartnerAddress 参数设置为加入设备地址，Action 参数设置为响应，RandomChallenge 参数设置为一个新的随机值。现在加入的设备被认为通过验证，并应进入商业模式的正常操作状态。

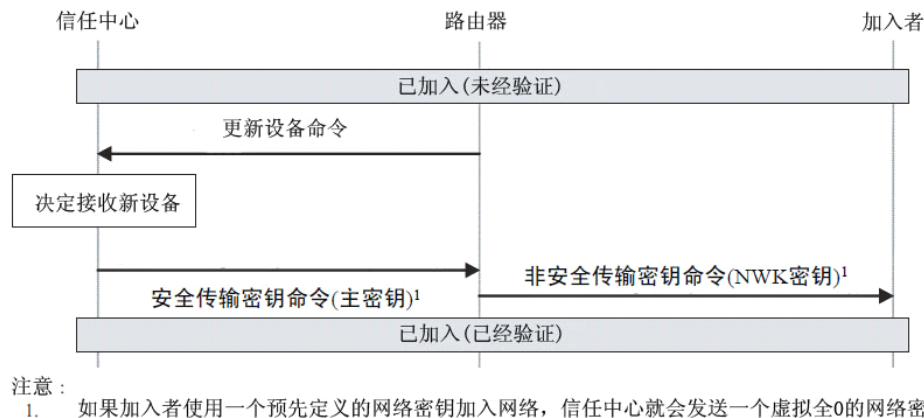


图 4.27 标准安全模式验证过程的例子

4.6.3.2.4 邻居设备认证

如果一个邻居设备尝试通信，邻居设备未知，且设备处于高安全模式，帧计数器检查正在进行，因为帧计数器没有启动，通信会被拒绝。邻居设备通过使用 APSME-AUTHENTICATION.request 发起一个实体认证，PartnerAddress 参数设置为邻居设备地址，Action 参数设置为发起，RandomChallenge 参数设置为一个新的随机值。APSME-AUTHENTICATION.request 发送时带有一个随机抖动。

然后邻居设备会用APSME-AUTHENTICATION.request 响应, PartnerAddress参数设置为设备地址, Action参数设置为响应, RandomChallenge参数设置为一个新的随机值。邻居设备现在被认为通过认证。⁴⁸

4.6.3.2.5 信息序列图表

图4.27和图4.28给出了当路由器和信任中心是分别运行在标准模式或高安全模式下的不同设备, 认证步骤的实例信息序列图表。

在图 4.27 中, 更新设备和传输密钥命令在信任中心和路由器之间的通信, 应该在 APS 层根据活动网络密钥加密。从路由器发送到加入者的传输密钥命令无需加密。

在图 4.28 中, 更新设备和传输密钥命令在信任中心和路由器之间的通信, 应该在 APS 层根据信任中心连接密钥进行加密。如果 NIB 中 *nwkSecureAllFrames* 属性是 TRUE, 它也应该在 NWK 层用活动网络密钥加密。从路由器发送到加入者的传输密钥命令无需加密。SKKE 命令在发送时, 使用路由器作为一个联结, 当 NIB 中 *nwkSecureAllFrames* 属性是 TRUE, 这样 SKKE 命令在信任中心和路由器之间的通信应该在 NWK 层用活动网络密钥加密, 在路由器和加入者之间的命令无需加密。否则, 信任中心和加入者之间的 SKKE 命令无需加密。最终的传输密钥在信任中心到加入者之间的通信需要在 APS 层根据信任中心连接密钥加密, 如果 NIB 中 *nwkSecureAllFrames* 属性是 TRUE, 也要在 NWK 层用活动网络密钥加密。

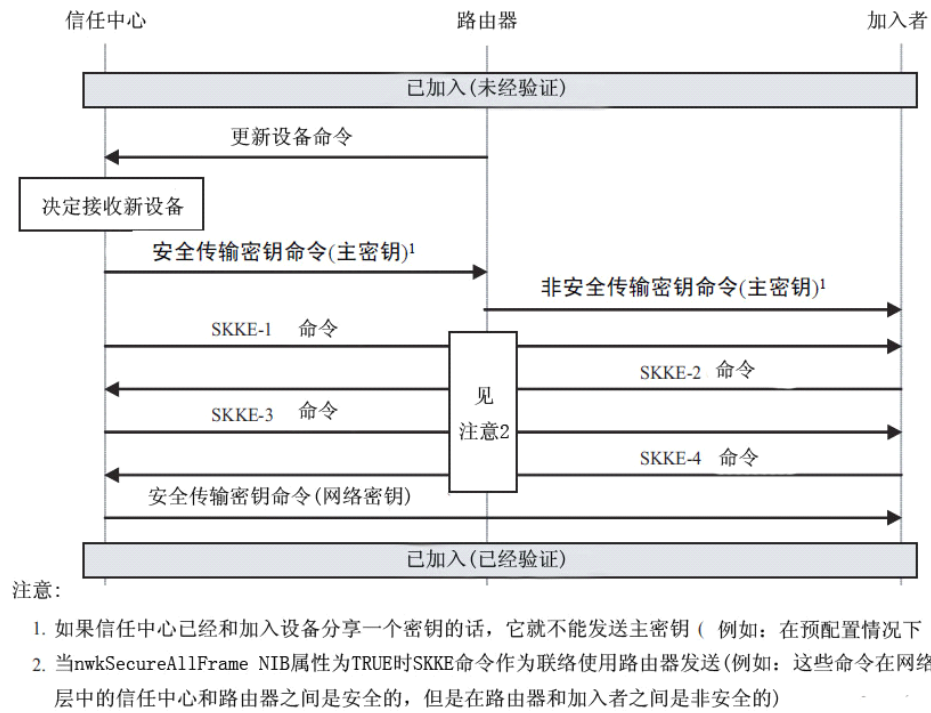


图 4.28 高安全模式认证过程例子

4.6.3.3 Intra-PAN 可移植性

遵守本小节所述程序的设备必须支持可移植性, 参看 2.5.5.5.2.2 小节所述的机制。

目前, Intra-PAN 可移植性的安全支持被最小化了。在大多数情况下, 安全操作和正常的没有什么不同, 如这些章节所述。

4.6.3.3.1 路由器操作

本节介绍了为支持 Intra-PAN 可移植性, ZigBee 协调器和 ZigBee 路由器要进行的安全操作。

按照 2.5.5.5.2.2 小节所述的步骤, 一个孤设备暂时被接受到网络至少要经过 *apsSecurityTimeOutPeriod* 毫秒的时间。在此期间, 它应被要求发送至少一个形式正确的 ZigBee 信息到新的父设备, 使用网络密钥加密。如果这信息顺利通过本文件中所描述的所有安全处理步骤, 它应被接纳为网络的一个成员。

如果来自一个孤设备的一个信息未能通过以上在本文件中任何地方所规定的安全处理, 本规范既不规定, 也不需要父设备采取任何行动。

4.6.3.3.2 终端设备操作

按照 2.5.5.5.2.2 小节所述的步骤, 一个孤设备暂时被接受到网络至少要经过 *apsSecurityTimeOutPeriod* 毫秒的时间。在此期间, 它应被要求发送至少一个形式正确的 ZigBee 信息到新的父设备, 使用网络密钥加密。进一步, 它应证实从新的父设备收到至少一个 ZigBee 信息, 使用当前网络密钥加密。如果这信息顺利通过本文件中所描述的所有安全处理步骤, 设备应作为网络成员操作。如果该协议没有正确完成, 设备会放弃使用这个父设备重新加入网络的尝试。

一般情况下, 当运行在标准安全模式, 终端设备不得接受从信任中心发来的未加密的网络密钥, 除了当被激活加入一个新的网络时本文件的要求。

一般情况下, 当运行在高安全模式, 终端设备配置为接受从信任中心发来的更新后的网络密钥, 甚至当没有为认证程序激活也是如此。在此情况下, 该密钥应该用 APSME-TRANSPORT-KEY.request 原语传输, 并且要证实合适的安全。

注意 ZigBee 路由器也可以执行一个孤扫描, 参考 2.5.5.5.2.2 小节所述。在此情况下, 它也应该遵守本小节描述的步骤。

4.6.3.4 网络密钥更新

信任中心和网络设备更新活动网络密钥应该遵守本小节的步骤。

4.6.3.4.1 信任中心操作

当用同样类型的新密钥更新一个标准的网络密钥，信任中心会通过发出 APSME-TRANSPORT-KEY.request 原语，广播这个新密钥到所有的设备，DestAddress 参数设置为广播地址，KeyType 参数设置为 0x01（即标准网络密钥）。TransportKeyData 子参数应该做如下设置：

- KeySeqNumber 子参数应该设置为新的网络密钥的序列计数值；
- NetworkKey 子参数应该设置为新的网络密钥；
- UseParent 子参数应该设置为 FALSE 。

如果前面分发给网络中的密钥顺序计数以 N 表示，那么这个新网络密钥的顺序计数应是 $(N+1) \bmod 256$ 。信任中心会通过发出 APSEM-ESWITCH-KEY.request 更换新密钥，DestAddress 参数设置为广播地址，KeySeqNumber 参数应该设置为更新后的网络密钥的序列计数值。

当运行在标准安全模式，信任中心会保持一个网络中所有设备的清单；在高安全模式下，也会保持一个这样的清单。为了使用这个清单更新活动网络密钥，信任中心先会发送新的网络密钥到清单上每个设备，然后让每个设备更换成新的密钥。新的网络密钥应该发送到清单上一个设备，通过发出 APSME-TRANSPORT-KEY.request 原语，DestAddress 参数设置为清单上设备地址，KeyType 参数设置为 0x05（即高安全网络密钥）。TransportKeyData 子参数应该做如下设置：

- KeySeqNumber 子参数应该设置为新的网络密钥的序列计数值；
- NetworkKey 子参数应该设置为新的网络密钥；
- UseParent 子参数应该设置为 FALSE 。

如果前面分发给网络中的密钥顺序计数以 N 表示，那么这个新网络密钥的顺序计数应是 $(N+1) \bmod 256$ 。信任中心会通过发出 APSEME-SWITCH-KEY.request 要求一个设备更换成这个新密钥，DestAddress 参数设置为设备地址，KeySeqNumber 参数应该设置为更新后的网络密钥的序列计数值。

4.6.3.4.2 网络设备操作

当在正常操作状态下,在接收到 APSME-TRANSPORT-KEY.Indication 原语之后,KeyType 参数设置为 0x01 或 0x05 (即网络密钥),只有在 SrcAddress 参数等于信任中心地址 (见 AIB 中 *apsTrustCenterAddress* 属性) 时,设备才接受 TransportKeyData 参数作为网络密钥。如果接受,且设备可以存储一个备用网络密钥,TransportKeyData 参数中包含的密钥和序列号数据将会取代备用网络密钥。否则,TransportKeyData 参数中包含的密钥和序列号数据将会取代活动网络密钥。在这两种情况下,合适的网络密钥的所有输入和输出帧计数器都应该设置为 0。

当在正常操作状态下,在接收到 APSME-SWITCH-KEY.Indication 原语之后,仅当 SrcAddress 参数和信任中心地址 (存储在 AIB *apsTrustCenterAddress* 属性) 相同时,设备应该更换它的活动网络密钥成为 KeySeqNumber 参数指定的密钥。

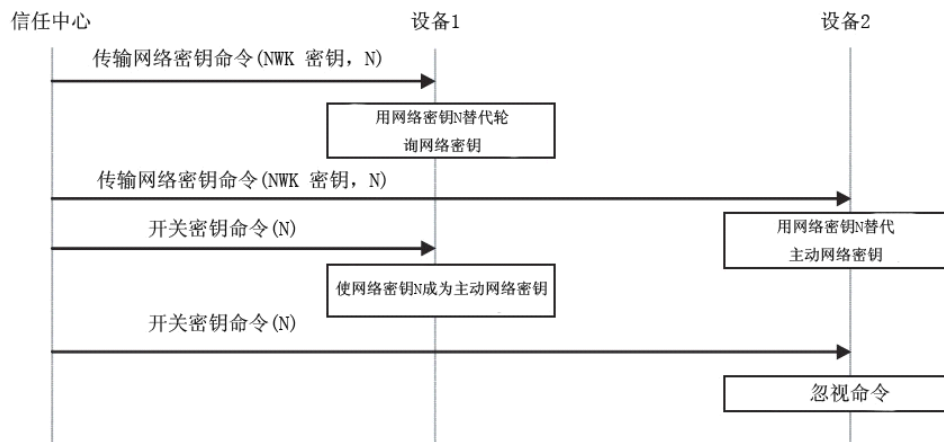


图 4.29 网络密钥更新过程例子

4.6.3.4.3 信息序列图表

如图 4.29 展示了两个设备成功执行网络密钥更新程序的一个例子。在这个例子中,信任中心传送网络密钥到设备 1 和 2,序列数为 N 。在这个例子中,设备 1 是一个 FFD,可存储两个网络密钥,活动的和备用的,设备 2 是一个 RFD,只能存储一个网络密钥。在收到传输密钥命令后,设备 1 用新的网络密钥取代其备用网络密钥;然而设备 2 必须用新的密钥取代其活动的网络密钥。下一步,在收到更换密钥命令后,设备 1 使新的网络密钥成为其活动的网络密钥;但是设备 2 只有一个活动的网络密钥,因此它会忽略这命令。

4.6.3.5 端对端应用的密钥建立

为了发起者和响应者之间的端对端应用程序的安全，当建立一个连接密钥时，一个发起设备、一个信任中心和一个响应设备应该遵守本小节的步骤。

4.6.3.5.1 设备操作

发起设备应发出 APSME-REQUEST-KEY.request 原语和响应设备建立一个连接密钥开始这个程序。DstDevice 参数应设置为其信任中心地址，KeyType 参数应设置为 0x02（即应用程序密钥），PartnerAddress 参数应设置为响应设备的地址。

4.6.3.5.1.1 接收连接密钥

在接收 APSME-TRANSPORT-KEY.indication 原语之后，且 KeyType 参数设置为 0x03（即应用程序连接密钥），只有 PartnerAddress 参数等于 AIB 中 *apsTrustCenterAddress* 属性，设备才可以接受 TransportKeyData 参数作为一个连接密钥连接 PartnerAddress 指定的设备。如果接受，AIB 表中 *DeviceKeyPairSet* 属性应该被更新。为 PartnerAddress 参数指定的设备在 AIB 中创建一个密钥对描述符（或如果存在就更新），DeviceAddress 元素设置为 PartnerAddress 参数，LinkKey 元素设置为 TransportKeyData 参数中的连接密钥，OutgoingFrameCounter 和 IncomingFrameCounter 元素设置为 0。

4.6.3.5.1.2 接收主密钥

在接收到 APSME-TRANSPORT-KEY.indication 原语之后，且 KeyType 参数设置为 0x02（即应用程序主密钥），只有 SrcAddress 参数等于 AIB 中 *apsTrustCenterAddress* 属性，设备才可以接受 TransportKeyData 参数作为一个主密钥连接 PartnerAddress 子参数指定的设备。如果接受，AIB 中 *DeviceKeyPairSet* 属性应该被更新。为 PartnerAddress 参数指定的设备在 AIB 中创建一个密钥对描述符（或如果存在就更新），DeviceAddress 元素设置为 PartnerAddress 参数，MasterKey 元素设置为 TransportKeyData 参数中的主密钥，OutgoingFrameCounter 和 IncomingFrameCounter 元素设置为 0。

下一步, 如果 APSME-TRANSPORT-KEY.indication 原语 TransportKeyData 参数的子参数 Initiator 属性是 TRUE, 设备会发出 APSME-ESTABLISH-KEY.request 原语。ResponderAddress 参数设置为 TransportKeyData 参数的子参数 PartnerAddress; UseParent 设置为 FALSE, KeyEstablishmentMethod 应设置为 0x00 (即 SKKE)。

在收到 APSME-ESTABLISH-KEY.indication 原语之后, 发起设备应该通知响应设备, 愿意建立连接密钥。如果响应设备决定建立连接密钥, 它会发送 APSME-ESTABLISH-KEY.response 原语, InitiatorAddress 参数设置为发起者设备地址, Accept 参数设置为 TRUE。否则, 设置 Accept 参数为 FALSE。

如果响应设备决定和发起者建立连接密钥, SKKE 协议会保证 APSME-ESTABLISH-KEY.confirm 原语会被发到响应者和发起者。

4.6.3.5.2 信任中心操作

在收到 APSME-REQUEST-KEY .indication 原语之后, 且 KeyType 参数应设置为 0x02 (即应用程序密钥), 信任中心的行为取决于是否已经配置为发送应用程序连接密钥和主密钥。

信任中心会发出 2 个 APSME-TRANSPORT-KEY.request 原语。如果配置为发送应用程序连接密钥, KeyType 参数应设置为 0x03 (即应用程序连接密钥); 否则, KeyType 参数应设置为 0x02 (即应用程序主密钥)。第一个原语应该设置 DestAddress 参数为请求密钥的设备地址。TransportKeyData 子参数应该做如下设置:

- PartnerAddress 子参数应该设置为 APSME-REQUEST-KEY.indication 原语 TransportKeyData 参数的 PartnerAddress 子参数。

- Initiator 子参数应该设置为 TRUE。
- Key 子参数应该设置为一个新密钥 K (主密钥或者连接密钥)。

该密钥是随机产生的。第二个原语 DestAddress 参数设置为 APSME-REQUEST-KEY.indication 原语 TransportKeyData 参数的 PartnerAddress 子参数。TransportKeyData 子参数应该做如下设置:

- PartnerAddress 子参数应该设置为请求密钥的设备地址。
- Initiator 子参数设置为 FALSE。
- Key 子参数应该设置为 K。

4.6.3.5.3 信息序列图表

图 4.30 展示了端对端应用程序密钥建立的信息序列图的一个例子。程序开始于从发起者到信任中心传输请求密钥命令。下一步，信任中心启动一个超时计时器。在这个计时器时间内（即直到它到期），信任中心会为设备对丢弃任何新的请求密钥命令，除非它们来自发起者。

信任中心现在发送传输密钥命令，包括应用程序连接密钥或者主密钥到发起者和响应设备。只有发起者的传输密钥命令将 Initiator 设置为 1（即 TRUE），因此，如果主密钥被发送，只有发起设备通过发送 SKKE-1 命令开始建立密钥协议。如果响应者决定接受和发起者建立一个密钥，SKKE 协议通过交换 SKKE-2、SKKE-3 和 SKKE-4 命令继续进行。协议完成后（或超时），协议的状态报告给发起者和应答设备的 ZDO。如果成功的话，发起者和响应者现在共用一个连接密钥，且安全通信将成为可能。

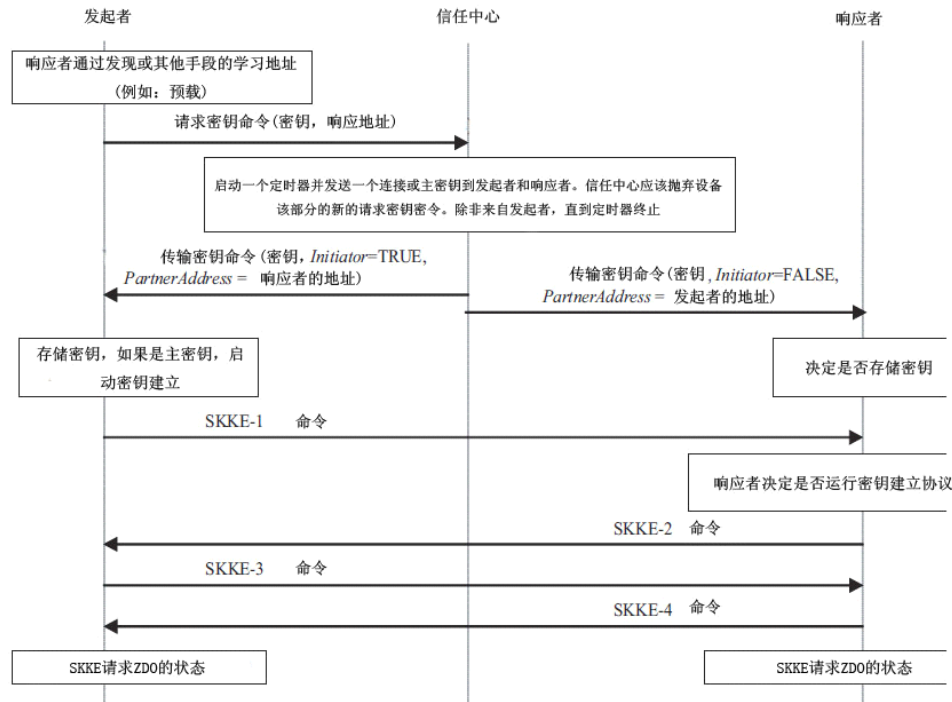


图 4.30 端到端应用程序密钥建立过程例子

4.6.3.6 网络离开

当一个设备离开网络，设备、它的路由器和信任中心将遵守本小节的步骤。

4.6.3.6.1 信任中心操作

如果信任中心想要一个设备离开，且如果信任中心不是该设备的路由器，信任中心会发出 APSME-REMOVE-DEVICE.request 原语，ParentAddress 参数设置为路由器地址，ChildAddress 设置为希望离开设备的地址。

信任中心还会被通知设备离开网络。在收到 APSME-UPDATE-DEVICE.indication 原语之后，且 Status 参数设置为 0x02 (即设备离开)，DeviceAddress 参数应该表示离开网络的设备的地址，SrcAddress 参数应该指示该设备的父设备地址。如果运行在高安全模式，信任中心会从它的网络设备清单中移除离开设备。

4.6.3.6.2 路由器操作

路由器负责接收移除设备命令并发送更新设备命令。

在收到 APSME-REMOVE-DEVICE.indication 原语之后，如果 SrcAddress 参数等于 AIB 中 *apsTrustCenterAddress* 属性，且对应于 DeviceAddress 的邻居表条目表示它有一个网络密钥，路由器应发出 NLME-LEAVE.request 原语，DeviceAddress 参数等于 APSME-REMOVEDEVICE.indication 原语的 DeviceAddress 参数，rejoin 参数设置为 0。其它域在栈 profile 中定义。路由器忽略 SrcAddress 参数不等于 AIB 中 *apsTrustCenterAddress* 属性的 APSME-REMOVE-DEVICE.indication 原语。

在收到 NLME-LEAVE.indication 原语之后，且 DeviceAddress 参数设置为它的子设备之一，路由器也不是信任中心，应该发出 APSME-UPDATE-DEVICE.request 原语：

- DstAddress 参数设置为信任中心地址；
- Status 参数设置为 0x02 (即设备离开)；
- DeviceAddress 参数设置为 NLME-LEAVE.indication 原语的 DeviceAddress 参数。

如果路由器是信任中心，它就简单地作为信任中心运行，不发出 APSME-UPDATE-DEVICE.request 原语（见 4.6.3.6.1 小节）。

4.6.3.6.3 离开设备操作

设备负责发送接收和发送离开命令。

在一个安全的 ZigBee 网络中，离开命令应该用活动网络密钥加密，发送时安全启用，安全级别见 AIB 中 *nwkSecurityLevel* 属性。

在一个安全的 ZigBee 网络中，离开命令只有用活动网络密钥加密后才能接收和处理，接收和处理中安全启用，安全级别见 AIB 中 *nwkSecurityLevel* 属性。

4.6.3.6.4 信息序列图表

图 4.31 显示一个信息序列图实例，其中一个信任中心要求路由器从网络移除它的一个子设备。如果信任中心希望一个设备离开，且如果信任中心不是该设备的路由器，该信任中心必须发给路由器一个移除设备的命令，带有它希望离开网络的设备的地址。在一个安全的网络中，如果存在的话，移除设备命令应用程序连接密钥加密；否则须用活动网络密钥加密。在收到移除设备的命令之后，路由器应发送一个离开命令到设备，通知其离开网络。

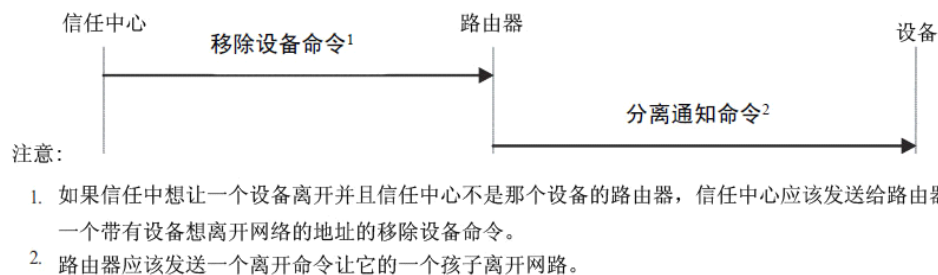


图 4.31 移除设备过程例子

图 4.32 展示了一个信息序列图实例，其中一个设备通知其路由器，设备正在离开网络。在本实例中，设备发送一个离开命令（用活动网络密钥加密）到它的路由器。然后路由器发送一个设备更新命令到信任中心。在一个安全的网络中，如果存在的话，更新设备命令必须用连接密钥加密，或用活动网络密钥加密。

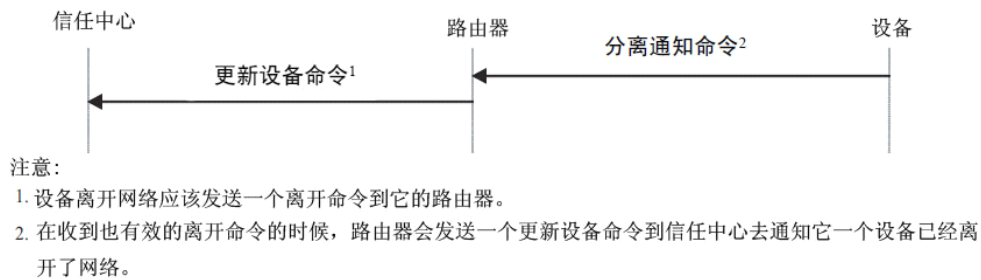


图 4.32 设备离开过程例子

4.6.3.6.5 信任中心操作

在本规范中，其它路由设备上的信任中心没有特殊的额外的作用。

正常情况下，当运行在标准的安全模式下，信任中心不能发出或重发一个未加密的网络密钥给孤设备，除非是为了认证程序激活。

正常情况下，当运行在高安全模式下，没有为认证程序激活时，信任中心可以有选择地发出或重发当前的或更新后的网络密钥给孤设备。在这种情况下，密钥应使用 APSME-TRANSPORT-KEY.request 原语传输，并应适当地加密。发起这样一个更新超过了本规范的范围。

4.6.3.7 命令通道

设备应该遵守本小节所述的程序，允许信任中心和没有当前网络密钥的远程设备进行加密通信。

4.6.3.7.1 信任中心操作

为了嵌入命令到通道命令，信任中心应该首先使用 4.4.1.1 小节中的安全保护，然后如果安全处理成功，加密的命令帧会嵌入到通道命令帧，如下：

- 1 APS 头域应该设置为要嵌入的命令的 APS 头域的值。
- 2 目标地址域应该设置为目标设备的 64 位扩展地址。
- 3 通道辅助帧域应该设置为已加密命令的辅助帧，作如下改变：

- 扩展临时子域应该设置为1。
 - 源地址域应该设置为信任中心的64位扩展地址。
 - 通道命令应该设置为嵌入命令的加密负载。
- 然后通道命令应该发送给目标设备的父设备或者其它邻居。

4.6.3.7.2 路由器操作

在收到一个APS通道命令之后，路由器应该提取嵌入命令如下：

- 1 APS头域应该设置为通道命令APS头域的值。
- 2 辅助帧域应该设置为通道命令的通道辅助帧域的值。
- 3 APS负载域应设置为通道命令的通道命令域。

提取的命令应该发送到目标地址域指明的目的地址，通过发出NLDE-DATA.request原语，安全功能禁用。

4.6.3.7.3 目标操作

在收到 APS 层加密的信息，且在 APS 辅助帧里有扩展临时，信息应该按通常处理，除非信息不能在 APS 重复拒绝表中查阅，或增加到 APS 重复拒绝表。

4.6.3.8 允许配置表

表 4.41 的允许配置表指明了哪个设备被授权去执行某种类型的命令，并决定每种情况下是否需要使用连接密钥。期望表中的值由上层填充，为了下面的输入命令被检查。如果一个输入命令不被允许，那么它就不能实施，应该给源设备发送一个错误信息。

表 4.41 允许配置表的元素

名称	类型	范围	描述	默认
ModifyPermissionsConfigurationTable	允许描述符, 见表 4.42	-	允许修改本表	-
NetworkSettings	允许描述符, 见表 4.42	-	允许配置网络启动和参加参数, 包括直接参加, 允许参加, 离开, 网络维护和复位。	-
ApplicationSettings	允许描述符, 见表 4.42	-	允许配置应用程序设置, 包括绑定和组, 以及其它的应用配置命令	-
SecuritySettings	允许描述符, 见表 4.42	-	允许配置安全设置	-
ApplicationCommands	允许描述符, 见表 4.42	-	允许发出应用层命令	-
保留	允许描述符, 见表 4.42	-	保留	-
保留	允许描述符, 见表 4.42	-	保留	-
SKKEWithMasterKey	允许描述符, 见表 4.42	任何有效的64位地址或者 0xFFFFFFFF FFFFFFFF FF	允许使用 SKKE 产生一个新的信任中心连接密钥。	-

表 4.42 允许描述符元素

名称	类型	范围	描述	默认
AuthorizedDeviceList	设备地址表	-	每个被允许设备的有效 64 位地址列表，或一个条目 0xFFFFFFFFFFFFFFFF 表示允许所有设备	0xFFFFFFFF FFFFFFFF FF
LinkKeyRequired	Boolean	TRUE FALSE	指示在接受命令时是否需要连接密钥	FALSE

4.6.3.8.1 服务

允许配置表中的这组服务详细描述如下：

- ModifyPermissionsConfigurationTable 条目对上层可用，来决定是否允许改变这个表格。
- NetworkSettings 条目应该被 ZDO 使用，由上层选择性使用，决定是否允许某些网络相关的配置命令。适用于本表的 ZDO 命令是 Mgmt_Direct_Join_req，Mgmt_Permit_Join_req 和 Mgmt_Leave_req 命令。
- ApplicationSettings 条目应该被 ZDO 使用，由上层选择性使用，决定是否允许某些应用相关的配置命令。适用于本表的 ZDO 命令是 Bind_req 和 Unbind_req 命令。
- SecuritySettings 条目对于上层可用，决定是否允许某些安全相关的配置命令。
- ApplicationCommands 条目对于上层可用，决定是否允许具体的应用命令。
- SKKEWithMasterKey 条目可选用于限制哪个设备被允许启动一个 SKKE 事务，使用信任中心主密钥，以产生一个新的信任中心连接密钥。这一表条目在没有执行 SKKE 时对设备没有影响。

4.6.3.8.2 使用细节

输入命令应该调用一个规定的允许检查，如果失败，输入命令不能执行。允许检查失败发送的任何错误信息在相关命令的规范中定义。

每个条目的 `AuthorizedDevicesList` 应该包含授权设备的 IEEE 地址列表，或者只包含值 `0xFFFFFFFFFFFFFFFF`，指示如果适当加密的话，所有设备被允许去执行相应的命令。

如上规定，除了本规范内委托的任何检查，表格的条目可以由上层检查。上层检查和任何相关错误信息的细节在本规范之外。

假定用信任中心连接密钥加密的输入命令来自信任中心，被免除这里所述的所有检查的命令也是这样，如果适当加密的话，总是被允许的。这一免除对没有信任中心连接密钥的设备没有影响。

本表格是可选的，如果支持的话，`ModifyPermissionsConfigurationTable` 和 `SKKEWithMasterKey` 条目应包括一个空集，表示这些设备不能被使用，所有其它条目应该使允许描述符的 `AuthorizedDeviceList` 条目设置为一个值 `0xFFFFFFFFFFFFFFFF`，表示没有限制设备。

附录 A

CCM*操作模式

CCM*是加密和验证分组密码模式的一般结合。CCM*只为使用 128 位块大小的分组密码而定义，比如 AES-128 [B8]。CCM* 的思想可以扩展到其它块大小，但是这将需要进一步的定义。

CCM*模式符合原始的 CCM 模式规范[B21]，它用于需要验证且可能加密的信息，但是也提供支持只需要加密的信息。和 CCM 模式相比较，CCM*模式只需要一个密钥。CCM 模式的安全证明（[B22]和[B23]）也适用于这里描述的 CCM*模式。CCM*模式的设计考虑了[B24]的结果，从而可以允许它安全用于使用可变长度的验证标签的执行环境，而不仅仅是固定长度的验证标签，这一点是很有益的。

先决条件：以下是一般 CCM*模式操作的先决条件：

- 1 必须选择一个分组密码加密函数 E ，块大小为 128 位。已选加密函数所用密钥的长度用 $keylen$ 来表示，以位为单位。
- 2 必须选择一个二进制字符串的一个字节固定表示法（例如，最高有效位优先或最低有效位优先的顺序）。
- 3 必须选择信息长度域的长度 L ，以字节为单位。 L 的有效值是整数 2, 3, ..., 8（值 $L=1$ 保留）。
- 4 必须选择验证域的长度 M ，以字节为单位。 M 的有效值是整数 0, 4, 6, 8, 10, 12, 14 和 16。（值 $M=0$ 对应于禁用验证，因为这样验证域是空字符串。）

A.1 符号和表示法

在本规范中，必须指定整数的表示法为字节字符串。所有整数必须表示为最高有效位优先的顺序的字节字符串。这一表示法符合 ANSI X9.63-2001 [B7]的 4.3 节的约定。

A.2 CCM*模式加密和验证转换

CCM*模式按顺序转发有关执行的转换，即输入转换（A.2.1），验证转换（A.2.2）和加密转换（A.2.3）。

输入：CCM*模式转发转换作为输入：

- 1 一个长度为 $keylen$ 位的位字符串 Key ，用作密钥。每个实体必须可以在实体本身的限制内获得这个密钥，并获得它的组成员共享的预定密钥。
- 2 一个 $15-L$ 子节的临时 N 。在任何加密密钥 Key 的范围内，该临时值必须是唯一的。
- 3 一个长度为 $l(m)$ 字节的字节字符串 m ，其中 $0 \leq l(m) < 28L$ 。
- 4 一个长度为 $l(a)$ 字节的字节字符串 a ，其中 $0 \leq l(a) < 2^{64}$ 。

临时的 N 必须为 M 的潜在值进行编码，这样可以从 N 中唯一确定一个实际使用的 M 值。临时 N 的准确格式在本规范的范围之外，且必须由 CCM*模式的实际执行环境来确定和固定下来。

注意：临时 N 的准确格式留给应用程序，以允许特定设置中简化硬件和软件实现。CCM*模式的实际执行可能会限制 M 的值，在加密密钥 Key 的整个生命周期是一个一般 CCM*模式允许的那些严格的子集。如果是这样的话，临时 N 的格式必须按以上所述，这样可以从 N 中唯一确定该子集中使用的一个 M 值。特别地，如果 M 固定了且值 $M \neq 0$ 不允许，那么对 N 就没有限制，在这种情况下，CCM*模式降为 CCM 模式。

A.2.1 输入转换

本步骤涉及输入字符串 a 和 m 的转换，分别到字符串 $AuthData$ 和 $PlaintextData$ ，分别用于验证转换和加密转换。

本步骤涉及以下步骤，顺序为：

- 1 形成字节字符串表示的 $L(a)$ ，字节字符串 a 的长度为 $l(a)$ ，如下：
 - a 如果 $l(a)=0$ ，那么 $L(a)$ 是空字符串。
 - b 如果 $0 < l(a) < 2^{16}-2^8$ ，那么 $L(a)$ 是 $l(a)$ 的 2 字节编码。
 - c 如果 $2^{16}-2^8 \leq l(a) < 2^{32}$ ，那么 $L(a)$ 是字节 0xff，字节 0xfe 和 $l(a)$ 的 4 字节编码的右串联。
 - d 如果 $2^{32} \leq l(a) < 2^{64}$ ，那么 $L(a)$ 是字节 0xff，字节 0xff 和 $l(a)$ 的 8 字节编码的右串联。
- 2 右串联字节字符串 $L(a)$ 和字节字符串 a 本身。注意由此产生的字符串包含 $l(a)$ 和以一个可逆方式编码的 a 。
- 3 通过把产生的字符串和全零字节的最小非负数右串联，形成填补信息 $AddAuthData$ ，这样字节字符串 $AddAuthData$ 的长度可以被 16 整除。
- 4 通过把字节字符串 m 和全零字节的最小非负数右串联，形成填补信息 $PlaintextData$ ，这样字节字符串 $PlaintextData$ 的长度可以被 16 整除。
- 5 包含字节字符串 $AddAuthData$ 和 $PlaintextData$ ，形成信息 $AuthData$ 。

$$AuthData = AddAuthData || PlaintextData$$

A.2.2 验证转换

上面建立的数据 $AuthData$ 必须如下的标记转换进行标记：

- 1 形成 1 字节的 $Flags$ 域，包含 1 位的 $Reserved$ 域，1 位的 $Adata$ 域，和 3 位表示的整数 M ，如下：

$$Flags = Reserved || Adata || M || L$$

这里，1 位的 $Reserved$ 域保留作未来的扩展，必须设置为“0”。如果 $l(a)=0$ ，1 位的 $Adata$ 域设置为“0”，如果 $l(a)>0$ ，设置为“1”。 L 域是 3 位表示的整数 $L-1$ ，按照最高有效位优先的顺序。如果 $M>0$ ， M 域是 3 位表示的整数 $(M-2)/2$ ，如果 $M=0$ ，是 3 位表示的整数 0，按照最高有效位优先的顺序。

- 2 形成 16 字节的 B_0 域, 包含上面定义的 1 字节的 $Flags$ 域, 15- L 字节的临时域 N 和 L 字节表示的长度域 $l(m)$, 如下:

$$B_0 = Flags || \text{临时域 } N || l(m)$$

- 3 解析信息 $AuthData$ as $B_1 || B_2 || \dots || B_t$, 其中每个信息块 B_i 是一个 16 字节的字符串。CBC-MAC 的值 X_{t+1} 定义为:

$$X_0 := 0^{28}; X_{i+1} := E(Key; X_i - B_i) \text{ for } i=0, \dots, t.$$

这里, $E(K, x)$ 是密码文字, 使用建立的分组密码加密函数 E 和密钥 Key 进行的明文 x 加密的结果; 字符串 0^{28} 是 16 位字节的全零位字符串。

验证标记 T 是忽略 CBC-MAC 的值 X_{n+1} 的所有字节, 但是除了最左边的 M 字节, 从而计算出来的。

A.2.3 加密转换

A.2.1 小节 (步骤 4) 建立的数据 $PlaintextData$ 和 A.2.2 小节 (步骤 3) 建立的验证标记 T 必须使用以下加密转换进行加密:

- 1 形成 1 字节的 $Flags$ 域, 包含两个 1 位的 $Reserved$ 域, 和 3 位表示的整数 0 和 L , 如下:

$$Flags = Reserved || Reserved || 0 || L$$

这里, 两个 1 位的 $Reserved$ 域保留用于未来的扩展, 必须设置为“0”。 L 域是 3 位表示的整数 $L-1$, 按照最高有效位优先的顺序。“0”域是 3 位表示的整数 0, 按照最高有效位优先的顺序。

定义 16 字节的 A_i 域, 包含上面定义的 1 字节的 $Flags$ 域, 15- L 字节的临时 N , 和 L 字节表示的整数 i , 如下:

$$A_i = Flags || \text{Nonce } N || \text{Counter } i, \text{ for } i=0, 1, 2, \dots$$

注意这个定义可保证所有的 A_i 域有别于实际使用的 B_0 域，因为这些有一个 M 非零编码的 $Flags$ 域，而所有 A_i 域有一个全零编码的整数 0（参见 A.2.2 小节的步骤 1）。

解析信息 $PlaintextData$ 为 $M_1 \| \dots \| M_t$ ，其中每个信息块 M_i 是一个 16 字节的字符串。

密文块 $C_1, \dots, C_t$ 定义为：

$$C_i := E(Key, A_i) \oplus M_i \text{ for } i=1, 2, \dots, t$$

字符串 $Ciphertext$ 是忽略字符串 $C_1 \| \dots \| C_t$ 所有字节，但是除了最左边的 $l(m)$ 字节的结果。

定义 16 字节的加密块 S_0 为：

$$S_0 := E(Key, A_0)$$

2 加密的验证标记 U 是把 S_0 最左边的 M 字节和验证标记 T 异或后得到的字符串。

输出：如果以上任何操作失败，那么输出“无效”。否则，输出加密的信息 $Ciphertext$ 和加密的验证标记 U 的右串联。

A.3 CCM*模式解密和验证检查转换

输入：CCM*进行逆转换作为输入：

- 1 一个位字符串 Key ，长度为 $keylen$ 位，用作密钥。每个实体必须可以在实体本身的限制内获得这个密钥，并获得它的组成员共享的预定密钥。
- 2 一个 $15-L$ 字节的临时 N 。在任何加密密钥 Key 的范围内，该临时值必须是唯一的。
- 3 一个长度为 $l(c)$ 字节的字节字符串 c ，其中 $0 \leq l(c)-M < 2^{8L}$ 。
- 4 一个长度为 $l(a)$ 字节的字节字符串 a ，其中 $0 \leq l(a) < 2^{64}$ 。

A.3.1 解密设备

解密转换涉及以下步骤，顺序为：

- 1 解析信息 c 为 $C\|U$ ，其中最右边的字符串 U 是一个 M 字节的字符串。如果这个操作失败，输出“无效”并停止。 U 是所谓的加密验证标记。注意最左边的字符串 C 长度为 $l(c)-M$ 字节。
- 2 通过把字符串 C 和全零字节的最小非负数右串联，形成填补信息 $CiphertextData$ 。这样子字节字符串 $CiphertextData$ 长度可以被 16 整除。
- 3 使用 A.2.3 小节的加密转换，以数据 $CipherTextData$ 和标记 U 作为输入。
- 4 解析应用这个转换产生的输出字符串 $m\|T$ ，其中最右边的字符串 T 是一个 M 字节字符串。 T 是所谓的验证标记。注意最左边的字符串 m 长度为 $l(c)-M$ 字节。

A.3.2 验证检查转换

验证检查转换涉及以下步骤：

- 1 使用 A.2.1 小节的输入转换，字符串 a 和 A.3.1 小节（步骤 4）建立的字节字符串 m 作为输入，形成信息 $AuthData$ 。
- 2 信息 $AuthData$ 作为输入，使用 A.2.2 小节的验证转换。
- 3 比较这个转换产生的输出标记 $MACTag$ 和 A.3.1 小节（步骤 4）建立的标记 T 。如果 $MACTag=T$ ，输出“有效”；否则，输出“无效”并停止。

输出：如果以上的任何验证失败，那么输出“无效”并拒绝字节字符串 m 。否则，接受子字节字符串，并接受组成员共享的密钥之一作为 m 的源。

A.4 限制

所有的执行必须限制以一个密钥加密数据的总数。对于 CBC-MAC 和 CTR 加密操作，CCM*加密转换总共只能激活不超过 2^{61} 个的分组密码加密函数操作。

在 CCM*解密中，必须在释放任何信息，比如 *Plaintext*，之前证实（截断的）CBC-MAC。如果 CBC-MAC 验证失败，只披露 CBC-MAC 验证失败的事实；所有其它信息必须被销毁。

附录 B

安全建立模块

本附录指明了用于执行本标准安全协议的加密原语和机制。

B.1 对称密钥加密建立模块

以下对称密钥加密原语和数据元素是为使用本标准指定的所有安全处理操作而定义的。

B.1.1 分组密码

本规范使用的分组密码必须是高级加密标准 AES-128，如 FIPS Pub 197 所定义。这个分组密码有一个密钥大小 $keylen$ ，它等于块的大小，以位为单位，比如， $keylen=128$ 。

B.1.2 操作模式

本规范使用的分组密码的操作模式必须是 CCM* 的操作模式，如 A.2.3 小节所定义，且带有以下实例：

- 1 每个实体必须使用 B.1.1 小节指明的分组密码 E ；
- 2 所有字节必须按照“约定和缩略语”指定的来表示；
- 3 参数 L 必须为整数值 2；
- 4 参数 M 必须为以下整数值：0，4，8 或 16 之一。

B.1.3 加密哈希函数

本规范的加密哈希函数必须是基于 B.6 节指明的加密哈希函数的分组密码，且带有以下实例：

- 1 每个实体必须使用 B.1.1 小节指明的分组密码 E ；
- 2 所有整数和字节必须按照 1.2.1 小节指明的来表示。

Matyas-Meyer-Oseas 哈希函数（B.6 小节指定的）有一个信息消化大小 $hashlen$ ，等于建立的分组密码的块大小，以位为单位。

B.1.4 信息验证的键控哈希函数

本规范使用的键控哈希信息验证代码（HMAC）必须是 FIPS Pub 198 [B9]指明的 HMAC，且带有以下实例：

- 1 每个实体必须使用 B.1.3 小节指明的加密哈希 H 函数；
- 2 块大小 B 必须是整数值 16（这个块大小指定了数据完整性密钥的长度，以字节为单位，用于键控哈希函数，比如，它使用一个 128 位的数据完整性密钥）；
- 3 HMAC 函数的输出大小 $HMAClen$ 必须是和 B.1.3 指明的信息消化参数 $hashlen$ 相同的整数值。

B.1.5 信息验证的专门键控哈希函数

本规范使用的专门键控哈希信息验证代码必须是 B.1.4 小节指明的键控哈希信息验证代码。

B.1.6 质疑域参数

本规范使用的质疑域参数必须是 B.3.1 小节规定的，且带有以下实例：
($minchallenge$, $maxchallenge$) = (128, 128)。

所有质疑必须使用 B.4 小节指明的证实原语进行证实。

B.2 密钥协议方案

B.2.1 对称密钥的密钥协议方案

本标准的对称密钥的密钥协议必须使用带有密钥确认方案的完全对称密钥，且带有以下实例：

- 1 每个实体必须按照 B.1.4 小节指明的确定。
- 2 每个实体必须使用 B.1.5 小节指明的专门 HMAC 方案。
- 3 每个实体必须使用 B.1.3 小节指明的加密哈希函数。
- 4 参数 *keydatalen* 必须和 B.1.1 小节指明密钥大小参数 *keylen* 具有相同的整数值。
- 5 参数 *SharedData* 必须是空字符串；参数 *shareddatalen* 必须为整数值 0。
- 6 B.7.1 小节和 B.7.2 小节指明的可选参数 *Text₁* 和 *Text₂* 必须都是空字符串。
- 7 每个实体必须使用 B.1.6 小节指明的质疑域参数。
- 8 所有字节必须按照 1.2.1 小节所述的进行表示。

B.3 质疑域参数生成和验证

本节指明了用于生成和验证质疑域参数的原语。

质疑域参数给一个方案预计的位的长度施加了限制。这样，这确定了质疑的熵的一个边界，从而，确定了这些质疑使用的加密方案的安全界限。在大多数方案中，质疑域参数将只接受一个固定长度的质疑（比如，128 位质疑）。但是，可以定义一个这样的质疑域参数，即可以接收变长的质疑。后者在希望参与加密方案的实体的板上随机数生成器可能坏了的情况下是有用的。允许参与一个方案的实体都贡献足够长的输入，使得每个这些输入都可为目前的方案提供足够的熵。

在本标准中，质疑域参数将被一些使用一个标准方案的实体共享。质疑域参数可以是公共的；系统安全不依赖这些作为秘密的参数。

B.3.1 质疑域参数生成

质疑域参数必须使用以下例程生成。

输入：这个例程不需要任何输入。

行为：采取以下行为：

- 1 选择两个非负整数 *minchallengelen* 和 *maxchallengelen*，且 $\text{minchallengelen} \leq \text{maxchallengelen}$ 。

输出：质疑域参数 $D = (\text{minchallengelen}, \text{maxchallengelen})$ 。

B.3.2 质疑域参数验证

质疑域参数必须使用以下例程验证。

输入：所谓的一组质疑域参数 $D = (\text{minchallengelen}, \text{maxchallengelen})$ 。

行为：进行以下检查：

- 1 检查 *minchallengelen* 和 *maxchallengelen* 是非负整数。
- 2 检查 $\text{minchallengelen} \leq \text{maxchallengelen}$ 。

输出：如果以上任何验证失败，那么输出“无效”并拒绝质疑域参数。否则，输出“有效”并接收质疑域参数。

B.4 质疑验证原语

它用于检查本标准中被一个方案使用的一个质疑是否有足够的长度（比如，太短的信息被丢弃，因为没有足够的熵）。

输入：验证转换的输入是一组有效的质疑域参数 $D = (\text{minchallengelen}, \text{maxchallengelen})$ ，以及位字符串 *Challenge*。

行为：采取以下行为：

- 1 计算位字符串 *Challenge* 的位长度 *challengelen*。

- 2 证实 $challenge_n \in [minchallenge_n, maxchallenge_n]$ 。(即证实质疑有一个合适的长度。)

输出：如果以上验证失败，那么输出“无效”并拒绝质疑。否则，输出“有效”并接收质疑。

B.5 密钥生成（SKG）原语

本节指明了本标准中对称密钥的密钥协议方案使用的 SKG 原语。

当所有的质疑共享同一个质疑域参数时，本原语从一个实体 U_1 拥有的一个质疑，和一个实体 U_2 拥有的一个质疑得到一个共享的秘密值。如果这两个实体都正确执行了这个原语，且以相应的质疑作为输入，将产生相同的共享的秘密值。

共享的秘密值将按照如下的方法计算：

先决条件：下面是使用 SKG 原语的先决条件：

- 1 每个实体必须绑定一个唯一的标识符（比如，可区分的名称）。所有标识符必须具有相同长度 $entlen$ 位的位字符串。实体 U_1 的标识符将由位字符串 U_1 表示。实体 U_2 的标识符将由位字符串 U_2 表示。
- 2 必须选择一个专门 MAC 方案，具有 ANSI X9.63-2001 [B7] 的 5.7.1 节所述的标记转换。专门 MAC 方案使用的密钥长度由 $mackeylen$ 表示，以位为单位。

输入：SKG 原语采用如下输入：

- 一个位字符串 $MACKey$ ，长度为 $mackeylen$ 位，用作建立的专门 MAC 方案的密钥。
- U_1 拥有的一个位字符串 QEU_1 。
- U_2 拥有的一个位字符串 QEU_2 。

行为：采取以下行为：

- 1 形成位字符串，包含 U_1 的标识符， U_2 的标识符， U_1 的质疑相应的位字符串 QEU_1 ，和 U_2 的质疑相应的位字符串 QEU_2 ：

$$MacData = U_1 || U_2 || QEU_1 || QEU_2$$

- 2 为 *MacData* 计算标记 *MacTag*，它在密钥 *MacKey* 之下，使用建立的专门 MAC 方案的标记转换：

$$MacTag = MAC_{MacKey}(MacData)$$

- 3 如果标记转换输出“无效”，则输出“无效”并停止。
- 4 设置 $Z = MacTag$ 。

输出：位字符串 Z 作为共享的秘密值。

B.6 基于分组密码的加密哈希函数

本节指明了 Matyas-Meyer-Oseas 哈希函数，一个基于分组密码的加密哈希函数。我们为分组密码，比如 AES-128，定义了这个哈希函数，其中有一个密钥大小等于块大小，且有一个特定选择的固定初始化向量 IV （我们取 $IV=0$ ）。对于 Matyas-Meyer-Oseas 哈希函数更多的一般定义，请参考[B19]的 9.4.1 节。

先决条件：下面是 Matyas-Meyer-Oseas 哈希函数操作的先决条件：

- 1 必须选择一个分组密码加密函数 E ，有一个密钥大小等于块大小。Matyas-Meyer-Oseas 哈希函数有一个信息消化大小，等于建立的加密函数的块大小。它在长度小于 2^n 的位字符串上可以操作，其中 n 是建立的分组密码的块大小，以字节为单位。
- 2 必须选择一个二进制字符串或字节字符串，固定表示为整数。

输出：Matyas-Meyer-Oseas 哈希函数的输出如下：

- 1 一个位字符串 M ，长度为 l 位，其中 $0 \leq l < 2^n$ 。

行为：哈希值必须按如下方法获得：

- 1 根据以下方法填充信息 M ：
 - a 右串联信息 M ，首先是二进制位组成的“1”，其次用 k “0”位，其中 k 是以下方程最小非负数的解：

$$l + l + k \equiv 7n \pmod{8n} \tag{1}$$

- b 通过右串联产生的字符串，即等于二进制表示的整数 l 的 n 位字符串，形成填补信息 M' 。

- 2 解析填补信息 M 为 $M_1 || M_2 || \dots || M_t$, 其中每个信息块 M_i 是一个 n 字节的字符串。
- 3 输出 $Hash_i$ 定义为

$$Hash_0 = 0^{8n}; Hash_j = E(Hash_{j-1}, M_j) \oplus M_j \text{ for } j=1, \dots, t \quad (2)$$

这里, $E(K, x)$ 是加密明文 x 产生的密文, 使用建立的分组密码加密函数 E , 且带有密钥 K ; 字符串 0^{8n} 是 n 字节的全零位字符串。

输出: 位字符串 $Hash_i$ 作为哈希值。

注意加密哈希函数只能在长度小于 2^n 位的位长度上操作, 其中 n 是建立的分组密码的块大小 (或密钥大小), 以字节为单位。比如, AES-128 的 Matyas-Meyer-Oseas 哈希函数函数只能在长度小于 2^{16} 位的位字符串上操作。假定所有哈希函数调用都是在长度小于 2^n 位的位字符串上。尝试在超过 2^n 位的位字符串上调用哈希函数的任何方案会输出“无效”并停止。

B.7 对称加密验证密钥协议方案

本节指明了完全对称密钥的密钥协议, 且带有密钥确认方案。一个 MAC 方案用于提供一个密钥确认。注意所有密钥交换和随机质疑都应在数据字符串范围之内, 按照网络传输的顺序假定。⁴⁹

图 B.1 说明了带有密钥确认方案的完全对称密钥的密钥协议涉及使用的信息。

49. CCB #785

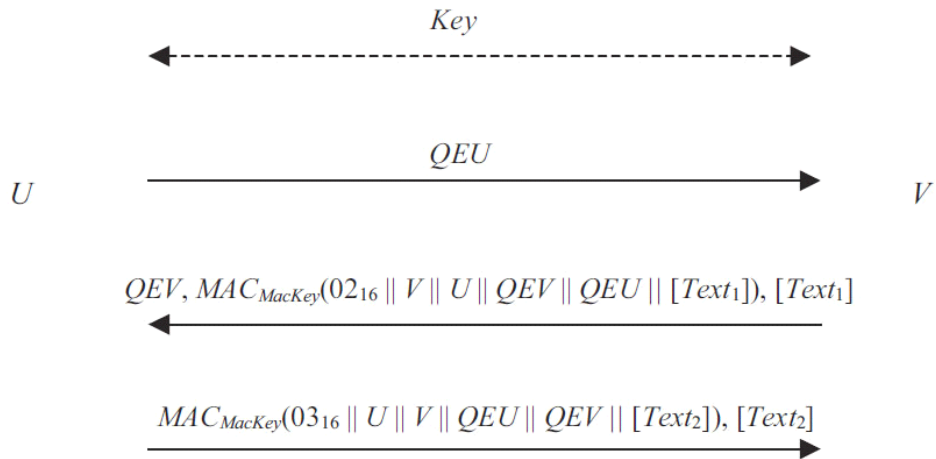


图 B.1 对称密钥验证密钥协议方案

方案是一个“对称的”方案，这样规定了两个转换。如果 U 是协议的发起者， U 使用 B.7.1 小节指明的转换来同意和 V 加密数据，如果 V 是协议的响应者， V 使用 B.7.2 小节指明的转换来同意和 U 加密数据。

发起者角色和响应者角色之间的本质区别仅仅是发起者发送第一个质疑的传递。

如果 U 执行发起者转换，且 V 执行响应者转换，以共享的秘密键控材料作为输入，那么 U 和 V 将计算相同的键控数据。

先决条件： 以下是使用该方案的先决条件：

- 1 每个实体有一个系统质疑域参数的验证副本 $D=(minchallenge, maxchallenge)$ 。
- 2 每个实体必须可以获得一个位字符串 Key ，长度为 $keylen$ 位，用作密钥。双方必须有证据证明，在实体本身和涉及对称密钥验证密钥协议方案的其它实体的限制范围内，可以获得这个密钥。
- 3 每个实体必须绑定一个唯一的标识符（比如，可区分的名称）。所有标识符必须是具有相同长度 $entlen$ 位的位字符串。实体 U 的标识符将由位字符串 U 表示。实体 V 的标识符将由位字符串 V 表示。
- 4 每个实体必须决定使用 ANSI X9.63-2001 [B7] 的 5.7 节指定的哪个 MAC 方案。选定的 MAC 方案使用的密钥长度由 $mackeylen$ 表示，以位为单位。
- 5 必须选择一个加密哈希函数使用，它带有密钥验证功能。

- 6 必须选择一个专门 MAC 方案使用,它带有秘密的密钥生成原语,和 ANSI X9.63-2001 [B7]的 5.7.1 节指明的标记转换。专门 MAC 方案使用的密钥长度由 $keylen$ 表示,以位为单位。
- 7 必须为二进制字符串选择一个字节的固定表示法。(比如,最高有效位优先的顺序或最低有效位优先的顺序)。

B.7.1 发起者转换

如果 U 是协议的发起者, U 必须执行以下转换来同意和 V 加密数据。 U 必须获得 V 的标识符的一个验证副本, 以及和 V 共享的静态秘密密钥 Key 的一个验证副本。

输入: 发起者转换的输入是:

- 1 一个整数 $keydatalen$, 要生成的键控数据的长度, 以位为单位。
- 2 (可选的) 一个位字符串 $SharedData$, 长度为 $shareddatalen$ 位, 包含 U 和 V 共享的一些数据。
- 3 (可选的) 一个位字符串 $Text_2$, 包含 U 给 V 提供的另外一些数据。

组成: 发起者转换使用 ANSI X9.63-2001 [B7]的 5.3 节的质疑生成原语, B.3.2 小节的质疑验证原语, B.5 小节的 SKG 原语, ANSI X9.63-2001 [B7]的 5.6.3 节密钥获得函数, 和 ANSI X9.63-2001 [B7]的 5.7 节的 MAC 方案之一。

行为: 密钥数据必须按以下方式获得:

- 1 使用 ANSI X9.63-2001 [B7]的 5.3 节的质疑生成原语, 为质疑域参数 D 生成一个质疑 QEU 。发送 QEU 给 V 。
- 2 然后从 V 接收 V 拥有的一个所谓质疑 QEV 。如果这个值没有接收, 输出“无效”并停止。
- 3 证实 QEV 对于 B.3.2 小节指明的质疑域参数 D 是一个有效的质疑。如果验证原语拒绝该质疑, 输出“无效”并停止。
- 4 使用 B.5 小节的 SKG 原语, 从 U 拥有的质疑 $QI=QEU$ 和 V 拥有的 $Q2=QEV$ 获得一个共享的秘密位字符串 Z , 作为共享的密钥 Key 使用。如果 SKG 原语输出“无效”, 输出“无效”并停止。



- 5 使用 ANSI X9.63-2001 [B7]的 5.6.3 节的密钥获得函数和建立的哈希函数，从共享的秘密值 Z 和共享的数据 [$SharedData$]，获得键控数据 $KKeyData$ ，长度为 $mackeylen+keydatalen$ 位。
- 6 解析 $KKeyData$ 最左边的 $mackeylen$ 位，作为一个 MAC 密钥 $MacKey$ ，剩余的位作为键控数据 $KeyData$ 。
- 7 形成位字符串，包含字节 02_{16} ， V 的标识符， U 的标识符，位字符串 QEV' ，位字符串 QEU ，以及如果 $Text_1$ 出现的话还有 $Text_1$ ：

$$MacData_1 = 02_{16} \parallel V \parallel U \parallel QEV' \parallel QEU \parallel [Text_1]$$

- 8 证实 $MacTag_1'$ 是 $MacData_1$ 的标记，在 $MacKey$ 之下，使用 ANSI X9.63-2001 [B7]的 5.7.2 节指明的合适 MAC 方案的标记检查转换。如果标记检查转换输出“无效”，输出“无效”并停止。
- 9 形成位字符串，包含字节 03_{16} ， U 的标识符， V 的标识符，对应 U 的质疑的位字符串 QEU ，对应 V 的质疑的位字符串 QEV' ，和可选的一个位字符串 $Text_2$ ：

$$MacData_2 = 03_{16} \parallel U \parallel V \parallel QEU \parallel QEV' \parallel [Text_2]$$

- 10 计算 $MacData_2$ 的标记 $MacTag_2$ ，在密钥 $MacKey$ 之下，使用 ANSI X9.63-2001 [B7]的 5.7.1 节指明的合适 MAC 方案的标记转换：

$$MacTag_2 = MAC_{MacKey}(MacData_2) \quad (3)$$

- 11 如果标记转换输出“无效”，输出“无效”并停止。给 V 发送 $MacTag_2$ ，以及如果 $Text_2$ 出现的话，也发送 $Text_2$ 。
- 12 从 V 接收一个合适的位字符串 $Text_1$ ，和一个所谓的标记 $MacTag_1'$ 。如果这些值没有收到，输出“无效”并停止。

输出：如果以上任何验证失败，那么输出“无效”并拒绝位字符串 $KeyData$ 和 $Text_1$ 。否则，输出“有效”，接受位字符串 $KeyData$ 作为和 V 共享的密钥数据，长度为 $keydatalen$ 位，并接受 V 作为位字符串 $Text_1$ 的源（如果出现的话）。

B.7.2 响应者转换

如果 V 是协议的响应者， V 必须执行以下转换来同意和 U 加密数据。 V 必须获得 U 的标识符的一个验证副本，以及和 U 共享的静态秘密密钥 Key 的一个验证副本。

输入：响应者转换的输入是：

- 1 U 拥有的一个所谓的质疑 QEU 。
- 2 一个整数 $keydatalen$ ，是要生成的键控数据的长度，以位为单位。
- 3 （可选的）一个位字符串 $SharedData$ ，长度为 $shareddatalen$ 位，包含 U 和 V 共享的一些数据。
- 4 （可选的）一个位字符串 $Text_1$ ，包含 V 给 U 提供的另外一些数据。

组成：响应者转换使用 ANSI X9.63-2001 [B7] 的 5.3 节的质疑生成原语，B.3.2 小节的质疑验证原语，B.5 小节的 SKG 原语，ANSI X9.63-2001 [B7] 的 5.6.3 节的密钥验证函数和 ANSI X9.63-2001 [B7] 的 5.7 节的 MAC 方案之一。

行为：键控数据必须按照以下方式获得：

- 1 证实 QEU 是 B.3.2 小节指明的质疑域参数 D 的有效质疑。如果验证原语拒绝该质疑，输出“无效”并停止。
- 2 使用 ANSI X9.63-2001 [B7] 的 5.3 节的质疑生成原语，为质疑域参数 D 生成一个质疑 QEV 。给 U 发送质疑 QEV 。
- 3 然后从 U 接收一个可选的位字符串 $Text_2$ 和一个所谓的标记 $MacTag_2'$ 。如果这个数据没有接收，输出“无效”并停止。
- 4 形成位字符串，包含字节 03_{16} ， U 的标识符， V 的标识符，对应 U 所谓质疑的位字符串 QEU ，对应 V 质疑的位字符串 QEV ，和位字符串 $Text_2$ （如果出现的话）：

$$MacData_2 = 03_{16} \parallel U \parallel V \parallel QEU' \parallel QEV \parallel [Text_2]$$

- 5 证实 $MacTag_2'$ 是 $MacData_2$ 的有效标记，在密钥 $MacKey$ 之下，使用 ANSI X9.63-2001 [B7] 的 5.7 节指定的合适 MAC 方案的标记检查转换。如果标记检查转换输出“无效”，输出“无效”并停止。
- 6 使用 B.5 小节的 SKG 原语，从 U 拥有的质疑 $Q_1=QEU$ 和 V 拥有的 $Q_2=QEV$ 获得一个共享的秘密位字符串 Z ，作为共享密钥 Key 使用。如果 SKG 原语输出“无效”，输出“无效”并停止。
- 7 使用 ANSI X9.63-2001 [B7] 的 5.6.3 节的密钥获得函数和建立的哈希函数，从共享的秘密值 Z 和共享的数据 $[SharedData]$ 获得键控数据 $KKeyData$ ，长度为 $mackeylen+keydatalen$ 位。



- 8 解析 $KKeyData$ 最左边的 $mackeylen$ 位作为一个 MAC 密钥 $MacKey$ ，剩余的位作为键控数据 $KeyData$ 。
- 9 形成位字符串，包含字节 02_{16} ， V 的标识符， U 的标识符，位字符串 QEV ，位字符串 QEU' ，和可选的一个位字符串 $Text_1$ ：

$$MacData_1 = 02_{16} \parallel V \parallel U \parallel QEV \parallel QEU' \parallel [Text_1]$$

- 10 为 $MacData_1$ 计算标记 $MacTag_1$ ，在密钥 $MacKey$ 之下，使用 ANSI X9.63-2001 [B7] 的 5.7 节指明的合适 MAC 方案的标记转换：

$$MacTag_1 = MAC_{MacKey}(MacData_1)$$

如果标记转换输出“无效”，输出“无效”并停止。给 U 发送 $MacTag_1$ ，如果 $Text_1$ 出现的话，也发送 $Text_1$ 。

输出：如果以上任何验证失败，那么输出“无效”并拒绝位字符串 $KeyData$ 和 $Text_2$ 。否则，输出“有效”，接受位字符串 $KeyData$ 作为和 U 的键控数据，长度为 $keydatalen$ ，并接受 U 作为位字符串 $Text_2$ 的源（如果出现的话）。

B.8 相互对称密钥实体认证

本节指明了相互对称密钥实体认证方案。一个 MAC 方案用来提供密钥确认。

图 B.2 表明了涉及相互对称密钥实体认证方案使用的信息。

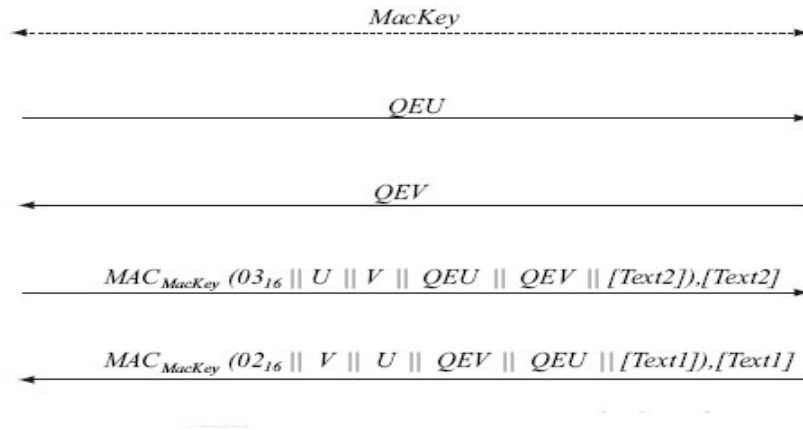


图 B.2 相互对称密钥实体认证方案

本方案是“非对称”的，所以规定了 2 个转换。 U 使用 B.8.1 小节的转换，通过和 V 共享一个密钥和合作通信，去建立真实性，并可选地从 V 获得经过验证的数据。 V 使用 B.8.2 小节指明的转换，通过和 U 共享一个密钥和合作通信，去建立真实性，并可选地从 U 获得经过验证的数据。

发起者和响应者的作用之间的本质区别在于发起者发送交换的第一个传输。

先决条件： 以下是使用本方案的先决条件：

- 1 每个实体有系统的质疑域参数的认证副本 $D=(minchallenge, maxchallenge)$ 。这些参数是使用 B.3.1 小节的参数产生原语产生的。此外，这些参数是使用 B.3.2 小节的参数证实原语证实的。
- 2 每个实体都应该有权访问被用作密钥的一个位字符串 $MacKey$ ，长度为 $mackeylen$ 位。每一方应该证明访问此密钥受实体本身，以及相互实体认证方案涉及的其它实体的限制。
- 3 每个实体都应该限制在唯一的标识符内（即有区别的名称）。所有的标识符都应该具有同样长度 $entlen$ 的字符串。实体 U 的标识符以位字符串 U 表示。实体 V 的标识符以位字符串 V 表示。

- 4 每个实体必须决定使用ANSI X9.63-2001 [B7]的5.7节指定的哪个MAC方案。选定的MAC方案使用的密钥长度由 $mackeylen$ 表示，以位为单位。
- 5 必须为二进制字符串选择一个字节的固定表示法。（比如，最高有效位优先的顺序或最低有效位优先的顺序）。

B.8.1 发起者转换

U 通过和 I 共享一个密钥并合作通信，执行以下转换来建立真实性，并可选地从 I 获得经过验证的数据。 U 必须获得 I 的标识符的一个验证副本，以及和 I 共享的静态秘密密钥 $MacKey$ 的一个验证副本。

输入：发起者转换的输入是：

- 1 （可选的）一个位字符串 $Text2$ ，包含 U 给 I 提供的另外一些数据。

组成：发起者转换使用ANSI X9.63-2001 [B7]的5.3节的质疑生成原语，B.4小节的质疑验证原语，和ANSI X9.63-2001 [B7]的5.7节的MAC方案之一。

行为：实体认证必须按以下方式建立：

- 1 使用ANSI X9.63-2001 [B7]的5.3节的质疑生成原语，为质疑域参数 D 生成一个质疑 QEU 。发送 QEU 给 I 。
- 2 然后从 I 接收 I 拥有的一个所谓质疑 QEV' 。如果这个值没有接收，输出“无效”并停止。
- 3 证实 QEV' 对于B.3.1小节指明的质疑域参数 D 是一个有效的质疑。如果验证原语拒绝该质疑，输出“无效”并停止。
- 4 形成位字符串，包含字节 03_{16} ， U 的标识符， I 的标识符，对应 U 的质疑的位字符串 QEU' ，对应 I 所谓的质疑的位字符串 QEV' ，以及可选的一个位字符串 $Text2$ ：

$$MacData2 = 03_{16} || U || V || QEU || QEV' || [Text2]$$

- 5 计算 $MacData2$ 的标记 $MacTag2$ ，在 $MacKey$ 之下，使用ANSI X9.63-2001 [B7]的5.7.1节指明的合适MAC方案的标记检查转换：

$$MacTag2 = MAC_{MacKey}(MacData2).$$

如果标记检查转换输出“无效”，输出“无效”并停止。发送 *MacTag2* 到 *V*，如果有 *Text2*，也发送字符串 *Text2* 到 *V*。

6 然后从 *V* 接收一个可选的位字符串 *Text1*，和一个所谓的标记 *MacTag1'*。如果这些值没有接收，输出“无效”并停止。

7 形成位字符串，包含字节 02_{16} ，*V* 的标识符，*U* 的标识符，对应 *V* 所谓的质疑的位字符串 *QEV'*，对应 *U* 的质疑的位字符串 *QEU'*，以及如果 *Text1* 出现的话还有 *Text1*：

$$MacData1 = 02_{16} || V || U || QEV' || QEU' || [Text1]$$

8 证实 *MacTag1'* 是 *MacData1* 的标记，在 *MacKey* 之下，使用 ANSI X9.63-2001 [B7] 的 5.7.2 节指明的合适 MAC 方案的标记检查转换。如果标记检查转换输出“无效”，输出“无效”并停止。

输出：如果以上任何验证失败，那么输出“无效”并拒绝 *V* 的认证和来自 *V* 的实体认证。否则，输出“有效”，接受 *V* 的认证，并接受经过验证的位字符串 *Text1* 的来自 *V* 的实体认证（如果出现的话）。

B.8.2 响应者转换

V 通过和 *U* 共享一个密钥并合作通信，执行以下转换来建立真实性，并可选地从 *U* 获得经过验证的数据。*V* 必须获得 *U* 的标识符的一个验证副本，以及和 *U* 共享的静态秘密密钥 *Key* 的一个验证副本。

输入：响应者转换的输入是：

- 1 *U* 拥有的一个所谓的质疑 *QEU'*。
- 2 （可选的）一个位字符串 *Text1*，包含 *V* 给 *U* 提供的另外一些数据。

组成：响应者转换使用 ANSI X9.63-2001 [B7] 的 5.3 节的质疑生成原语，B.4 小节的质疑验证原语，和 ANSI X9.63-2001 [B7] 的 5.7 节的 MAC 方案之一。

行为：实体认证必须按照以下方式建立：

- 1 证实 *QEU'* 是 B.3.1 小节指明的质疑域参数 *D* 的有效质疑。如果验证原语拒绝该质疑，输出“无效”并停止。



- 2 使用ANSI X9.63-2001[B7]的5.3节的质疑生成原语，为质疑域参数 D 生成一个质疑 QEV 。给 U 发送质疑 QEV 。
- 3 然后从 U 接收一个可选的位字符串 $Text2$ 和一个所谓的标记 $MacTag2$ 。如果这个数据没有接收，输出“无效”并停止。
- 4 形成位字符串，包含字节 03_{16} ， U 的标识符， V 的标识符，对应 U 所谓质疑的位字符串 QEU' ，对应 V 质疑的位字符串 QEV ，和位字符串 $Text2$ （如果出现的话）：

$$MacData_2 = 03_{16} || U || V || QEU' || QEV || [Text2]$$

- 5 计算 $MacData2$ 标记的 $MacTag2$ ，在密钥 $MacKey$ 之下，使用 ANSI X9.63-2001[B7] 的 5.7.1 节指定的合适 MAC 方案的标记转换。

$$MacTag2 = MAC_{MacKey}(MacData2)$$

如果标记转换输出“无效”，输出“无效”并停止。

- 6 证实 $MacTag2$ 是 $MacData2$ 的有效标记，在密钥 $MacKey$ 之下，使用 ANSI X9.63-2001[B7] 的 5.7.2 节指定的合适 MAC 方案的标记检查转换。如果标记检查转换输出“无效”，输出“无效”并停止。
- 7 形成位字符串，包含字节 02_{16} ， V 的标识符， U 的标识符，对应 V 的质疑的位字符串 QEV ，对应 U 所谓的质疑的位字符串 QEU' ，和可选的一个位字符串 $Text1$ ：

$$MacData1 = 02_{16} || V || U || QEV || QEU' || [Text1]$$

- 8 为 $MacData1$ 计算标记 $MacTag1$ ，在密钥 $MacKey$ 之下，使用 ANSI X9.63-2001 [B7] 的 5.7.1 节指明的合适 MAC 方案的标记转换：

$$MacTag1 = MAC_{MacKey}(MacData1)$$

如果标记转换输出“无效”，输出“无效”并停止。给 U 发送 $MacTag1$ ，如果 $Text1$ 出现的话，也发送 $Text1$ 。

输出：如果以上任何验证失败，那么输出“无效”并拒绝 U 的认证，并拒绝来自 U 的实体认证。否则，输出“有效”，接受 U 的认证，并接受经过验证的位字符串 $Text2$ 的来自 U 的实体认证（如果出现的话）。

附录 C

加密建立模块的测试向量

本附录为 ZigBee 团体提供了样本测试向量，目的是帮助建立兼容的安全实现。提供的样本测试向量是有待独立验证的。

C.1 数据转换

对于测试向量，参见 ANSI X9.63-2001 [B7]的附录 J1。

C.2 AES 分组密码

本附录为 B.1.1 小节指明的分组密码提供了样本测试向量。

C.3 CCM*模式加密和验证转换

本附录为 B.1.2 小节指明的操作模式提供了样本测试向量。

先决条件：为操作模式的运行建立以下先决条件：

- 1 参数 M 的值必须为整数值 8。



输入：操作模式的输入是：

- 1 使用密钥 Key ，大小为 $keylen=128$ 位：

$Key = C0\ C1\ C2\ C3\ C4\ C5\ C6\ C7\ C8\ C9\ CA\ CB\ CC\ CD\ CE\ CF$

- 2 使用临时 N ，为 $15-L=13$ 个字节：

$Nonce = A0\ A1\ A2\ A3\ A4\ A5\ A6\ A7 \parallel 03\ 02\ 01\ 00 \parallel 06$

- 3 使用字节字符串 m ，长度为 $l(m)=23$ 个字节：

$m = 08\ 09\ 0A\ 0B\ 0C\ 0D\ 0E\ 0F\ 10\ 11\ 12\ 13\ 14\ 15\ 16\ 17\ 18\ 19\ 1A\ 1B\ 1C\ 1D\ 1E$

- 4 使用字节字符串 a ，长度为 $l(a)=8$ 个字节：

$a = 00\ 01\ 02\ 03\ 04\ 05\ 06\ 07$

C.3.1 输入转换

本步骤涉及输入字符串 a 和 m 到字符串 $AuthData$ 和 $PlainTextData$ 的转换，分别用于验证转换和加密转换。

- 1 形成字节字符串表示的 $L(a)$ ，长度为字节字符串 a 的长度 $l(a)$ ：

$L(a) = 00\ 08$

- 2 右串联字节字符串 $L(a)$ 和字节字符串 a 本身：

$L(a) \parallel a = 00\ 08 \parallel 00\ 01\ 02\ 03\ 04\ 05\ 06\ 07$

- 3 通过右串联产生的字符串和全零字节的最小非负数，形成填补信息 $AddAuthData$ ，这样字节字符串 $AddAuthData$ 的长度可以被 16 整除：

$AddAuthData = 00\ 08 \parallel 00\ 01\ 02\ 03\ 04\ 05\ 06\ 07 \parallel 00\ 00\ 00\ 00\ 00\ 00$

- 4 通过右串联字节字符串 m 和全零字节的最小非负数，形成填补信息 $PlainTextData$ ，这样字节字符串 $PlainTextData$ 的长度可以被 16 整除：

$PlainTextData = 08\ 09\ 0A\ 0B\ 0C\ 0D\ 0E\ 0F\ 10\ 11\ 12\ 13\ 14\ 15\ 16\ 17 \parallel 18\ 19\ 1A\ 1B\ 1C\ 1D\ 1E \parallel 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00$

- 5 形成信息 $AuthData$ ，包含字节字符串 $AddAuthData$ 和 $PlainTextData$ 。

```
0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 00 00 00 00 00 00 00 00 00
```

C.3.2 验证转换

上面建立的数据 *AuthData* 必须使用如下的标记转换进行标记:

- 1 形成 1 字节的 *Flags* 域, 如下:

Flags = 59

- 2 形成 16 字节的 B_0 域, 如下:

$$B_0 = 59 \parallel A_0 A_1 A_2 A_3 A_4 A_5 A_6 A_7 03 02 01 00 06 \parallel 00 17$$

- 3 解析信息 *AuthData* 作为 $B_1 \| B_2 \| B_3$, 其中每个信息块 B_i 是一个 16 字节字符串。

- 4 CBC-MAC 的值 X_4 按照下面方法计算:

i	B_i	X_i
0	59 A0 A1 A2 A3 A4 A5 A6 A7 03 02 01 00 06 00 17	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1	00 08 00 01 02 03 04 05 06 07 00 00 00 00 00 00	F7 74 D1 6E A7 2D C0 B3 E4 5E 36 CA 8F 24 3B 1A
2	08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17	90 2E 72 58 AE 5A 4B 5D 85 7A 25 19 F3 C7 3A B3
3	18 19 1A 1B 1C 1D 1E 00 00 00 00 00 00 00 00 00	5A B2 C8 6E 3E DA 23 D2 7C 49 7D DF 49 BB B4 09
4	$\varnothing$	B9 D7 89 67 04 BC FA 20 B2 10 36 74 45 F9 83 D6

验证标记 T 是忽略 CBC-MAC 的值 X_4 的所有字节, 但是除了最左边的 $M=8$ 字节的结果:

$$T = B9\ D7\ 89\ 67\ 04\ BC\ FA\ 20$$

C.3.3 加密转换

数据 *PlaintextData* 必须使用如下的加密转换进行加密:

- 1 形成 1 字节的 Flags 域, 如下:

$$Flags = 01$$



- 2 定义 16 字节的 A_i 域，如下：

i	A_i
0	01 A0 A1 A2 A3 A4 A5 A6 A7 03 02 01 00 06 00 00
1	01 A0 A1 A2 A3 A4 A5 A6 A7 03 02 01 00 06 00 01
2	01 A0 A1 A2 A3 A4 A5 A6 A7 03 02 01 00 06 00 02

- 3 解析信息 *PlaintextData*，作为 $M_1 || M_2$ ，其中每个信息块 M_i 是一个 16 字节字符串。
4 密文块 C_1, C_2 按照如下方法计算：

i	AES(Key, A_i)	$C_i = AES(Key, A_i) - M_i$
1	12 5C A9 61 B7 61 6F 02 16 7A 21 66 70 89 F9 07	1A 55 A3 6A BB 6C 61 0D 06 6B 33 75 64 9C EF 10
2	CC 7F 54 D1 C4 49 B6 35 46 21 46 03 AA C6 2A 17	D4 66 4E CA D8 54 A8 35 46 21 46 03 AA C6 2A 17

- 5 字符串 *Ciphertext* 是忽略字符串 $C_1 || C_2$ 的所有字节，但是除了最左边 $l(m)=23$ 字节的结果：

$CipherText = 1A\ 55\ A3\ 6A\ BB\ 6C\ 61\ 0D\ 06\ 6B\ 33\ 75\ 64\ 9C\ EF\ 10 || D4\ 66\ 4E\ CA\ D8\ 54\ A8$

- 6 定义 16 字节的加密块 S_0 ，通过：

$S_0 = E(Key, A_0) = B3\ 5E\ D5\ A6\ DC\ 43\ 6E\ 49\ D6\ 17\ 2F\ 54\ 77\ EB\ B4\ 39$

- 7 加密验证标记 U 是字符串 S_0 最左边 $M=8$ 字节和验证标记 T 的异或：

$U = 0A\ 89\ 5C\ C1\ D8\ FF\ 94\ 69$

输出： c 是加密信息 *Ciphertext* 和加密验证标记 U 的右串联：

$c = 1A\ 55\ A3\ 6A\ BB\ 6C\ 61\ 0D\ 06\ 6B\ 33\ 75\ 64\ 9C\ EF\ 10 || D4\ 66\ 4E\ CA\ D8\ 54\ A8 || 0A\ 89\ 5C\ C1\ D8\ FF\ 94\ 69$

C.4 CCM*模式解密和验证检查转换

本附录为 B.1.2 小节指明的操作模式的逆操作提供了样本测试向量。

先决条件：为操作模式的操作建立以下先决条件：

- 1 参数 M 必须是整数值 8。

输入：操作模式的逆操作的输入是：

- 1 使用密钥 Key ，大小是 $keylen=128$ 位：

$$Key = C0\ C1\ C2\ C3\ C4\ C5\ C6\ C7\ C8\ C9\ CA\ CB\ CC\ CD\ CE\ CF$$

- 2 使用临时 N ，为 $15-L=13$ 字节：

$$Nonce = A0\ A1\ A2\ A3\ A4\ A5\ A6\ A7\ ||\ 03\ 02\ 01\ 00\ ||\ 06$$

- 3 使用字节字符串 c ，长度为 $\mathcal{L}(c)=31$ 字节：

$$c = 1A\ 55\ A3\ 6A\ BB\ 6C\ 61\ 0D\ 06\ 6B\ 33\ 75\ 64\ 9C\ EF\ 10\ ||\ D4\ 66\ 4E\ CA\ D8\ 54\ A8\ ||\ 0A\ 89\ 5C\ C1\ D8\ FF\ 94\ 69$$

- 4 使用字节字符串 a ，长度为 $\mathcal{L}(a)=8$ 字节：

$$a = 00\ 01\ 02\ 03\ 04\ 05\ 06\ 07$$

C.4.1 解密转换

解密转换涉及以下步骤，顺序为：

- 1 解析信息 c ，作为 $C||U$ ，其中最右边的字符串 U 是一个 M 字节的字符串：

$$C = 1A\ 55\ A3\ 6A\ BB\ 6C\ 61\ 0D\ 06\ 6B\ 33\ 75\ 64\ 9C\ EF\ 10\ ||\ D4\ 66\ 4E\ CA\ D8\ 54\ A8;$$
$$U = 0A\ 89\ 5C\ C1\ D8\ FF\ 94\ 69$$

- 2 通过右串联字符串 C 和全零字节的最小非负数，形成填补信息 $CiphertextData$ ，这样字节字符串 $CiphertextData$ 的长度可以被 16 整除。

$$CipherTextData = 1A\ 55\ A3\ 6A\ BB\ 6C\ 61\ 0D\ 06\ 6B\ 33\ 75\ 64\ 9C\ EF\ 10\ ||\ D4\ 66\ 4E\ CA\ D8\ 54\ A8\ ||\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00$$



- 3 形成 1 字节的 $Flags$ 域，如下：

$$Flags = 01$$

- 4 定义 16 字节的 A_i 域，如下：

i	A_i
0	01 A0 A1 A2 A3 A4 A5 A6 A7 03 02 01 00 06 00 00
1	01 A0 A1 A2 A3 A4 A5 A6 A7 03 02 01 00 06 00 01
2	01 A0 A1 A2 A3 A4 A5 A6 A7 03 02 01 00 06 00 02

- 5 解析信息 $CiphertextData$ ，作为 $C_1 || C_2$ ，其中每个信息块是一个 16 字节的字符串。
6 密文块 P_1, P_2 按照如下方法计算：

i	$AES(Key, A_i)$	$P_i = AES(Key, A_i) - C_i$
1	12 5C A9 61 B7 61 6F 02 16 7A 21 66 70 89 F9 07	08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17
2	CC 7F 54 D1 C4 49 B6 35 46 21 46 03 AA C6 2A 17	18 19 1A 1B 1C 1D 1E 00 00 00 00 00 00 00 00

- 7 字节字符串 m 是忽略字符串 $P_1 || P_2$ 的所有字节，但是除了最左边的 $l(m)=23$ 字节：

$$m = 08\ 09\ 0A\ 0B\ 0C\ 0D\ 0E\ 0F\ 10\ 11\ 12\ 13\ 14\ 15\ 16\ 17\ ||\ 18\ 19\ 1A\ 1B\ 1C\ 1D\ 1E$$

- 8 定义了 16 字节的加密块 S_0 ，通过

$$S_0 = E(Key, A_0) = B3\ 5E\ D5\ A6\ DC\ 43\ 6E\ 49\ D6\ 17\ 2F\ 54\ 77\ EB\ B4\ 39$$

- 9 所谓的验证标记 T 是字符串 S_0 最左边 $M=8$ 字节和字节字符串 U 的异或：

$$T = B9\ D7\ 89\ 67\ 04\ BC\ FA\ 20$$

C.4.2 验证检查转换

验证检查转换涉及以下步骤：

- 1 以 C.4.1 小节（步骤 7）建立的字符串 a 和字节字符串 m 作为输入，使用 C.3.1 小节的输入转换形成信息 $AuthData$ 。



$AuthData = 08\ 00\ 01\ 02\ 03\ 04\ 05\ 06\ 07\ 00\ 00\ 00\ 00\ 00\ 00\ ||$
 $08\ 09\ 0A\ 0B\ 0C\ 0D\ 0E\ 0F\ 10\ 11\ 12\ 13\ 14\ 15\ 16\ 17||$
 $18\ 19\ 1A\ 1B\ 1C\ 1D\ 1E\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00$

- 2 使用 C.3.2 小节的验证转换，以信息 $AuthData$ 作为输入，计算验证标记 $MACTag$:

$MACTag = B9\ D7\ 89\ 67\ 04\ BC\ FA\ 20$

- 3 比较这个转换产生的输出标记 $MACTag$ ，和 C.4.1 小节（步骤 9）建立的标记 T :

$T = B9\ D7\ 89\ 67\ 04\ BC\ FA\ 20 = MACTag$

输出：如果 $MACTag = T$ ，输出“有效”，接受字节字符串 m ，并接受共享组成员的密钥之一作为 m 的源。

C.5 加密哈希函数

本附录为 C.5 节指明的加密哈希函数提供了样本测试向量。

C.5.1 测试向量例 1

输入：加密哈希函数的输入如下：

- 1 使用位字符串 M ，长度为 $L=8$ 位：

$M = C0$

行为：哈希值必须按照以下方法获得：

- 1 通过右串联 M 和位“1”，其次是“0”位的最小非负数，填补信息 M ，这样产生的字符串长度为 14（模 16）字节：

$C0 || 80\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00$

- 2 通过右串联产生的字符串，和 16 位字符串，以及等于整数 1 的二进制表示，形成填补信息 M' 。

$M' = C0 || 80\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00 || 00\ 08$

- 3 解析填补信息 M' ，作为 M_1 ，其中每个信息块 M_i 是一个 16 字节的字符串。



4 哈希值 $Hash_1$ 按照如下计算:

i	$Hash_i$	M_i
0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	æ
1	AE 3A 10 2A 28 D4 3E E0 D4 A0 9E 22 78 8B 20 6C	C0 80 00 00 00 00 00 00 00 00 00 00 00 00 00 08

输出: 16 字节的字符串 $Hash = Hash_1 = \text{AE 3A 10 2A 28 D4 3E E0 D4 A0 9E 22 78 8B 20 6C}$ 。

C.5.2 测试向量例 2

输入: 加密哈希函数的输入如下:

1 使用位字符串 M , 长度为 $L=128$ 位:

$M = \text{C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF}$

行为: 哈希值必须按照以下方法获得:

1 通过右串联 M 和位“1”, 其次是“0”位的最小非负数, 填补信息 M , 这样产生的字符串长度为 14 (模 16) 字节:

$\text{C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF} \parallel 80 00 00 00 00 00 00 00 00 00 00 00 00 00$

2 通过右串联产生的字符串, 和 16 位字符串, 以及等于整数 L 的二进制表示, 形成填补信息 M' 。

$M' = \text{C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF} \parallel 80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 \parallel 00 80$

3 解析填补信息 M' , 作为 $M_1 \parallel M_2$, 其中每个信息块 M_i 是一个 16 字节的字符串。

4 哈希值 $Hash_2$ 按照如下计算:

i	$Hash_i$	M_i
0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	æ
1	84 EE 75 E5 4F 9A 52 0F 0B 30 9C 35 29 1F 83 4F	C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF
2	A7 97 7E 88 BC 0B 61 E8 21 08 27 10 9A 22 8F 2D	80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 08

输出: 16 字节的字符串 $Hash = Hash_2 = \text{A7 97 7E 88 BC 0B 61 E8 21 08 27 10 9A 22 8F 2D}$ 。

C.6 信息验证的键控哈希函数

本附录为 C.6 节指明的信息验证的键控哈希函数提供了样本测试向量。

C.6.1 测试向量例 1

输入：键控哈希函数的输入如下：

- 1 使用密钥 Key ，大小为 $keylen=128$ 位：

$$Key = 40\ 41\ 42\ 43\ 44\ 45\ 46\ 47\ 48\ 49\ 4A\ 4B\ 4C\ 4D\ 4E\ 4F$$

- 2 使用位字符串 M ，长度为 $l=8$ 位：

$$M = C0$$

行为：键控哈希函数值必须按照以下方式获得：

- 1 创建一个 16 字节的字符串 $ipad$ （内部填补），如下：

$$ipad = 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36$$

- 2 通过异或位字符串 Key 和字节字符串 $ipad$ ，形成内部密钥 Key_1 ：

$$Key_1 = Key - ipad = 76\ 77\ 74\ 75\ 72\ 73\ 70\ 71\ 7E\ 7F\ 7C\ 7D\ 7A\ 7B\ 78\ 79$$

- 3 通过右串联位字符串 Key_1 和位字符串 M ，形成填补信息 M_1 ：

$$M_1 = Key_1 \parallel M = 76\ 77\ 74\ 75\ 72\ 73\ 70\ 71\ 7E\ 7F\ 7C\ 7D\ 7A\ 7B\ 78\ 79 \parallel C0$$

- 4 计算位字符串 M_1 的哈希值 $Hash_1$ ：

$$Hash_1 = 3C\ 3D\ 53\ 75\ 29\ A7\ A9\ A0\ 3F\ 66\ 9D\ CD\ 88\ 6C\ B5\ 2C$$

- 5 创建 16 字节的字符串 $opad$ （外部填补），如下：

$$opad = 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C$$

- 6 通过异或字符串 Key 和字节字符串 $opad$ ，形成外部密钥 Key_2 ：

$$Key_2 = Key - opad = 1C\ 1D\ 1E\ 1F\ 18\ 19\ 1A\ 1B\ 14\ 15\ 16\ 17\ 10\ 11\ 12\ 13$$

- 7 通过右串联位字符串 Key_2 和位字符串 $Hash_1$ ，形成填补信息 M_2 ：

$M_2 = Key_2 \parallel Hash_1 = 1C\ 1D\ 1E\ 1F\ 18\ 19\ 1A\ 1B\ 14\ 15\ 16\ 17\ 10\ 11\ 12\ 13 \parallel 3C\ 3D\ 53\ 75\ 29\ A7\ A9\ A0\ 3F\ 66\ 9D\ CD\ 88\ 6C\ B5\ 2C$

- 8 计算位字符串 M_2 的哈希值 $Hash_2$ ：

$Hash_2 = 45\ 12\ 80\ 7B\ F9\ 4C\ B3\ 40\ 0F\ 0E\ 2C\ 25\ FB\ 76\ E9\ 99$

输出：16 字节字符串 $HMAC = Hash_2 = 45\ 12\ 80\ 7B\ F9\ 4C\ B3\ 40\ 0F\ 0E\ 2C\ 25\ FB\ 76\ E9\ 99$

C.6.2 测试向量例 2

输入：键控哈希函数的输入如下：

- 1 使用密钥 Key ，大小为 $keylen=256$ 位：

$Key = 40\ 41\ 42\ 43\ 44\ 45\ 46\ 47\ 48\ 49\ 4A\ 4B\ 4C\ 4D\ 4E\ 4F \parallel 50\ 51\ 52\ 53\ 54\ 55\ 56\ 57\ 58\ 59\ 5A\ 5B\ 5C\ 5D\ 5E\ 5F$

- 2 使用位字符串 M ，长度为 $l=128$ 位：

$M = C0\ C1\ C2\ C3\ C4\ C5\ C6\ C7\ C8\ C9\ CA\ CB\ CC\ CD\ CE\ CF$

行为：键控哈希函数值必须按照以下方式获得：

- 1 计算位字符串 Key 的哈希值 Key_0 ：

$Key_0 = 22\ F4\ 0C\ BE\ 15\ 66\ AC\ CF\ EB\ 77\ 77\ E1\ C4\ A9\ BB\ 43$

- 2 创建 16 字节的字符串 $ipad$ （内部填补），如下：

$ipad = 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36\ 36$

- 3 通过异或位字符串 Key_0 和字节字符串 $ipad$ ，形成内部密钥 Key_1 ：

$Key_1 = Key_0 - ipad = 14\ C2\ 3A\ 88\ 23\ 50\ 9A\ F9\ DD\ 41\ 41\ D7\ F2\ 9F\ 8D\ 75$

- 4 通过右串联位字符串 Key_1 和位字符串 M ，形成填补信息 M_1 ：

$M_1 = Key_1 \parallel M = 14\ C2\ 3A\ 88\ 23\ 50\ 9A\ F9\ DD\ 41\ 41\ D7\ F2\ 9F\ 8D\ 75 \parallel$

$C0\ C1\ C2\ C3\ C4\ C5\ C6\ C7\ C8\ C9\ CA\ CB\ CC\ CD\ CE\ CF$

- 5 计算位字符串 M_1 的哈希值 $Hash_1$ ：

$Hash_1 = 42\ 65\ BE\ 29\ 74\ 55\ 8C\ A2\ 7B\ 77\ 85\ AC\ 73\ F2\ 22\ 10$



- 6 创建 16 字节的字符串 *opad* (外部填补), 如下:

$$opad = 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C\ 5C$$

- 7 通过异或位字符串 *Key₀* 和字节字符串 *opad*, 形成外部密钥 *Key₂*:

$$Key_2 = Key_0 - opad = 7E\ A8\ 50\ E2\ 49\ 3A\ F0\ 93\ B7\ 2B\ 2B\ BD\ 98\ F5\ E7\ 1F$$

- 8 通过右串联位字符串 *Key₂* 和位字符串 *Hash₁*, 形成填补信息 *M₂*:

$$M_2 = Key_2 \parallel Hash_1 = 7E\ A8\ 50\ E2\ 49\ 3A\ F0\ 93\ B7\ 2B\ 2B\ BD\ 98\ F5\ E7\ 1F \parallel 42\ 65\ BE\ 29\ 74\ 55\ 8C\ A2\ 7B\ 77\ 85\ AC\ 73\ F2\ 22\ 10$$

- 9 计算位字符串 *M₂* 的哈希值 *Hash₂*:

$$Hash_2 = A3\ B0\ 07\ 99\ 84\ BF\ 15\ 57\ F7\ 4A\ 0D\ 63\ 87\ E0\ A1\ 1A$$

输出: 16 字节字符串 $HMAC = Hash_2 = A3\ B0\ 07\ 99\ 84\ BF\ 15\ 57\ F7\ 4A\ 0D\ 63\ 87\ E0\ A1\ 1A$

C.6.3 信息验证的专门键控哈希函数

本附录为 C.6.3 节指明的信息验证的专门键控哈希函数提供了样本测试向量。
对于测试向量, 参见 C.6。

C.6.4 对称密钥的密钥协议方案和实体认证方案

本节详细提供了 ZigBee 使用的 SKKE 和 EA 算法的中间步骤和结果。它的目的是通过证实自己的结果帮助那些希望执行该算法的用户。

希望 MAC 功能 (使用一个密钥的哈希数据) 已经被证实。

C.6.4.1 字节存储次序问题

以下是 SKKE / EA 信息解释数据元素的方式:

- 发起者和响应者 IEEE 地址是作为小端 (从小到大) 通过无线发送的, 但是它们串联在位字符串中还是作为大端值。



- 密钥被解释为作为小端通过无线传输，用作小端值。
- 发起者和响应者质疑被解释为作为小端通过无线传输。它们串联在位字符串中还是作为小端值。
- MacTag 数据被解释为作为小端，通过无线传输，和通过算法传输的方式相同。
- 帧计数器作为小端，通过无线传输，但是串联在位字符串中还是作为大端。

图例：

(<)表示数字是小端，例如(<)1234 以大端形式显示是 0x3412。

(>)表示数字是大端。

C.6.4.2 SKKE 发起者转换

名词	描述
Key	共享密钥（例如信任中心主密钥）
MacKey	产生密钥数据期间创建的一个密钥
U	发起者的唯一 ID（EUI64 地址）
V	响应者的唯一 ID（EUI64 地址）
QEU	发起者质疑（16 字节随机数）
QEV	响应者质疑（16 字节随机数）
MAC	我们的 HMAC 函数
Text1	可选的位字符串。SKKE 不使用。
Text2	""
SharedData	可选的数据，在发起者和响应者之间共享。对于 SKKE / EA 是空字符串。

注意：伪代码中的'||'表示串联

- 1 响应者的 IEEE (V) 必须提前知道
- 2 产生发起者质疑 QEU
- 3 发送发起者质疑和发起者 IEEE 给响应者 (SKKE-1)
- 4 接收响应者质疑，QEV (SKKE-2)

- 5 计算共享的秘密位字符串 Z。
 - a $\text{MacData1} = U \parallel V \parallel \text{QUE} \parallel \text{QEV}$
 - b $\text{MacTag1} = \text{MAC}(\text{MacData1}, \text{Key})$
 - c $Z = \text{MacTag1}$
- 6 获得键控数据。
 - a $\text{Hash-1} = Z \parallel 00\ 00\ 00\ 01 \parallel \text{SharedData}$
 - b $\text{Hash-2} = Z \parallel 00\ 00\ 00\ 02 \parallel \text{SharedData}$
 - c $\text{KeyingData} = \text{Hash-1} \parallel \text{Hash-2}$
- 7 解析键控数据如下：
 - a $\text{MacKey} = \text{KeyingData}$ 的首个 128 位 (Hash-1)
 - b $\text{KeyData} = \text{KeyingData}$ 的第二个 128 位 (Hash-2)
- 8 形成位字符串 MacData2。
 - a $\text{MacData2} = 03 \parallel U \parallel V \parallel \text{QEU} \parallel \text{QEV} \parallel \text{Text2}$
- 9 计算 MacTag2。
 - a $\text{MacTag2} = \text{MAC}(\text{MacData2}, \text{MacKey})$
- 10 发送 MacTag2 和 Text2 到响应者 (SKKE-3)。
- 11 从响应者接收 MacTag1' 和 Text1 (SKKE-4)。
- 12 创建位字符串 MacData1。
 - a $\text{MacData1} = 02 \parallel V \parallel U \parallel \text{QEV} \parallel \text{QEU} \parallel \text{Text1}$
- 13 证实 MacTag1 和 MacTag1' 匹配。
 - a $\text{MacTag1} = \text{MAC}(\text{MacData1}, \text{MacKey})$
 - b 检查 $\text{MacTag1} = \text{MacTag1}'$
- 14 存储新的连接密钥, KeyData。 (SKKE)



C.6.4.3 SKKE 测试向量#1

发起者 IEEE 地址 (U)	(>) 11 11 11 11 11 11 11 11
响应者 IEEE 地址 (V)	(>) 22 22 22 22 22 22 22 22
主密钥 (Key)	(<) 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
发起者质疑 (QUE)	(<) 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
响应者质疑(QEV)	(<) 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
SKKE-3 数据 (MacTag2)	(<) F9 A0 89 6C E2 73 8D 71 34 3D 27 9B 51 3F 07 9C
SKKE-4 数据 (MacTag1)	(<) 55 98 10 E9 BD A4 48 40 59 88 12 58 C1 B2 0A 4F
获得的连接密钥	(<)19 BB FC 94 57 55 E7 09 86 E8 8B 5D 21 1C 97 37

中间步骤

- 1 响应者的EUI 64 (V)应该提前知道
- 2 产生发起者质疑QEU
- 3 发送发起者质疑和发起者EUI 64到响应者(SKKE-1)
- 4 接收响应者质疑，QEV (SKKE-2)
- 5 计算共享的密钥位字符串Z。

$$a \quad \text{MacData1} = U \parallel V \parallel \text{QUE} \parallel \text{QEV}$$

0x11 0x11 0x11 0x11 0x11 0x11 0x11 0x11
 0x22 0x22 0x22 0x22 0x22 0x22 0x22 0x22
 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00

$$b \quad \text{MacTag1} = \text{MAC}(\text{MacData1}, \text{Key})$$

$$c \quad Z = \text{MacTag1}$$

0x4c 0x3f 0x60 0xa0 0x2e 0x07 0x13 0x5e
 0xdf 0xad 0xb0 0xe0 0xc1 0x46 0xfd 0xe2

- 6 获得键控数据



- a Hash-1 = Z || 00 00 00 01 || SharedData
位字符串 (在Hash之前)
0x4c 0x3f 0x60 0xa0 0x2e 0x07 0x13 0x5e
0xdf 0xad 0xb0 0xe0 0xc1 0x46 0xfd 0xe2
0x00 0x00 0x00 0x01
- b Hash-2 = Z || 00 00 00 02 || SharedData
位字符串 (在Hash之前)
0x4c 0x3f 0x60 0xa0 0x2e 0x07 0x13 0x5e
0xdf 0xad 0xb0 0xe0 0xc1 0x46 0xfd 0xe2
0x00 0x00 0x00 0x02
- c KeyingData = Hash-1 || Hash-2

7 解析键控数据如下:

- a MacKey = 键控数据的首个128位(Hash-1)

0xaf 0x6e 0x2b 0x7a 0x59 0xc0 0x98 0xb0
0xcf 0x0a 0x93 0x1c 0x8b 0x34 0xcd 0xbd
- b KeyData = 键控数据的第二个128位(Hash-2)

0x19 0xbb 0xfc 0x94 0x57 0x55 0xe7 0x09
0x86 0xe8 0x8b 0x5d 0x21 0x1c 0x97 0x37

8 形成位字符串MacData2。

- a MacData2 = 03 || U || V || QEU || QEV || Text2

0x03 0x11 0x11 0x11 0x11 0x11 0x11 0x11
0x11 0x22 0x22 0x22 0x22 0x22 0x22 0x22
0x22 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00

9 计算MacTag2。

- a MacTag2 = MAC(MacData2, MacKey)

0xf9 0xa0 0x89 0x6c 0xe2 0x73 0x8d 0x71
0x34 0x3d 0x27 0x9b 0x51 0x3f 0x07 0x9c



- 10 发送MacTag2和Text2到响应者(SKKE-).
- 11 从响应者接收MacTag1'和Text1 (SKKE-4).
- 12 创建位字符串MacData1。
 - a $\text{MacData1} = 02 \parallel V \parallel U \parallel \text{QEV} \parallel \text{QEU} \parallel \text{Text1}$

```

0x02 0x22 0x22 0x22 0x22 0x22 0x22 0x22
0x22 0x11 0x11 0x11 0x11 0x11 0x11 0x11
0x11 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00

```

- 13 证实MacTag1和MacTag1'匹配。
 - a $\text{MacTag1} = \text{MAC}(\text{MacData1}, \text{MacKey})$

```

0x55 0x98 0x10 0xe9 0xbd 0xa4 0x48 0x40
0x59 0x88 0x12 0x58 0xc1 0xb2 0x0a 0x4f

```

- 14 证实MacTag1和MacTag1'匹配。
 - a $\text{MacTag1} = \text{MAC}(\text{MacData1}, \text{MacKey})$
 - b 检查 $\text{MacTag1} == \text{MacTag1}'$
- 15 存储新的连接密钥, KeyData。(SKKE)

```

0x19 0xbb 0xfc 0x94 0x57 0x55 0xe7 0x09
0x86 0xe8 0x8b 0x5d 0x21 0x1c 0x97 0x37

```



C.6.4.4 SKKE 测试向量#2

发起者 IEEE 地址 (U)	(>) 55 73 65 72 20 55 0D 0A
响应者 IEEE 地址 (V)	(>) 55 73 65 72 20 56 0D 0A
主密钥 (Key)	(<) C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF
发起者质疑 (QEU)	(<) 9E 3F 0C 19 05 4B 05 44 D5 A7 17 62 0A F2 7D 96
响应者质疑 (QEV)	(<) BF 14 DF 94 94 39 D2 CE 24 C9 09 53 B5 72 D6 53
SKKE-3 数据 (MacTag2)	(<) 57 E1 D4 F9 7B 28 AB 72 B1 9B B7 76 F7 B5 55 1C
SKKE-4 数据 (MacTag1)	(<) A0 31 57 12 38 6A 25 EF 28 94 30 6C 6F BA A4 DA
获得的连接密钥	(<) 84 3D 71 7F 92 C3 D4 31 0A 9F 46 6B 58 B9 F2 1C

中间步骤

- 1 响应者的EUI 64 (V)应该提前知道。
- 2 产生发起者质疑QEU。
- 3 发送发起者质疑和发起者EUI 64到响应者(SKKE-1)。
- 4 接收响应者质疑，QEV (SKKE-2)。
- 5 计算共享的秘密位字符串Z。
 - a $\text{MacData1} = \text{U} \parallel \text{V} \parallel \text{QEU} \parallel \text{QEV}$

0x55 0x73 0x65 0x72 0x20 0x55 0x0d 0x0a
 0x55 0x73 0x65 0x72 0x20 0x56 0x0d 0x0a
 0x9e 0x3f 0x0c 0x19 0x05 0x4b 0x05 0x44
 0xd5 0xa7 0x17 0x62 0x0a 0xf2 0x7d 0x96
 0xbf 0x14 0xdf 0x94 0x94 0x39 0xd2 0xce
 0x24 0xc9 0x09 0x53 0xb5 0x72 0xd6 0x53

b $\text{MacTag1} = \text{MAC}(\text{MacData1}, \text{Key})$

c $Z = \text{MacTag1}$

0x78 0x7c 0xde 0xf6 0x80 0x13 0x12 0xcd
 0x41 0x1b 0xcd 0x62 0x14 0x91 0xf8 0x6d



6 获得键控数据

- a Hash-1 = Z || 00 00 00 01 || SharedData

位字符串(在Hash之前)

0x78 0x7c 0xde 0xf6 0x80 0x13 0x12 0xcd
0x41 0x1b 0xcd 0x62 0x14 0x91 0xf8 0x6d
0x00 0x00 0x00 0x01

- b Hash-2 = Z || 00 00 00 02 || SharedData

位字符串(在Hash之前)

0x78 0x7c 0xde 0xf6 0x80 0x13 0x12 0xcd
0x41 0x1b 0xcd 0x62 0x14 0x91 0xf8 0x6d
0x00 0x00 0x00 0x02

- c KeyingData = Hash-1 || Hash-2

7 解析键控数据如下:

- a MacKey = 键控数据的首个128位(Hash-1)

0x0e 0x6b 0x9f 0x70 0x8c 0xaa 0x84 0x63
0x44 0x76 0xec 0x17 0xa4 0x3e 0x6b 0xc5

- b KeyData = 键控数据的第二个128位(Hash-2)

0x84 0x3d 0x71 0x7f 0x92 0xc3 0xd4 0x31
0x0a 0x9f 0x46 0x6b 0x58 0xb9 0xf2 0x1c

8 形成位字符串MacData2

- a MacData2 = 03 || U || V || QEU || QEV || Text2

0x03 0x55 0x73 0x65 0x72 0x20 0x55 0x0d
0x0a 0x55 0x73 0x65 0x72 0x20 0x56 0x0d
0x0a 0x9e 0x3f 0x0c 0x19 0x05 0x4b 0x05
0x44 0xd5 0xa7 0x17 0x62 0x0a 0xf2 0x7d
0x96 0xbf 0x14 0xdf 0x94 0x94 0x39 0xd2
0xce 0x24 0xc9 0x09 0x53 0xb5 0x72 0xd6
0x53

9 计算MacTag2

- a MacTag2 = MAC(MacData2, MacKey)

0x57 0xe1 0xd4 0xf9 0x7b 0x28 0xab 0x72
0xb1 0x9b 0xb7 0x76 0xf7 0xb5 0x55 0x1c



- 10 发送MacTag2和Text2到响应者(SKKE-3)。
- 11 从响应者接收MacTag1'和Text1 (SKKE-4)。
- 12 创建位字符串MacData1
 - a MacData1 = 02 || V || U || QEV || QEU || Text1

```
0x02 0x55 0x73 0x65 0x72 0x20 0x56 0x0d
0x0a 0x55 0x73 0x65 0x72 0x20 0x55 0x0d
0x0a 0xbf 0x14 0xdf 0x94 0x94 0x39 0xd2
0xce 0x24 0xc9 0x09 0x53 0xb5 0x72 0xd6
0x53 0x9e 0x3f 0x0c 0x19 0x05 0x4b 0x05
0x44 0xd5 0xa7 0x17 0x62 0x0a 0xf2 0x7d
0x96
```

- 13 证实MacTag1和MacTag1'匹配
 - a MacTag1 = MAC(MacData1, MacKey)

```
0xa0 0x31 0x57 0x12 0x38 0x6a 0x25 0xef
0x28 0x94 0x30 0x6c 0x6f 0xba 0xa4 0xda
```

- b 检查MacTag1 == MacTag1'

- 14 存储新的连接密钥, KeyData。 (SKKE), 或

```
0x84 0x3d 0x71 0x7f 0x92 0xc3 0xd4 0x31
0x0a 0x9f 0x46 0x6b 0x58 0xb9 0xf2 0x1c
```



C.6.4.5 实体认证发起者转换

名词	描述
MacKey	共享密钥（网络密钥）
U	发起者的唯一 ID(EUI 64 地址)
V	响应者的唯一 ID(EUI 64 地址)
QEU	发起者质疑（16 字节随机地址）
QEV	响应者质疑（16 字节随机地址）
MAC	我们的 HMAC 函数
Text1	可选的位字符串，EA 中的帧计数器
Text2	“”
SharedData	在发起者和响应者间可选的共享数据。对于 SKKE / EA 是空字符串

注意：伪代码中的‘||’意思是串联。

- 1 响应者的EUI 64 (V)必须提前知道。
- 2 产生发起者质疑QEU。
- 3 发送发起者质疑和发起者IEEE到响应者(EA-Init-Challenge)。
- 4 接收响应者质疑，QEV (EA-Resp-Challenge)。
- 5 形成位字符串MacData2。
 - a $\text{MacData2} = 03 \parallel U \parallel V \parallel \text{QEU} \parallel \text{QEV} \parallel \text{Text2}$
- 6 计算MacTag2
 - a $\text{MacTag2} = \text{MAC}(\text{MacData2}, \text{MacKey})$
- 7 发送MacTag2和Text2到响应者(EA-Init-Mac-Data)。
- 8 从响应者接收MacTag1’和Text1 (EA-Resp-Mac-Data)。
- 9 创建位字符串MacData1
 - a $\text{MacData1} = 02 \parallel V \parallel U \parallel \text{QEV} \parallel \text{QEU} \parallel \text{Text1}$
- 10 证实MacTag1和MacTag1’匹配
 - a $\text{MacTag1} = \text{MAC}(\text{MacData1}, \text{MacKey})$
 - b 检查 $\text{MacTag1} == \text{MacTag1}'$
- 11 存储新的帧计数器，Text1 (EA)。

C.6.4.6 EA 测试向量#1

发起者 IEEE 地址 (U)	(>) 22 22 22 22 22 22 22 22
响应者 IEEE 地址(V)	(>) 11 11 11 11 11 11 11 11
网络密钥 (MacKey)	(<) 44 44 44 44 44 44 44 44 44 44 44 44 44 44 44 44
发起者质疑 (QUE)	(<) 55 55 55 55 55 55 55 55 55 55 55 55 55 55 55 55
响应者质疑 (QEV)	(<) 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66
EA 发起者数据 (MacTag2)	(<) b5 8e 4a a8 ce 51 06 bc 5e 71 9a 41 3e 2e 6e 94
EA 响应者数据(MacTag1)	(<) 83 75 96 ff 23 ea 8c 54 2a 9c 0d 7e 8c c4 ca b4
发起者帧计数器 (Text2)	(<) 00 00 00 00
响应者帧计数器 (Text1)	(<) 00 00 00 00

中间步骤

- 1 响应者的EUI 64 (V) 必须提前知道。
- 2 创建发起者质疑QEU。
- 3 发送发起者质疑和发起者IEEE到响应者(EA-Init-Challenge)。
- 4 接收响应者质疑，QEV (EA-Resp-Challenge)。
- 5 形成位字符串MacData2
 - a $\text{MacData2} = 03 \parallel U \parallel V \parallel \text{QEU} \parallel \text{QEV} \parallel \text{Text2}$

```

0x03 0x22 0x22 0x22 0x22 0x22 0x22 0x22
0x22 0x11 0x11 0x11 0x11 0x11 0x11 0x11
0x11 0x55 0x55 0x55 0x55 0x55 0x55 0x55
0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55
0x55 0x66 0x66 0x66 0x66 0x66 0x66 0x66
0x66 0x66 0x66 0x66 0x66 0x66 0x66 0x66
0x66 0x00 0x00 0x00 0x00 0x00 0x00 0x00

```

- 6 计算MacTag2
 - a $\text{MacTag2} = \text{MAC}(\text{MacData2}, \text{MacKey})$



0xb5 0x8e 0x4a 0xa8 0xce 0x51 0x06 0xbc

0x5e 0x71 0x9a 0x41 0x3e 0x2e 0x6e 0x94

- 7 发送MacTag2和Text2到响应者(EA-Init-Mac-Data)。
- 8 从响应者接收MacTag1'和Text1 (EA-Resp-Mac-Data)。
- 9 创建位字符串MacData1
 - a $\text{MacData1} = 02 \parallel V \parallel U \parallel \text{QEV} \parallel \text{QEU} \parallel \text{Text1}$

0x02 0x11 0x11 0x11 0x11 0x11 0x11 0x11

0x11 0x22 0x22 0x22 0x22 0x22 0x22 0x22

0x22 0x66 0x66 0x66 0x66 0x66 0x66 0x66

0x66 0x66 0x66 0x66 0x66 0x66 0x66 0x66

0x66 0x55 0x55 0x55 0x55 0x55 0x55 0x55

0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55

0x55 0x00 0x00 0x00 0x00

- 10 证实MacTag1和MacTag1'匹配
 - a $\text{MacTag1} = \text{MAC}(\text{MacData1}, \text{MacKey})$

0x83 0x75 0x96 0xff 0x23 0xea 0x8c 0x54

0x2a 0x9c 0x0d 0x7e 0x8c 0xc4 0xca 0xb4

- b 检查 $\text{MacTag1} = \text{MacTag1}'$
- 11 存储新的帧计数器，Text1 (EA)⁵⁰。



C.6.4.7 EA 测试向量#2

发起者 IEEE 地址 (U)	(>) ab cd ef 01 23 45 67 89
响应者 IEEE 地址 (V)	(>) 12 34 56 78 90 ab cd ef
网络密钥 (MacKey)	(<) 47 85 67 85 73 11 79 85 43 21 87 57 44 35 02 65
发起者质疑 (QUE)	(<) 43 4f 85 a1 61 d3 ba 99 7b 3f 25 ef 00 34 42 6f
响应者质疑 (QEV)	(<) e2 08 74 50 4a 0f 18 97 40 9d fb d0 49 78 ae 69
EA 发起者数据 (MacTag2)	(<) 9b f9 e2 48 5d 14 a8 be 6d 40 d3 ee c2 0d e6 6d
EA 响应者数据 (MacTag1)	(<) 67 59 9c a8 a3 1f 8e b0 87 9a de d3 7c ba fc c5
发起者帧计数器 (Text2)	(<) 1f 00 00 00
响应者帧计数器 (Text1)	(<) 20 00 00 00

中间步骤

- 1 响应者的 EUI 64 (V)必须提前知道。
- 2 产生发起者质疑 QEU
- 3 发送发起者质疑和发起者 IEEE 到响应者(EA-Init-Challenge)
- 4 接收响应者质疑, QEV (EA-Resp-Challenge)
- 5 形成位字符串 MacData2
 - a MacData2 = 03 || U || V || QEU || QEV || Text2

```

0x03 0xab 0xcd 0xef 0x01 0x23 0x45 0x67
0x89 0x12 0x34 0x56 0x78 0x90 0xab 0xcd
0xef 0x43 0x4f 0x85 0xa1 0x61 0xd3 0xba
0x99 0x7b 0x3f 0x25 0xef 0x00 0x34 0x42
0x6f 0xe2 0x08 0x74 0x50 0x4a 0x0f 0x18
0x97 0x40 0x9d 0xfb 0xd0 0x49 0x78 0xae
0x69 0x00 0x00 0x00 0x1f

```

- 6 计算 MacTag2
 - a MacTag2 = MAC(MacData2, MacKey)

```

0x9b 0xf9 0xe2 0x48 0x5d 0x14 0xa8 0xbe
0x6d 0x40 0xd3 0xee 0xc2 0x0d 0xe6 0x6d

```



- 7 发送 MacTag2 和 Text2 到响应者(EA-Init-Mac-Data)
- 8 从响应者接收 MacTag1'和 Text1 (EA-Resp-Mac-Data)
- 9 创建位字符串 MacData1
 - a $\text{MacData1} = 02 \parallel V \parallel U \parallel \text{QEV} \parallel \text{QEU} \parallel \text{Text1}$

0x02 0x12 0x34 0x56 0x78 0x90 0xab 0xcd
0xef 0xab 0xcd 0xef 0x01 0x23 0x45 0x67
0x89 0xe2 0x08 0x74 0x50 0x4a 0x0f 0x18
0x97 0x40 0x9d 0xfb 0xd0 0x49 0x78 0xae
0x69 0x43 0x4f 0x85 0xa1 0x61 0xd3 0xba
0x99 0x7b 0x3f 0x25 0xef 0x00 0x34 0x42
0x6f 0x00 0x00 0x00 0x20

- 10 证实 MacTag1 和 MacTag1'匹配
 - a $\text{MacTag1} = \text{MAC}(\text{MacData1}, \text{MacKey})$

0x67 0x59 0x9c 0xa8 0xa3 0x1f 0x8e 0xb0
0x87 0x9a 0xde 0xd3 0x7c 0xba 0xfc 0xc5

- b 检查 $\text{MacTag1} == \text{MacTag1}'$
- 11 存储新的帧计数器, Text1 (EA)

附录 D

MAC 和 PHY 子层说明

D.1 简介

D.1.1 范围

当用于与 ZigBee 规范定义的上层有关的情况下，本附录适用于 IEEE 802.15.4 2003 媒体访问控制子层（MAC）和物理层（PHY）规范。关于其它环境下的使用不做任默示。

D.1.2 目的

当前的 ZigBee 规范假定了 IEEE 802.15.4 2003 规范定义的 MAC 和 PHY 子层的用法。但是，因为开发者已经把 MAC 和 PHY 子层投入使用，他们发现了规范作者可能预见或没有预见的问题，或 IEEE 802.15.4 2003 规范没有涵盖的问题。本文件目的是提供这类问题的解决方案，由 ZigBee 联盟使用。

D.2 协议栈大小问题

MAC 和 ZigBee 协议栈开发者都发现，实施全套的 MAC 是一个重大的任务，且需要大量的代码空间。即使取消了可选的 GTS 和 MAC 安全功能，如果你发现 MAC 在一个 64K 可用空间的处理器占据了超过 24K 的代码空间，这也是毫不惊奇的。

ZigBee 联盟采用了一项补偿政策，即宣布不需要支持某个特定协议栈 profile 的 MAC 功能，对于该协议栈 profile 是可选的。具体地，因为某个协议栈 profile 的平台兼容性测试的结果，不使用的任何 MAC 功能就不需要列出来，以执行所宣布的平台兼容。比如，因为 ZigBee2006 协议栈 profile 依赖的是一个无信标网络，所以协议栈 profile 的平台兼容性测试不使用信标。在测试时，支持定期信标、信标跟踪等等的代码可以从设备的代码库中空缺，测试者不需要知道，至于平台兼容性验证不会出现问题。

D.3 MAC 连接

在连接期间，根据IEEE 802.15.4 2003规范，会发送一些帧，包括一个连接请求命令，一个连接响应命令和一个数据请求。关于这些帧头的地址域，规范中有一些模棱两可的值。表 D.1–D.3列出了允许的选项，它们被执行ZigBee规范的设备承认。在每种情况下，给定的第一个选项是首选的选项，是应该使用的。

表 D.1 Associate Request头域

DstPANId	DstAddr	SrcPANId	SrcAddr
目的设备的个域网ID	目的设备的16位短地址	0xffff	源设备的64位扩展地址
		因为在帧控制域 IntraPAN被设置为1， PANId被忽略	
		目的设备的个域网ID	
如果目的设备是个域网的 协调器就没有父节点	如果目的设备是个域网的 协调器就没有父节点		

注意在这种情况和下面的情况中，命令的源是请求连接的设备。

表 D.2 数据请求头域

DstPANId	DstAddr	SrcPANId	SrcAddr
目的设备的个域网ID	目的设备16位短地址	0xffff	源设备的64位扩展地址
		因为在帧控制域IntraPAN被设置为1，PANId被忽略。	
		目的设备的个域网ID	
如果目的设备是个域网协调器，没有父节点	如果目的设备是个域网协调器，没有父节点		

表 D.3 连接响应头域

DstPANId	DstAddr	SrcPANId	SrcAddr
目的设备的个域网ID	目的设备的64位扩展地址	因为在帧控制域IntraPAN被设置为1，PANId被忽略。	源设备64位扩展地址
		源设备的个域网ID	
0xffff			

D.4 aMaxMACFrameSize

在 IEEE 802.15.4 2003 规范中，有一个常量叫 aMaxMACFrameSize，被定义为 aMaxPHYPacketSize - aMaxFrameOverhead。本公式给定 $aMaxMACFrameSize = 127 - 25 = 102$ 字节。关于这个值的不同意见和 aMaxFrameOverhead 参数有关。这个参数的值为 25 对于只能使用短寻址模式的 ZigBee 网络是不正确的。ZigBee 联盟决定（而不是讨论）aMaxFrameOverhead 的正确值应该定义为一个可接受的数据包，小于 aMaxPHYFrameSize，是一个可解码的 802.15.4 和 ZigBee 数据包。

为了推动这项工作，`nsduLength`参数的最大值被定义为小于`aMaxPHYFrameSize-(nwkcMACFrameOverhead+nwkcMinHeaderOverhead)`。因此ZigBee的开发者需要保证他们的802.15.4的执行可以包含这个改进。

D.5 信标时序⁵¹

为了采用 NWK 信标调度算法，有必要对 IEEE Std 802.15.4-2003 MAC 子层执行下列改进。

一个新的参数，`StartTime`，需要增加到 `MLME-START.request` 原语，指定开始传输信标的时间。本原语的新格式如下：

```
MLME-START.request    {  
                        PANID,  
                        LogicalChannel,  
                        BeaconOrder,  
                        SuperframeOrder,  
                        PANCoordinator,  
                        BatteryLifeExtention,  
                        CoordRealignment,  
                        SecurityEnable,  
                        StartTime  
                        }  
                        
```

51. LB #0160

StartTime 参数见表 D.4 描述，关于所有其它参数的描述参见 IEEE 802.15.4-2003 [B1]。

表 D.4 信标传输的开始时间

名称	类型	有效范围	描述
StartTime	Integer	0x000000 - 0xffffffff	信标传输的开始时间。如果发出原语的设备是 PAN 协调器，这个参数被忽略，信标传输立即开始。否则，本参数指明的关于它的父节点信标的时间。该参数指定为符号秒，四舍五入到一个退避时槽边界。这个值精确值最低是 20 位，低 4 位是最低位。

macAutoRequest 的值应该设置为 IEEE 802.15.4 [B1]中的默认值⁵²。

附录 E

网络管理器作为网络信道管理器运行的 干扰报告和解决方案

一个单个设备能成为网络信道管理器。这个设备作为中央协调机制，接收网络干扰报告和如果检测到干扰时改变信道。网络管理器的默认地址是协调器，但是这可以通过发送 `Mgmt_NWK_Update_req` 命令更新，带有网络信道管理器的不同的短地址。是网络信道管理器的设备在节点描述符的服务器位掩码中设置网络管理器位，并应响应 `System_Server_Discovery_req` 命令。

每个路由器或协调器负责追踪传输失败，使用邻居表中的 `TransmitFailure` 域，也为总的传输尝试保持一个 NIB 计数器。一旦总的传输尝试超过 20，如果信息发送的传输失败超过 25%，设备可能已经在使用的信道上检测到干扰。然后设备负责采取以下步骤：

- 1 在所有的信道上实施能量扫描。如果能量扫描不表明当前信道比其它信道有更高的能量，就不采取行动。设备会继续正常操作，信息计数器不复位。但是，重复的能量扫描不可取，因为在扫描时设备是离开网络的。因此执行会限制失败的设备执行能量扫描的频率。

- 2 如果能量扫描指示所用的信道上能量增加，一个 `Mgmt_NWK_Update_notify` 会发送到网络管理器，指示存在干扰。本报告是作为带有确认的 APS 单播发送的，一旦收到确认，总传输和传输失败计数器被重置为零。

3 为了避免有通信问题的设备频繁发送报告到网络管理器，每个小时设备不能发送 Mgmt_NWK_Update_notify 超过四次。

在收到一个主动提供的 Mgmt_NWK_Update_notify 之后，网络管理器必须评估网络是否需要信道改变。网络管理器使用的决定信道改变的具体机制留给了执行者。希望执行者会使用不同的方法最佳地决定何时信道需要改变，以及如何选择最合适的信道。以下是提供的执行的指南。

网络管理器将会作如下行动：

1 等待和评估是否收到其它设备的报告。如果没有报告其它失败，这个报告是合适的。在此情况下，网络管理器会把报告的设备增加到一个报告干扰设备列表。这样一个列表上的设备数目取决于网络的大小。网络管理器可以将老化设备移除出清单。

2 使用 Mgmt_NWK_Update_req 命令请求其它干扰报告。如果其它的失败被报告，或者网络管理器设备本身失败希望改变信道，报告就可能产生。网络管理器可以从已经报告干扰的设备列表，以及其它在网络中随机抽选的路由器中请求数据。网络管理器不能从刚刚报告干扰的设备中请求更新，因为这已经是新的数据。

3 在收到 Mgmt_NWK_Update_notify 之后，网络管理器会决定是否需要改变信道，不管执行的具体机制是否合适。

4 如果上面的数据指示要考虑信道改变，网络管理器会完成下列步骤：

a 基于 Mgmt_NWK_Update_notify，根据最低能量选择一个信道。这是提议的新信道。如果这个新信道的能量等级不低于可以接受的阈值，不能进行信道改变。

5 在改变信道之前，网络管理器会储存能量扫描值作为最新的能量扫描值，现存的信道中的失败率作为最新的失败率。这些值对于比较之前信道的失败率和能量等级有用，以评估网络是否造成了自己的干扰。

6 网络管理器应该广播一个 Mgmt_NWK_Update_req 通知设备新信道。这个广播应该发到所有的路由器和协调器。网络管理器负责增加 NIB 中 *nwkUpdateId* 参数，并把它包括在 Mgmt_NWK_Update_req 中。网络管理器根据 *apsChannelTimer* 的值设置一个计时器，在发出 Mgmt_NWK_Update_req 之后，改变信道，在本计时器终止之前不能发出另外一个这样的命令。但是，在此期间，网络管理器会完成上述分析。但是，不用改变信道，网络管理器会使用 Mgmt_NWK_Update_notify 报告给本地应用程序，应用程序可以使用 Mgmt_NWK_Update_req 强制改变一个信道。

在收到带有改变信道请求的 `Mgmt_NWK_Update_req` 之后，本地网络管理器会设置一个计时器等于 `nwkNetworkBroadcastDeliveryTime`，在本计时器终止之后改变信道。每个节点也都应该增加 `nwkUpdateId` 参数，也要复位总的传输次数和传输失败计数器。

对于 `RxOnWhenIdle` 等于 `FALSE` 的设备，不能接收任何信道改变。在这些失去网络的设备或路由器上，应该在 APS IB 的 `apsChannelMask` 清单上实施一个主动的扫描，使用扩展的 PANID 去找到这个网络。如果在不同的信道上找到扩展的 PANID，设备会选择 `nwkUpdateId` 参数值更大的信道。如果使用 `apsChannelMask` 清单没有找到扩展的 PANID，应该使用所有信道进行一个扫描。

版权声明

郑州新双恒信息技术有限公司拥有本译文的版权。本译文免费供大家一起学习交流。译文出现的错误及误差，您可以通过以下方式联系本公司，我们将非常感谢。欢迎在网络上转载本译文，但必须保证本译文的完整，否则本公司不承担任何后果。如果你在转载的时候，需要对本译文进行修改或重新编辑，请征得本公司的同意。谢谢！

网址：<http://www.zigbee-sh.cn>

QQ 群：83028739

